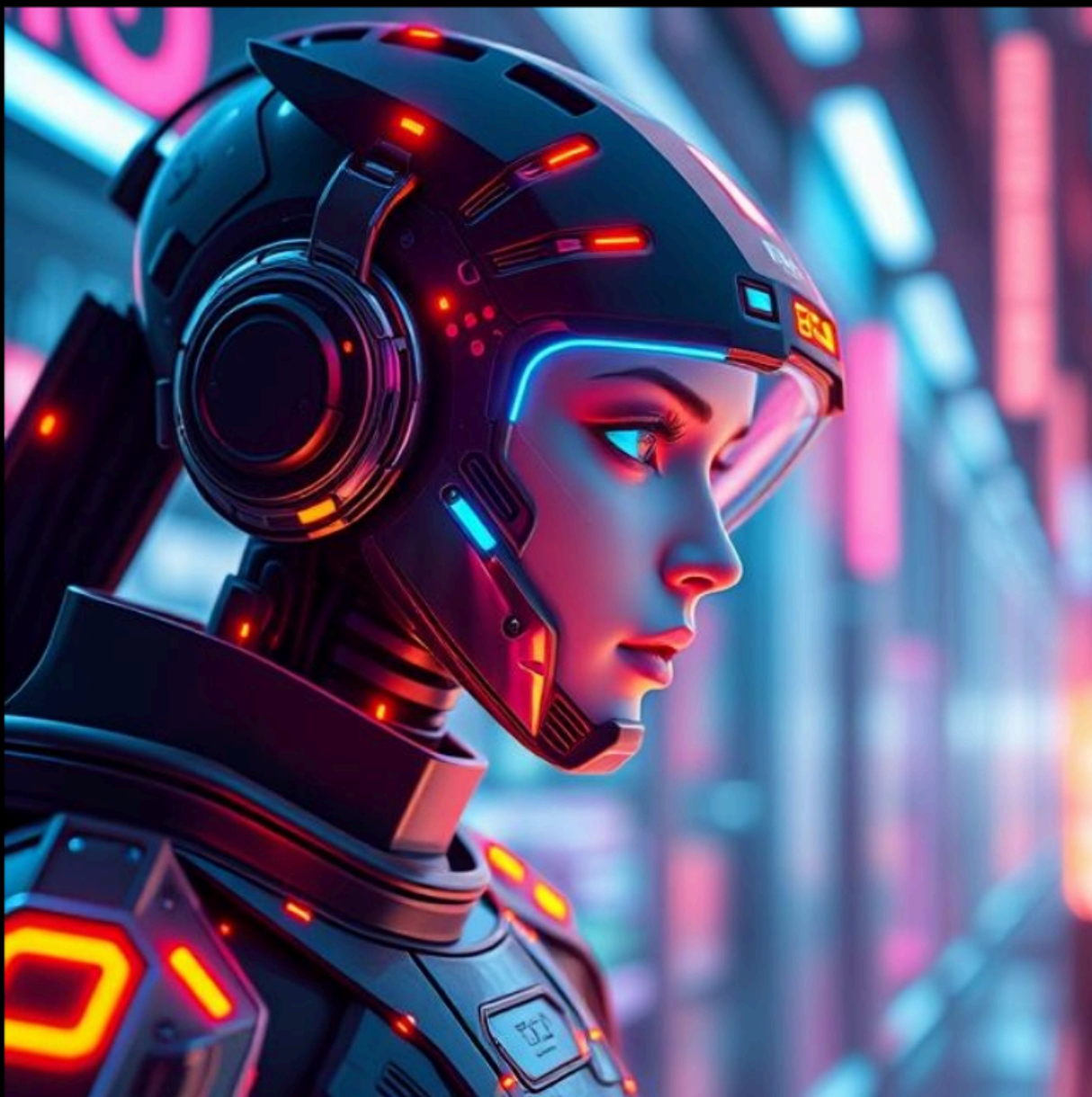




REVERSE ENGINEERING DATA

FIRST EDITION 1.0001

Copyright © 2026 Kevin Thomas

Contents

Part I: Foundations

- Chapter 1: Central Limit Theorem
- Chapter 2: Penguins Dataset - EDA
- Chapter 3: Penguins Dataset - Clustering

Part II: Simple Linear Regression

- Chapter 4: Penguins Dataset Simple Linear Regression - Linear Relationship
- Chapter 5: Penguins Dataset Simple Linear Regression - Non-Linear Relationship
- Chapter 6: Penguins Dataset Simple Linear Regression - Fitting Linear Feature
- Chapter 7: Penguins Dataset Simple Linear Regression - Fitting Non-Linear Feature
- Chapter 8: Penguins Dataset Simple Linear Regression - Fitting Linear Feature Predictions
- Chapter 9: Penguins Dataset Simple Linear Regression - Fitting Non-Linear Feature Predictions
- Chapter 10: Penguins Dataset Simple Linear Regression - Measuring Linear Feature Performance
- Chapter 11: Penguins Dataset Simple Linear Regression - Fitting Multiple Models

Part III: Multiple Linear Regression and Linear Regression

- Chapter 12: Multiple Linear Regression - Additive Features
- Chapter 13: Multiple Linear Regression - Interactive Features
- Chapter 14: Penguins Dataset - Linear Regression

Part IV: The Mathematics of Deep Learning

- Linear Algebra for Deep Learning
- Calculus, Gradients, and Automatic Differentiation
- Probability, Information Theory, and Loss Functions
- The Mathematics of the Multilayer Perceptron
- The Mathematics of Convolutional Networks
- The Mathematics of Recurrent Networks and LSTMs
- The Mathematics of Attention and Transformers
- The Mathematics of GPT
- The Mathematics of Autoencoders, Variational Autoencoders, and GANs

Part V: Classification and Deep Learning with PyTorch

- Chapter 15: Penguins Dataset - Logistic Regression
- Chapter 16: Penguins Dataset - MLP
- Chapter 17: Fashion-MNIST Dataset - CNN

Part VI: Sequences, Attention, and Language Models

- Chapter 18: Tiny Shakespeare Dataset - RNN
- Chapter 19: Tiny Shakespeare Dataset - Attention
- Chapter 20: Tiny Shakespeare Dataset - TinyGPT From Scratch

Part VII: Regularization, Evaluation, and Generative Vision

- Chapter 21: Breast Cancer Dataset - Regularization
- Chapter 22: Breast Cancer Dataset - Model Evaluation
- Chapter 23: Fashion-MNIST Dataset - Autoencoder

Chapter 24: Fashion-MNIST Dataset - GAN

Chapter 25: Fashion-MNIST Dataset - Vision Transformer

Part VIII: Segmentation, Reinforcement Learning, and Deployment

Chapter 26: Fashion-MNIST Dataset - Model Deployment

Chapter 27: Oxford-IIIT Pet Dataset - Semantic Segmentation

Chapter 28: CartPole Environment - Deep Q-Network

Chapter 29: Tiny Shakespeare Dataset - Fine-Tuning a Pretrained LLM

Part IX: A Kaggle Project from Scratch

Chapter 30: Kaggle Titanic Dataset - EDA and Binary Classifier

Appendices

Appendix A: Python Reference

Appendix B: PyTorch Reference

Appendix C: Mathematical Notation

Appendix D: Glossary

Appendix E: Further Reading

Appendix F: NumPy Reference

Appendix G: pandas Reference

Appendix H: Matplotlib Reference

Appendix I: Command-Line and Environment Reference

Appendix J: Function Index

Appendix K: Chapter Summaries

Part I: Foundations

Chapter 1: Central Limit Theorem

Introduction

Every measurement is a sample, and every sample is noisy. The central limit theorem is the single most important reason we can say anything trustworthy about a population from a finite set of observations. It explains why the average of many independent measurements is approximately normal, why the spread of that average shrinks with the square root of the sample size, and why a confidence interval is a meaningful object. Before you fit a single model, you need this theorem, because it is the foundation on which statistical inference is built.

This chapter builds the theorem from scratch. You will not import a library that hands you the answer. You will generate random numbers, average them, repeat the experiment thousands of times, and watch the distribution of the averages become a bell curve. Along the way you will write your own functions for the average, the variance, and the standard deviation, and you will see the standard error of the mean emerge as a measured quantity rather than a formula someone told you to memorize.

What You Will Learn

- How to generate random samples and compute their average by hand.
- Why the distribution of sample averages becomes normal as the sample size grows.
- The difference between the variance of the data and the variance of the average.
- How to compute the standard error of the mean and use it to build a confidence interval.
- Why small samples force us to express our uncertainty rather than hide it.

Overview

The logic of the chapter is a loop. We draw a sample, compute its average, and record it. We repeat that loop many times. The collection of recorded averages is itself a dataset, and its distribution is the sampling distribution of the mean. When each sample has only a few observations, the sampling distribution is wide and lumpy. As the sample size grows, two things happen at once: the distribution becomes smooth and bell-shaped, and it becomes narrower. The narrowing is the important part. It tells us that a larger sample gives a more precise estimate, and the rate of improvement is governed by the square root of the sample size.

The standard error of the mean is the standard deviation of that sampling distribution. It is not the standard deviation of the raw data. Confusing the two is one of the most common mistakes in applied statistics. The raw data can be wildly spread out while the average of a large sample is quite stable. Once you can measure the standard error, you can state how confident you are that your sample average is close to the true population average, and that statement is a confidence interval.

We close by comparing a small sample to a large sample and showing how much wider the small-sample interval is. The lesson is not that small samples are useless. The lesson is that small samples demand honesty about uncertainty.

Import Modules

```
import random
import matplotlib.pyplot as plt
```

For Loop

Basic for loop example.

```
numbers = [1,
           2,
           3,
           4,
           5]
```

```
for num in numbers:
    print(num)
```

```
1
2
3
4
5
```

Random Numbers

Generate 100 Random Numbers w/ a For Loop

```
random.seed(42)

for _ in range(100):
    print(random.random())
```

```
0.6394267984578837
0.025010755222666936
0.27502931836911926
0.22321073814882275
0.7364712141640124
0.6766994874229113
0.8921795677048454
0.08693883262941615
0.4219218196852704
0.029797219438070344
0.21863797480360336
0.5053552881033624
0.026535969683863625
0.1988376506866485
0.6498844377795232
0.5449414806032167
0.2204406220406967
0.5892656838759087
0.8094304566778266
0.006498759678061017
0.8058192518328079
0.6981393949882269
```

0.3402505165179919
0.15547949981178155
0.9572130722067812
0.33659454511262676
0.09274584338014791
0.09671637683346401
0.8474943663474598
0.6037260313668911
0.8071282732743802
0.7297317866938179
0.5362280914547007
0.9731157639793706
0.3785343772083535
0.552040631273227
0.8294046642529949
0.6185197523642461
0.8617069003107772
0.577352145256762
0.7045718362149235
0.045824383655662215
0.22789827565154686
0.28938796360210717
0.0797919769236275
0.23279088636103018
0.10100142940972912
0.2779736031100921
0.6356844442644002
0.36483217897008424
0.37018096711688264
0.2095070307714877
0.26697782204911336
0.936654587712494
0.6480353852465935
0.6091310056669882
0.171138648198097
0.7291267979503492
0.1634024937619284
0.3794554417576478
0.9895233506365952
0.6399997598540929
0.5569497437746462
0.6846142509898746
0.8428519201898096
0.7759999115462448
0.22904807196410437
0.03210024390403776
0.3154530480590819
0.26774087597570273
0.21098284358632646
0.9429097143350544
0.8763676264726689
0.3146778807984779
0.65543866529488
0.39563190106066426
0.9145475897405435
0.4588518525873988
0.26488016649805246
0.24662750769398345
0.5613681341631508
0.26274160852293527
0.5845859902235405
0.897822883602477

```
0.39940050514039727
0.21932075915728333
0.9975376064951103
0.5095262936764645
0.09090941217379389
0.04711637542473457
0.10964913035065915
0.62744604170309
0.7920793643629641
0.42215996679968404
0.06352770615195713
0.38161928650653676
0.9961213802400968
0.529114345099137
0.9710783776136181
0.8607797022344981
```

Generate 100 Random Numbers w/ a For Loop and Store in a List

```
random_numbers = []
```

```
random_numbers
```

```
[]
```

```
random.seed(42)
```

```
for _ in range(100):
    random_numbers.append(random.random())
```

```
random_numbers
```

```
[0.6394267984578837,
 0.02501075522666936,
 0.27502931836911926,
 0.22321073814882275,
 0.7364712141640124,
 0.6766994874229113,
 0.8921795677048454,
 0.08693883262941615,
 0.4219218196852704,
 0.029797219438070344,
 0.21863797480360336,
 0.5053552881033624,
 0.026535969683863625,
 0.1988376506866485,
 0.6498844377795232,
 0.5449414806032167,
 0.2204406220406967,
 0.5892656838759087,
 0.8094304566778266,
 0.006498759678061017,
 0.8058192518328079,
 0.6981393949882269,
```

```
0.3402505165179919,  
0.15547949981178155,  
0.9572130722067812,  
0.33659454511262676,  
0.09274584338014791,  
0.09671637683346401,  
0.8474943663474598,  
0.6037260313668911,  
0.8071282732743802,  
0.7297317866938179,  
0.5362280914547007,  
0.9731157639793706,  
0.3785343772083535,  
0.552040631273227,  
0.8294046642529949,  
0.6185197523642461,  
0.8617069003107772,  
0.577352145256762,  
0.7045718362149235,  
0.045824383655662215,  
0.22789827565154686,  
0.28938796360210717,  
0.0797919769236275,  
0.23279088636103018,  
0.10100142940972912,  
0.2779736031100921,  
0.6356844442644002,  
0.36483217897008424,  
0.37018096711688264,  
0.2095070307714877,  
0.26697782204911336,  
0.936654587712494,  
0.6480353852465935,  
0.6091310056669882,  
0.171138648198097,  
0.7291267979503492,  
0.1634024937619284,  
0.3794554417576478,  
0.9895233506365952,  
0.6399997598540929,  
0.5569497437746462,  
0.6846142509898746,  
0.8428519201898096,  
0.7759999115462448,  
0.22904807196410437,  
0.03210024390403776,  
0.3154530480590819,  
0.26774087597570273,  
0.21098284358632646,  
0.9429097143350544,  
0.8763676264726689,  
0.3146778807984779,  
0.65543866529488,  
0.39563190106066426,  
0.9145475897405435,  
0.4588518525873988,  
0.26488016649805246,  
0.24662750769398345,  
0.5613681341631508,  
0.26274160852293527,  
0.5845859902235405,  
0.897822883602477,
```

```
0.39940050514039727,  
0.21932075915728333,  
0.9975376064951103,  
0.5095262936764645,  
0.09090941217379389,  
0.04711637542473457,  
0.10964913035065915,  
0.62744604170309,  
0.7920793643629641,  
0.42215996679968404,  
0.06352770615195713,  
0.38161928650653676,  
0.9961213802400968,  
0.529114345099137,  
0.9710783776136181,  
0.8607797022344981]
```

List Comprehension

```
<output_list> = [<action> for <variable> in <sequence>]
```

```
random.seed(42)  
  
random_numbers_lc = [random.random() for _ in range(100)]
```

```
random_numbers_lc
```

```
[0.6394267984578837,  
0.025010755222666936,  
0.27502931836911926,  
0.22321073814882275,  
0.7364712141640124,  
0.6766994874229113,  
0.8921795677048454,  
0.08693883262941615,  
0.4219218196852704,  
0.029797219438070344,  
0.21863797480360336,  
0.5053552881033624,  
0.026535969683863625,  
0.1988376506866485,  
0.6498844377795232,  
0.5449414806032167,  
0.2204406220406967,  
0.5892656838759087,  
0.8094304566778266,  
0.006498759678061017,  
0.8058192518328079,  
0.6981393949882269,  
0.3402505165179919,  
0.15547949981178155,  
0.9572130722067812,  
0.33659454511262676,  
0.09274584338014791,  
0.09671637683346401,  
0.8474943663474598,  
0.6037260313668911,  
0.8071282732743802,
```


0.7297317866938179,
0.5362280914547007,
0.9731157639793706,
0.3785343772083535,
0.552040631273227,
0.8294046642529949,
0.6185197523642461,
0.8617069003107772,
0.577352145256762,
0.7045718362149235,
0.045824383655662215,
0.22789827565154686,
0.28938796360210717,
0.0797919769236275,
0.23279088636103018,
0.10100142940972912,
0.2779736031100921,
0.6356844442644002,
0.36483217897008424,
0.37018096711688264,
0.2095070307714877,
0.26697782204911336,
0.936654587712494,
0.6480353852465935,
0.6091310056669882,
0.171138648198097,
0.7291267979503492,
0.1634024937619284,
0.3794554417576478,
0.9895233506365952,
0.6399997598540929,
0.5569497437746462,
0.6846142509898746,
0.8428519201898096,
0.7759999115462448,
0.22904807196410437,
0.03210024390403776,
0.3154530480590819,
0.26774087597570273,
0.21098284358632646,
0.9429097143350544,
0.8763676264726689,
0.3146778807984779,
0.65543866529488,
0.39563190106066426,
0.9145475897405435,
0.4588518525873988,
0.26488016649805246,
0.24662750769398345,
0.5613681341631508,
0.26274160852293527,
0.5845859902235405,
0.897822883602477,
0.39940050514039727,
0.21932075915728333,
0.9975376064951103,
0.5095262936764645,
0.09090941217379389,
0.04711637542473457,
0.10964913035065915,
0.62744604170309,
0.7920793643629641,

```
0.42215996679968404,  
0.06352770615195713,  
0.38161928650653676,  
0.9961213802400968,  
0.529114345099137,  
0.9710783776136181,  
0.8607797022344981]
```

Average

1. Get the sum of all the numbers.

```
nums = [1,  
        2,  
        3,  
        4,  
        5]
```

```
a_sum = 0  
for num in nums:  
    a_sum += num
```

```
a_sum
```

```
15
```

2. Count the number of elements in the list.

```
nums_len = len(nums)
```

```
nums_len
```

```
5
```

3. Divide the sum by the number of elements in the list.

```
a_sum / nums_len
```

```
3.0
```

Create a Function to Calculate the Average

```
def my_avg(n):  
    """  
    Calculate the average of a list of numbers.  
  
    Parameters:
```

```
    n (list): A list of numbers.

Returns:
    float: The average of the numbers in the list.
"""
return sum(n) / len(n)
```

```
my_avg(nums)
```

```
3.0
```

Getting Help on a Function

```
help(my_avg)
```

```
Help on function my_avg in module __main__:

my_avg(n)
    Calculate the average of a list of numbers.

    Parameters:
        n (list): A list of numbers.

    Returns:
        float: The average of the numbers in the list.
```

Collect 3 Observations on Different Days for 10 People

```
random.seed(42)

day1 = [random.random() for _ in range(10)]
day2 = [random.random() for _ in range(10)]
day3 = [random.random() for _ in range(10)]
```

```
day1
```

```
[0.6394267984578837,
 0.025010755222666936,
 0.27502931836911926,
 0.22321073814882275,
 0.7364712141640124,
 0.6766994874229113,
 0.8921795677048454,
 0.08693883262941615,
 0.4219218196852704,
 0.029797219438070344]
```

```
day2
```

```
[0.21863797480360336,  
0.5053552881033624,  
0.026535969683863625,  
0.1988376506866485,  
0.6498844377795232,  
0.5449414806032167,  
0.2204406220406967,  
0.5892656838759087,  
0.8094304566778266,  
0.006498759678061017]
```

```
day3
```

```
[0.8058192518328079,  
0.6981393949882269,  
0.3402505165179919,  
0.15547949981178155,  
0.9572130722067812,  
0.33659454511262676,  
0.09274584338014791,  
0.09671637683346401,  
0.8474943663474598,  
0.6037260313668911]
```

All 3 Have Different Values, Let's Calculate Their Averages

```
my_avg(day1)
```

```
0.4006685751243019
```

```
my_avg(day2)
```

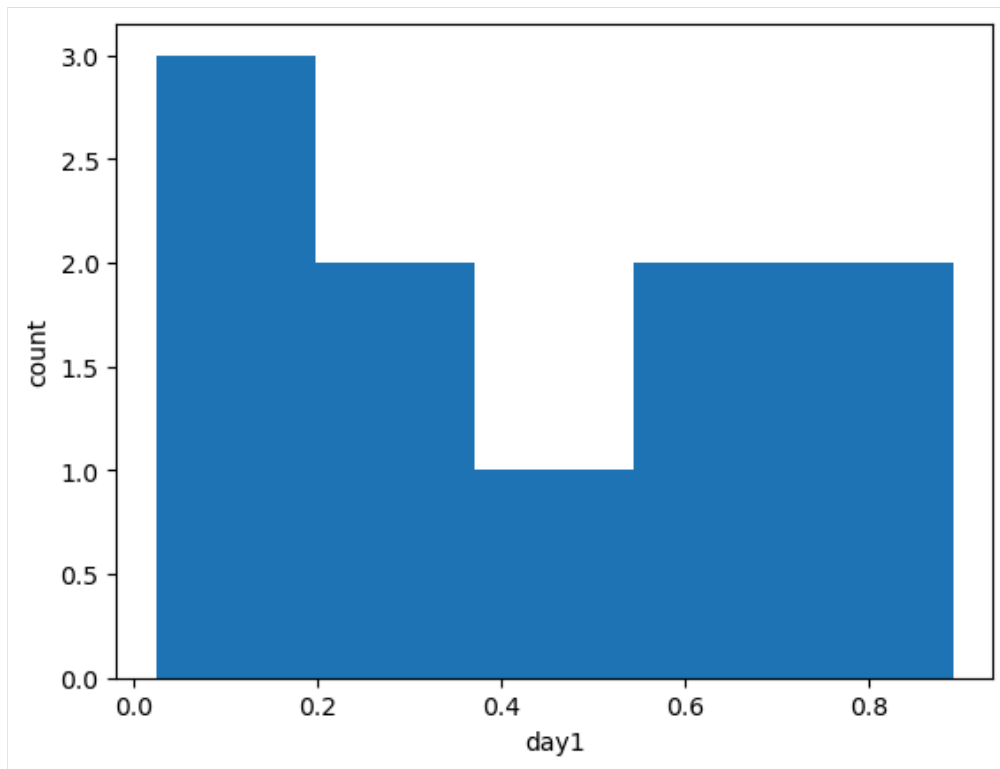
```
0.3769828323932711
```

```
my_avg(day3)
```

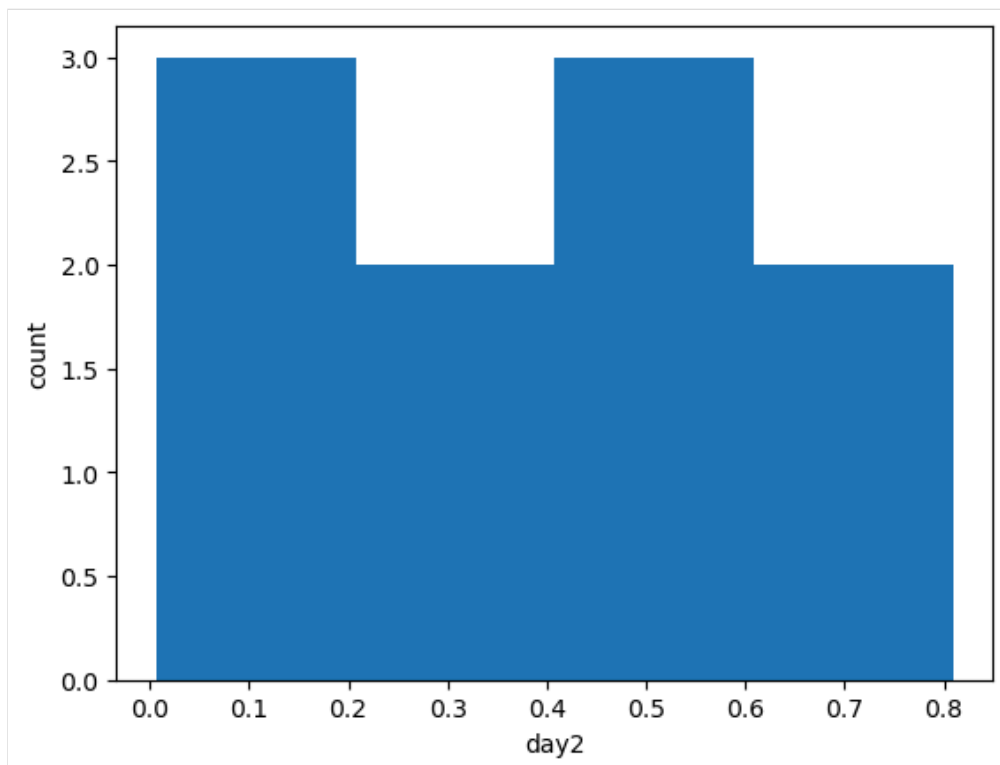
```
0.49341788983981794
```

Visualize the Distribution Using a Histogram

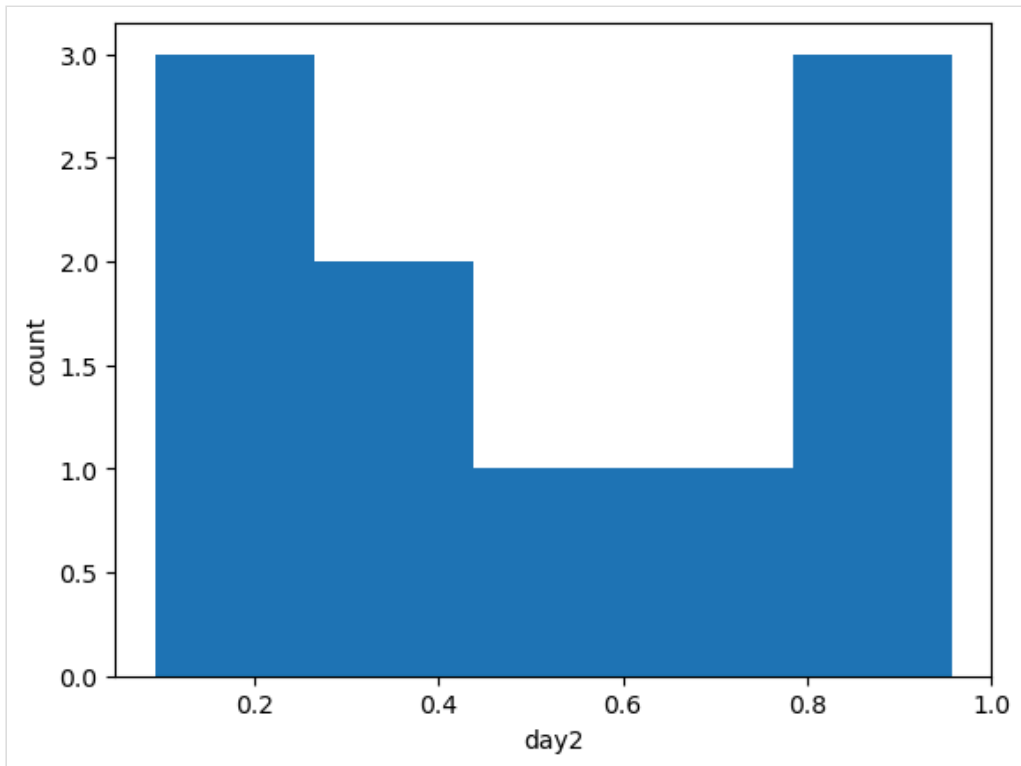
```
plt.hist(day1, bins=5)  
plt.xlabel('day1')  
plt.ylabel('count')  
plt.show()
```



```
plt.hist(day2, bins=4)
plt.xlabel('day2')
plt.ylabel('count')
plt.show()
```



```
plt.hist(day3, bins=5)
plt.xlabel('day2')
plt.ylabel('count')
plt.show()
```



Create a Function for Generating Samples and Calculating the Average

```
def gen_samples_calc_avg(n):
    """
    Assume we are generating values from a UNIFORM distribution between 0 and 1

    Parameters:
        n (int): The number of random numbers to generate.

    Returns:
        float: The average of the generated random numbers.
    """
    return my_avg([random.random() for _ in range(n)])
```

Let's Calculate the Average of 10 Numbers 100 Times

```
random.seed(42)

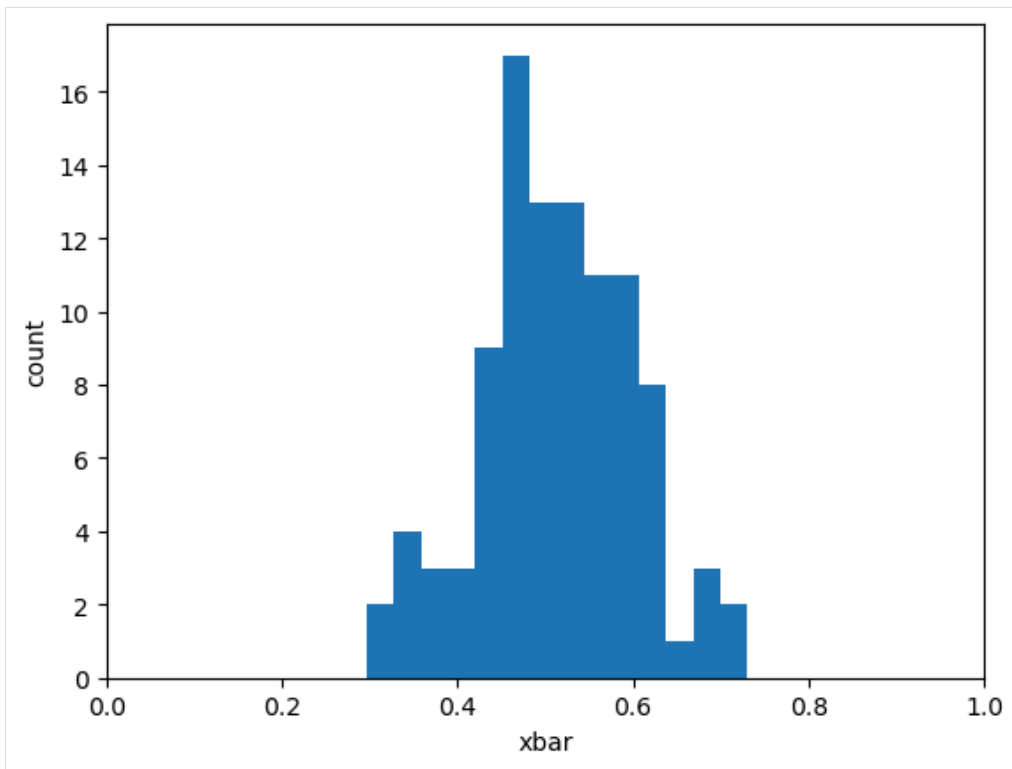
num_10_100s = [gen_samples_calc_avg(10) for _ in range(100)]
```

```
num_10_100s
```

```
[0.4006685751243019,  
0.3769828323932711,  
0.49341788983981794,  
0.686376238606863,  
0.29597569781632027,  
0.44836101802315814,  
0.533428117689419,  
0.528091574806805,  
0.4570329568579887,  
0.5753575301062241,  
0.4812527587386003,  
0.5300830160193748,  
0.5594560699987943,  
0.4486082829315742,  
0.4579606299821986,  
0.4323131269803091,  
0.4020481658999553,  
0.5180339984328821,  
0.4976098693315329,  
0.5614202376942153,  
0.5433716165239272,  
0.5605711093272987,  
0.595318519108694,  
0.42416779210070904,  
0.49869962802742684,  
0.507829604061832,  
0.6085403471214621,  
0.45059590489603646,  
0.4851957880754007,  
0.43623517532803885,  
0.6006227477552516,  
0.5237048733092099,  
0.7223651008715091,  
0.5508299621111297,  
0.47195243225557065,  
0.5491879723111697,  
0.5012411998355109,  
0.5261024506936252,  
0.46012254404717334,  
0.4784093951087044,  
0.4564321026172258,  
0.4333098071191711,  
0.48559709819968366,  
0.4589175404280795,  
0.7307533607570189,  
0.5448090085991202,  
0.3329021748699117,  
0.5799730139025867,  
0.5614311179565079,  
0.30844719005199556,  
0.3304273775552172,  
0.4155776220777579,  
0.5508967952577224,  
0.46814721725259484,  
0.5052942116360402,  
0.5877682384088321,  
0.5848666254043279,  
0.6108617556799849,  
0.46457267109406686,  
0.519906974170459,  
0.498734631981905,  
0.505973310643967,
```

```
0.35925702411496985,  
0.3308405771923436,  
0.606734037324806,  
0.3783690795132647,  
0.5339320857345057,  
0.5935450014887813,  
0.3343045623606754,  
0.6745753728979176,  
0.4658750784825184,  
0.4450376099408852,  
0.6153047520461203,  
0.47039242510117374,  
0.5917760985890255,  
0.5363212489382024,  
0.6053232093920518,  
0.46869479717005014,  
0.5822969490633502,  
0.6917193782153774,  
0.47845226531629487,  
0.47377576632527685,  
0.6157262328676482,  
0.5677634969182028,  
0.6419341084315839,  
0.5439147696833968,  
0.5100762422540623,  
0.48558759440706273,  
0.44349976005457803,  
0.4609708423963622,  
0.5850425572901449,  
0.6101524678814235,  
0.5827334097098966,  
0.5393070577269519,  
0.624376156241494,  
0.6310578737150109,  
0.5747700872274695,  
0.4611341010100567,  
0.5249278209488202,  
0.5015525305823275]
```

```
plt.hist(num_10_100s, bins=14)  
plt.xlabel('xbar')  
plt.ylabel('count')  
plt.xlim(0,1)  
plt.show()
```

Let's Calculate the Average of 10 Numbers 1000 Times

```
random.seed(42)

num_10_1000s = [gen_samples_calc_avg(10) for _ in range(1000)]
```

```
num_10_1000s
```

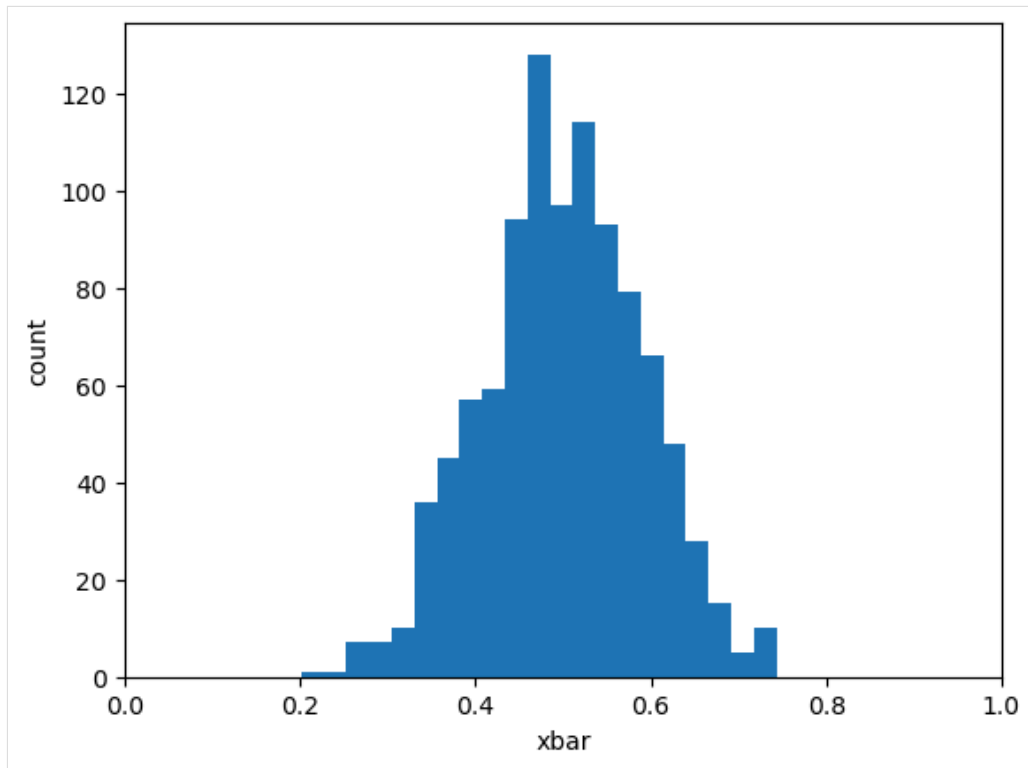
```
[0.4006685751243019,
0.3769828323932711,
0.49341788983981794,
0.686376238606863,
0.29597569781632027,
0.44836101802315814,
0.533428117689419,
0.528091574806805,
0.4570329568579887,
0.5753575301062241,
0.4812527587386003,
0.5300830160193748,
0.5594560699987943,
0.4486082829315742,
0.4579606299821986,
0.4323131269803091,
0.4020481658999553,
0.5180339984328821,
0.4976098693315329,
0.5614202376942153,
0.5433716165239272,
0.5605711093272987,
0.595318519108694,
```

0.42416779210070904,
0.49869962802742684,
0.507829604061832,
0.6085403471214621,
0.45059590489603646,
0.4851957880754007,
0.43623517532803885,
0.6006227477552516,
0.5237048733092099,
0.7223651008715091,
0.5508299621111297,
0.47195243225557065,
0.5491879723111697,
0.5012411998355109,
0.5261024506936252,
0.46012254404717334,
0.4784093951087044,
0.4564321026172258,
0.4333098071191711,
0.48559709819968366,
0.4589175404280795,
0.7307533607570189,
0.5448090085991202,
0.3329021748699117,
0.5799730139025867,
0.5614311179565079,
0.30844719005199556,
0.3304273775552172,
0.4155776220777579,
0.5508967952577224,
0.46814721725259484,
0.5052942116360402,
0.5877682384088321,
0.5848666254043279,
0.6108617556799849,
0.46457267109406686,
0.519906974170459,
0.498734631981905,
0.505973310643967,
0.35925702411496985,
0.3308405771923436,
0.606734037324806,
0.3783690795132647,
0.5339320857345057,
0.5935450014887813,
0.3343045623606754,
0.6745753728979176,
0.4658750784825184,
0.4450376099408852,
0.6153047520461203,
0.47039242510117374,
0.5917760985890255,
0.5363212489382024,
0.6053232093920518,
0.46869479717005014,
0.5822969490633502,
0.6917193782153774,
0.47845226531629487,
0.47377576632527685,
0.6157262328676482,
0.5677634969182028,
0.6419341084315839,

```
0.5439147696833968,  
0.5100762422540623,  
0.48558759440706273,  
0.44349976005457803,  
0.4609708423963622,  
0.5850425572901449,  
0.6101524678814235,  
0.5827334097098966,  
0.5393070577269519,  
0.624376156241494,  
0.6310578737150109,  
0.5747700872274695,  
0.4611341010100567,  
0.5249278209488202,  
0.5015525305823275,  
0.46788756032608403,  
0.44776094970986985,  
0.3805370707664448,  
0.27051823887983345,  
0.5382643200591601,  
0.57115187468849,  
0.4514224036332946,  
0.5599192042033933,  
0.36023663191351424,  
0.5266672282728357,  
0.573070784650344,  
0.5136718354722197,  
0.6127063776883179,  
0.40002858183614504,  
0.429561722077657,  
0.44528917137328944,  
0.38945238338939,  
0.34466905319659746,  
0.5566869335486978,  
0.48473639962721443,  
0.42578738208656,  
0.48206262822111273,  
0.6349617024786431,  
0.4554721989986765,  
0.4625700197277281,  
0.5920333277242678,  
0.3635726723873474,  
0.4558216290730157,  
0.5198965771797138,  
0.5560350202013289,  
0.6056699561229641,  
0.39861121292215407,  
0.5132080455656799,  
0.46635991491511086,  
0.5999094548255295,  
... (860 lines omitted) ...  
0.3652910975718219,  
0.4526580600425577,  
0.5797380404798498,  
0.3544587730381279,  
0.5293835478493831]
```

```
plt.hist(num_10_1000s, bins=21)  
plt.xlabel('xbar')  
plt.ylabel('count')
```

```
plt.xlim(0,1)
plt.show()
```



Let's Calculate the Average of 10 Numbers 10,000 Times

```
random.seed(42)

num_10_10000s = [gen_samples_calc_avg(10) for _ in range(10000)]
```

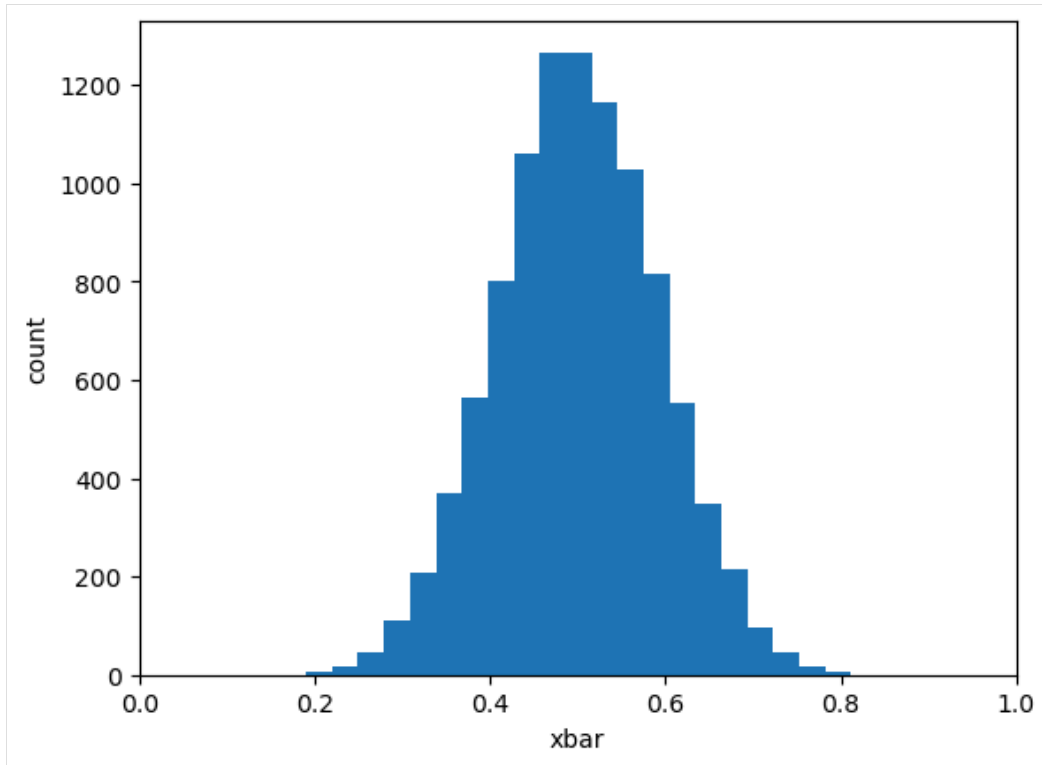
```
num_10_10000s
```

```
[0.4006685751243019,
0.3769828323932711,
0.49341788983981794,
0.686376238606863,
0.29597569781632027,
0.44836101802315814,
0.533428117689419,
0.528091574806805,
0.4570329568579887,
0.5753575301062241,
0.4812527587386003,
0.5300830160193748,
0.5594560699987943,
0.4486082829315742,
0.4579606299821986,
0.4323131269803091,
0.4020481658999553,
0.5180339984328821,
0.4976098693315329,
```

0.5614202376942153,
0.5433716165239272,
0.5605711093272987,
0.595318519108694,
0.42416779210070904,
0.49869962802742684,
0.507829604061832,
0.6085403471214621,
0.45059590489603646,
0.4851957880754007,
0.43623517532803885,
0.6006227477552516,
0.5237048733092099,
0.7223651008715091,
0.5508299621111297,
0.47195243225557065,
0.5491879723111697,
0.5012411998355109,
0.5261024506936252,
0.46012254404717334,
0.4784093951087044,
0.4564321026172258,
0.4333098071191711,
0.48559709819968366,
0.4589175404280795,
0.7307533607570189,
0.5448090085991202,
0.3329021748699117,
0.5799730139025867,
0.5614311179565079,
0.30844719005199556,
0.3304273775552172,
0.4155776220777579,
0.5508967952577224,
0.46814721725259484,
0.5052942116360402,
0.5877682384088321,
0.5848666254043279,
0.6108617556799849,
0.46457267109406686,
0.519906974170459,
0.498734631981905,
0.505973310643967,
0.35925702411496985,
0.3308405771923436,
0.606734037324806,
0.3783690795132647,
0.5339320857345057,
0.5935450014887813,
0.3343045623606754,
0.6745753728979176,
0.4658750784825184,
0.4450376099408852,
0.6153047520461203,
0.47039242510117374,
0.5917760985890255,
0.5363212489382024,
0.6053232093920518,
0.46869479717005014,
0.5822969490633502,
0.6917193782153774,
0.47845226531629487,

```
0.47377576632527685,  
0.6157262328676482,  
0.5677634969182028,  
0.6419341084315839,  
0.5439147696833968,  
0.5100762422540623,  
0.48558759440706273,  
0.44349976005457803,  
0.4609708423963622,  
0.5850425572901449,  
0.6101524678814235,  
0.5827334097098966,  
0.5393070577269519,  
0.624376156241494,  
0.6310578737150109,  
0.5747700872274695,  
0.4611341010100567,  
0.5249278209488202,  
0.5015525305823275,  
0.46788756032608403,  
0.44776094970986985,  
0.3805370707664448,  
0.27051823887983345,  
0.5382643200591601,  
0.57115187468849,  
0.4514224036332946,  
0.5599192042033933,  
0.36023663191351424,  
0.5266672282728357,  
0.573070784650344,  
0.5136718354722197,  
0.6127063776883179,  
0.40002858183614504,  
0.429561722077657,  
0.44528917137328944,  
0.38945238338939,  
0.34466905319659746,  
0.5566869335486978,  
0.48473639962721443,  
0.42578738208656,  
0.48206262822111273,  
0.6349617024786431,  
0.4554721989986765,  
0.462570019727281,  
0.5920333277242678,  
0.3635726723873474,  
0.4558216290730157,  
0.5198965771797138,  
0.5560350202013289,  
0.6056699561229641,  
0.39861121292215407,  
0.5132080455656799,  
0.46635991491511086,  
0.5999094548255295,  
... (861 lines omitted) ...  
0.4526580600425577,  
0.5797380404798498,  
0.3544587730381279,  
0.5293835478493831,  
...]
```

```
plt.hist(num_10_10000s, bins=21)
plt.xlabel('xbar')
plt.ylabel('count')
plt.xlim(0,1)
plt.show()
```



Let's Calculate the Average of 10 Numbers 100,000 Times

```
random.seed(42)

num_10_10000s = [gen_samples_calc_avg(10) for _ in range(100000)]
```

```
num_10_10000s
```

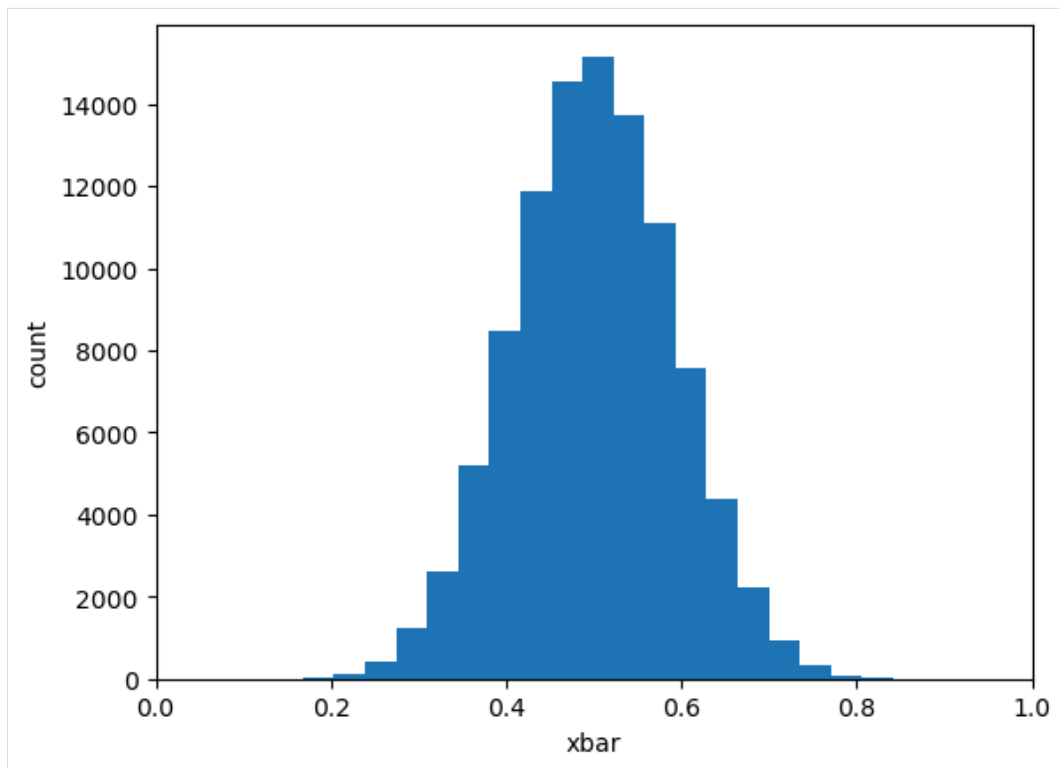
```
[0.4006685751243019,
 0.3769828323932711,
 0.49341788983981794,
 0.686376238606863,
 0.29597569781632027,
 0.44836101802315814,
 0.533428117689419,
 0.528091574806805,
 0.4570329568579887,
 0.5753575301062241,
 0.4812527587386003,
 0.5300830160193748,
 0.5594560699987943,
 0.4486082829315742,
 0.4579606299821986,
 0.4323131269803091,
```

0.4020481658999553,
0.5180339984328821,
0.4976098693315329,
0.5614202376942153,
0.5433716165239272,
0.5605711093272987,
0.595318519108694,
0.42416779210070904,
0.49869962802742684,
0.507829604061832,
0.6085403471214621,
0.45059590489603646,
0.4851957880754007,
0.43623517532803885,
0.6006227477552516,
0.5237048733092099,
0.7223651008715091,
0.5508299621111297,
0.47195243225557065,
0.5491879723111697,
0.5012411998355109,
0.5261024506936252,
0.46012254404717334,
0.4784093951087044,
0.4564321026172258,
0.4333098071191711,
0.48559709819968366,
0.4589175404280795,
0.7307533607570189,
0.5448090085991202,
0.3329021748699117,
0.5799730139025867,
0.5614311179565079,
0.30844719005199556,
0.3304273775552172,
0.4155776220777579,
0.5508967952577224,
0.46814721725259484,
0.5052942116360402,
0.5877682384088321,
0.5848666254043279,
0.6108617556799849,
0.46457267109406686,
0.519906974170459,
0.498734631981905,
0.505973310643967,
0.35925702411496985,
0.3308405771923436,
0.606734037324806,
0.3783690795132647,
0.5339320857345057,
0.5935450014887813,
0.3343045623606754,
0.6745753728979176,
0.4658750784825184,
0.4450376099408852,
0.6153047520461203,
0.47039242510117374,
0.5917760985890255,
0.5363212489382024,
0.6053232093920518,
0.46869479717005014,


```
0.5822969490633502,  
0.6917193782153774,  
0.47845226531629487,  
0.47377576632527685,  
0.6157262328676482,  
0.5677634969182028,  
0.6419341084315839,  
0.5439147696833968,  
0.5100762422540623,  
0.48558759440706273,  
0.44349976005457803,  
0.4609708423963622,  
0.5850425572901449,  
0.6101524678814235,  
0.5827334097098966,  
0.5393070577269519,  
0.624376156241494,  
0.6310578737150109,  
0.5747700872274695,  
0.4611341010100567,  
0.5249278209488202,  
0.5015525305823275,  
0.46788756032608403,  
0.44776094970986985,  
0.3805370707664448,  
0.27051823887983345,  
0.5382643200591601,  
0.57115187468849,  
0.4514224036332946,  
0.5599192042033933,  
0.36023663191351424,  
0.5266672282728357,  
0.573070784650344,  
0.5136718354722197,  
0.6127063776883179,  
0.40002858183614504,  
0.429561722077657,  
0.44528917137328944,  
0.38945238338939,  
0.34466905319659746,  
0.5566869335486978,  
0.48473639962721443,  
0.42578738208656,  
0.48206262822111273,  
0.6349617024786431,  
0.4554721989986765,  
0.4625700197277281,  
0.5920333277242678,  
0.3635726723873474,  
0.4558216290730157,  
0.5198965771797138,  
0.5560350202013289,  
0.6056699561229641,  
0.39861121292215407,  
0.5132080455656799,  
0.46635991491511086,  
0.5999094548255295,  
... (861 lines omitted) ...  
0.4526580600425577,  
0.5797380404798498,  
0.3544587730381279,
```

```
0.5293835478493831,  
...]
```

```
plt.hist(num_10_100000s, bins=21)  
plt.xlabel('xbar')  
plt.ylabel('count')  
plt.xlim(0,1)  
plt.show()
```



Let's Calculate the Average of 10 Numbers 1,000,000 Times

```
random.seed(42)  
  
num_10_1000000s = [gen_samples_calc_avg(10) for _ in range(1000000)]
```

```
num_10_1000000s
```

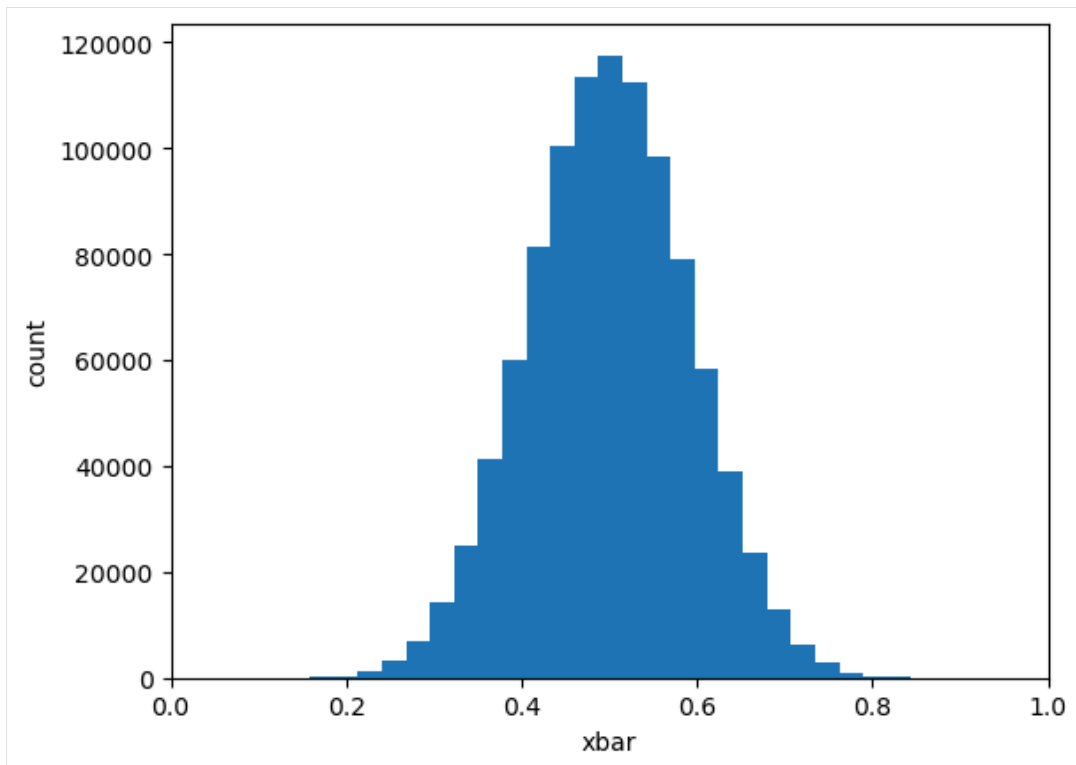
```
[0.4006685751243019,  
0.3769828323932711,  
0.49341788983981794,  
0.686376238606863,  
0.29597569781632027,  
0.44836101802315814,  
0.533428117689419,  
0.528091574806805,  
0.4570329568579887,  
0.5753575301062241,  
0.4812527587386003,
```

0.5300830160193748,
0.5594560699987943,
0.4486082829315742,
0.4579606299821986,
0.4323131269803091,
0.4020481658999553,
0.5180339984328821,
0.4976098693315329,
0.5614202376942153,
0.5433716165239272,
0.5605711093272987,
0.595318519108694,
0.42416779210070904,
0.49869962802742684,
0.507829604061832,
0.6085403471214621,
0.45059590489603646,
0.4851957880754007,
0.43623517532803885,
0.6006227477552516,
0.5237048733092099,
0.7223651008715091,
0.5508299621111297,
0.47195243225557065,
0.5491879723111697,
0.5012411998355109,
0.5261024506936252,
0.46012254404717334,
0.4784093951087044,
0.4564321026172258,
0.4333098071191711,
0.48559709819968366,
0.4589175404280795,
0.7307533607570189,
0.5448090085991202,
0.3329021748699117,
0.5799730139025867,
0.5614311179565079,
0.30844719005199556,
0.3304273775552172,
0.4155776220777579,
0.5508967952577224,
0.46814721725259484,
0.5052942116360402,
0.5877682384088321,
0.5848666254043279,
0.6108617556799849,
0.46457267109406686,
0.519906974170459,
0.498734631981905,
0.505973310643967,
0.35925702411496985,
0.3308405771923436,
0.606734037324806,
0.3783690795132647,
0.5339320857345057,
0.5935450014887813,
0.3343045623606754,
0.6745753728979176,
0.4658750784825184,
0.4450376099408852,
0.6153047520461203,

0.47039242510117374,
0.5917760985890255,
0.5363212489382024,
0.6053232093920518,
0.46869479717005014,
0.5822969490633502,
0.6917193782153774,
0.47845226531629487,
0.47377576632527685,
0.6157262328676482,
0.5677634969182028,
0.6419341084315839,
0.5439147696833968,
0.5100762422540623,
0.48558759440706273,
0.44349976005457803,
0.4609708423963622,
0.5850425572901449,
0.6101524678814235,
0.5827334097098966,
0.5393070577269519,
0.624376156241494,
0.6310578737150109,
0.5747700872274695,
0.4611341010100567,
0.5249278209488202,
0.5015525305823275,
0.46788756032608403,
0.44776094970986985,
0.3805370707664448,
0.27051823887983345,
0.5382643200591601,
0.57115187468849,
0.4514224036332946,
0.5599192042033933,
0.36023663191351424,
0.5266672282728357,
0.573070784650344,
0.5136718354722197,
0.6127063776883179,
0.40002858183614504,
0.429561722077657,
0.44528917137328944,
0.38945238338939,
0.34466905319659746,
0.5566869335486978,
0.48473639962721443,
0.42578738208656,
0.48206262822111273,
0.6349617024786431,
0.4554721989986765,
0.4625700197277281,
0.5920333277242678,
0.3635726723873474,
0.4558216290730157,
0.5198965771797138,
0.5560350202013289,
0.6056699561229641,
0.39861121292215407,
0.5132080455656799,
0.46635991491511086,
0.5999094548255295,

```
... (861 lines omitted) ...
0.4526580600425577,
0.5797380404798498,
0.3544587730381279,
0.5293835478493831,
...]
```

```
plt.hist(num_10_100000s, bins=28)
plt.xlabel('xbar')
plt.ylabel('count')
plt.xlim(0,1)
plt.show()
```



Create a Group of Charts Demonstrating a Normal Distribution Occuring w/ Increased Counts

- The distribution of the average is converging to a gaussian or normal distribution; bell curve.
- This is the basis for the **Central Limit Theorem**.
- The **Central Limit Theorem** tells us that the average is distributed normally, therefore, we can express the variability of the average using a statistic that represents the width of the bell curve.
- The variance is a statistic that describes variation or variability.

```
# Data preparation
if 'data_lists' not in locals():
    data_lists = [num_10_100s, num_10_1000s, num_10_10000s, num_10_100000s, num_10_1000000s]
if 'labels' not in locals():
    labels = [100,
              1000,
              10000,
              100000,
```

```

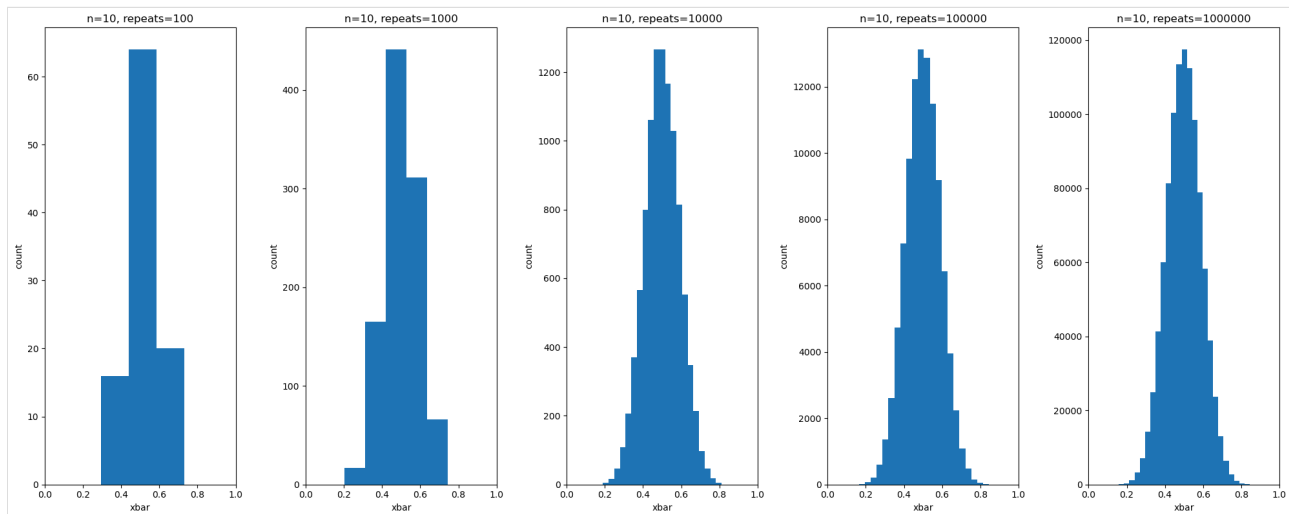
1000000]

# Create subplots
fig, axes = plt.subplots(1,
                        len(data_lists),
                        figsize=(20, 8))

# Iterate through data lists and axes to create histograms
for i, (ax, data, label) in enumerate(zip(axes,
                                         data_lists,
                                         labels)):

    if i == 0:
        bins = 3
    elif i == 1:
        bins = 5
    elif i == 2:
        bins = 21
    elif i == 3:
        bins = 24
    else:
        bins = 28
    ax.hist(data, bins=bins)
    ax.set_xlabel('xbar')
    ax.set_ylabel('count')
    ax.set_title('n=10, repeats={}'.format(label))
    ax.set_xlim(0, 1)
plt.tight_layout()
plt.show()

```



Central Limit Theorem, Variance, and Standard Deviation

Definition: Let $X_1, X_2, \dots, X_n$ be i.i.d. random variables with mean μ and variance $\sigma^2 < \infty$. The distribution of the sample mean $\bar{X}_n$ approaches a normal distribution as $n \rightarrow \infty$.

- Sample mean: $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$
- CLT (in distribution): $\sqrt{n}(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$

Equivalently: $\bar{X}_n \xrightarrow{d} \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right)$

Variance = the average of the squared deviations from the mean.

- Population variance: $\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$
- Sample (unbiased) variance: $s^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2$

Standard deviation = the square root of the variance.

- Population standard deviation: $\sigma = \sqrt{\sigma^2}$
- Sample standard deviation: $s = \sqrt{s^2}$

THE CENTRAL LIMIT THEOREM STATES THAT THE DISTRIBUTION OF THE SAMPLE MEAN OF A LARGE NUMBER OF INDEPENDENT, IDENTICALLY DISTRIBUTED RANDOM VARIABLES WITH FINITE MEAN AND VARIANCE APPROACHES A NORMAL DISTRIBUTION (WITH MEAN μ AND VARIANCE σ^2/n) AS THE SAMPLE SIZE n GROWS.

VARIANCE = THE AVERAGE OF THE SQUARED VERTICAL DISTANCES FROM EACH DATA POINT TO THE MEAN.

VARIANCE = THE AVERAGE OF THE SQUARED DEVIATIONS FROM THE MEAN.

STANDARD DEVIATION = THE SQUARE ROOT OF THE VARIANCE.

Let's Create a Variance Function

```
def sample_variance(data):  
    """  
    Calculate the sample variance of a list of numbers (Bessel's correction: divide by n-1).  
  
    Parameters:  
        data (list): A list of numbers.  
  
    Returns:  
        float: The sample variance of the numbers.  
    """  
    mean = sum(data) / len(data)  
    squared_deviation = [(x - mean) ** 2 for x in data]  
    return sum(squared_deviation) / (len(data) - 1)
```

Determine the Variance of the Average

```
sample_variance([5,  
                10,  
                15])
```

25.0

Let's Make a Function to Calculate the Sample Variance

```
def sample_sd(data):  
    """
```

Calculate the sample standard deviation of a list of numbers (using Bessel's correction, $n-1$).

Parameters:

data (list): A list of numbers.

Returns:

float: The sample standard deviation of the numbers.

"""

return sample_variance(data) ** 0.5

```
sample_sd([5,
           10,
           15])
```

5.0

Visualize How the Sample Variance and Sample Standard Deviation Work

```
# Data
data = [5,
        10,
        15]
mean = sum(data) / len(data)
n = len(data)
x = data

# Calculate squared deviations and SD
sq_diffs = [(val - mean)**2 for val in data]
variance = sum(sq_diffs) / (n - 1)
sample_sd_ = variance ** 0.5

# Plot points and mean
plt.figure(figsize=(12, 6))
plt.scatter(x,
            data,
            color='blue',
            s=120,
            label='Data Points')
plt.axhline(mean,
            color='red',
            linestyle='--',
            lw=2,
            label='Mean = {:.1f}'.format(mean))

# Draw deviation lines and annotate
for i, y in enumerate(data):
    plt.plot([x[i],
              x[i]],
             [mean, y],
             'k:', lw=1)
    diff = y - mean
    # Purple annotation
    plt.text(x[i],
             y+1 if y>mean else y-2,
             '({}-10)={:.1f}'.format(y, diff),
             color='purple',
             fontsize=8)
    # Orange annotation, just 0.5 units above/below the purple one
```



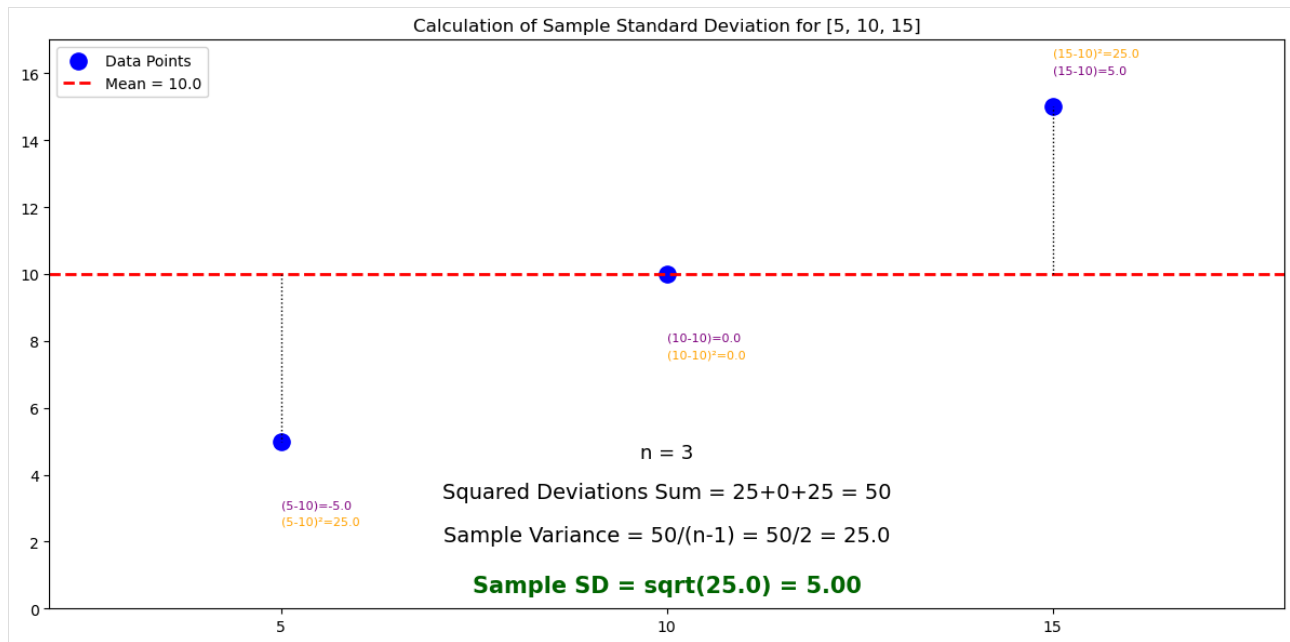
```

plt.text(x[i],
         (y+1.5 if y>mean else y-2.5),
         '({}-10)^2={:.1f}'.format(y, diff**2),
         color='orange',
         fontsize=8)

# SD calculation steps
plt.text(10,
         4.5,
         'n = {}'.format(n),
         fontsize=13,
         color='black',
         ha='center')
plt.text(10,
         3.3,
         'Squared Deviations Sum = {} = {}'.format("+" .join(str(int(sd)) for sd in sq_diffs),
                                                    int(sum(sq_diffs))),
         color='black',
         fontsize=14,
         ha='center')
plt.text(10,
         2.0,
         'Sample Variance = {}/(n-1) = {} / 2 = {:.1f}'.format(int(sum(sq_diffs)),
                                                            int(sum(sq_diffs)),
                                                            variance),
         color='black',
         fontsize=14,
         ha='center')
plt.text(10,
         0.5,
         'Sample SD = sqrt({:.1f}) = {:.2f}'.format(variance, sample_sd_),
         color='darkgreen',
         fontsize=15,
         fontweight='bold',
         ha='center')

# Calculation of Sample Standard Deviation
plt.title('Calculation of Sample Standard Deviation for [5, 10, 15]')
plt.xticks(data,
           [str(v) for v in data])
plt.xlim(2, 18)
plt.ylim(0, 17)
plt.legend(loc='upper left')
plt.grid(False)
plt.tight_layout()
plt.show()

```



```
sample_size_study = [3 * (2 ** ix) for ix in range(15)]
```

```
sample_size_study
```

```
[3, 6, 12, 24, 48, 96, 192, 384, 768, 1536, 3072, 6144, 12288, 24576, 49152]
```

Let's Replicate 2500 Times Calculating the Average from a Sample Size to Ensure a Normal Distribution for the Sample Average

```
sample_sd([1,
            2,
            3])
```

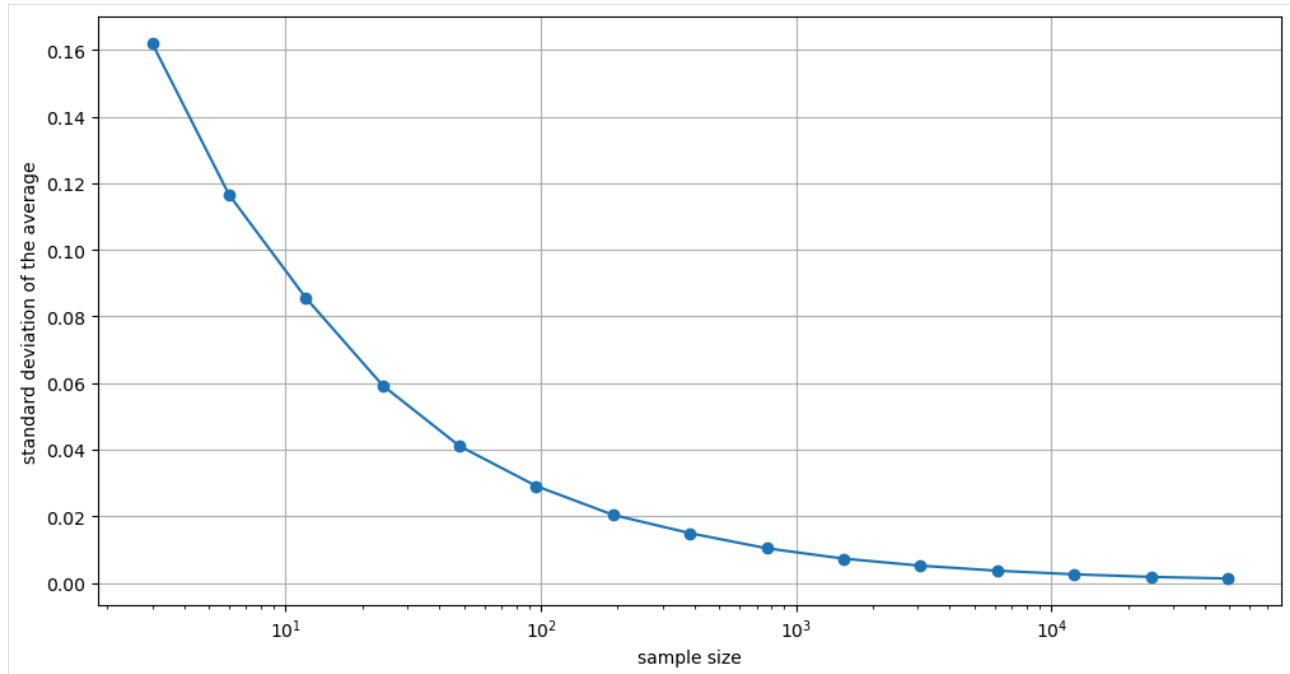
```
1.0
```

```
random.seed(42)
```

```
study_sd_vs_sample_size = [sample_sd([gen_samples_calc_avg(sample) for _ in range(2500)]) for sample
in sample_size_study]
```

Show the Standard Deviation of the Average wrt Sample Size

```
fig, ax = plt.subplots(figsize=(12, 6))
ax.plot(sample_size_study, study_sd_vs_sample_size, '-o')
ax.set_xscale('log')
ax.set_xlabel('sample size')
ax.set_ylabel('standard deviation of the average')
ax.grid(True)
plt.show()
```



The standard deviation on the average decreases as the sample size increases.

The standard deviation on the average is called the standard error on the mean or SEM.

SEM = (standard_deviation) / (square root of the number of samples)

$$\text{SEM} = \frac{\sigma}{\sqrt{N}}$$

The σ is the standard deviation of the measurements. We are currently using a random number generator that generates **uniform** random numbers. There is a known formula for the **variance** of a **uniform** distribution.

$$\text{SEM} = \frac{1}{\sqrt{12 \times N}}$$

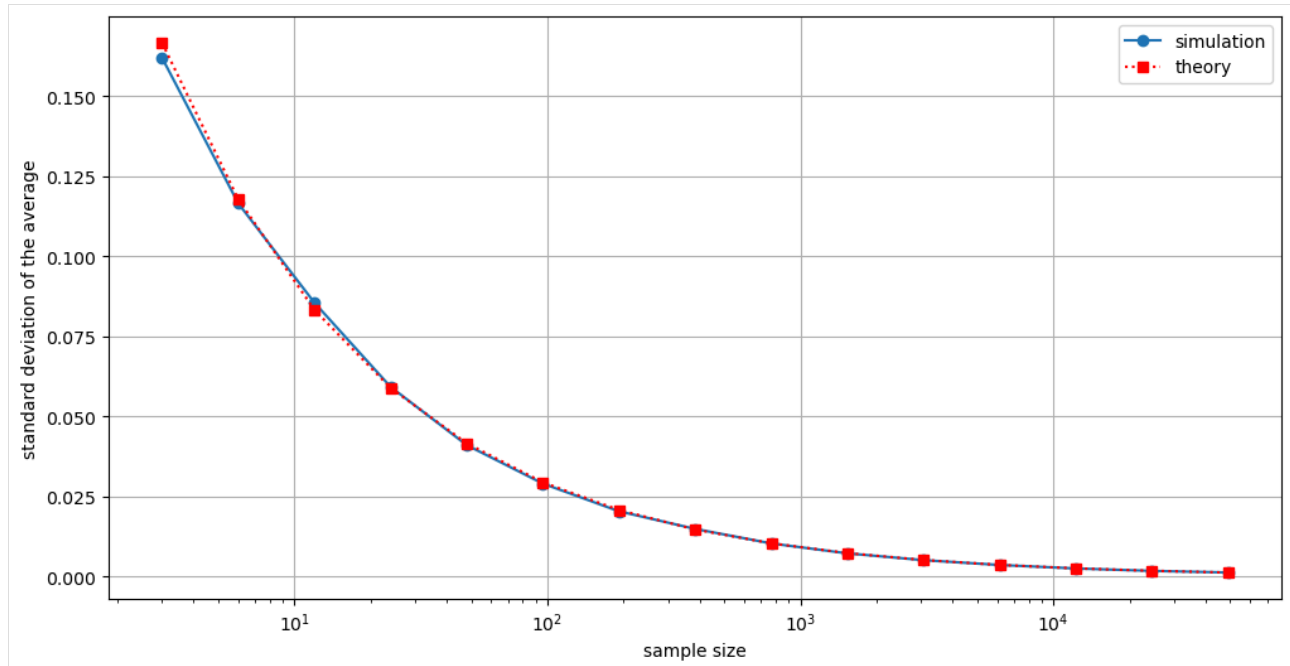
or

$$\text{SEM} = \left(\frac{1/12}{N} \right)^{0.5}$$

Let's use this formula and compare this theoretical to our simulation results...

```
sem_theory = [(1/12) / sample) ** 0.5 for sample in sample_size_study]
```

```
fig, ax = plt.subplots(figsize=(12, 6))
ax.plot( sample_size_study, study_sd_vs_sample_size, '-o', label='simulation')
ax.plot( sample_size_study, sem_theory, ':sr', label='theory')
ax.set_xscale('log')
ax.set_xlabel('sample size')
ax.set_ylabel('standard deviation of the average')
ax.grid(True)
ax.legend()
plt.show()
```



WE COLLECT DATA AND MORE OFTEN THAN NOT, WE HAVE RELATIVELY SMALL SAMPLE SIZES.

HOW CONFIDENT ARE WE ON AVERAGE BASED ON THESE SMALL SAMPLE SIZES REPRESENTING THE TOTAL POPULATION?

WE CAN EXPRESS CONFIDENCE USING THE CLT AND THE STANDARD ERROR.

The confidence interval represents where you anticipate the true population average to be, based on your sample data and the chosen confidence level.

The most common confidence interval is 95%.

Small Sample Set

```
random.seed(42)

small_sample = [random.random() for _ in range(10)]
```

```
small_sample
```

```
[0.6394267984578837,
 0.025010755222666936,
 0.27502931836911926,
 0.22321073814882275,
 0.7364712141640124,
 0.6766994874229113,
 0.8921795677048454,
 0.08693883262941615,
 0.4219218196852704,
 0.029797219438070344]
```

```
small_sample_avg = my_avg(small_sample)
```

REVERSE ENGINEERING DATA

```
small_sample_avg
```

```
0.4006685751243019
```

```
sample_sd_ = sample_sd(small_sample)
```

```
sample_sd_
```

```
0.3183924797690624
```

```
sem = sample_sd_ / (len(small_sample) ** 0.5)
```

```
sem
```

```
0.10068454259393185
```

```
conf_int = (small_sample_avg - 2 * sem,  
            small_sample_avg + 2 * sem)
```

```
conf_int
```

```
(0.19929948993643817, 0.6020376603121655)
```

```
small_sample_avg
```

```
0.4006685751243019
```

```
conf_int[1] - conf_int[0]
```

```
0.40273817037572734
```

Large Sample Set

```
random.seed(42)  
  
large_sample = [random.random() for _ in range(1000000)]
```

```
large_sample
```

```
[0.6394267984578837,  
0.025010755222666936,  
0.27502931836911926,  
0.22321073814882275,  
0.7364712141640124,  
0.6766994874229113,  
0.8921795677048454,  
0.08693883262941615,  
0.4219218196852704,  
0.029797219438070344,  
0.21863797480360336,  
0.5053552881033624,  
0.026535969683863625,  
0.1988376506866485,  
0.6498844377795232,  
0.5449414806032167,  
0.2204406220406967,  
0.5892656838759087,  
0.8094304566778266,  
0.006498759678061017,  
0.8058192518328079,  
0.6981393949882269,  
0.3402505165179919,  
0.15547949981178155,  
0.9572130722067812,  
0.33659454511262676,  
0.09274584338014791,  
0.09671637683346401,  
0.8474943663474598,  
0.6037260313668911,  
0.8071282732743802,  
0.7297317866938179,  
0.5362280914547007,  
0.9731157639793706,  
0.3785343772083535,  
0.552040631273227,  
0.8294046642529949,  
0.6185197523642461,  
0.8617069003107772,  
0.577352145256762,  
0.7045718362149235,  
0.045824383655662215,  
0.22789827565154686,  
0.28938796360210717,  
0.0797919769236275,  
0.23279088636103018,  
0.10100142940972912,  
0.2779736031100921,  
0.6356844442644002,  
0.36483217897008424,  
0.37018096711688264,  
0.2095070307714877,  
0.26697782204911336,  
0.936654587712494,  
0.6480353852465935,  
0.6091310056669882,  
0.171138648198097,  
0.7291267979503492,  
0.1634024937619284,  
0.3794554417576478,  
0.9895233506365952,  
0.6399997598540929,
```

0.5569497437746462,
0.6846142509898746,
0.8428519201898096,
0.7759999115462448,
0.22904807196410437,
0.03210024390403776,
0.3154530480590819,
0.26774087597570273,
0.21098284358632646,
0.9429097143350544,
0.8763676264726689,
0.3146778807984779,
0.65543866529488,
0.39563190106066426,
0.9145475897405435,
0.4588518525873988,
0.26488016649805246,
0.24662750769398345,
0.5613681341631508,
0.26274160852293527,
0.5845859902235405,
0.897822883602477,
0.39940050514039727,
0.21932075915728333,
0.9975376064951103,
0.5095262936764645,
0.09090941217379389,
0.04711637542473457,
0.10964913035065915,
0.62744604170309,
0.7920793643629641,
0.42215996679968404,
0.06352770615195713,
0.38161928650653676,
0.9961213802400968,
0.529114345099137,
0.9710783776136181,
0.8607797022344981,
0.011481021942819636,
0.7207218193601946,
0.6817103690265748,
0.5369703304087952,
0.2668251899525428,
0.6409617985798081,
0.11155217359587644,
0.434765250669105,
0.45372370632920644,
0.9538159275210801,
0.8758529403781941,
0.26338905075109076,
0.5005861130502983,
0.17865188053013137,
0.9126278393448205,
0.8705185698367669,
0.2984447914486329,
0.6389494948660052,
0.6089702114381723,
0.1528392685496348,
0.7625108000751513,
0.5393790301196257,
0.7786264786305582,
0.5303536721951775,

```
0.0005718961279435053,  
0.3241560570046731,  
0.019476742385832302,  
0.9290986162646171,  
0.8787218778231842,  
0.8316655293611794,  
0.30751412540266143,  
0.05792516649418755,  
0.8780095992040405,  
0.9469494452979941,  
0.08565345206787878,  
... (861 lines omitted) ...  
0.3705299587344354,  
0.44338308606070165,  
0.950555169851427,  
0.8554501933059546,  
...]
```

```
large_sample_avg = my_avg(large_sample)
```

```
large_sample_avg
```

```
0.5000145439034674
```

```
sample_sd_ = sample_sd(large_sample)
```

```
sample_sd_
```

```
0.28849403201032836
```

```
sem = sample_sd_ / (len(large_sample) ** 0.5)
```

```
sem
```

```
0.00028849403201032834
```

```
conf_int = (large_sample_avg - 2 * sem,  
            large_sample_avg + 2 * sem)
```

```
conf_int
```

```
(0.49943755583944677, 0.5005915319674881)
```

```
conf_int[1] - conf_int[0]
```



```
0.0011539761280413008
```

Key Takeaways

- The average of many independent observations is approximately normally distributed, regardless of the shape of the underlying distribution.
- The standard deviation of the sample average is the standard error of the mean, and it shrinks in proportion to the square root of the sample size.
- A confidence interval expresses how confident we are that the true population average lies in a range.
- Small samples produce wide intervals; pretending otherwise is the first step toward bad decisions.

Exercises

1. Modify the sample-size experiment to use a sample size of 100. How much narrower is the sampling distribution than with a sample size of 10?
2. Write a function that returns a 95 percent confidence interval for the mean of a list.
3. Generate data from an exponential distribution and confirm that the sampling distribution of the mean is still approximately normal for large samples.
4. Explain in two sentences why the standard error of the mean is not the standard deviation of the data.
5. Halve the sample size. By what factor does the standard error change?

Guided Walkthrough

The notebook opens by building the smallest possible tools. A `for` loop prints numbers, then stores random numbers in a list, then a list comprehension does the same job in one line. The point is not the syntax; it is that every later step is built from these primitives. The `my_avg` function is the first abstraction, and it is deliberately tiny so that nothing is hidden.

The first experiment generates three observations for ten people and averages them. Because each person has only three measurements, the ten averages are spread out. The histogram is lumpy and wide. The notebook then repeats the experiment with ten, a thousand, ten thousand, one hundred thousand, and one million samples per average. Watch the histograms in sequence. Each one is smoother and narrower than the last. That sequence is the central limit theorem happening in front of you.

The second half separates two quantities that beginners constantly confuse. The variance of the raw data measures how spread out individual observations are, and it does not shrink as the sample grows. The variance of the average measures how spread out the averages are, and it does shrink. The notebook computes both, plots the standard deviation of the average against the sample size, and shows the curve falling like one over the square root of n . That curve is the standard error of the mean, and it is the bridge to the confidence interval at the end, where a small sample produces a wide interval and a large sample produces a narrow one.

Common Pitfalls

- Confusing the standard deviation of the data with the standard error of the mean. They answer different questions.
- Assuming the theorem applies to dependent observations; independence is required.
- Believing a large sample makes a biased estimator unbiased. It does not.

- Reporting a point estimate without an interval.

Going Further

- Repeat the experiment with a skewed distribution, such as exponential waiting times, and confirm the sampling distribution still becomes normal.
- Derive the one-over-root-n rule from the variance of a sum of independent variables.
- Read about the law of large numbers, which guarantees convergence, and the central limit theorem, which describes the shape of the fluctuations around the limit.

Exercise Solutions

1. With a sample size of 100 the sampling distribution is about the square root of ten, roughly 3.2 times, narrower than with a sample size of 10.
2. A 95 percent interval is the sample mean plus or minus 1.96 times the standard error, where the standard error is the sample standard deviation divided by the square root of n .
3. The exponential distribution is skewed, yet the averages of large samples still form a bell curve, which is the theorem in action.
4. The standard deviation of the data describes individual observations; the standard error of the mean describes the precision of the average.
5. Halving the sample size multiplies the standard error by the square root of two, about 1.41.

Chapter 2: Penguins Dataset - EDA

Introduction

Exploratory data analysis is the conversation you have with a dataset before you ask it to answer a question. It is where you learn what the columns mean, where the holes are, which variables move together, and which values are impossible. Skipping it is the most reliable way to build a model that is confidently wrong. This chapter performs a complete exploratory analysis of the Palmer Penguins dataset, one of the cleanest and most instructive datasets in statistics.

The dataset describes three species of penguins measured on three islands. Each row is a bird, and each column is a physical measurement or a category. By the end of the chapter you will know how to profile a dataset's shape and types, quantify its missing values, visualize the distribution of every variable, and read relationships between variables from plots. Every technique here generalizes to datasets a thousand times larger.

What You Will Learn

- How to load a dataset and inspect its shape, types, and summary statistics.
- How to find and reason about missing values.
- How to visualize the distribution of continuous and categorical variables.
- How to read relationships between variables with scatter plots and pair plots.
- How to spot outliers, impossible values, and class imbalance before modeling.

Overview

The chapter follows a fixed checklist that you can reuse on any tabular dataset. First, size and types: how many rows, how many columns, and which columns are numeric versus categorical. Second, missingness: how many values are absent, and is the absence random or systematic. Third, univariate distributions: what does each variable look like on its own. Fourth, bivariate relationships: how does each input relate to the target and to the other inputs. Fifth, anomalies: values that violate the definitions of the columns.

The Penguins dataset rewards careful looking. The three species are separated most cleanly by flipper length and bill length, but there is overlap, which is exactly what makes a classification task interesting rather than trivial. Body mass correlates with the other measurements, which matters for regression. The island variable is entangled with species, which is a trap: a model can appear accurate by learning the island rather than the bird. Recognizing that entanglement during exploration is what saves you from a leaked feature later.

By the time you finish, you will have a mental picture of the dataset that no summary table can give you, and that picture is what guides every modeling decision that follows.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island       object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm  float64
body_mass_g   float64
sex          object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island       0
bill_length_mm    2
bill_depth_mm    2
flipper_length_mm  2
body_mass_g      2
sex          11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island       0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm  0
body_mass_g   0
sex          0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island       3
bill_length_mm 163
bill_depth_mm  79
flipper_length_mm 54
body_mass_g   93
sex          2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm 163
bill_depth_mm  79
flipper_length_mm 54
body_mass_g   93
dtype: int64
```

Save the cleaned dataset for use in other project notebooks.

```
df_clean.to_csv('data/penguins-clean.csv', index=False)
```

Convert Data: Long Format

Categorical Names

```
df_clean_category_names = df_clean.select_dtypes(['object',  
                                                  'category']). \  
    columns.tolist()
```

```
df_clean_category_names
```

```
['species', 'island', 'sex']
```

Numeric Names

```
df_clean_numeric_names = df_clean.select_dtypes('number'). \  
    columns.tolist()
```

```
df_clean_numeric_names
```

```
['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
```

Reshape

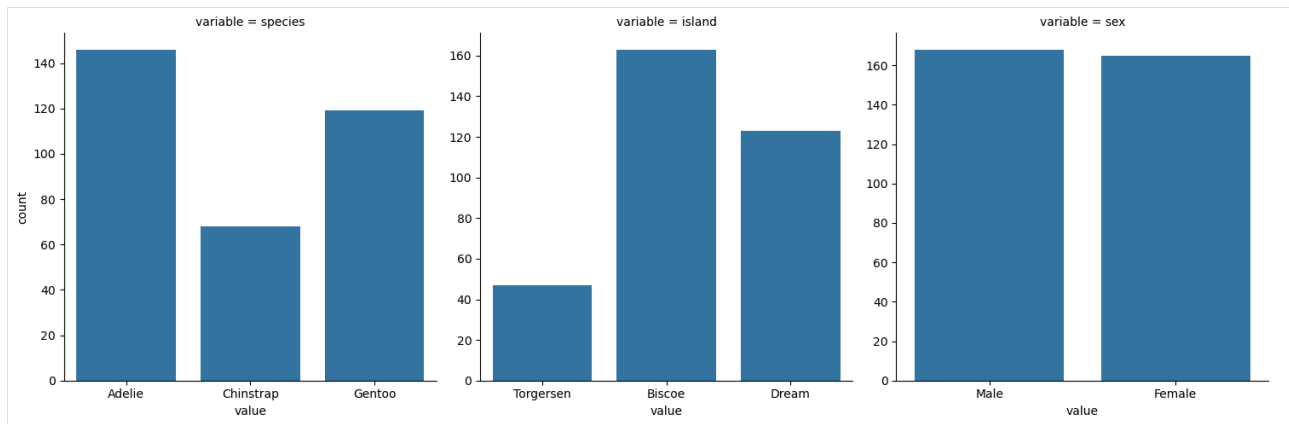
```
df_clean_lf = df_clean.reset_index(). \  
    rename(columns={'index': 'rowid'}). \  
    melt(id_vars=['rowid'] + df_clean_category_names, \  
         value_vars=df_clean_numeric_names)
```

Visually Explore Data

Count Plots – Marginal Distributions of Each Categorical Variable, with Facets for Each Categorical Variable

The data reveals a roughly equal distribution of male and female penguins, with Adelie being the most common species and Biscoe the most represented island, while Chinstrap and Torgersen are the least frequent species and island, respectively.

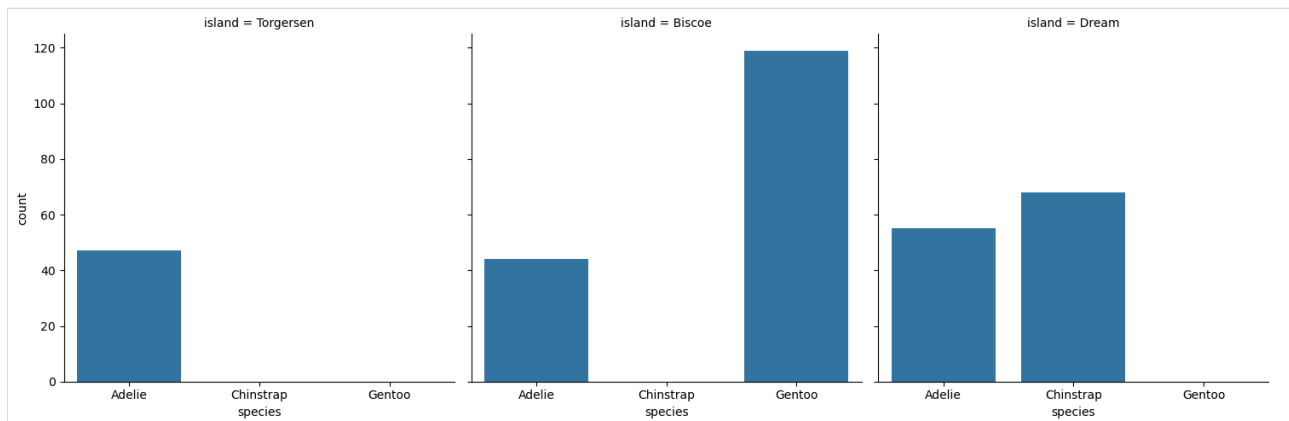
```
sns.catplot(data=df_clean.melt(value_vars=df_clean_category_names),  
            x='value',  
            col='variable',  
            col_wrap=3,  
            kind='count',  
            sharex=False,  
            sharey=False)  
  
plt.show()
```



Count Plots – Marginal Distributions of the `species` Categorical Variable, with Facets for Each `island` Categorical Variable

Gentoo penguins are found exclusively on Biscoe island, Chinstrap only on Dream island, while Adelie is present on all three islands.

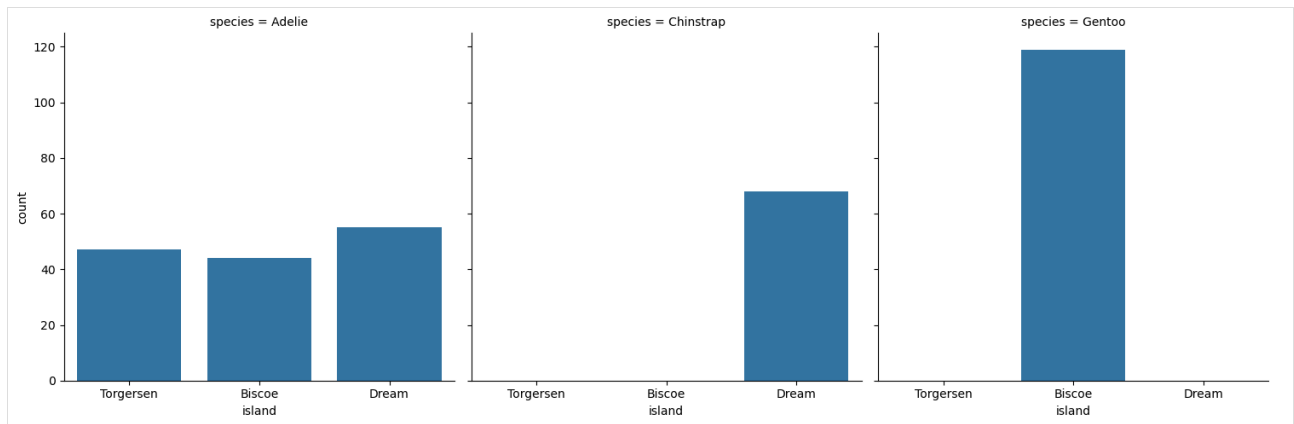
```
sns.catplot(data=df_clean,
            x='species',
            col='island',
            kind='count')
plt.show()
```



Count Plots – Marginal Distributions of the `island` Categorical Variable, with Facets for Each `species` Categorical Variable

Adelie penguins are evenly distributed across all three islands, Chinstrap is found only on Dream island, and Gentoo is present exclusively on Biscoe island.

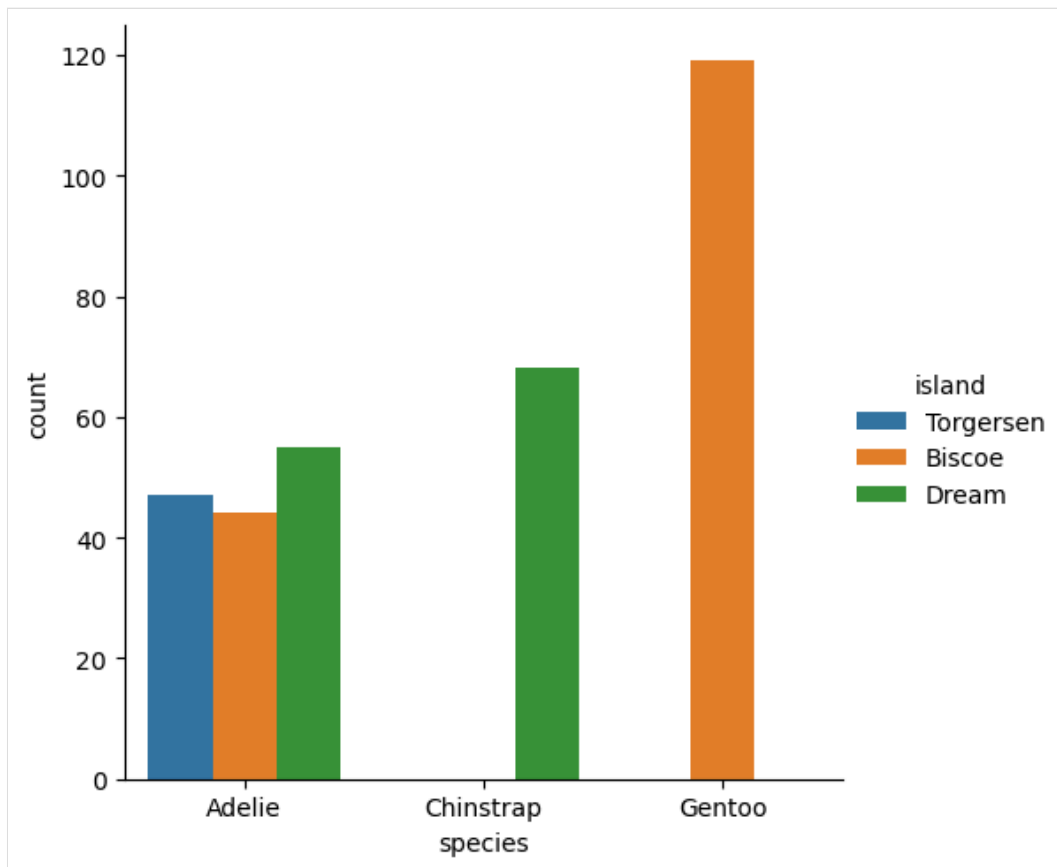
```
sns.catplot(data=df_clean,
            x='island',
            col='species',
            kind='count')
plt.show()
```



Count Plots – Marginal Distributions of the `species` Categorical Variable, Colored by Each `island` Categorical Variable

Adelie penguins are distributed across all islands, Chinstrap is exclusive to Dream island, and Gentoo is found only on Biscoe island.

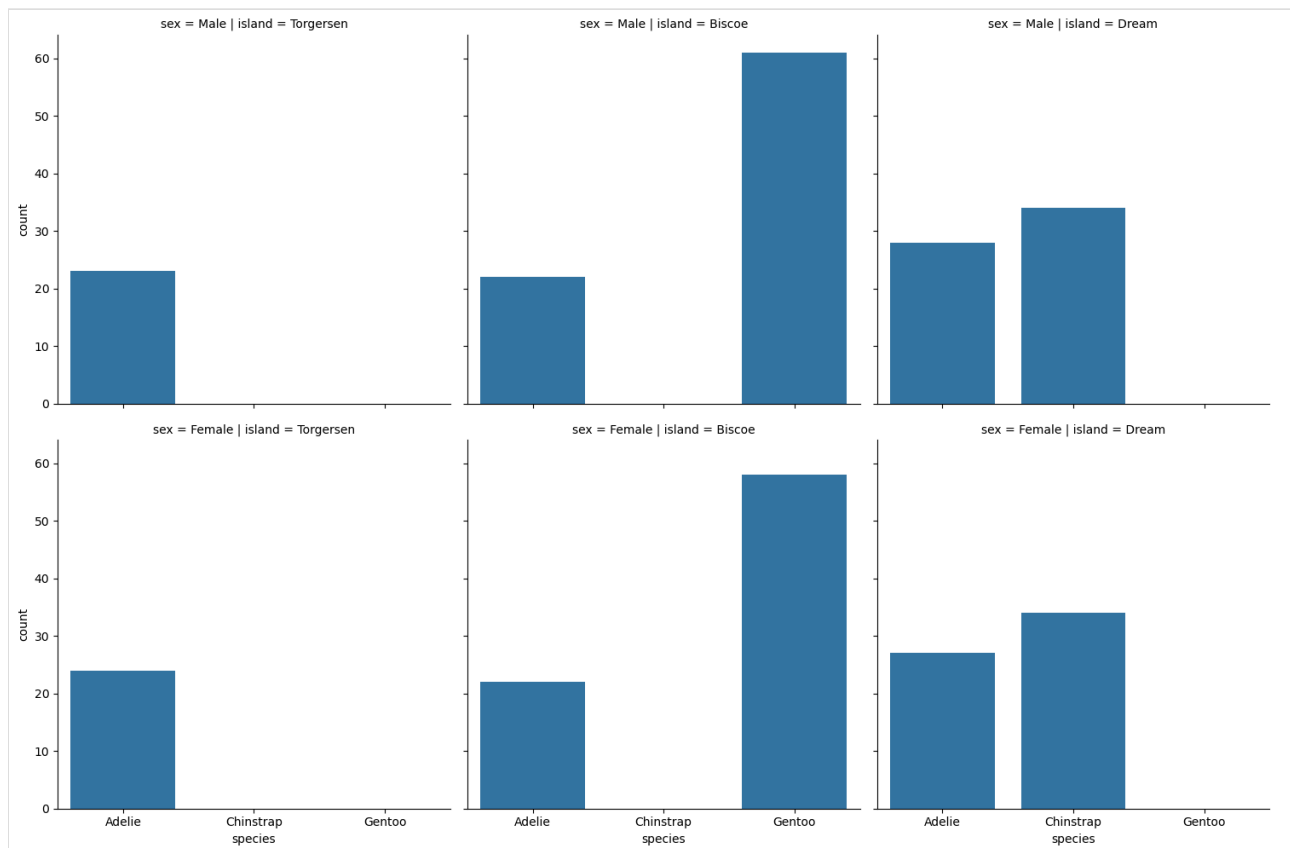
```
sns.catplot(data=df_clean,  
            x='species',  
            hue='island',  
            kind='count')  
plt.show()
```



Count Plots – Marginal Distributions of the `species` Categorical Variable, with Facets for Each Combination of `sex` and `island` Categorical Variables

Across all islands, Adelie penguins are evenly distributed between sexes, Chinstrap is found only on Dream island with more females than males, and Gentoo is exclusive to Biscoe island, where males are slightly more numerous than females.

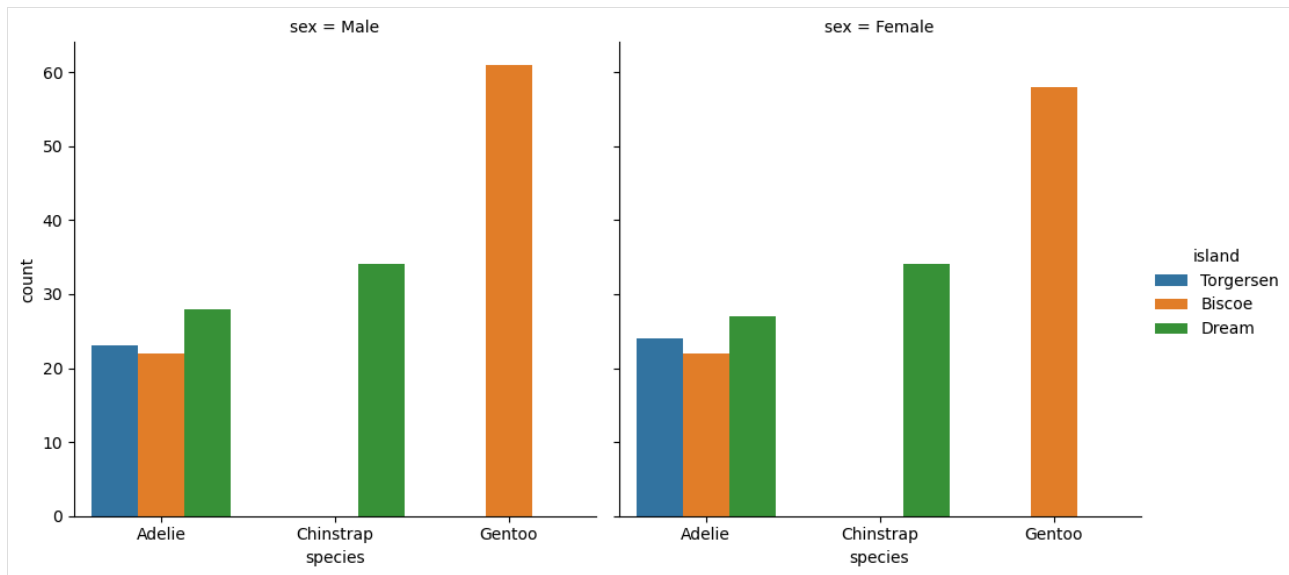
```
sns.catplot(data=df_clean,
            x='species',
            row='sex',
            col='island',
            kind='count')
plt.show()
```



Count Plots – Marginal Distributions of the `species` Categorical Variable, Colored by Each `island` Categorical Variable, with Facets for Each `sex` Categorical Variable

Both males and females show a similar distribution: Adelie penguins are present on all islands, Chinstrap is exclusive to Dream island, and Gentoo is found only on Biscoe island.

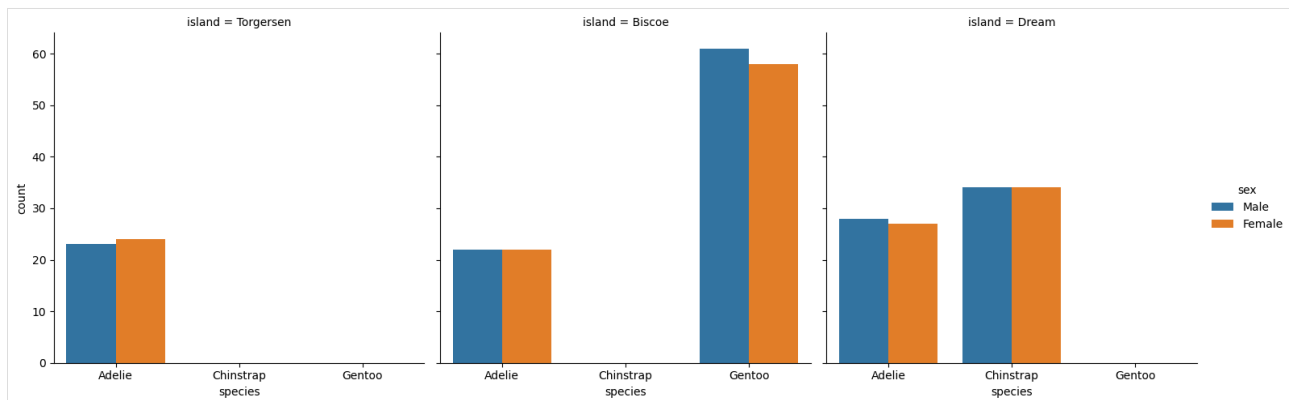
```
sns.catplot(data=df_clean,
            x='species',
            hue='island',
            col='sex',
            kind='count')
plt.show()
```



Count Plots – Marginal Distributions of the `species` Categorical Variable, Colored by Each `sex` Categorical Variable, with Facets for Each `island` Categorical Variable

On each island, Adelie penguins are evenly split between sexes, Gentoo is exclusive to Biscoe with similar male and female counts, and Chinstrap is found only on Dream island with equal representation of both sexes.

```
sns.catplot(data=df_clean,
            x='species',
            hue='sex',
            col='island',
            kind='count')
plt.show()
```

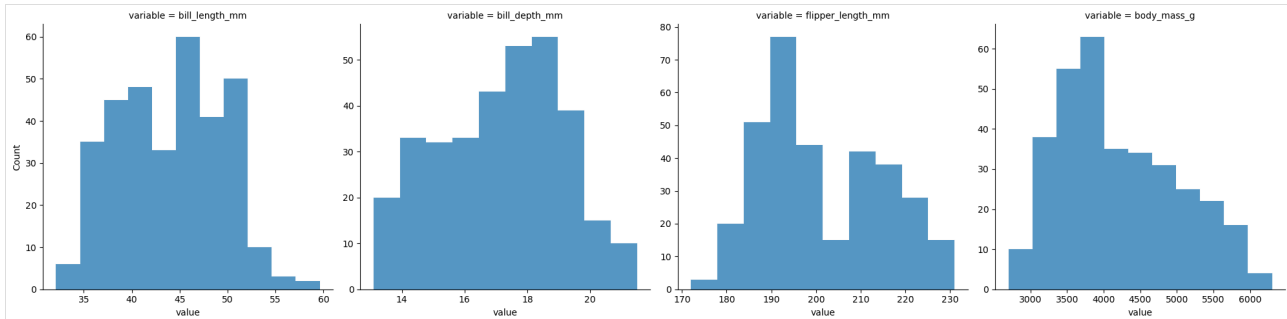


Histogram Plots – Marginal Distributions of Each Numerical Variable, with Facets for Each Numerical Variable

The distributions of bill length, bill depth, flipper length, and body mass are all approximately unimodal and show moderate variation, with most measurements concentrated around their respective central values.

```
sns.displot(data=df_clean_lf,
            x='value',
            col='variable',
            col_wrap=4,
```

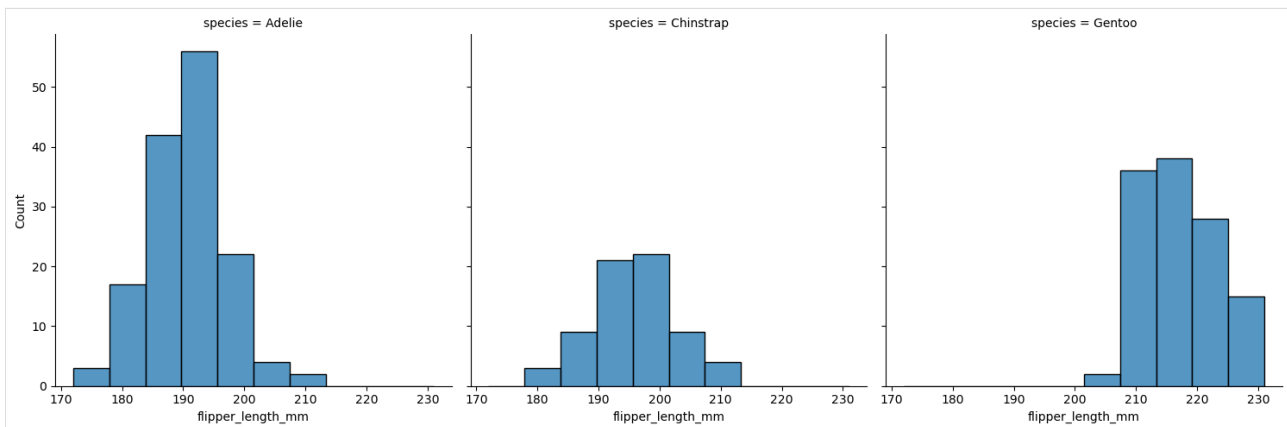
```
kind='hist',
facet_kws={'sharex': False,
           'sharey': False},
common_bins=False)
plt.show()
```



Histogram Plots – Marginal Distributions of the `flipper_length_mm` Continuous Variable, with Facets for Each `species` Categorical Variable

Adelie and Chinstrap penguins have overlapping flipper length distributions centered around 190–200 mm, while Gentoo exhibits distinctly longer flipper lengths, mostly between 210 and 230 mm.

```
sns.displot(data=df_clean,
            x='flipper_length_mm',
            col='species',
            kind='hist')
plt.show()
```

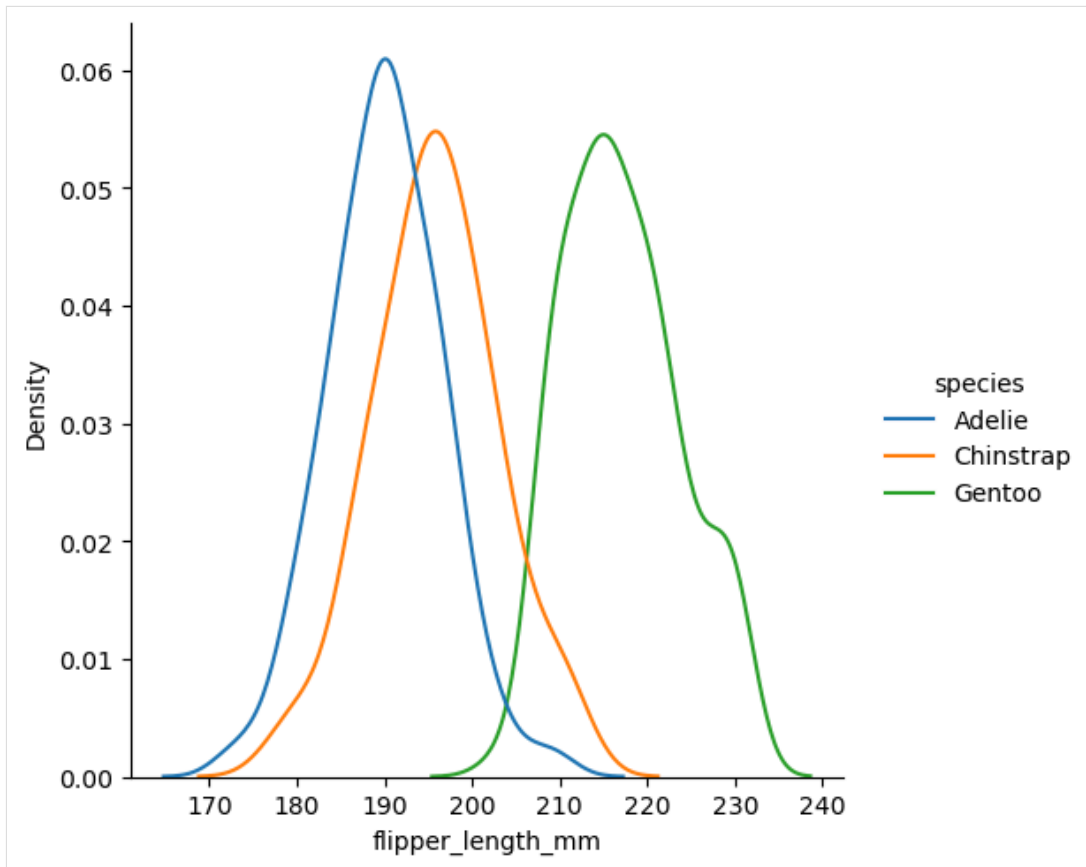


KDE Plot – Marginal Distribution Density of the `flipper_length_mm` Continuous Variable, Colored by Each `species` Categorical Variable

The density plot shows that Adelie and Chinstrap penguins have overlapping flipper length distributions centered around 190–200 mm, while Gentoo exhibits distinctly longer flipper lengths, mostly between 210 and 230 mm.

```
sns.displot(data=df_clean,
            x='flipper_length_mm',
            hue='species',
            common_norm=False,
```

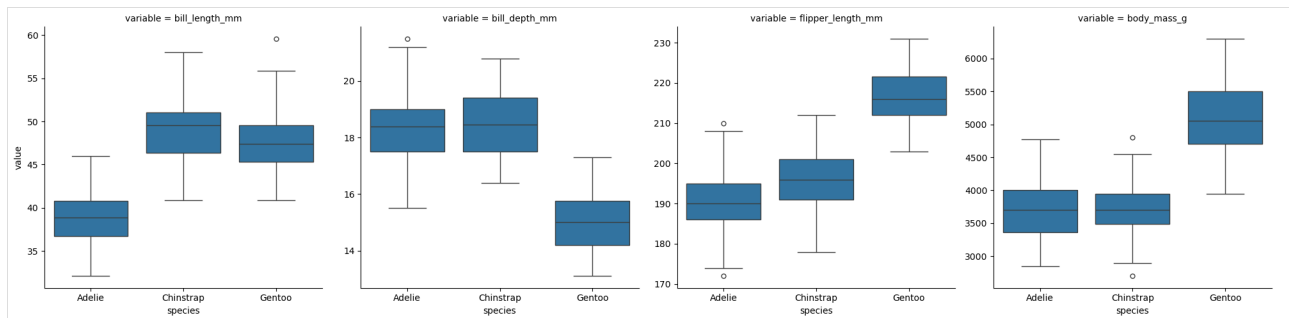
```
kind='kde')  
plt.show()
```



Box Plots – Distribution of Each Numerical Variable by `species`, Faceted by the Respective Numerical Variable

Gentoo penguins show higher values for flipper length and body mass, Chinstrap tend to have the longest and deepest bills, while Adelie generally have the smallest measurements across all variables.

```
sns.catplot(data=df_clean_lf,  
            x='species',  
            y='value',  
            col='variable',  
            col_wrap=4,  
            kind='box',  
            sharey=False)  
plt.show()
```

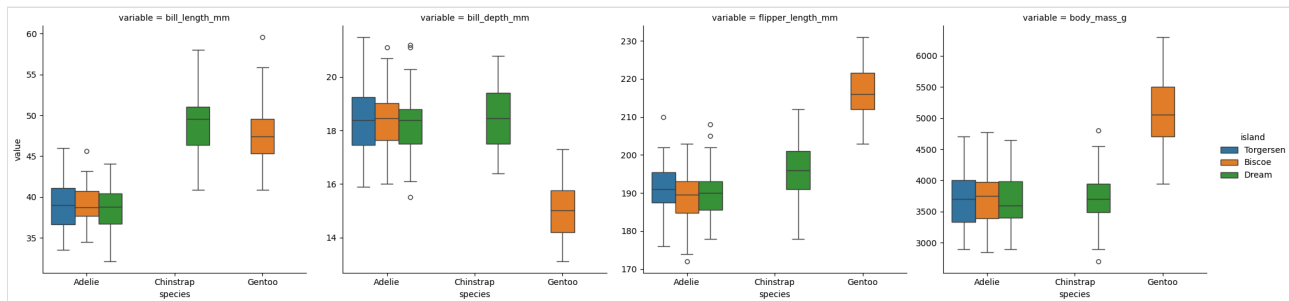


Box Plots – Distribution of Each Numerical Variable by species, Colored by island Categorical Variable, Faceted by the Respective Numerical Variable

Adelie penguins show similar measurements across all islands, Chinstrap are present only on Dream with relatively larger bills, while Gentoo is exclusive to Biscoe and stands out for its greater flipper length and body mass.

```
sns.catplot(data=df_clean_lf,
            x='species',
            y='value',
            col='variable',
            col_wrap=4,
            hue='island',
            kind='box',
            sharey=False)

plt.show()
```

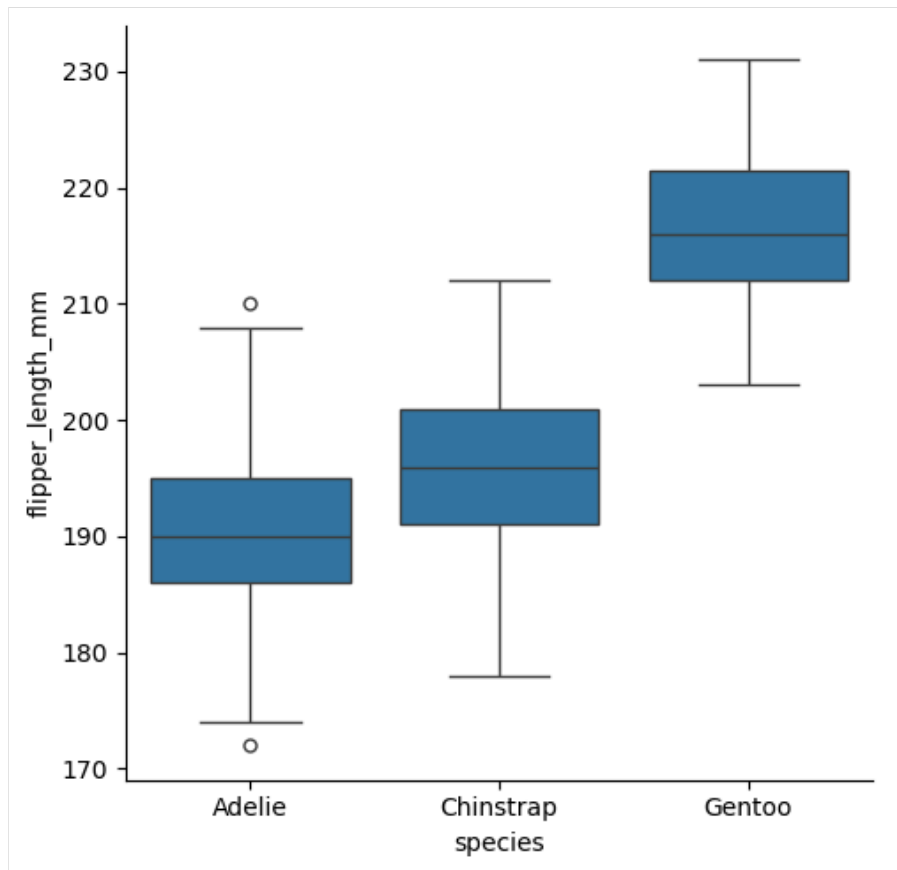


Box Plot – Marginal Distributions of the flipper_length_mm Continuous Variable, Grouped by Each species Categorical Variable

Gentoo penguins have the longest flipper lengths, followed by Chinstrap, and then Adelie. The boxplots show clear separation in flipper length among the three species, with Gentoo consistently measuring higher.

```
sns.catplot(data=df_clean,
            x='species',
            y='flipper_length_mm',
            kind='box')

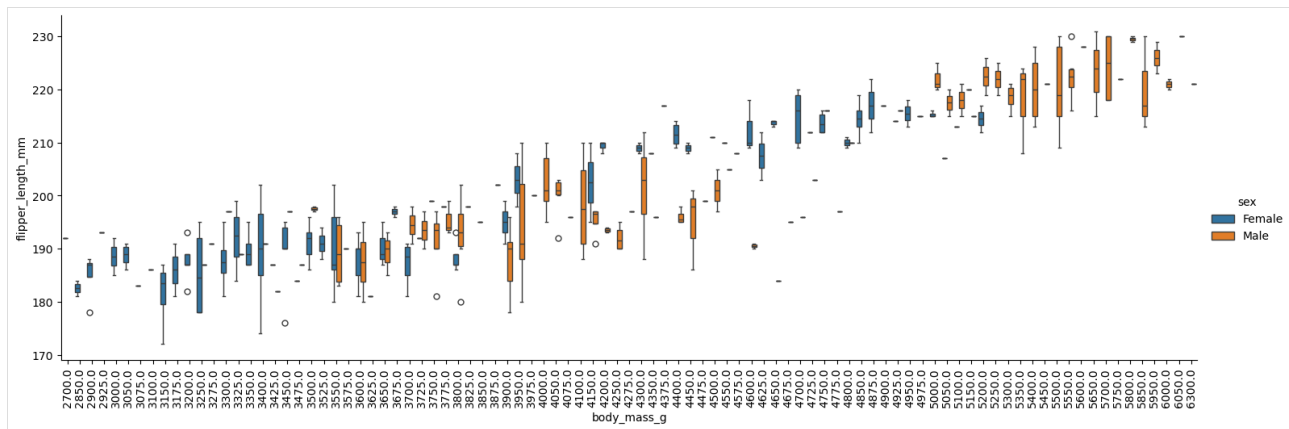
plt.show()
```



Box Plot – Distribution of the `flipper_length_mm` **Continuous Variable Across** `body_mass_g` **Continuous Variable, Colored by Each** `sex` **Categorical Variable**

The plot shows that as penguins' body mass increases, their flipper length also tends to increase. This trend is consistent for both male and female penguins, with heavier penguins generally having longer flippers.

```
sns.catplot(data=df_clean,
            x='body_mass_g',
            y='flipper_length_mm',
            hue='sex',
            kind='box',
            aspect=3)
plt.xticks(rotation=90)
plt.show()
```

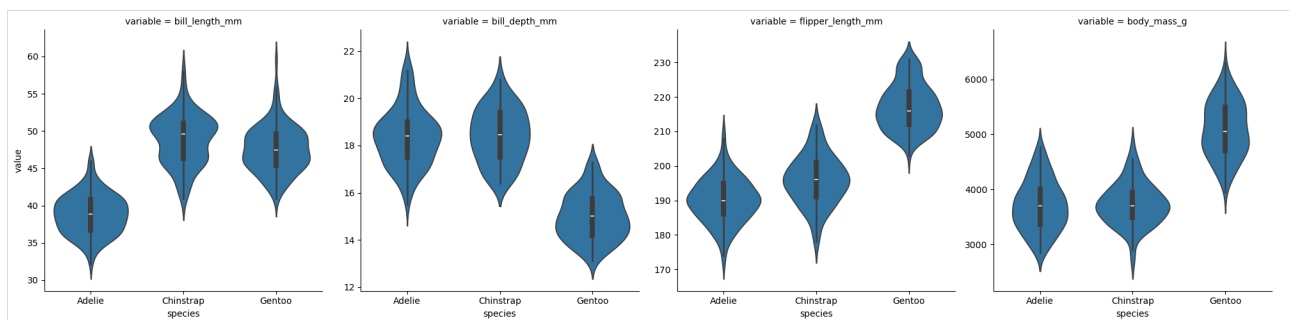


Violin Plots – Distribution Density of Each Numerical Variable by species, Faceted by the Respective Numerical Variable

Chinstrap and Gentoo penguins have longer bills compared to Adelie penguins, with Chinstrap showing the greatest bill depth. Gentoo penguins stand out for having the longest flipper lengths and the highest body mass, while Adelie penguins have the smallest measurements across all variables.

```
sns.catplot(data=df_clean_lf,
            x='species',
            y='value',
            col='variable',
            col_wrap=4,
            kind='violin',
            sharey=False)

plt.show()
```

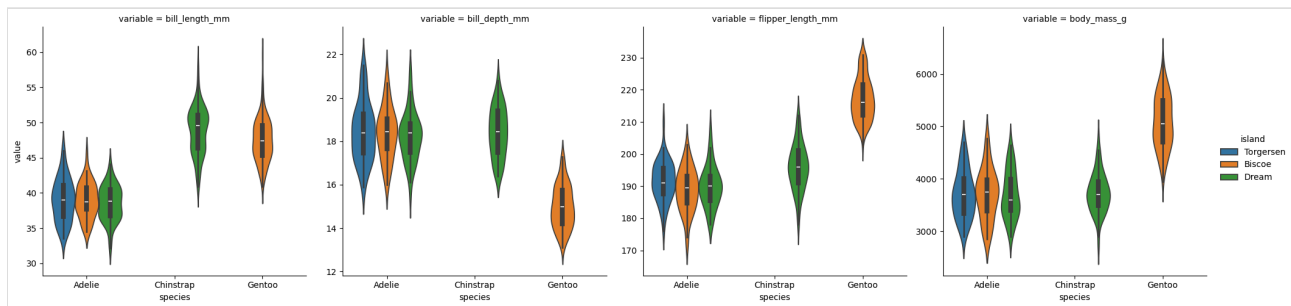


Violin Plots – Distribution Density of Each Numerical Variable by species, Colored by island, Faceted by the Respective Numerical Variable

Adelie penguins have similar measurements across all islands. Chinstrap penguins, only found on Dream, have greater bill length and depth compared to Adelie. Gentoo penguins, exclusive to Biscoe, stand out for their larger flipper length and body mass. This visualization highlights clear differences in physical traits among species and islands.

```
sns.catplot(data=df_clean_lf,
            x='species',
            y='value',
            col='variable',
            col_wrap=4,
            hue='island',
```

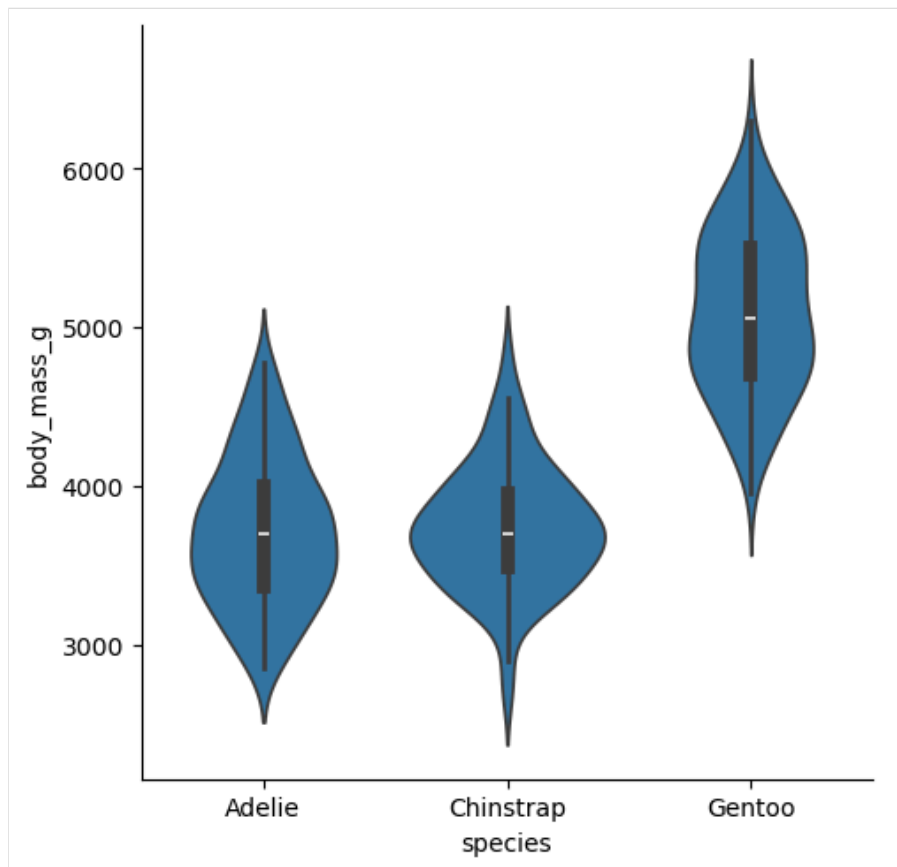
```
kind='violin',
sharey=False)
plt.show()
```



Violin Plot – Marginal Distribution Density of the `body_mass_g` Continuous Variable, Grouped by Each species Categorical Variable

Gentoo penguins have the highest body mass, followed by Chinstrap, while Adelie penguins have the lowest body mass. Gentoo penguins also display greater variation in body mass compared to the other species.

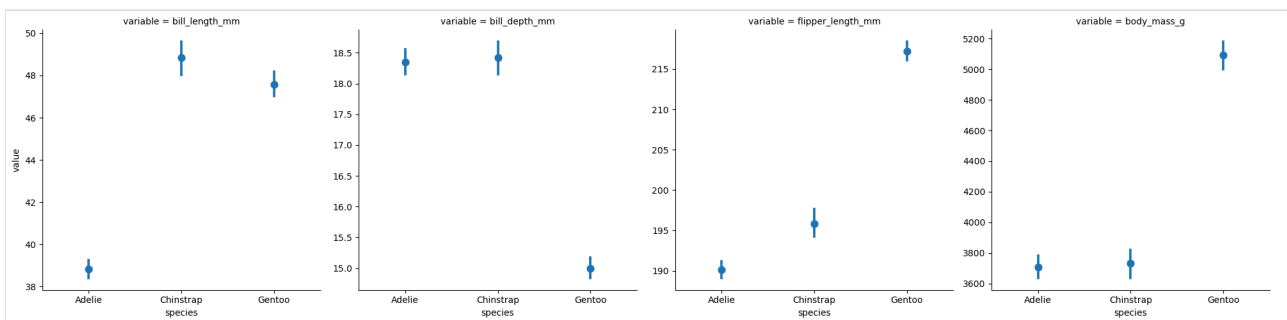
```
sns.catplot(data=df_clean,
            x='species',
            y='body_mass_g',
            kind='violin')
plt.show()
```



Point Plots – Mean and Confidence Interval of Each Numerical Variable by species, Faceted by the Respective Numerical Variable

Chinstrap penguins have the longest bills and greatest bill depth, Gentoo penguins have the longest flipper length and highest body mass, while Adelie penguins have the smallest measurements for all variables. Error bars indicate the variability around the mean values for each species.

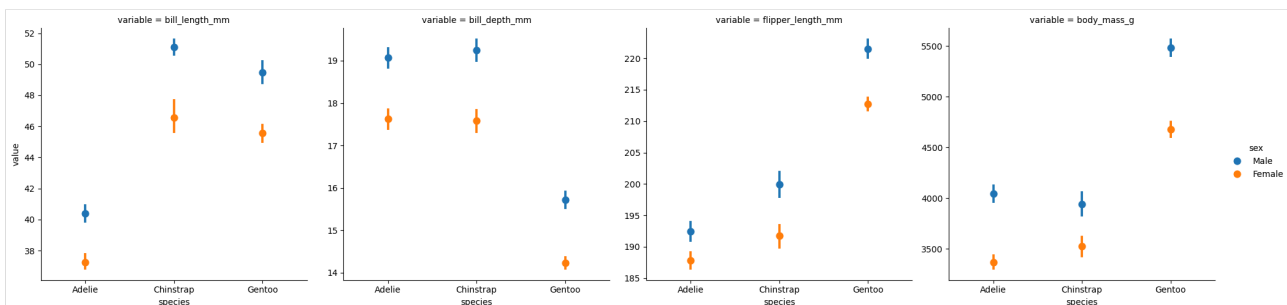
```
sns.catplot(data=df_clean_lf,
            x='species',
            y='value',
            col='variable',
            col_wrap=4,
            kind='point',
            sharey=False,
            linestyle='none')
plt.show()
```



Point Plots – Mean and Confidence Interval of Each Numerical Variable by species, Colored by sex Categorical Variable, Faceted by the Respective Numerical Variable

For all species, male penguins have higher measurements than female penguins across bill length, bill depth, flipper length, and body mass. Chinstrap males have the largest bills, Gentoo males have the longest flippers and greatest body mass, while Adelie females have the lowest values for all traits. The differences between sexes are consistent and pronounced for each variable.

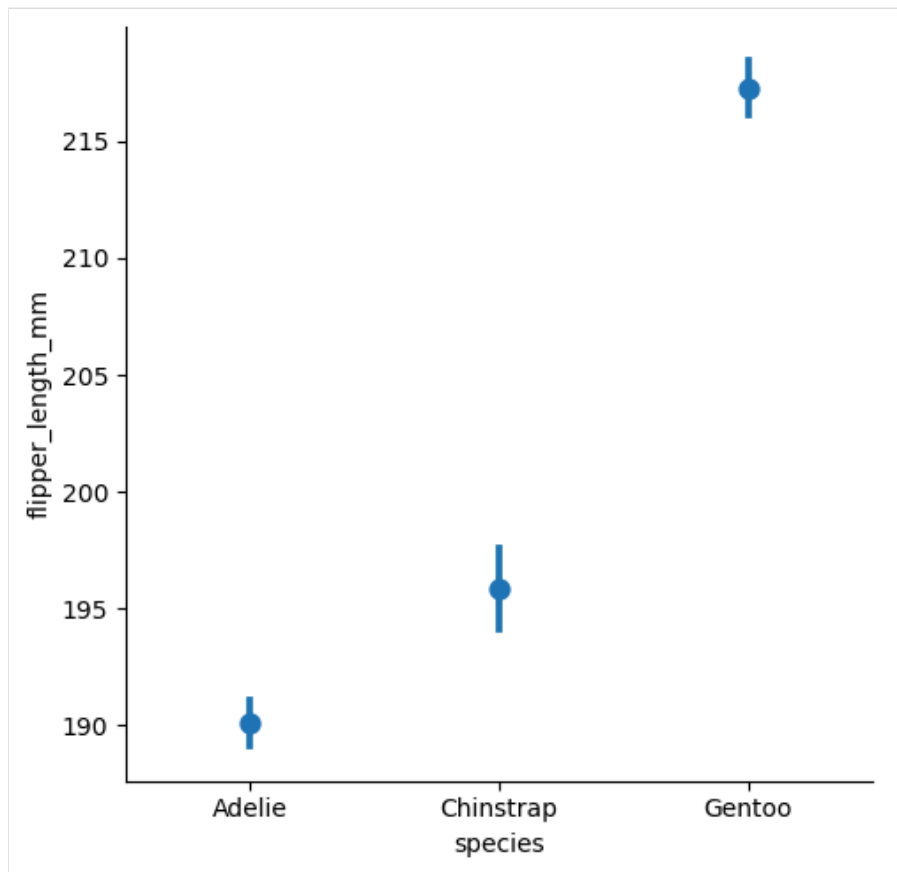
```
sns.catplot(data=df_clean_lf,
            x='species',
            y='value',
            col='variable',
            col_wrap=4,
            hue='sex',
            kind='point',
            sharey=False,
            linestyle='none')
plt.show()
```



Point Plot – Mean and Confidence Interval of the `flipper_length_mm` Continuous Variable, Grouped by Each `species` Categorical Variable

Gentoo penguins have the longest average flipper length, followed by Chinstrap penguins, with Adelie penguins having the shortest flippers. The error bars indicate the variability around the mean for each species.

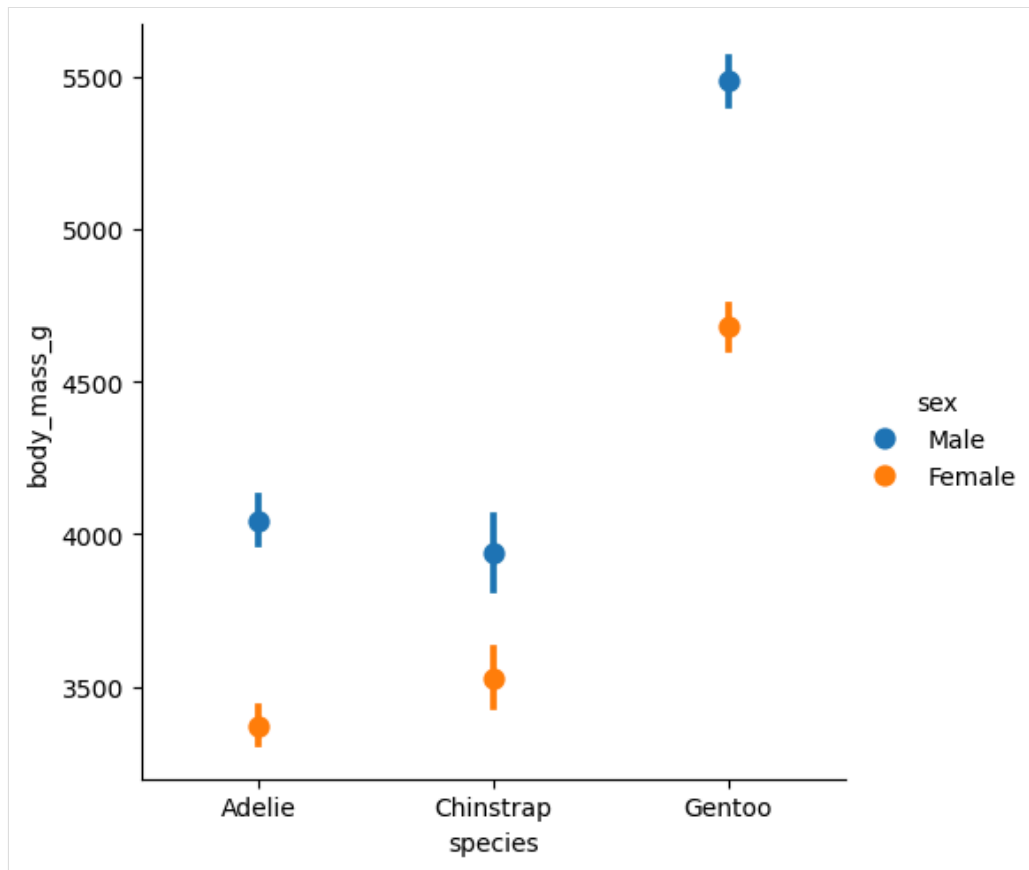
```
sns.catplot(data=df_clean,
            x='species',
            y='flipper_length_mm',
            kind='point',
            linestyle='none')
plt.show()
```



Point Plot – Mean and Confidence Interval of the `body_mass_g` Continuous Variable, Grouped by Each Combination of `species` Categorical Variable, Colored by `sex` Categorical Variable

For each species, male penguins have a higher average body mass than female penguins. Gentoo penguins are the heaviest, followed by Chinstrap and then Adelie. The differences between males and females are consistent across species, with males weighing more.

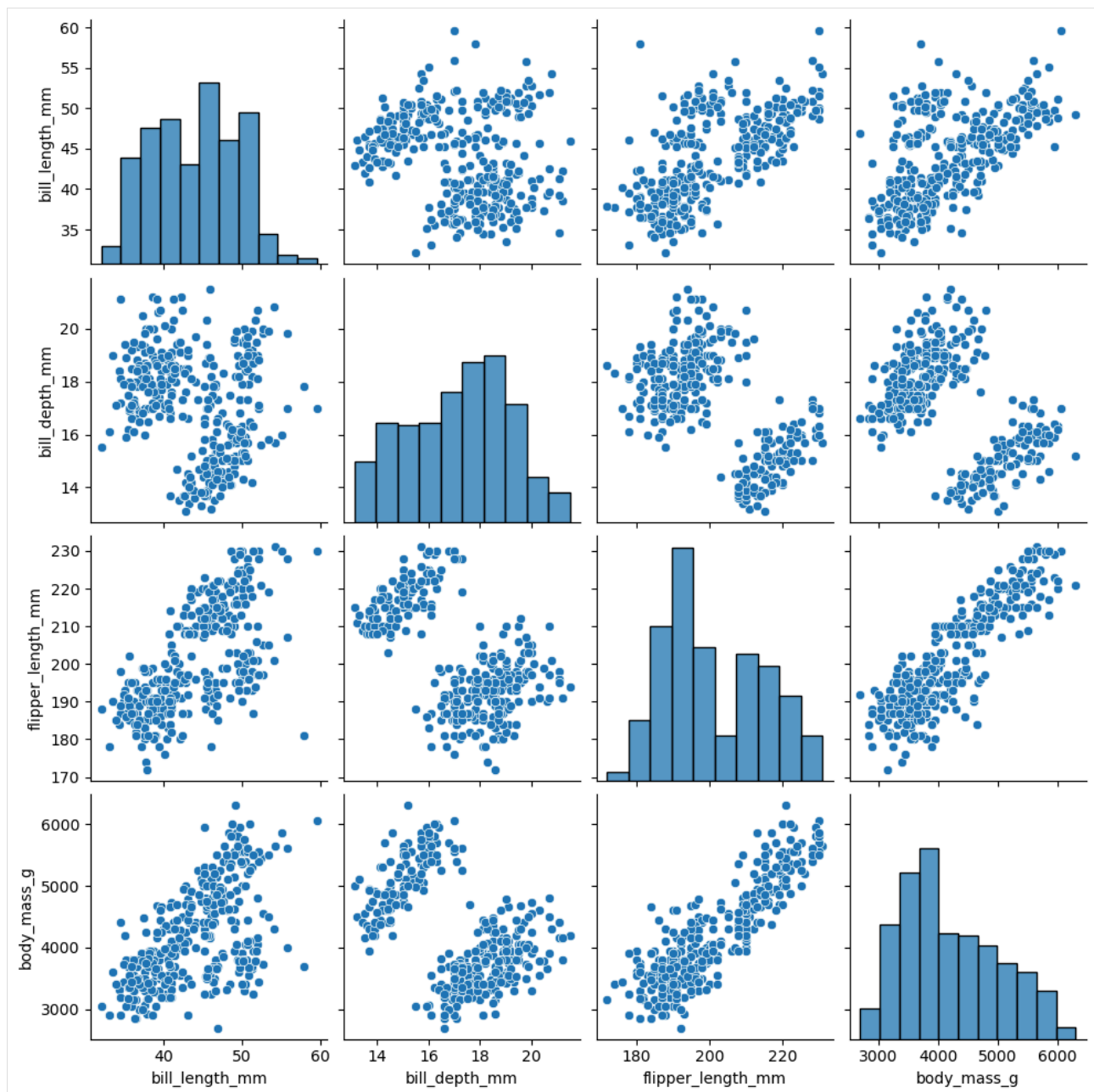
```
sns.catplot(data=df_clean,
            x='species',
            y='body_mass_g',
            hue='sex',
            kind='point',
            linestyle='none')
plt.show()
```



Pair Plots – Pairwise Relationships of All Numerical Variables

There are clear positive relationships among bill length, flipper length, and body mass in penguins—penguins with longer bills tend to have longer flippers and greater body mass. Bill depth displays less pronounced correlation with the other variables. Each variable also shows its own distribution along the diagonal, with flipper length and body mass having wider ranges compared to bill measurements.

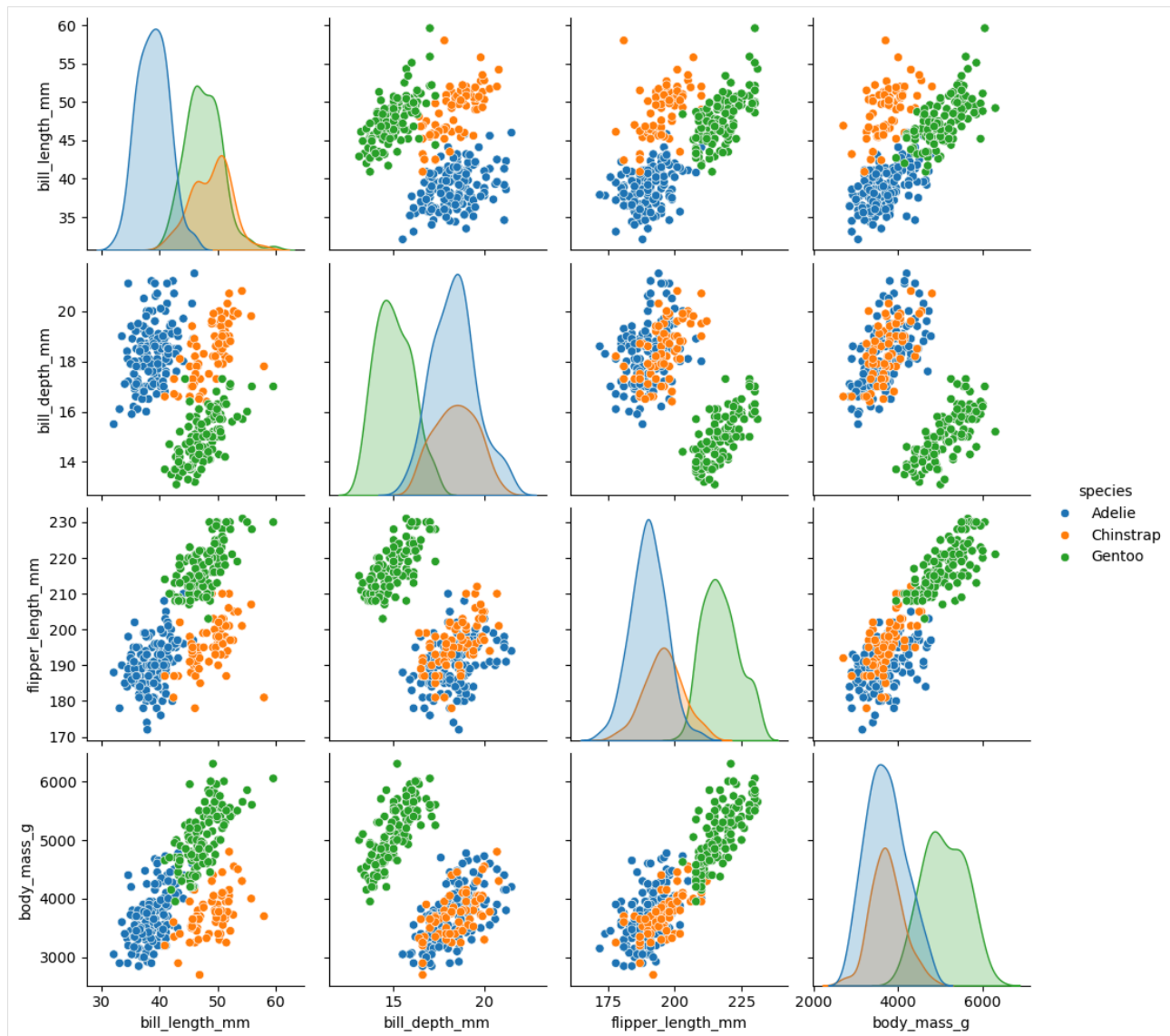
```
sns.pairplot(df_clean)
plt.show()
```



Pair Plots – Pairwise Relationships of All Numerical Variables, Colored by species **Categorical Variable**

There are clear distinctions among Adelie, Chinstrap, and Gentoo penguins for all measured variables. Gentoo penguins generally have the largest bill length, flipper length, and body mass, while Adelie penguins have the smallest. Chinstrap penguins typically have the greatest bill depth. The scatter plots reveal strong positive relationships between flipper length and body mass, as well as between bill length and flipper length. Each species forms distinct clusters, highlighting the differences in physical traits.

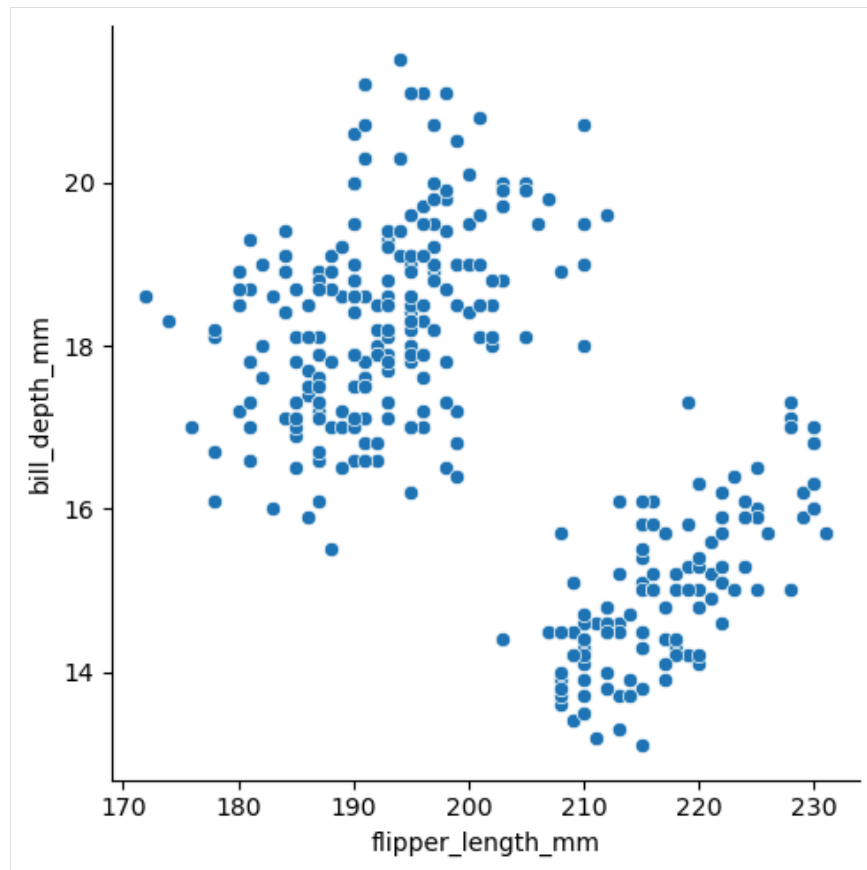
```
sns.pairplot(df_clean,
             hue='species')
plt.show()
```



Scatter Plot – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables

Penguins with longer flippers tend to have lower bill depths, while those with shorter flippers generally have higher bill depths. This suggests a negative relationship between flipper length and bill depth among penguins, with two distinct clusters visible in the data.

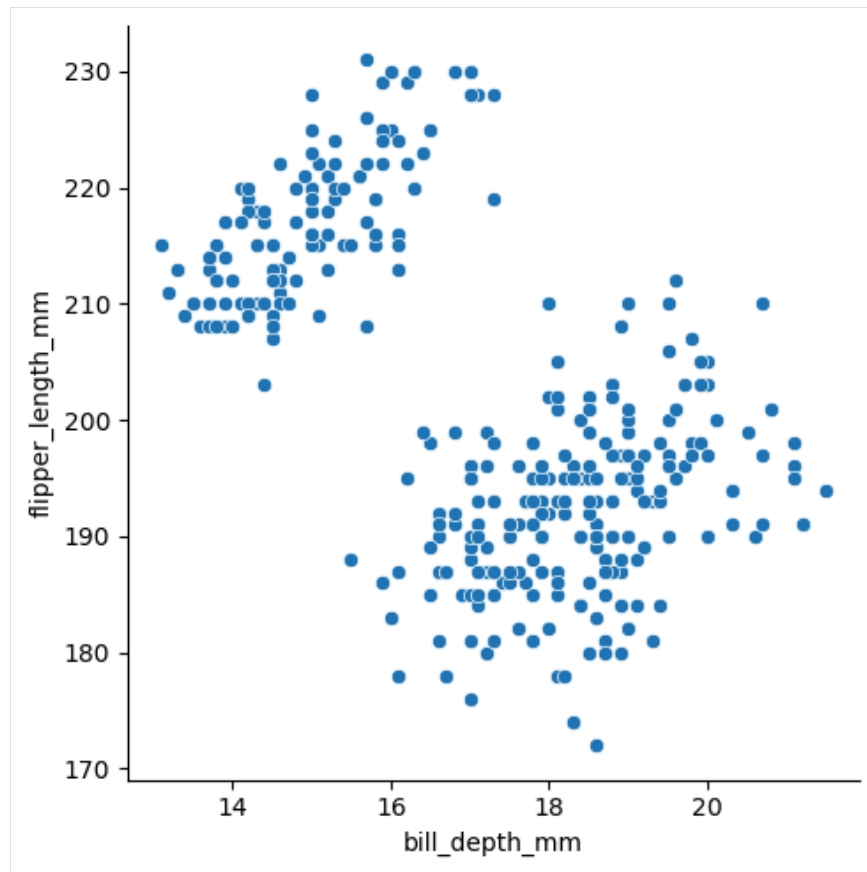
```
sns.relplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm')
plt.show()
```



Scatter Plot – Joint Distribution of the `bill_depth_mm` and `flipper_length_mm` Continuous Variables

There is a distinct pattern: penguins with lower bill depth tend to have longer flipper lengths, while those with higher bill depth generally have shorter flipper lengths. There are two clear clusters, indicating a negative relationship between bill depth and flipper length among the penguin data.

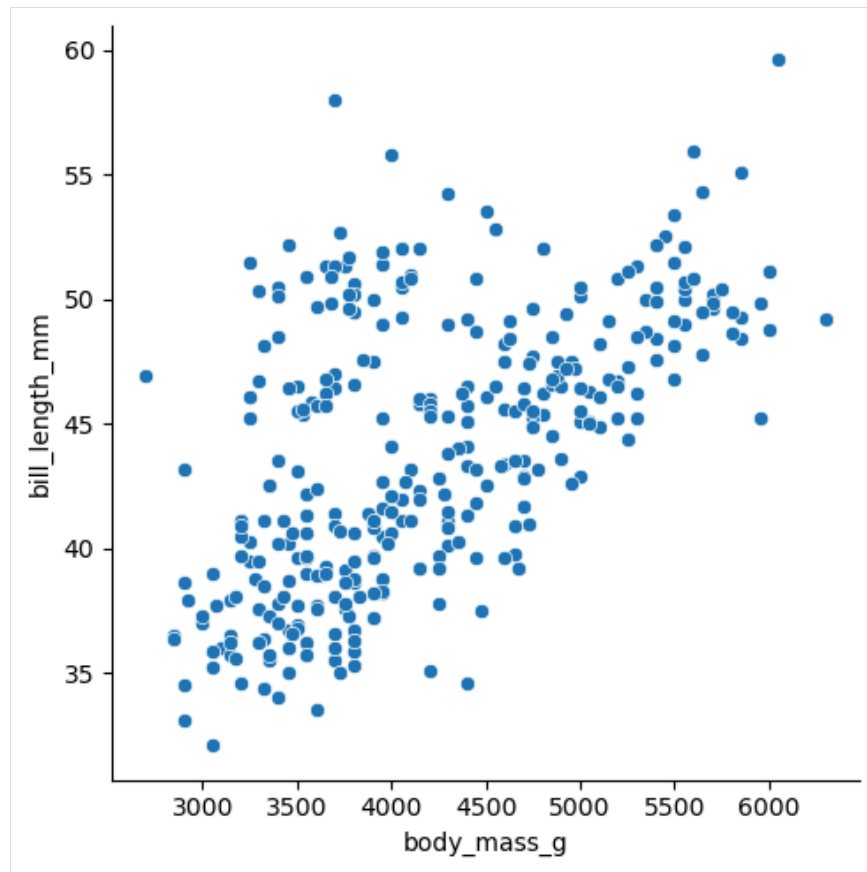
```
sns.relplot(data=df_clean,
            x='bill_depth_mm',
            y='flipper_length_mm')
plt.show()
```



Scatter Plot – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables

There is a clear positive relationship between body mass and bill length in penguins. As body mass increases, bill length also tends to increase. The data points are fairly spread out but show a general upward trend, indicating that larger penguins tend to have longer bills.

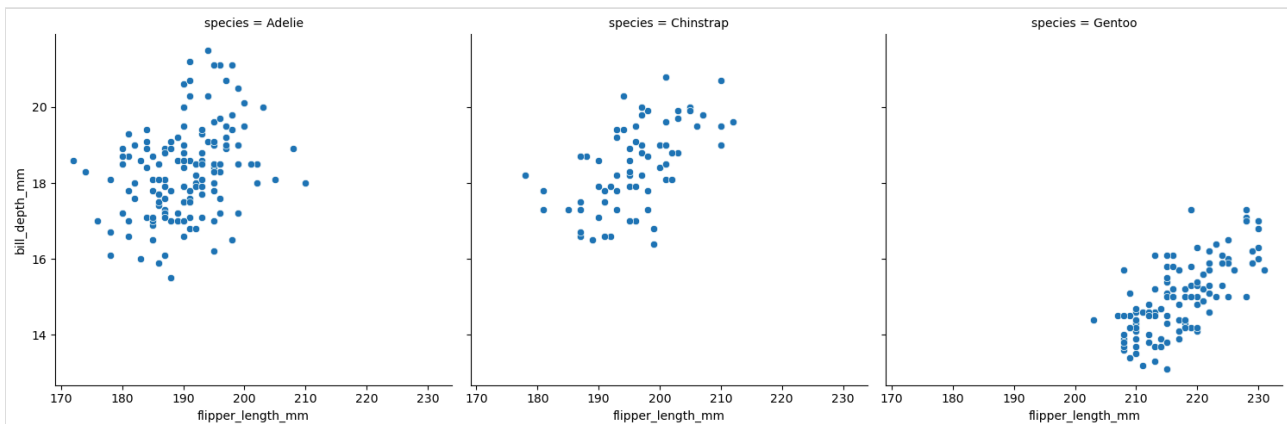
```
sns.relplot(data=df_clean,
            x='body_mass_g',
            y='bill_length_mm')
plt.show()
```



Scatter Plot Facets – Joint Distribution of the `flipper_length_mm` and `bill_depth_mm` Continuous Variables, Faceted by Each `species` Categorical Variable

Gentoo penguins have longer flipper lengths and lower bill depths compared to the other species.

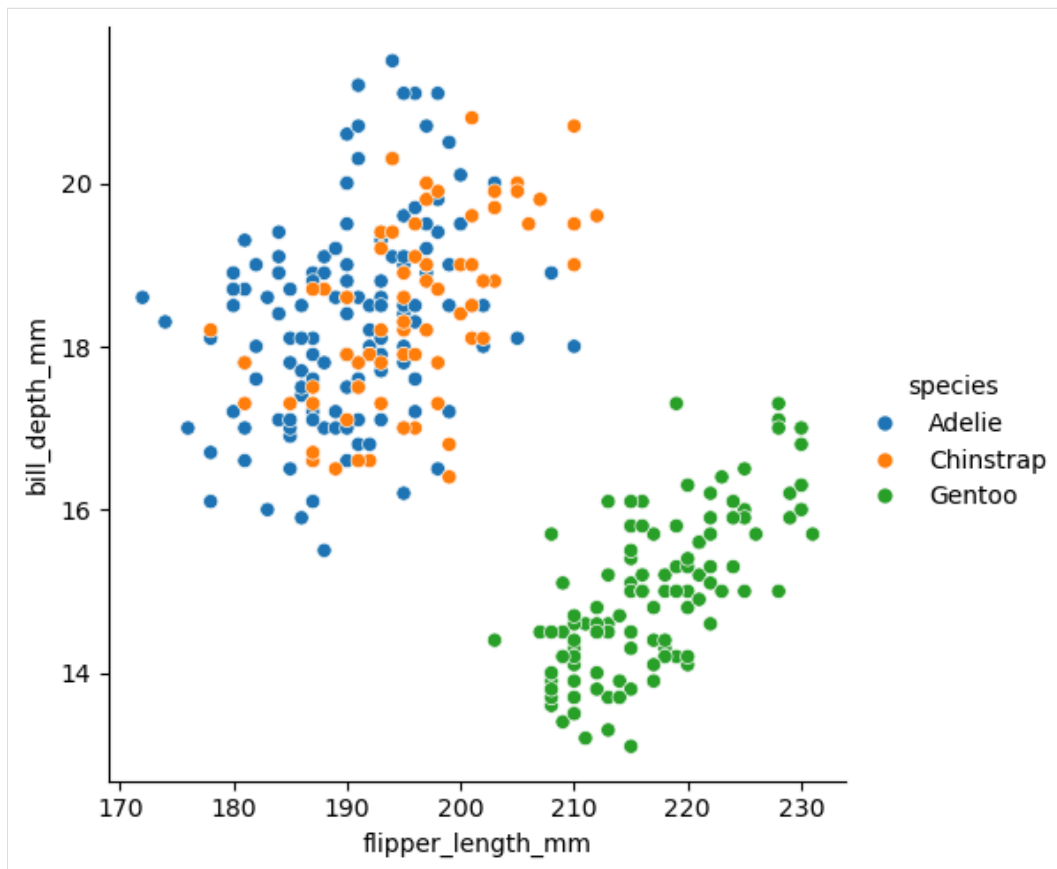
```
sns.relplot(data=df_clean,  
            x='flipper_length_mm',  
            y='bill_depth_mm',  
            col='species')  
plt.show()
```



Scatter Plot – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables, Colored by species Categorical Variable

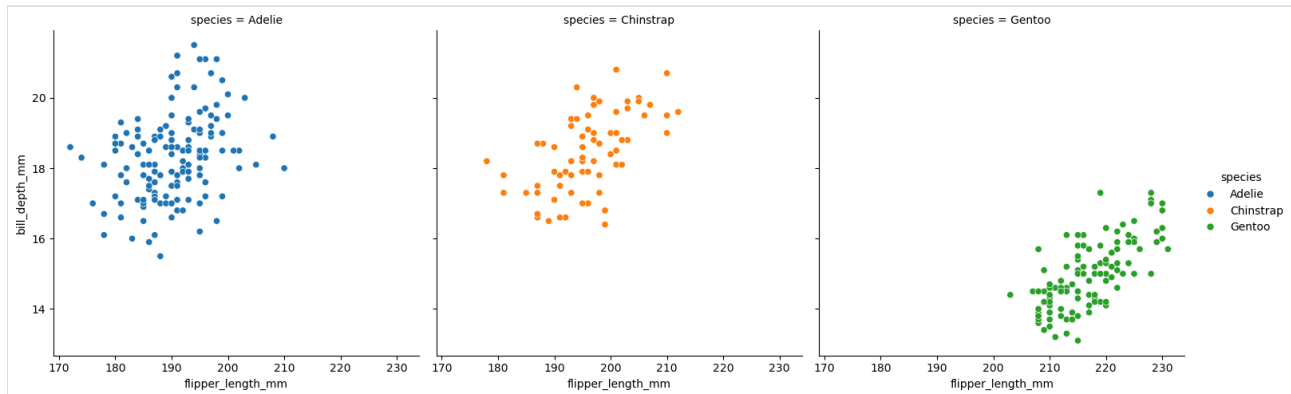
Adelie and Chinstrap penguins cluster together with higher bill depths and shorter flipper lengths, while Gentoo penguins are clearly separated—they have longer flipper lengths and lower bill depths. The color coding makes it easy to distinguish the species' differences and reveals that bill depth and flipper length are useful for differentiating Gentoo penguins from the other two species.

```
sns.relplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm',
            hue='species')
plt.show()
```

**Scatter Plot Facets – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables, Faceted by Each species Categorical Variable, Colored by species Categorical Variable**

Adelie penguins are shown in blue with flipper lengths mostly between 180 and 200 mm, and bill depths ranging from 16 to 21 mm. Chinstrap penguins, represented in orange, overlap with Adelie in flipper length but generally have slightly higher bill depths. Gentoo penguins, shown in green, form a distinct cluster with much longer flipper lengths (over 210 mm) and lower bill depths, clearly separated from the other two species. The color legend on the right facilitates species identification, and overall, the plot highlights the morphological differences among the three penguin species.

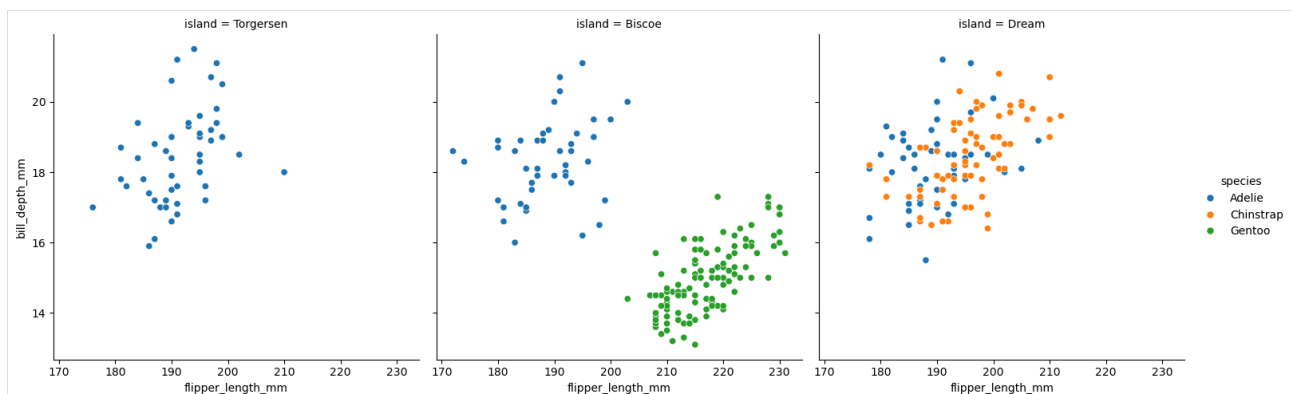
```
sns.relplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm',
            hue='species',
            col='species')
plt.show()
```



Scatter Plot Facets – Joint Distribution of the `flipper_length_mm` and `bill_depth_mm` Continuous Variables, Faceted by Each `island` Categorical Variable, Colored by `species` Categorical Variable

On Torgersen island (left), only Adelie penguins are present, and they cluster around shorter flipper lengths and higher bill depths. Biscoe island (middle) hosts both Adelie and Gentoo penguins; Adelie penguins overlap with those from Torgersen, while Gentoo penguins form a distinct group with longer flippers and lower bill depths. Dream island (right) features Adelie and Chinstrap penguins, both with higher bill depths and shorter flipper lengths, but Chinstrap penguins tend to have slightly greater bill depths than Adelie. The color legend clarifies species distribution, and the plots reveal how island location is associated with both species composition and their morphological characteristics.

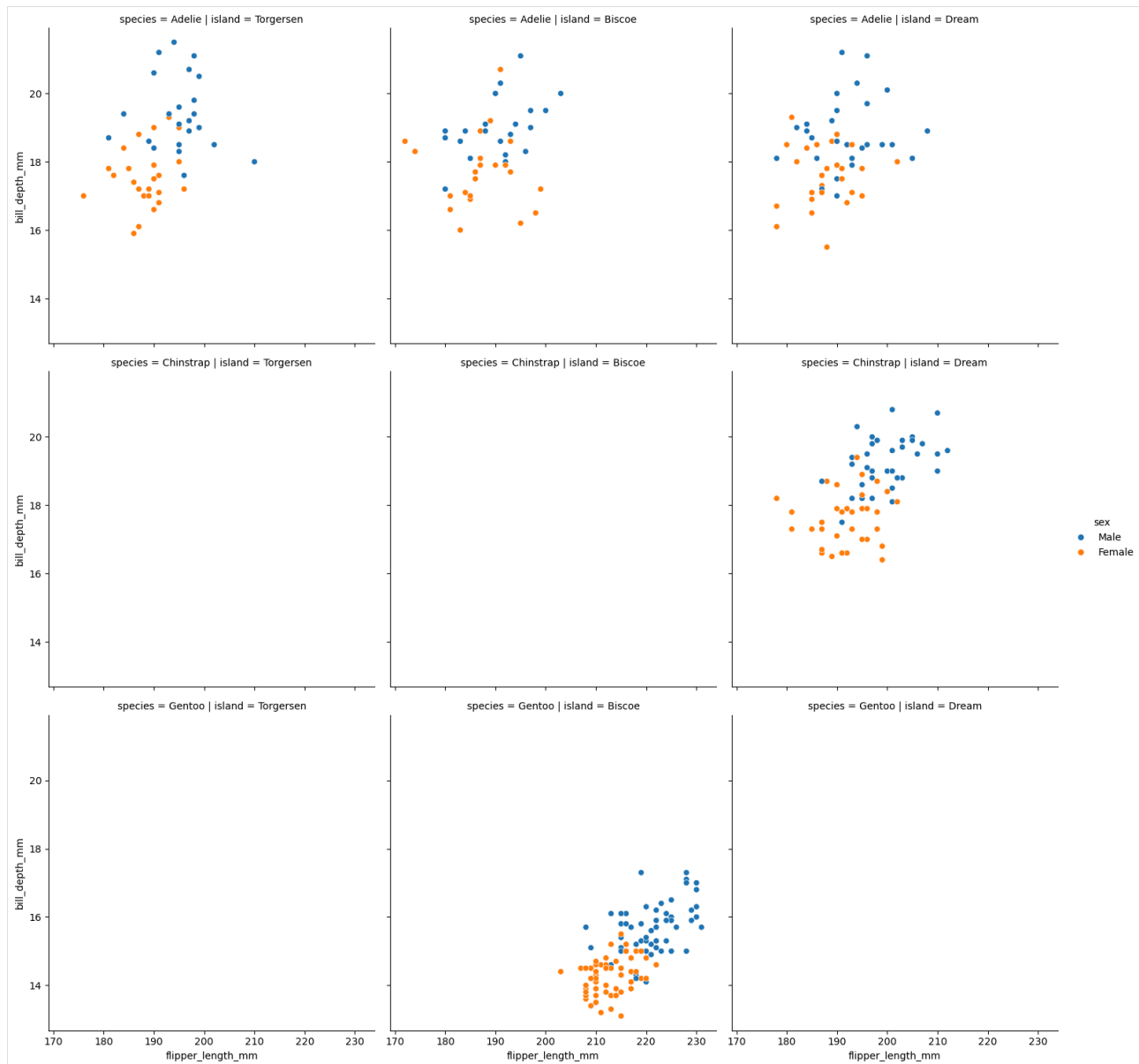
```
sns.relplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm',
            hue='species',
            col='island')
plt.show()
```



Scatter Plot Facets – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables, Faceted by Each Combination of species and island Categorical Variables, Colored by sex Categorical Variable

The Adelie penguin panels show data across all three islands, with some variation in the spread and clustering of male and female measurements. Chinstrap penguins are only present on Dream island, where males tend to have slightly greater flipper lengths and bill depths than females. Gentoo penguins appear only on Biscoe island, with males generally showing longer flipper lengths and higher bill depths compared to females. This visualization makes it easy to compare how sex, species, and island location interact to influence penguin morphology.

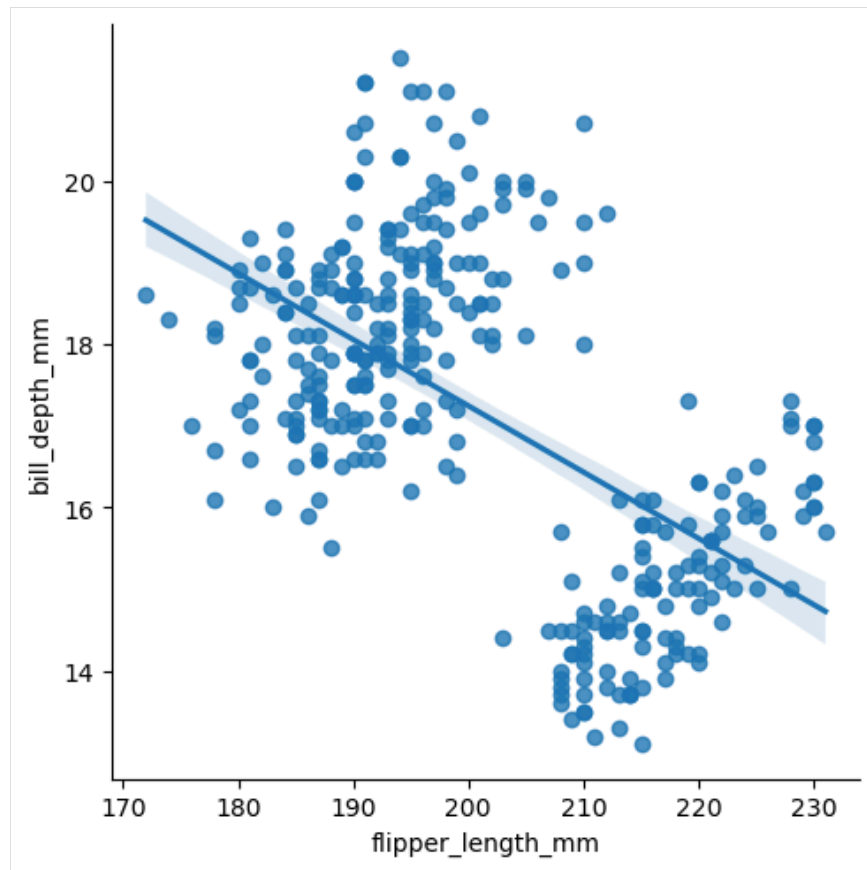
```
sns.relplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm',
            hue='sex',
            col='island',
            row='species')
plt.show()
```



Trend Plot – Joint Distribution of the `flipper_length_mm` and `bill_depth_mm` Continuous Variables, with Fitted Linear Regression and Confidence Interval

The negative slope of the regression line indicates an inverse relationship: as flipper length increases, bill depth tends to decrease. The spread of points suggests that there are distinct clusters, likely representing different species, but overall, longer flippers are associated with shallower bills. The shaded area around the line provides a visual indication of the uncertainty or variability in this trend.

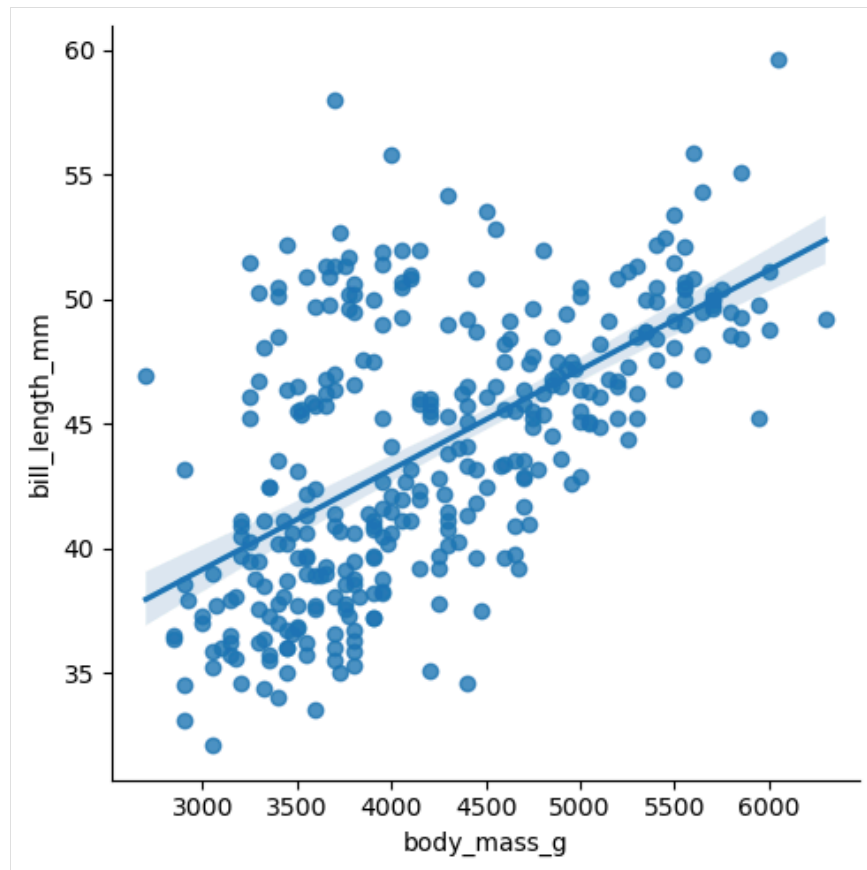
```
sns.lmplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm')
plt.show()
```



Trend Plot – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables, with Fitted Linear Regression and Confidence Interval

The positive slope of the regression line indicates a direct correlation: as body mass increases, bill length also tends to increase. The data points are spread across a range of values, but the overall trend suggests that larger penguins generally have longer bills. The shaded area around the regression line represents the confidence interval, illustrating the variability in this relationship.

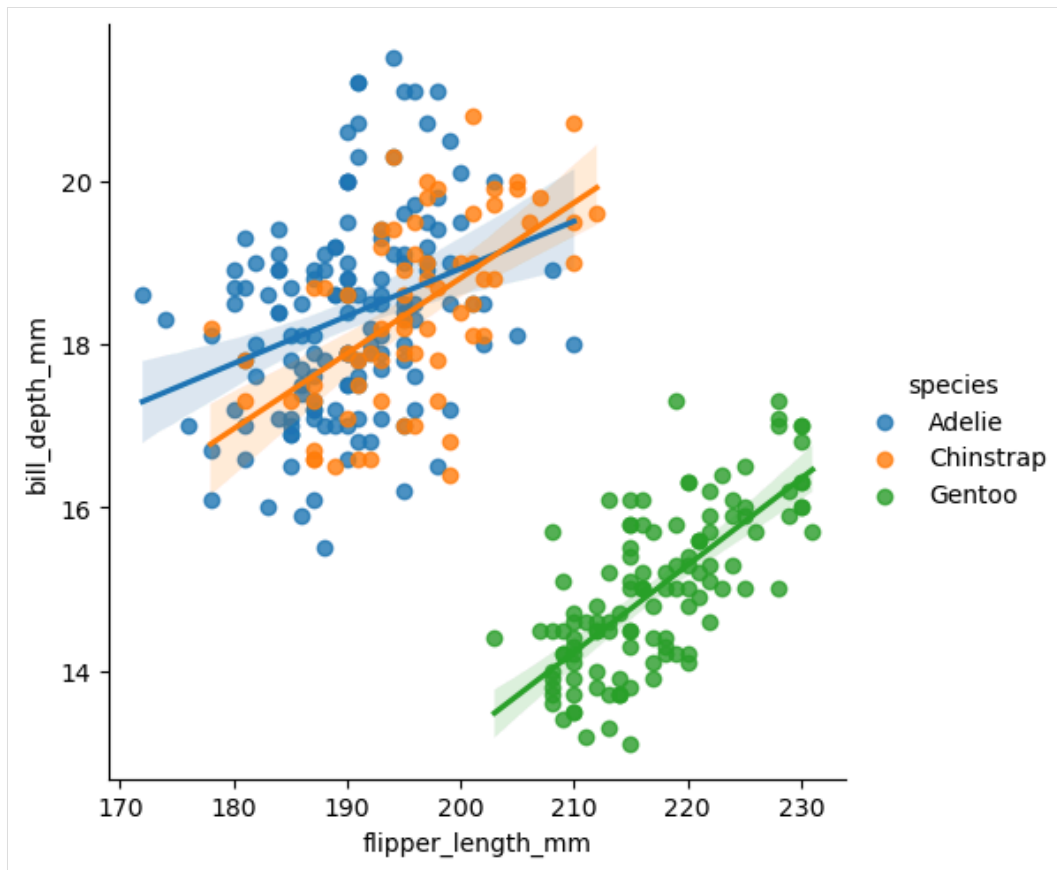
```
sns.lmplot(data=df_clean,
           x='body_mass_g',
           y='bill_length_mm')
plt.show()
```



Trend Plot – Joint Distribution of the `flipper_length_mm` and `bill_depth_mm` Continuous Variables, Colored by `species` Categorical Variable, with Fitted Linear Regression and Confidence Interval

Adelie and Chinstrap regression lines indicate a positive correlation between the two variables for both species. Gentoo penguins form a distinct cluster with longer flipper lengths and lower bill depths, and their regression line also shows a positive association. The shaded regions around each regression line reflect the confidence intervals, highlighting the variability of the data within each species. Overall, the plot emphasizes both the differences in morphology between species and the positive relationship between flipper length and bill depth within each group.

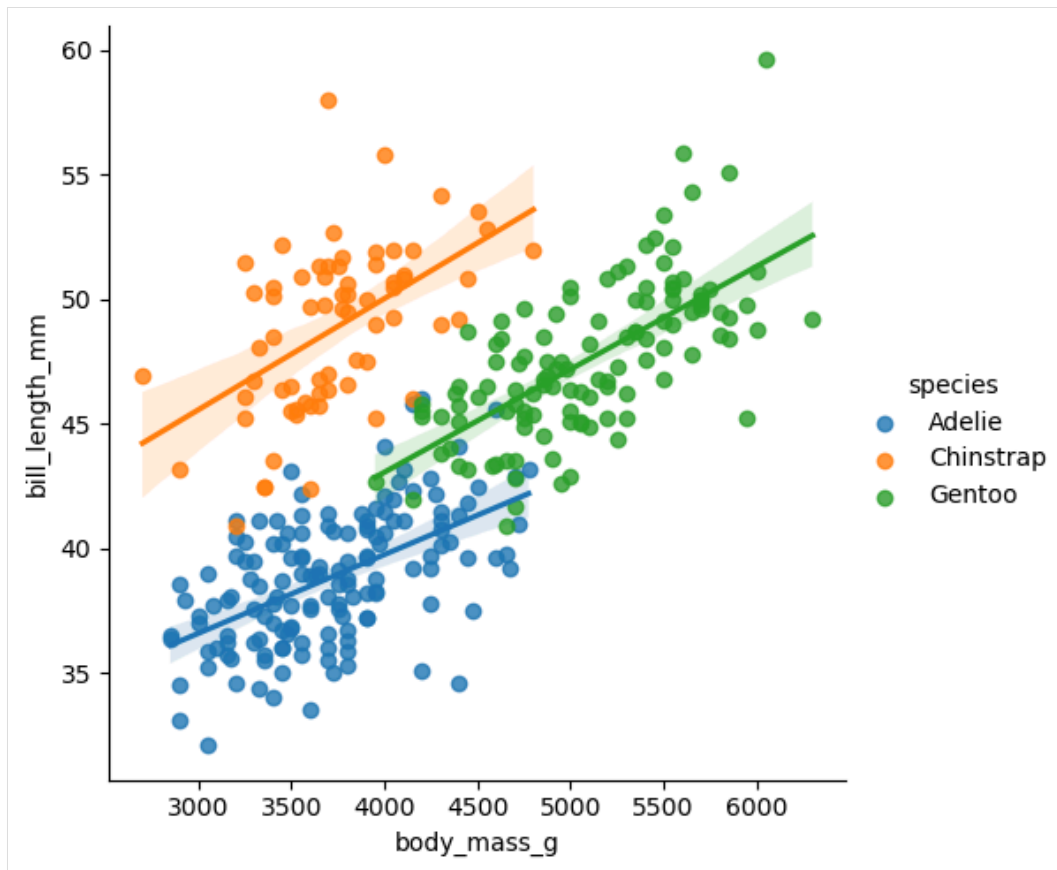
```
sns.lmplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm',
            hue='species')
plt.show()
```



Trend Plot – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables, Colored by `species` Categorical Variable, with Fitted Linear Regression and Confidence Interval

All three species show a positive correlation between body mass and bill length, but their regression lines are vertically separated, indicating clear differences in average bill length among species for a given body mass. Chinstrap penguins (orange) generally have the longest bills for a given body mass, followed by Gentoo (green) and then Adelie (blue). The shaded regions around each regression line represent the variability within each species, and the plot overall highlights both the species-specific trends and the general pattern that heavier penguins tend to have longer bills.

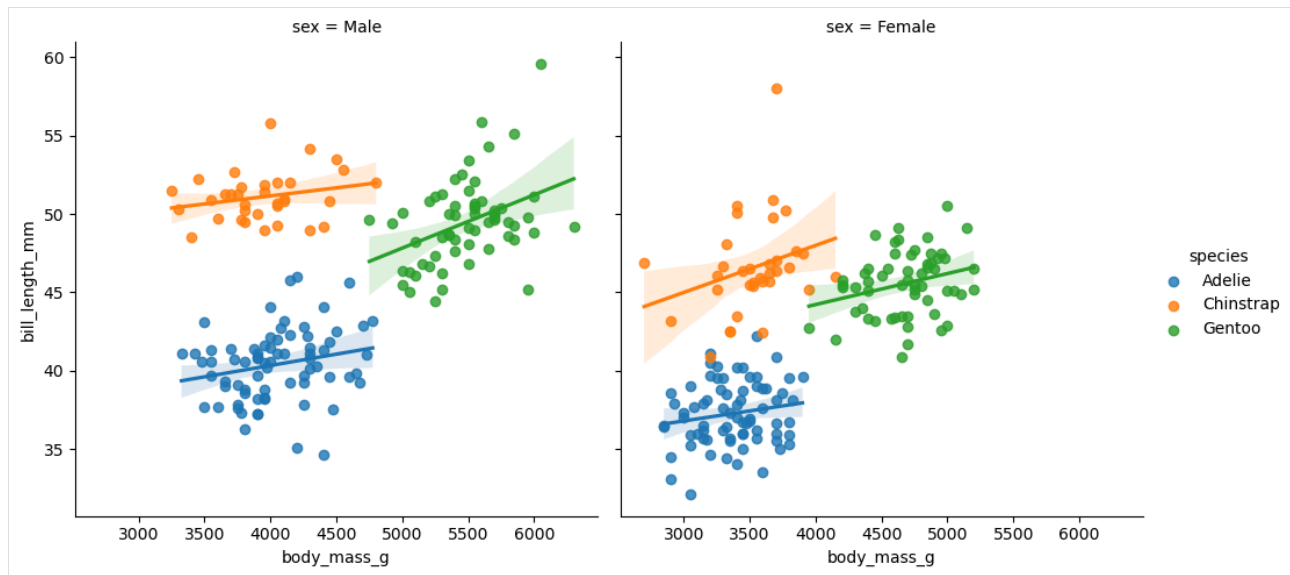
```
sns.lmplot(data=df_clean,
            x='body_mass_g',
            y='bill_length_mm',
            hue='species')
plt.show()
```



Trend Plots – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables, Faceted by `sex` Categorical Variable, Colored by `species` Categorical Variable, with Fitted Linear Regression and Confidence Interval

Males generally have longer bills than females for a given body mass, and that Chinstrap penguins have the longest bills, followed by Gentoo and Adelie.

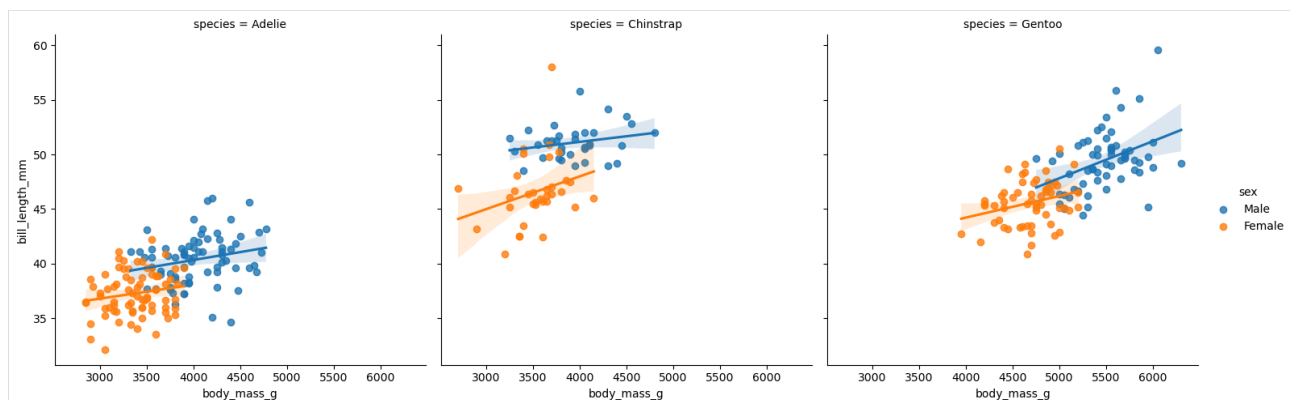
```
sns.lmplot(data=df_clean,
            x='body_mass_g',
            y='bill_length_mm',
            hue='species',
            col='sex')
plt.show()
```

Trend Plots – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables, Faceted by `species` Categorical Variable, Colored by `sex` Categorical Variable, with Fitted Linear Regression and Confidence Interval

Males generally have longer bills than females for a given body mass across all species.

```
sns.lmplot(data=df_clean,
            x='body_mass_g',
            y='bill_length_mm',
            hue='sex',
            col='species')
plt.show()
```



Trend Plots – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables, Faceted by Each Combination of `island` and `species` Categorical Variables, Colored by `sex` Categorical Variable, with Fitted Linear Regression and Confidence Interval

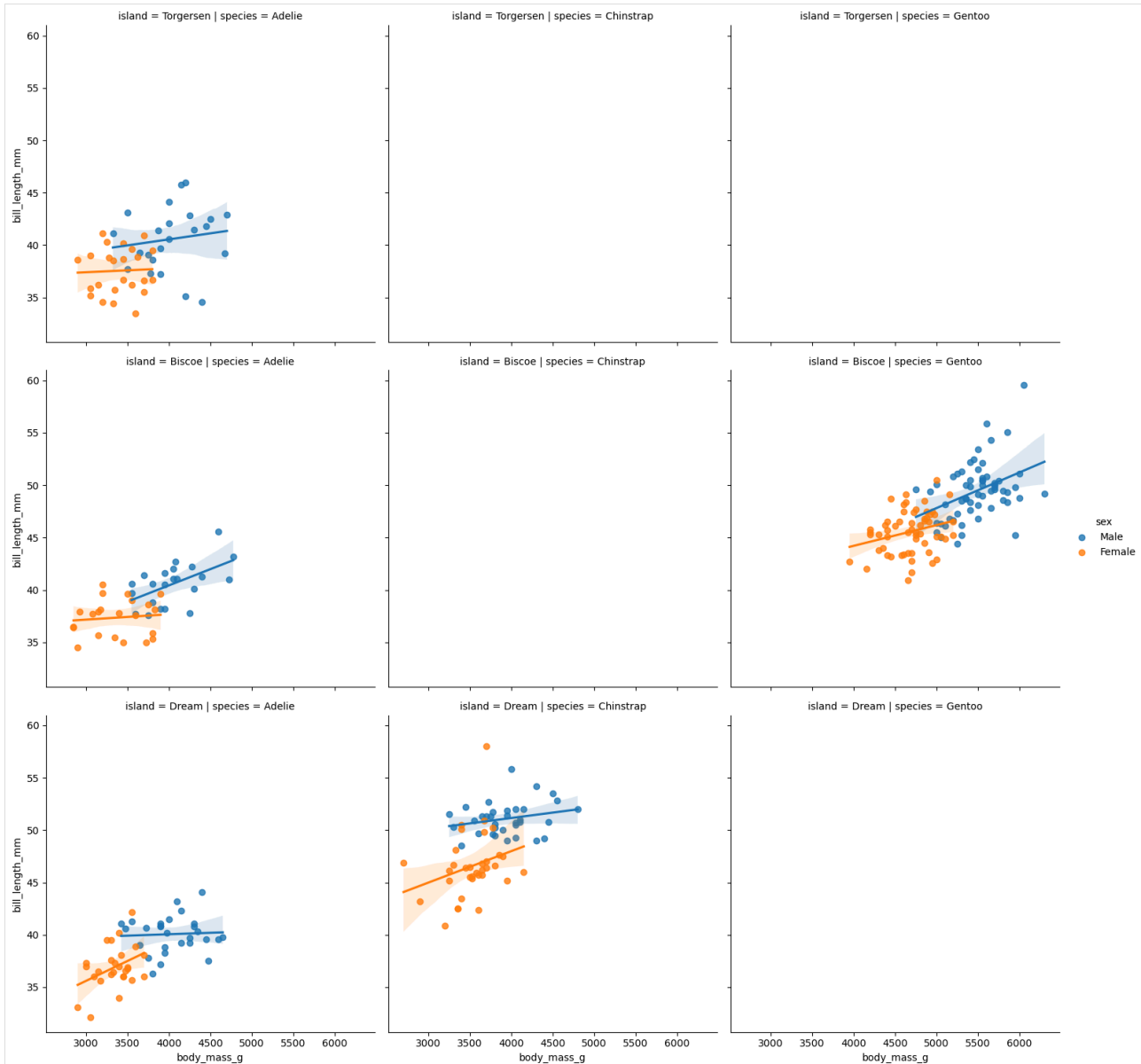
Adelie penguins appear on all islands, Chinstrap penguins only on Dream, and Gentoo penguins only on Biscoe. In each group, males generally have longer bills than females for a given body mass, and the regression lines with confidence intervals highlight the differences and variability within each island-species-sex combination.

```
sns.lmplot(data=df_clean,
            x='body_mass_g',
```

```

y='bill_length_mm',
hue='sex',
col='species',
row='island')
plt.show()

```



Trend Plots – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables, Faceted by Each Combination of island and species Categorical Variables, Colored by sex Categorical Variable, with Fitted Linear Regression and Confidence Interval

Adelie penguins appear on all islands, Chinstrap on Dream, and Gentoo on Biscoe and Dream, showing that males generally have deeper bills than females for a given flipper length, and highlighting differences in morphology across islands and species.

```

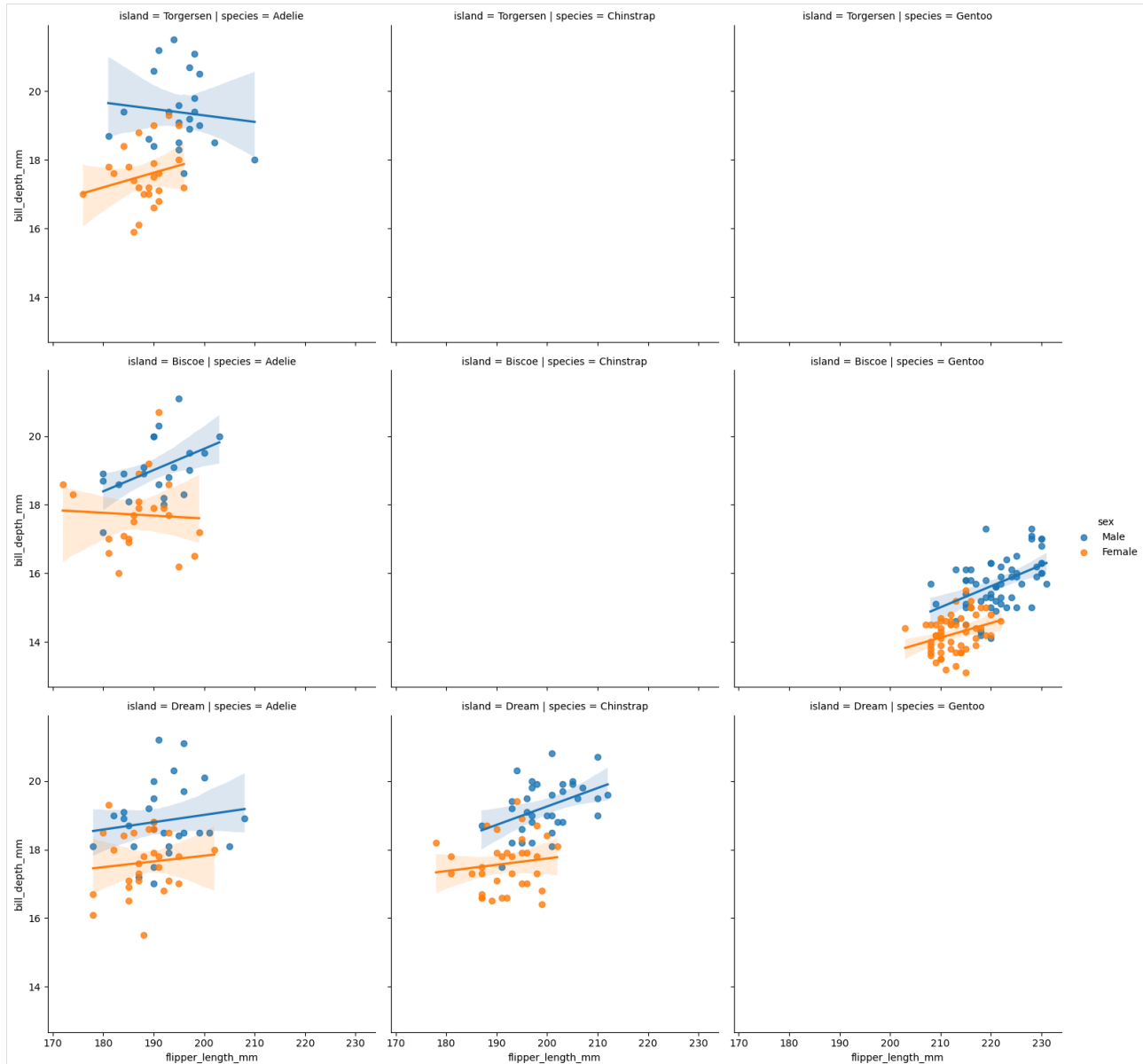
sns.lmplot(data=df_clean,
           x='flipper_length_mm',

```

```

y='bill_depth_mm',
hue='sex',
col='species',
row='island')
plt.show()

```



Correlation Plots - Correlation Matrix of All Numerical Variables

Flipper length and body mass are strongly positively correlated (0.87), bill length is moderately correlated with both flipper length (0.65) and body mass (0.59), and bill depth is negatively correlated with both flipper length (-0.58) and body mass (-0.47).

```

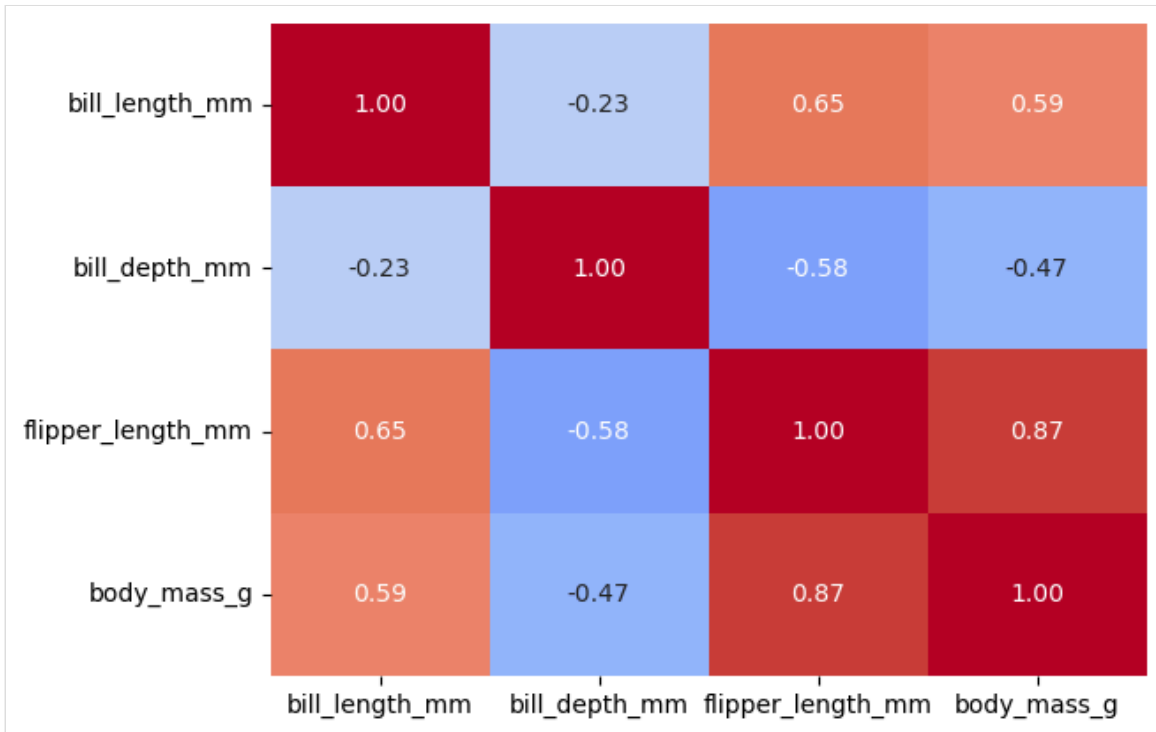
fig, ax = plt.subplots()
sns.heatmap(data=df_clean.corr(numeric_only=True),
            vmin=-1,
            vmax=1,

```

```

center=0,
cmap='coolwarm',
cbar=False,
annot=True,
fmt='.2f',
annot_kws={'size': 10},
ax=ax)
plt.show()

```



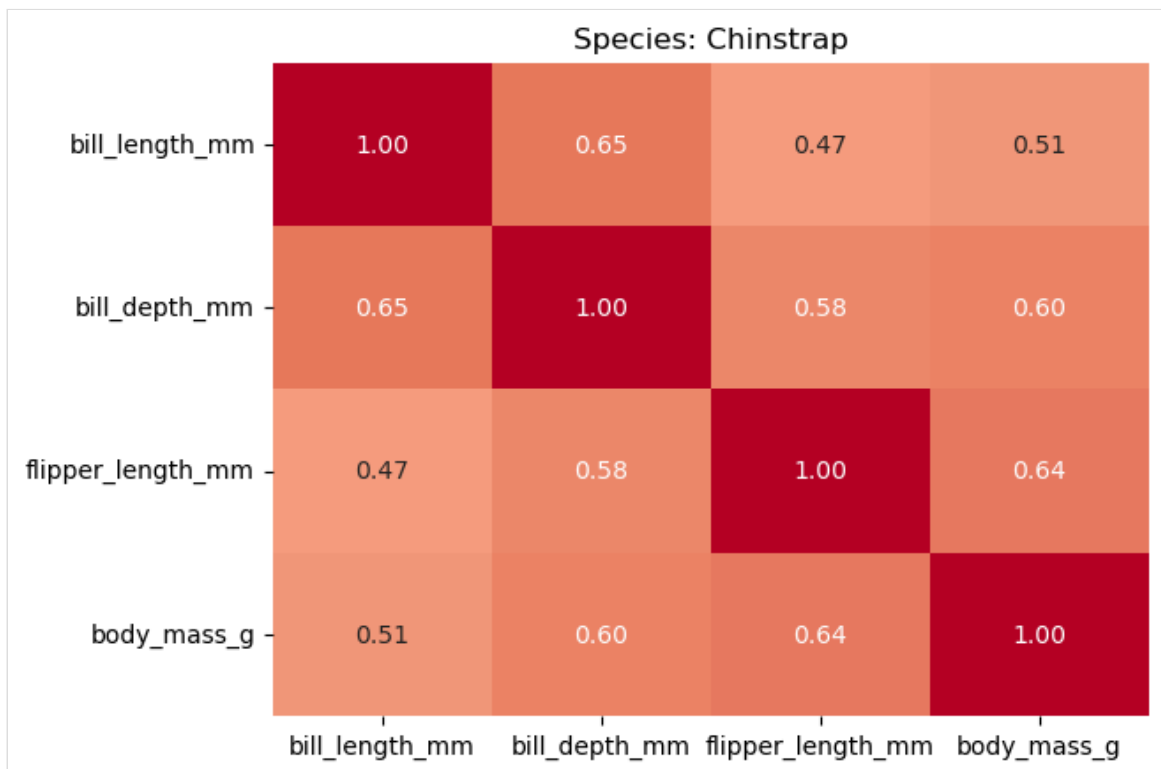
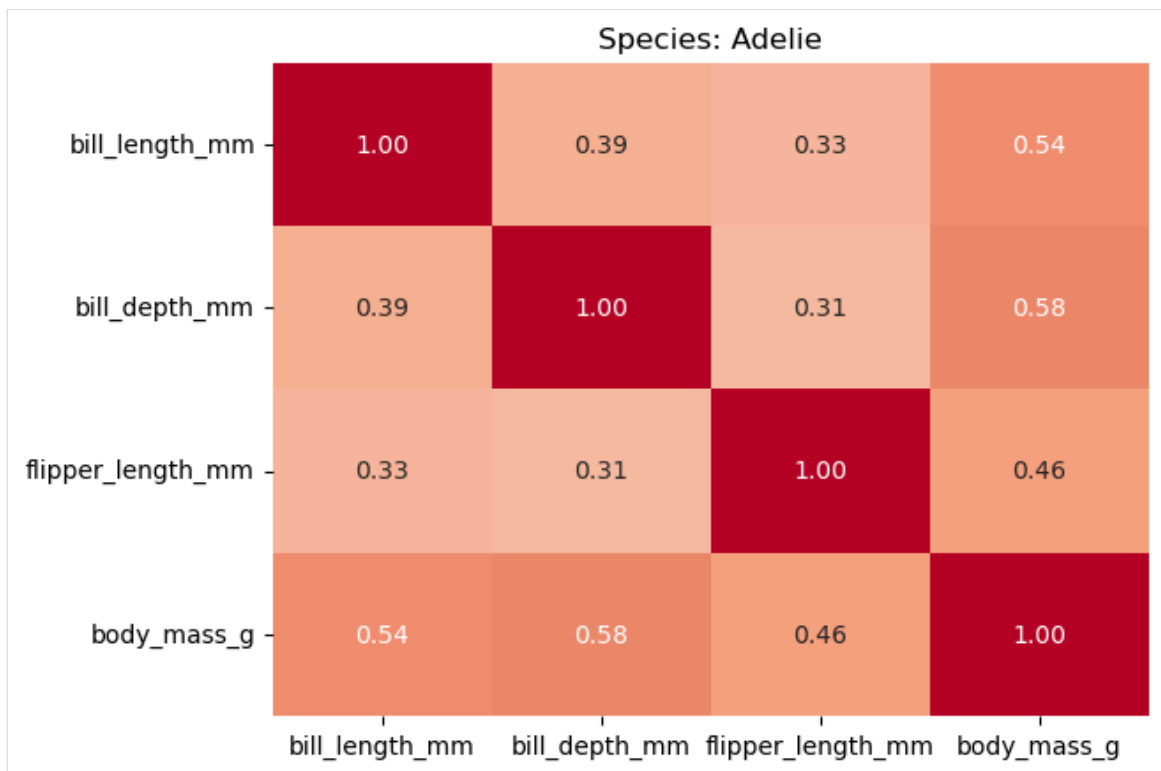
Correlation Plots – Correlation Matrix of All Numerical Variables, Faceted by `species` Categorical Variable

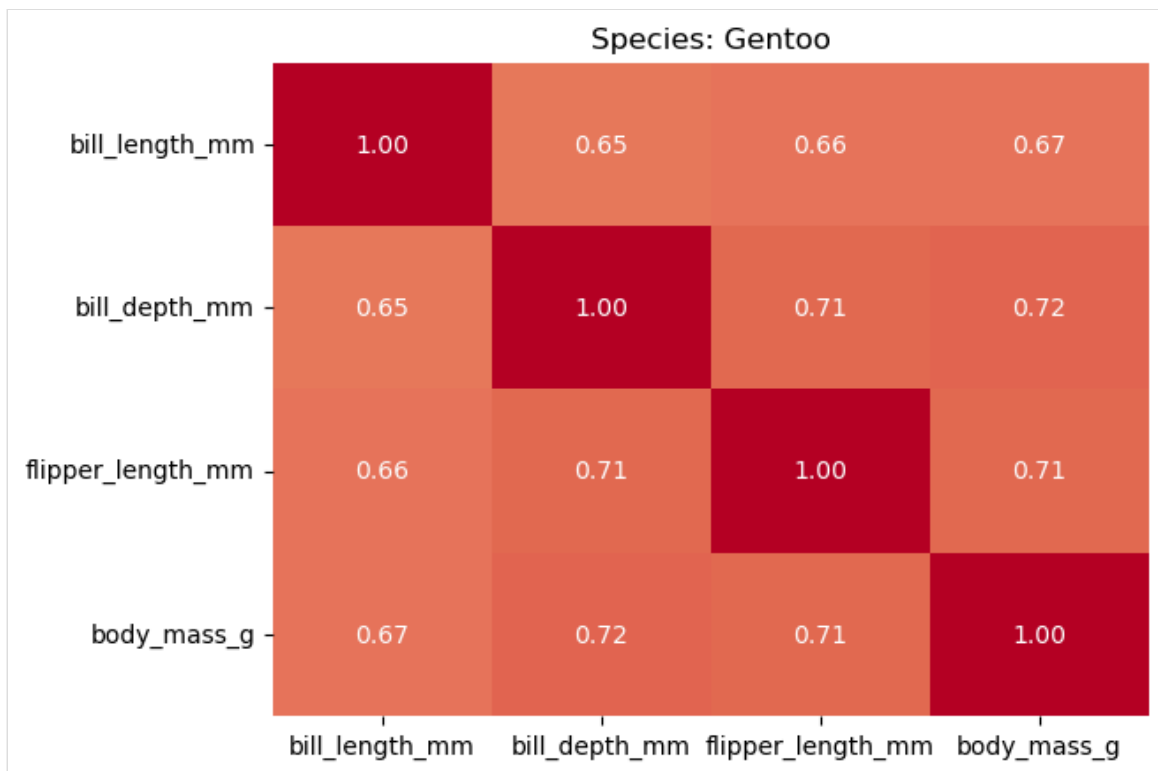
Correlations among bill length, bill depth, flipper length, and body mass are stronger in Gentoo penguins (all above 0.65), moderate in Chinstrap (mostly 0.47–0.65), and weaker in Adelie (mostly 0.31–0.58), indicating species-specific differences in how morphological traits are related.

```

species_list = df_clean['species'].unique()
for species in species_list:
    fig, ax = plt.subplots()
    subset = df_clean[df_clean['species'] == species]
    sns.heatmap(data=subset.corr(numeric_only=True),
                vmin=-1,
                vmax=1,
                center=0,
                cmap='coolwarm',
                cbar=False,
                annot=True,
                fmt='.2f',
                annot_kws={'size': 10},
                ax=ax)
    ax.set_title(f'Species: {species}')
    plt.show()

```



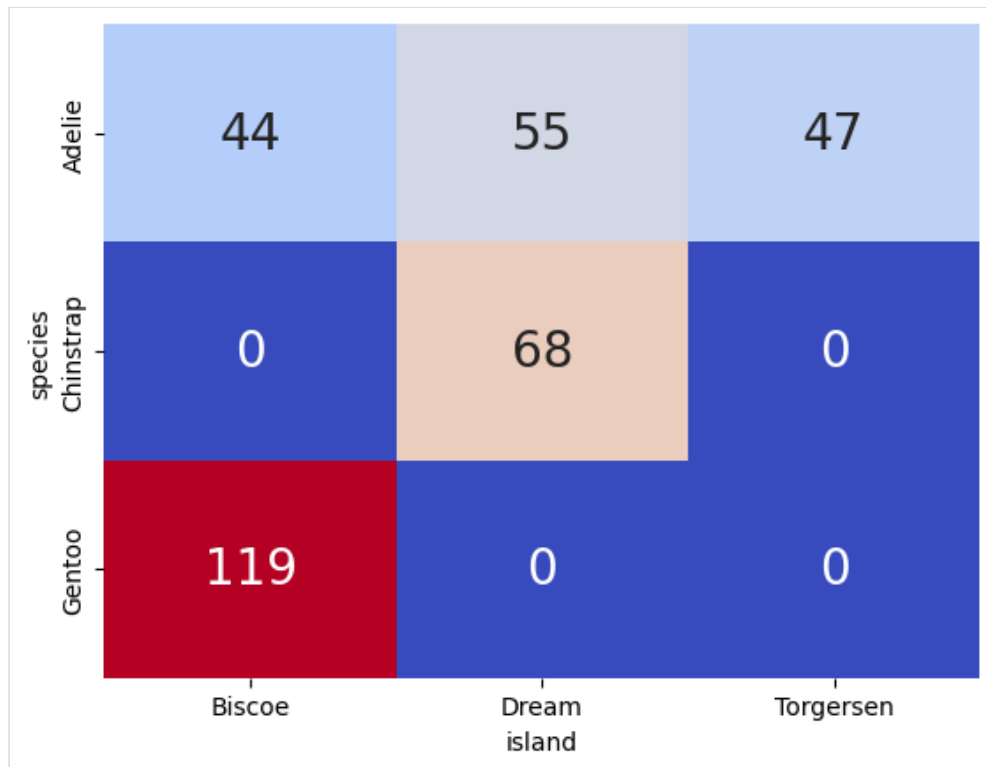


Heat Map – Counts of Observations for Each Combination of `species` and `island` Categorical Variables

Adelie penguins are present on all three islands, Chinstrap penguins are found only on Dream island, and Gentoo penguins are found only on Biscoe island.

```
fig, ax = plt.subplots()
sns.heatmap(pd.crosstab(df_clean.species,
                        df_clean.island),
            ax=ax,
            annot=True,
            annot_kws={'size': 20},
            fmt='d',
            cmap='coolwarm',
            cbar=False)

plt.show()
```

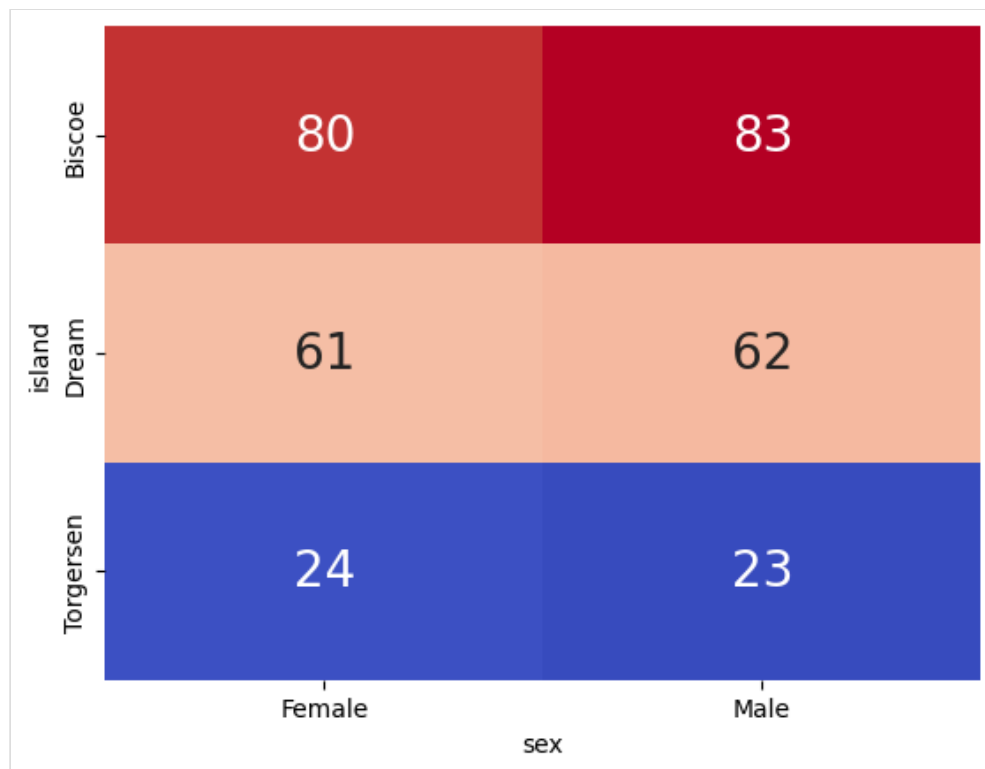


Heat Map – Counts of Observations for Each Combination of island and sex Categorical Variables

Biscoe has the highest numbers of both females (80) and males (83), Dream island also has similar counts of females (61) and males (62), and Torgersen has the lowest counts, with 24 females and 23 males. The distribution of sexes is fairly balanced across all islands.

```
fig, ax = plt.subplots()
sns.heatmap(pd.crosstab(df_clean.island,
                        df_clean.sex ),
            ax=ax,
            annot=True,
            annot_kws={'size': 20},
            fmt='d',
            cmap='coolwarm',
            cbar=False)

plt.show()
```



Final Summation of EDA and Key Features for Modeling

The exploratory data analysis of the Palmer Penguins dataset, comprising 333 cleaned records after removing missing values, revealed distinct morphological differences among the three species: Adelie, Chinstrap, and Gentoo. Univariate analyses showed that `flipper_length_mm` and `body_mass_g` have the widest ranges and are strongly right-skewed, while `bill_length_mm` and `bill_depth_mm` are more normally distributed. Categorical variables like `sex` (balanced male/female) and `island` (Biscoe most common) showed varying distributions, with Gentoo exclusive to Biscoe, Chinstrap to Dream, and Adelie on all islands. Bivariate and multivariate visualizations, including pair plots and correlation matrices, indicated that `flipper_length_mm` (correlation with `body_mass_g` ~ 0.87 overall) and `body_mass_g` are the strongest linear predictors for distinguishing species, with Gentoo showing the largest values, followed by Chinstrap and Adelie. Bill measurements also differ, with Chinstrap having the longest bills and Adelie the shortest. For modeling species classification, key features should include `flipper_length_mm`, `body_mass_g`, `bill_length_mm`, and `bill_depth_mm` as primary predictors, while considering `island` and `sex` for interaction effects, and potentially feature engineering to capture non-linear relationships or ratios (e.g., bill length to depth) to enhance model performance.

Key Takeaways

- Always profile shape, types, missingness, and distributions before modeling.
- Visualize relationships, not just summary statistics; a correlation can hide a curve.
- Entangled features, such as `island` and `species`, can leak the target and inflate accuracy.
- Outliers and impossible values are easier to find in plots than in tables.

Exercises

1. Compute the number and percentage of missing values in each column and decide how you would handle each one.
2. Plot the distribution of body mass separately for each species and describe the overlap.
3. Find a pair of features whose scatter plot separates the species almost perfectly.
4. Explain why using island as a feature could be a form of leakage.
5. Write a reusable function that prints the shape, dtypes, and missing-value counts of any dataframe.

Guided Walkthrough

The notebook begins by loading the Penguins data and immediately asking three questions: how big is it, what types are its columns, and where are the missing values. Those three answers determine almost everything that follows. The dataset has 344 rows and seven columns; three are continuous measurements, and the rest are categories. A handful of values are missing in the continuous columns, which forces a decision about imputation before any model can be fit.

The univariate plots come next. Histograms and count plots show the shape of each variable on its own. Body mass is roughly bell shaped, bill length is bimodal because it mixes species, and the categorical columns have uneven counts. Reading shape is a skill; a bimodal histogram is often the first clue that an unseen grouping variable is hiding in the data.

The bivariate section is where the dataset becomes interesting. Scatter plots of each measurement against every other, colored by species, show that the three species occupy overlapping but distinct regions. Flipper length and bill length separate them best, while bill depth overlaps. A correlation heatmap summarizes the linear relationships and warns that several measurements move together, which matters for regression because correlated predictors make individual coefficients unstable.

The chapter closes by looking for anomalies and class imbalance, the two things that most often break a model silently. The habit to take away is the order of operations: size, types, missingness, distributions, relationships, anomalies. Run it every time.

Common Pitfalls

- Jumping to modeling before understanding the data.
- Dropping rows with missing values without checking whether the missingness is informative.
- Trusting a correlation coefficient to describe a curved relationship.
- Ignoring class imbalance until after the model has been trained.

Going Further

- Compare listwise deletion with median imputation on a downstream model.
- Add a new engineered feature and see whether it separates the species better than the raw measurements.
- Read about Simpson's paradox and find a grouping variable that reverses a trend.

Exercise Solutions

1. The continuous columns have a few missing values each, under five percent. Median imputation is reasonable because the distributions are close to symmetric, but the missingness should be reported.
2. Body mass distributions overlap across species, with Gentoo shifted higher and Adelie lowest; the overlap means body mass alone cannot classify a penguin.
3. Flipper length against bill length separates the species almost perfectly, which foreshadows why those features matter most.
4. Island and species are entangled, because Gentoo live only on Bisco, so a model using island can learn the label indirectly.
5. The function loops over the columns and prints the dtype and the count of missing values for each.

Chapter 3: Penguins Dataset - Clustering

Introduction

Clustering is the art of finding groups in data without being told what the groups are. It is the first unsupervised method in this course, and it is the natural next question after exploratory analysis: now that we can see the data, can we let an algorithm partition it? This chapter applies KMeans clustering and principal component analysis to the Penguins dataset and shows both the power and the pitfalls of unsupervised learning.

KMeans is simple to state and subtle to use well. It alternates between assigning each point to the nearest cluster center and moving each center to the average of its points. Principal component analysis is a different idea: it finds the directions in which the data varies most and projects the data onto them, which lets us draw a two-dimensional picture of a higher-dimensional dataset. Together they form the standard toolkit for exploring structure.

What You Will Learn

- How KMeans clustering works and how to choose the number of clusters.
- Why features must be standardized before distance-based clustering.
- How to use the elbow method and silhouette-style reasoning to pick k .
- How principal component analysis compresses many features into a few.
- How to compare discovered clusters against known labels.

Overview

The chapter begins with a lesson that is easy to state and expensive to forget: distance-based methods are sensitive to scale. If one feature is measured in millimeters and another in grams, the larger-numbered feature dominates every distance calculation. We therefore standardize the numeric features so that each contributes equally, and we inspect the data before and after to see the effect.

With scaled features we run KMeans and confront the central question of clustering: how many clusters. The elbow method plots the within-cluster variation against the number of clusters and looks for the point where adding another cluster stops buying much. It is a heuristic, not a proof, and we treat it as one. We then compare the discovered clusters to the known species labels using a heatmap, which shows how well the unsupervised structure lines up with the truth. Perfect agreement would be suspicious; partial agreement is the honest and expected result, because the species overlap in measurement space.

Finally we turn to principal component analysis. We project the four-dimensional measurements onto two principal components and plot the result colored by species. The projection is a compression, so it loses information, but it reveals the dominant axes of variation. The chapter ends by combining clustering and PCA so you can see structure and groups in the same picture.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island       object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm  float64
body_mass_g    float64
sex          object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island       0
bill_length_mm  2
bill_depth_mm  2
flipper_length_mm  2
body_mass_g    2
sex          11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island       0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm  0
body_mass_g   0
sex          0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island       3
bill_length_mm 163
bill_depth_mm  79
flipper_length_mm 54
body_mass_g   93
sex          2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm 163
bill_depth_mm   79
flipper_length_mm 54
body_mass_g    93
dtype: int64
```

Data Standardization

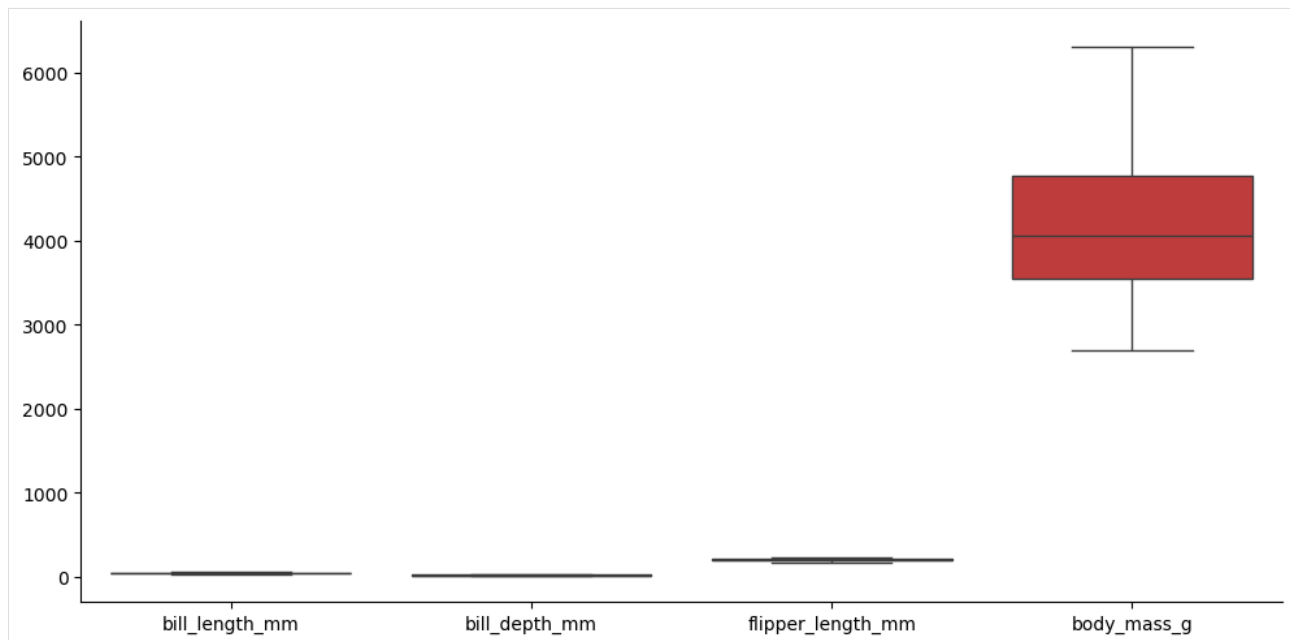
Examine raw numeric features magnitude and scale.

The actual values in the boxplot represent the distributions of the penguin dataset's numeric features: bill length (mm), bill depth (mm), flipper length (mm), and body mass (g). The plot shows that body mass is on a much larger scale than the other features, indicating that its values are much higher (in grams) compared to the millimeter measurements of the other features, which is important for understanding feature scaling and the need for standardization in data analysis.

```
df_clean.describe()
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	333.000000	333.000000	333.000000	333.000000
mean	43.992793	17.164865	200.966967	4207.057057
std	5.468668	1.969235	14.015765	805.215802
min	32.100000	13.100000	172.000000	2700.000000
25%	39.500000	15.600000	190.000000	3550.000000
50%	44.500000	17.300000	197.000000	4050.000000
75%	48.600000	18.700000	213.000000	4775.000000
max	59.600000	21.500000	231.000000	6300.000000

```
sns.catplot(data=df_clean,
            kind='box',
            aspect=2)
plt.show()
```



Prepare the dataset by extracting our numeric features.

```
df_clean_features = df_clean.select_dtypes('number').\
    copy()
```

Standardize the numeric features.

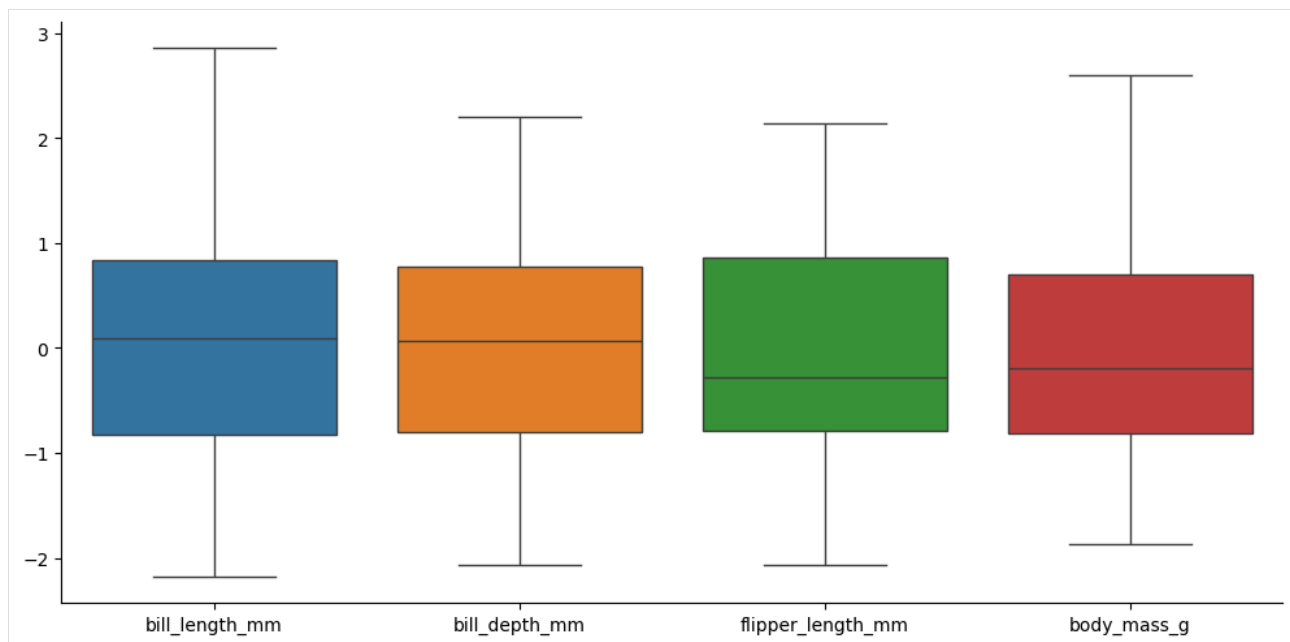
```
X = StandardScaler().fit_transform(df_clean_features)
```

Examine standardized numeric features magnitude and scale.

After standardization, all numeric features (bill length, bill depth, flipper length, and body mass) have similar distributions centered around zero with comparable spread, indicating that differences in scale have been removed and the features are now suitable for clustering and other analyses that assume equal variance.

```
sns.catplot(data=pd.DataFrame(X,
                               columns=df_clean_features.columns),
            kind='box',
            aspect=2)

plt.show()
```

**KMeans****Initialize, fit and predict our KMeans by starting with two clusters.**

```
clusters_2 = KMeans(n_clusters=2,
                    random_state=42,
                    n_init=25,
                    max_iter=500).\
                    fit_predict(X)
```

Assign cluster labels to a clean copied DataFrame.

```
df_clean_copy = df_clean.copy()
```

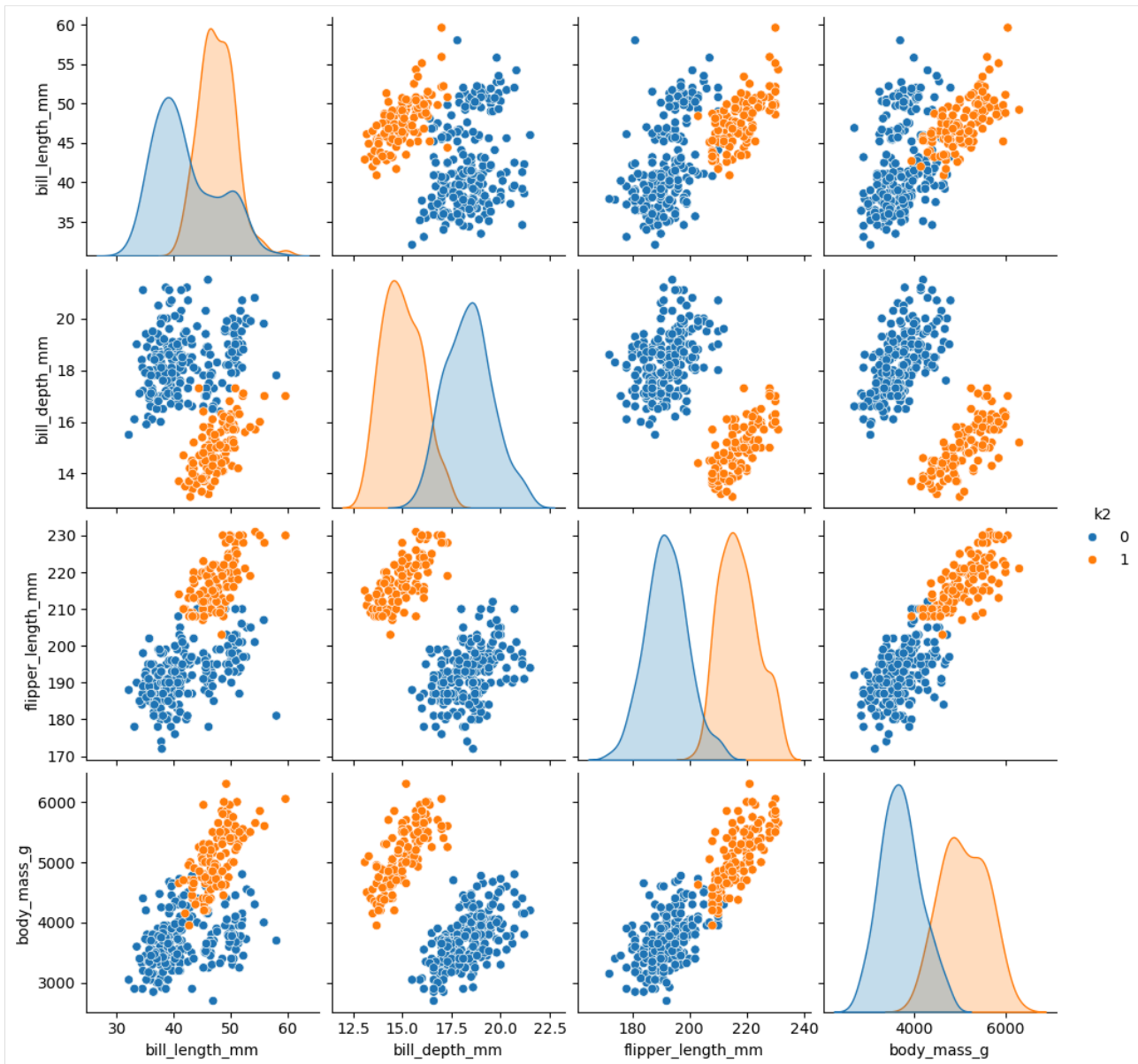
```
df_clean_copy['k2'] = pd.Series(clusters_2,
                               index=df_clean_copy.index). \
```

```
astype('category')
```

Visualize the clusters using a pair plot.

The pair plot shows that the KMeans clustering with two clusters separates the penguins into two distinct groups based on their physical measurements, with clear differences in bill length, bill depth, flipper length, and body mass between the clusters, indicating strong feature-based separation in the dataset.

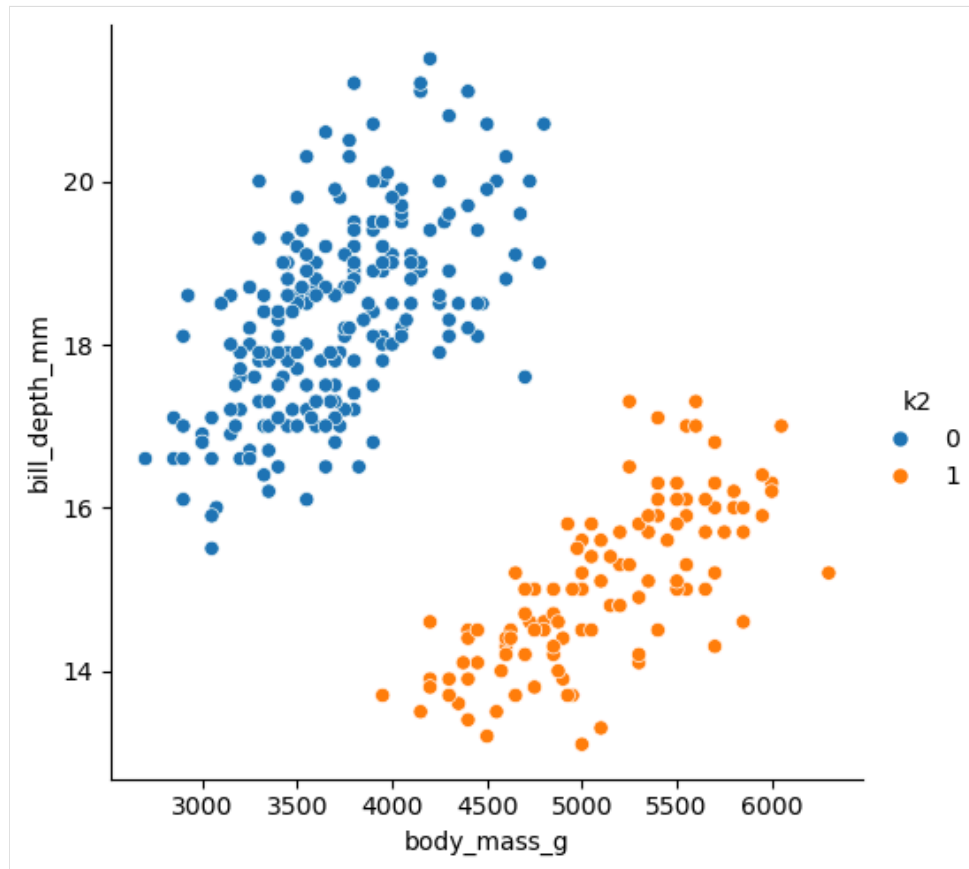
```
sns.pairplot(data=df_clean_copy,
             hue='k2',
             diag_kws={'common_norm': False})
plt.show()
```



Visualize the cluster results on a scatter plot.

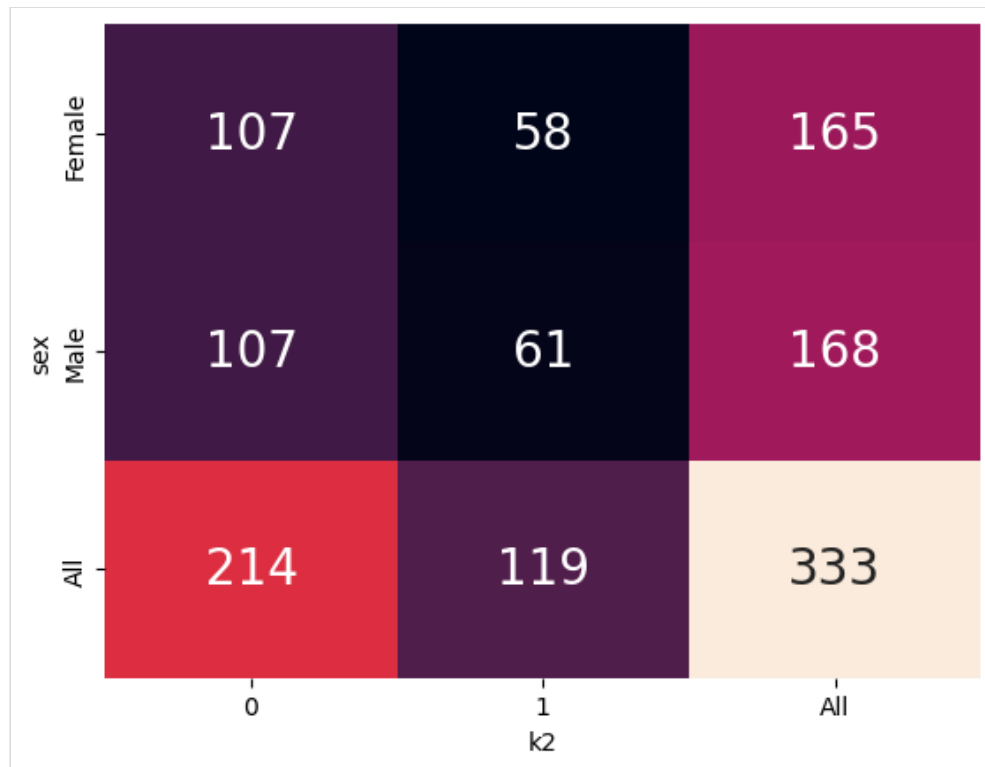
This scatter plot demonstrates that the two clusters identified by KMeans are well separated, with one group of penguins having higher body mass and lower bill depth, and the other group having lower body mass and higher bill depth, highlighting distinct physical differences between the clusters.

```
sns.relplot(data=df_clean_copy,
            x='body_mass_g',
            y='bill_depth_mm',
            hue='k2')
plt.show()
```

**Use a heatmap to visually show how the clusters align with the known groupings.**

The heatmap shows that both clusters contain similar numbers of male and female penguins, indicating that the clustering is not strongly driven by sex and that both groups are well represented across this categorical variable.

```
fig, ax = plt.subplots()
sns.heatmap(data=pd.crosstab(df_clean_copy.sex,
                             df_clean_copy.k2,
                             margins=True),
            annot=True,
            annot_kws={"fontsize": 20},
            fmt='g',
            cbar=False,
            ax=ax)
plt.show()
```



Determine the optimal number of clusters.

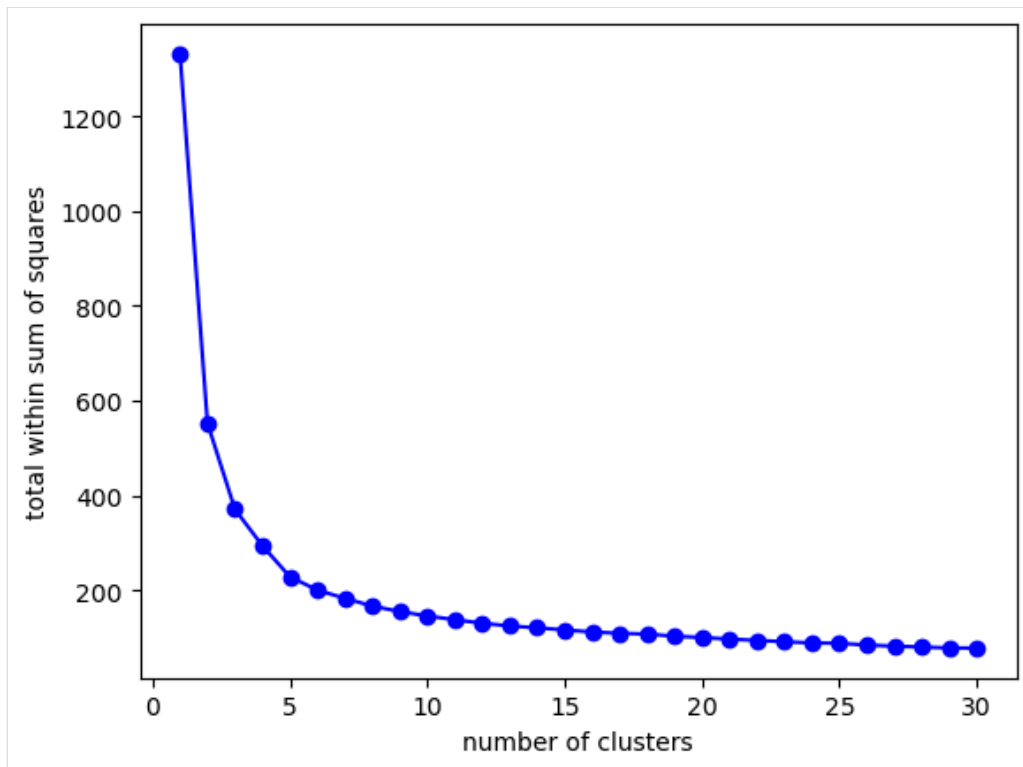
```
tots_within = []
K = range(1, 31)
for k in K:
    km = KMeans(n_clusters=k,
                random_state=42,
                n_init=25,
                max_iter=500)
    km = km.fit(X)
    tots_within.append(km.inertia_)
```

Visualize the optimal number of clusters.

The elbow plot shows a sharp decrease in total within sum of squares as the number of clusters increases, with the curve flattening after about 2–3 clusters, suggesting that two or three clusters are optimal for capturing the main structure in the data.

```
fig, ax = plt.subplots()
ax.plot(K, tots_within, 'bo-')
ax.set_xlabel('number of clusters')
ax.set_ylabel('total within sum of squares')

plt.show()
```



Use three clusters.

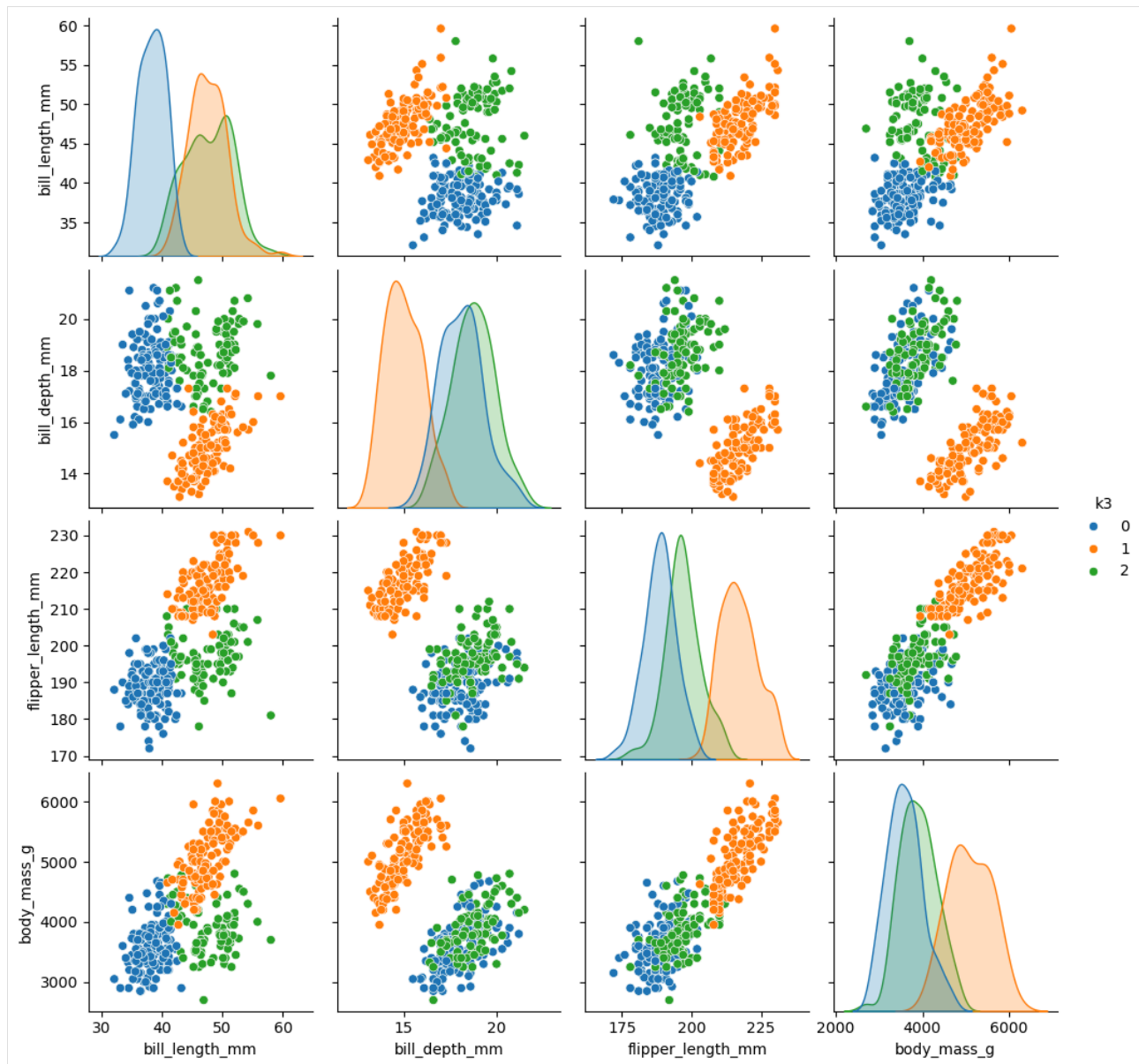
```
clusters_3 = KMeans(n_clusters=3,  
                    random_state=42,  
                    n_init=25,  
                    max_iter=500). \  
    fit_predict(X)
```

```
df_clean_copy['k3'] = pd.Series(clusters_3,  
                               index=df_clean_copy.index). \  
    astype('category')
```

Visualize the clusters using a pair plot.

The pair plot with three clusters reveals that the penguins are grouped into three distinct sets based on their physical measurements, with each cluster showing unique combinations of bill length, bill depth, flipper length, and body mass, suggesting the presence of three underlying subpopulations in the dataset.

```
sns.pairplot(data=df_clean_copy,  
            hue='k3',  
            diag_kws={'common_norm': False})  
plt.show()
```



Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean_copy.select_dtypes(exclude='number').nunique()
```

```
species    3
island     3
sex        2
k2         2
k3         3
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

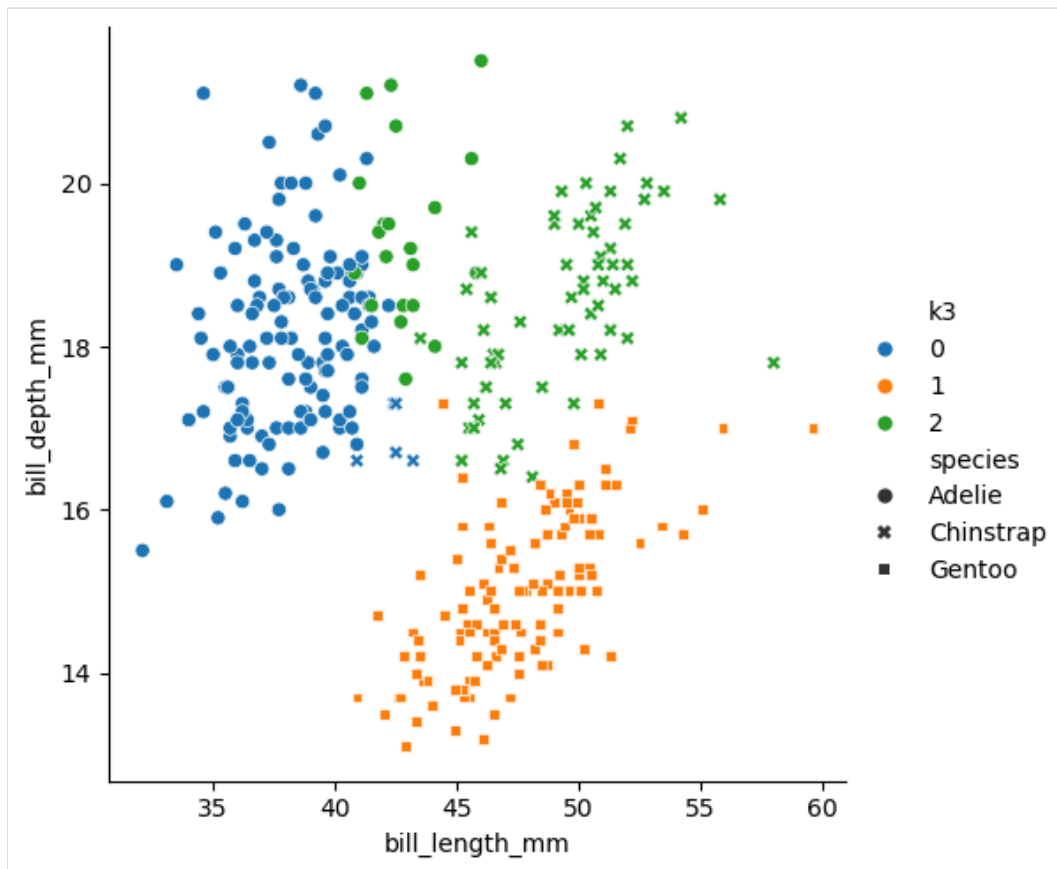
```
df_clean_copy.select_dtypes(include='number').nunique()
```

```
bill_length_mm    163
bill_depth_mm     79
flipper_length_mm  54
body_mass_g       93
dtype: int64
```

Visualize the cluster results on a scatter plot.

This scatter plot shows that the three clusters correspond closely to the three penguin species, with each species occupying a distinct region in the bill length and bill depth space, demonstrating that the clustering effectively captures species-level differences in these physical traits.

```
sns.relplot(data=df_clean_copy,
            x='bill_length_mm',
            y='bill_depth_mm',
            hue='k3',
            style='species')
plt.show()
```

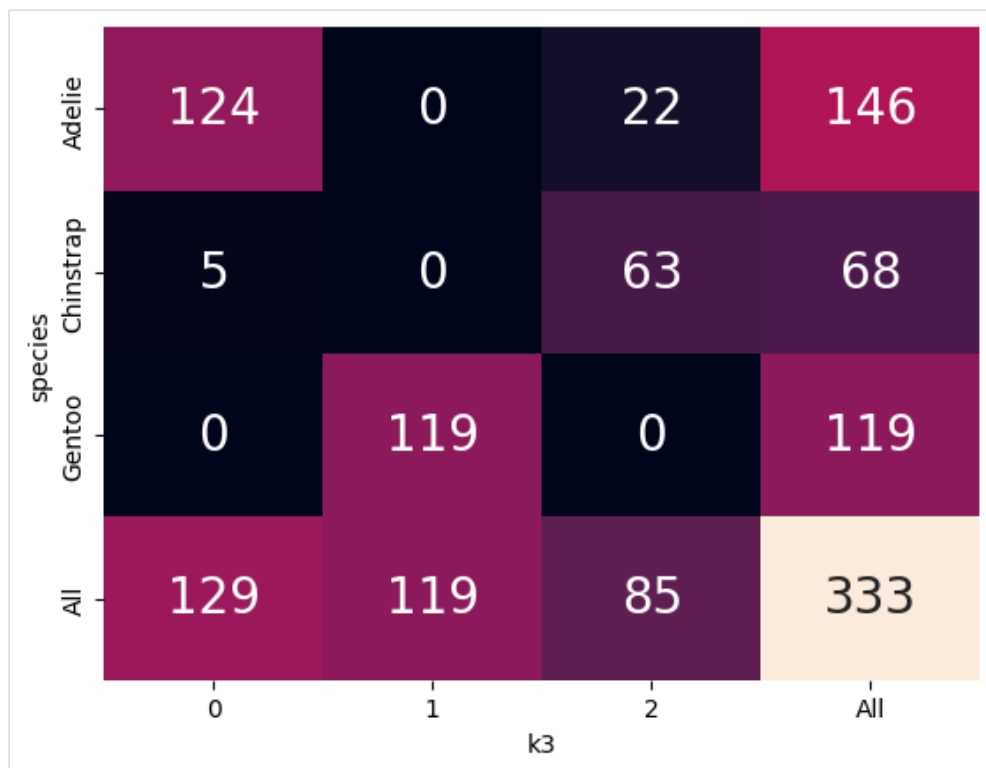


Use a heatmap to visually show how the clusters align with the known groupings.

The heatmap shows a strong correspondence between clusters and species: each cluster almost exclusively contains one species, confirming that the clustering algorithm has successfully separated Adelie, Chinstrap, and Gentoo penguins based on their physical measurements.

```
fig, ax = plt.subplots()
sns.heatmap(data=pd.crosstab(df_clean_copy.species,
                             df_clean_copy.k3,
                             margins=True),
            annot=True,
            annot_kws={"fontsize": 20},
            fmt='g',
            cbar=False,
            ax=ax)

plt.show()
```



PCA

```
pca_df_clean_copy = PCA(n_components=2). \
    fit_transform(X)
```

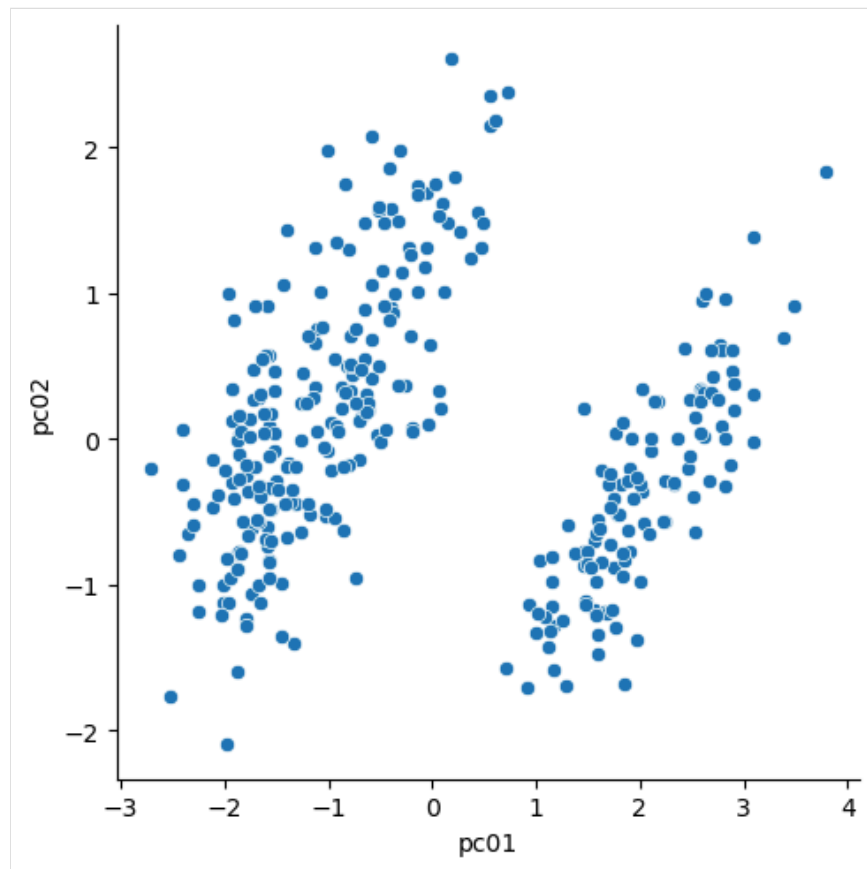
```
pca_df_clean_copy_df = pd.DataFrame(pca_df_clean_copy,
                                     columns=['pc01',
                                              'pc02'])
```

```
pca_df_clean_copy_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 333 entries, 0 to 332
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype  
---  -
0   pc01     333 non-null     float64
1   pc02     333 non-null     float64
dtypes: float64(2)
memory usage: 5.3 KB
```

This PCA scatter plot shows that the first two principal components capture clear structure in the data, with the points forming two well-separated groups, indicating strong underlying variation and suggesting that dimensionality reduction effectively reveals major patterns in the penguin measurements.

```
sns.relplot(data=pca_df_clean_copy_df,
            x='pc01',
            y='pc02')
plt.show()
```



The heatmap of the PCA components' correlation matrix shows that pc01 and pc02 are completely uncorrelated (correlation ≈ 0), confirming that the principal components are orthogonal and capture independent sources of variance in the standardized penguin measurements.

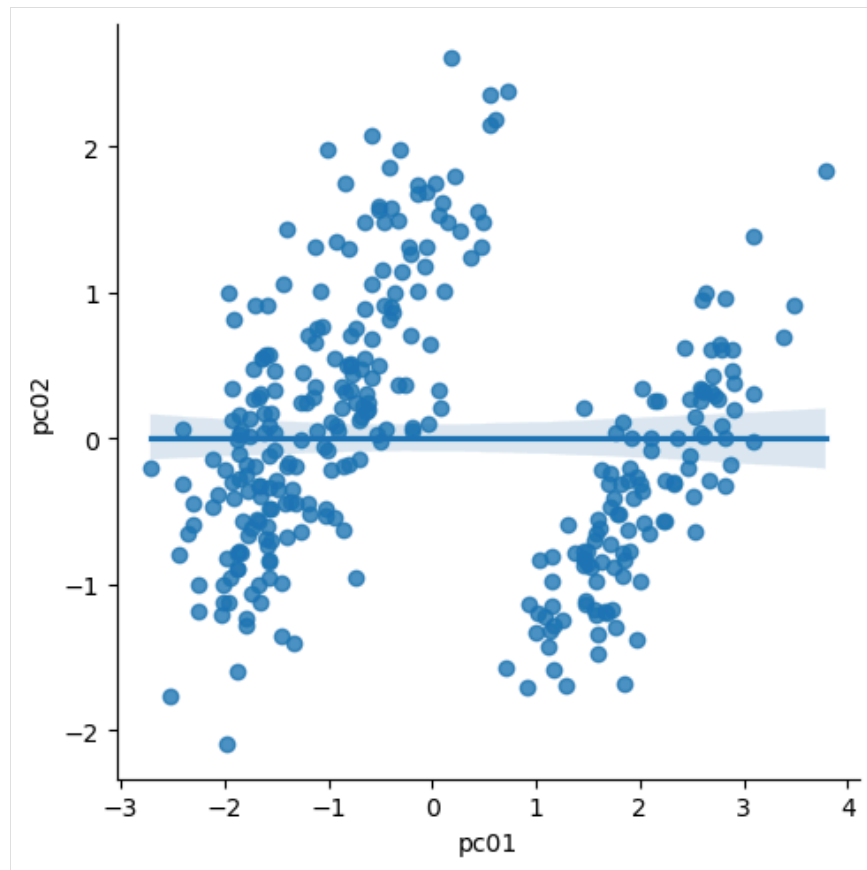
```
fig, ax = plt.subplots()
sns.heatmap(pca_df_clean_copy_df.corr(numeric_only=True),
```

```
vmin=-1,  
vmax=1,  
center=0,  
cmap='coolwarm',  
annot=True,  
annot_kws={'fontsize': 20},  
fmt='.3f',  
ax=ax)  
  
plt.show()
```



This scatter plot with regression line shows that there is no significant linear relationship between the first and second principal components, confirming their statistical independence and the effectiveness of PCA in separating the main sources of variance in the data.

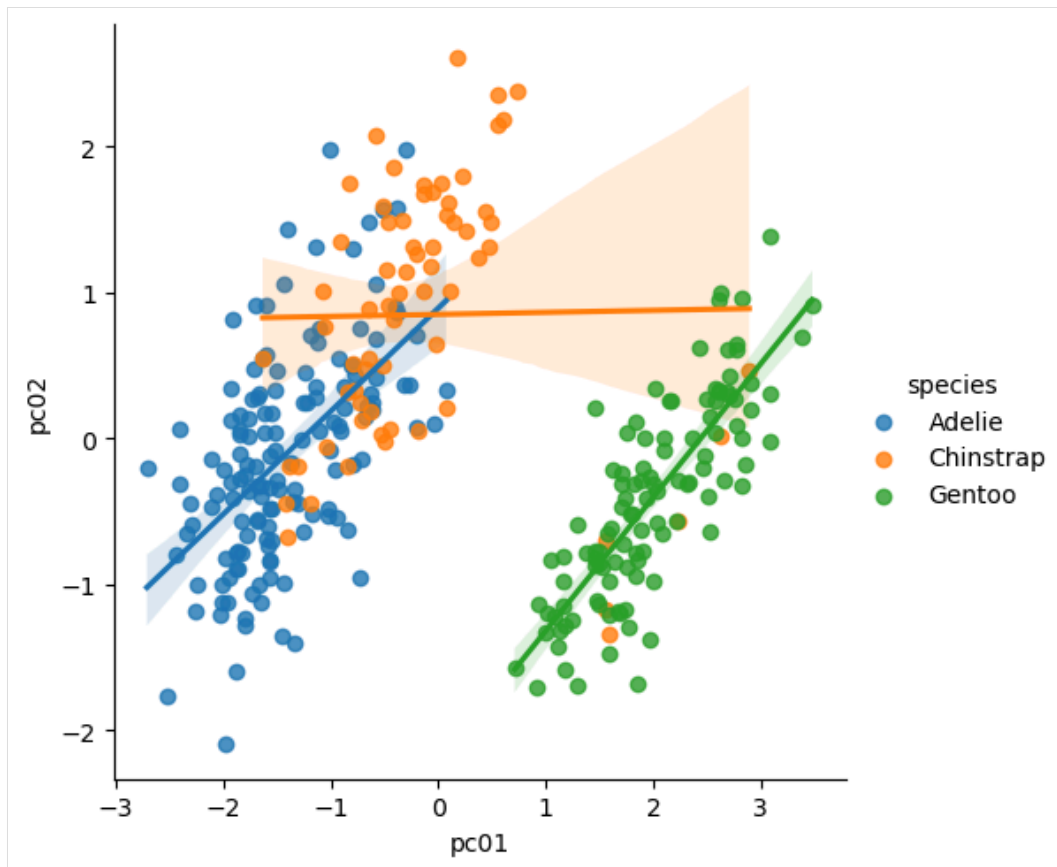
```
sns.lmplot(data=pca_df_clean_copy_df,  
           x='pc01',  
           y='pc02')  
  
plt.show()
```

The regression lines in this PCA scatter plot show that each penguin species has a distinct linear trend in the principal component space, highlighting species-specific variation and confirming that PCA effectively separates the three species based on their physical measurements.

```
pca_df_clean_copy_df['species'] = df_clean_copy['species']
```

```
sns.lmplot(data=pca_df_clean_copy_df,  
           x='pc01',  
           y='pc02',  
           hue='species')  
plt.show()
```



Clustering & PCA

Both scatter plots demonstrate that the KMeans clustering algorithm effectively separates the penguins into two distinct groups, whether visualized in the original feature space (body mass vs. bill depth) or in the reduced PCA space (pc01 vs. pc02). In both cases, the clusters are well separated, indicating that the underlying physical measurements and their principal components capture strong, non-overlapping patterns in the data. The PCA plot further confirms that dimensionality reduction preserves the cluster structure, with each group occupying a distinct region in the principal component space, validating the consistency and robustness of the clustering results across different representations of the data.

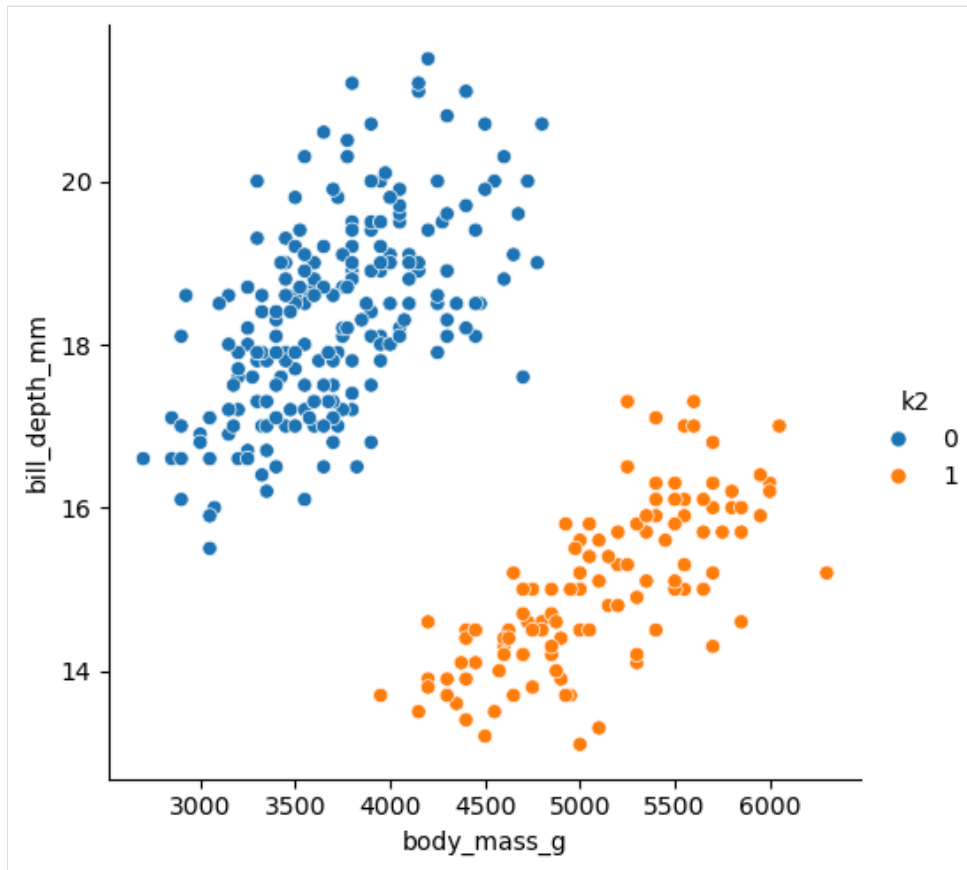
```
pca_df_clean_copy_df['k2'] = pd.Series(clusters_2,
                                       index=pca_df_clean_copy_df. \
                                       index). \
                                       astype('category')
```

```
pca_df_clean_copy_df.info()
```

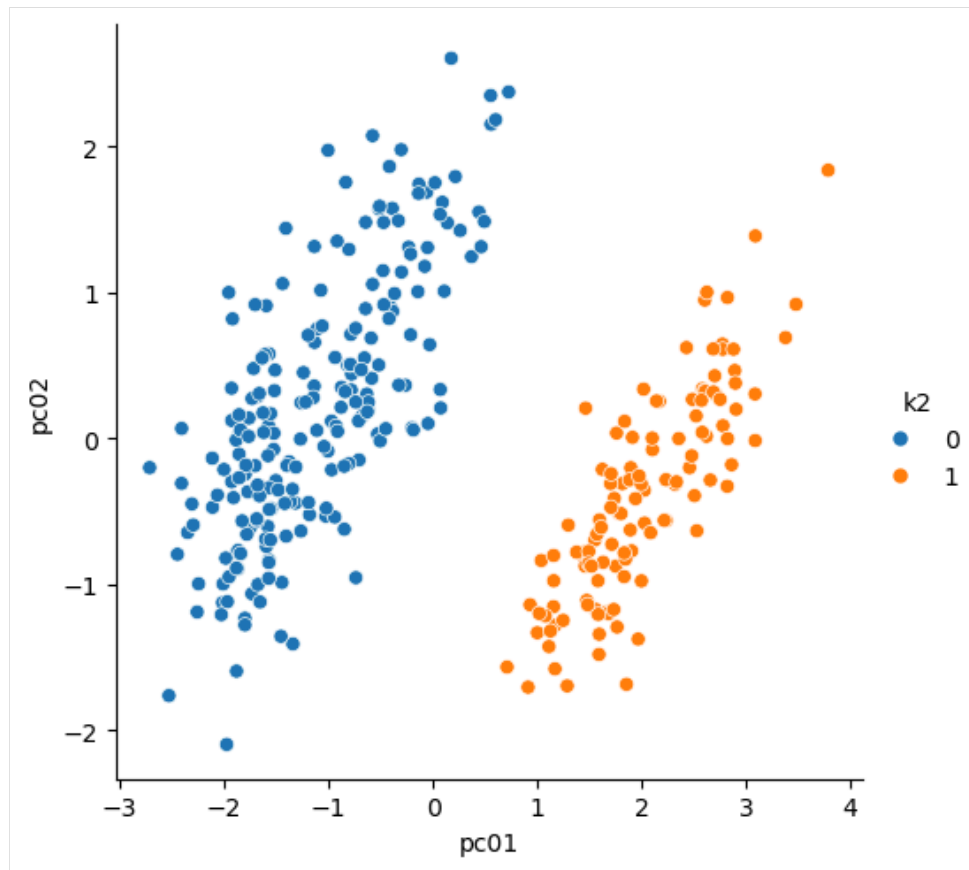
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 333 entries, 0 to 332
Data columns (total 4 columns):
#   Column   Non-Null Count  Dtype
---  ---
0   pc01      333 non-null    float64
1   pc02      333 non-null    float64
```

```
2  species  324 non-null  object
3  k2       333 non-null  category
dtypes: category(1), float64(2), object(1)
memory usage: 8.4+ KB
```

```
sns.relplot(data=df_clean_copy,
            x='body_mass_g',
            y='bill_depth_mm',
            hue='k2')
plt.show()
```



```
sns.relplot(data=pca_df_clean_copy_df,
            x='pc01',
            y='pc02',
            hue='k2')
plt.show()
```



Key Takeaways

- Standardize features before any distance-based method.
- The number of clusters is a modeling choice; use the elbow method as a guide, not a verdict.
- Clusters are not labels; compare them to ground truth with a cross-tabulation.
- Principal component analysis finds the directions of greatest variance and enables low-dimensional visualization.
- Overlap between classes is normal and is what makes a problem worth solving.

Exercises

1. Run KMeans with two, three, and four clusters and compare the cross-tabulations with species.
2. Repeat the analysis without standardization and describe how the clusters change.
3. Plot the explained variance ratio of the principal components and decide how many to keep.
4. Color the PCA projection by cluster instead of species and compare the two pictures.
5. Write a function that standardizes a dataframe's numeric columns and returns the result.

Guided Walkthrough

The notebook starts with a lesson in units. Before clustering, it prints the raw ranges of the numeric features and shows that body mass is measured in thousands while bill depth is measured in tens. Any distance-based method will be dominated by the largest-numbered feature, so the first real step is standardization. After scaling, the notebook reprints the ranges to confirm that every feature now has mean zero and standard deviation one.

With scaled features the notebook runs KMeans. It begins with two clusters and plots the result, then uses the elbow method to justify a larger number. The elbow plot shows within-cluster variation falling as clusters are added, and the bend in the curve suggests three, which happens to match the number of species. That agreement is reassuring but should never be assumed; the notebook then cross-tabulates clusters against species with a heatmap and finds strong but imperfect alignment.

Principal component analysis is introduced as a way to see four dimensions at once. The notebook projects the scaled measurements onto the first two principal components, plots the points colored by species, and shows that most of the separation survives the compression. The explained variance ratio confirms how much information the two components retained. The chapter ends by overlaying clusters and components, so structure and grouping appear in the same picture.

Common Pitfalls

- Clustering unscaled features, which lets one variable dominate the distance.
- Treating the elbow as a precise answer rather than a guide.
- Assuming clusters correspond to real categories.
- Reading too much into a two-dimensional projection that discards variance.

Going Further

- Try a density-based method such as DBSCAN and compare the clusters it finds.
- Standardize with a robust scaler and see whether outliers change the result.
- Color the PCA plot by a variable the clustering never saw, such as island, to look for hidden structure.

Exercise Solutions

1. With two clusters the split is coarse; with three it aligns best with species; with four one species is split into two clusters.
2. Without standardization the clusters are driven almost entirely by body mass, because its scale is largest.
3. The explained variance ratio typically shows the first component capturing most of the variation and the second a smaller but meaningful share.
4. Coloring by cluster shows the discovered groups; coloring by species shows the truth; the differences are where clustering fails.
5. The function selects numeric columns, fits a scaler, transforms the data, and returns a dataframe with the same columns.

Part II: Simple Linear Regression

Chapter 4: Penguins Dataset Simple Linear Regression - Linear Relationship

Introduction

Simple linear regression is the workhorse of statistics and the first model every practitioner should master. It fits a straight line through a cloud of points by choosing the slope and intercept that minimize the sum of squared errors. This chapter introduces the idea on a genuinely linear relationship in the Penguins dataset, so that the model's assumptions are not yet strained and you can see clearly what fitting a line means.

What You Will Learn

- What a simple linear regression model is and what its coefficients mean.
- How to fit a line with the statsmodels formula interface.
- How to visualize the fitted line against the data.
- How to interpret the slope as a rate of change and the intercept as a baseline.
- How to read the residual pattern to judge whether a line is appropriate.

Overview

We pick one continuous input and one continuous output and ask for the best straight line between them. The formula interface lets us write the model as an equation, which keeps the statistics close to the mathematics. We fit the model, read the coefficients, and then draw the line over the scatter plot. The slope tells us how much the output changes for a one-unit change in the input, and the intercept tells us the output when the input is zero, which may or may not be meaningful depending on the domain.

The critical habit introduced here is looking at residuals, the differences between the observed and predicted values. A good linear fit leaves residuals that look like random noise with no pattern. A pattern means the relationship is not linear, and the next chapters show what to do about that.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island        0
bill_length_mm  2
bill_depth_mm  2
flipper_length_mm 2
body_mass_g    2
sex          11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island        0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm 0
body_mass_g    0
sex           0
dtype: int64
```


Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island       3
bill_length_mm 163
bill_depth_mm 79
flipper_length_mm 54
body_mass_g   93
sex          2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm 163
bill_depth_mm  79
flipper_length_mm 54
body_mass_g    93
dtype: int64
```

Linear Relationship

We will start with a single continuous input that is linearly related to the trend. In our case we will attempt to use `flipper_length_mm` to predict `body_mass_g`.

$$\mu = \beta_0 + \beta_1 \times x$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **slope**: change in μ for a one-unit increase in x

x **feature** or **input variable**: independent variable used to predict μ

Visualize the Trend

The trend is increasing as the input increases because we have a positive **slope**.

```
x = df_clean['flipper_length_mm']
y = df_clean['body_mass_g']
```

```
my_slope, my_intercept = np.polyfit(x,
                                     y,
                                     1)
```

```
my_slope
```

```
50.153265942241134
```

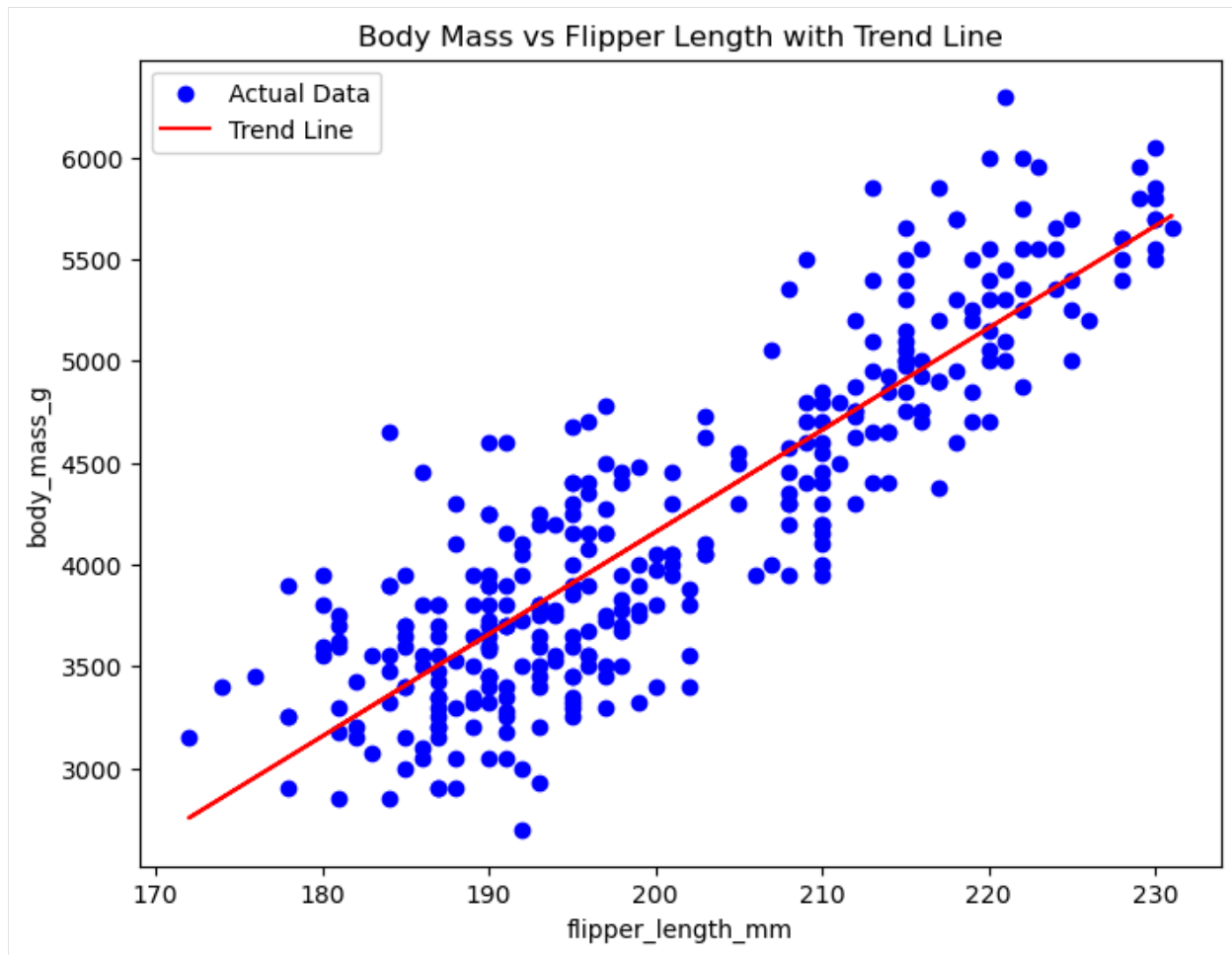
```
my_intercept
```

```
-5872.09268284282
```

```
df_viz = df_clean.copy()
```

```
df_viz['trend'] = my_intercept + \
                  my_slope * \
                  df_viz['flipper_length_mm']
```

```
plt.figure(figsize=(8,6))
plt.scatter(df_viz['flipper_length_mm'],
            df_viz['body_mass_g'],
            color='blue',
            label='Actual Data')
plt.plot(df_viz['flipper_length_mm'],
         df_viz['trend'],
         color='red',
         label='Trend Line')
plt.xlabel('flipper_length_mm')
plt.ylabel('body_mass_g')
plt.title('Body Mass vs Flipper Length with Trend Line')
plt.legend()
plt.show()
```



Calculating σ

The likelihood of an output around the trend is a Gaussian distribution and the variation of a Gaussian is specified by the **standard deviation** or σ .

To calculate **sigma** (σ), the **standard deviation** of the residuals in linear regression, use the following formula.

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \mu_i)^2}$$

where:

- y_i is the actual observed value for the i th point,
- μ_i is the predicted value for the i th point (from your regression line),
- n is the number of data points.

Step-by-step explanation.

1. **Calculate the residuals:** $y_i - \mu_i$.
2. **Square the residuals:** $(y_i - \mu_i)^2$.
3. **Sum squared residuals:** $\sum_{i=1}^n (y_i - \mu_i)^2$.
4. **Divide by the number of points:** $\frac{1}{n} \sum_{i=1}^n (y_i - \mu_i)^2$.
5. **Take the square root:** $\sqrt{\cdot}$.

If you want the **sample standard deviation** (which is common), use $n - 1$ in the denominator.

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (y_i - \mu_i)^2}$$

σ measures how much your actual values deviate from the values predicted by the regression line.

```
my_residuals = df_viz['body_mass_g'] - df_viz['trend']
```

```
my_sigma = np.sqrt(np.sum(my_residuals**2) / (len(my_residuals) - 1))
```

```
my_sigma
```

```
392.7504293927682
```

Visualize the variation around the trend or uncertainty of a single output measurement around the average output.

```
df_viz['obs_lwr_68'] = df_viz.trend - my_sigma
df_viz['obs_upr_68'] = df_viz.trend + my_sigma
```

```
df_viz['obs_lwr_95'] = df_viz.trend - 2 * my_sigma
df_viz['obs_upr_95'] = df_viz.trend + 2 * my_sigma
```

```
# Sort by flipper_length_mm for continuous plotting
df_viz_sorted = df_viz.sort_values('flipper_length_mm')

# Create the figure and axis
fig, ax = plt.subplots()

# True trend
ax.plot(df_viz_sorted.flipper_length_mm,
        df_viz_sorted.trend,
        color='crimson',
        linewidth=1.5)

# True variation around the trend showing 2 sigma intervals
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_95,
                df_viz_sorted.obs_upr_95,
                facecolor='crimson',
                alpha=0.35)
```

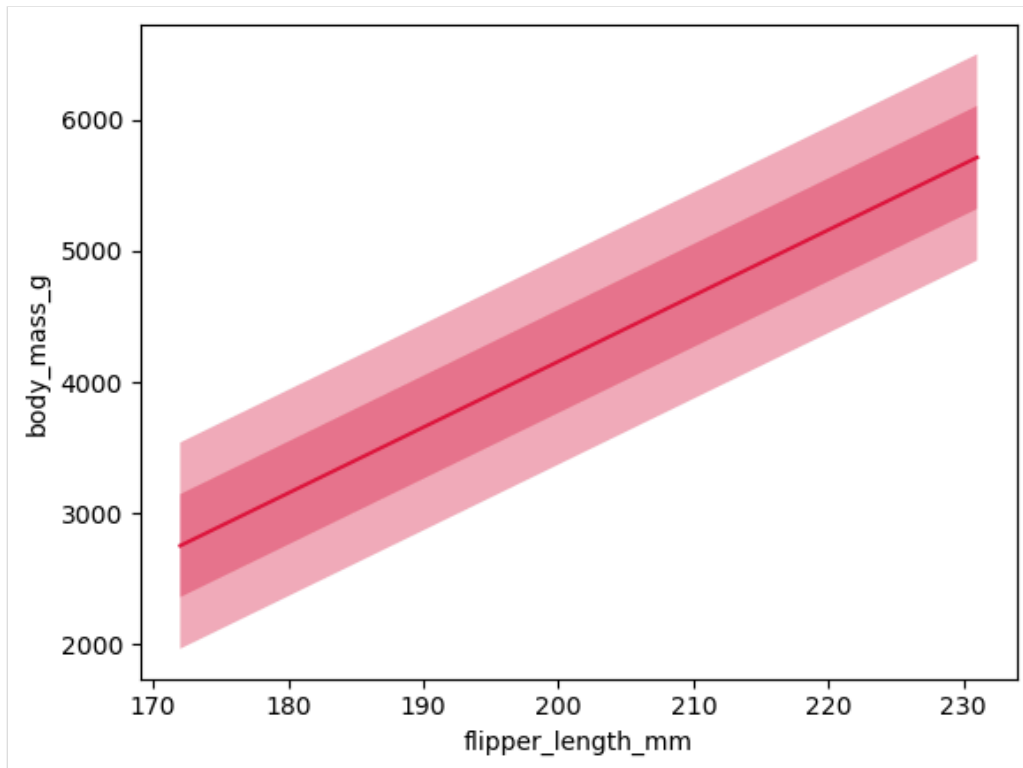
```

ax.fill_between(df_viz_sorted.flipper_length_mm,
               df_viz_sorted.obs_lwr_68,
               df_viz_sorted.obs_upr_68,
               facecolor='crimson',
               alpha=0.35)

# Set labels
ax.set_xlabel('flipper_length_mm')
ax.set_ylabel('body_mass_g')

# Show the plot
plt.show()

```



Calculate the trend given the input values.

```

# Sort by flipper_length_mm for continuous plotting
df_viz_sorted = df_viz.sort_values('flipper_length_mm')

# Create the figure and axis
fig, ax = plt.subplots()

# True trend
ax.plot(df_viz_sorted.flipper_length_mm,
        df_viz_sorted.trend,
        color='crimson',
        linewidth=1.5)

# True variation around the trend showing 2 sigma intervals
ax.fill_between(df_viz_sorted.flipper_length_mm,
               df_viz_sorted.obs_lwr_95,
               df_viz_sorted.obs_upr_95,
               facecolor='crimson',

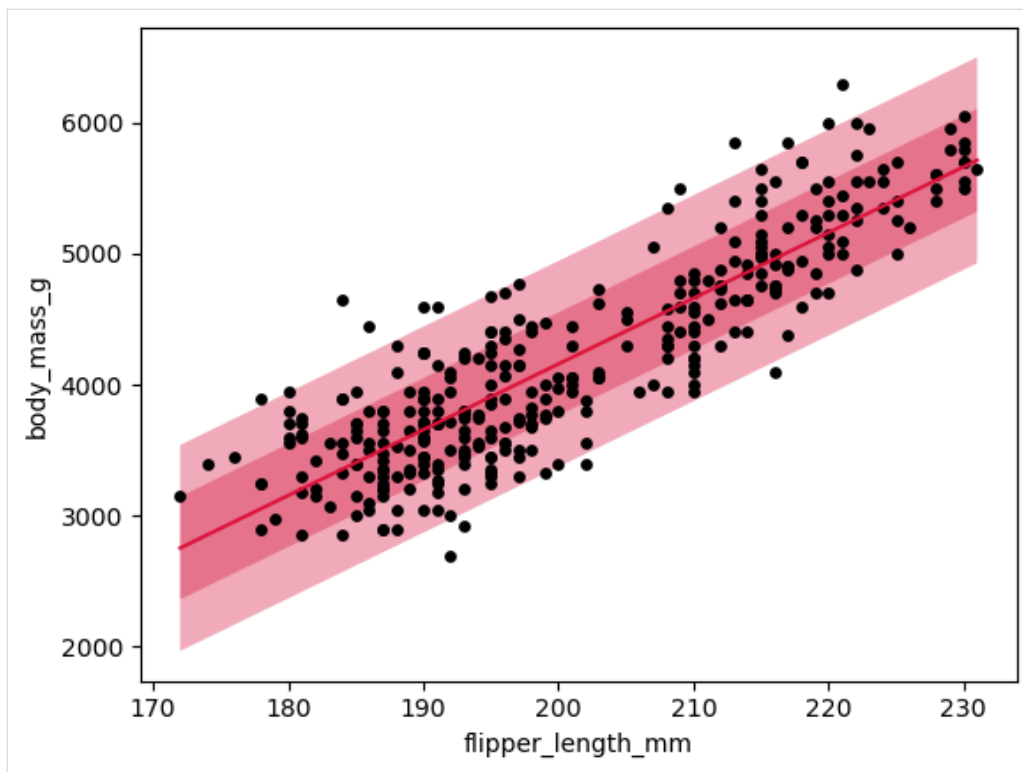
```

```
alpha=0.35)
ax.fill_between(df_viz_sorted.flipper_length_mm,
               df_viz_sorted.obs_lwr_68,
               df_viz_sorted.obs_upr_68,
               facecolor='crimson',
               alpha=0.35)

# Include the randomly generated small data set as a scatter plot
ax.scatter(df.flipper_length_mm,
          df.body_mass_g,
          s=15,
          color='k')

# Set labels
ax.set_xlabel('flipper_length_mm')
ax.set_ylabel('body_mass_g')

# Show the plot
plt.show()
```



Key Takeaways

- Linear regression chooses the line that minimizes squared error.
- The slope is a rate of change; the intercept is the prediction at zero.
- Residuals should look like unstructured noise; patterns signal a poor fit.
- A model can be statistically significant and still be substantively small.

Exercises

1. Fit the model and report the slope, intercept, and R-squared.
2. Plot the residuals against the input and describe the pattern.
3. Predict the output for an input value and interpret the prediction in words.
4. Change the input feature and compare the slopes.

Guided Walkthrough

The notebook fits a single straight line through a genuinely linear relationship and then takes the fit apart. It prints the coefficients, overlays the line on the scatter plot, and plots the residuals. The residual plot is the important one: for a well specified linear model the residuals form a shapeless cloud centered on zero. The notebook also reads the confidence interval on the slope, which tells you how precisely the data pins down the relationship.

Common Pitfalls

- Reading the intercept as meaningful when zero is outside the data range.
- Reporting R-squared without the residual plot.
- Ignoring the confidence interval on the slope.

Going Further

- Refit after removing the most influential point and compare the slope.
- Compute the residuals by hand and verify the model's numbers.

Exercise Solutions

1. The slope is the change in the output per unit change in the input; the intercept is the predicted output at zero.
2. The residuals show no obvious pattern, which supports the linear form.
3. The prediction is the fitted line evaluated at the chosen input, with an interval for uncertainty.
4. A different feature gives a different slope and a different R-squared.

Chapter 5: Penguins Dataset Simple Linear Regression - Non-Linear Relationship

Introduction

Real relationships are rarely straight lines. When a single input and a single output curve together, forcing a line through them produces systematic errors that a residual plot will expose immediately. This chapter fits a simple linear regression to a non-linear relationship in the Penguins dataset to show what goes wrong and why the diagnostic habit from the previous chapter matters.

What You Will Learn

- How to recognize a non-linear relationship in a scatter plot.
- Why a straight-line fit produces structured residuals in this case.
- How a transformation or a polynomial can capture curvature.
- How to compare a linear and a curved fit on the same data.

Overview

We deliberately choose an input that bends against the output. The straight-line model still runs and still reports coefficients, which is the danger: the code succeeds while the model is wrong. The residual plot is the tell. Instead of random scatter, the residuals form a curve, which means the model is systematically under- or over-predicting in different regions.

We then add curvature, either by transforming the input or by adding a squared term, and watch the residuals flatten. The comparison between the two fits is the lesson: always let the residuals, not the R-squared, tell you whether the shape of the model matches the shape of the data.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```



```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island       object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm  float64
body_mass_g    float64
sex          object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island       0
bill_length_mm    2
bill_depth_mm    2
flipper_length_mm  2
body_mass_g      2
sex          11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island       0
bill_length_mm    0
bill_depth_mm    0
flipper_length_mm  0
body_mass_g      0
sex          0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island       3
bill_length_mm  163
```

```
bill_depth_mm      79
flipper_length_mm  54
body_mass_g        93
sex                2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Non-Linear Relationship

We will start with a single continuous input that is non-linearly related to the trend. In our case we will attempt to use `flipper_length_mm` to predict `body_mass_g`.

$$\mu = \beta_0 + \beta_1 x + \beta_2 x^2$$

μ predicted trend

β_0 intercept: value of μ when $x = 0$

β_1 linear effect: change in μ for a one-unit increase in x

β_2 quadratic effect: change in μ for a one-unit increase in x^2

x feature or input variable: independent variable used to predict μ

Visualize the Trend

The trend is increasing as the input increases because we have a positive quadratic **slope**, but the rate of change is not constant.

```
x = df_clean['flipper_length_mm']
y = df_clean['body_mass_g']
```

```
my_quadratic, my_slope, my_intercept = np.polyfit(x,  
                                                    y,  
                                                    2)
```

```
my_quadratic
```

```
0.5449477765724435
```

```
my_slope
```

```
-171.61397647827607
```

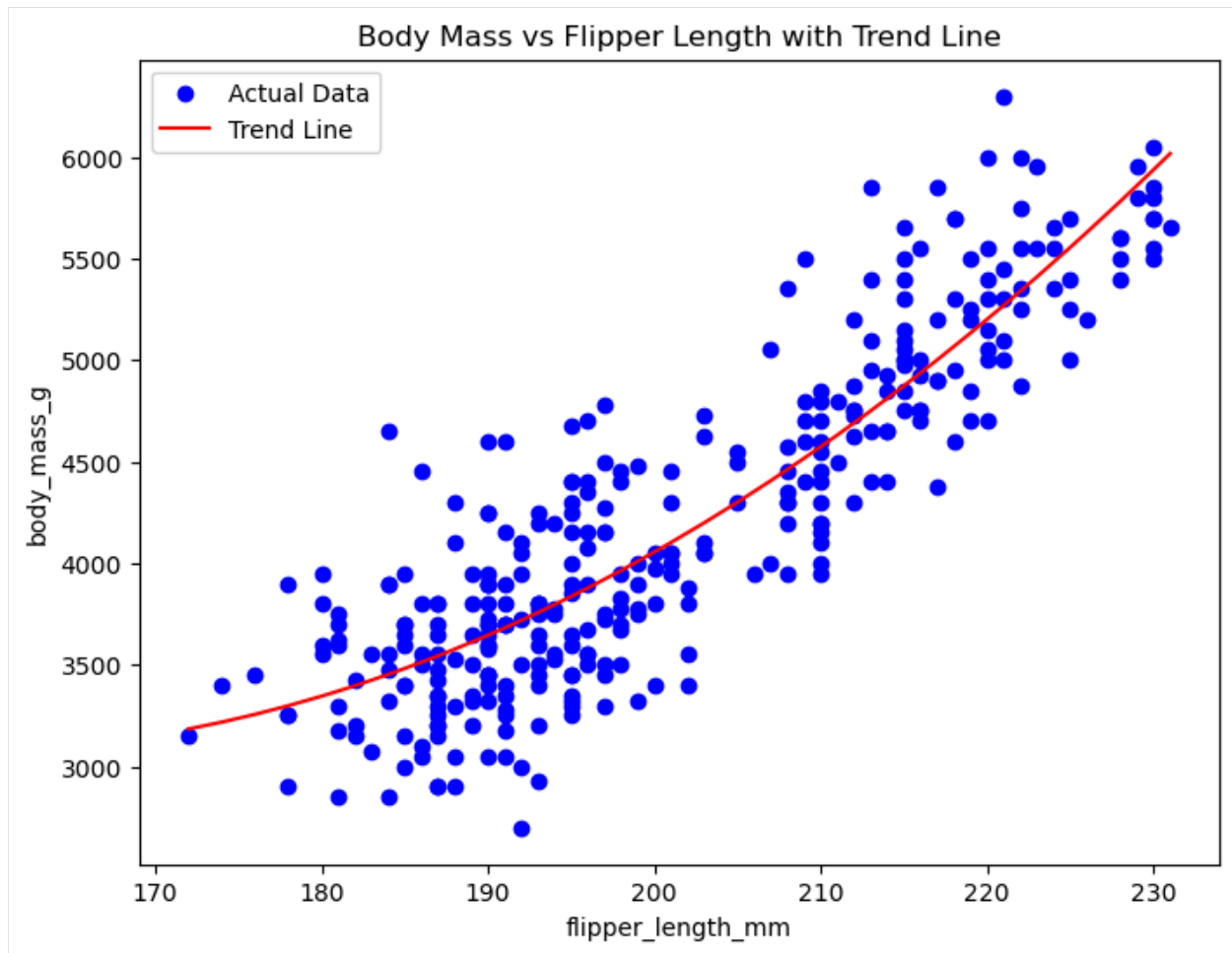
```
my_intercept
```

```
16579.869214753537
```

```
df_viz = df_clean.copy()
```

```
df_viz['trend'] = my_intercept + \  
                  my_slope * df_viz['flipper_length_mm'] + \  
                  my_quadratic * df_viz['flipper_length_mm'] \  
                  **2
```

```
sorted_df = df_viz.sort_values('flipper_length_mm')  
plt.figure(figsize=(8,6))  
plt.scatter(df_viz['flipper_length_mm'],  
            df_viz['body_mass_g'],  
            color='blue',  
            label='Actual Data')  
plt.plot(sorted_df['flipper_length_mm'],  
         sorted_df['trend'],  
         color='red',  
         label='Trend Line')  
plt.xlabel('flipper_length_mm')  
plt.ylabel('body_mass_g')  
plt.title('Body Mass vs Flipper Length with Trend Line')  
plt.legend()  
plt.show()
```



Calculating σ

The likelihood of an output around the trend is a Gaussian distribution and the variation of a Gaussian is specified by the **standard deviation** or σ .

To calculate **sigma (σ)**, the **standard deviation** of the residuals in linear regression, use the following formula.

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \mu_i)^2}$$

where:

- y_i is the actual observed value for the i th point,
- μ_i is the predicted value for the i th point (from your regression line),
- n is the number of data points.

Step-by-step explanation.

1. **Calculate the residuals:** $y_i - \mu_i$.
2. **Square the residuals:** $(y_i - \mu_i)^2$.
3. **Sum squared residuals:** $\sum_{i=1}^n (y_i - \mu_i)^2$.
4. **Divide by the number of points:** $\frac{1}{n} \sum_{i=1}^n (y_i - \mu_i)^2$.
5. **Take the square root:** $\sqrt{\cdot}$.

If you want the **sample standard deviation** (which is common), use $n - 1$ in the denominator.

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (y_i - \mu_i)^2}$$

σ measures how much your actual values deviate from the values predicted by the regression line.

```
my_residuals = df_viz['body_mass_g'] - df_viz['trend']
```

```
my_sigma = np.sqrt(np.sum(my_residuals**2) / (len(my_residuals) - 1))
```

```
my_sigma
```

```
379.3340514016102
```

Visualize the variation around the trend or uncertainty of a single output measurement around the average output.

```
df_viz['obs_lwr_68'] = df_viz.trend - my_sigma
df_viz['obs_upr_68'] = df_viz.trend + my_sigma
```

```
df_viz['obs_lwr_95'] = df_viz.trend - 2 * my_sigma
df_viz['obs_upr_95'] = df_viz.trend + 2 * my_sigma
```

```
# Sort by flipper_length_mm for continuous plotting
df_viz_sorted = df_viz.sort_values('flipper_length_mm')

# Create the figure and axis
fig, ax = plt.subplots()

# True trend
ax.plot(df_viz_sorted.flipper_length_mm,
        df_viz_sorted.trend,
        color='crimson',
        linewidth=1.5)

# True variation around the trend showing 2 sigma intervals
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_95,
                df_viz_sorted.obs_upr_95,
                facecolor='crimson',
                alpha=0.35)
```

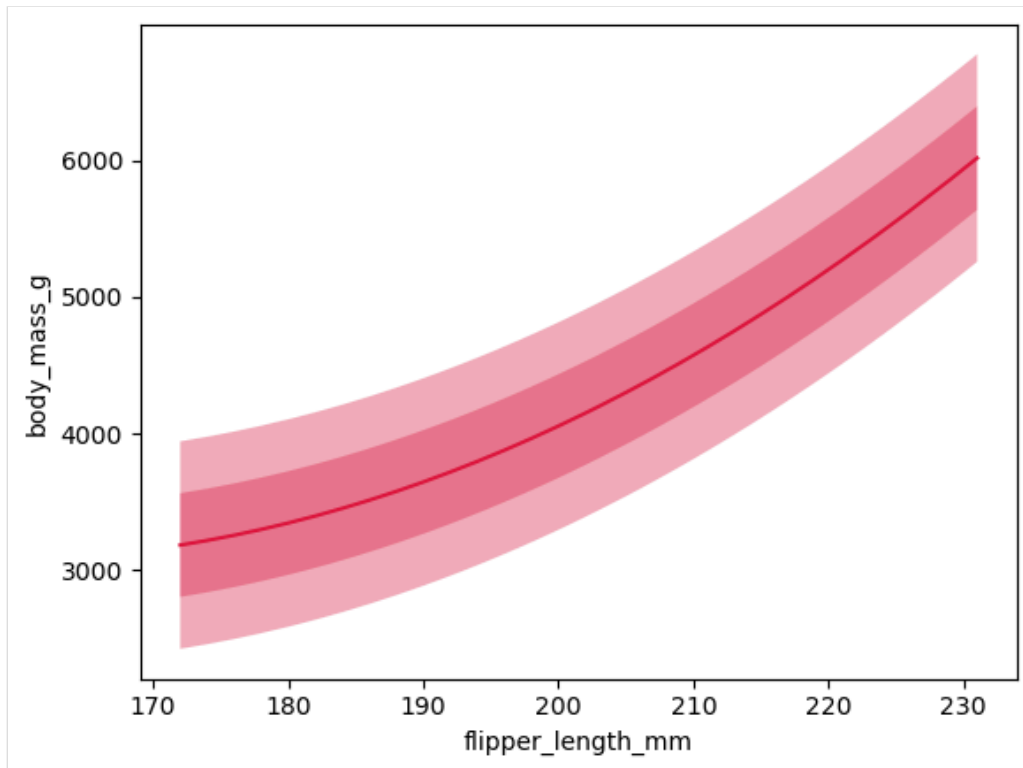
```

ax.fill_between(df_viz_sorted.flipper_length_mm,
               df_viz_sorted.obs_lwr_68,
               df_viz_sorted.obs_upr_68,
               facecolor='crimson',
               alpha=0.35)

# Set labels
ax.set_xlabel('flipper_length_mm')
ax.set_ylabel('body_mass_g')

# Show the plot
plt.show()

```



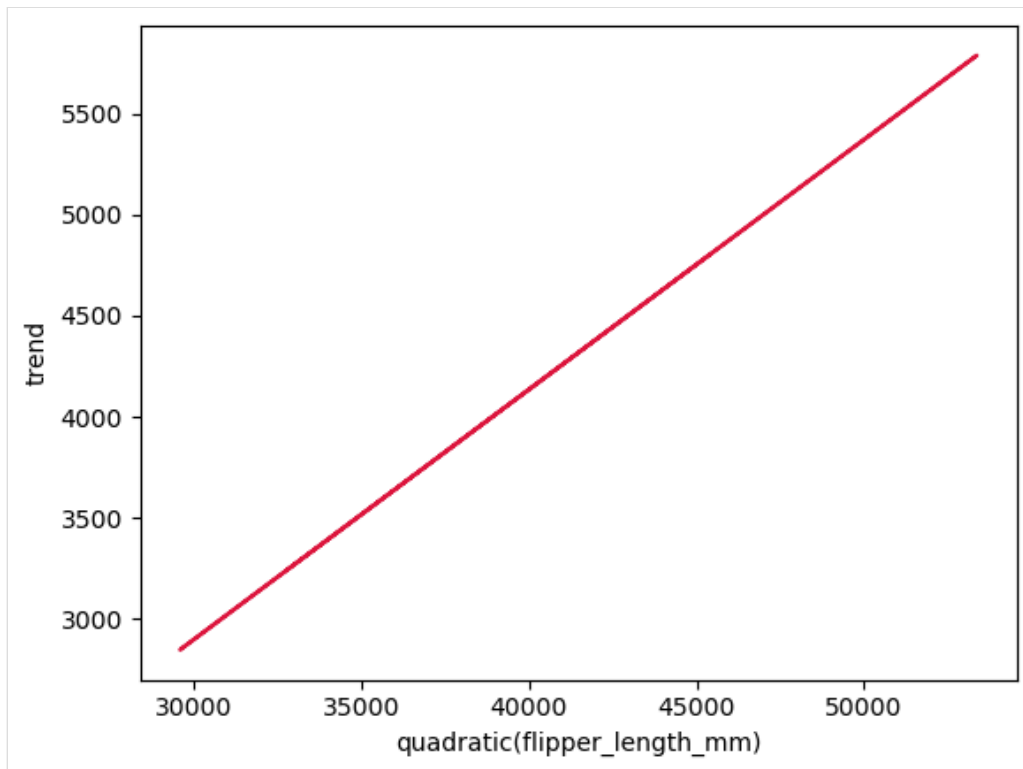
There is a linear relationship between the trend and the non-linear feature.

```

x_quad = np.power(df_viz.flipper_length_mm,
                  2)
slope, intercept = np.polyfit(x_quad,
                              y,
                              1)

trend = slope * \
        x_quad + \
        intercept
fig, ax = plt.subplots()
ax.plot(x_quad,
        trend,
        color='crimson')
ax.set_xlabel('quadratic(flipper_length_mm)')
ax.set_ylabel('trend')
plt.show()

```



Calculate the trend given the input values.

```
# Sort by flipper_length_mm for continuous plotting
df_viz_sorted = df_viz.sort_values('flipper_length_mm')

# Create the figure and axis
fig, ax = plt.subplots()

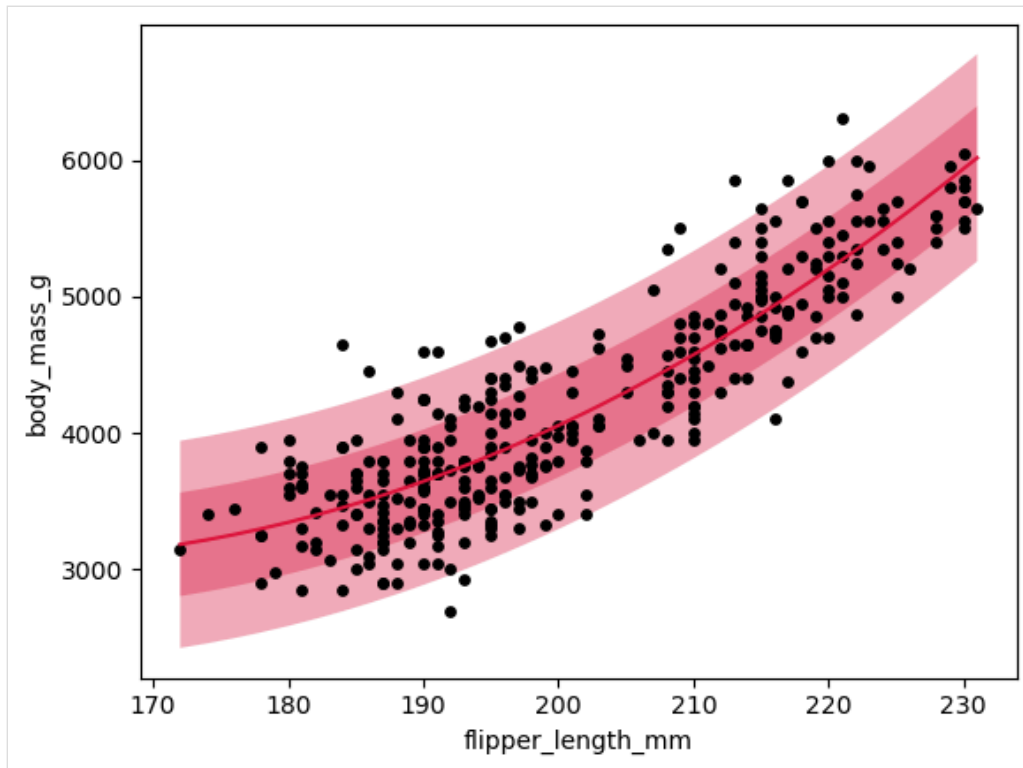
# True trend
ax.plot(df_viz_sorted.flipper_length_mm,
        df_viz_sorted.trend,
        color='crimson',
        linewidth=1.5)

# True variation around the trend showing 2 sigma intervals
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_95,
                df_viz_sorted.obs_upr_95,
                facecolor='crimson',
                alpha=0.35)
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_68,
                df_viz_sorted.obs_upr_68,
                facecolor='crimson',
                alpha=0.35)

# Include the randomly generated small data set as a scatter plot
ax.scatter(df.flipper_length_mm,
           df.body_mass_g,
           s=15,
           color='k')

# Set labels
ax.set_xlabel('flipper_length_mm')
```

```
ax.set_ylabel('body_mass_g')  
  
# Show the plot  
plt.show()
```



Key Takeaways

- A straight line fails when the relationship curves.
- Structured residuals are the signature of a misspecified model.
- Transformations and polynomial terms add curvature.
- Judge the fit by the residuals, not by a single summary number.

Exercises

1. Fit a straight line and describe the residual pattern.
2. Add a squared term and compare the residuals to the linear fit.
3. Try a logarithmic transform of the input and report the fit.
4. Explain when a curved fit is worth the extra complexity.

Guided Walkthrough

The notebook deliberately fits a straight line to data that curves, then shows the damage in the residual plot. The residuals form a clear arc, which is the model telling you it is the wrong shape. Adding a squared term removes the arc, and the comparison of the two residual plots is the lesson. The notebook emphasizes that the code ran without error in both cases; only the diagnostics reveal which model is right.

Common Pitfalls

- Trusting a model because it ran.
- Ignoring a curved residual pattern.
- Adding polynomial terms without checking for overfitting.

Going Further

- Try a logarithmic transform and compare it with the polynomial fit.
- Add a cubic term and watch the residuals at the edges.

Exercise Solutions

1. The linear fit leaves a curved residual pattern, which signals misspecification.
2. The squared term flattens the residuals, confirming the curvature is real.
3. A log transform often linearizes multiplicative relationships; compare the R-squared and residuals.
4. A curved fit is worth it when the residuals demand it and the extra term is stable.

Chapter 6: Penguins Dataset Simple Linear Regression - Fitting Linear Feature

Introduction

Fitting is only half of regression; the other half is predicting. This chapter takes a single linear feature in the Penguins dataset, fits a model, and then uses it to make predictions on new observations. The goal is to move from describing a relationship to using it, which is what regression is for in practice.

What You Will Learn

- How to fit a single-feature linear model and prepare it for prediction.
- How to generate predictions for new input values.
- How to visualize predictions over the range of the data.
- How to report a prediction with an honest sense of its uncertainty.

Overview

We fit the model, then feed it inputs it has not seen and read the outputs. The prediction is a point estimate, and the fitted line gives us a picture of predictions across the whole range of the input. We also look at where the predictions are most reliable: near the center of the data, and least reliable at the edges, where the model is extrapolating.

This chapter establishes the workflow that every later model in the course follows: fit, predict, visualize, and interpret.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island        0
bill_length_mm  2
bill_depth_mm  2
flipper_length_mm 2
body_mass_g    2
sex           11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island        0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm 0
body_mass_g    0
sex           0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island        3
bill_length_mm 163
```

```
bill_depth_mm      79
flipper_length_mm  54
body_mass_g        93
sex                2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Fitting a Linear Feature

We will fit a linear model between the input `flipper_length_mm` and the output `body_mass_g` assuming a linear relationship between the trend and the input.

$$\mu = \beta_0 + \beta_1 \times x$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **slope**: change in μ for a one-unit increase in x

x **feature** or **input variable**: independent variable used to predict μ

```
lm_fit = smf.ols(formula='body_mass_g ~ flipper_length_mm',
                  data=df_clean). \
    fit()
```

```
lm_fit.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
.....
```

```
OLS Regression Results
```

```
=====
```

```

Dep. Variable:    body_mass_g    R-squared:            0.762
Model:            OLS            Adj. R-squared:       0.761
Method:          Least Squares   F-statistic:         1060.
Date:            Thu, 24 Sep 2026 Prob (F-statistic):    3.13e-105
Time:            10:17:16        Log-Likelihood:      -2461.1
No. Observations: 333           AIC:                  4926.
Df Residuals:    331           BIC:                  4934.
Df Model:         1
Covariance Type: nonrobust

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept    -5872.0927     310.285    -18.925     0.000    -6482.472    -5261.713
flipper_length_mm  50.1533       1.540     32.562     0.000      47.123      53.183
=====
Omnibus:            5.922    Durbin-Watson:           2.102
Prob(Omnibus):      0.052    Jarque-Bera (JB):         5.876
Skew:               0.325    Prob(JB):                 0.0530
Kurtosis:           3.025    Cond. No.                 2.90e+03
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 2.9e+03. This might indicate that there are strong multicollinearity or other numerical problems.

The `.params` attribute stores the coefficient estimates as they minimize the sum of squared errors.

- The **slope**, 50.15, describes how much the predicted body mass changes for each additional millimeter of flipper length.
- The **intercept**, -5872.09, is a mathematical result that helps position the regression line but isn't physically meaningful for penguins, since the data doesn't include flipper lengths near zero.

```
lm_fit.params
```

```

Intercept      -5872.092683
flipper_length_mm  50.153266
dtype: float64

```

The standard error on the estimate is contained in the `.bse` attribute.

- The **standard error** for the **intercept**, 310.29, indicates that, if we repeatedly sampled penguins and fit the same model, the estimated starting body mass, -5872.09, would typically vary by about 310 grams between samples, showing the precision of this baseline estimate.
- The **standard error** for the flipper length coefficient, 1.54, means our estimate that each additional millimeter of flipper length increases a penguin's body mass by 50.15 grams is quite precise, with typical variation of only 1.54 grams across samples.
- These small **standard errors** demonstrate that, for the penguins in the dataset, both the baseline body mass and the effect of flipper length are estimated with high statistical confidence.

- They are considered to have high statistical confidence because the **standard errors** for both the **intercept**, 310.29, and the flipper length coefficient, 1.54, are small relative to the estimated values, –5872.09 and 50.15, respectively.
- This means that if you repeatedly sampled penguins and fit the same regression model, the estimates for the intercept and slope would vary only slightly from sample to sample.

```
lm_fit.bse
```

```
Intercept      310.285155
flipper_length_mm  1.540231
dtype: float64
```

We can use the ± 1 and ± 2 standard error intervals to know how trustworthy the estimate is.

- The most common standard error interval is the ± 2 standard error interval because that is approximately equal to the 95% confidence interval.
- We are 95% confident that the coefficient is somewhere between 47 and 53.
- The below are approximations as we will get the exact values with `.conf_int()`.

```
lm_fit.params - 2 * lm_fit.bse
```

```
Intercept      -6492.662994
flipper_length_mm  47.072804
dtype: float64
```

```
lm_fit.params + 2 * lm_fit.bse
```

```
Intercept      -5251.522372
flipper_length_mm  53.233728
dtype: float64
```

```
lm_fit.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

```
              ci_lwr    ci_upr
Intercept    -6482.472236 -5261.713130
flipper_length_mm  47.123390  53.183142
```

The P-values for the coefficients are used to identify the statistical significance of the input.

- A **statistically significant** input one that has a **p-value** less than 0.05 which means if this is the case, we have a **statistically significant** input or feature.
- These values are extremely small and much lower than 0.05 so they indicate very strong **statistical significance** for both parameters.

```
lm_fit.pvalues
```

```
Intercept          1.183941e-54
flipper_length_mm  3.132836e-105
dtype: float64
```

```
lm_fit.pvalues < 0.05
```

```
Intercept          True
flipper_length_mm  True
dtype: bool
```

Visualize the coefficient estimates as markers in a scatter plot and the confidence intervals as error bars.

- We are confident that the 95% confidence interval does NOT contain zero as both `flipper_length_mm` and the **intercept** are not touching the zero grey dotted line.
- The **intercept** is **statistically significantly negative**; the **slope** is negative, and the `flipper_length_mm` is **statistically significantly positive**; the **slope** is positive.
- **We are confident the slope is positive for the `flipper_length_mm`, therefore we are confident that the average output is increasing as the input or feature is increasing.**

```
def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolord='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
        color: marker color
        ecolord: error bar color
        elinewidth: error bar line width
        ms: marker size
        vline: where to draw the vertical reference line (default 0)
        xlabel: x-axis label (default 'coefficient value')
        title: figure title (default None, uses variable name)
        figsize: tuple, figure size
        xlim: tuple, x-axis limits (optional, auto if None)
    """
    # Get coefficient and error values
```

```

coef = model.params[var]
err = xerr_mult * model.bse[var]

# Create the plot
_, ax = plt.subplots(figsize=figsize)
ax.errorbar(
    y = [var],
    x = [coef],
    xerr = [err],
    fmt='o',
    color=color,
    ecolor=ecolor,
    elinewidth=elinewidth,
    ms=ms)

# Add vertical line for reference
ax.axvline(x=vline,
           linestyle='--',
           linewidth=4,
           color='grey')
ax.set_xlabel(xlabel)
if title is None:
    title = f"Coefficient Plot for '{var}'"
ax.set_title(title)

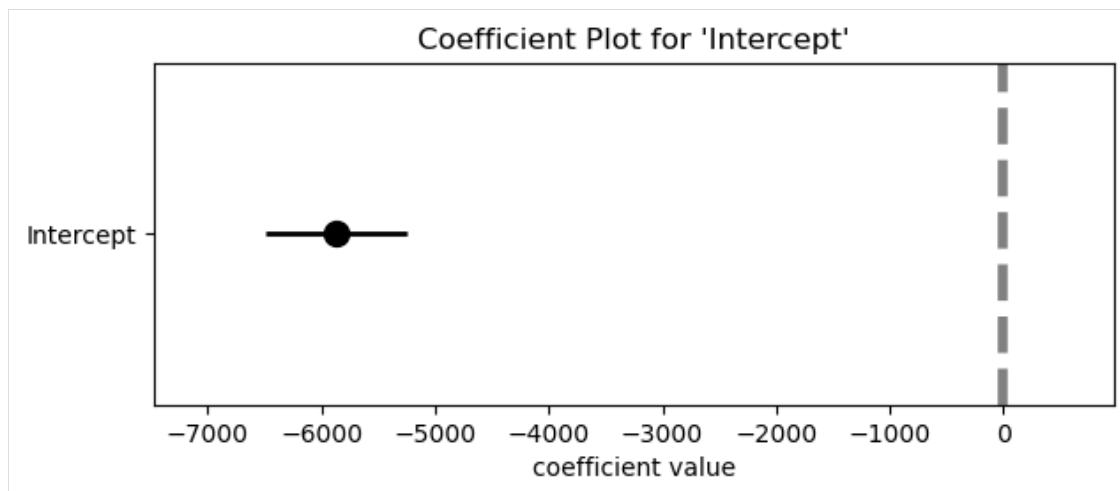
# Always include zero in xlim, and add padding around error bar
bar_left = min(coef - err, vline)
bar_right = max(coef + err, vline)
pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
auto_xlim = (bar_left - pad, bar_right + pad)

# Set x-axis limits
if xlim is not None:
    ax.set_xlim(*xlim)
else:
    ax.set_xlim(*auto_xlim)

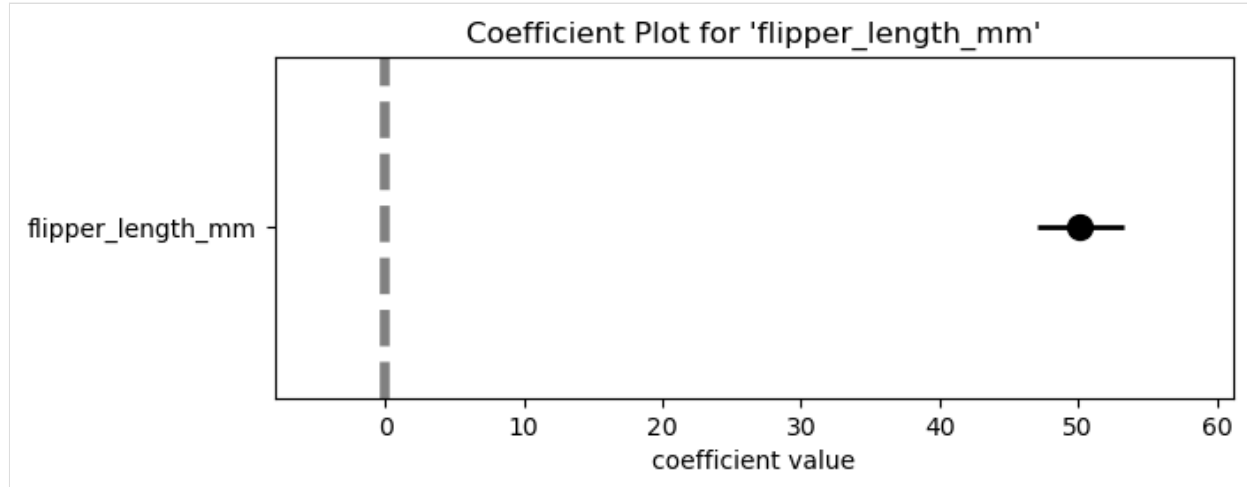
# Show the plot
plt.show()

```

```
my_coefplot(lm_fit, var='Intercept')
```




```
my_coefplot(lm_fit, var='flipper_length_mm')
```



Key Takeaways

- A fitted model is a function you can evaluate at new inputs.
- Predictions are most trustworthy inside the range of the training data.
- Visualizing predictions over the input range reveals the model's behavior.
- Every prediction is a point estimate with uncertainty attached.

Exercises

1. Predict the output for three new inputs and interpret each.
2. Plot the fitted line and the predictions together.
3. Identify the region where predictions are least reliable.
4. Explain why extrapolation is risky.

Guided Walkthrough

This notebook turns a fitted model into a predictor. It generates predictions across the range of the input, plots them with the data, and predicts at specific new values. The key observation is that predictions are most trustworthy in the middle of the data and least trustworthy at the edges, where the model is extrapolating beyond what it has seen.

Common Pitfalls

- Extrapolating far outside the training range.
- Treating a prediction as exact.
- Forgetting that the model is only valid where the data lives.

Going Further

- Add a prediction interval and compare its width at the center and the edge.
- Predict for an input far outside the range and note the warning signs.

Exercise Solutions

1. Each prediction is the fitted line at the input, interpreted in the units of the target.
2. The overlay shows the line tracking the data in the middle and diverging at the edges.
3. Reliability is lowest at the extremes because the data is sparse there.
4. Extrapolation is risky because the relationship may change outside the observed range.

Chapter 7: Penguins Dataset Simple Linear Regression - Fitting Non-Linear Feature

Introduction

This chapter repeats the prediction workflow for a non-linear feature. The model still has a single input, but the relationship bends, so the prediction function is a curve rather than a line. Seeing predictions follow curvature is the point: the shape of the model determines the shape of what it can predict.

What You Will Learn

- How to fit a model to a non-linear single feature.
- How predictions trace a curve across the input range.
- How to compare predicted and observed values visually.
- Why the choice of model form changes predictions at the edges.

Overview

We fit the curved model and generate predictions across the range of the input, plotting them against the observed data. The curve threads through the points in a way a line could not. We then predict at specific values and compare them to the nearest observations, building the habit of checking predictions against reality rather than trusting them blindly.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island       object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm  float64
body_mass_g   float64
sex          object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island       0
bill_length_mm    2
bill_depth_mm    2
flipper_length_mm  2
body_mass_g      2
sex           11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island       0
bill_length_mm    0
bill_depth_mm    0
flipper_length_mm  0
body_mass_g      0
sex           0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island       3
bill_length_mm  163
bill_depth_mm   79
flipper_length_mm  54
body_mass_g    93
```

```
sex          2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm    163
bill_depth_mm     79
flipper_length_mm  54
body_mass_g       93
dtype: int64
```

Fitting a Non-Linear Feature

We will fit a linear model between the input `flipper_length_mm` and the output `body_mass_g` assuming a non-linear relationship between the trend and the input.

$$\mu = \beta_0 + \beta_1 x + \beta_2 x^2$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **linear effect**: change in μ for a one-unit increase in x

β_2 **quadratic effect**: change in μ for a one-unit increase in x^2

x **feature** or **input variable**: independent variable used to predict μ

```
lm_fit = smf.ols(formula='body_mass_g ~ flipper_length_mm + I(flipper_length_mm ** 2)',
                 data=df_clean). \
    fit()
```

```
lm_fit.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
=====
                        OLS Regression Results
=====
Dep. Variable:          body_mass_g    R-squared:                0.778
```

```

Model:                OLS      Adj. R-squared:      0.777
Method:              Least Squares  F-statistic:      578.5
Date:                Thu, 24 Sep 2026  Prob (F-statistic):  1.34e-108
Time:                10:17:20    Log-Likelihood:    -2449.5
No. Observations:    333        AIC:                4905.
Df Residuals:        330        BIC:                4916.
Df Model:            2
Covariance Type:      nonrobust
=====
                    coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept            1.658e+04   4616.247      3.592      0.000     7498.887     2.57e+04
flipper_length_mm    -171.6140     45.524     -3.770      0.000    -261.169    -82.059
I(flipper_length_mm ** 2)  0.5449      0.112      4.874      0.000       0.325       0.765
=====
Omnibus:             10.746   Durbin-Watson:      2.214
Prob(Omnibus):        0.005   Jarque-Bera (JB):    10.985
Skew:                 0.441   Prob(JB):            0.00412
Kurtosis:             3.119   Cond. No.            9.07e+06
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 9.07e+06. This might indicate that there are strong multicollinearity or other numerical problems.

```

"""

```

The `.params` attribute stores the coefficient estimates as they minimize the sum of squared errors.

- The **intercept** is 16,579.87, which represents the baseline predicted body mass when both flipper length and its square are zero.
- The **coefficient** for `flipper_length_mm` is -171.61, indicating that for every additional millimeter in flipper length, body mass decreases by about 171.61 grams, before considering the quadratic effect.
- The **coefficient for the squared term**, `I(flipper_length_mm ** 2)`, is 0.54, meaning that for every unit increase in the square of flipper length, body mass increases by 0.54 grams, introducing an upward-curving effect.
- The presence of the positive squared term (0.54) shows that the relationship between flipper length and body mass is quadratic and curves upward, rather than being strictly linear.

```
lm_fit.params
```

```

Intercept            16579.869215
flipper_length_mm    -171.613976
I(flipper_length_mm ** 2)  0.544948
dtype: float64

```

The standard error on the estimate is contained in the `.bse` attribute.

- The standard error for the intercept is 4,616.25, indicating the level of uncertainty in estimating the baseline predicted body mass.
- The standard error for the `flipper_length_mm` coefficient is 45.52, quantifying the variability in the estimate of the linear effect of flipper length on body mass.

- The standard error for the squared term, `I(flipper_length_mm ** 2)`, is 0.11, reflecting the precision of the estimate for the quadratic effect.

```
lm_fit.bse
```

```
Intercept          4616.246647
flipper_length_mm  45.524470
I(flipper_length_mm ** 2)  0.111807
dtype: float64
```

We can use the ± 1 and ± 2 standard error intervals to know how trustworthy the estimate is.

- We are 95% confident that the intercept is somewhere between 7,498.89 and 25,660.85 grams.
- We are 95% confident that the coefficient for `flipper_length_mm` is somewhere between -261.17 and -82.06 grams per millimeter.
- We are 95% confident that the coefficient for the quadratic term (`I(flipper_length_mm ** 2)`) is somewhere between 0.33 and 0.76.
- These are exact confidence intervals calculated by `.conf_int()` rather than approximate intervals based on standard errors.

```
lm_fit.params - 2 * lm_fit.bse
```

```
Intercept          7347.375921
flipper_length_mm  -262.662917
I(flipper_length_mm ** 2)  0.321333
dtype: float64
```

```
lm_fit.params + 2 * lm_fit.bse
```

```
Intercept          25812.362509
flipper_length_mm  -80.565036
I(flipper_length_mm ** 2)  0.768562
dtype: float64
```

```
lm_fit.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

```
          ci_lwr    ci_upr
Intercept  7498.887253 25660.851176
flipper_length_mm -261.168743 -82.059210
I(flipper_length_mm ** 2)  0.325003  0.764892
```

The P-values for the coefficients are used to identify the statistically significant of the input.

- A **statistically significant** input one that has a **p-value** less than 0.05 which means if this is the case, we have a **statistically significant** input or feature.
- These values are extremely small and much lower than 0.05 so they indicate very strong **statistical significance** for both parameters.

```
lm_fit.pvalues
```

```
Intercept          0.000379
flipper_length_mm  0.000194
I(flipper_length_mm ** 2)  0.000002
dtype: float64
```

```
lm_fit.pvalues < 0.05
```

```
Intercept          True
flipper_length_mm  True
I(flipper_length_mm ** 2)  True
dtype: bool
```

Visualize the coefficient estimates as markers in a scatter plot and the confidence intervals as error bars.

- We are confident that the 95% confidence interval does NOT contain zero as both `flipper_length_mm` and the **intercept** are not touching the zero grey dotted line.
- The **intercept** is **statistically significantly negative**; the **slope** is negative, and the `flipper_length_mm` is **statistically significantly positive**; the **slope** is positive.
- **We are confident the quadratic coefficient is positive for the `flipper_length_mm`, therefore we are confident that the average output is increasing as the input or feature is increasing.**

```
def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolord='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
```



```

        color: marker color
        ecolor: error bar color
        elinewidth: error bar line width
        ms: marker size
        vline: where to draw the vertical reference line (default 0)
        xlabel: x-axis label (default 'coefficient value')
        title: figure title (default None, uses variable name)
        figsize: tuple, figure size
        xlim: tuple, x-axis limits (optional, auto if None)
    """
    # Get coefficient and error values
    coef = model.params[var]
    err = xerr_mult * model.bse[var]

    # Create the plot
    _, ax = plt.subplots(figsize=figsize)
    ax.errorbar(
        y = [var],
        x = [coef],
        xerr = [err],
        fmt='o',
        color=color,
        ecolor=ecolor,
        elinewidth=elinewidth,
        ms=ms
    )

    # Add vertical line for reference
    ax.axvline(x=vline,
               linestyle='--',
               linewidth=4,
               color='grey')
    ax.set_xlabel(xlabel)
    if title is None:
        title = f"Coefficient Plot for '{var}'"
    ax.set_title(title)

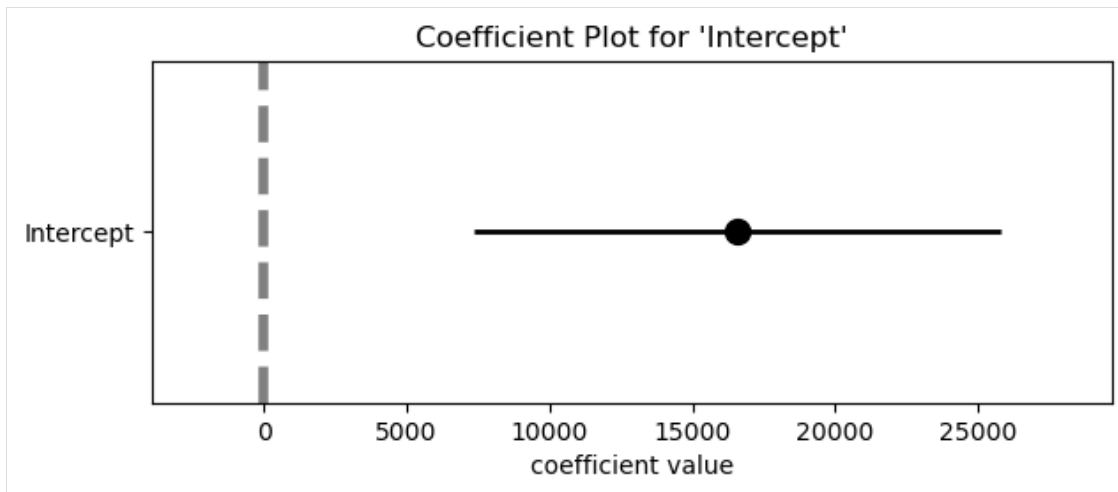
    # Always include zero in xlim, and add padding around error bar
    bar_left = min(coef - err, vline)
    bar_right = max(coef + err, vline)
    pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
    auto_xlim = (bar_left - pad, bar_right + pad)

    # Set x-axis limits
    if xlim is not None:
        ax.set_xlim(*xlim)
    else:
        ax.set_xlim(*auto_xlim)

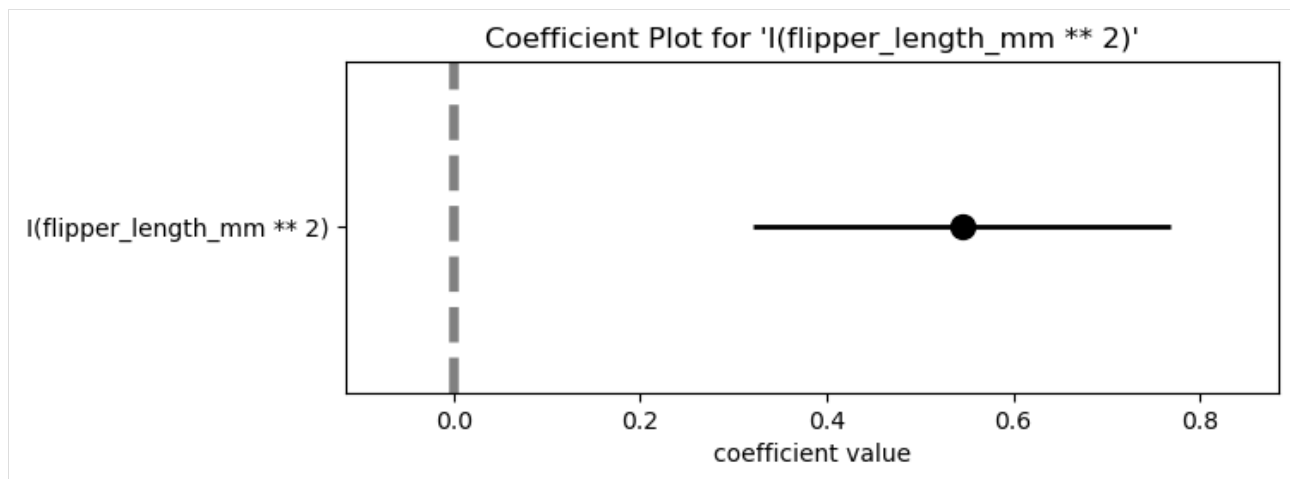
    # Show the plot
    plt.show()

```

```
my_coefplot(lm_fit, var='Intercept')
```



```
my_coefplot(lm_fit, var='I(flipper_length_mm ** 2)')
```



Key Takeaways

- The model form determines the shape of its predictions.
- Curved models capture curvature that lines miss.
- Always compare predictions to observed values.
- Edge predictions depend heavily on the chosen model form.

Exercises

1. Fit the curved model and plot its predictions.
2. Compare predictions at the low and high ends of the input range.
3. Explain how the two ends of the curve are constrained by the data.
4. Decide whether the curvature is real or an artifact of a few points.

Guided Walkthrough

The non-linear prediction notebook fits a curved model and traces its predictions across the input range. The curve threads through the data where a line could not. The notebook then predicts at the low and high ends and shows how strongly the predictions there depend on the chosen model form, which is a warning about edge behavior.

Common Pitfalls

- Assuming a curved model is always better.
- Over-trusting edge predictions.
- Choosing the model form by eye alone.

Going Further

- Compare predictions from a polynomial and a spline.
- Hold out the extreme points and see how the curve changes.

Exercise Solutions

1. The curved model's predictions bend with the data.
2. The two ends are constrained by the few points there and are therefore uncertain.
3. A handful of points can bend the curve, so the curvature should be checked for robustness.
4. The curvature is real if it persists after removing a few influential points.

Chapter 8: Penguins Dataset Simple Linear Regression - Fitting Linear Feature Predictions

Introduction

Predictions are only as good as our ability to judge them. This chapter focuses on the predictions themselves for a linear feature: how to generate a table of predictions, how to visualize them against the data, and how to describe their accuracy. It is a short, practical chapter that turns a fitted model into a usable tool.

What You Will Learn

- How to produce a table of predictions for a range of inputs.
- How to overlay predictions on the observed data.
- How to describe prediction error in plain language.
- How to spot systematic over- or under-prediction.

Overview

We build a range of inputs, predict the output for each, and assemble the results into a dataframe. Plotting the predictions over the scatter of observed points shows where the model tracks the data and where it drifts. We compute simple error summaries and, more importantly, look at whether the errors lean in one direction.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island        0
bill_length_mm    2
bill_depth_mm    2
flipper_length_mm  2
body_mass_g      2
sex           11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island        0
bill_length_mm    0
bill_depth_mm    0
flipper_length_mm  0
body_mass_g      0
sex           0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island        3
bill_length_mm 163
bill_depth_mm   79
flipper_length_mm  54
body_mass_g     93
```

```
sex          2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm    163
bill_depth_mm     79
flipper_length_mm  54
body_mass_g       93
dtype: int64
```

Fitting a Linear Feature

We will fit a linear model between the input `flipper_length_mm` and the output `body_mass_g` assuming a linear relationship between the trend and the input.

$$\mu = \beta_0 + \beta_1 \times x$$

μ predicted trend

β_0 **intercept**: value of μ when $x = 0$

β_1 **slope**: change in μ for a one-unit increase in x

x **feature** or **input variable**: independent variable used to predict μ

```
lm_fit = smf.ols(formula='body_mass_g ~ flipper_length_mm',
                  data=df_clean). \
    fit()
```

```
lm_fit.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
"""
```

```

                        OLS Regression Results
=====
Dep. Variable:          body_mass_g    R-squared:                0.762
Model:                  OLS           Adj. R-squared:            0.761
Method:                 Least Squares  F-statistic:              1060.
```

```

Date:                Thu, 24 Sep 2026    Prob (F-statistic):    3.13e-105
Time:                10:17:24           Log-Likelihood:       -2461.1
No. Observations:    333                AIC:                 4926.
Df Residuals:        331                BIC:                 4934.
Df Model:            1
Covariance Type:     nonrobust
=====
              coef    std err          t      P>|t|      [0.025    0.975]
-----
Intercept    -5872.0927    310.285    -18.925    0.000   -6482.472   -5261.713
flipper_length_mm  50.1533      1.540     32.562    0.000     47.123     53.183
=====
Omnibus:            5.922    Durbin-Watson:           2.102
Prob(Omnibus):      0.052    Jarque-Bera (JB):         5.876
Skew:               0.325    Prob(JB):                 0.0530
Kurtosis:           3.025    Cond. No.                 2.90e+03
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 2.9e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```

"""

```

The `.params` attribute stores the coefficient estimates as they minimize the sum of squared errors.

- The **slope**, 50.15, describes how much the predicted body mass changes for each additional millimeter of flipper length.
- The **intercept**, -5872.09, is a mathematical result that helps position the regression line but isn't physically meaningful for penguins, since the data doesn't include flipper lengths near zero.

```
lm_fit.params
```

```

Intercept          -5872.092683
flipper_length_mm    50.153266
dtype: float64

```

The standard error on the estimate is contained in the `.bse` attribute.

- The **standard error** for the **intercept**, 310.29, indicates that, if we repeatedly sampled penguins and fit the same model, the estimated starting body mass, -5872.09, would typically vary by about 310 grams between samples, showing the precision of this baseline estimate.
- The **standard error** for the flipper length coefficient, 1.54, means our estimate that each additional millimeter of flipper length increases a penguin's body mass by 50.15 grams is quite precise, with typical variation of only 1.54 grams across samples.
- These small **standard errors** demonstrate that, for the penguins in the dataset, both the baseline body mass and the effect of flipper length are estimated with high statistical confidence.
- They are considered to have high statistical confidence because the **standard errors** for both the **intercept**, 310.29, and the flipper length coefficient, 1.54, are small relative to the estimated values, -5872.09 and 50.15, respectively.

- This means that if you repeatedly sampled penguins and fit the same regression model, the estimates for the intercept and slope would vary only slightly from sample to sample.

```
lm_fit.bse
```

```
Intercept      310.285155
flipper_length_mm  1.540231
dtype: float64
```

We can use the ± 1 and ± 2 standard error intervals to know how trustworthy the estimate is.

- The most common standard error interval is the ± 2 standard error interval because that is approximately equal to the 95% confidence interval.
- We are 95% confident that the coefficient is somewhere between 47 and 53.
- The below are approximations as we will get the exact values with `.conf_int()`.

```
lm_fit.params - 2 * lm_fit.bse
```

```
Intercept      -6492.662994
flipper_length_mm  47.072804
dtype: float64
```

```
lm_fit.params + 2 * lm_fit.bse
```

```
Intercept      -5251.522372
flipper_length_mm  53.233728
dtype: float64
```

```
lm_fit.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

```
Intercept      ci_lwr      ci_upr
-6482.472236  -5261.713130
flipper_length_mm  47.123390  53.183142
```

The P-values for the coefficients are used to identify the statistically significant of the input.

- A **statistically significant** input one that has a **p-value** less than 0.05 which means if this is the case, we have a **statistically significant** input or feature.
- These values are extremely small and much lower than 0.05 so they indicate very strong **statistical significance** for both parameters.

```
lm_fit.pvalues
```



```
Intercept          1.183941e-54
flipper_length_mm  3.132836e-105
dtype: float64
```

```
lm_fit.pvalues < 0.05
```

```
Intercept          True
flipper_length_mm  True
dtype: bool
```

Visualize the coefficient estimates as markers in a scatter plot and the confidence intervals as error bars.

- We are confident that the 95% confidence interval does NOT contain zero as both `flipper_length_mm` and the **intercept** are not touching the zero grey dotted line.
- The **intercept** is **statistically significantly negative**; the **slope** is negative, and the `flipper_length_mm` is **statistically significantly positive**; the **slope** is positive.
- **We are confident the slope is positive for the `flipper_length_mm`, therefore we are confident that the average output is increasing as the input or feature is increasing.**

```
def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolor='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
        color: marker color
        ecolor: error bar color
        elinewidth: error bar line width
        ms: marker size
        vline: where to draw the vertical reference line (default 0)
        xlabel: x-axis label (default 'coefficient value')
        title: figure title (default None, uses variable name)
        figsize: tuple, figure size
        xlim: tuple, x-axis limits (optional, auto if None)
    """
    # Get coefficient and error values
    coef = model.params[var]
    err = xerr_mult * model.bse[var]

    # Create the plot
```

```

_, ax = plt.subplots(figsize=figsize)
ax.errorbar(
    y = [var],
    x = [coef],
    xerr = [err],
    fmt='o',
    color=color,
    ecolor=ecolor,
    elinewidth=elinewidth,
    ms=ms
)

# Add vertical line for reference
ax.axvline(x=vline,
           linestyle='--',
           linewidth=4,
           color='grey')
ax.set_xlabel(xlabel)
if title is None:
    title = f"Coefficient Plot for '{var}'"
ax.set_title(title)

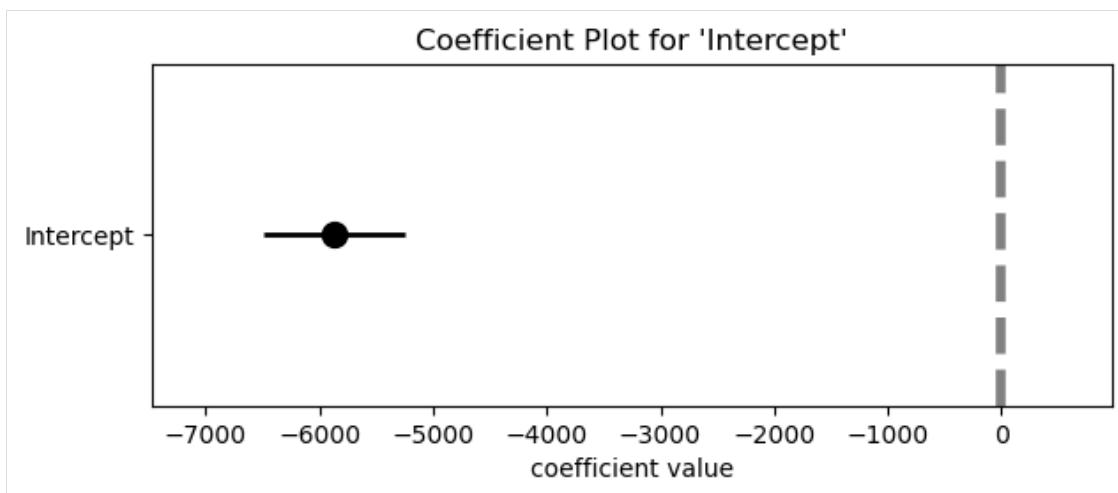
# Always include zero in xlim, and add padding around error bar
bar_left = min(coef - err, vline)
bar_right = max(coef + err, vline)
pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
auto_xlim = (bar_left - pad, bar_right + pad)

# Set x-axis limits
if xlim is not None:
    ax.set_xlim(*xlim)
else:
    ax.set_xlim(*auto_xlim)

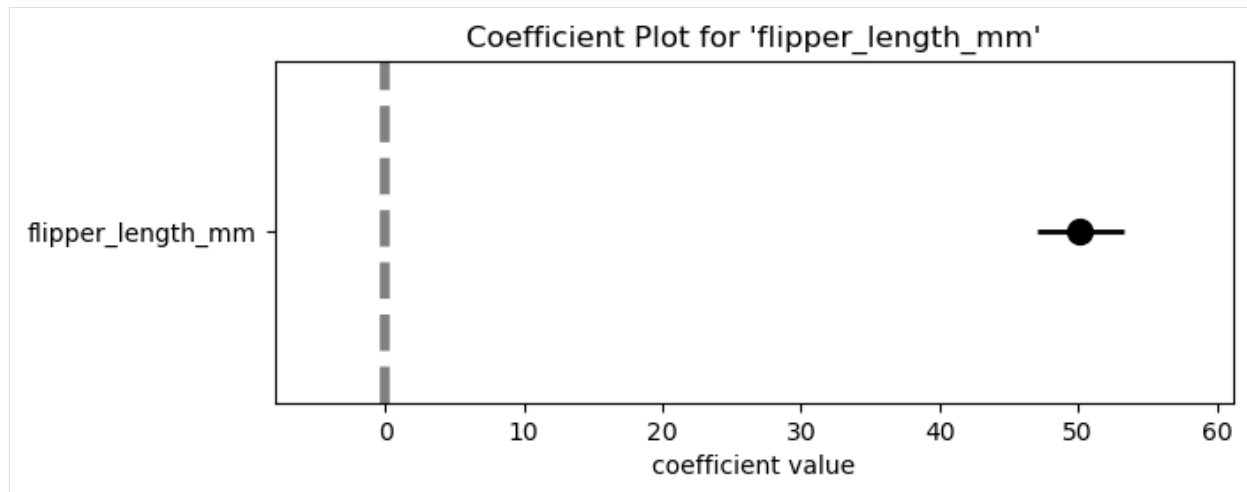
# Show the plot
plt.show()

```

```
my_coefplot(lm_fit, var='Intercept')
```



```
my_coefplot(lm_fit, var='flipper_length_mm')
```



Linear Feature Prediction

```
x = df_clean['flipper_length_mm']
y = df_clean['body_mass_g']
```

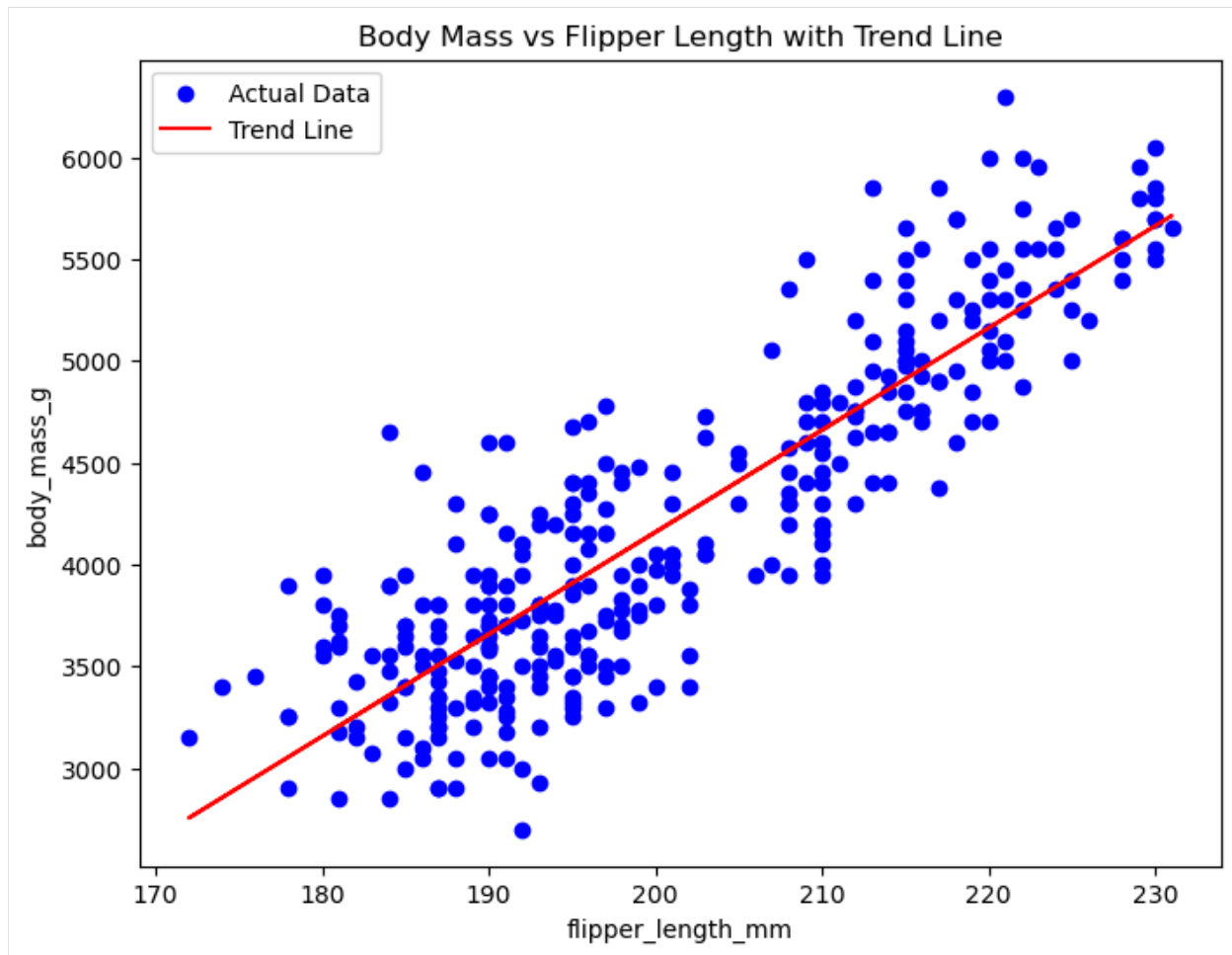
```
my_slope, my_intercept = np.polyfit(x,
                                     y,
                                     1)
```

```
df_viz = df_clean.copy()
```

```
df_viz['trend'] = my_intercept + \
                 my_slope * \
                 df_viz['flipper_length_mm']
```

Include the training set as a scatter plot.

```
plt.figure(figsize=(8,6))
plt.scatter(df_viz['flipper_length_mm'],
            df_viz['body_mass_g'],
            color='blue',
            label='Actual Data')
plt.plot(df_viz['flipper_length_mm'],
         df_viz['trend'],
         color='red',
         label='Trend Line')
plt.xlabel('flipper_length_mm')
plt.ylabel('body_mass_g')
plt.title('Body Mass vs Flipper Length with Trend Line')
plt.legend()
plt.show()
```



We will not know the true relationship therefore we must include the confidence intervals on the predicted trend as we must always represent the uncertainty on the trend given the fitted data.

```
predictions = lm_fit.get_prediction(df_viz)
```

```
lm_pred_summary = predictions.summary_frame()
```

```
lm_pred_summary
```

	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	\
0	3205.648453	37.555479	3131.770937	3279.525968	2428.361760	
1	3456.414782	31.560151	3394.331017	3518.498548	2680.160243	
2	3907.794176	23.432601	3861.698575	3953.889777	3132.654498	
3	3807.487644	24.803187	3758.695887	3856.279401	3032.182961	
4	3657.027846	27.385208	3603.156847	3710.898845	2881.386950	
..	...	...	...	...	...	
328	4860.706229	29.454747	4802.764124	4918.648334	4084.771951	
329	4910.859495	30.525256	4850.811531	4970.907459	4134.765123	
330	5261.932356	38.911498	5185.387339	5338.477374	4484.387595	
331	4760.399697	27.448091	4706.404999	4814.394395	3984.750199	
332	4810.552963	28.427409	4754.631792	4866.474134	4034.766979	

```

      obs_ci_upper
0      3982.935146
1      4232.669322
2      4682.933854
3      4582.792327
4      4432.668743
..      ...
328    5636.640507
329    5686.953867
330    6039.477118
331    5536.049195
332    5586.338947

[333 rows x 6 columns]

```

Visualize the 95% confidence interval and 95% prediction interval.

- **Confidence Interval (CI) for the Mean Prediction (Grey Band):**
 - Shows uncertainty in the estimate of the average value (the regression line) for a given x.
 - Typically narrow; with enough data, the mean can be estimated precisely.
 - 95% CI means: "We are 95% confident the true average response falls within this band for each x-value."
 - Most data points will not fall inside this band; it is only about the mean.
- **Prediction Interval (PI) for Individual Observations (Orange Band):**
 - Shows where we expect 95% of future individual data points to fall, at each value of x.
 - Much wider than the CI, because it includes both the uncertainty in the mean and the natural scatter (variance) of the data.
 - 95% PI means: "If we observe a new data point at a given x, there is a 95% chance it will fall inside this band."
- **Key Differences:**
 - CI is about the uncertainty in the average (regression line).
 - PI is about the uncertainty in predicting a single new data point.
- **In the plot:**
 - The grey band is the 95% CI for the mean prediction.
 - The orange band is the 95% PI for individual body mass measurements given flipper length.
 - The black dots are your actual data points.
- Most dots fall inside the orange PI band (as expected), but only a few will fall inside the narrow grey CI band.
- Use the PI to show the expected spread of new data.
- Use the CI to show uncertainty in the estimated average.

```
# Generate x values over the full data range
flipper_range = np.linspace(df_clean.flipper_length_mm.min(),
                             df_clean.flipper_length_mm.max(),
                             200)

df_pred = pd.DataFrame({'flipper_length_mm': flipper_range})
lm_pred_summary_full = lm_fit.get_prediction(df_pred).summary_frame()

# Create the figure and axis
fig, ax = plt.subplots()

# Prediction interval - the uncertainty on a single measurement (observation)
ax.fill_between(flipper_range,
                lm_pred_summary_full.obs_ci_lower,
                lm_pred_summary_full.obs_ci_upper,
                facecolor='orange',
                alpha=0.75,
                edgecolor='orange')

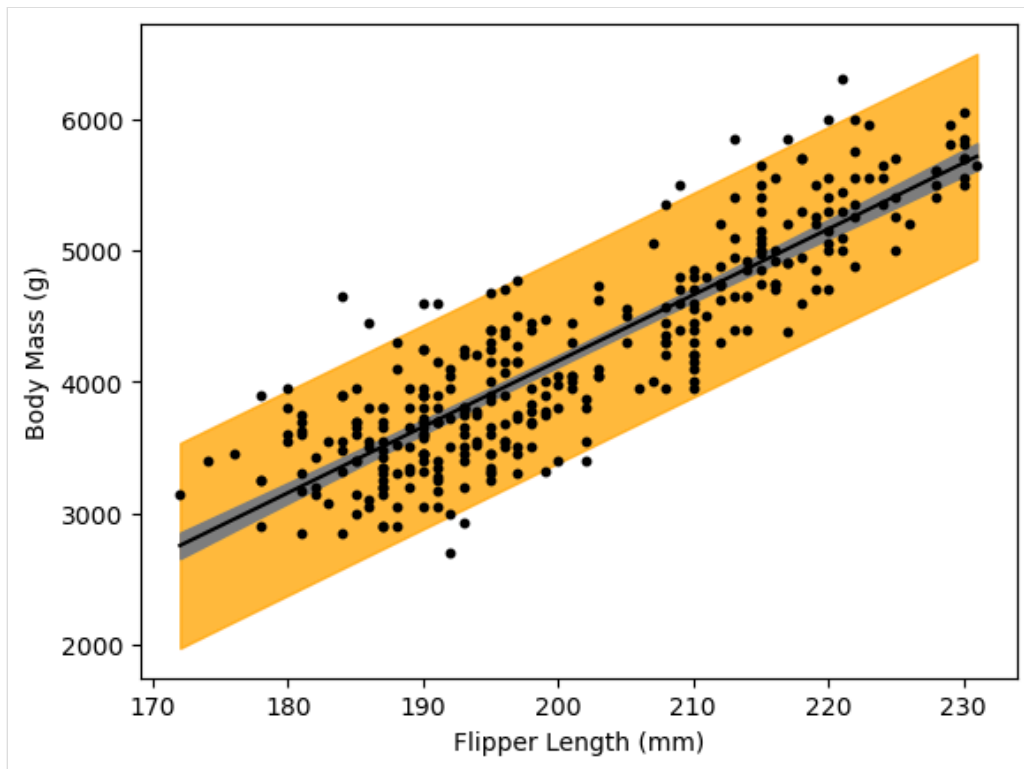
# Confidence interval - the uncertainty on the mean output
ax.fill_between(flipper_range,
                lm_pred_summary_full.mean_ci_lower,
                lm_pred_summary_full.mean_ci_upper,
                facecolor='grey',
                edgecolor='grey')

# Output mean - the predictive trend - the best fit line
ax.plot(flipper_range,
        lm_pred_summary_full['mean'],
        color='k',
        linewidth=1.5)

# Include the training data
ax.scatter(df_clean.flipper_length_mm,
           df_clean.body_mass_g,
           color='k',
           s=10)

# Set the labels
ax.set_xlabel('Flipper Length (mm)')
ax.set_ylabel('Body Mass (g)')

# Show the plot
plt.show()
```



Key Takeaways

- A prediction table makes a model's behavior concrete.
- Overlaying predictions on data exposes systematic bias.
- Simple error summaries are useful but can hide structure.
- Direction of error matters as much as its size.

Exercises

1. Build a prediction table for twenty evenly spaced inputs.
2. Overlay the predictions on the scatter plot.
3. Compute the mean and standard deviation of the errors.
4. Describe any region where the model systematically over-predicts.

Guided Walkthrough

The notebook builds a table of predictions for a range of inputs and overlays them on the data. It then summarizes the errors and, crucially, checks their direction. A model that over-predicts in one region and under-predicts in another is misspecified even if its average error is small.

Common Pitfalls

- Judging a model by a single average error.
- Ignoring the sign of the errors.
- Comparing predictions without a common scale.

Going Further

- Plot the errors against the input to expose regional bias.
- Compare two models' prediction tables side by side.

Exercise Solutions

1. The table lists inputs and their predicted outputs.
2. The overlay shows where the line tracks and where it drifts.
3. The mean error is near zero if the model is unbiased; the standard deviation measures typical error size.
4. A region where the model systematically over-predicts is a sign of a missing term.

Chapter 9: Penguins Dataset Simple Linear Regression - Fitting Non-Linear Feature Predictions

Introduction

This chapter completes the prediction arc for a non-linear feature. The model curves, so the predictions do too, and the comparison between predictions and observations reveals how well a curved model follows the data. It is the non-linear twin of the previous chapter and reinforces the same discipline with a harder shape.

What You Will Learn

- How to generate and tabulate predictions from a curved model.
- How to judge a curved fit by eye and by summary.
- How predictions behave at the extremes of the data.
- How to decide whether the added curvature is justified.

Overview

We predict across the input range and compare the curve to the observed points. The curved model tracks the bend, but the extremes are where models earn or lose trust, because the data is sparsest there. We summarize the error and inspect the plot, then weigh the gain in fit against the added complexity of the curved form.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island        0
bill_length_mm  2
bill_depth_mm  2
flipper_length_mm 2
body_mass_g    2
sex           11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island        0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm 0
body_mass_g    0
sex           0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island        3
bill_length_mm 163
bill_depth_mm  79
flipper_length_mm 54
body_mass_g    93
```

```
sex          2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm    163
bill_depth_mm     79
flipper_length_mm  54
body_mass_g       93
dtype: int64
```

Fitting a Non-Linear Feature

We will fit a linear model between the input `flipper_length_mm` and the output `body_mass_g` assuming a non-linear relationship between the trend and the input.

$$\mu = \beta_0 + \beta_1 x + \beta_2 x^2$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **linear effect**: change in μ for a one-unit increase in x

β_2 **quadratic effect**: change in μ for a one-unit increase in x^2

x **feature** or **input variable**: independent variable used to predict μ

```
lm_fit = smf.ols(formula='body_mass_g ~ flipper_length_mm + I(flipper_length_mm ** 2)',
                  data=df_clean). \
    fit()
```

```
lm_fit.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
=====
                        OLS Regression Results
=====
Dep. Variable:          body_mass_g    R-squared:                0.778
```

```

Model:                OLS      Adj. R-squared:      0.777
Method:              Least Squares  F-statistic:      578.5
Date:                Thu, 24 Sep 2026  Prob (F-statistic): 1.34e-108
Time:                10:17:28    Log-Likelihood:   -2449.5
No. Observations:    333        AIC:                4905.
Df Residuals:        330        BIC:                4916.
Df Model:             2
Covariance Type:     nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.658e+04	4616.247	3.592	0.000	7498.887	2.57e+04
flipper_length_mm	-171.6140	45.524	-3.770	0.000	-261.169	-82.059
I(flipper_length_mm ** 2)	0.5449	0.112	4.874	0.000	0.325	0.765

```

=====
Omnibus:            10.746    Durbin-Watson:      2.214
Prob(Omnibus):      0.005    Jarque-Bera (JB):   10.985
Skew:               0.441    Prob(JB):           0.00412
Kurtosis:           3.119    Cond. No.           9.07e+06
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 9.07e+06. This might indicate that there are strong multicollinearity or other numerical problems.

```

"""

```

The `.params` attribute stores the coefficient estimates as they minimize the sum of squared errors.

- The **intercept** is 16,579.87, which represents the baseline predicted body mass when both flipper length and its square are zero.
- The **coefficient** for `flipper_length_mm` is -171.61, indicating that for every additional millimeter in flipper length, body mass decreases by about 171.61 grams, before considering the quadratic effect.
- The **coefficient for the squared term**, `I(flipper_length_mm ** 2)`, is 0.54, meaning that for every unit increase in the square of flipper length, body mass increases by 0.54 grams, introducing an upward-curving effect.
- The presence of the positive squared term (0.54) shows that the relationship between flipper length and body mass is quadratic and curves upward, rather than being strictly linear.

```
lm_fit.params
```

```

Intercept          16579.869215
flipper_length_mm  -171.613976
I(flipper_length_mm ** 2)    0.544948
dtype: float64

```

The standard error on the estimate is contained in the `.bse` attribute.

- The standard error for the intercept is 4,616.25, indicating the level of uncertainty in estimating the baseline predicted body mass.
- The standard error for the `flipper_length_mm` coefficient is 45.52, quantifying the variability in the estimate of the linear effect of flipper length on body mass.

- The standard error for the squared term, `I(flipper_length_mm ** 2)`, is 0.11, reflecting the precision of the estimate for the quadratic effect.

```
lm_fit.bse
```

```
Intercept          4616.246647
flipper_length_mm  45.524470
I(flipper_length_mm ** 2)  0.111807
dtype: float64
```

We can use the ± 1 and ± 2 standard error intervals to know how trustworthy the estimate is.

- We are 95% confident that the intercept is somewhere between 7,498.89 and 25,660.85 grams.
- We are 95% confident that the coefficient for `flipper_length_mm` is somewhere between -261.17 and -82.06 grams per millimeter.
- We are 95% confident that the coefficient for the quadratic term (`I(flipper_length_mm ** 2)`) is somewhere between 0.33 and 0.76.
- These are exact confidence intervals calculated by `.conf_int()` rather than approximate intervals based on standard errors.

```
lm_fit.params - 2 * lm_fit.bse
```

```
Intercept          7347.375921
flipper_length_mm  -262.662917
I(flipper_length_mm ** 2)  0.321333
dtype: float64
```

```
lm_fit.params + 2 * lm_fit.bse
```

```
Intercept          25812.362509
flipper_length_mm  -80.565036
I(flipper_length_mm ** 2)  0.768562
dtype: float64
```

```
lm_fit.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

```
          ci_lwr    ci_upr
Intercept  7498.887253 25660.851176
flipper_length_mm -261.168743 -82.059210
I(flipper_length_mm ** 2)  0.325003  0.764892
```

The P-values for the coefficients are used to identify the statistically significant of the input.

- A **statistically significant** input one that has a **p-value** less than 0.05 which means if this is the case, we have a **statistically significant** input or feature.
- These values are extremely small and much lower than 0.05 so they indicate very strong **statistical significance** for both parameters.

```
lm_fit.pvalues
```

```
Intercept          0.000379
flipper_length_mm  0.000194
I(flipper_length_mm ** 2)  0.000002
dtype: float64
```

```
lm_fit.pvalues < 0.05
```

```
Intercept          True
flipper_length_mm  True
I(flipper_length_mm ** 2)  True
dtype: bool
```

Visualize the coefficient estimates as markers in a scatter plot and the confidence intervals as error bars.

- We are confident that the 95% confidence interval does NOT contain zero as both `flipper_length_mm` and the **intercept** are not touching the zero grey dotted line.
- The **intercept** is **statistically significantly negative**; the **slope** is negative, and the `flipper_length_mm` is **statistically significantly positive**; the **slope** is positive.
- **We are confident the slope is positive for the `flipper_length_mm`, therefore we are confident that the average output is increasing as the input or feature is increasing.**

```
def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolord='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
```

```

        color: marker color
        ecolor: error bar color
        elinewidth: error bar line width
        ms: marker size
        vline: where to draw the vertical reference line (default 0)
        xlabel: x-axis label (default 'coefficient value')
        title: figure title (default None, uses variable name)
        figsize: tuple, figure size
        xlim: tuple, x-axis limits (optional, auto if None)
    """
    # Get coefficient and error values
    coef = model.params[var]
    err = xerr_mult * model.bse[var]

    # Create the plot
    _, ax = plt.subplots(figsize=figsize)
    ax.errorbar(
        y = [var],
        x = [coef],
        xerr = [err],
        fmt='o',
        color=color,
        ecolor=ecolor,
        elinewidth=elinewidth,
        ms=ms
    )

    # Add vertical line for reference
    ax.axvline(x=vline,
               linestyle='--',
               linewidth=4,
               color='grey')
    ax.set_xlabel(xlabel)
    if title is None:
        title = f"Coefficient Plot for '{var}'"
    ax.set_title(title)

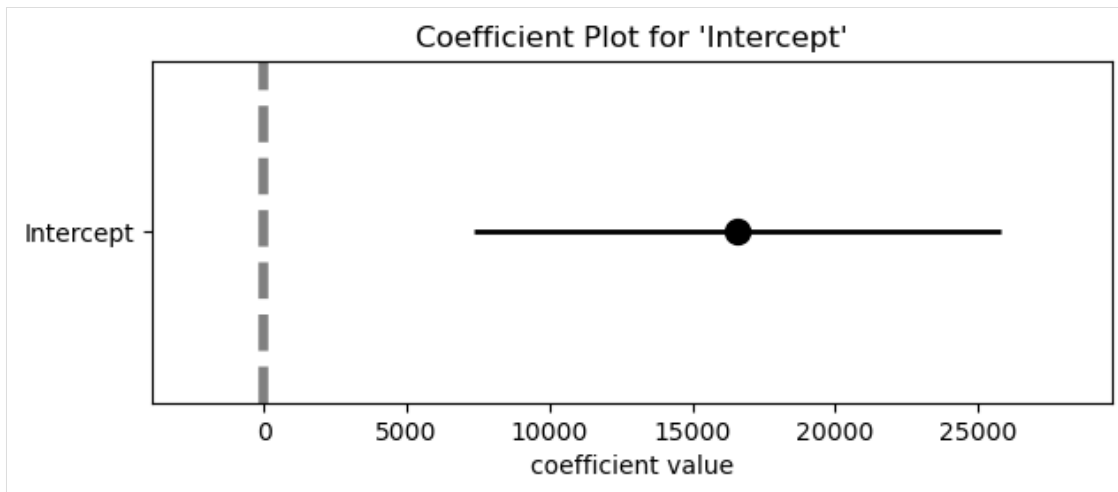
    # Always include zero in xlim, and add padding around error bar
    bar_left = min(coef - err, vline)
    bar_right = max(coef + err, vline)
    pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
    auto_xlim = (bar_left - pad, bar_right + pad)

    # Set x-axis limits
    if xlim is not None:
        ax.set_xlim(*xlim)
    else:
        ax.set_xlim(*auto_xlim)

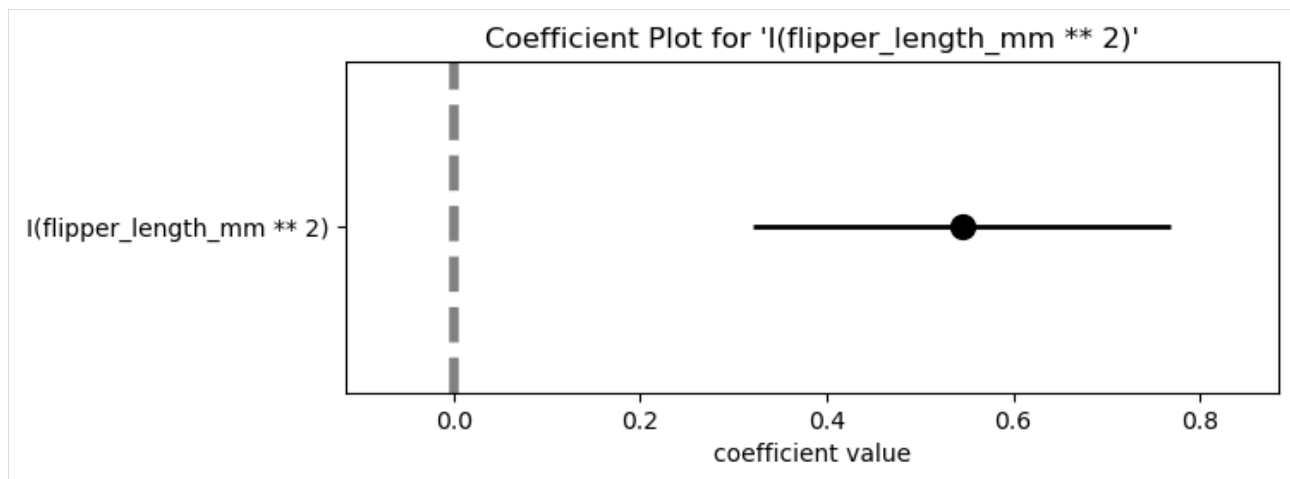
    # Show the plot
    plt.show()

```

```
my_coefplot(lm_fit, var='Intercept')
```



```
my_coefplot(lm_fit, var='I(flipper_length_mm ** 2)')
```



Non-Linear Feature Prediction

```
x = df_clean['flipper_length_mm']
y = df_clean['body_mass_g']
```

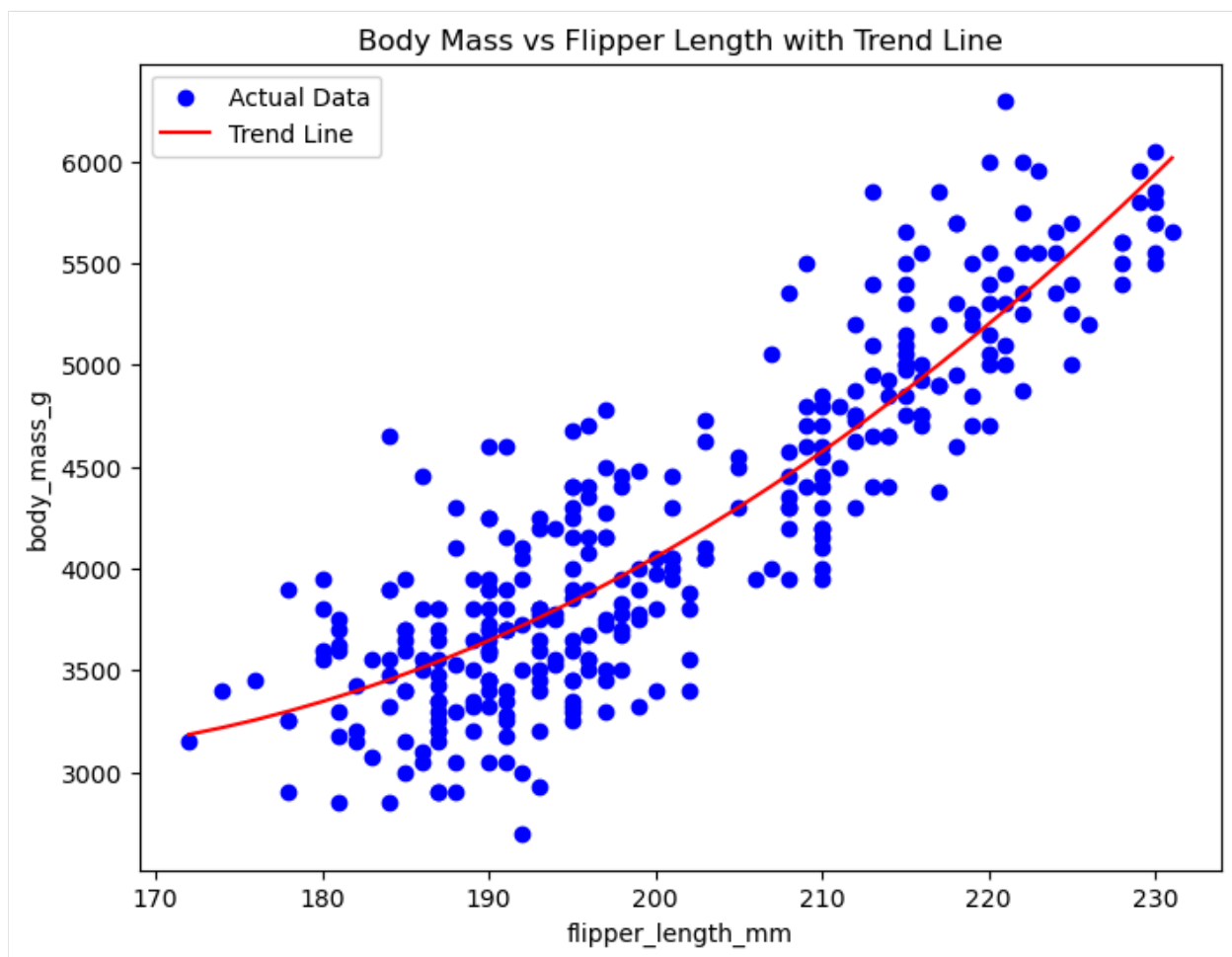
```
my_quadratic, my_slope, my_intercept = np.polyfit(x,
                                                    y,
                                                    2)
```

```
df_viz = df_clean.copy()
```

```
df_viz['trend'] = my_intercept + \
    my_slope * df_viz['flipper_length_mm'] + \
    my_quadratic * df_viz['flipper_length_mm'] \
    **2
```


Include the training set as a scatter plot.

```
sorted_df = df_viz.sort_values('flipper_length_mm')
plt.figure(figsize=(8,6))
plt.scatter(df_viz['flipper_length_mm'],
            df_viz['body_mass_g'],
            color='blue',
            label='Actual Data')
plt.plot(sorted_df['flipper_length_mm'],
         sorted_df['trend'],
         color='red',
         label='Trend Line')
plt.xlabel('flipper_length_mm')
plt.ylabel('body_mass_g')
plt.title('Body Mass vs Flipper Length with Trend Line')
plt.legend()
plt.show()
```



We will not know the true relationship therefore we must include the confidence intervals on the predicted trend as we must always represent the uncertainty on the trend given the fitted data.

```
predictions = lm_fit.get_prediction(df_viz)
```

```
lm_pred_summary = predictions.summary_frame()
```

lm_pred_summary

```

      mean    mean_se mean_ci_lower mean_ci_upper obs_ci_lower \
0    3370.773580  49.673534   3273.056864   3468.490297   2615.946156
1    3512.682868  32.638135   3448.477825   3576.887911   2761.458420
2    3836.783006  26.945004   3783.777368   3889.788644   3086.432766
3    3757.131484  26.122145   3705.744557   3808.518411   3006.893855
4    3645.828418  26.589245   3593.522623   3698.134213   2895.527292
..      ...      ...      ...      ...      ...
328  4810.906624  30.268285   4751.363500   4870.449749   4060.066250
329  4873.075244  30.527844   4813.021521   4933.128967   4122.194206
330  5338.772657  40.807527   5258.496960   5419.048355   4586.004385
331  4689.839072  30.240975   4630.349671   4749.328472   3939.002956
332  4749.827900  30.188751   4690.441234   4809.214567   3998.999917

      obs_ci_upper
0    4125.601005
1    4263.907316
2    4587.133245
3    4507.369113
4    4396.129544
..      ...
328  5561.746999
329  5623.956282
330  6091.540929
331  5440.675187
332  5500.655883

[333 rows x 6 columns]
```

Visualize the 95% confidence interval and 95% prediction interval.

- **Confidence Interval (CI) for the Mean Prediction (Grey Band):**

- Shows uncertainty in the estimate of the average value (the regression line) for a given x.
- Typically narrow; with enough data, the mean can be estimated precisely.
- 95% CI means: "We are 95% confident the true average response falls within this band for each x-value."
- Most data points will not fall inside this band; it is only about the mean.

- **Prediction Interval (PI) for Individual Observations (Orange Band):**

- Shows where we expect 95% of future individual data points to fall, at each value of x.
- Much wider than the CI, because it includes both the uncertainty in the mean and the natural scatter (variance) of the data.
- 95% PI means: "If we observe a new data point at a given x, there is a 95% chance it will fall inside this band."

- **Key Differences:**

- CI is about the uncertainty in the average (regression line).
- PI is about the uncertainty in predicting a single new data point.

- **In the plot:**

- The grey band is the 95% CI for the mean prediction.
- The orange band is the 95% PI for individual body mass measurements given flipper length.
- The black dots are your actual data points.
- Most dots fall inside the orange PI band (as expected), but only a few will fall inside the narrow grey CI band.
- Use the PI to show the expected spread of new data.
- Use the CI to show uncertainty in the estimated average.

```
# Generate x values over the full data range
flipper_range = np.linspace(df_clean.flipper_length_mm.min(),
                             df_clean.flipper_length_mm.max(),
                             200)

df_pred = pd.DataFrame({'flipper_length_mm': flipper_range})
lm_pred_summary_full = lm_fit.get_prediction(df_pred).summary_frame()

# Create the figure and axis
fig, ax = plt.subplots()

# Prediction interval – the uncertainty on a single measurement (observation)
ax.fill_between(flipper_range,
                 lm_pred_summary_full.obs_ci_lower,
                 lm_pred_summary_full.obs_ci_upper,
                 facecolor='orange',
                 alpha=0.75,
                 edgecolor='orange')

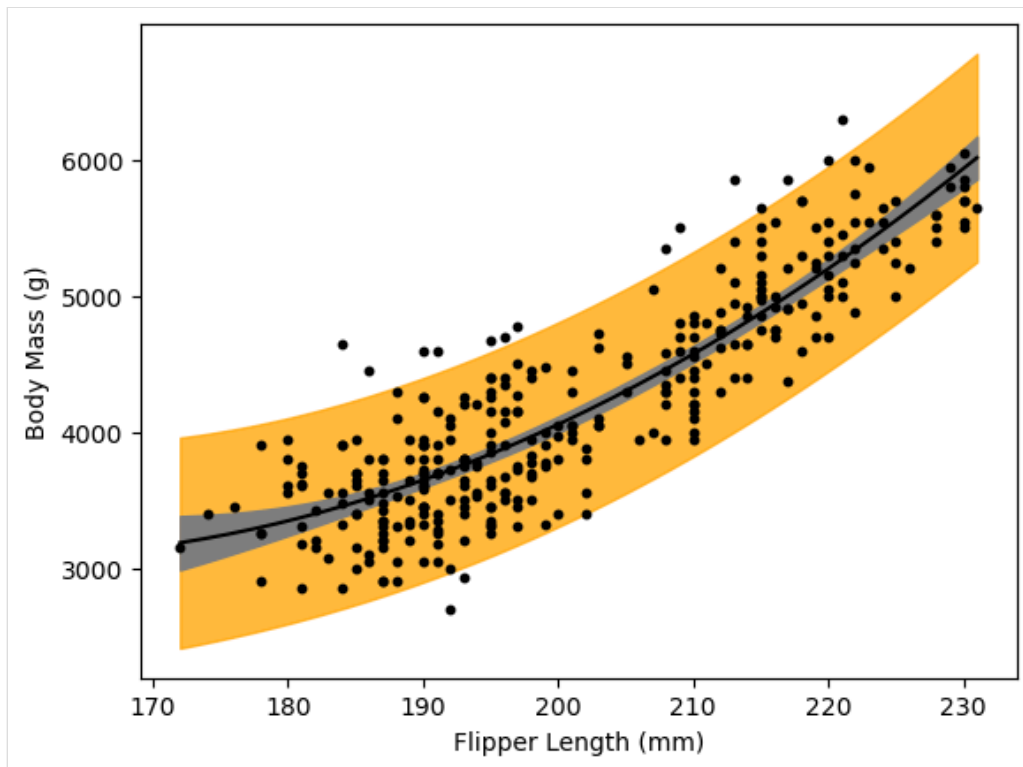
# Confidence interval – the uncertainty on the mean output
ax.fill_between(flipper_range,
                 lm_pred_summary_full.mean_ci_lower,
                 lm_pred_summary_full.mean_ci_upper,
                 facecolor='grey',
                 edgecolor='grey')

# Output mean – the predictive trend – the best fit line
ax.plot(flipper_range,
        lm_pred_summary_full['mean'],
        color='k',
        linewidth=1.5)

# Include the training data
ax.scatter(df_clean.flipper_length_mm,
           df_clean.body_mass_g,
           color='k',
           s=10)

# Set the labels
ax.set_xlabel('Flipper Length (mm)')
ax.set_ylabel('Body Mass (g)')

# Show the plot
plt.show()
```



Key Takeaways

- Curved models predict curvature but are less stable at the edges.
- Extreme inputs are where the data is thinnest and the model is boldest.
- Weigh improved fit against added complexity.
- Visual comparison remains the fastest diagnostic.

Exercises

1. Generate predictions across the full input range and plot them.
2. Compare the errors near the center and near the edges.
3. Fit both a line and a curve and compare their error summaries.
4. State which model you would ship and why.

Guided Walkthrough

This notebook generates predictions from a curved model and compares them to observations across the whole range. The center fits well, and the edges are where the curve is boldest. The notebook weighs the improved fit against the added complexity, which is the recurring theme of model selection.

Common Pitfalls

- Preferring a complex model for a small gain in fit.
- Ignoring edge instability.
- Reporting predictions without error estimates.

Going Further

- Fit a line and a curve and compare their errors on held-out points.
- Use cross-validation to compare the two forms.

Exercise Solutions

1. The predictions follow the bend and track the center closely.
2. Edge errors are larger because the data is sparse there.
3. The curved model usually wins on fit but at the cost of stability.
4. Ship the simpler model unless the curved model's gain is clear and stable.

Chapter 10: Penguins Dataset Simple Linear Regression - Measuring Linear Feature Performance

Introduction

A model that fits the data it was trained on is not necessarily a good model. This chapter introduces performance measurement for a single-feature model: how to quantify error, how to compare models fairly, and how to avoid fooling yourself. It is the bridge from descriptive regression to model selection.

What You Will Learn

- How to measure a model's performance with R-squared and root mean squared error.
- Why training performance overstates real-world performance.
- How to hold out data to get an honest estimate.
- How to compare two models on the same footing.

Overview

We compute R-squared, which measures the fraction of variance the model explains, and root mean squared error, which measures the typical size of the errors in the units of the target. Both are useful and neither is sufficient. We then introduce the idea of evaluating on data the model has not seen, which is the foundation of every honest evaluation in the rest of the course.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island        0
bill_length_mm    2
bill_depth_mm    2
flipper_length_mm 2
body_mass_g      2
sex           11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island        0
bill_length_mm    0
bill_depth_mm    0
flipper_length_mm 0
body_mass_g      0
sex           0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island        3
bill_length_mm 163
bill_depth_mm   79
flipper_length_mm 54
body_mass_g    93
```

```
sex          2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm    163
bill_depth_mm     79
flipper_length_mm  54
body_mass_g       93
dtype: int64
```

Fitting a Linear Feature

We will fit a linear model between the input `flipper_length_mm` and the output `body_mass_g` assuming a linear relationship between the trend and the input.

$$\mu = \beta_0 + \beta_1 \times x$$

μ predicted trend

β_0 **intercept**: value of μ when $x = 0$

β_1 **slope**: change in μ for a one-unit increase in x

x **feature** or **input variable**: independent variable used to predict μ

```
lm_fit = smf.ols(formula='body_mass_g ~ flipper_length_mm',
                  data=df_clean). \
    fit()
```

```
lm_fit.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
"""
```

```

                        OLS Regression Results
=====
Dep. Variable:          body_mass_g    R-squared:                0.762
Model:                  OLS           Adj. R-squared:            0.761
Method:                 Least Squares  F-statistic:              1060.
```



```

Date:                Thu, 24 Sep 2026    Prob (F-statistic):    3.13e-105
Time:                10:17:32           Log-Likelihood:       -2461.1
No. Observations:    333                AIC:                 4926.
Df Residuals:        331                BIC:                 4934.
Df Model:            1
Covariance Type:     nonrobust
=====
              coef    std err          t      P>|t|      [0.025    0.975]
-----
Intercept      -5872.0927    310.285    -18.925    0.000   -6482.472   -5261.713
flipper_length_mm  50.1533      1.540     32.562    0.000     47.123     53.183
=====
Omnibus:            5.922    Durbin-Watson:           2.102
Prob(Omnibus):      0.052    Jarque-Bera (JB):         5.876
Skew:               0.325    Prob(JB):                 0.0530
Kurtosis:           3.025    Cond. No.                 2.90e+03
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 2.9e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```

"""

```

The `.params` attribute stores the coefficient estimates as they minimize the sum of squared errors.

- The **slope**, 50.15, describes how much the predicted body mass changes for each additional millimeter of flipper length.
- The **intercept**, -5872.09, is a mathematical result that helps position the regression line but isn't physically meaningful for penguins, since the data doesn't include flipper lengths near zero.

```
lm_fit.params
```

```

Intercept      -5872.092683
flipper_length_mm  50.153266
dtype: float64

```

The standard error on the estimate is contained in the `.bse` attribute.

- The **standard error** for the **intercept**, 310.29, indicates that, if we repeatedly sampled penguins and fit the same model, the estimated starting body mass, -5872.09, would typically vary by about 310 grams between samples, showing the precision of this baseline estimate.
- The **standard error** for the flipper length coefficient, 1.54, means our estimate that each additional millimeter of flipper length increases a penguin's body mass by 50.15 grams is quite precise, with typical variation of only 1.54 grams across samples.
- These small **standard errors** demonstrate that, for the penguins in the dataset, both the baseline body mass and the effect of flipper length are estimated with high statistical confidence.
- They are considered to have high statistical confidence because the **standard errors** for both the **intercept**, 310.29, and the flipper length coefficient, 1.54, are small relative to the estimated values, -5872.09 and 50.15, respectively.

- This means that if you repeatedly sampled penguins and fit the same regression model, the estimates for the intercept and slope would vary only slightly from sample to sample.

```
lm_fit.bse
```

```
Intercept      310.285155
flipper_length_mm  1.540231
dtype: float64
```

We can use the ± 1 and ± 2 standard error intervals to know how trustworthy the estimate is.

- The most common standard error interval is the ± 2 standard error interval because that is approximately equal to the 95% confidence interval.
- We are 95% confident that the coefficient is somewhere between 47 and 53.
- The below are approximations as we will get the exact values with `.conf_int()`.

```
lm_fit.params - 2 * lm_fit.bse
```

```
Intercept      -6492.662994
flipper_length_mm  47.072804
dtype: float64
```

```
lm_fit.params + 2 * lm_fit.bse
```

```
Intercept      -5251.522372
flipper_length_mm  53.233728
dtype: float64
```

```
lm_fit.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

```
Intercept      ci_lwr      ci_upr
-6482.472236 -5261.713130
flipper_length_mm  47.123390  53.183142
```

The P-values for the coefficients are used to identify the statistically significant of the input.

- A **statistically significant** input one that has a **p-value** less than 0.05 which means if this is the case, we have a **statistically significant** input or feature.
- These values are extremely small and much lower than 0.05 so they indicate very strong **statistical significance** for both parameters.

```
lm_fit.pvalues
```

```
Intercept          1.183941e-54
flipper_length_mm  3.132836e-105
dtype: float64
```

```
lm_fit.pvalues < 0.05
```

```
Intercept          True
flipper_length_mm  True
dtype: bool
```

Visualize the coefficient estimates as markers in a scatter plot and the confidence intervals as error bars.

- We are confident that the 95% confidence interval does NOT contain zero as both `flipper_length_mm` and the **intercept** are not touching the zero grey dotted line.
- The **intercept** is **statistically significantly negative**; the **slope** is negative, and the `flipper_length_mm` is **statistically significantly positive**; the **slope** is positive.
- **We are confident the slope is positive for the `flipper_length_mm`, therefore we are confident that the average output is increasing as the input or feature is increasing.**

```
def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolor='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
        color: marker color
        ecolor: error bar color
        elinewidth: error bar line width
        ms: marker size
        vline: where to draw the vertical reference line (default 0)
        xlabel: x-axis label (default 'coefficient value')
        title: figure title (default None, uses variable name)
        figsize: tuple, figure size
        xlim: tuple, x-axis limits (optional, auto if None)
    """
    # Get coefficient and error values
    coef = model.params[var]
    err = xerr_mult * model.bse[var]

    # Create the plot
```

```

_, ax = plt.subplots(figsize=figsize)
ax.errorbar(
    y = [var],
    x = [coef],
    xerr = [err],
    fmt='o',
    color=color,
    ecolor=ecolor,
    elinewidth=elinewidth,
    ms=ms
)

# Add vertical line for reference
ax.axvline(x=vline,
           linestyle='--',
           linewidth=4,
           color='grey')
ax.set_xlabel(xlabel)
if title is None:
    title = f"Coefficient Plot for '{var}'"
ax.set_title(title)

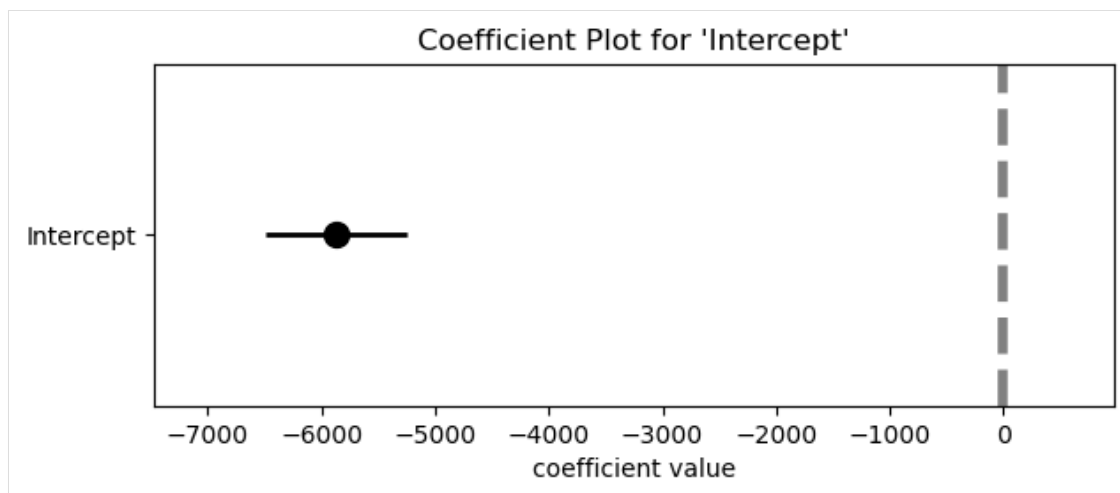
# Always include zero in xlim, and add padding around error bar
bar_left = min(coef - err, vline)
bar_right = max(coef + err, vline)
pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
auto_xlim = (bar_left - pad, bar_right + pad)

# Set x-axis limits
if xlim is not None:
    ax.set_xlim(*xlim)
else:
    ax.set_xlim(*auto_xlim)

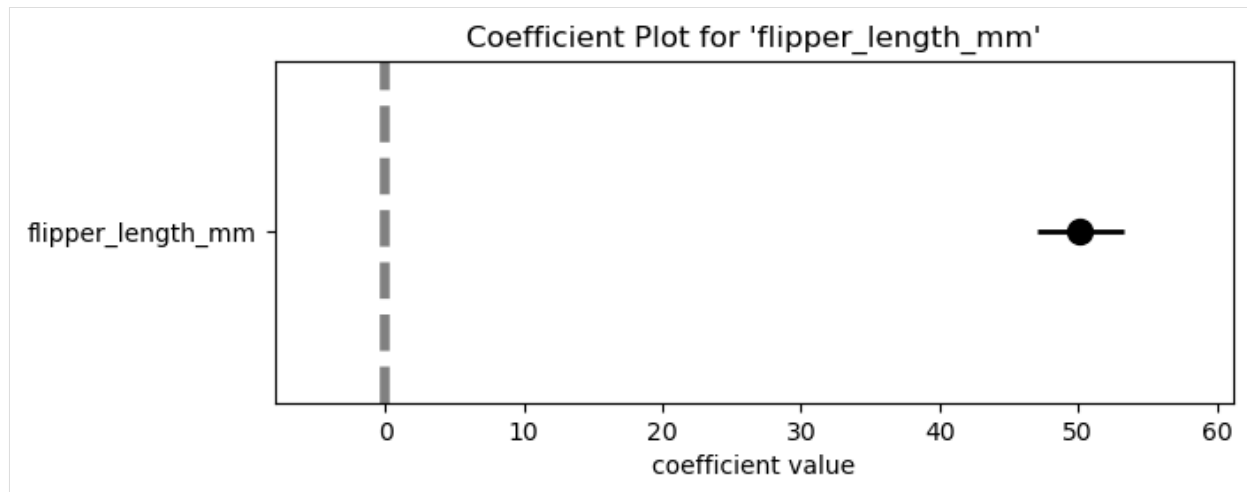
# Show the plot
plt.show()

```

```
my_coefplot(lm_fit, var='Intercept')
```



```
my_coefplot(lm_fit, var='flipper_length_mm')
```



Measure Linear Feature Performance

```
lm_fit.fittedvalues
```

```
0      3205.648453
1      3456.414782
2      3907.794176
4      3807.487644
5      3657.027846
...
338    4860.706229
340    4910.859495
341    5261.932356
342    4760.399697
343    4810.552963
Length: 333, dtype: float64
```

Visually compare the model fitted values or training predictions to the observed outputs.

```
df_clean_copy = df_clean.loc[:,
                             ['flipper_length_mm',
                              'body_mass_g']]. \
    copy()
```

```
df_clean_copy
```

```
   flipper_length_mm  body_mass_g
0              181.0      3750.0
1              186.0      3800.0
2              195.0      3250.0
4              193.0      3450.0
5              190.0      3650.0
..              ...          ...
338             214.0      4925.0
340             215.0      4850.0
341             222.0      5750.0
342             212.0      5200.0
343             213.0      5400.0
```

```
[333 rows x 2 columns]
```

```
df_clean_copy['fitted'] = lm_fit.fittedvalues
```

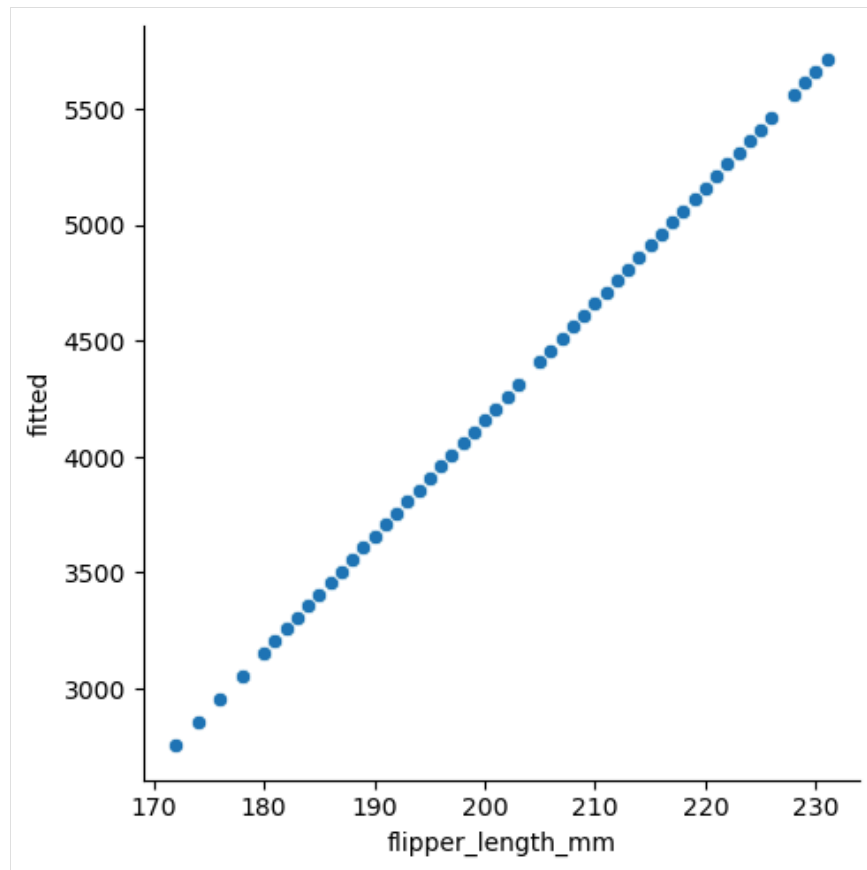
```
df_clean_copy
```

```
   flipper_length_mm  body_mass_g      fitted
0             181.0      3750.0  3205.648453
1             186.0      3800.0  3456.414782
2             195.0      3250.0  3907.794176
4             193.0      3450.0  3807.487644
5             190.0      3650.0  3657.027846
..              ...          ...          ...
338            214.0      4925.0  4860.706229
340            215.0      4850.0  4910.859495
341            222.0      5750.0  5261.932356
342            212.0      5200.0  4760.399697
343            213.0      5400.0  4810.552963
```

```
[333 rows x 3 columns]
```

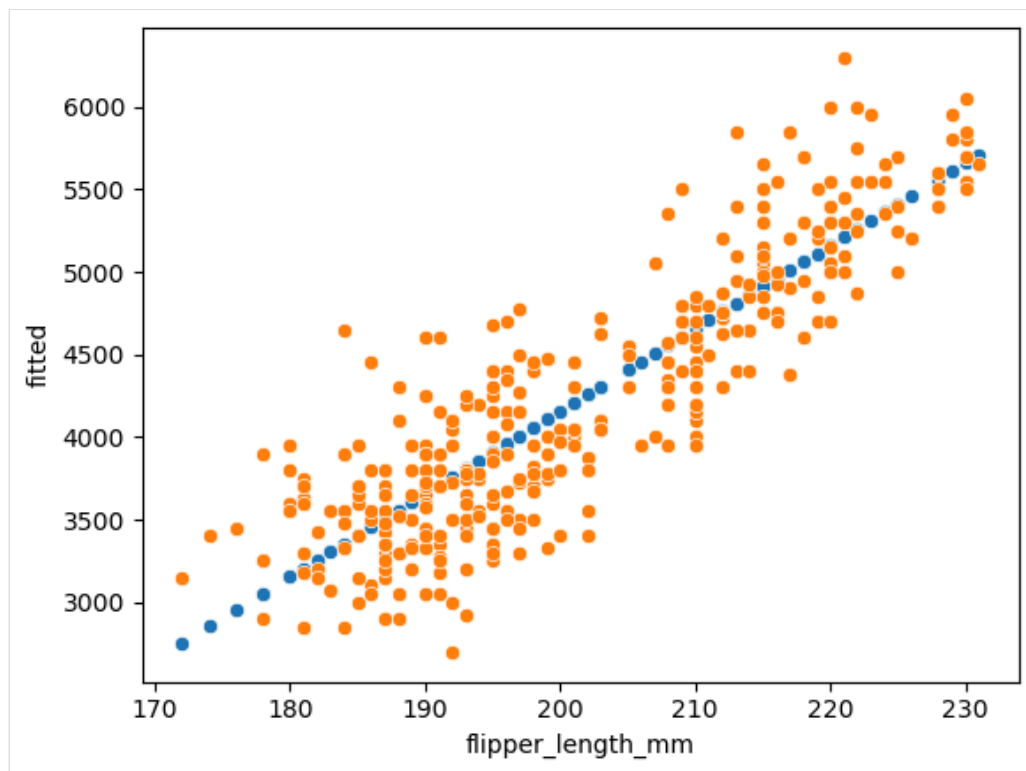
Visualize the Predictions

```
sns.relplot(data=df_clean_copy,
            x='flipper_length_mm',
            y='fitted')
plt.show()
```



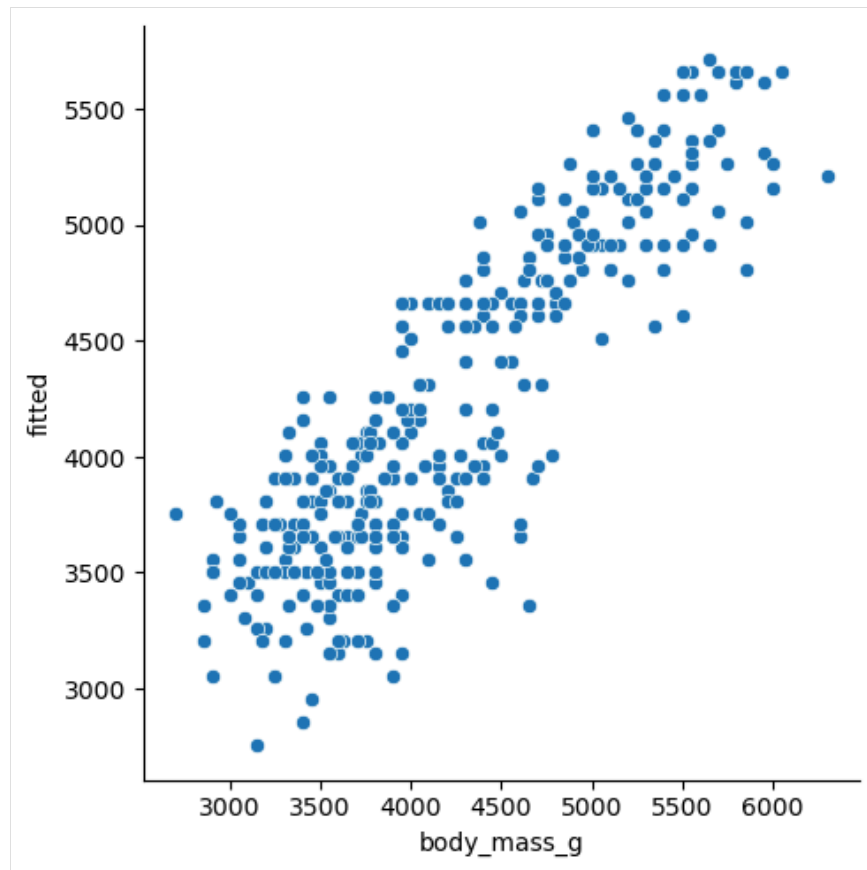
Compare the observed output to the model fits by overlaying the observed output vs the input with the predicted trend or fitted values with the input.

```
fig, ax = plt.subplots()
sns.scatterplot(data=df_clean_copy,
                x='flipper_length_mm',
                y='fitted',
                ax=ax)
sns.scatterplot(data=df_clean_copy,
                x='flipper_length_mm',
                y='body_mass_g',
                ax=ax)
plt.show()
```



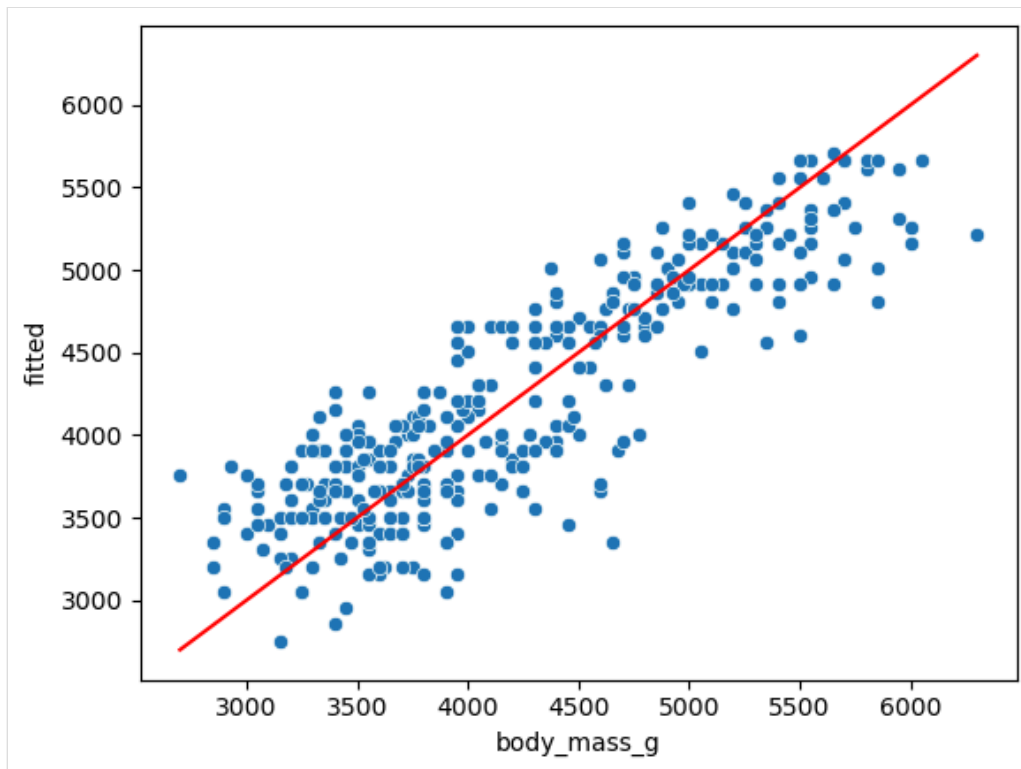
Directly relate the observed output and the model fits or the predicted vs observed figure.

```
sns.relplot(data=df_clean_copy,  
            x='body_mass_g',  
            y='fitted')  
plt.show()
```

Include a "45 degree" reference line with our predicted vs observed figure.

```
fig, ax = plt.subplots()
sns.scatterplot(data=df_clean_copy,
                x='body_mass_g',
                y='fitted',
                ax=ax)
sns.lineplot(data=df_clean_copy,
             x='body_mass_g',
             y='body_mass_g',
             color='red',
             ax=ax)
plt.show()
```



Often we want a number or quantitative metric for the correlation between the model predictions and the observed outputs by calculating the correlation coefficient between the fits and the observed output.

```
df_clean_copy.loc[:,
    ['body_mass_g', 'fitted']]. \
    corr(numeric_only=True)
```

```
body_mass_g    body_mass_g    fitted
body_mass_g    1.000000    0.872979
fitted          0.872979    1.000000
```

```
df_clean_copy.loc[:,
    ['body_mass_g', 'fitted']]. \
    corr(numeric_only=True). \
    iloc[0, 1]
```

```
0.8729788985653614
```

The correlation coefficient can be negative or positive so we will square it.

```
df_clean_copy.loc[:,
    ['body_mass_g', 'fitted']]. \
    corr(numeric_only=True). \
    iloc[0, 1] ** 2
```

```
0.7620921573403915
```

This calculation is called R-Squared as it comes from the predicted vs observed figure above as it will show whether the observed output increase or decrease with an increase or decrease of the predicted output from the model.

IF THERE IS A GENERAL LINEAR RELATIONSHIP BETWEEN THE OBSERVED AND PREDICTED OUTPUT, R-SQUARED WILL BE HIGH CLOSE TO 1.

```
lm_fit.rsquared
```

```
0.7620921573403914
```

The error is the difference between the observed output and the predicted trend or fitted values as the errors can be positive or negative.

```
df_clean_copy['errors'] = df_clean_copy.body_mass_g - df_clean_copy.fitted
```

```
df_clean_copy
```

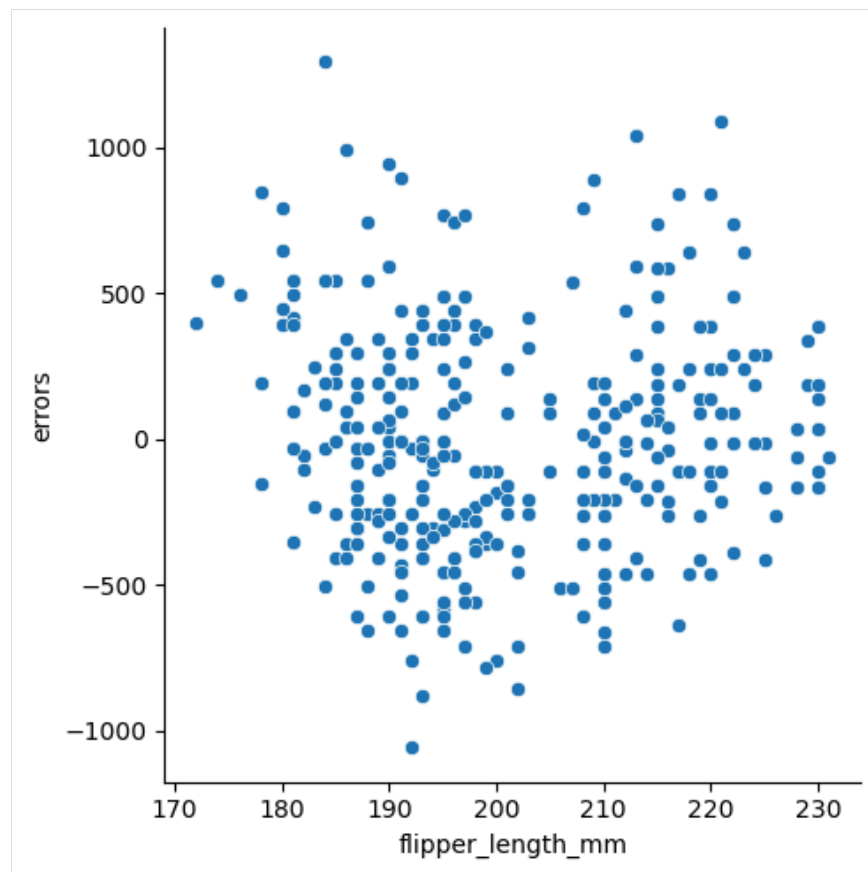
	flipper_length_mm	body_mass_g	fitted	errors
0	181.0	3750.0	3205.648453	544.351547
1	186.0	3800.0	3456.414782	343.585218
2	195.0	3250.0	3907.794176	-657.794176
4	193.0	3450.0	3807.487644	-357.487644
5	190.0	3650.0	3657.027846	-7.027846
..	...	...	...	...
338	214.0	4925.0	4860.706229	64.293771
340	215.0	4850.0	4910.859495	-60.859495
341	222.0	5750.0	5261.932356	488.067644
342	212.0	5200.0	4760.399697	439.600303
343	213.0	5400.0	4810.552963	589.447037

```
[333 rows x 4 columns]
```

We can visualize the errors with respect to an input as ideally there should be no clear correlation between an error and an input as the errors are meant to represent unexplained variability or everything the model can't capture.

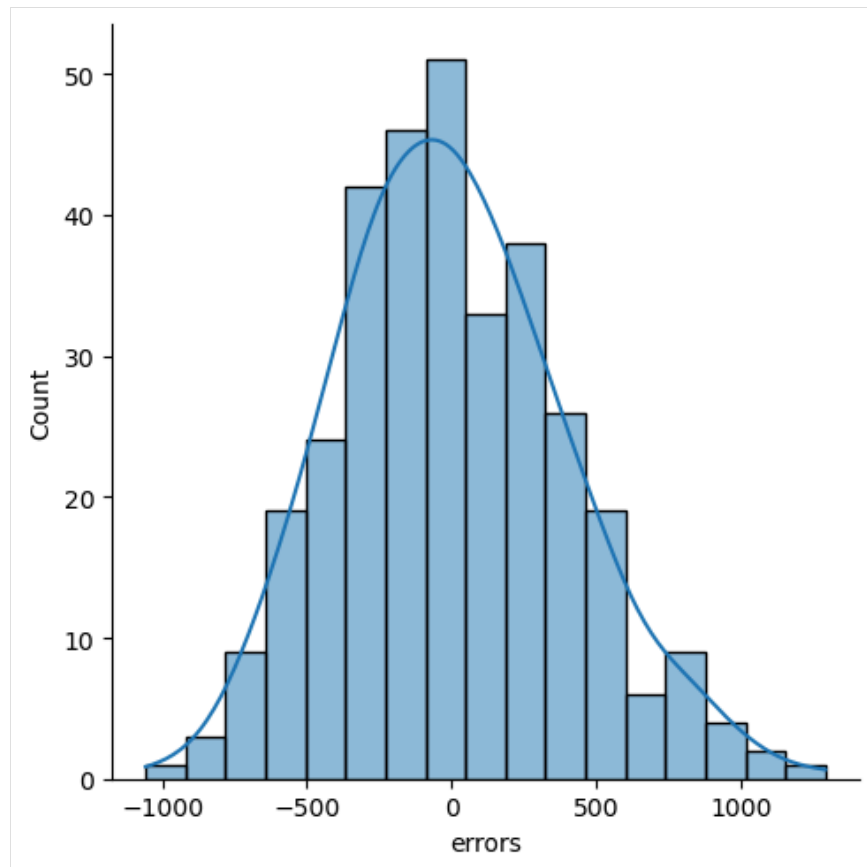
IF YOU SEE A LINEAR OR NON-LINEAR RELATIONSHIP THAT MEANS THERE IS SOME FEATURE YOU NEED TO DERIVE FROM YOUR INPUT TO BETTER CAPTURE THE OUTPUT TO INPUT RELATIONSHIP.

```
sns.relplot(data=df_clean_copy,
            x='flipper_length_mm',
            y='errors')
plt.show()
```



The distribution of the errors should be Gaussian.

```
sns.displot(data=df_clean_copy,
            x='errors',
            kind='hist',
            kde=True)
plt.show()
```



The other term for the error is the residual.

```
lm_fit.resid
```

```
0      544.351547
1      343.585218
2     -657.794176
4     -357.487644
5      -7.027846
...
338     64.293771
340    -60.859495
341     488.067644
342     439.600303
343     589.447037
Length: 333, dtype: float64
```

```
df_clean_copy['residuals'] = lm_fit.resid
```

```
df_clean_copy
```

	flipper_length_mm	body_mass_g	fitted	errors	residuals
0	181.0	3750.0	3205.648453	544.351547	544.351547
1	186.0	3800.0	3456.414782	343.585218	343.585218

REVERSE ENGINEERING DATA

```
2          195.0      3250.0  3907.794176 -657.794176 -657.794176
4          193.0      3450.0  3807.487644 -357.487644 -357.487644
5          190.0      3650.0  3657.027846  -7.027846  -7.027846
..          ...          ...          ...          ...          ...
338        214.0      4925.0  4860.706229   64.293771   64.293771
340        215.0      4850.0  4910.859495  -60.859495  -60.859495
341        222.0      5750.0  5261.932356  488.067644  488.067644
342        212.0      5200.0  4760.399697  439.600303  439.600303
343        213.0      5400.0  4810.552963  589.447037  589.447037
```

```
[333 rows x 5 columns]
```

To summarize the errors or residuals across all of the data by squaring the error which will convert any negative value to a positive error or SSE, sum of squared errors.

```
(lm_fit.resid ** 2).sum()
```

```
51211962.72968365
```

THE SSE INCREASES AS THE NUMBER OF DATA POINTS INCREASES SO WE WANT THE AVERAGE OR THE MSE, MEAN SQUARED ERROR OR AVERAGE SQUARED ERROR PER DATA POINT.

```
(lm_fit.resid ** 2).mean()
```

```
153789.6778669179
```

The MSE is not in the same units as the output therefore we must take the square root called RMSE or root mean squared error.

```
np.sqrt((lm_fit.resid ** 2).mean())
```

```
392.16027063806183
```

The MAE or mean absolute error makes all values positive and is the average absolute error of the training data and is less sensitive to extreme values because its not squaring the error.

```
np.abs(lm_fit.resid).mean()
```

```
312.08474571800986
```

Key Takeaways

- R-squared and RMSE describe different things; report both.
- Training error is optimistic; held-out error is honest.
- Comparing models requires the same evaluation procedure.
- A single number never tells the whole story.

Exercises

1. Compute R-squared and RMSE for the fitted model.
2. Split the data into training and test sets and compare the errors.
3. Explain why test error is usually larger than training error.
4. Compare two features by their held-out error.

Guided Walkthrough

The notebook introduces honest evaluation. It computes R-squared and root mean squared error on the training data, then holds out a portion of the data and recomputes them. The held-out numbers are worse, and the gap between training and test is the first appearance of the overfitting idea that dominates the rest of the course.

Common Pitfalls

- Reporting training error as if it were test error.
- Comparing models evaluated on different splits.
- Optimizing a single metric in isolation.

Going Further

- Repeat the split with several random seeds and report the spread.
- Compare a simple and a complex model on the same held-out set.

Exercise Solutions

1. R-squared measures explained variance; RMSE measures typical error in target units.
2. Test error is larger than training error because the model has not seen the test points.
3. The gap exists because the model fits noise in the training set.
4. The feature with the lower held-out error is the better single predictor.

Chapter 11: Penguins Dataset Simple Linear Regression - Fitting Multiple Models

Introduction

Choosing a model is the central activity of applied statistics. This chapter fits several single-feature models to the Penguins data and compares them, which turns model selection into a repeatable procedure rather than a guess. The habit established here, fitting many candidates and comparing them on held-out data, scales all the way up to deep learning.

What You Will Learn

- How to fit multiple candidate models in a loop.
- How to assemble their metrics into a comparison table.
- How to weigh accuracy against simplicity.
- How to pick a final model and justify the choice.

Overview

We fit a family of single-feature models, one per candidate input, and collect their R-squared and RMSE. The comparison table makes the trade-offs visible: the best-fitting feature is not always the most useful, and a simpler model with nearly the same performance is often the better engineering choice. We end by selecting a model and writing down the reasoning, which is the beginning of the technical communication that a research role demands.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```



```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island        0
bill_length_mm  2
bill_depth_mm  2
flipper_length_mm 2
body_mass_g    2
sex           11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island        0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm 0
body_mass_g    0
sex           0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island        3
bill_length_mm 163
```

```
bill_depth_mm      79
flipper_length_mm  54
body_mass_g        93
sex                2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Fitting Multiple Models

Create plotting function to be used for multiple models.

```
def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolord='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
        color: marker color
        ecolord: error bar color
        elinewidth: error bar line width
        ms: marker size
```

```

    vline: where to draw the vertical reference line (default 0)
    xlabel: x-axis label (default 'coefficient value')
    title: figure title (default None, uses variable name)
    figsize: tuple, figure size
    xlim: tuple, x-axis limits (optional, auto if None)
"""
# Get coefficient and error values
coef = model.params[var]
err = xerr_mult * model.bse[var]

# Create the plot
_, ax = plt.subplots(figsize=figsize)
ax.errorbar(
    y = [var],
    x = [coef],
    xerr = [err],
    fmt='o',
    color=color,
    ecolor=ecolor,
    elinewidth=elinewidth,
    ms=ms
)

# Add vertical line for reference
ax.axvline(x=vline,
           linestyle='--',
           linewidth=4,
           color='grey')
ax.set_xlabel(xlabel)
if title is None:
    title = f"Coefficient Plot for '{var}'"
ax.set_title(title)

# Always include zero in xlim, and add padding around error bar
bar_left = min(coef - err, vline)
bar_right = max(coef + err, vline)
pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
auto_xlim = (bar_left - pad, bar_right + pad)

# Set x-axis limits
if xlim is not None:
    ax.set_xlim(*xlim)
else:
    ax.set_xlim(*auto_xlim)

# Show the plot
plt.show()

```

The simplest possible linear model is an unknown constant average or intercept only model where we are saying `body_mass_g` is a constant.

```

lm_fit_00 = smf.ols(formula='body_mass_g ~ 1',
                    data=df_clean). \
    fit()

```

```

lm_fit_00.summary()

```

```
<class 'statsmodels.iolib.summary.Summary'>
''''
```

OLS Regression Results

```
=====
Dep. Variable:          body_mass_g    R-squared:                0.000
Model:                  OLS            Adj. R-squared:           0.000
Method:                 Least Squares   F-statistic:              nan
Date:                   Thu, 24 Sep 2026 Prob (F-statistic):       nan
Time:                   10:17:38        Log-Likelihood:          -2700.1
No. Observations:       333            AIC:                    5402.
Df Residuals:           332            BIC:                    5406.
Df Model:                0
Covariance Type:        nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	4207.0571	44.126	95.343	0.000	4120.256	4293.858

```
=====
Omnibus:                 30.464    Durbin-Watson:           0.972
Prob(Omnibus):            0.000    Jarque-Bera (JB):        19.874
Skew:                     0.470    Prob(JB):                4.84e-05
Kurtosis:                 2.260    Cond. No.:               1.00
=====
```

Notes:

```
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
''''
```

```
lm_fit_00.params
```

```
Intercept    4207.057057
dtype: float64
```

```
lm_fit_00.bse
```

```
Intercept    44.125554
dtype: float64
```

```
lm_fit_00.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

```
          ci_lwr    ci_upr
Intercept  4120.256133  4293.857981
```

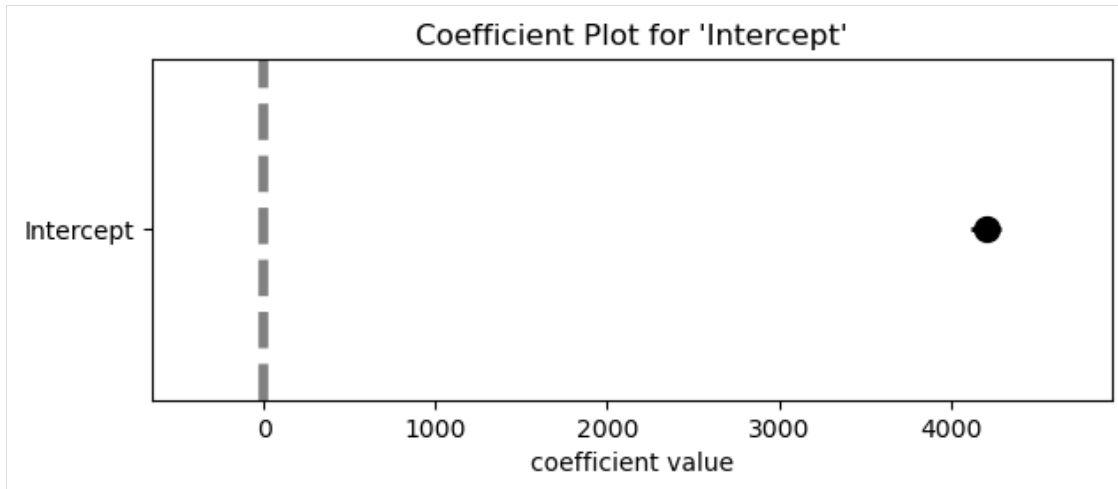
```
lm_fit_00.pvalues
```

```
Intercept    2.805040e-243
dtype: float64
```

```
lm_fit_00.pvalues < 0.05
```

```
Intercept    True
dtype: bool
```

```
my_coefplot(lm_fit_00, var='Intercept')
```



```
lm_fit_00.rsquared
```

```
4.440892098500626e-16
```

```
np.sqrt((lm_fit_00.resid ** 2).mean())
```

```
804.005860159563
```

Our next step is to model the linear relationship or classic best fit line.

```
lm_fit_01 = smf.ols(formula='body_mass_g ~ flipper_length_mm',
                    data=df_clean). \
    fit()
```

```
lm_fit_01.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
.....
```

```

                        OLS Regression Results
=====
Dep. Variable:          body_mass_g    R-squared:                0.762
Model:                  OLS           Adj. R-squared:            0.761
Method:                 Least Squares  F-statistic:              1060.
Date:                  Thu, 24 Sep 2026 Prob (F-statistic):      3.13e-105
```

```

Time:                10:17:38   Log-Likelihood:        -2461.1
No. Observations:    333       AIC:                4926.
Df Residuals:        331       BIC:                4934.
Df Model:            1
Covariance Type:     nonrobust

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----+-----
Intercept      -5872.0927     310.285    -18.925     0.000    -6482.472    -5261.713
flipper_length_mm   50.1533       1.540     32.562     0.000      47.123      53.183
=====
Omnibus:                5.922   Durbin-Watson:           2.102
Prob(Omnibus):          0.052   Jarque-Bera (JB):         5.876
Skew:                   0.325   Prob(JB):                 0.0530
Kurtosis:               3.025   Cond. No.                 2.90e+03
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 2.9e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```

"""

```

```
lm_fit_01.params
```

```

Intercept      -5872.092683
flipper_length_mm   50.153266
dtype: float64

```

```
lm_fit_01.bse
```

```

Intercept      310.285155
flipper_length_mm   1.540231
dtype: float64

```

```

lm_fit_01.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})

```

```

              ci_lwr      ci_upr
Intercept    -6482.47236  -5261.713130
flipper_length_mm   47.123390   53.183142

```

```
lm_fit_01.pvalues
```

```

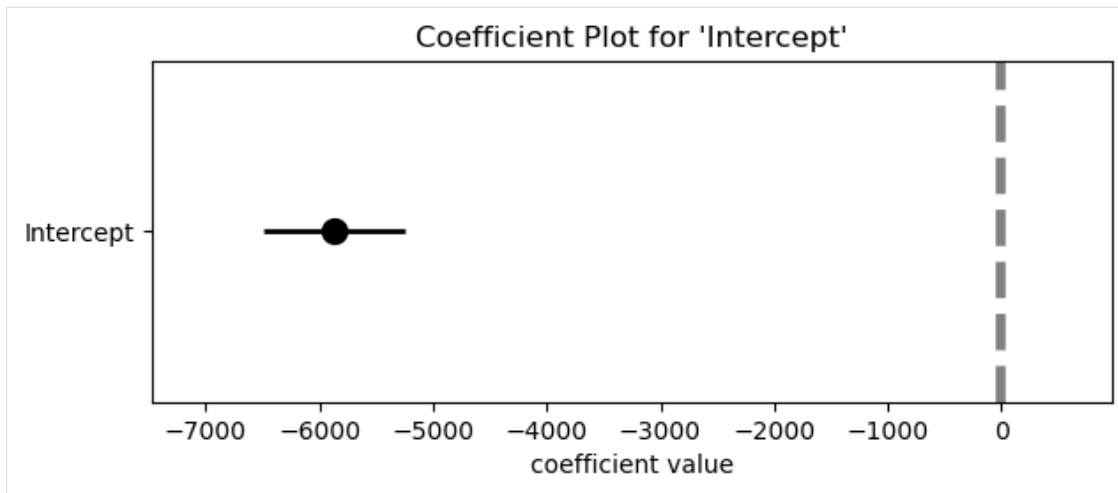
Intercept      1.183941e-54
flipper_length_mm   3.132836e-105
dtype: float64

```

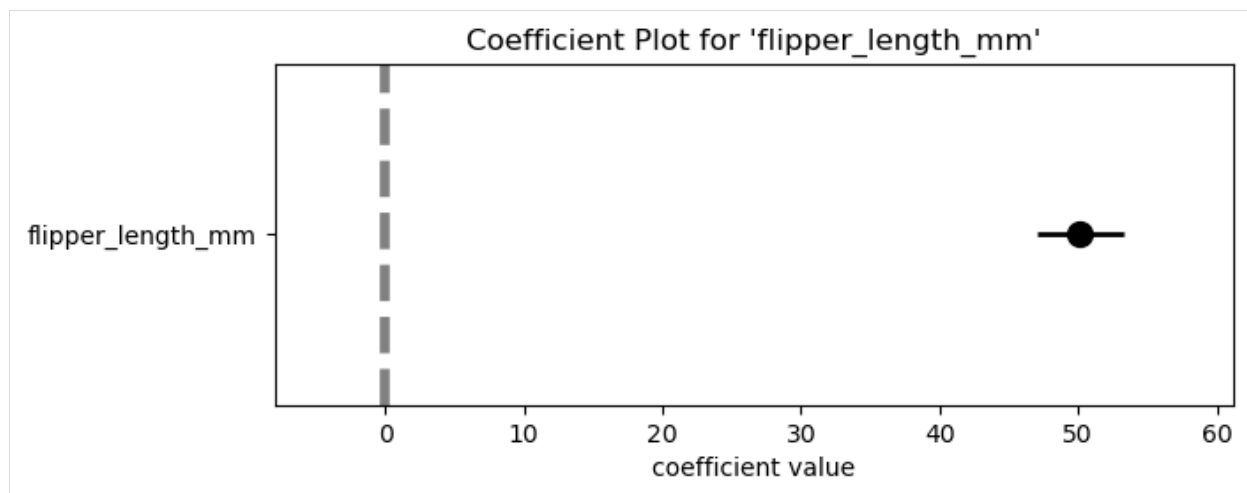
```
lm_fit_01.pvalues < 0.05
```

```
Intercept      True  
flipper_length_mm  True  
dtype: bool
```

```
my_coefplot(lm_fit_01, var='Intercept')
```



```
my_coefplot(lm_fit_01, var='flipper_length_mm')
```



```
lm_fit_01.rsquared
```

```
0.7620921573403914
```

```
np.sqrt((lm_fit_01.resid ** 2).mean())
```

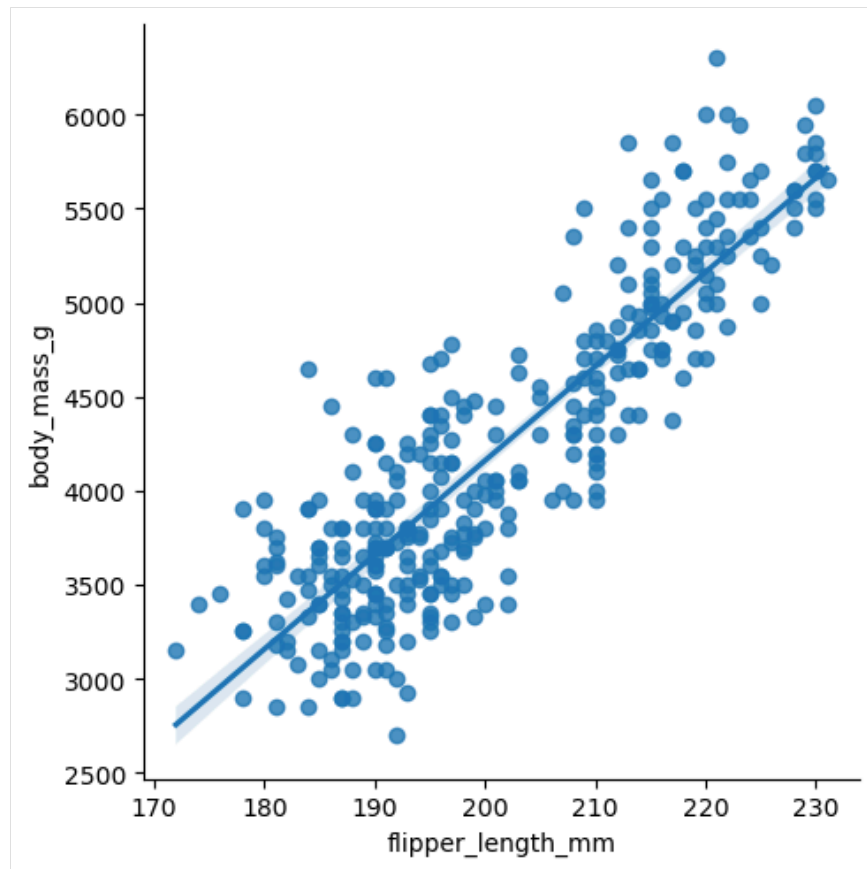
```
392.16027063806183
```

This model has a lower RMSE so it is doing better than the intercept-only model.

```
np.sqrt((lm_fit_01.resid ** 2).mean()) < np.sqrt((lm_fit_00.resid ** 2).mean())
```

True

```
sns.lmplot(data=df_clean,
           x='flipper_length_mm',
           y='body_mass_g')
plt.show()
```



We will try a second degree polynomial or quadratic model.

```
lm_fit_02 = smf.ols(formula='body_mass_g ~ flipper_length_mm + np.power(flipper_length_mm, 2)',
                    data=df_clean). \
                    fit()
```

```
lm_fit_02.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
.....
```

OLS Regression Results

```
=====
```



```

Dep. Variable:    body_mass_g    R-squared:        0.778
Model:            OLS            Adj. R-squared:    0.777
Method:            Least Squares  F-statistic:      578.5
Date:              Thu, 24 Sep 2026  Prob (F-statistic): 1.34e-108
Time:              10:17:38      Log-Likelihood:   -2449.5
No. Observations: 333           AIC:              4905.
Df Residuals:     330           BIC:              4916.
Df Model:         2
Covariance Type:  nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.658e+04	4616.247	3.592	0.000	7498.887	2.57e+04
flipper_length_mm	-171.6140	45.524	-3.770	0.000	-261.169	-82.059
np.power(flipper_length_mm, 2)	0.5449	0.112	4.874	0.000	0.325	0.765

```

=====
Omnibus:          10.746    Durbin-Watson:      2.214
Prob(Omnibus):    0.005    Jarque-Bera (JB):   10.985
Skew:             0.441    Prob(JB):           0.00412
Kurtosis:         3.119    Cond. No.           9.07e+06
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 9.07e+06. This might indicate that there are strong multicollinearity or other numerical problems.

""""

lm_fit_02.params

```

Intercept          16579.869215
flipper_length_mm  -171.613976
np.power(flipper_length_mm, 2)    0.544948
dtype: float64

```

lm_fit_02.bse

```

Intercept          4616.246647
flipper_length_mm  45.524470
np.power(flipper_length_mm, 2)    0.111807
dtype: float64

```

```

lm_fit_02.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})

```

```

              ci_lwr    ci_upr
Intercept    7498.887253 25660.851176
flipper_length_mm  -261.168743 -82.059210
np.power(flipper_length_mm, 2)    0.325003    0.764892

```

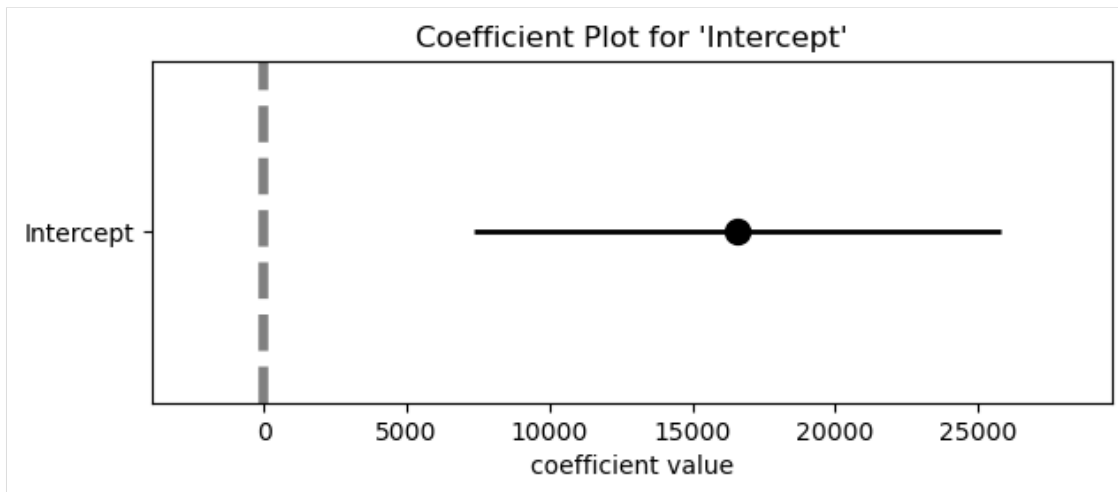
lm_fit_02.pvalues

```
Intercept                0.000379
flipper_length_mm        0.000194
np.power(flipper_length_mm, 2) 0.000002
dtype: float64
```

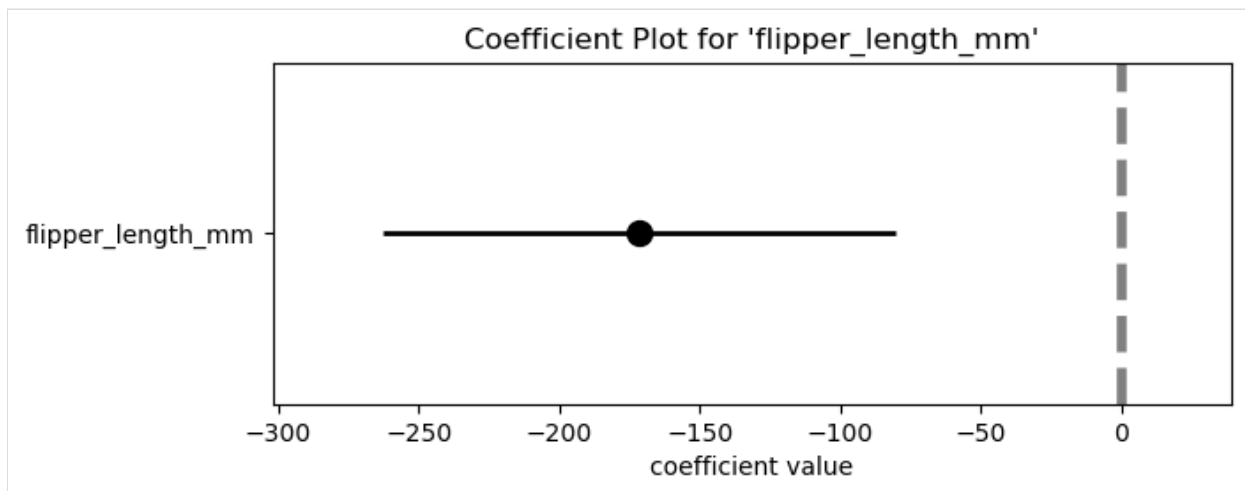
```
lm_fit_02.pvalues < 0.05
```

```
Intercept                True
flipper_length_mm        True
np.power(flipper_length_mm, 2)  True
dtype: bool
```

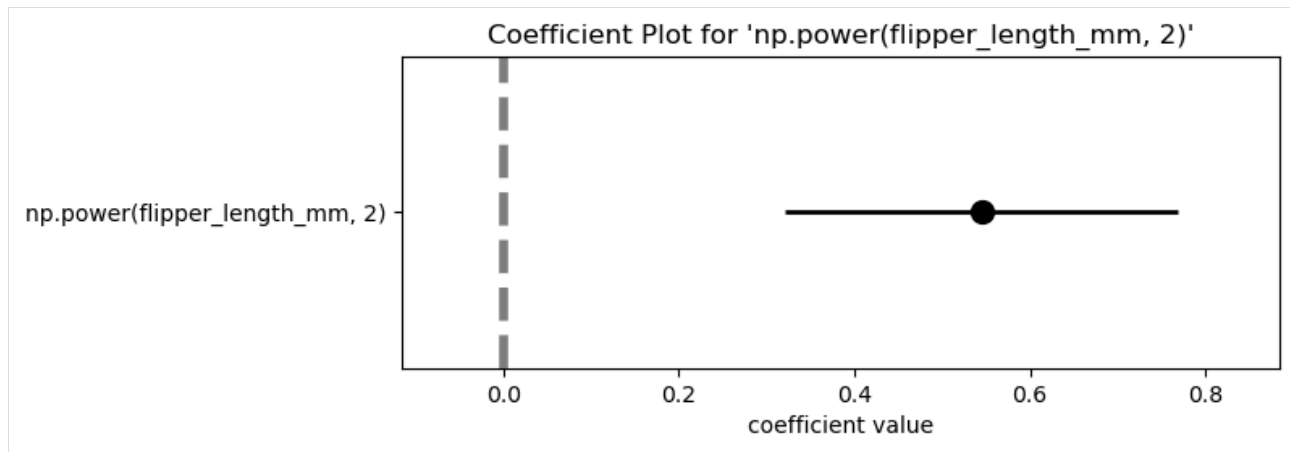
```
my_coefplot(lm_fit_02, var='Intercept')
```



```
my_coefplot(lm_fit_02, var='flipper_length_mm')
```



```
my_coefplot(lm_fit_02, var='np.power(flipper_length_mm, 2)')
```



```
lm_fit_02.rsquared
```

```
0.7780684324476419
```

```
np.sqrt((lm_fit_02.resid ** 2).mean())
```

```
378.7640525050106
```

This model has a lower RMSE than the other two so it is doing better.

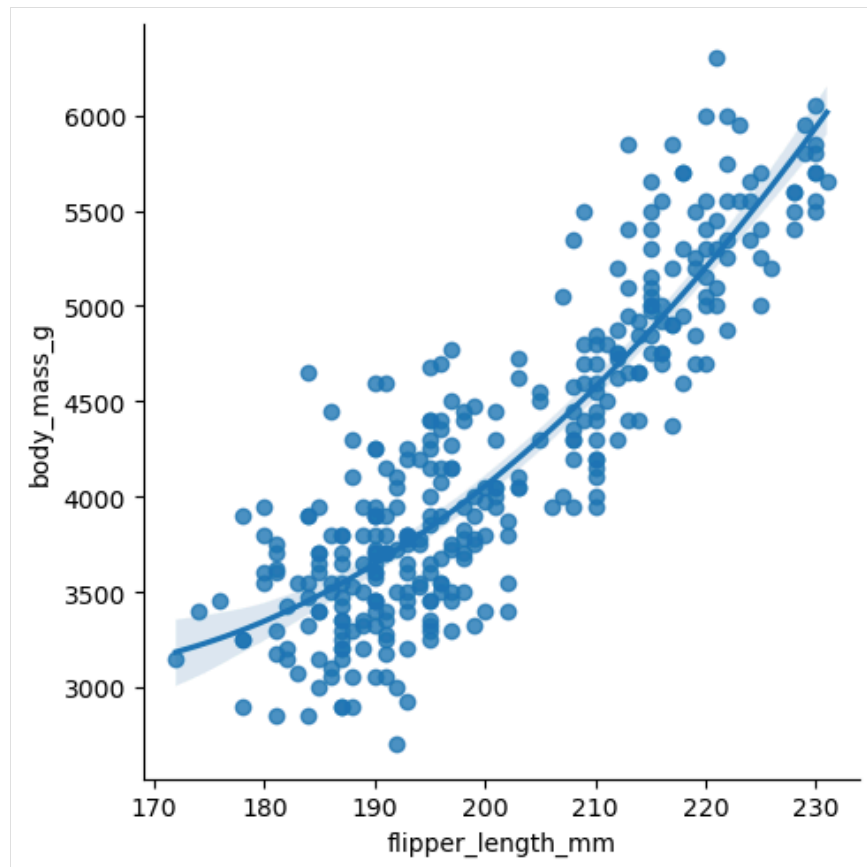
```
np.sqrt((lm_fit_02.resid ** 2).mean()) < np.sqrt((lm_fit_00.resid ** 2).mean())
```

```
True
```

```
np.sqrt((lm_fit_02.resid ** 2).mean()) < np.sqrt((lm_fit_01.resid ** 2).mean())
```

```
True
```

```
sns.lmplot(data=df_clean,
            x='flipper_length_mm',
            y='body_mass_g',
            order=2)
plt.show()
```



We will review a cubic model.

```
lm_fit_03 = smf.ols(formula='body_mass_g ~ flipper_length_mm + np.power(flipper_length_mm, 2) +
np.power(flipper_length_mm, 3)',
                    data=df_clean). \
fit()
```

```
lm_fit_03.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
.....
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          body_mass_g      R-squared:                0.784
Model:                  OLS              Adj. R-squared:           0.782
Method:                 Least Squares    F-statistic:               397.5
Date:                   Thu, 24 Sep 2026  Prob (F-statistic):       5.07e-109
Time:                   10:17:38         Log-Likelihood:            -2445.2
No. Observations:       333              AIC:                     4898.
Df Residuals:           329              BIC:                     4914.
Df Model:                3
Covariance Type:        nonrobust

=====
                        coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept              1.795e+05  5.55e+04     3.232     0.001     7.02e+04  2.89e+05
flipper_length_mm      -2591.3912   823.371    -3.147     0.002    -4211.126  -971.656
=====
```

```
np.power(flipper_length_mm, 2)    12.4909    4.060    3.076    0.002    4.504    20.478
np.power(flipper_length_mm, 3)   -0.0196    0.007   -2.943    0.003   -0.033   -0.006
```

```
=====
Omnibus:                11.383    Durbin-Watson:                2.320
Prob(Omnibus):          0.003    Jarque-Bera (JB):            11.688
Skew:                   0.454    Prob(JB):                    0.00290
Kurtosis:               3.130    Cond. No.                    2.27e+10
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 2.27e+10. This might indicate that there are strong multicollinearity or other numerical problems.
 """"

```
lm_fit_03.params
```

```
Intercept                179467.306901
flipper_length_mm        -2591.391180
np.power(flipper_length_mm, 2)    12.490930
np.power(flipper_length_mm, 3)   -0.019598
dtype: float64
```

```
lm_fit_03.bse
```

```
Intercept                55530.217107
flipper_length_mm         823.370574
np.power(flipper_length_mm, 2)    4.060253
np.power(flipper_length_mm, 3)    0.006659
dtype: float64
```

```
lm_fit_03.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

```
                ci_lwr    ci_upr
Intercept    70228.225861  288706.387940
flipper_length_mm  -4211.126337  -971.656024
np.power(flipper_length_mm, 2)    4.503598    20.478261
np.power(flipper_length_mm, 3)   -0.032697   -0.006499
```

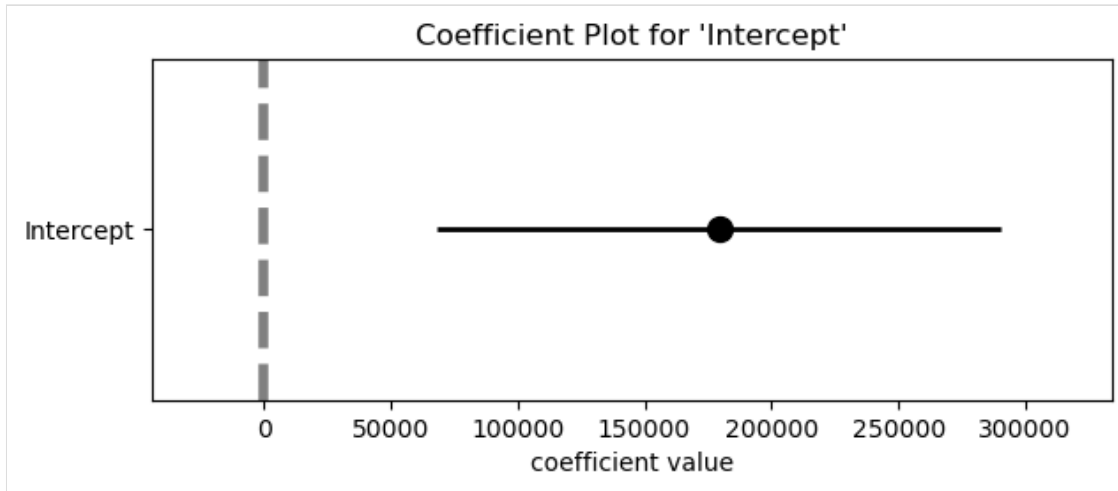
```
lm_fit_03.pvalues
```

```
Intercept                0.001354
flipper_length_mm        0.001799
np.power(flipper_length_mm, 2)    0.002271
np.power(flipper_length_mm, 3)    0.003478
dtype: float64
```

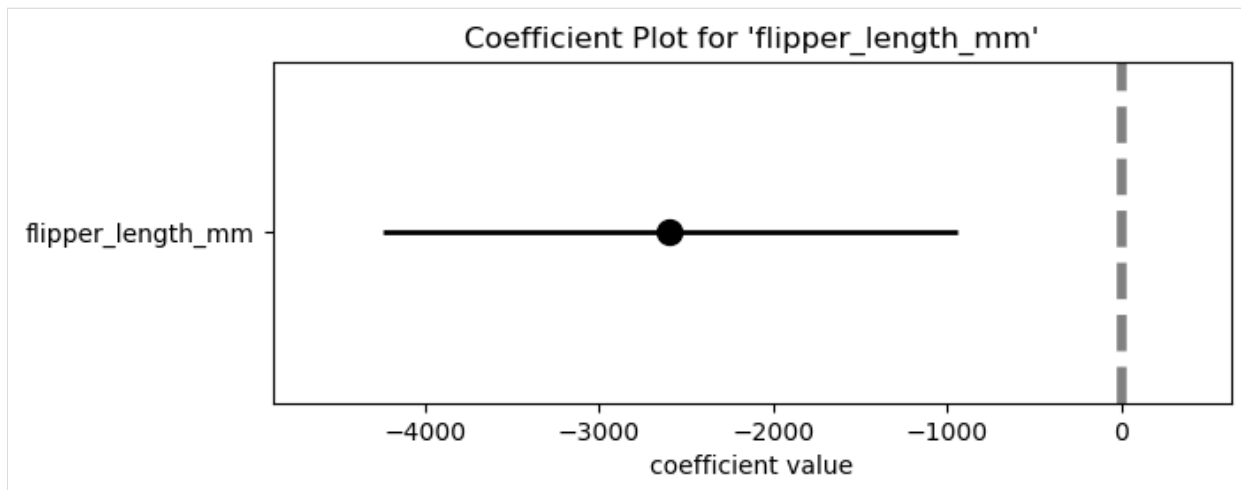
```
lm_fit_03.pvalues < 0.05
```

```
Intercept                True
flipper_length_mm        True
np.power(flipper_length_mm, 2)  True
np.power(flipper_length_mm, 3)  True
dtype: bool
```

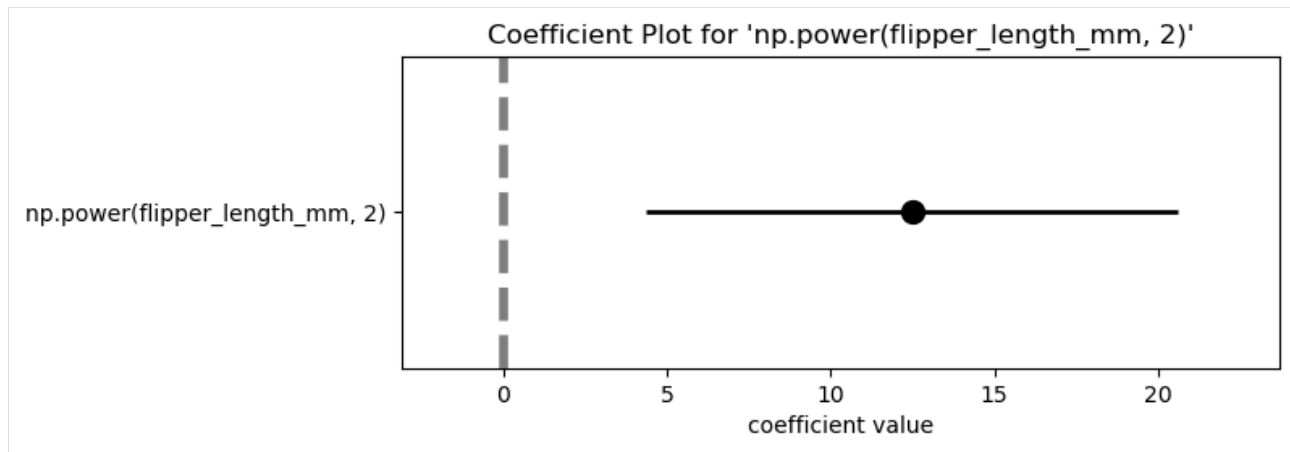
```
my_coefplot(lm_fit_03, var='Intercept')
```



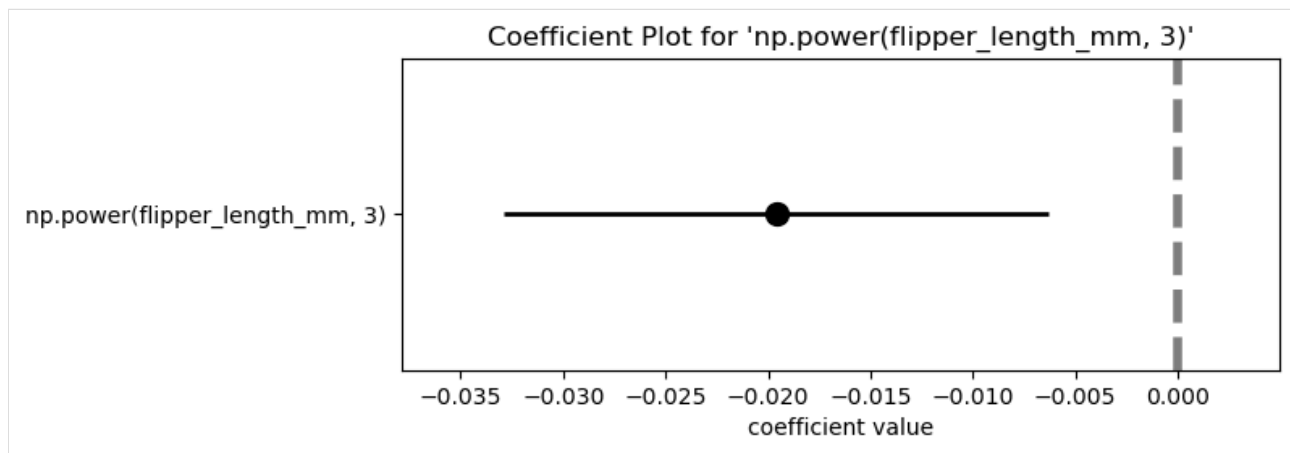
```
my_coefplot(lm_fit_03, var='flipper_length_mm')
```



```
my_coefplot(lm_fit_03, var='np.power(flipper_length_mm, 2)')
```



```
my_coefplot(lm_fit_03, var='np.power(flipper_length_mm, 3)')
```



```
lm_fit_03.rsquared
```

```
0.7837621428152766
```

This model has a lower RMSE than the other three so it is doing best so far.

```
np.sqrt((lm_fit_03.resid ** 2).mean()) < np.sqrt((lm_fit_00.resid ** 2).mean())
```

```
True
```

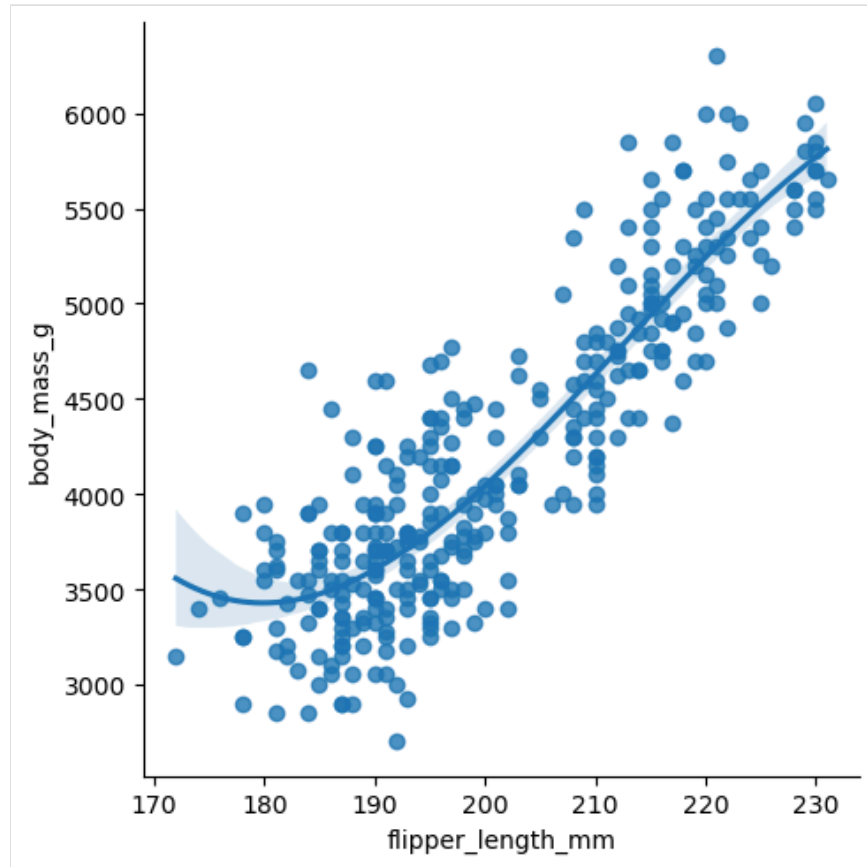
```
np.sqrt((lm_fit_03.resid ** 2).mean()) < np.sqrt((lm_fit_01.resid ** 2).mean())
```

```
True
```

```
np.sqrt((lm_fit_03.resid ** 2).mean()) < np.sqrt((lm_fit_02.resid ** 2).mean())
```

True

```
sns.lmplot(data=df_clean,  
           x='flipper_length_mm',  
           y='body_mass_g',  
           order=3)  
plt.show()
```



Key Takeaways

- Fit many candidates and compare them systematically.
- A comparison table turns model selection into evidence.
- Prefer the simplest model that performs nearly as well.
- Write down why you chose the model you chose.

Exercises

1. Fit a model for every continuous feature and build a comparison table.
2. Sort the models by held-out RMSE and by R-squared and compare the rankings.
3. Choose a final model and write two sentences justifying it.
4. Add a categorical feature and see whether the comparison changes.

Guided Walkthrough

The notebook fits one model per candidate feature and collects the results into a comparison table. Sorting the table by held-out error ranks the features by predictive value. The lesson is procedural: model selection is fitting many candidates and letting evidence choose, not picking one model and hoping.

Common Pitfalls

- Cherry-picking the feature that looks best on training data.
- Ignoring simplicity when accuracy is nearly tied.
- Failing to document why a model was chosen.

Going Further

- Add engineered features to the comparison.
- Use cross-validation instead of a single split for a more stable ranking.

Exercise Solutions

1. The comparison table lists each feature with its R-squared and RMSE.
2. The rankings by R-squared and RMSE usually agree but can differ near ties.
3. Justify the final model by its held-out performance and its simplicity.
4. Adding a categorical feature can improve the fit and change the ranking.

Part III: Multiple Linear Regression and Linear Regression

Chapter 12: Multiple Linear Regression - Additive Features

Introduction

Most real outcomes depend on more than one input. Multiple linear regression extends the single-feature model by letting several predictors add their effects together. This chapter introduces additive features: each input contributes its own amount to the prediction, and the contributions simply sum. The geometric picture is a line that shifts up or down as the other features change.

What You Will Learn

- How a multiple regression model combines several predictors.
- How to read a coefficient as the effect of one feature holding the others fixed.
- How additive features shift the fitted relationship up or down.
- How to fit and interpret a multiple regression with the formula interface.

Overview

We fit a model with several continuous predictors and interpret each coefficient carefully. The phrase "holding the others fixed" is the heart of multiple regression, and it is the phrase people most often forget. A coefficient is not the effect of a feature in isolation; it is the effect of that feature once the rest of the model has accounted for the others. We then visualize how changing one feature shifts the predicted output while the others stay constant, which makes the additive structure visible.

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Understanding How Additive Features Affect Data

Additive features add the effect of input 2 to the effect of input 1.

$$\mu = \beta_0 + \beta_1 \times x_1 + \beta_2 \times x_2$$

μ predicted trend

β_0 intercept: value of μ when $x = 0$

β_1 slope: change in μ for a one-unit increase in x_1

x_1 feature or input variable: one of the independent variables used to predict μ

β_2 slope: change in μ for a one-unit increase in x_2

x_2 feature or input variable: one of the independent variables used to predict μ

Additive linear relationships is where we calculate the average output or trend given input 1 and input 2 and the 3 regression coefficients as we will create a function that will allow visualization of the trend

with respect to input 1.

```
def calc_trend_wrt_x1(x1,
                     x2,
                     b0,
                     b1,
                     b2):
    """
    Calculates the linear trend based on two independent variables x1 and x2, using the provided
    coefficients.

    Args:
        x1 (array-like): Values for the first independent variable.
        x2 (array-like): Values for the second independent variable.
        b0 (float): Intercept coefficient.
        b1 (float): Coefficient for x1.
        b2 (float): Coefficient for x2.

    Returns:
        pandas.DataFrame: DataFrame containing columns for x1, x2, and the calculated trend.
    """
    res_df = pd.DataFrame({'x1': x1})
    res_df['x2'] = x2
    res_df['trend'] = b0 + b1 * res_df.x1 + b2 * res_df.x2
    return res_df
```

To demonstrate the relationships, let's define the following coefficients.

```
b0 = -0.25
b1 = 1.95
b2 = 0.2
```

Let's define 101 evenly or uniformly spaced values of `x1` between -3 and 3.

```
x1_values = np.linspace(-3,
                        3,
                        101)
```

```
x1_values
```

```
array([-3.    , -2.94, -2.88, -2.82, -2.76, -2.7 , -2.64, -2.58, -2.52,
       -2.46, -2.4 , -2.34, -2.28, -2.22, -2.16, -2.1 , -2.04, -1.98,
       -1.92, -1.86, -1.8 , -1.74, -1.68, -1.62, -1.56, -1.5 , -1.44,
       -1.38, -1.32, -1.26, -1.2 , -1.14, -1.08, -1.02, -0.96, -0.9 ,
       -0.84, -0.78, -0.72, -0.66, -0.6 , -0.54, -0.48, -0.42, -0.36,
       -0.3 , -0.24, -0.18, -0.12, -0.06,  0.   ,  0.06,  0.12,  0.18,
        0.24,  0.3 ,  0.36,  0.42,  0.48,  0.54,  0.6 ,  0.66,  0.72,
        0.78,  0.84,  0.9 ,  0.96,  1.02,  1.08,  1.14,  1.2 ,  1.26,
        1.32,  1.38,  1.44,  1.5 ,  1.56,  1.62,  1.68,  1.74,  1.8 ,
        1.86,  1.92,  1.98,  2.04,  2.1 ,  2.16,  2.22,  2.28,  2.34,
        2.4 ,  2.46,  2.52,  2.58,  2.64,  2.7 ,  2.76,  2.82,  2.88,
        2.94,  3.   ])
```

```
x1_values.size
```

```
101
```

```
x1_values.ndim
```

```
1
```

Let's calculate the average output for a single value of `x2` at 0 where `x2` has 1 unique value 0, and it appears 101 times.

```
calc_trend_wrt_x1(x1_values,
                  0,
                  b0,
                  b1,
                  b2)
```

```

      x1  x2  trend
0  -3.00  0 -6.100
1  -2.94  0 -5.983
2  -2.88  0 -5.866
3  -2.82  0 -5.749
4  -2.76  0 -5.632
..    ...  ..    ...
96   2.76  0  5.132
97   2.82  0  5.249
98   2.88  0  5.366
99   2.94  0  5.483
100  3.00  0  5.600

[101 rows x 3 columns]
```

```
calc_trend_wrt_x1(x1_values,
                  0,
                  b0,
                  b1,
                  b2). \
x2.value_counts()
```

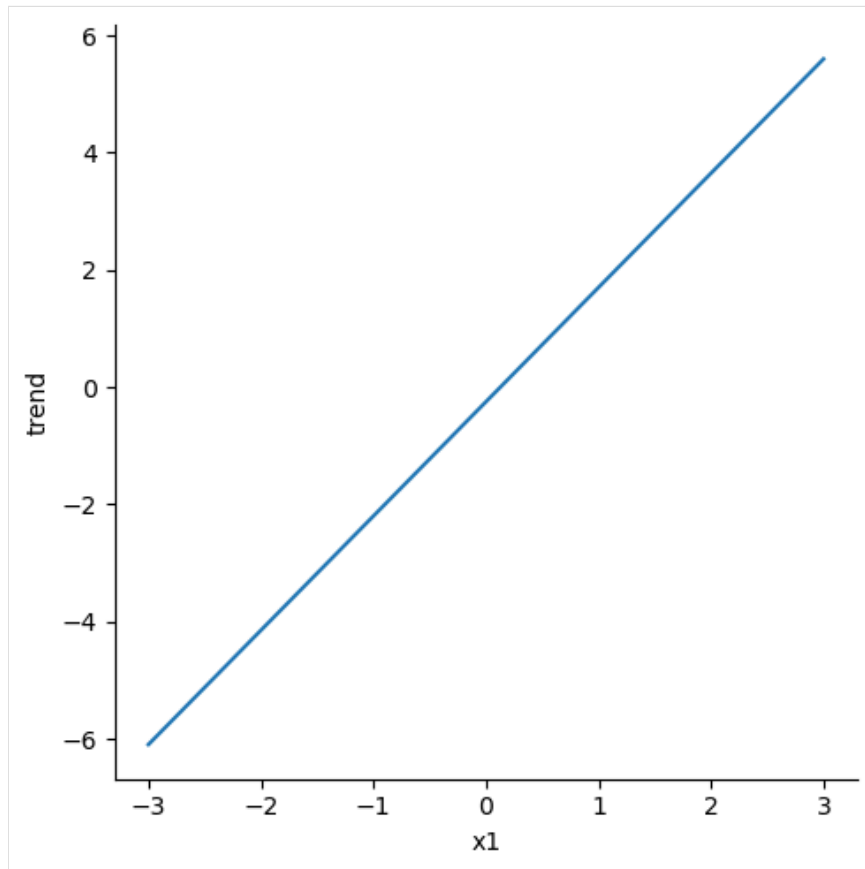
```

x2
0    101
Name: count, dtype: int64
```

Let's visualize the relationship between the average output and input 1 where the average output has roughly a change of 2 on the trend for a 1 unit change on the input as the slope is how much does the output change on average for a one unit change of the input.

```
sns.relplot(data=calc_trend_wrt_x1(x1_values,
                                   0,
                                   b0,
                                   b1,
                                   b2),
            x='x1',
            y='trend',
```

```
plt.show()                                kind='line')
```



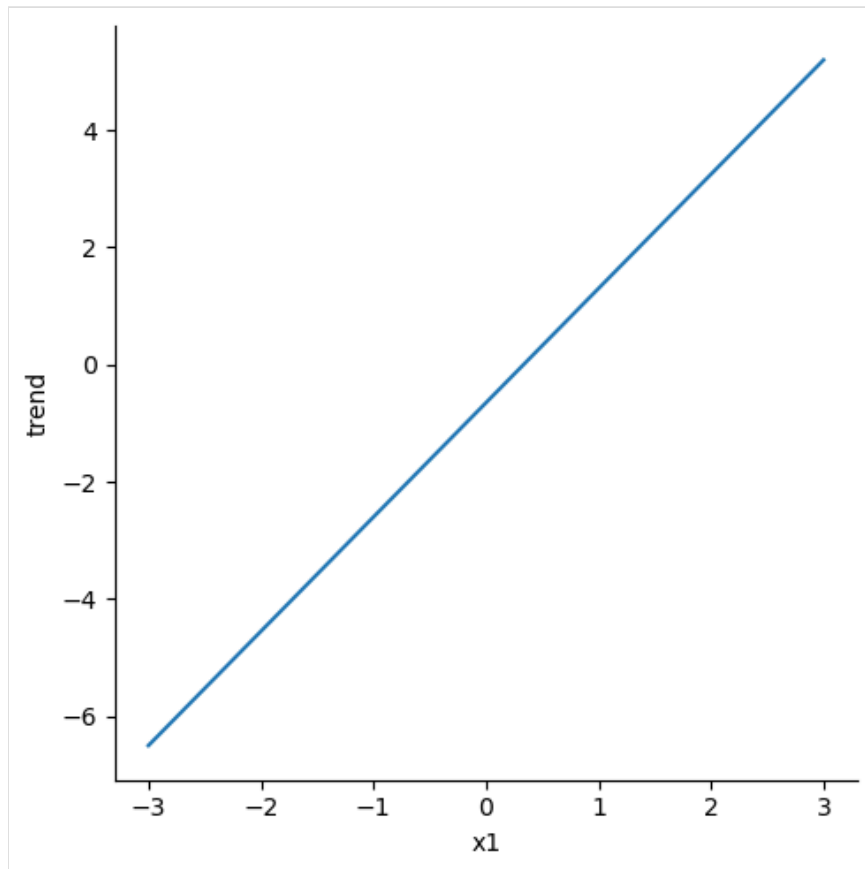
Now we will change `x2` from 0 to -2 as we can visualize the new trend as a 1 unit change in the input produces an almost 2 unit change on the trend but it is hard to see any difference from the prior graphic.

```
calc_trend_wrt_x1(x1_values,  
                  -2,  
                  b0,  
                  b1,  
                  b2)
```

```
      x1  x2  trend  
0  -3.00 -2 -6.500  
1  -2.94 -2 -6.383  
2  -2.88 -2 -6.266  
3  -2.82 -2 -6.149  
4  -2.76 -2 -6.032  
..   ... ..   ...  
96  2.76 -2  4.732  
97  2.82 -2  4.849  
98  2.88 -2  4.966  
99  2.94 -2  5.083  
100 3.00 -2  5.200  
  
[101 rows x 3 columns]
```

```
sns.relplot(data=calc_trend_wrt_x1(x1_values,
                                   -2,
                                   b0,
                                   b1,
                                   b2),
            x='x1',
            y='trend',
            kind='line')

plt.show()
```



Let's repeat or replicate calculating the trend with respect to `x1` for different values of `x2`.

```
x2_values = np.linspace(-3,
                        3,
                        9)
```

```
x2_values
```

```
array([-3. , -2.25, -1.5 , -0.75,  0.  ,  0.75,  1.5 ,  2.25,  3.  ])
```

```
study_wrt_x1_list = [calc_trend_wrt_x1(x1_values, x2, b0, b1, b2) for x2 in x2_values]
```

```
len(study_wrt_x1_list)
```

9

study_wrt_x1_list[0]

```

      x1  x2  trend
0  -3.00 -3.0 -6.700
1  -2.94 -3.0 -6.583
2  -2.88 -3.0 -6.466
3  -2.82 -3.0 -6.349
4  -2.76 -3.0 -6.232
..    ...  ...    ...
96   2.76 -3.0  4.532
97   2.82 -3.0  4.649
98   2.88 -3.0  4.766
99   2.94 -3.0  4.883
100  3.00 -3.0  5.000

[101 rows x 3 columns]
```

Combine or concat the separate DataFrames together.

```
study_wrt_x1_df = pd.concat(study_wrt_x1_list, ignore_index=True)
```

study_wrt_x1_df

```

      x1  x2  trend
0  -3.00 -3.0 -6.700
1  -2.94 -3.0 -6.583
2  -2.88 -3.0 -6.466
3  -2.82 -3.0 -6.349
4  -2.76 -3.0 -6.232
..    ...  ...    ...
904  2.76  3.0  5.732
905  2.82  3.0  5.849
906  2.88  3.0  5.966
907  2.94  3.0  6.083
908  3.00  3.0  6.200

[909 rows x 3 columns]
```

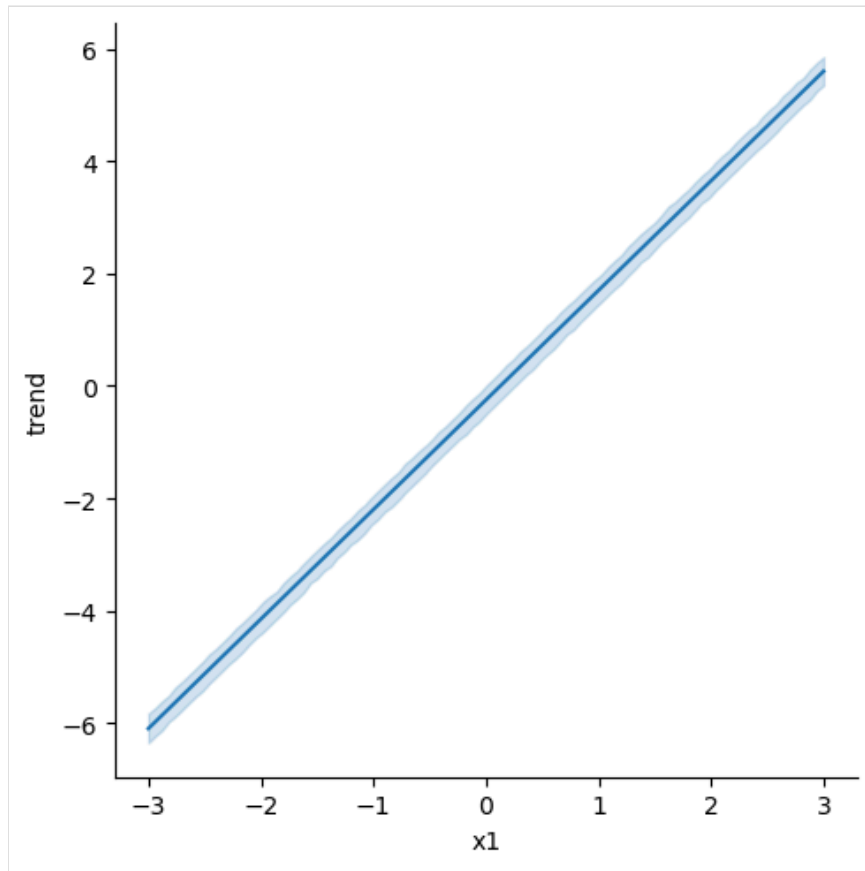
```
study_wrt_x1_df.x2.value_counts()
```

```

x2
-3.00    101
-2.25    101
-1.50    101
-0.75    101
 0.00    101
 0.75    101
 1.50    101
 2.25    101
 3.00    101
Name: count, dtype: int64
```

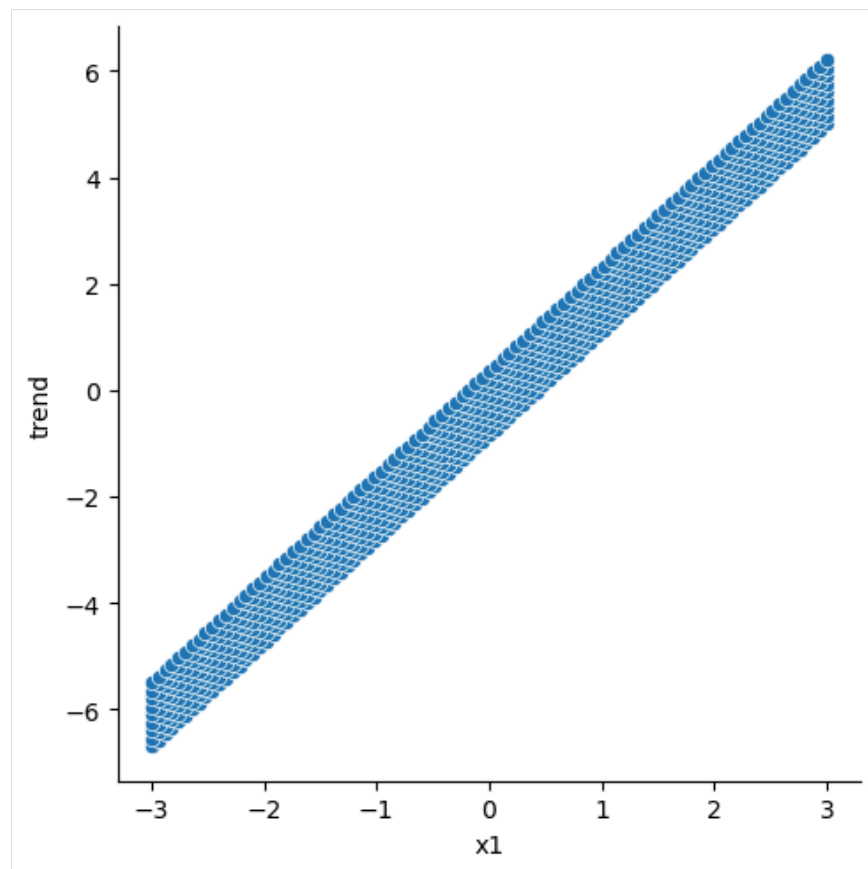

Visualize the trend or average output with respect to `x1` for each unique value of `x2` as a line chart as it will include the 95% confidence interval as this is a summation of the data.

```
sns.relplot(data=study_wrt_x1_df,  
            x='x1',  
            y='trend',  
            kind='line')  
plt.show()
```



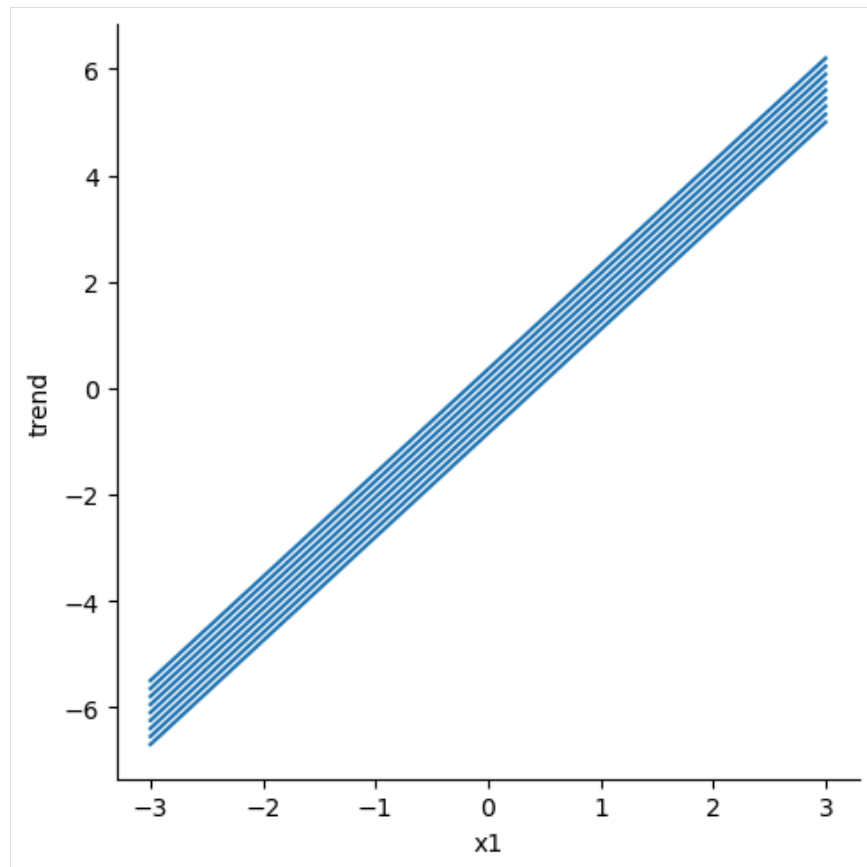
Visualize the raw data.

```
sns.relplot(data=study_wrt_x1_df,  
            x='x1',  
            y='trend',  
            kind='scatter')  
plt.show()
```



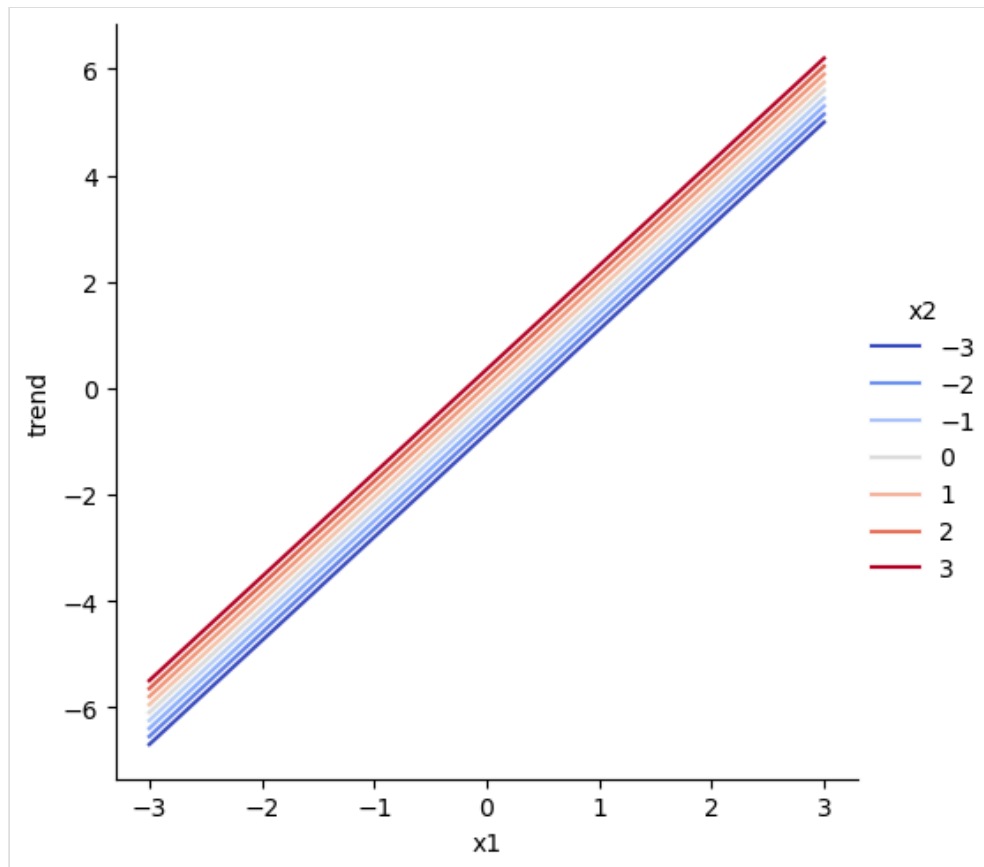
We need to disable Seaborn's default line chart averaging.

```
sns.relplot(data=study_wrt_x1_df,  
            x='x1',  
            y='trend',  
            kind='line',  
            estimator=None,  
            units='x2')  
plt.show()
```



We can associate the line color to `x2` via the `hue` argument.

```
sns.relplot(data=study_wrt_x1_df,  
            x='x1',  
            y='trend',  
            hue='x2',  
            kind='line',  
            palette='coolwarm',  
            estimator=None,  
            units='x2')  
plt.show()
```



WHEN WE HAVE ADDITIVE FEATURES WHICH ARE ADDING THE EFFECT OF INPUT 2 TO THE EFFECT OF INPUT 1, WE ARE LITERALLY SHIFTING THE LINE UP OR DOWN.

Additive features add the effect of one input to the other and if the inputs have the same magnitude of and scale, the magnitude of the slope tells you which input causes the greater change on the average output.

b1

1.95

In the above we have the slope β_1 very close to 2, therefore, a 1 unit change in x_1 is producing a 2 unit change in the trend or average output.

Let's focus on the relationship with respect to x_2 .

```
def calc_trend_wrt_x2(x1,
                      x2,
                      b0,
                      b1,
                      b2):
    """
    Calculates the linear trend based on two independent variables x1 and x2, using the provided
    coefficients.

    Args:
        x1 (array-like): Values for the first independent variable.
```

x2 (array-like): Values for the second independent variable.
 b0 (float): Intercept coefficient.
 b1 (float): Coefficient for x1.
 b2 (float): Coefficient for x2.

Returns:

pandas.DataFrame: DataFrame containing columns for x1, x2, and the calculated trend.

"""

```

res_df = pd.DataFrame({'x2': x2})
res_df['x1'] = x1
res_df['trend'] = b0 + b1 * res_df.x1 + b2 * res_df.x2
return res_df

```

Let's define 2 new arrays so we can visualize the trends with respect to `x2` for different values of `x1`.

```
x2_values_b = np.linspace(-3, 3, 101)
```

```
x2_values_b.shape
```

```
(101,)
```

```
x1_values_b = np.linspace(-3, 3, 9)
```

```
x1_values_b.shape
```

```
(9,)
```

Iterate over values `x1`.

```
study_wrt_x2_list = [calc_trend_wrt_x2(x1, x2_values_b, b0, b1, b2) for x1 in x1_values_b]
```

```
len(study_wrt_x2_list)
```

```
9
```

```
study_wrt_x2_df = pd.concat(study_wrt_x2_list, ignore_index=True)
```

```
study_wrt_x2_df
```

```

      x2  x1  trend
0 -3.00 -3.0 -6.700
1 -2.94 -3.0 -6.688
2 -2.88 -3.0 -6.676
3 -2.82 -3.0 -6.664
4 -2.76 -3.0 -6.652
..   ...  ...   ...

```

```
904  2.76  3.0  6.152
905  2.82  3.0  6.164
906  2.88  3.0  6.176
907  2.94  3.0  6.188
908  3.00  3.0  6.200
```

```
[909 rows x 3 columns]
```

```
study_wrt_x2_df.x1.value_counts()
```

```
x1
-3.00    101
-2.25    101
-1.50    101
-0.75    101
 0.00    101
 0.75    101
 1.50    101
 2.25    101
 3.00    101
Name: count, dtype: int64
```

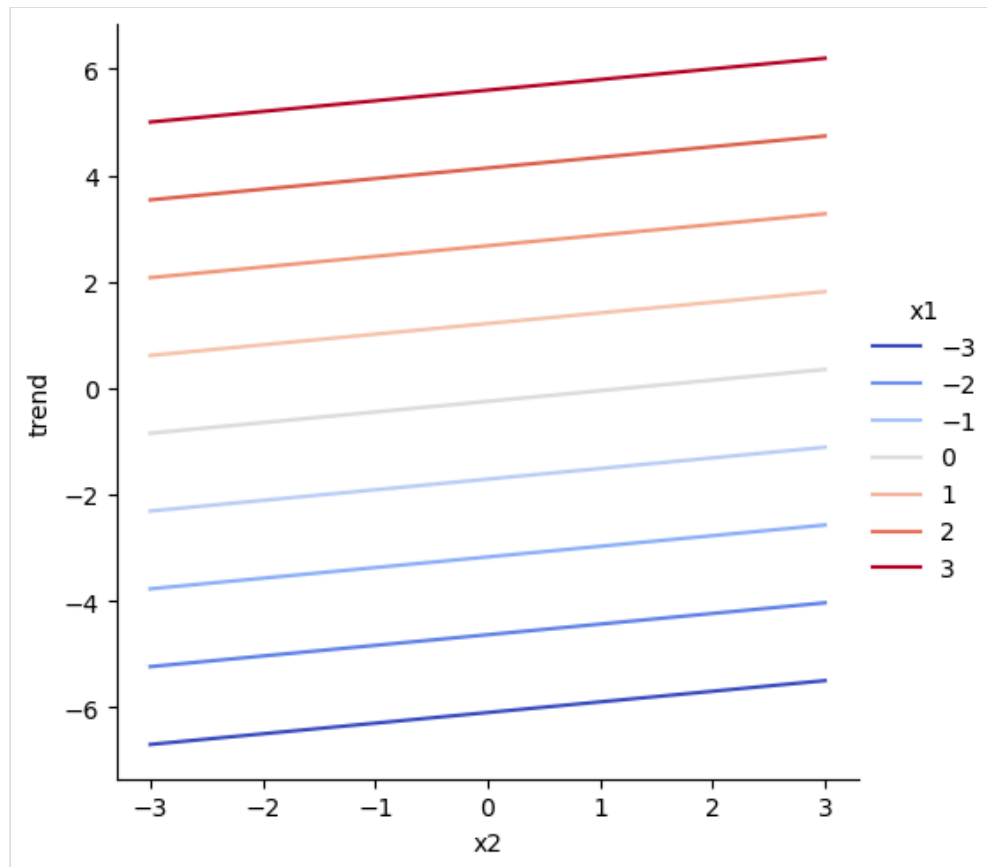
Visualize the trend or average output with respect to `x2` for each unique value of `x1` as we will use the diverging color palette to help distinguish `x1` values above and below the `x1` midpoint.

```
b2
```

```
0.2
```

In the above we have the slope β_2 0.2, therefore, a 1 unit change in `x2` is producing a very small change in the trend or average output.

```
sns.relplot(data=study_wrt_x2_df,
            x='x2',
            y='trend',
            hue='x1',
            kind='line',
            palette='coolwarm',
            estimator=None,
            units='x1')
plt.show()
```



Key Takeaways

- Multiple regression adds the effects of several features.
- Each coefficient is interpreted holding the other features fixed.
- Additive features shift the prediction up or down.
- Correlated predictors make individual coefficients harder to interpret.

Exercises

1. Fit a model with three continuous predictors and interpret each coefficient.
2. Plot the predicted output as one feature varies and the others are fixed.
3. Compare a coefficient in the multiple model to the same feature alone.
4. Explain why the coefficients can change when predictors are correlated.

Guided Walkthrough

The notebook fits a model with several continuous predictors and then interprets each coefficient while holding the others fixed. It plots the prediction as one feature varies and the others stay constant, which makes the additive structure visible: changing a feature shifts the whole prediction up or down. The notebook also shows that coefficients can change when predictors are correlated, because the model redistributes credit among features that move together.

Common Pitfalls

- Reading a coefficient as a standalone effect rather than a partial effect.
- Ignoring multicollinearity, which inflates coefficient variance.
- Comparing coefficients across features measured in different units.

Going Further

- Standardize the predictors and compare the standardized coefficients.
- Compute variance inflation factors and flag correlated predictors.

Exercise Solutions

1. Each coefficient is the change in the prediction for a one-unit change in that feature, others fixed.
2. The plot shows the prediction moving up or down as the chosen feature changes.
3. A coefficient in the multiple model often differs from the single-feature model because other predictors absorb some effect.
4. Correlated predictors share explanatory power, so their individual coefficients become unstable.

Chapter 13: Multiple Linear Regression - Interactive Features

Introduction

Additive features shift a prediction; interacting features rotate it. An interaction term lets the effect of one input depend on the value of another, which is how regression captures context. This chapter adds interactions to the Penguins model and shows how they change the slope rather than the height of the fitted relationship.

What You Will Learn

- What an interaction term represents.
- How interactions rotate the fitted relationship and change slopes.
- How to add an interaction with the formula interface.
- When an interaction is worth the added complexity.

Overview

We add a term that multiplies two predictors and watch the fitted relationship change. With an interaction, the slope of one feature is no longer a single number; it depends on the other feature. We visualize this by plotting the prediction against one input at several fixed values of the other, producing a family of lines with different slopes. The chapter closes by weighing the interpretive cost of interactions against the gain in fit.

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Understanding How Interactive Features Affect Data

Interactive features include the linear main effects of each input and also the product of the two inputs, which serves as the interaction term.

The interaction feature captures how the effect on the output depends on both input variables simultaneously.

$$\mu = \beta_0 + \beta_1 \times x_1 + \beta_2 \times x_2 + \beta_3 \times x_1 \times x_2$$

μ **predicted trend**

β_0 **intercept:** value of μ when $x = 0$

β_1 **slope:** change in μ for a one-unit increase in x_1

x_1 **feature or input variable:** one of the independent variables used to predict μ

β_2 **slope:** change in μ for a one-unit increase in x_2

x_2 **feature or input variable:** one of the independent variables used to predict μ

β_3 **slope**: change in μ for a one-unit increase in the product $x_1 \times x_2$

Let's define a function that calculates the trend or average output focusing on the relationship with x_1 where we will have the interaction features which equals the product of the two inputs.

```
def calc_trend_wrt_x1_w_interactions(x1,
                                     x2,
                                     b0,
                                     b1,
                                     b2,
                                     b3):
    """
    Calculates the linear trend based on two independent variables x1 and x2, using the provided
    coefficients.

    Args:
        x1 (array-like): Values for the first independent variable.
        x2 (array-like): Values for the second independent variable.
        b0 (float): Intercept coefficient.
        b1 (float): Coefficient for x1.
        b2 (float): Coefficient for x2.
        b3 (float): Coefficient for the interaction term (x1 * x2).

    Returns:
        pandas.DataFrame: DataFrame containing columns for x1, x2, and the calculated trend.
    """
    res_df = pd.DataFrame({'x1': x1})
    res_df['x2'] = x2
    res_df['trend'] = b0 + \
        b1 * \
            res_df.x1 + \
        b2 * \
            res_df.x2 + \
        b3 * \
            res_df.x1 * \
            res_df.x2

    return res_df
```

We need a slope defined that multiplies the interaction feature.

```
b0 = -0.25
b1 = 1.95
b2 = 0.2
b3 = 1
```

```
x1_values = np.linspace(-3,
                        3,
                        101)
```

```
x2_values = np.linspace(-3,
                        3,
                        9)
```

Let's calculate the trend with respect to x_1 for different values of x_2 .

```
study_interaction_wrt_x1_list = [calc_trend_wrt_x1_w_interactions(x1_values, x2, b0, b1, b2, b3) for
                                x2 in x2_values]
```

```
len(study_interaction_wrt_x1_list)
```

```
9
```

```
study_interaction_wrt_x1_df = pd.concat(study_interaction_wrt_x1_list, ignore_index=True)
```

```
study_interaction_wrt_x1_df
```

```

      x1  x2  trend
0  -3.00 -3.0  2.300
1  -2.94 -3.0  2.237
2  -2.88 -3.0  2.174
3  -2.82 -3.0  2.111
4  -2.76 -3.0  2.048
..    ...  ...    ...
904  2.76  3.0 14.012
905  2.82  3.0 14.309
906  2.88  3.0 14.606
907  2.94  3.0 14.903
908  3.00  3.0 15.200

```

```
[909 rows x 3 columns]
```

```
study_interaction_wrt_x1_df.x2.value_counts()
```

```

x2
-3.00    101
-2.25    101
-1.50    101
-0.75    101
 0.00    101
 0.75    101
 1.50    101
 2.25    101
 3.00    101
Name: count, dtype: int64

```

Let's visualize the average output or trend with respect to `x1` for each `x2` unique value.

We still have 9 unique values of `x2` however the slopes are not shifting up or down as we now have different slopes.

When `x2` is -3, the trend or average output is decreasing for a 1 unit change of input 1 and when `x2` is 3, the trend or average output is increasing for a 1 unit change of input 1.

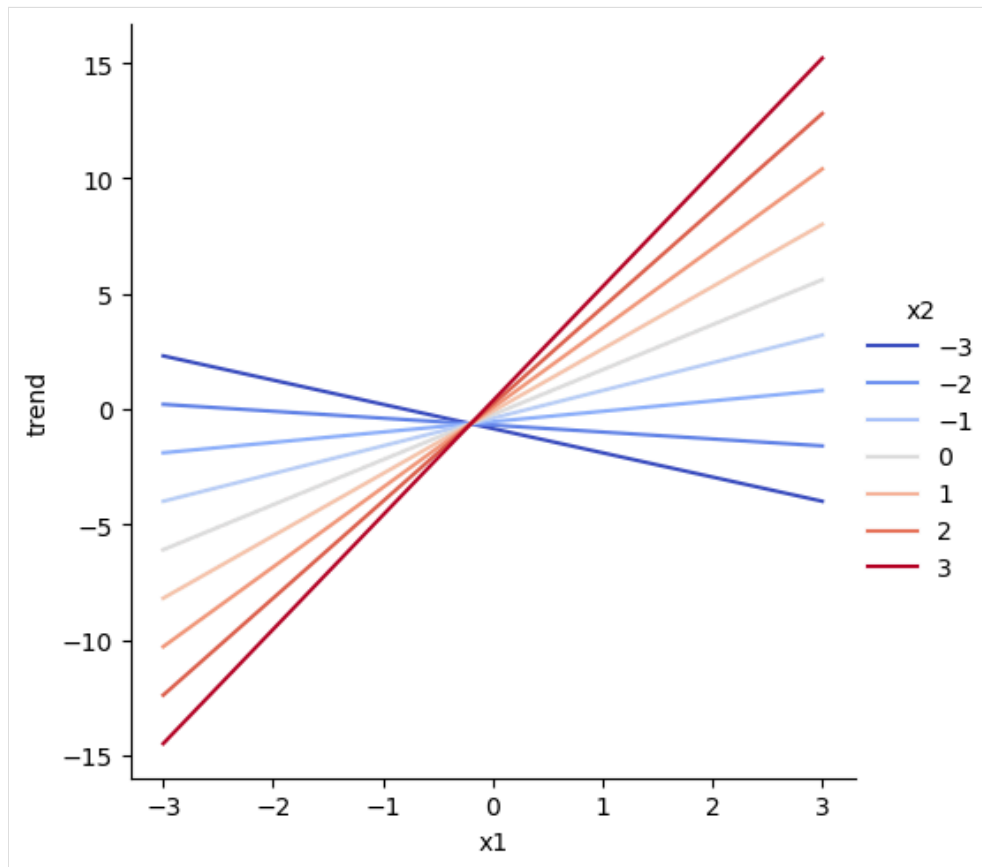
```
b1
```

```
1.95
```

We know β_1 is > 0 , positive, and yet we have a negative slope, however, when x_2 is 3 we have a positive slope.

The change when x_2 is 3, it is greater than β_{x1} .

```
sns.relplot(data=study_interaction_wrt_x1_df,
            x='x1',
            y='trend',
            hue='x2',
            kind='line',
            palette='coolwarm',
            estimator=None,
            units='x2')
plt.show()
```



An interaction represents that the relationship with respect to 1 input depends on the other input and the same idea holds for the relationship with respect to x_2 .

```
def calc_trend_wrt_x2_w_interactions(x1,
                                     x2,
                                     b0,
                                     b1,
                                     b2,
                                     b3):
    """
    Calculates the linear trend based on two independent variables x1 and x2, using the provided
    coefficients.
```

```

Args:
    x1 (array-like): Values for the first independent variable.
    x2 (array-like): Values for the second independent variable.
    b0 (float): Intercept coefficient.
    b1 (float): Coefficient for x1.
    b2 (float): Coefficient for x2.
    b3 (float): Coefficient for the interaction term (x1 * x2).

Returns:
    pandas.DataFrame: DataFrame containing columns for x1, x2, and the calculated trend.
"""
res_df = pd.DataFrame({'x2': x2})
res_df['x1'] = x1
res_df['trend'] = b0 + \
    b1 * \
    res_df.x1 + \
    b2 * \
    res_df.x2 + \
    b3 * \
    res_df.x1 * \
    res_df.x2

return res_df

```

```
x2_values_b = np.linspace(-3, 3, 101)
```

```
x1_values_b = np.linspace(-3, 3, 9)
```

```
study_interaction_wrt_x2_list = [calc_trend_wrt_x2_w_interactions(x1, x2_values_b, b0, b1, b2, b3) for
x1 in x1_values_b]
```

```
study_interaction_wrt_x2_df = pd.concat(study_interaction_wrt_x2_list, ignore_index=True)
```

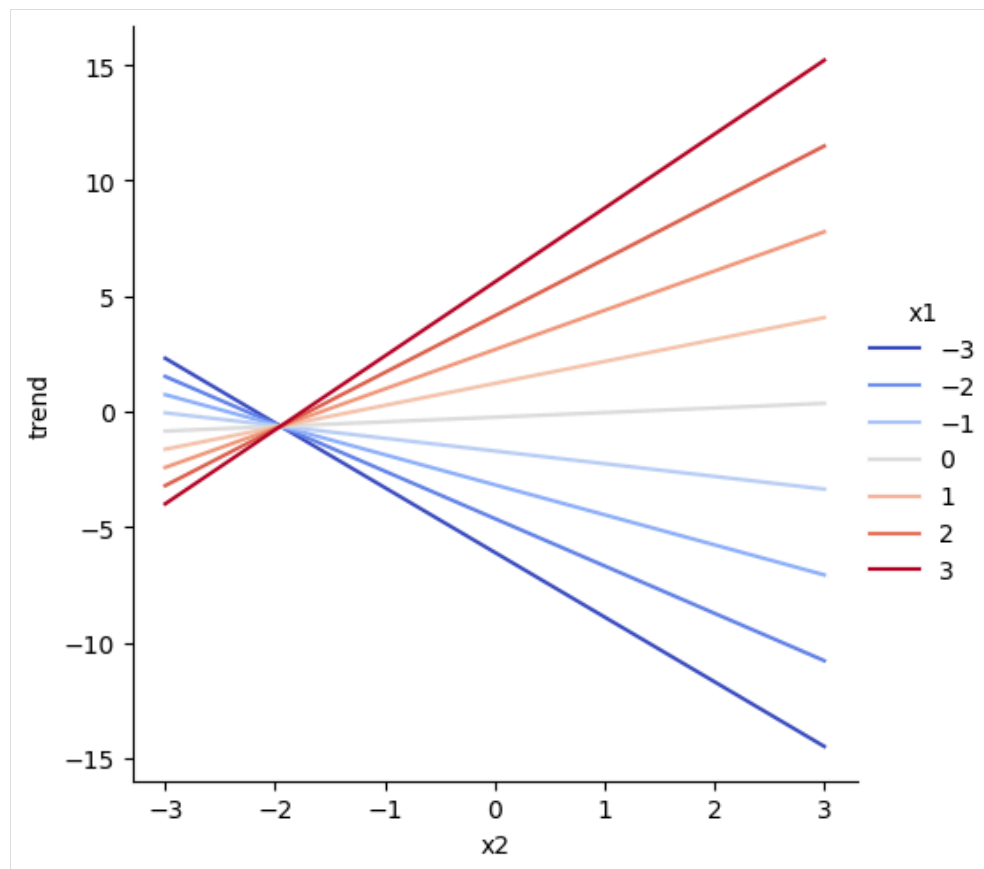
Here we see $\beta_2 < \beta_1$ as the magnitude and sign depend on the value of `x1`.

```

sns.relplot(data=study_interaction_wrt_x2_df,
            x='x2',
            y='trend',
            hue='x1',
            kind='line',
            palette='coolwarm',
            estimator=None,
            units='x1')

plt.show()

```



THE RELATIONSHIP OF THE AVERAGE OUTPUT WITH RESPECT TO 1 INPUT DEPENDS ON THE OTHER INPUT AND ROTATES THE LINE BY CHANGING THE SLOPE.

Interactions provide a lot of flexibility and allow the inputs to effect the behavior of other inputs.

Key Takeaways

- An interaction makes the effect of one feature depend on another.
- Interactions rotate the fitted relationship rather than shifting it.
- Interaction models are more flexible but harder to interpret.
- Add interactions when the domain suggests the effect should vary.

Exercises

1. Fit a model with an interaction and compare it to the additive model.
2. Plot the fitted lines at several values of the interacting feature.
3. Explain in words what the interaction coefficient means.
4. Decide whether the interaction is justified by the data.

Guided Walkthrough

The notebook adds an interaction term and watches the fitted relationship rotate rather than shift. It plots the prediction against one input at several fixed values of the interacting feature, producing a family of lines with different slopes. The interaction coefficient quantifies how much the slope changes, and the notebook weighs the gain in fit against the loss of interpretability.

Common Pitfalls

- Adding interactions without a domain reason.
- Interpreting a main effect when an interaction is present.
- Overfitting by adding many interaction terms at once.

Going Further

- Center the predictors before adding interactions to make the main effects interpretable.
- Use cross-validation to check whether the interaction generalizes.

Exercise Solutions

1. The interaction model fits better if the effect truly varies with the other feature.
2. The family of lines has different slopes, which is the interaction at work.
3. The interaction coefficient is the change in one feature's slope per unit of the other.
4. Keep the interaction only if it improves held-out performance meaningfully.

Chapter 14: Penguins Dataset - Linear Regression

Introduction

This is the culmination of the regression arc and the template for every serious modeling project in this course. It performs a full linear regression study on the Penguins dataset: visualizing relationships, fitting a ladder of candidate models from intercept-only to interactions, comparing them with cross-validation, checking residuals, and finally saving the best model and building a small inference engine that predicts on new data.

If you learn one workflow from this book, learn this one. It is the discipline that separates a model that merely runs from a model you can defend.

What You Will Learn

- How to build a sequence of candidate models of increasing complexity.
- How to compare models with cross-validated R-squared and RMSE.
- How to check residuals to confirm the model's assumptions.
- How to save a fitted model and load it for inference later.
- How to wrap a model in a function that predicts on new observations.

Overview

The chapter begins by visualizing the target against every continuous input, colored by species, to build intuition before any fitting. It then fits a ladder of models: an intercept-only baseline, a single categorical predictor, continuous predictors alone, continuous plus categorical, interactions and quadratic terms, and finally a full candidate. Comparing them on training data is a start, but the real comparison uses cross-validation, which rotates through held-out folds and reports the mean and spread of performance. The spread matters: a model that wins by a fraction of a standard error has not really won.

Once a model is chosen, we check its residuals against each input, expecting no pattern. A residual plot with structure means the model is missing something. We then save the model to disk with pickle and demonstrate loading it back and predicting on a new penguin. Saving and reloading is not glamorous, but it is what makes a model usable by anyone other than its author.

The inference engine at the end is deliberately small: a function that takes raw measurements and returns a prediction. It is the same idea that the deployment chapter turns into a command-line tool, and the same idea that underlies every model service in production.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```



```
import seaborn as sns
import statsmodels.formula.api as smf
from sklearn.linear_model import LinearRegression
from patsy import dmatrices
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
```

Load Dataset

```
df = pd.read_csv('data/penguins-clean.csv')
```

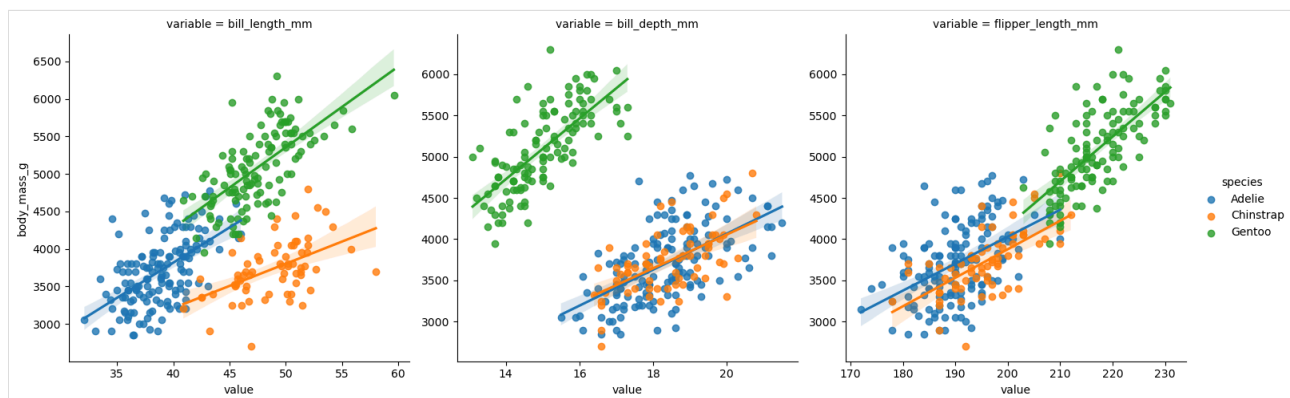
Linear Regression

Visualize Relationship Between the Output and Each Continuous Input, Colored by species

Body mass generally increases with each predictor (`bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`), and the 3 species form distinct clusters with Gentoo (green) largest, Chinstrap (orange) intermediate, and Adelie (blue) smallest.

There are clear relationships between all 3 continuous inputs and the target, with the strongest positive trends for `flipper_length_mm` and `bill_length_mm` (especially for Gentoo) and a somewhat noisier, more overlapping pattern for `bill_depth_mm` (Adelie and Chinstrap overlap while Gentoo remains separated).

```
sns.lmplot(
    data=df. \
        reset_index(). \
        rename(columns={'index':'rowid'}). \
        melt(id_vars=['rowid',
                     'body_mass_g',
                     'species'],
             value_vars=['bill_length_mm',
                        'bill_depth_mm',
                        'flipper_length_mm']),
    x='value',
    y='body_mass_g',
    col='variable',
    hue='species',
    facet_kws={'sharey': False,
              'sharex': False})
plt.show()
```

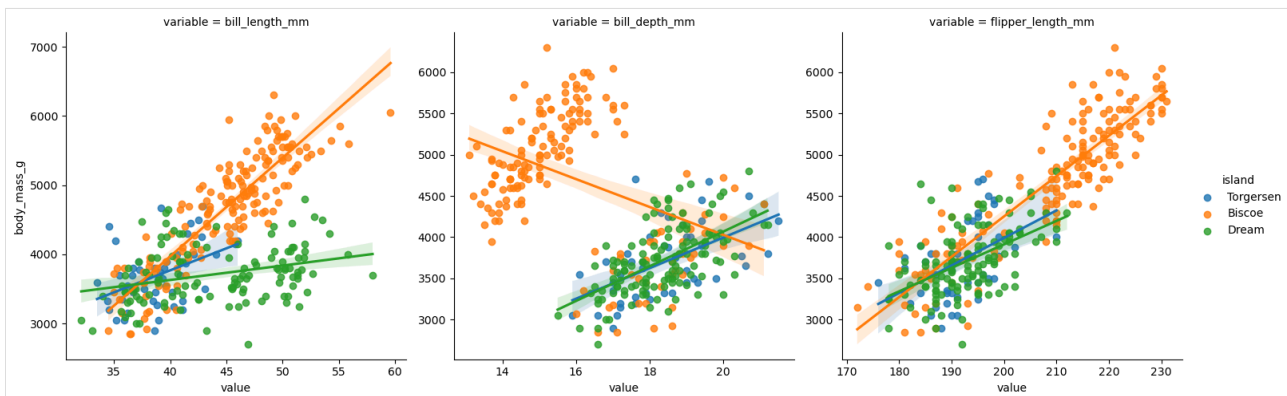


Visualize Relationship Between the Output and Each Continuous Input, Colored by island

Body mass generally increases with each predictor (`bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`), and the 3 islands show overlapping but distinguishable clusters with Biscoe (orange) largest, Torgersen (blue) intermediate, and Dream (green) smallest.

There are clear relationships between all 3 continuous inputs and the target, with the strongest positive trends for `bill_length_mm` and `flipper_length_mm` (especially on Biscoe) and a weaker, island-specific pattern for `bill_depth_mm`.

```
sns.lmplot(
    data=df. \
        reset_index(). \
        rename(columns={'index':'rowid'}). \
        melt(id_vars=['rowid',
                      'body_mass_g',
                      'island'],
            value_vars=['bill_length_mm',
                      'bill_depth_mm',
                      'flipper_length_mm']),
    x='value',
    y='body_mass_g',
    col='variable',
    hue='island',
    facet_kws={'sharey': False,
              'sharex': False})
plt.show()
```



Visualize Relationship Between the Output and Each Continuous Input, Colored by sex

Body mass generally increases with each predictor (`bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`), and the 2 sexes form overlapping but distinguishable bands with Males (blue) larger and Females (orange) smaller across most of the measurement ranges; the main separation is vertical (mass) rather than horizontal, with notable overlap in the mid-ranges of the predictors.

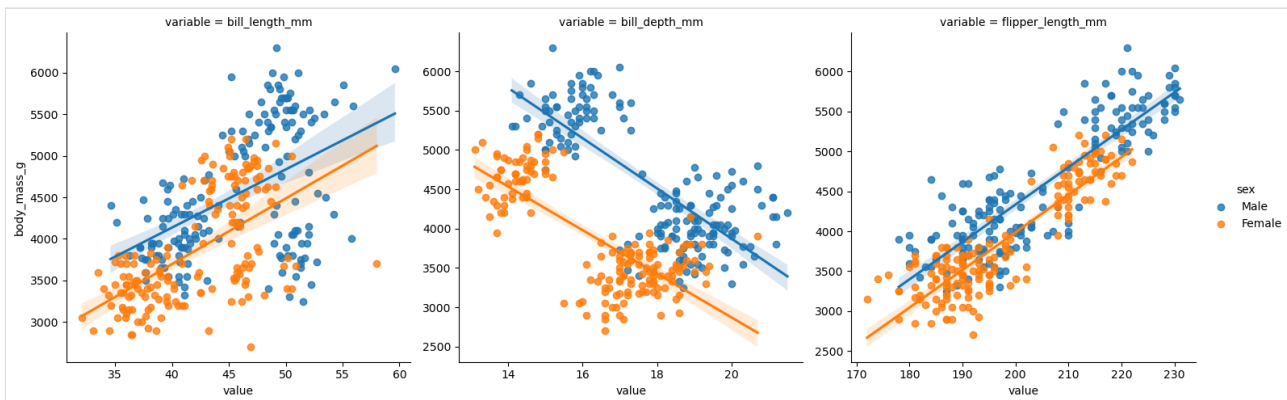
There are clear relationships between all 3 continuous inputs and the target, with the strongest positive trend for `flipper_length_mm`, a moderate positive trend for `bill_length_mm`, and a contrasting negative within-sex trend for `bill_depth_mm` (body mass declines as `bill_depth_mm` increases), while parallel sex-specific lines indicate a fairly consistent mass offset between Males and Females.

```
sns.lmplot(
    data=df. \
```

```

reset_index(). \
rename(columns={'index':'rowid'}). \
melt(id_vars=['rowid',
              'body_mass_g',
              'sex'],
      value_vars=['bill_length_mm',
                  'bill_depth_mm',
                  'flipper_length_mm']),
x='value',
y='body_mass_g',
col='variable',
hue='sex',
facet_kws={'sharey': False,
           'sharex': False})
plt.show()

```



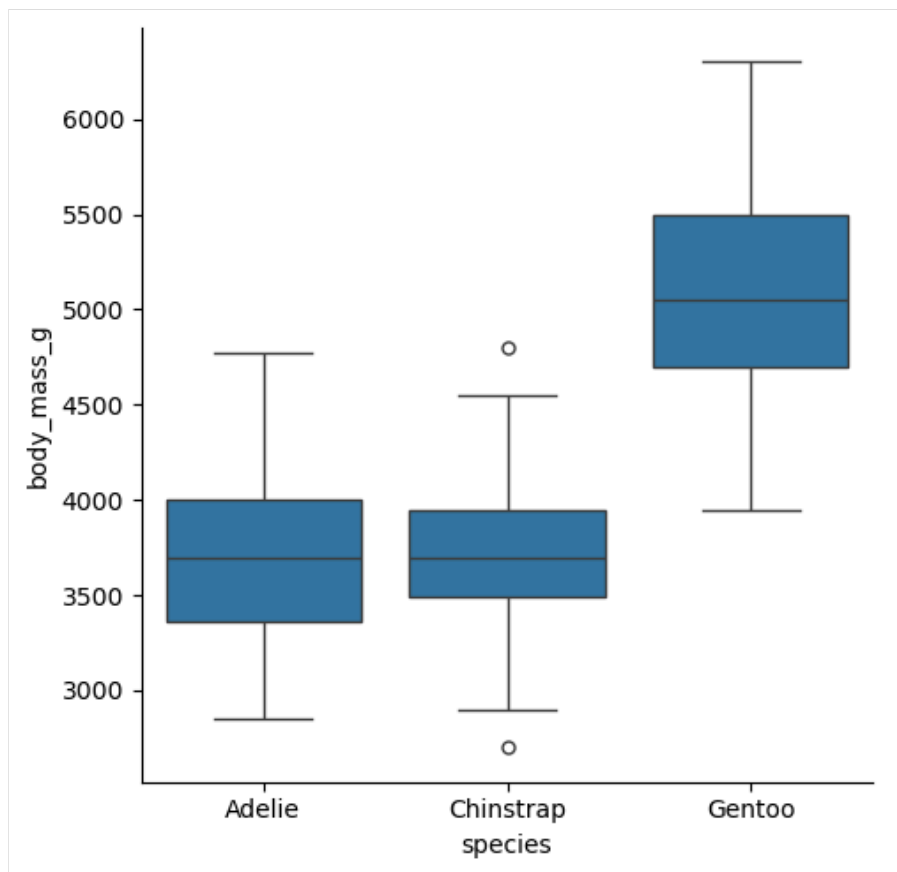
Visualize Relationship Between the Output and species

Across all three visualizations, Gentoo penguins show a clear upward shift in body mass relative to Adelie and Chinstrap as the boxplot emphasizes Gentoo's higher median and broader IQR, the violin plot reveals a bimodal/shouldered density suggesting internal variation (possibly sex or subgroups) while Adelie and Chinstrap have tighter, more centralized distributions with overlapping ranges, and the point plot with error bars (mean $\pm$ likely CI) succinctly confirms the large mean gap (Gentoo far higher, Chinstrap only slightly above Adelie); together they indicate strong inter-species separation driven by uniformly higher and more variable Gentoo masses, with Adelie and Chinstrap largely overlapping in central tendency but Chinstrap extending a bit higher in upper distribution tails.

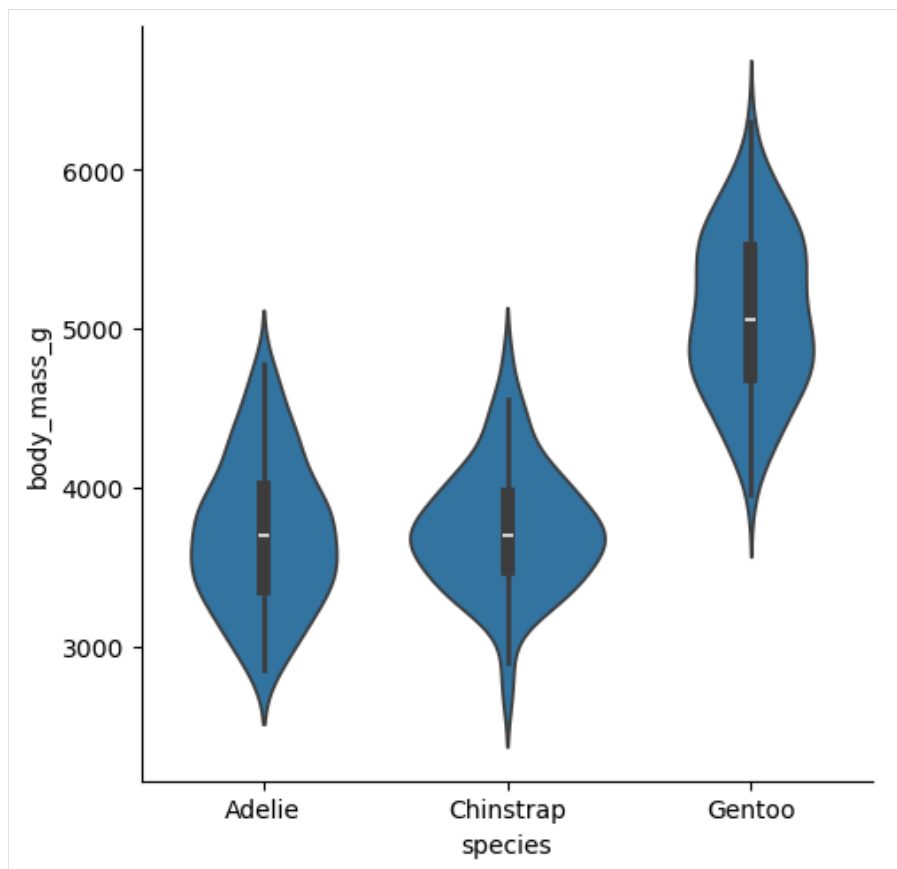
```

sns.catplot(data=df,
            x='species',
            y='body_mass_g',
            kind='box')
plt.show()

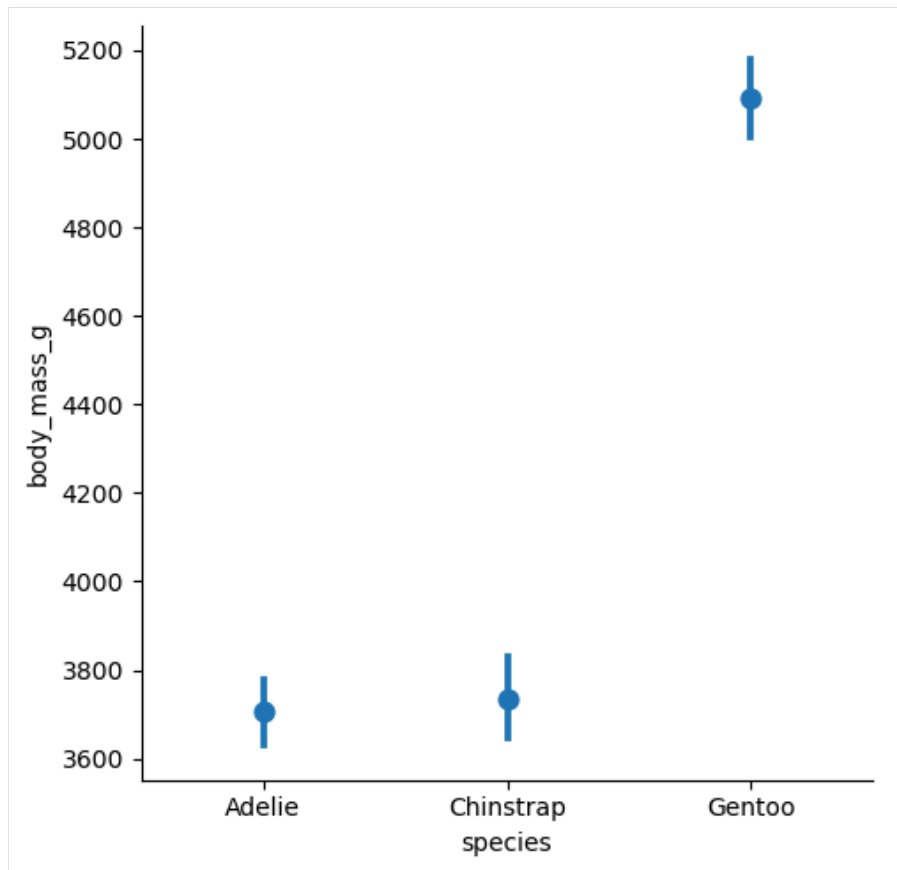
```



```
sns.catplot(data=df,  
            x='species',  
            y='body_mass_g',  
            kind='violin')  
plt.show()
```



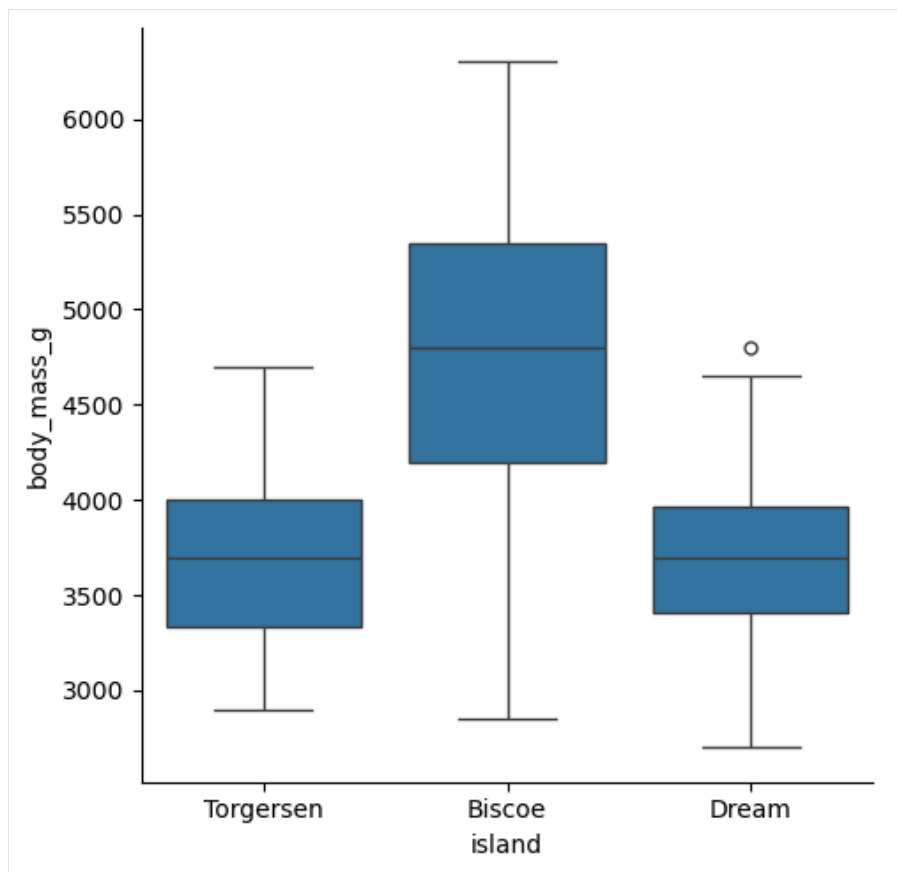
```
sns.catplot(data=df,  
            x='species',  
            y='body_mass_g',  
            kind='point',  
            linestyle='none')  
plt.show()
```



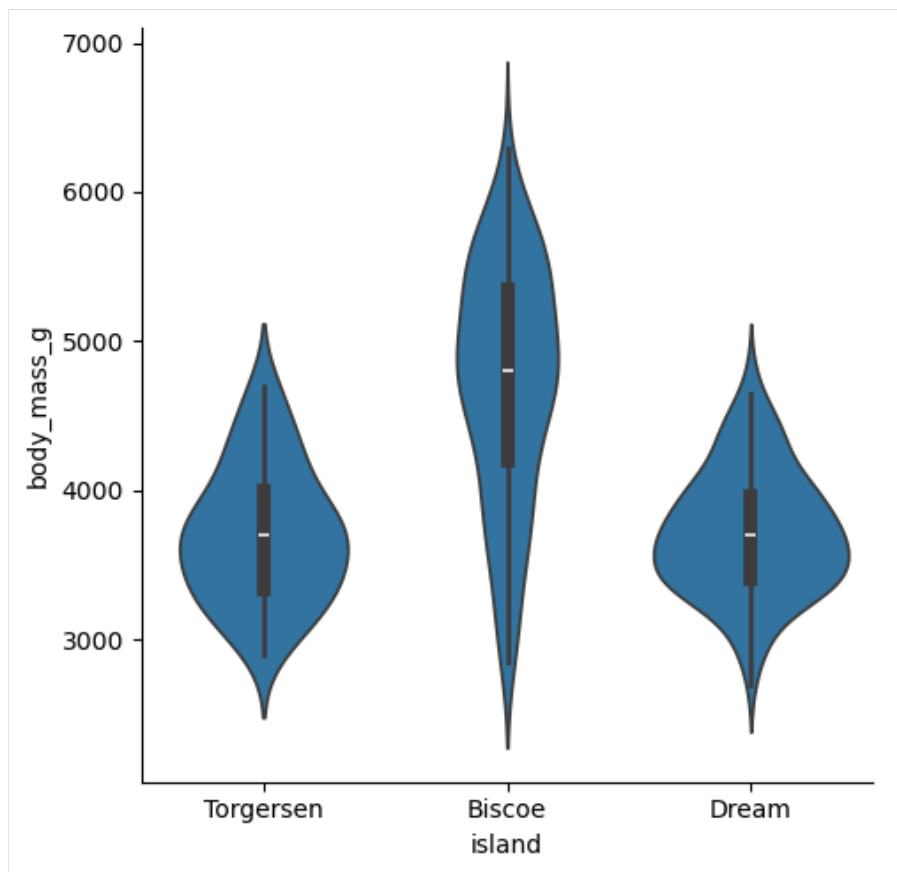
Visualize Relationship Between the Output and island

Across the island visualizations, penguins from Biscoe exhibit a clear upward shift in body mass relative to Torgersen and Dream as the boxplot shows Biscoe's median markedly higher with a much wider IQR and whiskers spanning the broadest range (including the heaviest individuals), while Torgersen and Dream have lower, overlapping medians and tighter boxes; the violin plot reinforces this separation with a tall, multi-shouldered Biscoe density (suggesting internal substructure, likely species or sex mix) contrasted against the more compact, single-peaked shapes for Torgersen and Dream; and the point plot (means with intervals) succinctly confirms the large mean gap—Biscoe well above, Dream only slightly exceeding Torgersen—indicating that inter-island differences in body mass are driven primarily by the consistently higher and more variable masses of penguins sampled on Biscoe, with Torgersen and Dream forming a lower, overlapping baseline.

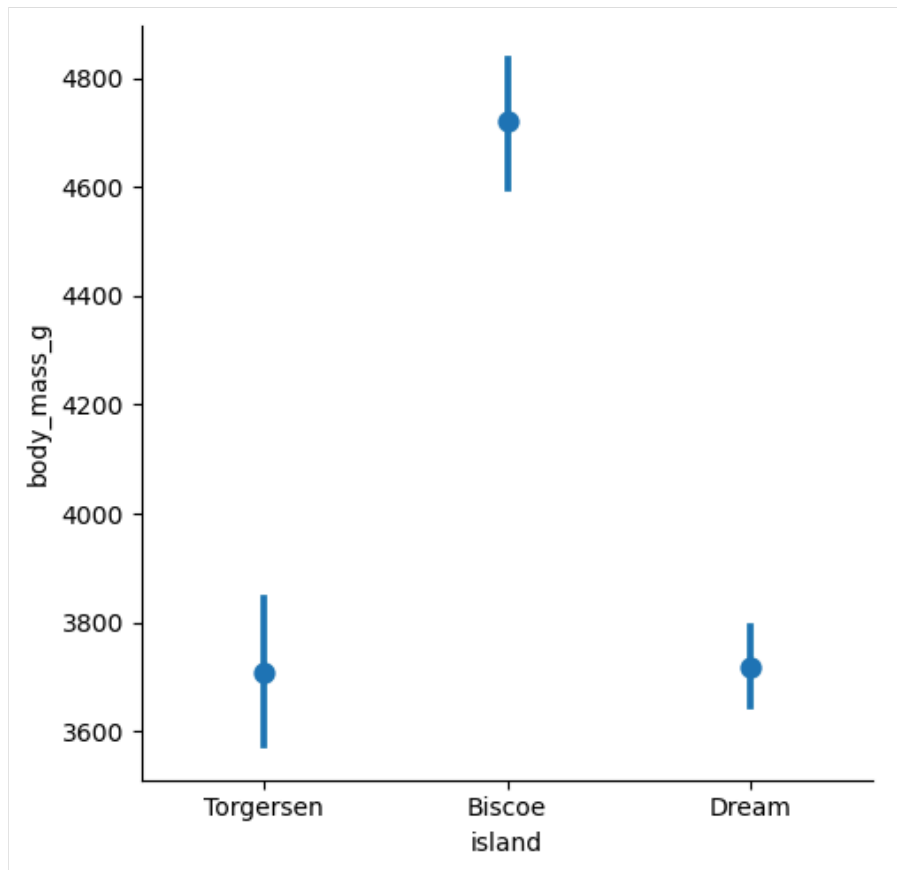
```
sns.catplot(data=df,  
            x='island',  
            y='body_mass_g',  
            kind='box')  
plt.show()
```



```
sns.catplot(data=df,  
            x='island',  
            y='body_mass_g',  
            kind='violin')  
plt.show()
```



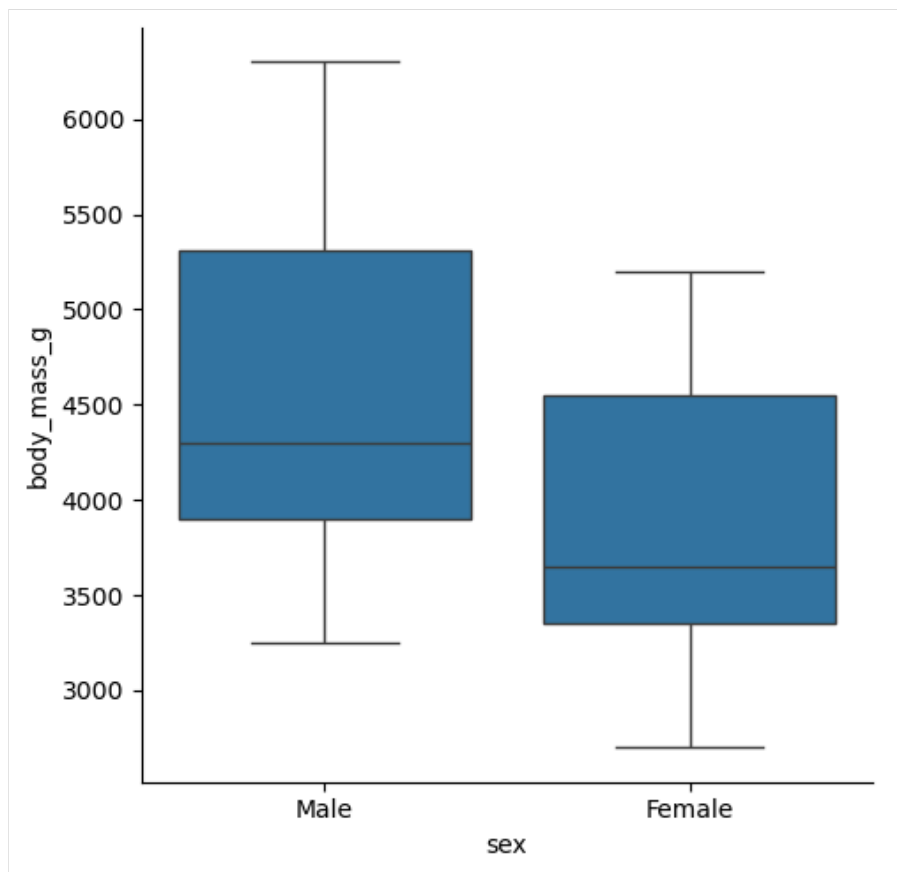
```
sns.catplot(data=df,  
            x='island',  
            y='body_mass_g',  
            kind='point',  
            linestyle='none')  
plt.show()
```

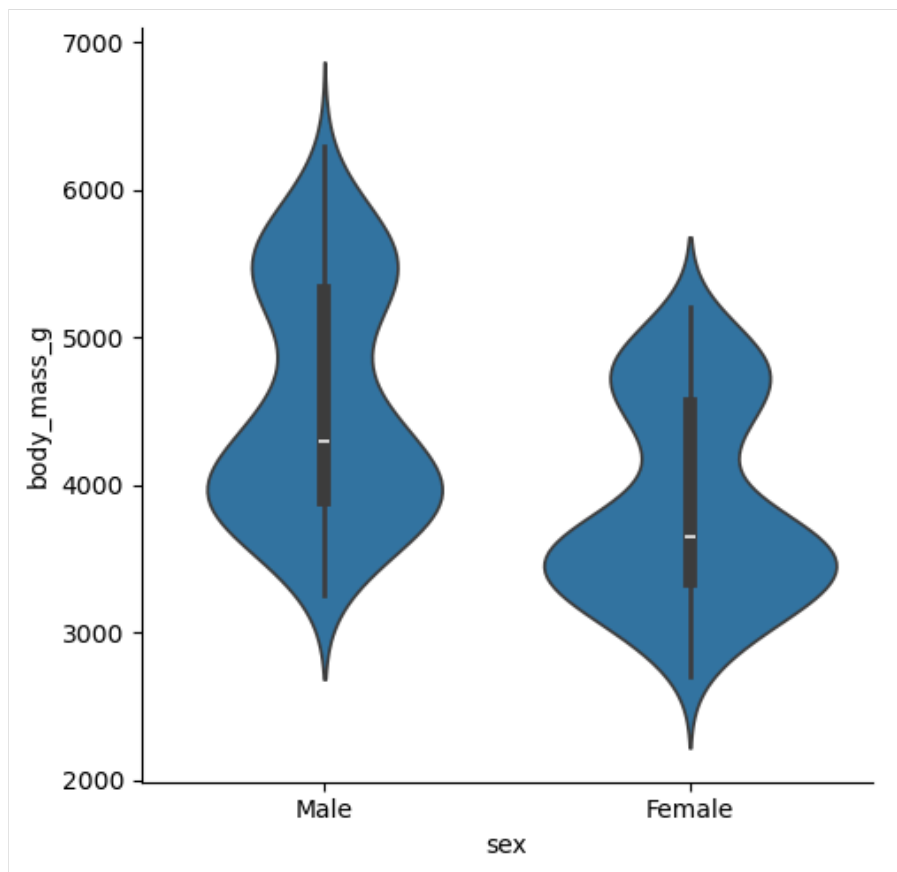
Visualize Relationship Between the Output and sex

Across the sex-focused visualizations, Male penguins show a clear upward shift in body mass versus Females as the boxplot highlights a higher Male median with an IQR spanning substantially larger values and whiskers reaching the heaviest birds, while the Female box sits lower with a narrower upper range; the violin plot reinforces this by displaying a broader, multi-shouldered Male density (suggesting internal subgrouping, e.g., species differences layered atop sex) contrasted with a more compact Female distribution that tapers earlier at the high end; and the point plot (mean with interval) succinctly confirms the gap, with the Male mean well above the Female mean and only modest overlap in confidence intervals—together indicating a consistent vertical (mass) separation where overlap exists mainly in the mid to lower mass range.

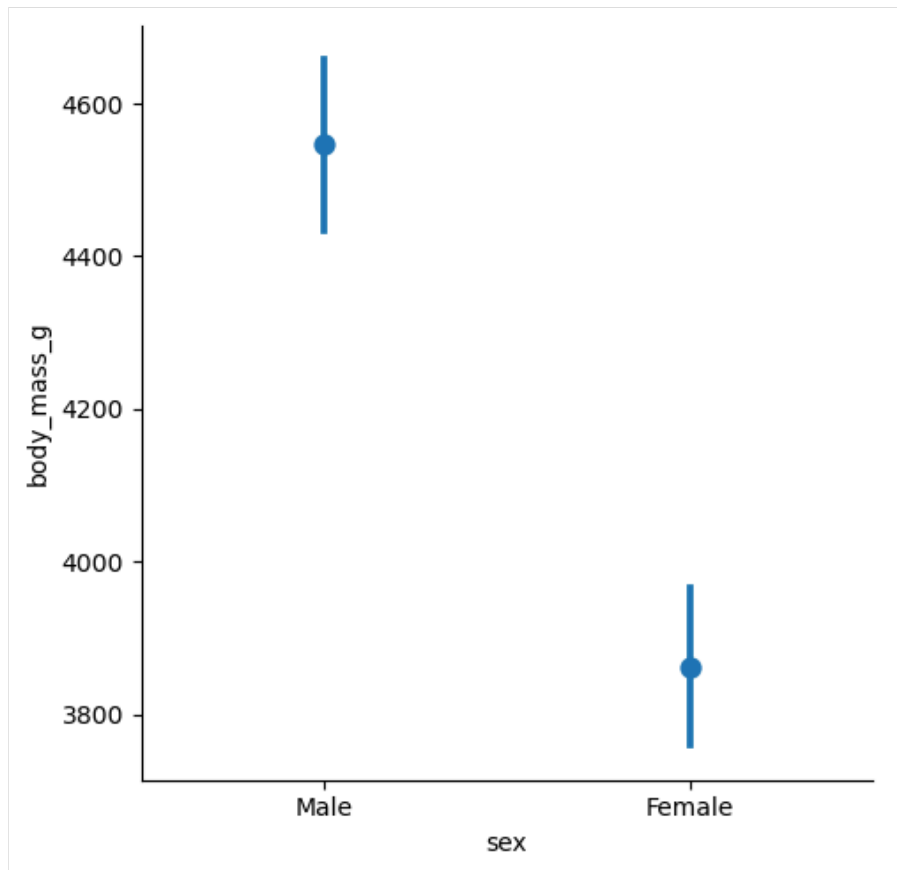
```
sns.catplot(data=df,  
            x='sex',  
            y='body_mass_g',  
            kind='box')  
plt.show()
```



```
sns.catplot(data=df,  
            x='sex',  
            y='body_mass_g',  
            kind='violin')  
plt.show()
```



```
sns.catplot(data=df,  
            x='sex',  
            y='body_mass_g',  
            kind='point',  
            linestyle='none')  
plt.show()
```



Initially Fit Linear Regression Models - Training Data

Functions

```
def fit_and_assess_ols(model,
                       formula,
                       df):
    """
    Fits an Ordinary Least Squares (OLS) regression model using statsmodels and returns model metrics.

    Parameters
    -----
    model : str
        A descriptive name for the model.
    formula : str
        A statsmodels formula string specifying the regression model.
    df : pandas.DataFrame
        The input data to fit the model.

    Returns
    -----
    pandas.DataFrame
        A single-row DataFrame containing:
        - 'model_name': Name of the model
        - 'model_formula': Formula used for fitting
        - 'num_coefs': Number of estimated coefficients
        - 'R-squared': Model R-squared statistic
        - 'RMSE': Root Mean Squared Error of residuals
    """
    # Fit the model
```

```

a_model = smf.ols(formula=formula, data=df).fit()

# Extract model metrics
results_dict = {'model_name': model,
                'model_formula': formula,
                'num_coefs': len(a_model.params),
                'R-squared': a_model.rsquared,
                'RMSE': np.sqrt((a_model.resid ** 2).mean())}

# Return results as DataFrame
return pd.DataFrame(results_dict, index=[0])

```

```

def my_coefplot(model,
                figsize_use=(10, 8),
                zero_nudge=125):
    """
    Create a coefficient plot of a fitted statsmodels regression model.

    This function visualizes the estimated coefficients of a regression model with their
    confidence intervals (approximated as  $\pm 2$  standard errors).
    Statistically significant coefficients ( $p < 0.05$ ) are shown in red.

    Parameters
    -----
    model : statsmodels.regression.linear_model.RegressionResultsWrapper
        A fitted statsmodels regression model, e.g., the result of smf.ols(...).fit().
    figsize_use : tuple, optional
        Size of the matplotlib figure, default is (10, 4).
    zero_nudge : float, optional
        Amount to nudge significant coefficients away from zero for visual clarity. Default is 125.

    Returns
    -----
    None
        Displays a matplotlib plot showing the model coefficients and their error bars.
    """
    # Create the figure and axis
    _, ax = plt.subplots(figsize=figsize_use)

    # Nudge coefficients very close to zero for visual clarity
    coefs = model.params.copy()
    coefs[np.isclose(coefs, 0, atol=zero_nudge)] = zero_nudge

    # Determine colors for coefficients based on significance
    colors = ['red' if p < 0.05 else 'black' for p in model.pvalues]

    # Plot each coefficient with its color
    for _, (y, x, xerr, color) in enumerate(zip(coefs.index, coefs, 2 * model.bse, colors)):
        ax.errorbar([x],
                    [y],
                    xerr=[xerr],
                    fmt='o',
                    color=color,
                    ecolor=color,
                    elinewidth=2,
                    ms=10)

    # Add a vertical line at x=0
    ax.axvline(x=0,
               linestyle='--',
               linewidth=3.5,

```

```

        color='grey')

# Add labels and title
ax.set_xlabel('Coefficient Value')

# Show the plot
plt.show()

```

Formulas

```

formula_list = [
    # Model 00: Intercept-Only (Baseline)
    'Q("body_mass_g") ~ '
    '1',

    # Model 01: Species (Categorical) Only
    'Q("body_mass_g") ~ '
    'C(Q("species"))',

    # Model 02: Continuous Predictors, Linear Additive
    'Q("body_mass_g") ~ '
    'Q("bill_length_mm")'
    ' + Q("bill_depth_mm")'
    ' + Q("flipper_length_mm")',

    # Model 03: Continuous Predictors + Species (Additive)
    'Q("body_mass_g") ~ '
    'Q("bill_length_mm")'
    ' + Q("bill_depth_mm")'
    ' + Q("flipper_length_mm")'
    ' + C(Q("species"))',

    # Model 04: Continuous Predictors with Pairwise Interactions and Quadratic Terms
    'Q("body_mass_g") ~ '
    'Q("bill_length_mm")'
    ' + Q("bill_depth_mm")'
    ' + Q("flipper_length_mm")'
    ' + Q("bill_length_mm"):Q("bill_depth_mm")'
    ' + Q("bill_length_mm"):Q("flipper_length_mm")'
    ' + Q("bill_depth_mm"):Q("flipper_length_mm")'
    ' + I(Q("bill_length_mm") ** 2)'
    ' + I(Q("bill_depth_mm") ** 2)'
    ' + I(Q("flipper_length_mm") ** 2)',

    # Model 05: Species Interacting with All Continuous Predictors (Allows Species-Specific Slopes)
    'Q("body_mass_g") ~ '
    'C(Q("species")) * ('
    'Q("bill_length_mm")'
    ' + Q("bill_depth_mm")'
    ' + Q("flipper_length_mm")'
    ')',

    # Model 06: Higher-Order Polynomials on Flipper Length (If Non-Linear Scaling Suspected)
    'Q("body_mass_g") ~ '
    'Q("bill_length_mm")'
    ' + Q("bill_depth_mm")'
    ' + Q("flipper_length_mm")'
    ' + I(Q("flipper_length_mm") ** 2)'
    ' + I(Q("flipper_length_mm") ** 3)',

    # Model 07: All Predictors, Interactions Among Continuous Terms, and Species (Full Candidate)

```

```

'Q("body_mass_g") ~ '
'(Q("bill_length_mm") + Q("bill_depth_mm") + Q("flipper_length_mm"))**2'
' + C(Q("species"))',

# Model 08: Final Model (Species-Specific Slope for Flipper Length)
'Q("body_mass_g") ~ '
'Q("flipper_length_mm")'
' + Q("bill_length_mm")'
' + C(Q("species"))'
' + Q("flipper_length_mm"):C(Q("species"))'
]

```

Model 00: Intercept-Only (Baseline)

```

model_00 = smf.ols(formula=formula_list[0],
                    data=df). \
fit()

```

```
model_00.summary()
```

```

<class 'statsmodels.iolib.summary.Summary'>
"""

```

OLS Regression Results

```

=====
Dep. Variable:      Q("body_mass_g")    R-squared:                0.000
Model:              OLS                  Adj. R-squared:            0.000
Method:             Least Squares        F-statistic:              nan
Date:               Thu, 24 Sep 2026     Prob (F-statistic):       nan
Time:               10:17:56             Log-Likelihood:           -2700.1
No. Observations:   333                  AIC:                      5402.
Df Residuals:       332                  BIC:                      5406.
Df Model:           0
Covariance Type:    nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	4207.0571	44.126	95.343	0.000	4120.256	4293.858

```

=====
Omnibus:            30.464    Durbin-Watson:           0.972
Prob(Omnibus):      0.000    Jarque-Bera (JB):       19.874
Skew:               0.470    Prob(JB):               4.84e-05
Kurtosis:           2.260    Cond. No.               1.00
=====

```

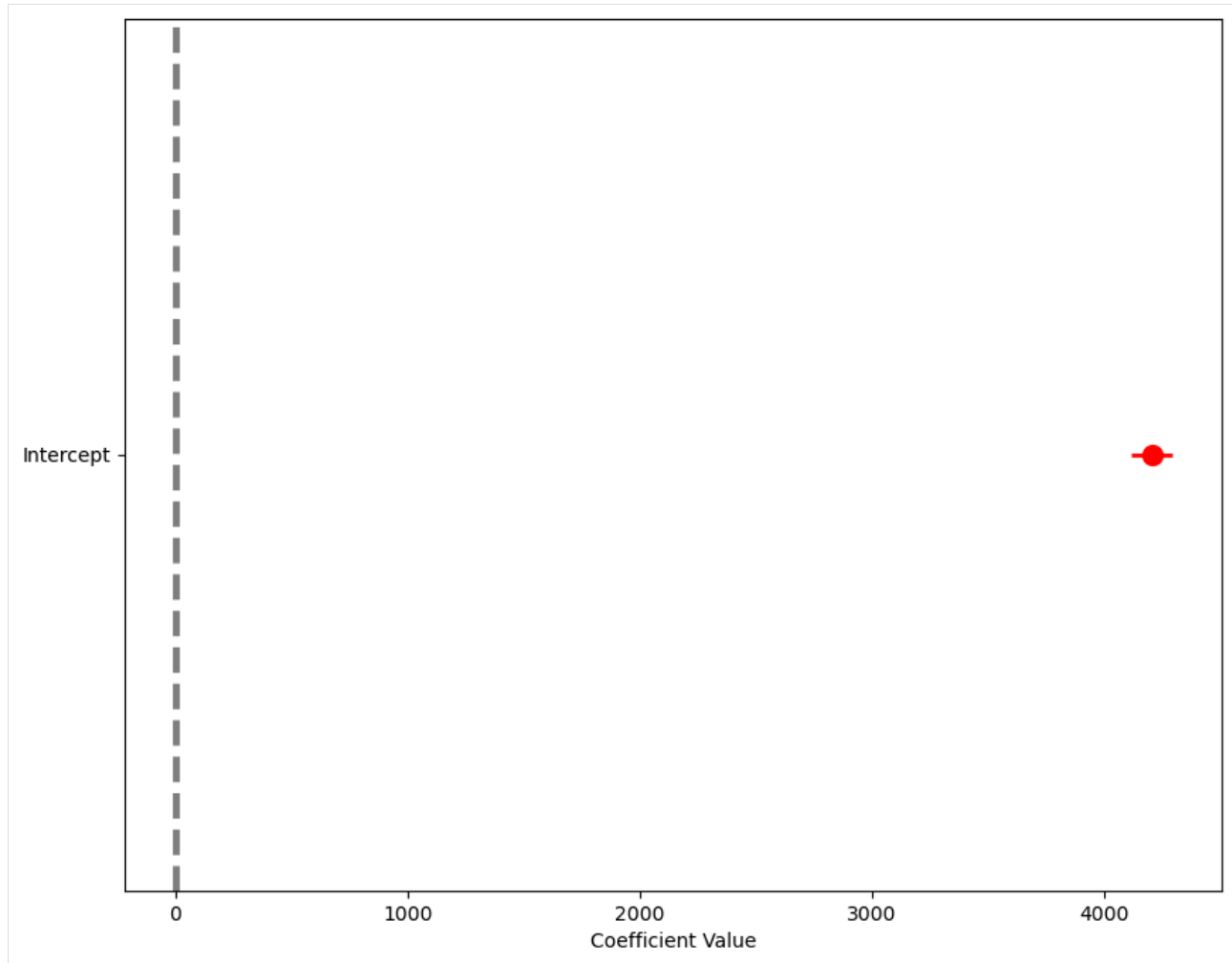
Notes:

```

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
"""

```

```
my_coefplot(model_00)
```



Model 01: Species (Categorical) Only

```
model_01 = smf.ols(formula=formula_list[1],
                    data=df). \
                    fit()
```

```
model_01.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
.....
```

OLS Regression Results

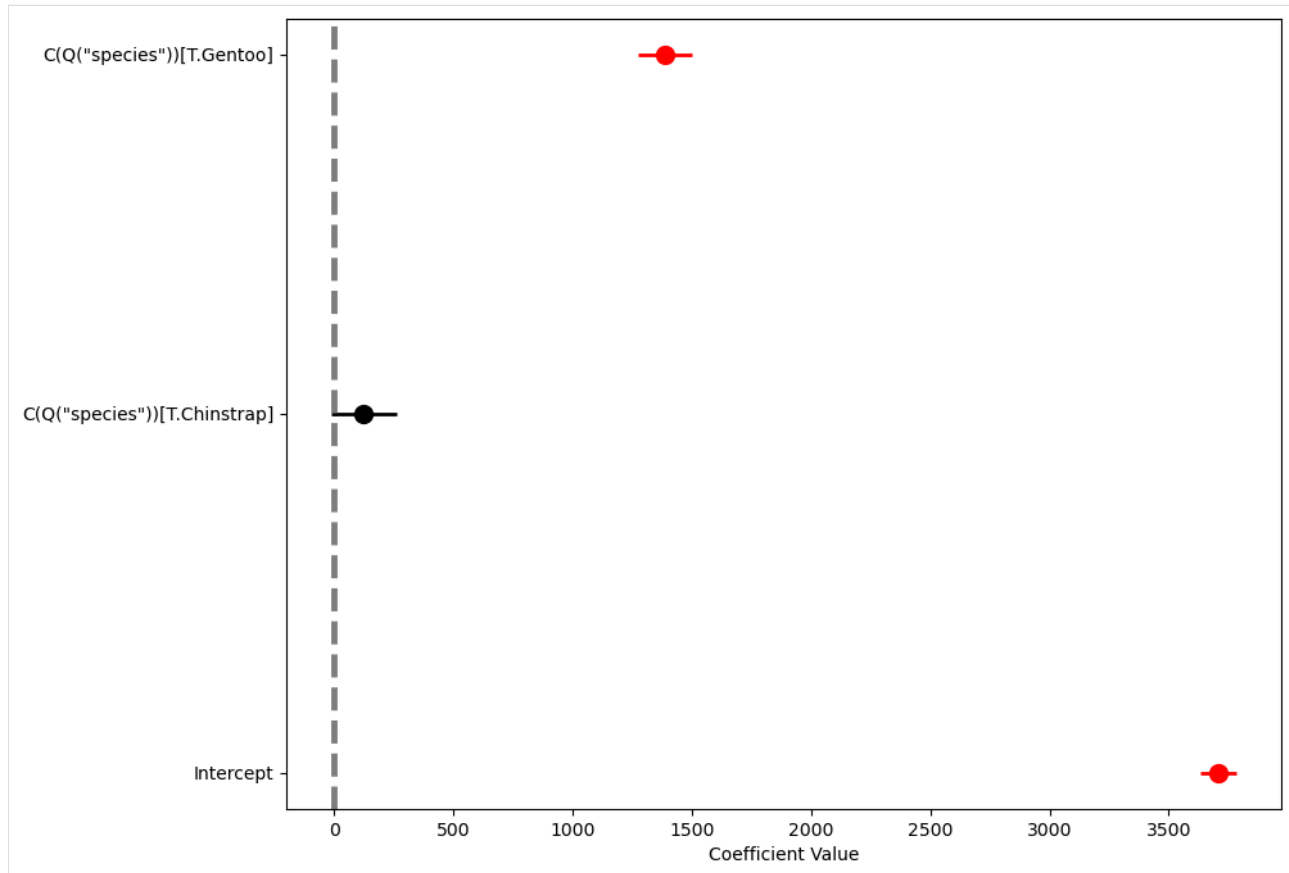
```
=====
Dep. Variable:    Q("body_mass_g")    R-squared:                0.674
Model:            OLS                  Adj. R-squared:            0.673
Method:            Least Squares        F-statistic:              341.9
Date:              Thu, 24 Sep 2026      Prob (F-statistic):       3.74e-81
Time:              10:17:56              Log-Likelihood:           -2513.3
No. Observations: 333                  AIC:                      5033.
Df Residuals:      330                  BIC:                      5044.
Df Model:           2
Covariance Type:   nonrobust
=====
```


	coef	std err	t	P> t	[0.025	0.975]
Intercept	3706.1644	38.136	97.184	0.000	3631.145	3781.184
C(Q("species"))[T.Chinstrap]	26.9239	67.652	0.398	0.691	-106.161	160.008
C(Q("species"))[T.Gentoo]	1386.2726	56.909	24.359	0.000	1274.323	1498.223
Omnibus:	6.944	Durbin-Watson:	2.983			
Prob(Omnibus):	0.031	Jarque-Bera (JB):	4.976			
Skew:	0.169	Prob(JB):	0.0831			
Kurtosis:	2.506	Cond. No.	3.45			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
my_coefplot(model_01)
```



Model 02: Continuous Predictors, Linear Additive

```
model_02 = smf.ols(formula=formula_list[2],
                    data=df). \
                    fit()
```

```
model_02.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
'''''
```

OLS Regression Results

```
=====
Dep. Variable:      Q("body_mass_g")    R-squared:                0.764
Model:              OLS                  Adj. R-squared:           0.762
Method:             Least Squares        F-statistic:              354.9
Date:               Thu, 24 Sep 2026     Prob (F-statistic):       9.26e-103
Time:               10:17:56             Log-Likelihood:           -2459.8
No. Observations:   333                  AIC:                      4928.
Df Residuals:       329                  BIC:                      4943.
Df Model:           3
Covariance Type:    nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-6445.4760	566.130	-11.385	0.000	-7559.167	-5331.785
Q("bill_length_mm")	3.2929	5.366	0.614	0.540	-7.263	13.849
Q("bill_depth_mm")	17.8364	13.826	1.290	0.198	-9.362	45.035
Q("flipper_length_mm")	50.7621	2.497	20.327	0.000	45.850	55.675

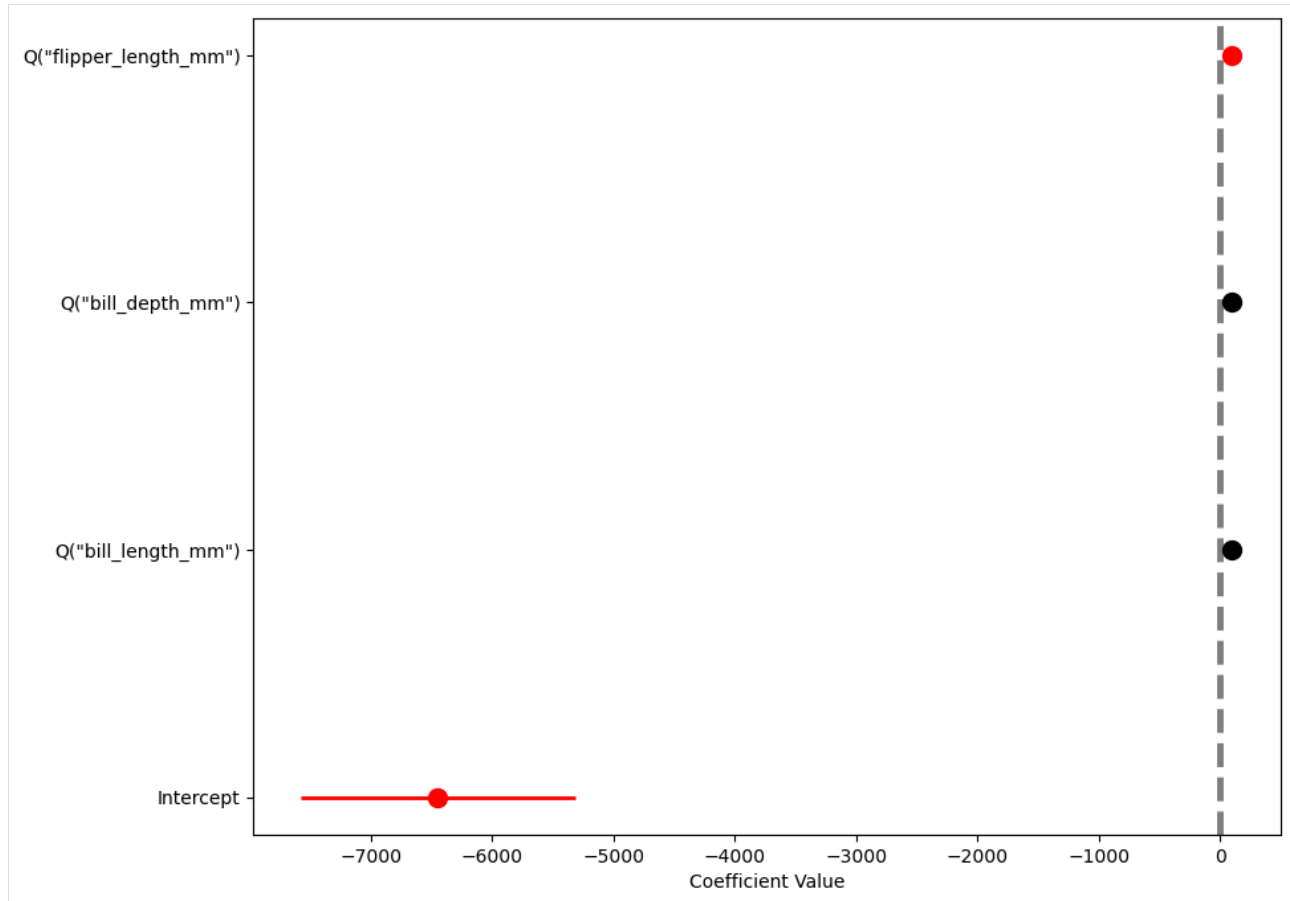
```
=====
Omnibus:            5.596    Durbin-Watson:           1.968
Prob(Omnibus):      0.061    Jarque-Bera (JB):        5.469
Skew:               0.312    Prob(JB):                 0.0649
Kurtosis:           3.068    Cond. No.                 5.44e+03
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 5.44e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
'''''
```

```
my_coefplot(model_02, zero_nudge=90)
```



Model 03: Continuous Predictors + Species (Additive)

```
model_03 = smf.ols(formula=formula_list[3],
                    data=df). \
                    fit()
```

```
model_03.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
.....
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          Q("body_mass_g")    R-squared:                0.849
Model:                  OLS                 Adj. R-squared:           0.847
Method:                 Least Squares       F-statistic:              369.1
Date:                   Thu, 24 Sep 2026    Prob (F-statistic):       4.22e-132
Time:                   10:17:56           Log-Likelihood:          -2384.8
No. Observations:       333                AIC:                     4782.
Df Residuals:           327                BIC:                     4805.
Df Model:                5
Covariance Type:        nonrobust

=====
                        coef      std err      t      P>|t|      [0.025      0.975]
-----
Intercept               -4282.0802    497.832    -8.601    0.000    -5261.438   -3302.723
C(Q("species"))[T.Chinstrap] -496.7583    82.469    -6.024    0.000    -658.995   -334.521
=====
```

```

C(Q("species"))[T.Gentoo]      965.1983    141.770    6.808    0.000    686.301    1244.096
Q("bill_length_mm")            39.7184     7.227    5.496    0.000    25.501    53.936
Q("bill_depth_mm")            141.7714    19.163    7.398    0.000    104.072    179.470
Q("flipper_length_mm")         20.2264     3.135    6.452    0.000    14.059    26.394
=====
Omnibus:                        7.321    Durbin-Watson:                2.247
Prob(Omnibus):                  0.026    Jarque-Bera (JB):              7.159
Skew:                          0.348    Prob(JB):                     0.0279
Kurtosis:                      3.179    Cond. No.                     6.01e+03
=====

```

Notes:

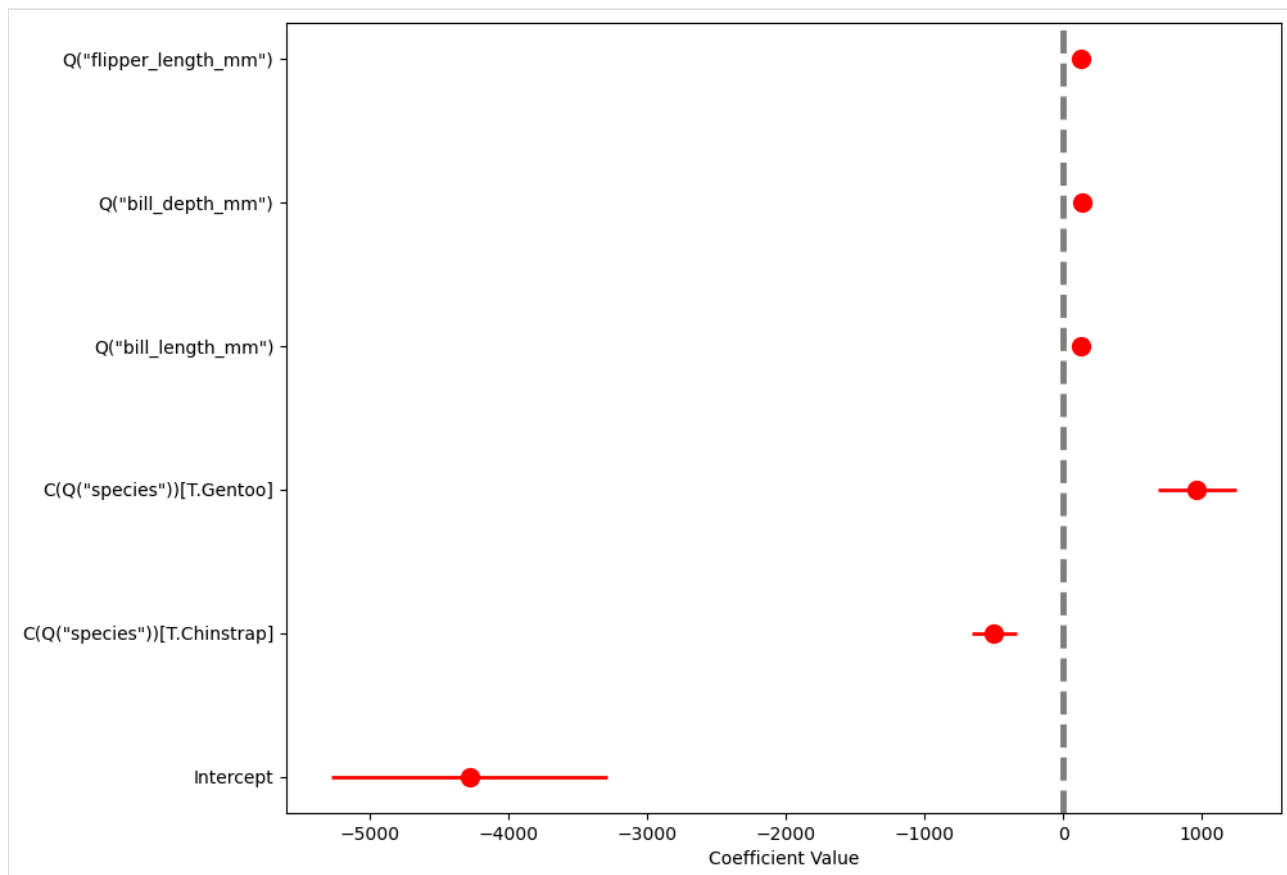
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 6.01e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```

"""

```

```
my_coefplot(model_03)
```



Model 04: Continuous Predictors with Pairwise Interactions and Quadratic Terms

```

model_04 = smf.ols(formula=formula_list[4],
                    data=df). \
                    fit()

```

```
model_04.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
''''''
```

OLS Regression Results

```
=====
Dep. Variable:      Q("body_mass_g")    R-squared:                0.816
Model:              OLS                  Adj. R-squared:           0.811
Method:             Least Squares        F-statistic:              159.1
Date:               Thu, 24 Sep 2026     Prob (F-statistic):       4.75e-113
Time:               10:17:57             Log-Likelihood:           -2418.4
No. Observations:   333                  AIC:                      4857.
Df Residuals:       323                  BIC:                      4895.
Df Model:           9
Covariance Type:    nonrobust
=====
```

	coef	std err	t	P> t	[0.025
Intercept	-1.001e+04	1.14e+04	-0.878	0.381	-3.25e+04
Q("bill_length_mm")	203.8636	122.035	1.671	0.096	-36.221
Q("bill_depth_mm")	1178.9782	539.781	2.184	0.030	117.048
Q("flipper_length_mm")	-54.0439	84.854	-0.637	0.525	-220.981
Q("bill_length_mm"):Q("bill_depth_mm")	-10.7817	3.224	-3.344	0.001	-17.125
Q("bill_length_mm"):Q("flipper_length_mm")	0.9314	0.564	1.652	0.099	-0.178
Q("bill_depth_mm"):Q("flipper_length_mm")	-4.1625	2.086	-1.996	0.047	-8.266
I(Q("bill_length_mm")) ** 2)	-2.0777	0.824	-2.522	0.012	-3.699
I(Q("bill_depth_mm")) ** 2)	4.8728	6.879	0.708	0.479	-8.661
I(Q("flipper_length_mm")) ** 2)	0.3112	0.186	1.673	0.095	-0.055

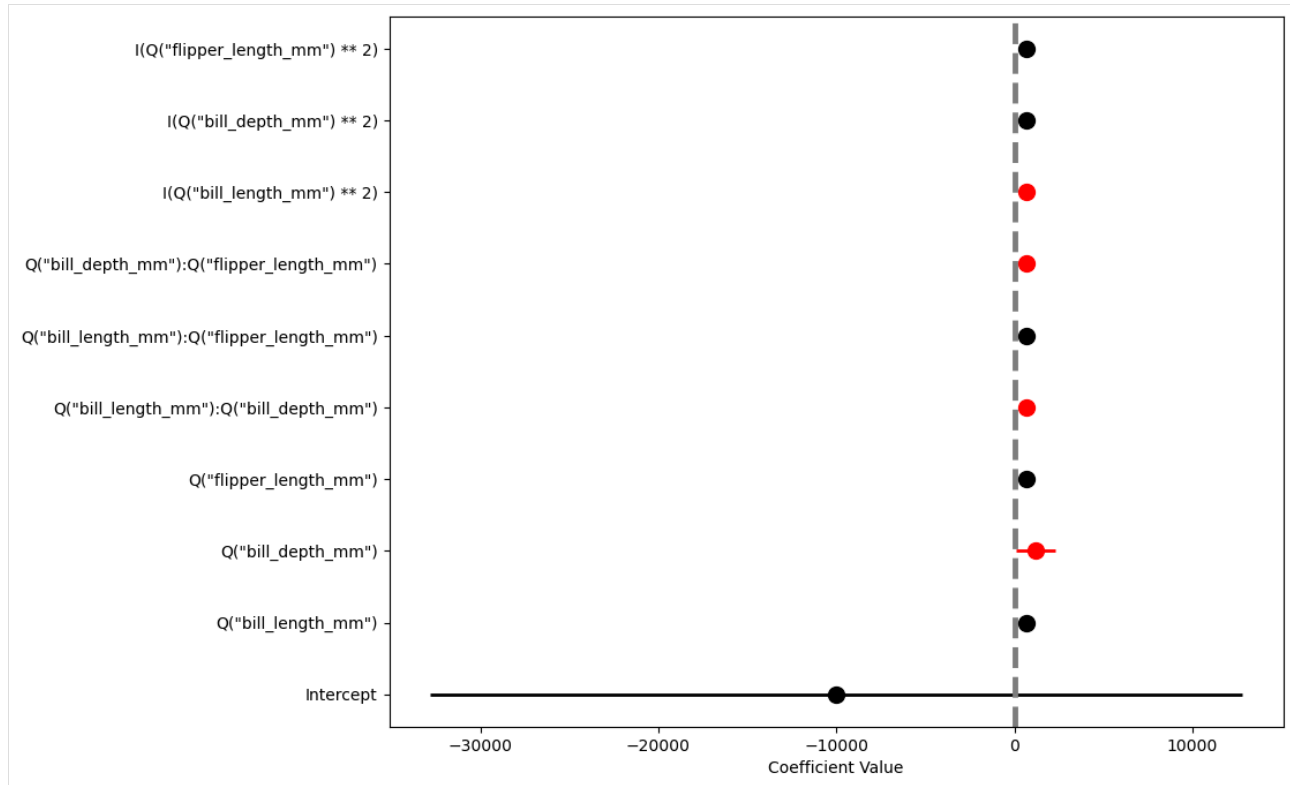
```
=====
Omnibus:            9.149    Durbin-Watson:           2.059
Prob(Omnibus):      0.010    Jarque-Bera (JB):        9.132
Skew:               0.395    Prob(JB):                 0.0104
Kurtosis:           3.181    Cond. No.                 2.51e+07
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 2.51e+07. This might indicate that there are strong multicollinearity or other numerical problems.

```
''''''
```

```
my_coefplot(model_04, zero_nudge=700)
```



Model 05: Species Interacting with All Continuous Predictors (Allows Species-Specific Slopes)

```
model_05 = smf.ols(formula=formula_list[5],
                    data=df). \
                    fit()
```

```
model_05.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
.....
```

OLS Regression Results

```
=====
Dep. Variable:      Q("body_mass_g")    R-squared:                0.856
Model:              OLS                  Adj. R-squared:            0.851
Method:              Least Squares        F-statistic:              173.4
Date:                Thu, 24 Sep 2026     Prob (F-statistic):       7.27e-128
Time:                10:17:57             Log-Likelihood:           -2377.5
No. Observations:    333                  AIC:                      4779.
Df Residuals:        321                  BIC:                      4825.
Df Model:            11
Covariance Type:     nonrobust
=====
```

	coef	std err	t	P> t
Intercept	-4270.6550	774.961	-5.511	0.000
C(Q("species"))[T.Chinstrap]	1113.1250	1301.811	0.855	0.393

```

C(Q("species"))[T.Gentoo]          -173.7598   1274.837   -0.136   0.892
-2681.851   2334.332
Q("bill_length_mm")                54.5124    10.828    5.034   0.000
33.209     75.816
C(Q("species"))[T.Chinstrap]:Q("bill_length_mm") -38.4733    18.657   -2.062   0.040
-75.179    -1.768
C(Q("species"))[T.Gentoo]:Q("bill_length_mm")    -16.9957    17.021   -0.999   0.319
-50.482    16.490
Q("bill_depth_mm")                 144.1567    23.465    6.144   0.000
97.993    190.320
C(Q("species"))[T.Chinstrap]:Q("bill_depth_mm")  -52.6440    53.766   -0.979   0.328
-158.422    53.134
C(Q("species"))[T.Gentoo]:Q("bill_depth_mm")     35.8488    49.827    0.719   0.472
-62.180    133.877
Q("flipper_length_mm")              16.9148     4.291    3.942   0.000
8.472     25.357
C(Q("species"))[T.Chinstrap]:Q("flipper_length_mm") 5.6649     7.881    0.719   0.473
-9.840     21.170
C(Q("species"))[T.Gentoo]:Q("flipper_length_mm")   6.3447     7.923    0.801   0.424
-9.242     21.931
=====
Omnibus:                8.160   Durbin-Watson:          2.165
Prob(Omnibus):          0.017   Jarque-Bera (JB):        8.027
Skew:                   0.357   Prob(JB):                0.0181
Kurtosis:               3.264   Cond. No.                2.19e+04
=====

```

Notes:

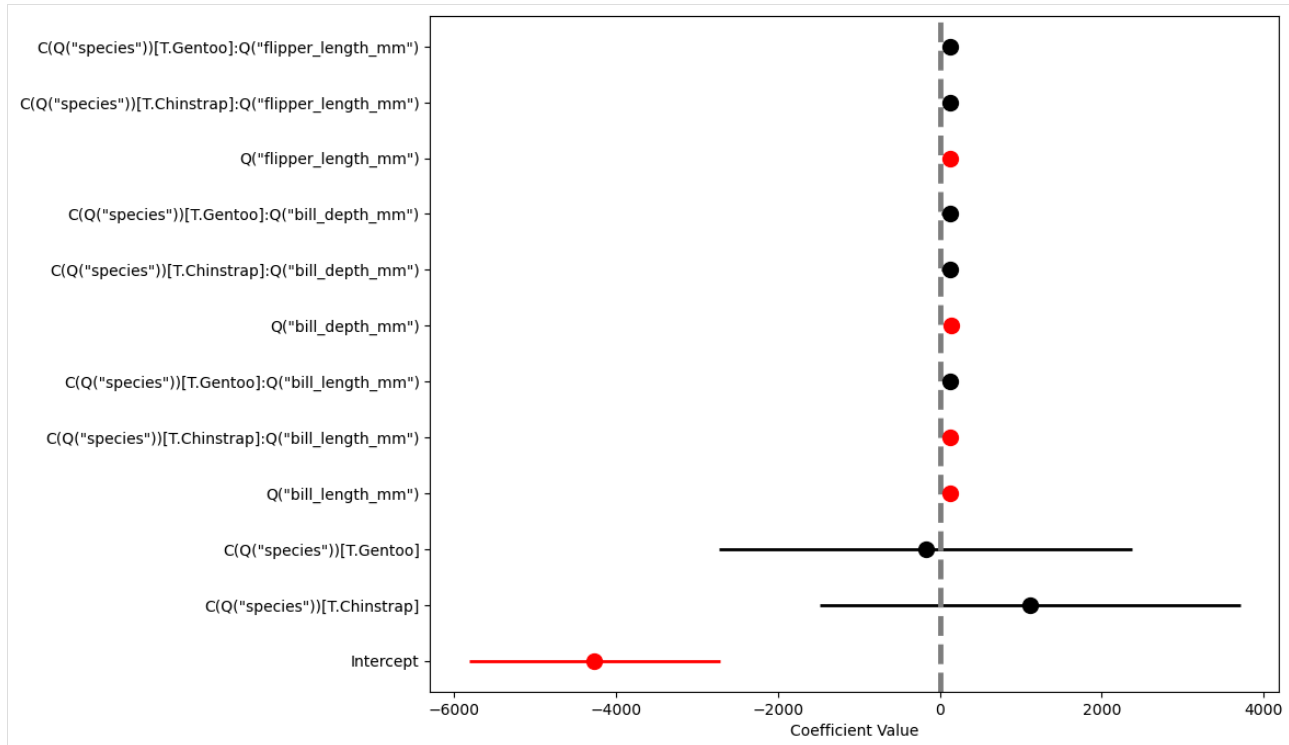
- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 2.19e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```

#####

```

```
my_coefplot(model_05, zero_nudge=120)
```



Model 06: Higher-Order Polynomials on Flipper Length (If Non-Linear Scaling Suspected)

```
model_06 = smf.ols(formula=formula_list[6],
                    data=df). \
                    fit()
```

```
model_06.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
'''
```

OLS Regression Results

```
=====
Dep. Variable:      Q("body_mass_g")    R-squared:                0.793
Model:              OLS                  Adj. R-squared:            0.790
Method:             Least Squares         F-statistic:              250.8
Date:               Thu, 24 Sep 2026      Prob (F-statistic):       1.43e-109
Time:               10:17:57              Log-Likelihood:           -2437.8
No. Observations:   333                  AIC:                      4888.
Df Residuals:       327                  BIC:                      4910.
Df Model:           5
Covariance Type:    nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	2.831e+05	6.13e+04	4.616	0.000	1.62e+05	4.04e+05
Q("bill_length_mm")	1.4184	5.070	0.280	0.780	-8.555	11.392
Q("bill_depth_mm")	53.4670	14.687	3.640	0.000	24.573	82.361
Q("flipper_length_mm")	-4143.6361	911.346	-4.547	0.000	-5936.478	-2350.795
I(Q("flipper_length_mm") ** 2)	20.1267	4.493	4.479	0.000	11.287	28.966
I(Q("flipper_length_mm") ** 3)	-0.0320	0.007	-4.354	0.000	-0.047	-0.018

```
=====
Omnibus:              10.214    Durbin-Watson:              2.101
```



```

Prob(Omnibus):          0.006   Jarque-Bera (JB):          10.260
Skew:                  0.408   Prob(JB):              0.00592
Kurtosis:              3.272   Cond. No.             2.55e+10
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

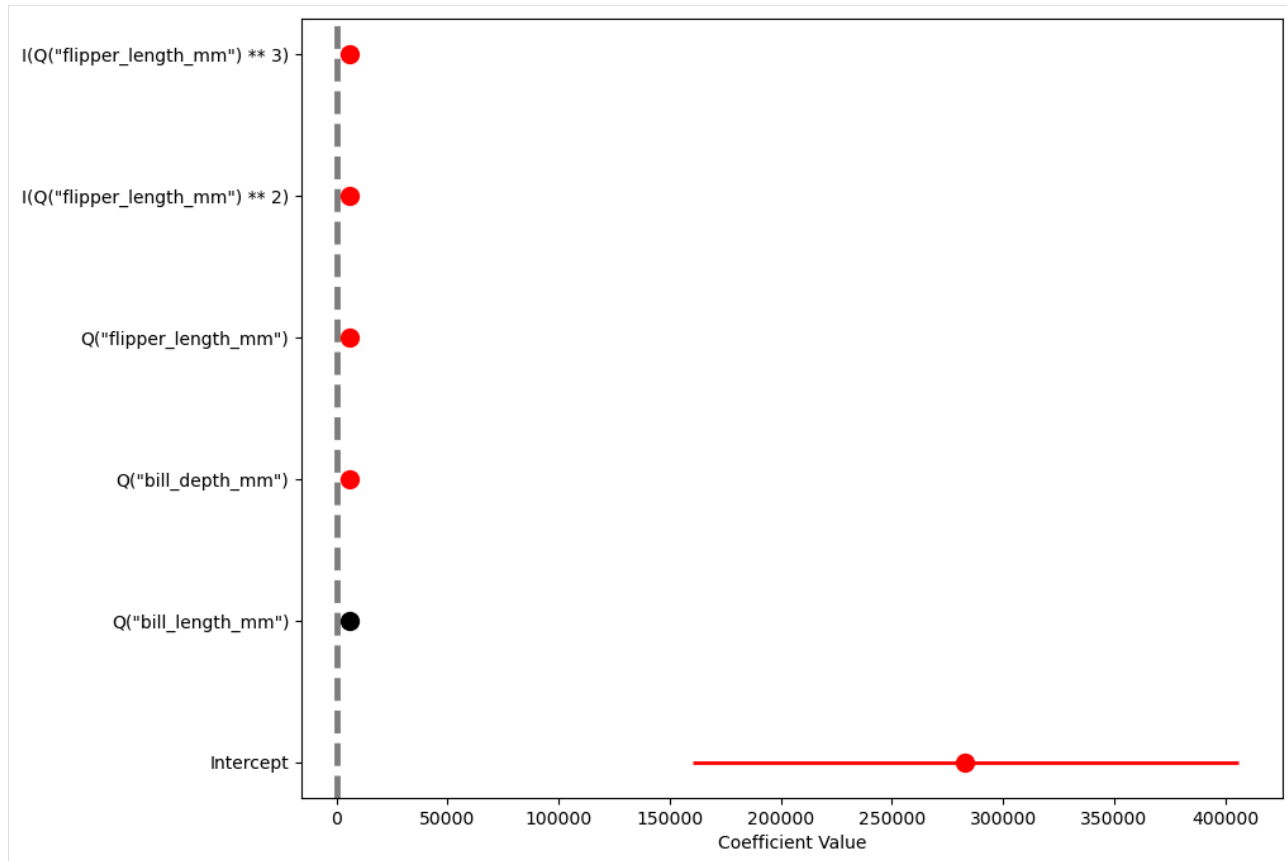
[2] The condition number is large, 2.55e+10. This might indicate that there are strong multicollinearity or other numerical problems.

```

"""

```

```
my_coefplot(model_06, zero_nudge=6200)
```

**Model 07: All Predictors, Interactions Among Continuous Terms, and Species (Full Candidate)**

```

model_07 = smf.ols(formula=formula_list[7],
                    data=df). \
fit()

```

```
model_07.summary()
```

```

<class 'statsmodels.iolib.summary.Summary'>
"""

```

OLS Regression Results

```

=====

```

```

Dep. Variable:    Q("body_mass_g")    R-squared:                0.852
Model:            OLS                  Adj. R-squared:           0.849
Method:           Least Squares        F-statistic:              233.7
Date:             Thu, 24 Sep 2026     Prob (F-statistic):       1.30e-129
Time:             10:17:57             Log-Likelihood:           -2381.7
No. Observations: 333                  AIC:                      4781.
Df Residuals:     324                  BIC:                      4816.
Df Model:          8
Covariance Type:  nonrobust

```

```

=====
                                coef    std err          t      P>|t|      [0.025
-----
0.975]
-----
Intercept                -6490.4539    5383.685     -1.206     0.229    -1.71e+04
4100.938
C(Q("species"))[T.Chinstrap]    -441.1314     89.228     -4.944     0.000    -616.671
-265.592
C(Q("species"))[T.Gentoo]         908.7480    162.852      5.580     0.000      588.366
1229.130
Q("bill_length_mm")           139.4687     83.530      1.670     0.096     -24.860
303.798
Q("bill_depth_mm")            297.0222    288.078      1.031     0.303    -269.718
863.762
Q("flipper_length_mm")          7.1624     28.747      0.249     0.803     -49.391
63.716
Q("bill_length_mm"):Q("bill_depth_mm")    -6.4401      2.765     -2.329     0.020    -11.880
-1.000
Q("bill_length_mm"):Q("flipper_length_mm")  0.0691      0.294      0.235     0.815     -0.510
0.648
Q("bill_depth_mm"):Q("flipper_length_mm")  0.5889      1.598      0.369     0.713     -2.554
3.732

```

```

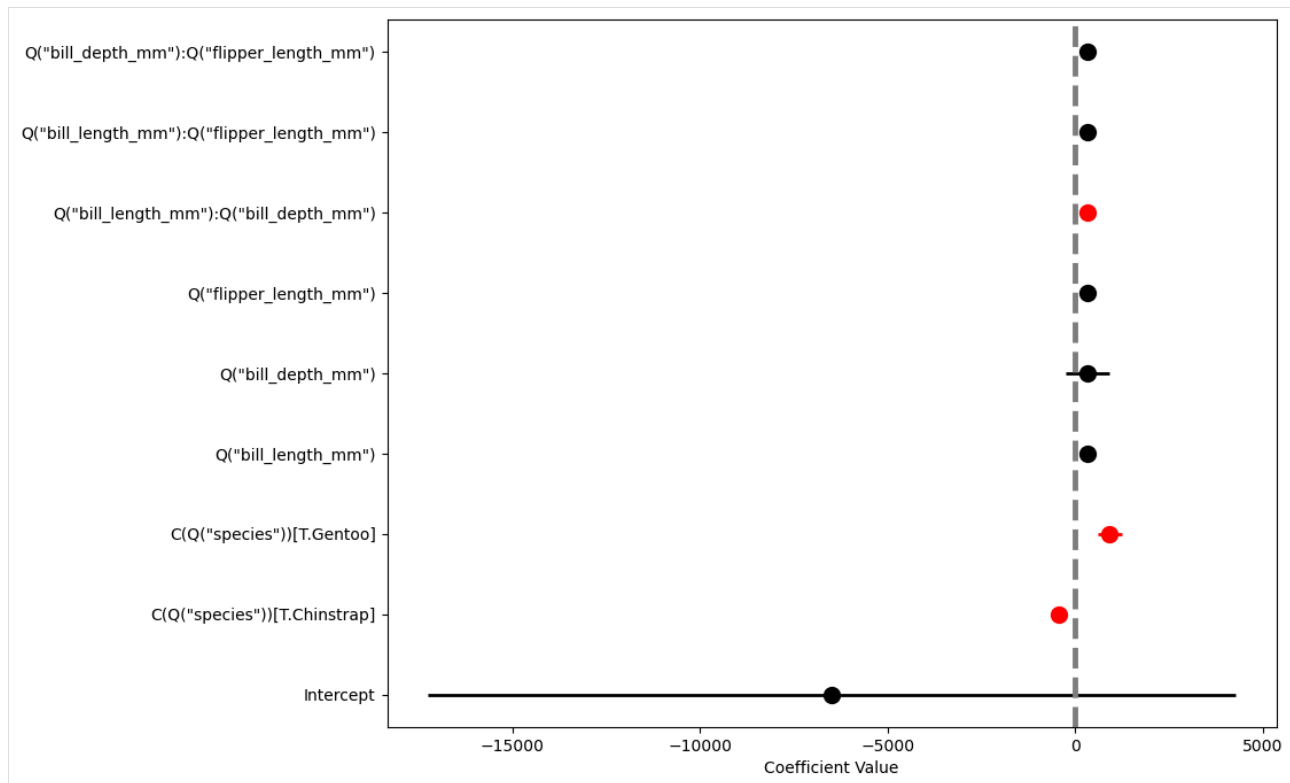
=====
Omnibus:                 6.344    Durbin-Watson:           2.195
Prob(Omnibus):           0.042    Jarque-Bera (JB):        6.131
Skew:                    0.321    Prob(JB):                 0.0466
Kurtosis:                3.173    Cond. No.                 3.04e+06
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 3.04e+06. This might indicate that there are strong multicollinearity or other numerical problems.
""""

```
my_coefplot(model_07, zero_nudge=325)
```



Model 08: Final Model (Species-Specific Slope for Flipper Length)

```
model_08 = smf.ols(formula=formula_list[8],
                    data=df). \
                    fit()
```

```
model_08.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
=====
```

OLS Regression Results

```
=====
Dep. Variable:    Q("body_mass_g")    R-squared:                0.827
Model:            OLS                 Adj. R-squared:            0.823
Method:            Least Squares       F-statistic:              259.2
Date:             Thu, 24 Sep 2026     Prob (F-statistic):       7.91e-121
Time:             10:17:57             Log-Likelihood:           -2408.3
No. Observations: 333                 AIC:                     4831.
Df Residuals:     326                 BIC:                     4857.
Df Model:         6
Covariance Type:  nonrobust
=====
```

		coef	std err	t	P> t
[0.025	0.975]				
Intercept		-3259.5585	824.921	-3.951	0.000
-4882.398	-1636.719				
C(Q("species"))[T.Chinstrap]		-100.0775	1401.346	-0.071	0.943
-2856.901	2656.746				

```

C(Q("species"))[T.Gentoo]          -2233.5848   1337.167   -1.670   0.096
-4864.150    396.980
Q("flipper_length_mm")             24.8689    4.421    5.625   0.000
16.172     33.566
Q("flipper_length_mm"):C(Q("species"))[T.Chinstrap]  -3.0247    7.248   -0.417   0.677
-17.282    11.233
Q("flipper_length_mm"):C(Q("species"))[T.Gentoo]    11.2368    6.528    1.721   0.086
-1.605     24.079
Q("bill_length_mm")                57.6468    7.327    7.867   0.000
43.232     72.061
=====
Omnibus:                6.111   Durbin-Watson:           2.281
Prob(Omnibus):          0.047   Jarque-Bera (JB):        6.069
Skew:                   0.330   Prob(JB):                0.0481
Kurtosis:               3.029   Cond. No.                2.13e+04
=====

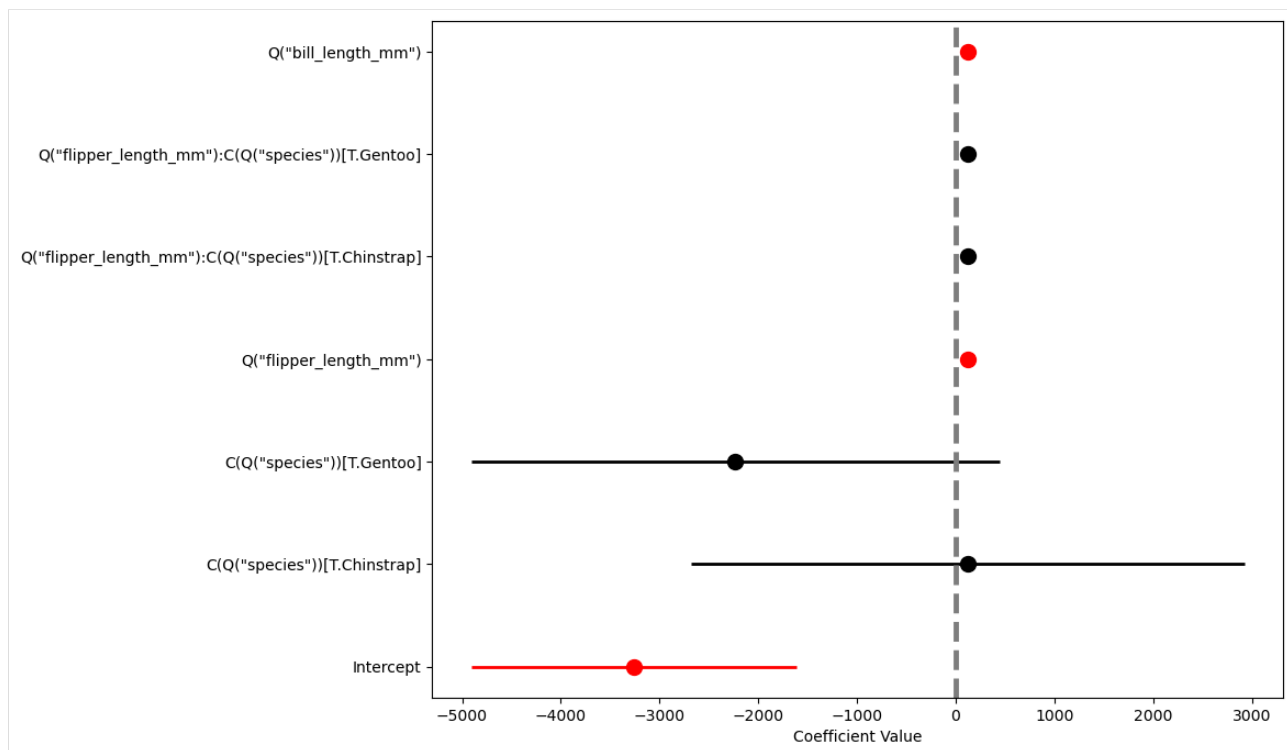
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.13e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
my_coefplot(model_08)
```

**Compare Fitted Models - Training Data**

```

ols_results_list = []

for model in range(len(formula_list)):
    ols_results_list.append(fit_and_assess_ols(model,

```

```
formula_list[model],
df))
```

```
ols_results_df = pd.concat(ols_results_list,
                           ignore_index=True)
```

```
ols_results_df
```

	model_name	model_formula	num_coefs	\
0	0	Q("body_mass_g") ~ 1	1	
1	1	Q("body_mass_g") ~ C(Q("species"))	3	
2	2	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	4	
3	3	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6	
4	4	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	10	
5	5	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...	12	
6	6	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6	
7	7	Q("body_mass_g") ~ (Q("bill_length_mm") + Q("b...	9	
8	8	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...	7	

	R-squared	RMSE
0	4.440892e-16	804.005860
1	6.744887e-01	458.714246
2	7.639367e-01	390.637086
3	8.494949e-01	311.913936
4	8.159175e-01	344.957359
5	8.559855e-01	305.114117
6	7.931653e-01	365.654520
7	8.522711e-01	309.023817
8	8.266837e-01	334.717896

Fit Linear Regression Models w/ Cross-Validation - Test Data

Functions

```
def linear_regression_cross_val_score(model, formula, init_model, df, cv):
    """
    Perform cross-validated evaluation of a linear-style model using a Patsy formula.

    This function builds design matrices from `formula` and `df` using patsy.dmatrices,
    then runs cross-validation using sklearn.model_selection.cross_val_score to compute per-fold
    R-squared and root-mean-squared-error (RMSE) scores. The per-fold results and some metadata
    are returned as a pandas DataFrame.

    Parameters
    -----
    model : str
        A human-readable name for the model; stored in the returned DataFrame.
    formula : str
        A Patsy-compatible formula string (e.g. "y ~ x1 + x2 + C(group)") used to create y and X.
    init_model : object
        An unfitted scikit-learn-style estimator (must implement fit/predict and be compatible with
        sklearn.model_selection.cross_val_score). Example: LinearRegression().
    df : pandas.DataFrame
        DataFrame containing all variables referenced in `formula`.
    cv : int or cross-validation splitter
        Number of folds (int) or a cross-validation splitter instance passed to cross_val_score.
```

Returns

pandas.DataFrame

A DataFrame with one row per CV fold containing:

- 'R-squared' : R^2 score on the test fold (from cross_val_score default scoring)
- 'RMSE' : Root mean squared error on the test fold
- 'fold_id' : 1-based fold identifier
- 'model_name' : the provided `model`
- 'model_formula' : the provided `formula`
- 'num_coefs' : number of columns in the design matrix X

.....

```
# Create the feature and output arrays using the formula
y, X = dmatrices( formula, data=df )
```

```
# Train and test within each fold - return the test set scores
```

```
# R-squared
```

```
test_r2 = cross_val_score(init_model,
                          X,
                          y.ravel(),
                          cv=cv)
```

```
# RMSE
```

```
test_rmse = -cross_val_score(init_model,
                              X,
                              y.ravel(),
                              cv=cv,
                              scoring='neg_root_mean_squared_error')
```

```
# Extract model metrics
```

```
results_df=pd.DataFrame({'R-squared': test_r2,
                          'RMSE': test_rmse})
```

```
results_df['fold_id'] = results_df.index + 1
```

```
results_df['model_name'] = model
```

```
results_df['model_formula'] = formula
```

```
results_df['num_coefs'] = X.shape[1]
```

```
# Return the results DataFrame
```

```
return results_df
```

Fit w/ Cross-Validation

```
kf = KFold(n_splits=5, shuffle=True, random_state=101)
```

```
lm = LinearRegression(fit_intercept=False)
```

Compare Fitted Models - Test Data

```
cv_score_list = []
```

```
for model in range(len(formula_list)):
```

```
    print('-- Formula ID %d --' % model)
```

```
    cv_score_list.append(linear_regression_cross_val_score(model,
                                                            formula_list[model],
                                                            init_model=lm,
                                                            df=df,
                                                            cv=kf) )
```

```
-- Formula ID 0 --
-- Formula ID 1 --
-- Formula ID 2 --
-- Formula ID 3 --
-- Formula ID 4 --
-- Formula ID 5 --
-- Formula ID 6 --
-- Formula ID 7 --
-- Formula ID 8 --
```

```
cv_score_df=pd.concat(cv_score_list, ignore_index=True)
```

```
cv_score_df
```

	R-squared	RMSE	fold_id	model_name	\
0	-4.526133e-02	812.420097	1		0
1	-3.363930e-07	735.247849	2		0
2	-6.889831e-05	792.792792	3		0
3	-6.097962e-03	839.647092	4		0
4	-1.400301e-02	845.708003	5		0
5	6.381451e-01	478.008282	1		1
6	6.301439e-01	447.146775	2		1
7	7.303516e-01	411.664747	3		1
8	6.918962e-01	464.649173	4		1
9	6.414559e-01	502.888977	5		1
10	7.606020e-01	388.801957	1		2
11	6.965466e-01	405.022996	2		2
12	7.292803e-01	412.481694	3		2
13	8.160409e-01	359.035398	4		2
14	7.528083e-01	417.558889	5		2
15	8.554658e-01	302.101930	1		3
16	7.868855e-01	339.422137	2		3
17	8.310358e-01	325.868316	3		3
18	8.922926e-01	274.725835	4		3
19	8.375904e-01	338.459441	5		3
20	8.081037e-01	348.098027	1		4
21	7.827729e-01	342.681524	2		4
22	7.949057e-01	359.022429	3		4
23	7.490267e-01	419.363278	4		4
24	7.793542e-01	394.501524	5		4
25	8.614673e-01	295.763394	1		5
26	8.023321e-01	326.890091	2		5
27	8.253085e-01	331.345110	3		5
28	8.846118e-01	284.352655	4		5
29	8.414184e-01	334.446902	5		5
30	8.012127e-01	354.292979	1		6
31	7.207259e-01	388.551851	2		6
32	7.618194e-01	386.899294	3		6
33	8.527424e-01	321.229617	4		6
34	7.679106e-01	404.602406	5		6
35	8.526903e-01	304.988836	1		7
36	7.951294e-01	332.792458	2		7
37	8.344838e-01	322.526193	3		7
38	8.937218e-01	272.897008	4		7
39	8.285500e-01	347.751957	5		7
40	8.393879e-01	318.461806	1		8
41	7.781769e-01	346.287729	2		8
42	8.005445e-01	354.052601	3		8

43	8.481728e-01	326.175606	4	8
44	8.173335e-01	358.946943	5	8

	model_formula	num_coefs
0	Q("body_mass_g") ~ 1	1
1	Q("body_mass_g") ~ 1	1
2	Q("body_mass_g") ~ 1	1
3	Q("body_mass_g") ~ 1	1
4	Q("body_mass_g") ~ 1	1
5	Q("body_mass_g") ~ C(Q("species"))	3
6	Q("body_mass_g") ~ C(Q("species"))	3
7	Q("body_mass_g") ~ C(Q("species"))	3
8	Q("body_mass_g") ~ C(Q("species"))	3
9	Q("body_mass_g") ~ C(Q("species"))	3
10	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	4
11	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	4
12	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	4
13	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	4
14	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	4
15	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
16	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
17	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
18	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
19	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
20	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	10
21	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	10
22	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	10
23	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	10
24	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	10
25	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...	12
26	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...	12
27	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...	12
28	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...	12
29	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...	12
30	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
31	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
32	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
33	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
34	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
35	Q("body_mass_g") ~ (Q("bill_length_mm") + Q("b...	9
36	Q("body_mass_g") ~ (Q("bill_length_mm") + Q("b...	9
37	Q("body_mass_g") ~ (Q("bill_length_mm") + Q("b...	9
38	Q("body_mass_g") ~ (Q("bill_length_mm") + Q("b...	9
39	Q("body_mass_g") ~ (Q("bill_length_mm") + Q("b...	9
40	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...	7
41	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...	7
42	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...	7
43	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...	7
44	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...	7

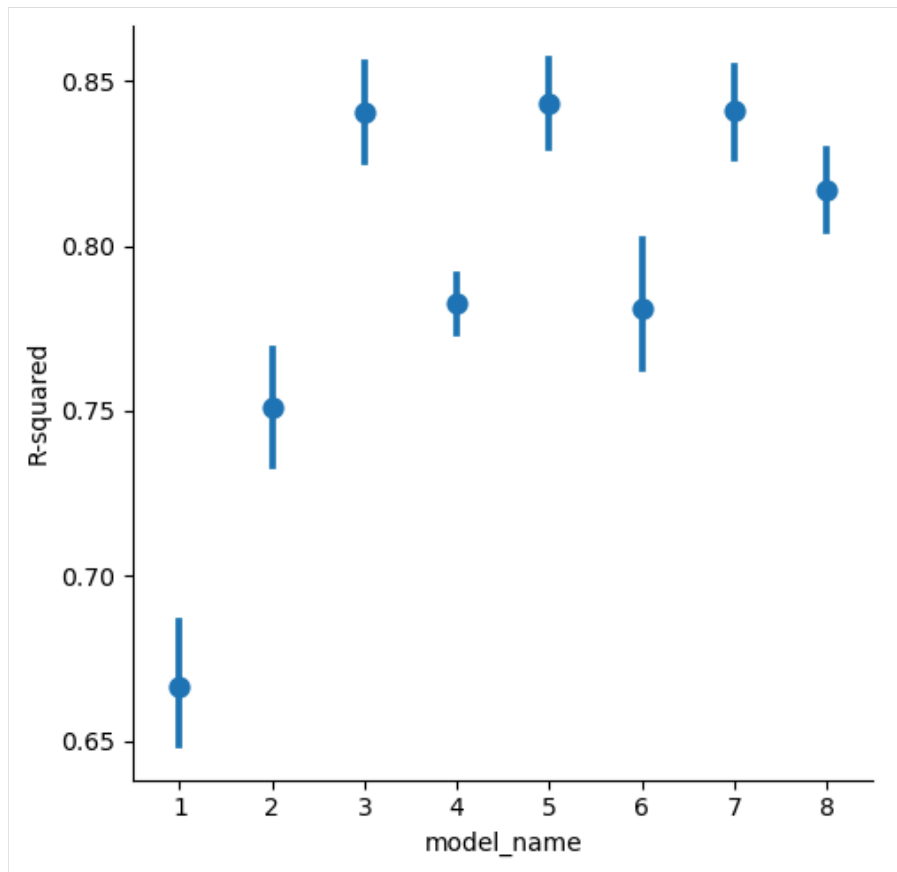
Visualize Average R-squared w/ 1 Standard Error Interval 68% Confidence Interval on the Average

```
cv_score_df.groupby(['model_name',
                    'num_coefs',
                    'model_formula']). \
    aggregate(r2_avg=('R-squared', 'mean'),
             r2_sem=('R-squared', 'sem')). \
    reset_index(). \
    sort_values(by='r2_avg', ascending=False). \
    reset_index(drop=True)
```


	model_name	num_coefs	model_formula
0	5	12	$Q(\text{"body_mass_g"}) \sim C(Q(\text{"species"})) * (Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"}))$
1	7	9	$Q(\text{"body_mass_g"}) \sim (Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"}))$
2	3	6	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"})$
3	8	7	$Q(\text{"body_mass_g"}) \sim Q(\text{"flipper_length_mm"}) + Q(\text{"body_mass_g"})$
4	4	10	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"})$
5	6	6	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"})$
6	2	4	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"})$
7	1	3	$Q(\text{"body_mass_g"}) \sim C(Q(\text{"species"}))$
8	0	1	$Q(\text{"body_mass_g"}) \sim 1$

	r2_avg	r2_sem
0	0.843028	0.014216
1	0.840915	0.016152
2	0.840654	0.017145
3	0.816723	0.012749
4	0.782833	0.009847
5	0.780882	0.022054
6	0.751056	0.019701
7	0.666399	0.019333
8	-0.013086	0.008443

```
sns.catplot(data=cv_score_df.loc[cv_score_df.model_name > 0, :],
            x='model_name',
            y='R-squared',
            kind='point',
            linestyle='none',
            errorbar=('ci', 68))
plt.show()
```



Visualize Average RMSE w/ 1 Standard Error Interval 68% Confidence Interval on the Average

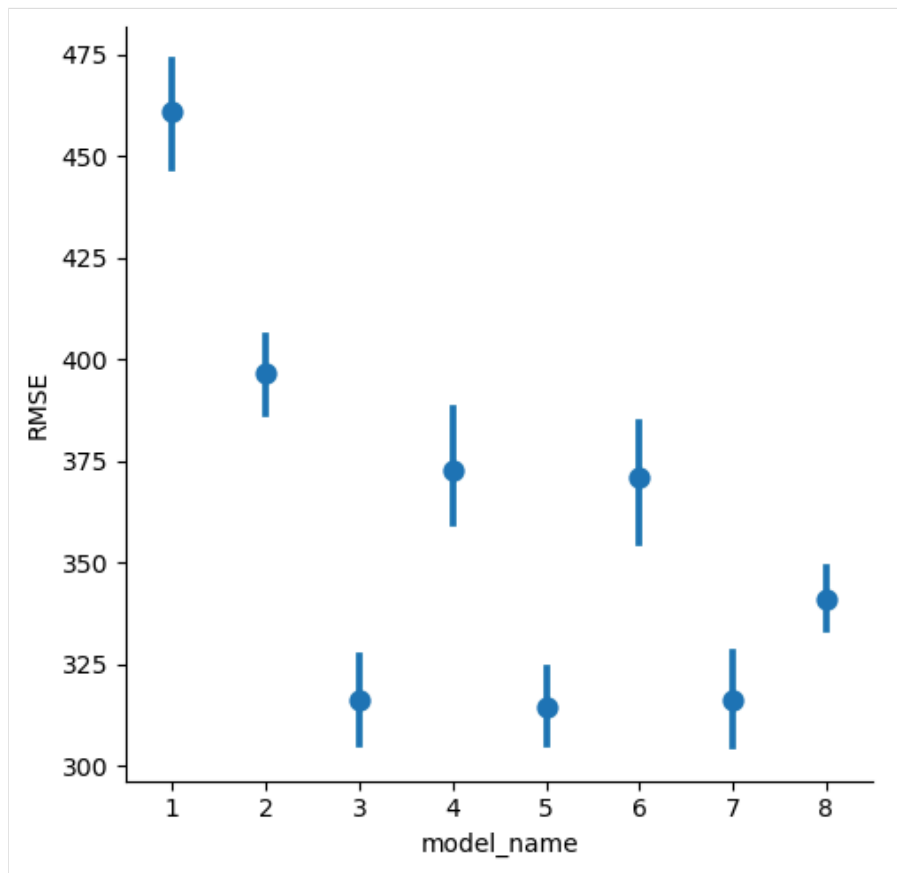
```
cv_score_df.groupby(['model_name',
                    'num_coefs',
                    'model_formula']). \
    aggregate(rmse_avg=('RMSE', 'mean'),
              rmse_sem=('RMSE', 'sem')). \
    reset_index(). \
    sort_values(by='rmse_avg', ascending=True). \
    reset_index(drop=True)
```

	model_name	num_coefs	model_formula \
0	5	12	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...
1	3	6	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...
2	7	9	Q("body_mass_g") ~ (Q("bill_length_mm") + Q("b...
3	8	7	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...
4	6	6	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...
5	4	10	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...
6	2	4	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...
7	1	3	Q("body_mass_g") ~ C(Q("species"))
8	0	1	Q("body_mass_g") ~ 1

	rmse_avg	rmse_sem
0	314.559630	10.234882
1	316.115532	12.343121
2	316.191290	12.867974
3	340.784937	7.899129
4	371.115229	14.909940
5	372.733356	14.741446

```
6 396.580187 10.569666
7 460.871591 15.302093
8 805.163167 19.908372
```

```
sns.catplot(data=cv_score_df.loc[cv_score_df.model_name > 0, :],
            x='model_name',
            y='RMSE',
            kind='point',
            linestyle='none',
            errorbar=('ci', 68))
plt.show()
```



Visualize Model 3 and Statistical Significance

```
model_03.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
.....
```

OLS Regression Results

Dep. Variable:	Q("body_mass_g")	R-squared:	0.849
Model:	OLS	Adj. R-squared:	0.847
Method:	Least Squares	F-statistic:	369.1
Date:	Thu, 24 Sep 2026	Prob (F-statistic):	4.22e-132
Time:	10:17:57	Log-Likelihood:	-2384.8

```

No. Observations:      333   AIC:      4782.
Df Residuals:         327   BIC:      4805.
Df Model:              5
Covariance Type:      nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept          -4282.0802     497.832     -8.601     0.000    -5261.438    -3302.723
C(Q("species"))[T.Chinstrap] -496.7583     82.469     -6.024     0.000    -658.995    -334.521
C(Q("species"))[T.Gentoo]    965.1983    141.770     6.808     0.000     686.301    1244.096
Q("bill_length_mm")      39.7184      7.227      5.496     0.000      25.501      53.936
Q("bill_depth_mm")     141.7714     19.163      7.398     0.000     104.072     179.470
Q("flipper_length_mm")    20.2264      3.135      6.452     0.000      14.059      26.394
=====
Omnibus:             7.321   Durbin-Watson:      2.247
Prob(Omnibus):       0.026   Jarque-Bera (JB):      7.159
Skew:                0.348   Prob(JB):             0.0279
Kurtosis:            3.179   Cond. No.             6.01e+03
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 6.01e+03. This might indicate that there are strong multicollinearity or other numerical problems.

.....

model_03.params

```

Intercept          -4282.080164
C(Q("species"))[T.Chinstrap] -496.758314
C(Q("species"))[T.Gentoo]    965.198291
Q("bill_length_mm")      39.718385
Q("bill_depth_mm")     141.771362
Q("flipper_length_mm")    20.226447
dtype: float64

```

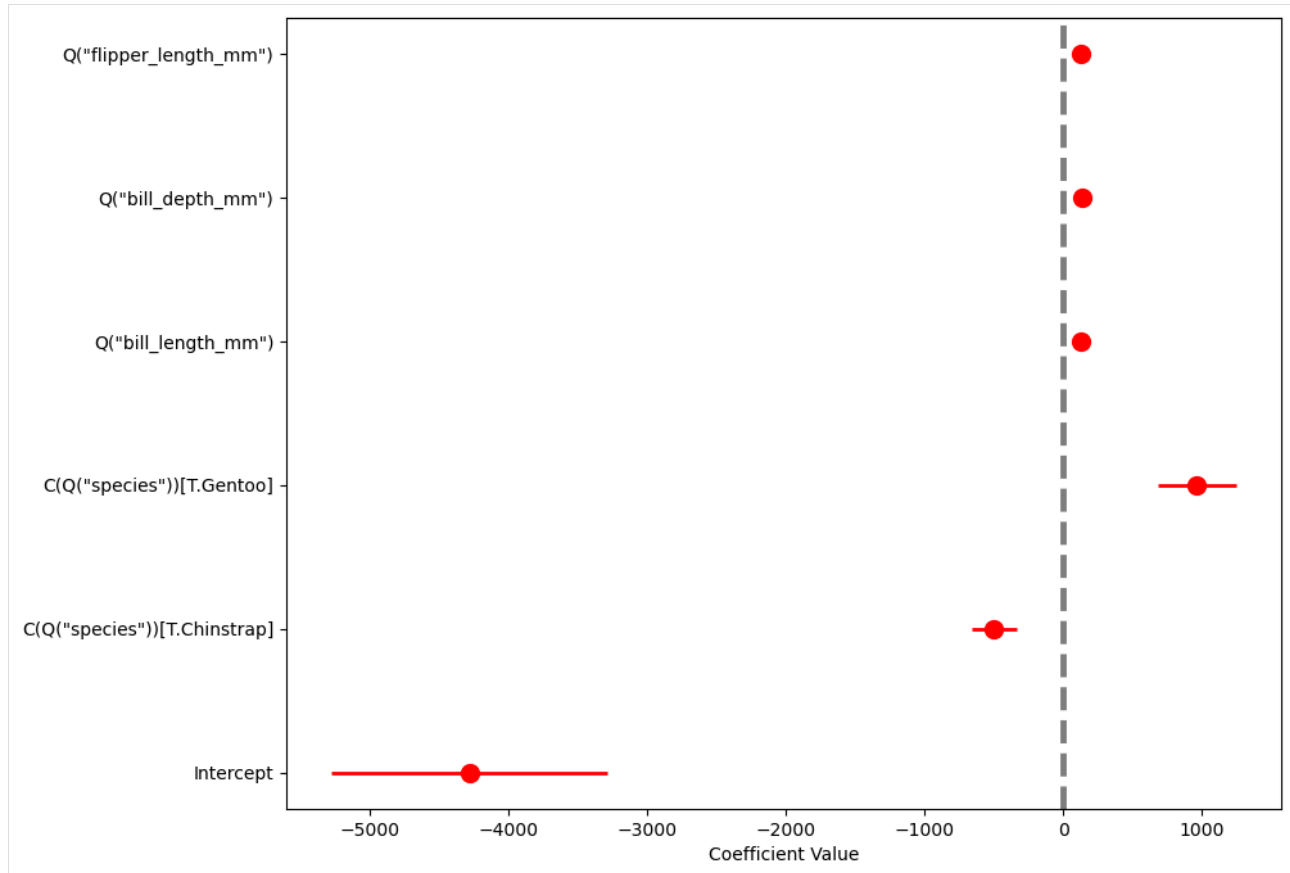
model_03.pvalues

```

Intercept          3.332545e-16
C(Q("species"))[T.Chinstrap] 4.590481e-09
C(Q("species"))[T.Gentoo]    4.740533e-11
Q("bill_length_mm")      7.845842e-08
Q("bill_depth_mm")     1.174106e-12
Q("flipper_length_mm")    3.982016e-10
dtype: float64

```

my_coefplot(model_03)



Visualize Model 3 Predicted Response vs `bill_depth_mm` with 95% Mean CI and 95% Prediction Interval

```
formula_list[3]
```

```
'Q("body_mass_g") ~ Q("bill_length_mm") + Q("bill_depth_mm") + Q("flipper_length_mm") +
C(Q("species"))'
```

```
pd.DataFrame({'coefficient': model_03.params[model_03.pvalues < 0.05].abs(),
              'sign': model_03.params[model_03.pvalues < 0.05]. \
                  apply(lambda x: 'positive' if x > 0 else 'negative')}). \
reset_index(). \
groupby('index'). \
aggregate({'coefficient': 'first',
            'sign': 'first'}). \
sort_values(by='coefficient', ascending=False)
```

index	coefficient	sign
Intercept	4282.080164	negative
C(Q("species"))[T.Gentoo]	965.198291	positive
C(Q("species"))[T.Chinstrap]	496.758314	negative
Q("bill_depth_mm")	141.771362	positive
Q("bill_length_mm")	39.718385	positive
Q("flipper_length_mm")	20.226447	positive

```
# Sequence for bill_depth_mm (primary x1)
df_viz_3 = pd.DataFrame({'bill_depth_mm': np.linspace(df.bill_depth_mm.min() - 0.02,
                                                    df.bill_depth_mm.max() + 0.02,
                                                    num=151)})

# Fix bill_length_mm at midpoint between its own min and mean (lower half)
df_viz_3['bill_length_mm'] = 0.5 * (df.bill_length_mm.min() + df.bill_length_mm.mean())

# Hold flipper_length_mm at mean
df_viz_3['flipper_length_mm'] = df.flipper_length_mm.mean()

# Fix species at a chosen level (baseline)
df_viz_3['species'] = 'Gentoo'
```

```
# Get predictions (must pass exactly the columns expected by the model)
pred_summary_03 = model_03.get_prediction(
    df_viz_3[['bill_depth_mm',
              'bill_length_mm',
              'flipper_length_mm',
              'species']]).summary_frame()
```

```
# Create the figure and axis
fig, ax = plt.subplots()

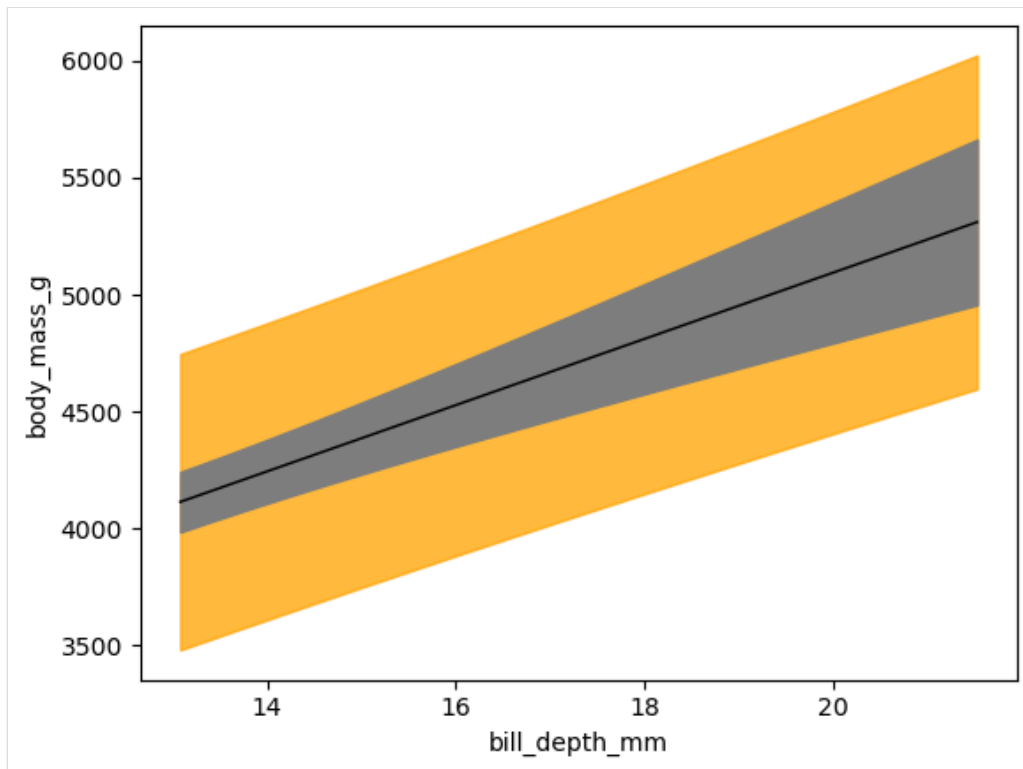
# Prediction interval
ax.fill_between(df_viz_3.bill_depth_mm,
               pred_summary_03.obs_ci_lower,
               pred_summary_03.obs_ci_upper,
               facecolor='orange',
               alpha=0.75,
               edgecolor='orange')

# Confidence interval
ax.fill_between(df_viz_3.bill_depth_mm,
               pred_summary_03.mean_ci_lower,
               pred_summary_03.mean_ci_upper,
               facecolor='grey',
               edgecolor='grey')

# Trend
ax.plot(df_viz_3.bill_depth_mm,
        pred_summary_03['mean'],
        color='k',
        linewidth=1)

# Set the labels
ax.set_xlabel('bill_depth_mm')
ax.set_ylabel('body_mass_g')

# Show the plot
plt.show()
```



Visualize Model 3 Predicted Response vs `bill_depth_mm` Across Multiple Fixed `bill_length_mm` Values (Mean Curves Only)

```
formula_list[3]
```

```
'Q("body_mass_g") ~ Q("bill_length_mm") + Q("bill_depth_mm") + Q("flipper_length_mm") +  
C(Q("species"))'
```

```
# Build the varying grid for the two predictors of interest
input_grid = pd.DataFrame([(bill_depth_mm, bill_length_mm) for bill_depth_mm in
np.linspace(df.bill_depth_mm.min(),
df.bill_depth_mm.max(),
num=101)
for bill_length_mm in np.linspace(df.bill_length_mm.min(),
df.bill_length_mm.max(),
num=9)],
columns=['bill_depth_mm', 'bill_length_mm'])

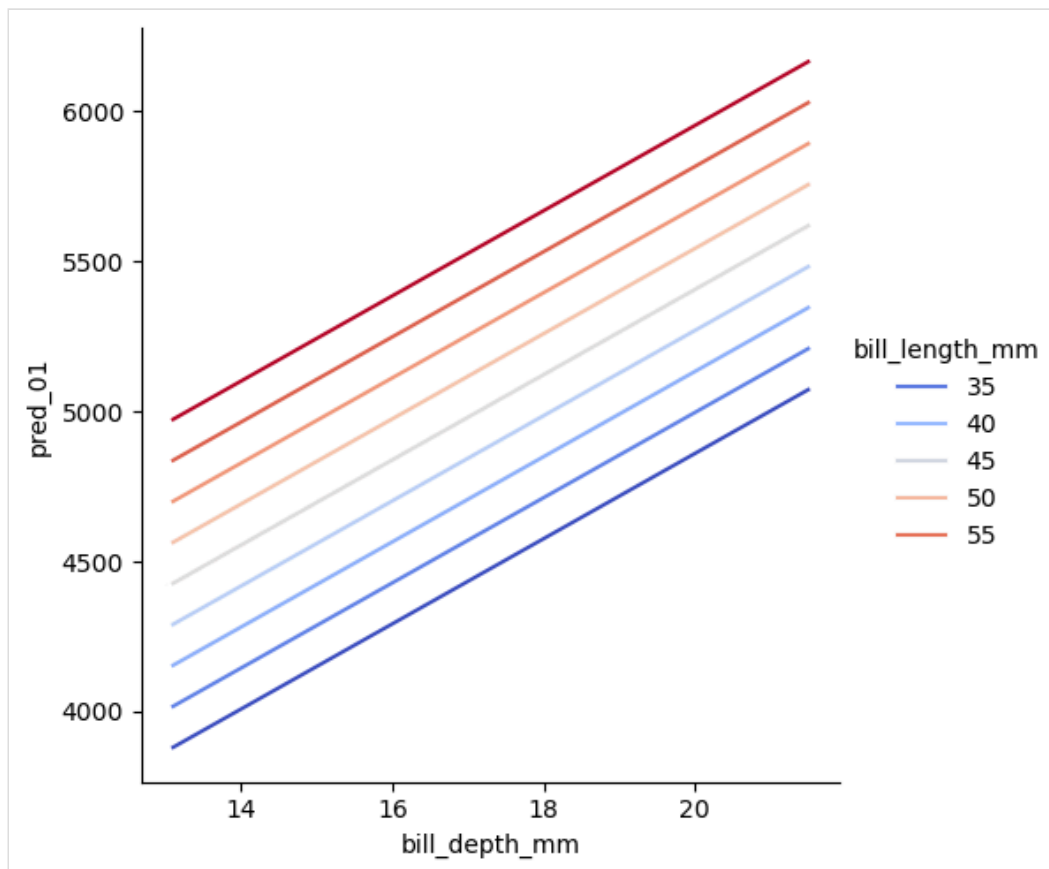
# Add fixed values for the remaining predictors
input_grid['flipper_length_mm'] = df.flipper_length_mm.mean()
input_grid['species'] = 'Gentoo'
```

```
# Create a copy of the input grid for visualization
df_viz_1 = input_grid.copy()
```

```
# Make predictions on the varying grid
df_viz_1['pred_01'] = model_03.predict(input_grid)
```

```
# Create a line plot for the predictions
sns.relplot(data=df_viz_1,
            x='bill_depth_mm',
            y='pred_01',
            kind='line',
            hue='bill_length_mm',
            estimator=None,
            units='bill_length_mm',
            palette='coolwarm')
```

```
# Show the plot
plt.show()
```



Visualize Model 3 Predicted Response vs `bill_length_mm` Across Multiple Fixed `bill_depth_mm` Values (Mean Curves Only)

```
formula_list[3]
```

```
'Q("body_mass_g") ~ Q("bill_length_mm") + Q("bill_depth_mm") + Q("flipper_length_mm") +  
C(Q("species"))'
```



```
# Build the varying grid for the two predictors of interest
input_grid = pd.DataFrame([(bill_length_mm, bill_depth_mm) for bill_length_mm in
np.linspace(df.bill_length_mm.min(),
df.bill_length_mm.max(),
num=101)
for bill_depth_mm in np.linspace(df.bill_depth_mm.min(),
df.bill_depth_mm.max(),
num=9)],
columns=['bill_length_mm', 'bill_depth_mm'])

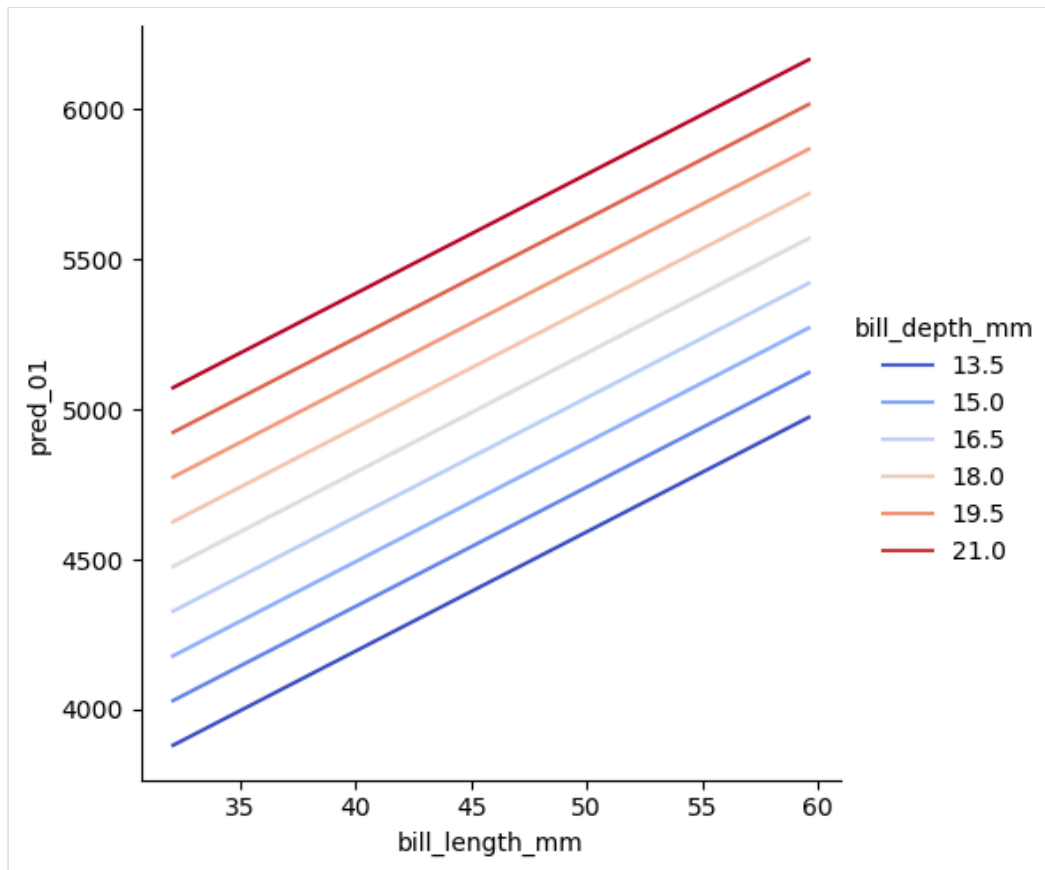
# Add fixed values for the remaining predictors
input_grid['flipper_length_mm'] = df.flipper_length_mm.mean()
input_grid['species'] = 'Gentoo'
```

```
# Create a copy of the input grid for visualization
df_viz_1 = input_grid.copy()
```

```
# Make predictions on the varying grid
df_viz_1['pred_01'] = model_03.predict(input_grid)
```

```
# Create a line plot for the predictions
sns.relplot(data=df_viz_1,
            x='bill_length_mm',
            y='pred_01',
            kind='line',
            hue='bill_depth_mm',
            estimator=None,
            units='bill_depth_mm',
            palette='coolwarm')

# Show the plot
plt.show()
```



Verify No Residual Pattern w/ Inputs

```
formula_list[3]
```

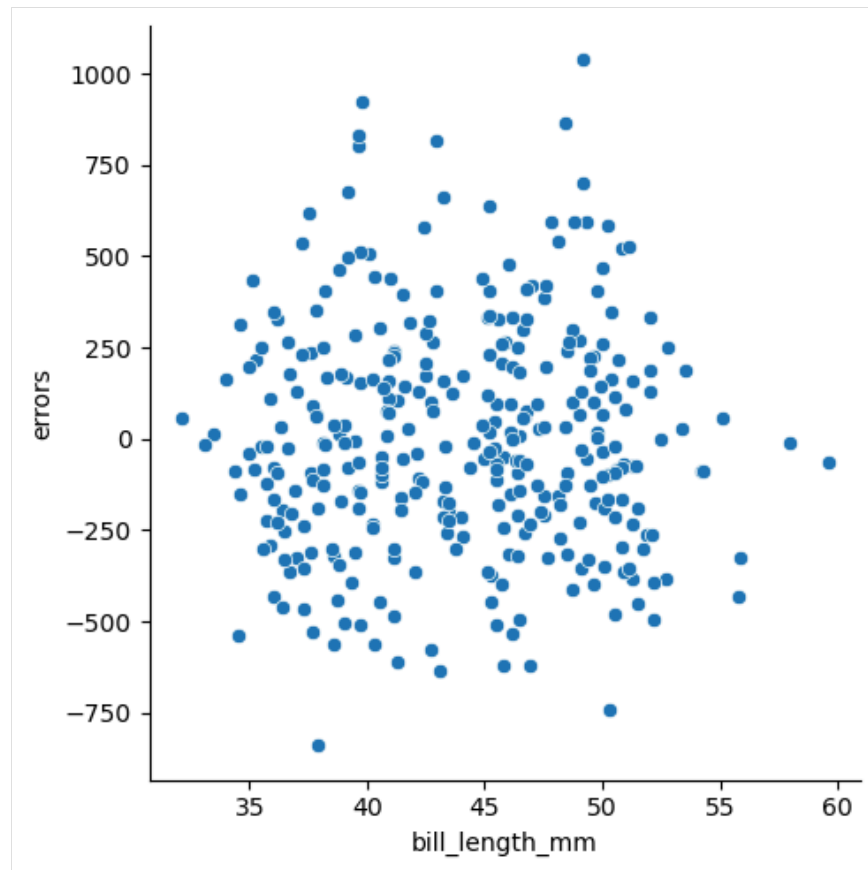
```
'Q("body_mass_g") ~ Q("bill_length_mm") + Q("bill_depth_mm") + Q("flipper_length_mm") +  
C(Q("species"))'
```

```
df['fitted'] = model_03.fittedvalues
```

```
df['errors'] = df['body_mass_g'] - df['fitted']
```

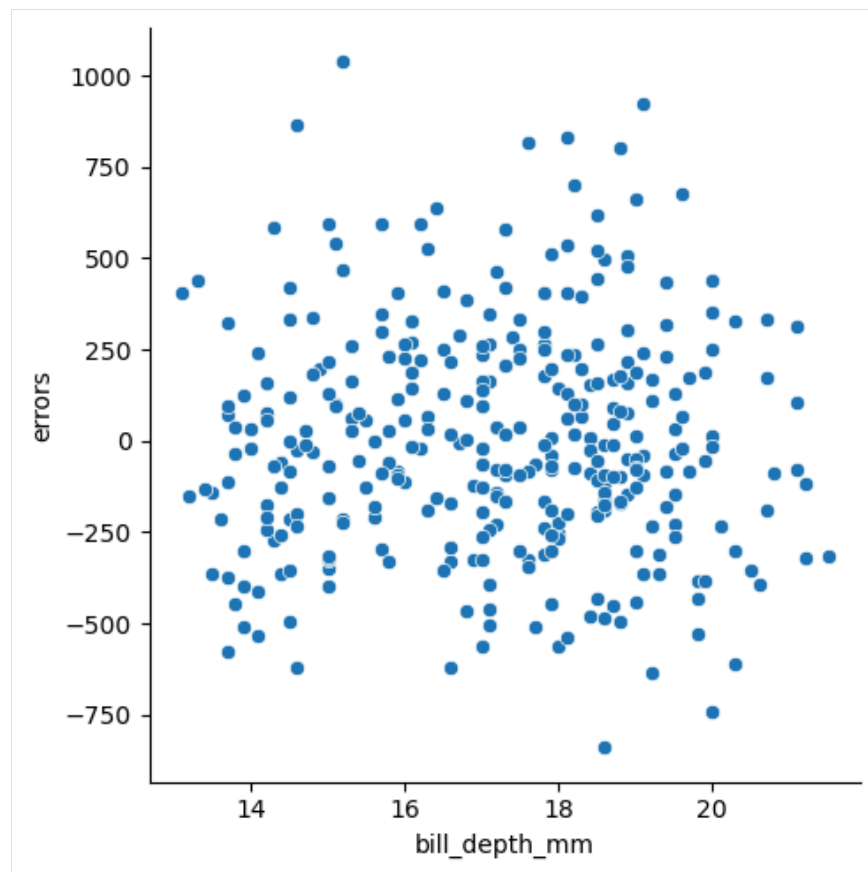
Model 3 Residuals vs bill_length_mm (No Pattern Expected)

```
sns.relplot(data=df,  
            x='bill_length_mm',  
            y='errors')  
plt.show()
```

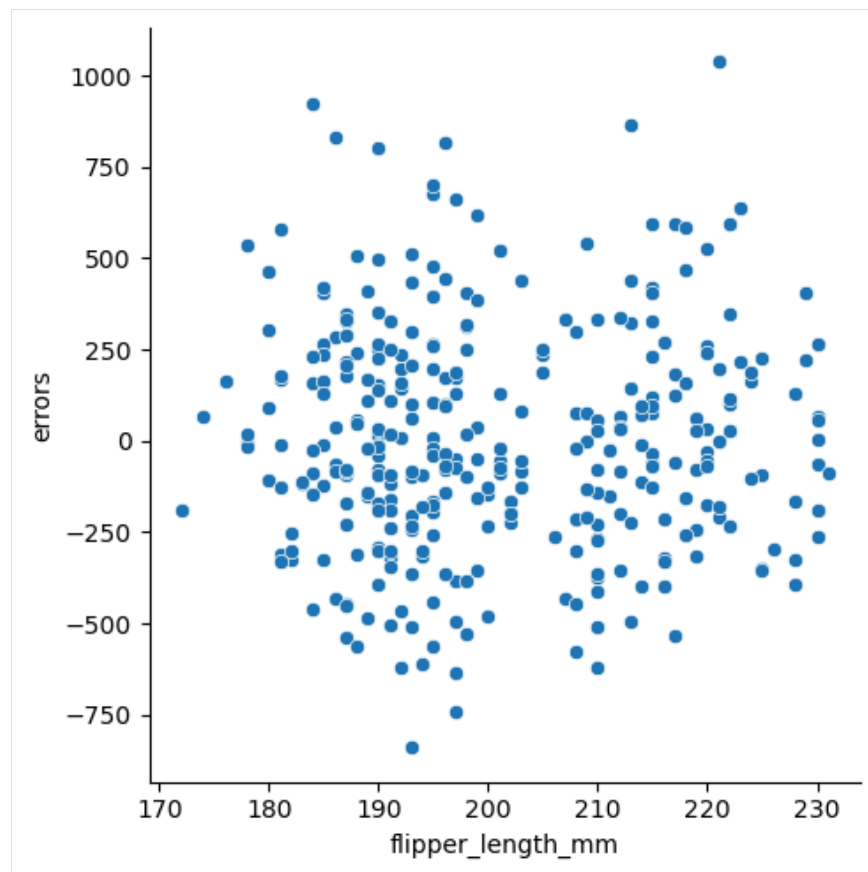


Model 3 Residuals vs bill_depth_mm (No Pattern Expected)

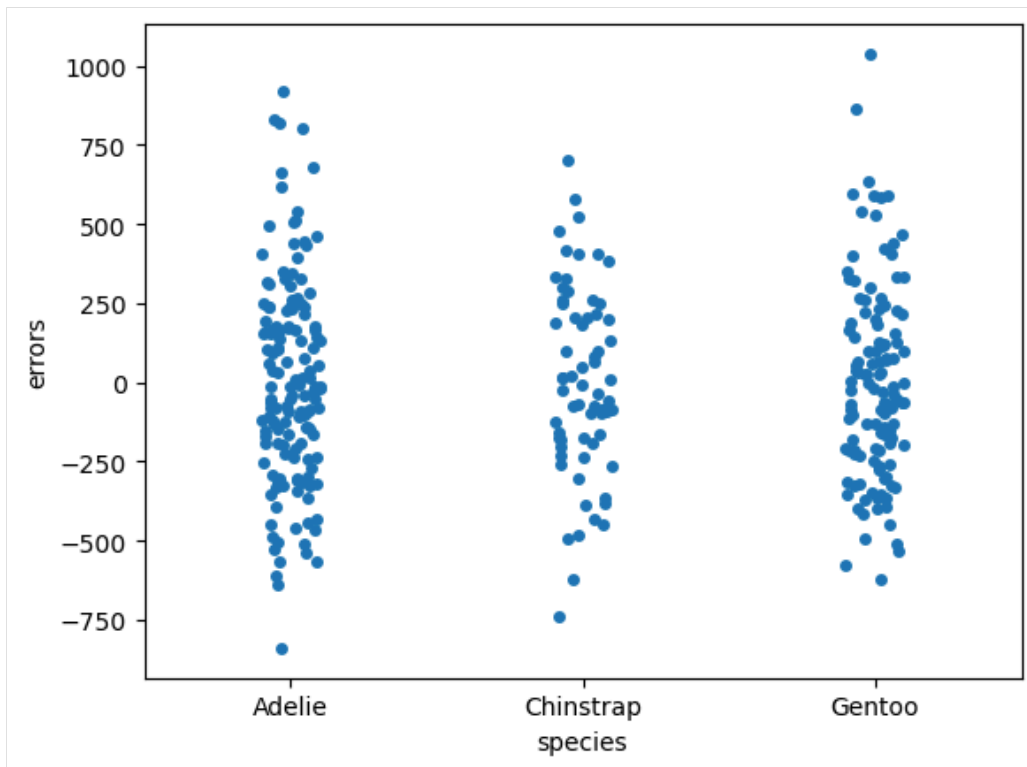
```
sns.relplot(data=df,  
            x='bill_depth_mm',  
            y='errors')  
plt.show()
```

**Model 3 Residuals vs flipper_length_mm (No Pattern Expected)**

```
sns.relplot(data=df,  
            x='flipper_length_mm',  
            y='errors')  
plt.show()
```

**Model 3 Residuals vs species (No Pattern Expected)**

```
sns.stripplot(data=df,  
              x='species',  
              y='errors')  
plt.ylabel('errors')  
plt.show()
```



Save Best Model - Model 3 w/ Highest R-squared and Lowest RMSE w/ Least Coefficients

Save Model

```
with open('models/model_03.pkl', 'wb') as f:
    pickle.dump(model_03, f)
print('model saved to model_03.pkl')
```

```
model saved to model_03.pkl
```

Load Model

```
try:
    with open('models/model_03.pkl', 'rb') as f:
        model_03 = pickle.load(f)
except FileNotFoundError:
    print('model_03.pkl not found')
except pickle.UnpicklingError:
    print('failed to unpickle model_03.pkl (file may be corrupted or unsafe)')
else:
    print('model loaded from model_03.pkl')
```

```
model loaded from model_03.pkl
```

Inference on New Data

```
new_penguin_data = pd.DataFrame({
    'bill_length_mm': [22.0],
    'bill_depth_mm': [15.0],
    'flipper_length_mm': [200.0],
    'species': ['Gentoo']
})

predicted_body_mass = model_03.predict(new_penguin_data)
print(f'predicted body mass for the new penguin: {predicted_body_mass[0]:.2f} grams')
```

```
predicted body mass for the new penguin: 3728.78 grams
```

Key Takeaways

- Build a ladder of models and let cross-validation choose among them.
- Report the spread of cross-validated performance, not just the mean.
- Check residuals; structure means the model is incomplete.
- Save the fitted model so it can be reused.
- Wrap prediction in a function so the model is easy to use.

Exercises

1. Reproduce the model comparison table and identify the best model.
2. Add a new candidate model and see whether it beats the incumbent.
3. Plot residuals against each input and describe any pattern.
4. Save a model, reload it, and confirm identical predictions.
5. Extend the inference function to accept a dataframe of new penguins.

Guided Walkthrough

This is the full regression workflow and the template for everything after. It begins with visualization, then fits a ladder of models from an intercept-only baseline to a full candidate with interactions and polynomial terms. Training metrics are reported first, then cross-validation, which rotates through folds and reports the mean and the standard error of performance. The spread matters: a model that wins by less than a standard error has not really won.

The residual analysis is the quality gate. The notebook plots residuals against each input and expects no pattern. Structure in the residuals means the model is missing something, and the fix is a new feature or a new term, not more data. Once the model passes, it is saved with pickle and reloaded, and an inference function predicts on a new penguin. Saving and reloading is what makes a model usable by anyone, and the inference function is the seed of the deployment chapter.

Common Pitfalls

- Selecting a model on training metrics.
- Reporting a mean cross-validation score without its spread.
- Skipping the residual check.

- Saving a model without recording its inputs and preprocessing.

Going Further

- Add a regularized model and compare it with the ordinary least squares fits.
- Package the inference function with the saved scaler so predictions are reproducible.
- Read about the Akaike and Bayesian information criteria as alternatives to cross-validation.

Exercise Solutions

1. The comparison table shows cross-validated R-squared and RMSE for each model; the best model balances accuracy and simplicity.
2. A new candidate wins only if its mean score exceeds the incumbent's by more than the standard error.
3. Residual plots with no pattern support the model; a pattern indicates a missing term.
4. Reloading the saved model reproduces identical predictions, which confirms the save is faithful.
5. The extended function accepts a dataframe, applies the same preprocessing, and returns predictions for every row.

Part IV: The Mathematics of Deep Learning

Linear Algebra for Deep Learning

Why Linear Algebra Comes First

Every neural network, however deep or modern, is built from one repeated operation: multiply by a matrix, add a vector, pass the result through a simple non-linear function. GPT is that operation composed tens of billions of times. If you truly understand the linear part, the rest of deep learning stops being mysterious. This chapter develops linear algebra at a level suitable for graduate study: precise definitions, proofs of the central results, and the connections to optimization and generalization that we will use throughout the book.

The treatment is self-contained. We begin with the axioms of a vector space and end with the singular value decomposition and principal component analysis. Along the way we prove the Cauchy-Schwarz inequality (which gives the triangle inequality and the cosine formula), the triangular eigenvalue theorem, the spectral theorem for symmetric matrices, and the spectral-norm case of the Eckart-Young theorem for low-rank approximation. The numeric claims in this chapter are checked by the companion script `build/verify/linear-algebra.py` against SymPy and NumPy.

Vector Spaces

A **vector space** over the real numbers is a set V with two operations, addition and scalar multiplication, satisfying eight axioms: addition is commutative and associative, there is an additive identity 0 and additive inverses, scalar multiplication distributes over vector addition and over scalar addition, scalar multiplication is compatible with scalar multiplication, and multiplying by 1 fixes every vector. The canonical example is $\mathbb{R}^n$, the set of ordered n -tuples of real numbers.

A **subspace** is a nonempty subset $U \subseteq V$ closed under addition and scalar multiplication. Every subspace contains the zero vector. The span of any set of vectors is a subspace, and a **basis** is a linearly independent spanning set. Every basis of a given space has the same size, which we call the **dimension**, $\dim V$. In $\mathbb{R}^n$ the standard basis is $e_1, \dots, e_n$, where e_i has a one in position i and zeros elsewhere.

Inner Products, Norms, and the Cauchy-Schwarz Inequality

An **inner product** on a real vector space is a function $\langle \cdot, \cdot \rangle$ that is symmetric, linear in each argument, and **positive definite**: $\langle x, x \rangle \geq 0$ with equality only for $x = 0$. On $\mathbb{R}^n$ the standard inner product is the dot product

$$\langle x, y \rangle = x^\top y = \sum_{i=1}^n x_i y_i.$$

The **induced norm** is $\|x\| = \sqrt{\langle x, x \rangle}$. We write $\|x\|_2$ for the Euclidean norm, $\|x\|_1 = \sum_i |x_i|$, and $\|x\|_\infty = \max_i |x_i|$.

Theorem (Cauchy-Schwarz). For all x, y in an inner product space, $|\langle x, y \rangle| \leq \|x\| \|y\|$, with equality if and only if x and y are linearly dependent.

Proof. If $y = 0$ the claim is trivial. Otherwise, for every real t ,

$$0 \leq \langle x - ty, x - ty \rangle = \|x\|^2 - 2t\langle x, y \rangle + t^2\|y\|^2.$$

This is a quadratic in t that is nonnegative everywhere, so its discriminant is at most zero: $4\langle x, y \rangle^2 - 4\|x\|^2\|y\|^2 \leq 0$, which rearranges to the claim. Equality forces the quadratic to have a real double root, so $x = ty$. ■

Two consequences follow immediately. First, the **triangle inequality**:

$$\|x + y\|^2 = \|x\|^2 + 2\langle x, y \rangle + \|y\|^2 \leq \|x\|^2 + 2\|x\|\|y\| + \|y\|^2 = (\|x\| + \|y\|)^2.$$

Second, the **cosine formula**: for nonzero vectors, $\cos \theta = \langle x, y \rangle / (\|x\|\|y\|)$ defines an angle $\theta \in [0, \pi]$, because Cauchy-Schwarz guarantees the ratio lies in $[-1, 1]$. Two vectors are **orthogonal** when the inner product is zero. The dot product therefore measures alignment, and this is exactly the quantity a single neuron computes between its weights and its input.

Tip: The three norms are not interchangeable. The L_1 norm used as a penalty encourages sparse solutions; the L_∞ norm governs robustness to the single largest coordinate change; the Euclidean norm is the default. The choice of norm is a modeling decision.

Linear Maps and Matrices

A map $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is **linear** if $T(ax + by) = aT(x) + bT(y)$ for all vectors and scalars. Fixing standard bases, every linear map is multiplication by a unique $m \times n$ matrix A :

$$y = Ax, \quad y_i = \sum_{j=1}^n a_{ij}x_j.$$

The j -th column of A is the image $T(e_j)$. For a product $C = AB$, with A of shape $m \times n$ and B of shape $n \times p$,

$$c_{ij} = \sum_{k=1}^n a_{ik}b_{kj},$$

and the shapes must satisfy the inner-dimension rule. The key fact is that **matrix multiplication is composition**: if B represents S and A represents T , then AB represents $T \circ S$.

Theorem. Matrix multiplication is associative.

Proof. Let A, B, C have compatible shapes. The (i, l) entry of $(AB)C$ is $\sum_j (\sum_k a_{ik}b_{kj})c_{jl}$, and of $A(BC)$ is $\sum_k a_{ik}(\sum_j b_{kj}c_{jl})$. Both equal the triple sum $\sum_{j,k} a_{ik}b_{kj}c_{jl}$, so the two are equal. ■

Associativity is why the bracketing of a long chain of layers does not change the function, though it does change the cost and the numerical behavior. Multiplication is **not commutative** in general; our verifier exhibits two random 3×3 matrices with $AB \neq BA$. The **transpose** satisfies $(A^\top)^\top = A$ and $(AB)^\top = B^\top A^\top$, the latter because the order of composition reverses under transposition.

Range, Null Space, and Rank

The **range** (image) of A is $\{Ax : x \in \mathbb{R}^n\}$, a subspace of $\mathbb{R}^m$. The **null space** (kernel) is $\{x : Ax = 0\}$, a subspace of $\mathbb{R}^n$. The **rank** $\text{rank}(A)$ is the dimension of the range, equal to the number of linearly independent columns. The **rank-nullity theorem** states

$$\text{rank}(A) + \dim \ker(A) = n.$$

A matrix is **full rank** when $\text{rank}(A) = \min(m, n)$; otherwise it maps some nonzero direction to zero and loses information. This is what happens when two columns of a weight matrix are identical, and it is a common cause of degenerate layers. For example, a matrix whose columns are all identical has rank one, and repeating one column of a full-rank matrix can only lower the rank; the verifier checks both cases.

Determinants

The **determinant** $\det(A)$ of a square matrix is the signed volume scale factor of the linear map: it multiplies every volume by $|\det(A)|$ and flips orientation when negative. It is multiplicative, $\det(AB) = \det(A)\det(B)$, and A is invertible exactly when $\det(A) \neq 0$. For a triangular matrix the determinant is the product of its diagonal entries, which is the fact we use next.

Eigenvalues and Eigenvectors

A nonzero vector v is an **eigenvector** of a square matrix A with **eigenvalue** λ if

$$Av = \lambda v.$$

The eigenvalue is the factor by which A scales v ; it may be negative (a flip), zero (a collapse), or complex (a rotation together with scaling), so "scale" is the precise word rather than "stretch". Eigenvalues are the roots of the **characteristic polynomial** $p(\lambda) = \det(A - \lambda I)$.

Theorem (triangular case). If A is triangular then its eigenvalues are its diagonal entries.

Proof. For triangular A , the matrix $A - \lambda I$ is triangular with diagonal entries $a_{ii} - \lambda$. The determinant of a triangular matrix is the product of its diagonal entries, so $p(\lambda) = \prod_i (a_{ii} - \lambda)$, whose roots are the diagonal entries. ■

This is why the example $A = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}$ has eigenvalues 3 and 2, and why the verifier can check a general triangular matrix.

A matrix is **diagonalizable** if it has a basis of eigenvectors, in which case $A = P\Lambda P^{-1}$ with Λ diagonal. A matrix that is not diagonalizable is **defective**; the matrix $J = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ is the smallest example.

Diagonalizability is not guaranteed, which is why the dynamics of A^k must be described with care.

Dynamics and the spectral radius. The behavior of repeated application of A is governed by the **spectral radius** $\rho(A) = \max_i |\lambda_i|$. For a diagonalizable matrix, writing v in the eigenbasis, the coefficient on an eigenvector with $|\lambda| > 1$ grows without bound, so if that coefficient is nonzero then $A^k v$ grows; conversely $\rho(A) < 1$ implies $A^k v \rightarrow 0$ for every v . Two cautions matter for non-normal matrices, which are the generic case in deep learning. First, "nonzero component in the unstable eigenspace" must mean a nonzero coefficient in the eigenbasis, not merely a nonzero projection. For $A = \begin{bmatrix} 2 & 100 \\ 0 & 1 \end{bmatrix}$ and $v = (1, -1/100)$, one computes $Av = v$ exactly, so $A^k v$ is constant although $\rho(A) = 2 > 1$ and v has a nonzero projection onto the unstable eigenvector $(1, 0)$. Second, a defective matrix can grow polynomially even when $\rho(A) = 1$: for $J = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$

we have $J^k(0, 1)^\top = (k, 1)^\top$, which is unbounded. When A is symmetric (indeed whenever A is normal, $AA^\top = A^\top A$), these pathologies disappear and the eigenbasis picture is exact. This spectral-radius analysis is exactly what explains vanishing and exploding gradients in recurrent networks in a later chapter.

The spectral theorem. If A is real and symmetric ($A = A^\top$), then all eigenvalues are real, eigenvectors for distinct eigenvalues are orthogonal, and there is an orthonormal basis of eigenvectors, so $A = Q\Lambda Q^\top$ with Q orthogonal. Symmetric matrices therefore behave like diagonal operators in a rotated coordinate system, which is why covariance matrices and Hessians are so tractable.

Proof. Let A be real and symmetric. The function $f(x) = x^\top Ax$ is continuous on the compact unit sphere, so it attains a maximum at some unit vector v , with value $\lambda = v^\top Av$, which is real because v and A are real. Applying the method of Lagrange multipliers to f subject to the smooth constraint $\|x\|^2 = 1$ (whose gradient $2x$ is nonzero on the sphere) gives $2Av = 2\lambda v$, so $Av = \lambda v$. The orthogonal complement $v^\perp$ is invariant under A : if $\langle x, v \rangle = 0$, then $\langle Ax, v \rangle = \langle x, Av \rangle = \lambda \langle x, v \rangle = 0$, using symmetry. Restricted to $v^\perp$, the map A is again symmetric, so by induction on the dimension it has an orthonormal eigenbasis; the base case is dimension one, where the claim is immediate. Adjoining v gives an orthonormal eigenbasis of the whole space, hence $A = Q\Lambda Q^\top$. ■

Here Q is **orthogonal**, meaning square with $Q^\top Q = QQ^\top = I$; its columns form an orthonormal basis, and it preserves lengths and angles.

Positive Definiteness and Quadratic Forms

A symmetric matrix A is **positive definite** if $x^\top Ax > 0$ for every $x \neq 0$, and **positive semidefinite** if $x^\top Ax \geq 0$ for every x . Symmetry is part of the definition, and it is not redundant: the skew matrix $K = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$ satisfies $x^\top Kx = 0$ for all x , because the quadratic form depends only on the symmetric part $\frac{1}{2}(K + K^\top) = 0$, yet K is not positive semidefinite. The verifier confirms this numerically.

By the spectral theorem, a symmetric A is positive definite exactly when all eigenvalues are positive, and positive semidefinite exactly when all are nonnegative. The quadratic form $x^\top Ax$ is a bowl when A is positive definite, an inverted bowl when A is negative definite, and a saddle or ridge otherwise. A symmetric positive definite matrix has a **Cholesky factorization** $A = LL^\top$ with L lower triangular, which is the numerically preferred way to solve linear systems and to sample from multivariate Gaussians.

The Singular Value Decomposition

Not every matrix is square, so eigenvalues are not always available. The **singular value decomposition** replaces them with singular values and works for any matrix. For $A \in \mathbb{R}^{m \times n}$ there exist orthogonal $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ (square orthogonal, as defined above) and a **rectangular-diagonal** $\Sigma \in \mathbb{R}^{m \times n}$ (nonzero entries only on the main diagonal) such that

$$A = U\Sigma V^\top.$$

The diagonal entries $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m,n)} \geq 0$ are the **singular values**, the columns of V are the **right singular vectors**, and the columns of U are the **left singular vectors**. The nonzero singular values are the square roots of the nonzero eigenvalues of $A^\top A$ (equivalently of $AA^\top$), and the right singular vectors are the corresponding eigenvectors. Our verifier checks that all singular values equal $\sqrt{\lambda_i(A^\top A)}$ and that the first right singular vector equals the first principal component of the centered data.

The spectral norm. The largest singular value is the **spectral norm** (operator two-norm),

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \sigma_1,$$

the maximum factor by which A stretches a vector. The verifier estimates the maximum ratio by sampling many unit vectors and confirms that it approaches σ_1 without ever calling the SVD for the estimate, and separately confirms $\sigma_1 = \sqrt{\lambda_{\max}(A^\top A)}$. The first singular vector gives the input direction of maximal stretch.

Low-rank approximation (Eckart-Young). The **Frobenius norm** of a matrix is $\|A\|_F = (\sum_{i,j} a_{ij}^2)^{1/2}$. Truncating the SVD to its largest k singular values gives the best rank- k approximation of A in the spectral norm, and by the companion Frobenius case of the same theorem it is optimal in the Frobenius norm as well. The truncation and its error are

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^\top, \quad \|A - A_k\|_2 = \sigma_{k+1}, \quad \|A - A_k\|_F = \left(\sum_{i=k+1}^{\min(m,n)} \sigma_i^2 \right)^{1/2}.$$

Proof (spectral norm case). Let B have rank k , so its null space has dimension $n - k$. The span of the top $k + 1$ right singular vectors $v_1, \dots, v_{k+1}$ has dimension $k + 1$, so it meets $\ker B$ in a nonzero unit vector x . Then $Bx = 0$ and $Ax = \sum_{i=1}^{k+1} \sigma_i (v_i^\top x) u_i$, so $\|Ax\|_2^2 = \sum_{i=1}^{k+1} \sigma_i^2 (v_i^\top x)^2 \geq \sigma_{k+1}^2 \sum_{i=1}^{k+1} (v_i^\top x)^2 = \sigma_{k+1}^2$, because each $\sigma_i \geq \sigma_{k+1}$. Hence $\|A - B\|_2 \geq \|Ax - Bx\|_2 = \|Ax\|_2 \geq \sigma_{k+1}$. The truncated A_k has rank k and achieves $\|A - A_k\|_2 = \sigma_{k+1}$, so it is optimal in the spectral norm. ■

This is the mathematical basis of principal component analysis and of low-rank compression of weight matrices.

Principal Component Analysis as an SVD

Given centered data arranged as rows of a matrix $X \in \mathbb{R}^{N \times d}$, principal component analysis seeks the directions of greatest variance. The sample covariance is $C = \frac{1}{N-1} X^\top X$. If $X = U \Sigma V^\top$, then $C = V \frac{\Sigma^2}{N-1} V^\top$, so the eigenvectors of C are the right singular vectors of X and the variances are $\sigma_i^2 / (N - 1)$. Projecting the data onto the first k right singular vectors gives the best k -dimensional linear summary in the least-squares sense. This is precisely what the clustering and autoencoder notebooks use to draw a two-dimensional picture of high-dimensional data.

A Worked Example: A Linear Layer by Hand

Suppose a layer has weights

$$W = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

and receives the input $x = (5, 6)$. Then

$$Wx = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 6 \\ 3 \cdot 5 + 4 \cdot 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}, \quad Wx + b = \begin{bmatrix} 18 \\ 38 \end{bmatrix}.$$

This is exactly what `nn.Linear(2, 2)` computes for every row of a batch at once. A deep network is this computation composed with non-linearities, and the linear algebra above is the entire story of the linear part.

Connections to Deep Learning

- **A layer is an affine map.** $x \mapsto Wx + b$ is the general form of a fully connected layer, and the mass of parameters lives in W .
- **Rank controls capacity.** A rank-deficient weight matrix cannot represent all input directions, so it behaves like a narrower layer.
- **Spectral norm controls robustness.** Since $\|W\delta\|_2 \leq \|W\|_2 \|\delta\|_2$, the largest singular value bounds how much a perturbed input can change a layer's output. Robustness certification in the final part of this book refines this bound layer by layer.
- **The SVD controls initialization and conditioning.** Weight initialization schemes aim to keep the singular values of each layer near one so that signals neither vanish nor explode with depth.
- **Covariance and PCA summarize representations.** The eigenvalues of a representation's covariance describe how many effective dimensions it uses.

Key Takeaways

- A vector space with an inner product has lengths, angles, and the Cauchy-Schwarz and triangle inequalities.
- A linear map is a matrix, and matrix multiplication is composition; it is associative but not commutative.
- Rank measures how much information a linear map preserves; rank-nullity relates rank to the null space.
- Eigenvalues describe scaling and the dynamics of iterated maps; the spectral radius governs growth, with the cautions stated above.
- Symmetric matrices are orthogonally diagonalizable and have real eigenvalues; positive semidefiniteness of the Hessian is the second-order condition for convexity, and positive definiteness gives strict convexity.
- The SVD exists for every matrix, its largest singular value is the spectral norm, and truncation gives the best low-rank approximation.
- PCA is the SVD of centered data.

Exercises

1. Compute $x^\top x$ for $x = (1, 2, 2)$ and use it to find the Euclidean norm.
2. Prove the parallelogram identity $\|x + y\|^2 + \|x - y\|^2 = 2\|x\|^2 + 2\|y\|^2$.
3. Multiply $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ by $\begin{bmatrix} 5 \\ 6 \end{bmatrix}$ and verify with NumPy.
4. Determine whether $(1, 0)$, $(0, 1)$, and $(1, 1)$ are linearly independent.
5. Find the eigenvalues of $\begin{bmatrix} 4 & 0 \\ 0 & -2 \end{bmatrix}$ and describe the transformation as scaling with possible reflections.
6. For a random 3×3 matrix, estimate the spectral norm by sampling many unit vectors and taking the maximum of $\|Ax\|_2$, then compare it to the largest singular value computed with NumPy.
7. Explain why a matrix with repeated columns has low rank and what that means for a layer.
8. Show that the skew matrix $\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$ has $x^\top Ax = 0$ for all x , and explain why it is not positive semidefinite.

Calculus, Gradients, and Automatic Differentiation

Why Calculus Is the Engine of Learning

A neural network is a function, its error is a function of its parameters, and learning is the search for parameters that make that error small. The only tool that makes this search tractable is the derivative, which tells us, at the current parameters, the direction in which the error increases most steeply. Moving the other way is gradient descent, and the chain rule is what lets us compute the derivative of a network with a billion parameters without a billion separate derivations. This chapter develops the calculus from definitions and proves the results that backpropagation depends on.

We move from single-variable derivatives to gradients, Jacobians, the multivariable chain rule, the Hessian, and then to automatic differentiation, which is the algorithmic realization of the whole story. The key formulas are checked in `build/verify/calculus.py`, and the backpropagation gradients are checked against PyTorch's autograd engine.

Limits and the Derivative

Let f be a real function and x_0 a point in its domain. The **derivative** of f at x_0 is the limit

$$f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h) - f(x_0)}{h},$$

when this limit exists. If it exists, f is **differentiable** at x_0 , and the derivative is the slope of the tangent line. The linear map $h \mapsto f'(x_0)h$ is the best linear approximation of f near x_0 :

$$f(x_0 + h) = f(x_0) + f'(x_0)h + o(h),$$

where $o(h)/h \rightarrow 0$. This "best linear approximation" view is the one that generalizes to many variables and to vector-valued functions, and it is the definition we will actually use.

Theorem. If f is differentiable at x_0 , then f is continuous at x_0 .

Proof. Write $f(x_0 + h) - f(x_0) = \left(\frac{f(x_0 + h) - f(x_0)}{h} \right) h$. As $h \rightarrow 0$ the bracket tends to $f'(x_0)$, a finite number, and $h \rightarrow 0$, so the product tends to 0. Hence $f(x_0 + h) \rightarrow f(x_0)$. ■

The converse fails: the absolute value function is continuous at 0 but not differentiable there, because the left and right difference quotients disagree.

Differentiation Rules

The rules that make calculus usable follow from the limit definition.

Linearity. $(af + bg)' = af' + bg'$.

Product rule. $(fg)' = f'g + fg'$.

Proof.

$$\frac{f(x+h)g(x+h) - f(x)g(x)}{h} = \frac{f(x+h) - f(x)}{h}g(x+h) + f(x)\frac{g(x+h) - g(x)}{h}.$$

As $h \rightarrow 0$, $g(x+h) \rightarrow g(x)$ by continuity, and the two difference quotients tend to $f'(x)$ and $g'(x)$. ■

Chain rule. If g is differentiable at x and f is differentiable at $g(x)$, then

$$(f \circ g)'(x) = f'(g(x)) \cdot g'(x).$$

Proof. Write $y = g(x)$ and $k = g(x+h) - g(x)$, so $g(x+h) = y + k$ and $k \rightarrow 0$ as $h \rightarrow 0$. Since f is differentiable at y , $f(y+k) - f(y) = f'(y)k + \varepsilon(k)k$ with $\varepsilon(k) \rightarrow 0$ as $k \rightarrow 0$ (define $\varepsilon(0) = 0$).

Substituting,

$$\frac{f(g(x+h)) - f(g(x))}{h} = (f'(y) + \varepsilon(k)) \frac{k}{h} = (f'(y) + \varepsilon(k)) \frac{g(x+h) - g(x)}{h}.$$

As $h \rightarrow 0$, $k \rightarrow 0$ so $\varepsilon(k) \rightarrow 0$, and the second factor tends to $g'(x)$. The product tends to $f'(g(x))g'(x)$. ■

The chain rule is the single most important identity in this book. A neural network is a composition of layers, and the chain rule turns that composition into a product of local derivatives, which is exactly what backpropagation multiplies together.

Higher Derivatives and Taylor's Theorem

The second derivative f'' is the derivative of f' . Taylor's theorem with remainder says that if f is k times continuously differentiable on an interval containing x and $x+h$ and $k+1$ times differentiable on its interior, then

$$f(x+h) = \sum_{j=0}^k \frac{f^{(j)}(x)}{j!} h^j + R_k,$$

where the remainder is $R_k = \frac{f^{(k+1)}(\xi)}{(k+1)!} h^{k+1}$ for some ξ between x and $x+h$. The first-order case is the linear approximation used above; the second-order case underlies Newton's method and the analysis of optimization.

Functions of Several Variables

For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **partial derivative** $\partial f / \partial x_i$ differentiates with respect to x_i while holding the others fixed. The **gradient** collects them into a vector:

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)^\top.$$

For a unit direction u , the **directional derivative** is $\nabla f(x)^\top u$; by Cauchy-Schwarz its magnitude is at most $\|\nabla f(x)\|$, with equality at $u = \nabla f(x) / \|\nabla f(x)\|$ (and the minimum at the opposite unit vector). Therefore the gradient is the direction of steepest ascent when it is nonzero, and $-\nabla f$ is the direction of steepest descent. This is the geometric fact that gives gradient descent its name.

For a vector-valued function $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the first derivatives form the **Jacobian** matrix $J_F(x) \in \mathbb{R}^{m \times n}$ with entries $J_{ij} = \partial F_i / \partial x_j$. The multivariable chain rule states that for a composition $F = H \circ G$,

$$J_F(x) = J_H(G(x)) J_G(x),$$

a product of Jacobians, which reduces to $f'(g(x))g'(x)$ in the scalar case.

The Hessian and Convexity

For $f : \mathbb{R}^n \rightarrow \mathbb{R}$ the **Hessian** is the matrix of second partial derivatives, $H_{ij} = \partial^2 f / \partial x_i \partial x_j$. When f is twice continuously differentiable, H is symmetric. The second-order Taylor expansion is

$$f(x + h) \approx f(x) + \nabla f(x)^\top h + \frac{1}{2} h^\top H(x) h.$$

By the previous chapter, f is convex if and only if $H(x)$ is positive semidefinite for all x ; if H is positive definite everywhere, f is strictly convex and has at most one minimum. A convex function has no bad local minima, which is why convex problems are "easy". Neural network losses are non-convex, but the local quadratic model $h^\top H h$ still describes the curvature and controls the largest stable learning rate.

Gradient Descent

To minimize f , **gradient descent** iterates

$$x_{t+1} = x_t - \eta \nabla f(x_t),$$

where $\eta > 0$ is the **learning rate**. For a positive definite quadratic $f(x) = \frac{1}{2} x^\top A x - b^\top x$ with minimizer $x^* = A^{-1}b$, the iteration satisfies

$$x_{t+1} - x^* = (I - \eta A)(x_t - x^*).$$

Writing the error in the eigenbasis of A with eigenvalues $0 < \lambda_1 \leq \dots \leq \lambda_n$, each component is multiplied by $1 - \eta \lambda_i$ per step. Convergence requires $|1 - \eta \lambda_i| < 1$ for every i , i.e., $0 < \eta < 2/\lambda_n$. The optimal constant step size is $\eta = 2/(\lambda_1 + \lambda_n)$; at that step the worst-case contraction factor per iteration is $(\kappa - 1)/(\kappa + 1)$ where $\kappa = \lambda_n/\lambda_1$ is the **condition number**. Because $(\kappa - 1)/(\kappa + 1) \approx 1 - 2/\kappa$ for large κ , ill-conditioned problems converge slowly. This is the precise reason we standardize features.

Backpropagation Derived for a Two-Layer Network

Now we derive the gradient of a real loss through a two-layer network, which is the smallest example that exhibits every feature of the general case. Let the input be x , with hidden pre-activation $z_1 = W_1 x + b_1$, hidden activation $a_1 = \sigma(z_1)$, output pre-activation $z_2 = W_2 a_1 + b_2$, and loss L . Define the **error signals** (the derivative of the loss with respect to the pre-activations)

$$\delta_2 = \frac{\partial L}{\partial z_2}, \quad \delta_1 = \frac{\partial L}{\partial z_1}.$$

By the chain rule, starting from the output,

$$\begin{aligned} \delta_2 &= \frac{\partial L}{\partial z_2}, \\ \frac{\partial L}{\partial W_2} &= \delta_2 a_1^\top, \quad \frac{\partial L}{\partial b_2} = \delta_2, \\ \delta_1 &= (W_2^\top \delta_2) \odot \sigma'(z_1), \end{aligned}$$

$$\frac{\partial L}{\partial W_1} = \delta_1 x^\top, \quad \frac{\partial L}{\partial b_1} = \delta_1,$$

where $\odot$ is the elementwise product. Read the equations as a backward pass: the output error δ_2 is propagated back through $W_2^\top$, multiplied elementwise by the local activation derivative $\sigma'(z_1)$, to give δ_1 . Each weight gradient is the outer product of that layer's error signal with its layer's input. The general rule is the same: **error signal times input**. The verifier computes these gradients by hand and confirms they match PyTorch's autograd within numerical tolerance. The derivation assumes the loss depends on the parameters only through the output pre-activation, $L = L(z_2)$; a penalty on the weights directly would add its own term.

As a concrete consequence, for the softmax output with cross-entropy loss, the output error signal collapses to the famously simple

$$\delta_2 = p - y,$$

where p is the predicted class distribution and y is the one-hot target. The verifier confirms this cancellation symbolically. It is the reason cross-entropy is the standard classification loss: it makes the output gradient exactly the prediction error.

Automatic Differentiation

Automatic differentiation computes exact derivatives by applying the chain rule to the sequence of elementary operations in a program, rather than by finite differences or symbolic manipulation. It is exact up to floating-point rounding, because it evaluates the derivatives of the primitive operations exactly and combines them by the chain rule. There are two modes.

Forward mode propagates derivatives with respect to one input forward through the computation. If the function has one input and m outputs, one forward pass gives the full Jacobian; if it has n inputs, forward mode costs n passes.

Reverse mode makes one forward pass to record the computation graph, then one backward pass to propagate the derivative of a scalar output with respect to every input, in cost proportional to a small multiple of the forward pass. For a scalar loss with n parameters, reverse mode gets all n partial derivatives in one backward pass. This is precisely the regime of neural network training, which is why reverse mode (backpropagation) is the universal choice. The verifier checks the consistency of both modes and the exactness of the reverse-mode gradient.

A subtlety worth stating: automatic differentiation is neither symbolic differentiation (which manipulates expressions and can explode in size) nor numerical differentiation (which is approximate and suffers cancellation). It accumulates exact derivatives of the primitive operations and combines them by the chain rule, so its results are exact up to floating-point rounding.

A Worked Example

Let $f(x) = (3x + 1)^2$. Then by the chain rule $f'(x) = 2(3x + 1) \cdot 3 = 6(3x + 1)$, and at $x = 2$ we get $f'(2) = 6 \cdot 7 = 42$. In reverse mode: the outer square has local derivative $2(3x + 1)$, the inner affine has local derivative 3, and the product is 42. The verifier checks the analytic value, a central finite difference, and SymPy's symbolic derivative all agree.

Connections to Deep Learning

- **The chain rule is backpropagation.** Every deep learning framework is an engine for applying it efficiently.
- **The gradient points uphill.** Training steps move against $-\nabla L$.
- **The Hessian controls the learning rate and conditioning.** Its largest eigenvalue bounds the stable step size; its condition number explains slow training on unnormalized features.
- **Cross-entropy plus softmax gives the gradient $p - y$.** This elegant cancellation is why the pair is ubiquitous.
- **Reverse-mode AD is what makes deep learning feasible.** It computes billions of gradients at the cost of a few forward passes.

Key Takeaways

- The derivative is the best linear approximation; differentiability implies continuity.
- The chain rule turns composition into a product of local derivatives.
- The gradient is the direction of steepest ascent; the Hessian governs curvature and convexity.
- For a quadratic, gradient descent converges for step sizes below $2/\lambda_{\max}$ (locally in general); the condition number sets the speed.
- Backpropagation is reverse-mode automatic differentiation; each weight gradient is an error signal times an input.
- For softmax with cross-entropy the output error is exactly $p - y$.

Exercises

1. Differentiate $f(x) = 3x^3 - 2x + 1$ and evaluate at $x = 2$.
2. Use the product rule to differentiate $f(x) = x^2 e^x$.
3. Use the chain rule to differentiate $f(x) = \sin(3x^2)$.
4. Compute the gradient of $f(x, y) = x^2 y + y^3$ and evaluate at $(1, 2)$.
5. Show that $\frac{d}{dx} \sigma(x) = \sigma(x)(1 - \sigma(x))$ for the sigmoid.
6. Derive the update for gradient descent on $f(w) = (w - 4)^2$ and find the largest stable step size.
7. Explain why reverse mode costs about one forward pass per backward pass regardless of the number of parameters.
8. Verify by hand that for a single softmax output with cross-entropy the gradient with respect to the logits is $p - y$.

Probability, Information Theory, and Loss Functions

Why Probability and Information Theory

A neural network does not output answers; it outputs probabilities, and it is trained to make the probabilities it assigns to the observed data as high as possible. Loss functions are not arbitrary penalties invented for convenience; most regression and classification losses in deep learning are the negative log-likelihood of a probabilistic model. This chapter builds the probability and information theory that explain *why* cross-entropy, mean squared error, and softmax are the standard choices, and it proves the properties (nonnegativity of KL divergence, the maximum-entropy characterization of the uniform distribution, the bias-variance decomposition) that the later chapters rely on.

Probability Spaces and Random Variables

A **probability space** is a triple $(\Omega, \mathcal{F}, P)$: a set of outcomes Ω , a collection of events $\mathcal{F}$, and a function $P : \mathcal{F} \rightarrow [0, 1]$ that is countably additive and satisfies $P(\Omega) = 1$. A **random variable** X is a measurable function from Ω to a value set, usually $\mathbb{R}$ or a finite set of classes.

A discrete random variable has a **probability mass function** $p(x) = P(X = x)$ with $\sum_x p(x) = 1$. A continuous random variable has a **probability density** $p(x) \geq 0$ with $\int p(x) dx = 1$. The **cumulative distribution function** is $F(x) = P(X \leq x)$.

Expectation, Variance, and Covariance

The **expectation** of a function g of X is

$$\mathbb{E}[g(X)] = \sum_x g(x)p(x) \quad \text{or} \quad \int g(x)p(x) dx.$$

Expectation is linear: $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$, always. The **variance** is

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2,$$

and the **covariance** is $\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]$. Two variables are **uncorrelated** when the covariance is zero. Independent variables are uncorrelated, but the converse fails. For example, let X be uniform on $\{-1, 0, 1\}$ and $Y = X^2$; then $\text{Cov}(X, Y) = \mathbb{E}[X^3] - \mathbb{E}[X]\mathbb{E}[X^2] = 0$, yet X and Y are dependent, because Y is determined by X .

Common Distributions

Distribution	Support	Parameters	Density or mass
Bernoulli	$\{0, 1\}$	p	$p^x(1-p)^{1-x}$
Categorical	one-hot vectors in $\mathbb{R}^K$	$\boldsymbol{\pi}$	$\prod_k \pi_k^{x_k}$
Uniform	$[a, b]$	a, b	$1/(b-a)$
Gaussian	$\mathbb{R}$	μ, σ^2	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/(2\sigma^2)}$

Distribution	Support	Parameters	Density or mass
Multivariate Gaussian	$\mathbb{R}^d$	μ, Σ	$(2\pi)^{-d/2} \Sigma ^{-1/2} e^{-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)}$

The parameters satisfy $p \in [0, 1]$, $\pi_k \geq 0$ with $\sum_k \pi_k = 1$, $a < b$, $\sigma^2 > 0$, and Σ symmetric positive definite. The **central limit theorem**, which the first notebook demonstrates experimentally, states that the standardized sum of many independent, identically distributed finite-variance variables converges in distribution to a standard Gaussian. This is why Gaussian noise appears everywhere.

Conditional Probability and Bayes' Rule

The **conditional probability** of A given B with $P(B) > 0$ is $P(A | B) = P(A \cap B) / P(B)$. **Bayes' rule** rearranges the symmetry $P(A \cap B) = P(A | B)P(B) = P(B | A)P(A)$:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}.$$

The terms have standard names: $P(A)$ is the **prior**, $P(B | A)$ the **likelihood**, $P(A | B)$ the **posterior**, and $P(B)$ the **evidence** or marginal likelihood. Bayes' rule is the foundation of probabilistic inference and, in the form of the posterior predictive, of Bayesian neural networks.

Maximum Likelihood Estimation

Given independent samples $\mathcal{D} = (x_1, \dots, x_N)$ from a parametric model $p(x; \theta)$, the **likelihood** is $L(\theta) = \prod_i p(x_i; \theta)$ and the **log-likelihood** is

$$\ell(\theta) = \sum_{i=1}^N \log p(x_i; \theta).$$

The logarithm is monotone, so maximizing ℓ is the same as maximizing L , and the sum is far easier to differentiate than the product. The **maximum likelihood estimate** (MLE) is $\hat{\theta} = \arg \max_{\theta} \ell(\theta)$, and training a model by minimizing a loss is usually minimizing $-\ell$ plus a regularizer.

Example (Bernoulli MLE). For $x_i \in \{0, 1\}$ with $p(x; \theta) = \theta^x (1 - \theta)^{1-x}$,

$$\ell(\theta) = \left(\sum_i x_i \right) \log \theta + \left(N - \sum_i x_i \right) \log(1 - \theta).$$

For $0 < \theta < 1$, differentiating and setting to zero gives $\hat{\theta} = \frac{1}{N} \sum_i x_i$, the sample mean. (If every observation is 0, or every observation is 1, the maximum lies on the boundary at 0 or 1, and the stationary-point argument does not apply; the boundary values are the correctly defined limits.) The verifier confirms this symbolically.

Example (Gaussian MLE and mean squared error). For a scalar target with Gaussian noise, $y_i | x_i \sim \mathcal{N}(f_{\theta}(x_i), \sigma^2)$, the log-likelihood is

$$\ell(\theta) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_i (y_i - f_{\theta}(x_i))^2.$$

Maximizing ℓ over θ is exactly minimizing the sum of squared errors; the negative log-likelihood is an affine function of the mean squared error (a positive scale times it plus a constant). This is the probabilistic justification of the mean squared error loss under Gaussian noise.

Information Theory

The **entropy** of a discrete distribution p measures its uncertainty:

$$H(p) = - \sum_x p(x) \log p(x).$$

Entropy is nonnegative, and it is maximized by the uniform distribution on a finite set, where it equals $\log K$ for K outcomes. This is the first hint of a general principle: under a set of constraints, the distribution with the largest entropy is the least committal.

The **cross-entropy** between a true distribution p and a model q is

$$H(p, q) = - \sum_x p(x) \log q(x),$$

and the **Kullback-Leibler (KL) divergence** is

$$D_{\text{KL}}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = H(p, q) - H(p).$$

Theorem (Gibbs' inequality). $D_{\text{KL}}(p \parallel q) \geq 0$, with equality if and only if $p = q$.

Proof. Since $-\log$ is convex, Jensen's inequality gives

$$D_{\text{KL}}(p \parallel q) = \sum_x p(x) \left(-\log \frac{q(x)}{p(x)} \right) \geq -\log \sum_x p(x) \frac{q(x)}{p(x)} = -\log \sum_x q(x) = 0,$$

where the sums are over the support of p (for x with $p(x) = 0$ the term is defined as zero). Equality holds in Jensen's inequality only when $q(x)/p(x)$ is constant on the support of p ; since q places no mass off that support and both distributions are normalized, that constant is 1, so $q = p$. ■

Because cross-entropy is KL divergence plus the entropy of the truth, and the entropy of the truth does not depend on the model, **minimizing cross-entropy is equivalent to minimizing KL divergence**, that is, making the model distribution as close to the true distribution as possible.

Theorem (maximum entropy). Among all distributions on a finite set of K outcomes, the entropy is maximized by the uniform distribution, with value $\log K$.

Proof. Let p be uniform and let q be arbitrary. By Gibbs' inequality, $D_{\text{KL}}(q \parallel p) \geq 0$. Expanding with $p(x) = 1/K$,

$$D_{\text{KL}}(q \parallel p) = \sum_x q(x) \log \frac{q(x)}{1/K} = \sum_x q(x) \log q(x) + \log K = \log K - H(q),$$

so $\log K - H(q) \geq 0$, that is, $H(q) \leq \log K = H(p)$. ■

Softmax and Cross-Entropy

A classifier produces scores (logits) $z = (z_1, \dots, z_K)$ and turns them into a distribution with the **softmax**

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

Softmax is the exponential-family distribution obtained by matching the expected one-hot sufficient statistic: maximizing entropy subject to $\mathbb{E}_p[t_k(X)] = \mu_k$ with one-hot t gives $p(x) \propto e^{\eta \cdot t(x)}$, a softmax in the natural parameters η . This is one reason it is the natural output layer. Pairing softmax with the cross-entropy loss

$$L = - \sum_k y_k \log \hat{p}_k, \quad \hat{p} = \text{softmax}(z),$$

where y is the one-hot target, gives the clean gradient $\partial L / \partial z_k = \hat{p}_k - y_k$ that we derived in the calculus chapter. This is not a coincidence: softmax is the **inverse canonical link** (the canonical mean function ∇A) of the categorical exponential family. The natural parameter equals the logit vector, the canonical link maps class probabilities to the logits through the multinomial logit $\log(p_k/p_K)$, and softmax inverts that map. The exponential-family moment identity then gives the gradient directly, and the derivative of the softmax cancels the logarithm in the cross-entropy. Note that the log-likelihood is not linear in the logits, because of the log-partition term $\log \sum_j e^{z_j}$.

The Bias-Variance Decomposition

For squared-error regression with a target $y = f(x) + \varepsilon$, where the noise has mean zero and variance σ^2 , and a learned predictor $\hat{f}$ trained on a random dataset, the expected test error at a point x decomposes as

$$\mathbb{E} \left[(y - \hat{f}(x))^2 \right] = \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x) \right)^2}_{\text{bias}^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}.$$

Proof. Add and subtract $\mathbb{E}[\hat{f}(x)]$ inside the square, expand, and use that the cross term vanishes because $\mathbb{E}[y - f(x)] = 0$ and ε is independent of the fitted model and has mean zero. The three surviving terms are exactly bias squared, variance, and noise. ■

The decomposition formalizes the trade-off discussed in the modeling chapters: a too-simple model has high bias, a too-flexible model has high variance, and the total error is minimized in between.

Connections to Deep Learning

- **Losses are negative log-likelihoods.** MSE is the Gaussian negative log-likelihood; cross-entropy is the categorical one.
- **Cross-entropy is KL plus a constant.** Minimizing it drives the model distribution toward the truth.
- **Softmax is the maximum-entropy output subject to matching the class probabilities** (the exponential-family moment-matching result) and yields the gradient $\hat{p} - y$.
- **Entropy and KL underpin regularization and evaluation.** They appear in information bottleneck methods, in variational inference (the ELBO is a negative KL plus a reconstruction term), and in generative modeling.

- **The bias-variance decomposition explains overfitting and underfitting** and motivates regularization and ensembling.

Key Takeaways

- Probability gives the language of uncertainty; expectation is linear, variance is not.
- Bayes' rule updates a prior with a likelihood to give a posterior.
- Maximum likelihood turns training into optimization; MSE and cross-entropy are MLE objectives in disguise.
- Entropy measures uncertainty; Gibbs' inequality gives $KL \geq 0$; minimizing cross-entropy minimizes KL.
- Softmax is the maximum-entropy classifier output and yields the gradient $\hat{p} - y$.
- Test error decomposes into bias squared, variance, and irreducible noise.

Exercises

1. A fair die is rolled; compute the expectation and variance of the result.
2. State Bayes' rule and identify the prior, likelihood, posterior, and evidence for a medical test.
3. Derive the Bernoulli MLE and show it equals the sample mean.
4. Show that maximizing the Gaussian log-likelihood is minimizing mean squared error.
5. Compute the entropy of a Bernoulli distribution and find the value of p that maximizes it.
6. Prove that cross-entropy equals entropy plus KL divergence.
7. Explain why minimizing cross-entropy and minimizing KL divergence are equivalent for a fixed dataset.
8. Derive the bias-variance decomposition for squared error.

The Mathematics of the Multilayer Perceptron

From a Single Neuron to a Network

A **neuron** computes an affine function of its inputs followed by a fixed non-linear **activation**:

$$a = \phi(w^\top x + b),$$

where w is the weight vector, b the bias, and ϕ the activation. The affine part selects a direction and a threshold; the activation introduces the non-linearity that makes composition powerful. A **layer** applies this to many neurons at once, which is a matrix multiplication $Wx + b$ followed by an elementwise activation ϕ . A **multilayer perceptron** (MLP) is a composition of such layers:

$$h^{(0)} = x, \quad z^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}, \quad h^{(\ell)} = \phi(z^{(\ell)}),$$

for $\ell = 1, \dots, L$, with the final layer often linear (for regression) or followed by a softmax (for classification). This chapter develops the mathematics of this object: the activations and their derivatives, the forward and backward passes in full generality, the optimizers, the initialization schemes, the normalization layers, and the regularization techniques. Selected formulas are checked in `build/verify/mlp.py`.

Why Nonlinearity Is Essential

If every ϕ were the identity, the composition of layers would collapse:

$$W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)} = (W^{(2)}W^{(1)})x + (W^{(2)}b^{(1)} + b^{(2)}),$$

which is again affine. No matter how many linear layers you stack, the result is a single linear map. A non-linear activation is what allows depth to increase expressive power. The verifier confirms this collapse explicitly for a two-layer stack.

Activation Functions

Activation	Formula	Derivative	Range
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	$\sigma(z)(1 - \sigma(z))$	$(0, 1)$
Tanh	$\tanh(z)$	$1 - \tanh^2(z)$	$(-1, 1)$
ReLU	$\max(0, z)$	1 if $z > 0$, else 0	$[0, \infty)$
Leaky ReLU	$\max(\alpha z, z)$	1 if $z > 0$, else α	$(-\infty, \infty)$
Softplus	$\log(1 + e^z)$	$\sigma(z)$	$(0, \infty)$
SiLU (Swish)	$z \sigma(z)$	$\sigma(z) + z \sigma(z)(1 - \sigma(z))$	$[-0.2785, \infty)$
GELU	$z \Phi(z)$	$\Phi(z) + z \varphi(z)$	$[-0.1700, \infty)$

Here Φ and φ are the standard Gaussian CDF and PDF. The **sigmoid** and **tanh** saturate: for large $|z|$ their derivatives tend to zero, which is the origin of vanishing gradients. **ReLU** does not saturate for positive inputs and has an exactly zero derivative for negative inputs, which makes it fast and the default for hidden layers, at

the cost of "dead" units that never activate again. Modern smooth activations such as SiLU and GELU combine the non-saturating positive region with a small negative region and are common in transformers. The verifier confirms the sigmoid, tanh, and softplus derivatives symbolically and the GELU derivative numerically.

The Forward Pass as a Composition

It is useful to view the network as a single function $f = f_L \circ \dots \circ f_1$, where each $f_\ell(u) = \phi(W^{(\ell)}u + b^{(\ell)})$. The gradient of the loss with respect to the parameters is then obtained by the chain rule through this composition, which is exactly backpropagation. The **Jacobian** of a layer with respect to its input is

$$\frac{\partial f_\ell}{\partial u} = \text{diag}(\phi'(z^{(\ell)})) W^{(\ell)},$$

a diagonal matrix of local activation derivatives times the weight matrix. This is the object that the backward pass multiplies.

Backpropagation in General

Write the loss L , and define the error signal of layer ℓ as the gradient with respect to its pre-activation, $\delta^{(\ell)} = \partial L / \partial z^{(\ell)}$. The backward pass is:

$$\delta^{(\ell)} = \frac{\partial L}{\partial z^{(\ell)}}, \quad \delta^{(\ell)} = \left(W^{(\ell+1)\top} \delta^{(\ell+1)} \right) \odot \phi'(z^{(\ell)}),$$

and the parameter gradients are the outer products

$$\frac{\partial L}{\partial W^{(\ell)}} = \delta^{(\ell)} h^{(\ell-1)\top}, \quad \frac{\partial L}{\partial b^{(\ell)}} = \delta^{(\ell)}.$$

The recursion propagates the error signal backward through the transpose weight matrices, scale by the activation derivative, and form the gradients as error-signal-times-input. This is the same pattern as the two-layer derivation in the calculus chapter, now written for arbitrary depth. The verifier checks the general recursion against PyTorch's autograd for a three-layer network.

The Vanishing and Exploding Gradient Problem

Unrolling the recursion, the error signal at an early layer is a product of Jacobians:

$$\delta^{(\ell)} = \left(\prod_{k=\ell}^{L-1} \text{diag}(\phi'(z^{(k)})) W^{(k+1)\top} \right) \delta^{(L)}.$$

If every Jacobian satisfies $\|J_k\|_2 \leq q < 1$, then by submultiplicativity $\|\prod_k J_k\|_2 \leq \prod_k \|J_k\|_2$ decays geometrically with depth, and the early layers receive almost no gradient (**vanishing gradients**); it is enough that the expected log-norm $\mathbb{E}[\log \|J_k\|_2]$ be negative, which is strictly weaker than a uniform bound. For growing gradients the correct hypothesis is a uniform lower bound on the smallest singular value: if every Jacobian is nonsingular with $\sigma_{\min}(J_k) \geq q > 1$ (equivalently $\|J_k^{-1}\|_2 \leq 1/q$), the product grows geometrically (**exploding gradients**). A large spectral norm alone is not enough, because a matrix can have a large norm and a tiny direction: $J_1 = \text{diag}(3, 10^{-6})$ and $J_2 = \text{diag}(10^{-6}, 3)$ both have spectral norm 3, yet $\|J_1 J_2\|_2 = 3 \times 10^{-6}$. Likewise the vanishing hypothesis is a uniform bound, not an average: alternating

Jacobians of norm 0.1 and 3 have arithmetic mean above one, yet their product vanishes. These are statements about the singular values of the layer Jacobians, not about the spectral radius of a single matrix. They are why normalization, careful initialization, residual connections, and non-saturating activations matter.

Loss Functions

For regression, the mean squared error $L = \frac{1}{N} \sum_i \|\hat{y}_i - y_i\|^2$ is the Gaussian negative log-likelihood up to a positive scale and an additive constant. For classification with a softmax output, the cross-entropy $L = -\sum_k y_k \log \hat{p}_k$ is the categorical negative log-likelihood and has the output error $\delta^{(L)} = \hat{p} - y$. For regression with a linear output and MSE loss, $\delta^{(L)} = \hat{y} - y$ up to a positive scale. These two cases cover the vast majority of supervised learning.

Optimizers

Let $g_t = \nabla_{\theta} L_t$ be the gradient of the mini-batch loss at step t .

Stochastic gradient descent (SGD). $\theta_{t+1} = \theta_t - \eta g_t$.

Momentum. Accumulate a velocity: $v_t = \beta v_{t-1} + g_t$, $\theta_{t+1} = \theta_t - \eta v_t$. This is an exponential moving average of the gradients, which smooths the update and accelerates along consistent directions. With $v_0 = 0$, unrolling gives $v_t = \sum_{k=0}^{t-1} \beta^k g_{t-k}$, a weighted sum with geometric weights.

Nesterov momentum evaluates the gradient at the look-ahead point $\theta_t - \eta \beta v_{t-1}$.

AdaGrad scales each coordinate by the inverse square root of the accumulated squared gradient: $\theta_{t+1} = \theta_t - \eta g_t / \sqrt{\sum_{s \leq t} g_s^2 + \epsilon}$.

RMSProp replaces the sum with an exponential moving average $s_t = \beta s_{t-1} + (1 - \beta) g_t^2$ and updates $\theta_{t+1} = \theta_t - \eta g_t / \sqrt{s_t + \epsilon}$.

Adam combines momentum and RMSProp with running estimates

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2,$$

and applies **bias correction**, $\hat{m}_t = m_t / (1 - \beta_1^t)$ and $\hat{s}_t = s_t / (1 - \beta_2^t)$, before the update

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{s}_t + \epsilon}}.$$

Why bias correction? At initialization $m_0 = 0$, so $m_t = (1 - \beta_1) \sum_{k=0}^{t-1} \beta_1^k g_{t-k}$ has expectation $(1 - \beta_1^t) \mathbb{E}[g]$ for a constant gradient. Dividing by $1 - \beta_1^t$ removes the initial underestimate. The verifier confirms this unbiasedness exactly for a constant gradient.

Weight Initialization

Consider a layer with n_{in} inputs, weights W_{ij} drawn independently of each other and of the inputs, with mean zero and variance σ_w^2 , and inputs h_j that are uncorrelated with mean zero. The pre-activation $z_i = \sum_j W_{ij} h_j$ then satisfies

$$\text{Var}(z_i) = \sum_j \mathbb{E}[W_{ij}^2] \mathbb{E}[h_j^2] = n_{\text{in}} \sigma_w^2 \mathbb{E}[h^2],$$

using independence and the zero mean of the weights. To keep the pre-activation variance constant across layers, we need $n_{\text{in}}\sigma_w^2 \mathbb{E}[h^2] = \text{Var}(h)$; for inputs with $\mathbb{E}[h^2] = \text{Var}(h)$ this gives $\sigma_w^2 = 1/n_{\text{in}}$. This is the **LeCun** initialization. The **Xavier/Glorot** initialization compromises between the forward condition $\sigma_w^2 = 1/n_{\text{in}}$ and the backward condition $\sigma_w^2 = 1/n_{\text{out}}$ by averaging the fan sizes, $\sigma_w^2 = 2/(n_{\text{in}} + n_{\text{out}})$, and the **He** initialization uses $\sigma_w^2 = 2/n_{\text{in}}$, which compensates for the ReLU halving the second moment $\mathbb{E}[h^2]$ of a symmetric zero-mean pre-activation (a ReLU zeroes about half its inputs, so $\mathbb{E}[\text{ReLU}(z)^2] = \mathbb{E}[z^2]/2$). Note that the ReLU does not halve the variance: for a zero-mean Gaussian pre-activation, $\text{Var}(\text{ReLU}(z)) \approx 0.341 \text{Var}(z)$. The initialization analysis uses the second moment, which is why the factor 2 is correct. The verifier confirms the variance-preservation numerically.

Normalization Layers

Batch normalization standardizes each feature across the mini-batch and then rescales:

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{z}_i + \beta,$$

where μ_B and σ_B^2 are the batch mean and variance and γ, β are learned. At inference, running averages of the mean and variance are used. **Layer normalization** standardizes across the features of a single example instead of across the batch, which makes it independent of batch size and is the standard choice in transformers. The verifier confirms that the standardized output has zero mean and unit variance on its normalization axis.

Regularization

L_2 regularization adds $\lambda \|W\|_F^2$ to the loss, equivalent to a Gaussian prior on the weights under a maximum a posteriori view, and shrinks weights toward zero.

Dropout zeroes each activation independently with probability p during training. To keep the expected activation unchanged at inference, the surviving activations are scaled by $1/(1 - p)$ during training (the **inverted dropout** convention). For any input,

$$\mathbb{E}[\text{dropout}(h)] = (1 - p) \cdot \frac{h}{1 - p} = h.$$

The verifier confirms this expectation and the exact mean of the dropout mask.

Universal Approximation

The **universal approximation theorem** states that a feed-forward network with a single hidden layer of sufficient width, a suitably regular non-polynomial activation (locally bounded and piecewise continuous), and a linear output layer can approximate any continuous function on a compact set to arbitrary accuracy in the uniform norm. The theorem guarantees existence, not that gradient descent finds the approximating weights, and it says nothing about how wide the layer must be. Later results (the "depth versus width" literature) show that some functions need exponentially many neurons in a shallow network but only polynomially many with depth. The practical lesson is that depth is efficient, and the theoretical lesson is that a shallow network is not fundamentally limited in what it can represent, only in how economically. The verifier does not attempt to prove this; we state it as the standard existence result.

Key Takeaways

- A neuron is affine plus activation; a layer is a matrix plus bias plus activation.
- Without non-linear activations, any stack of layers collapses to one affine map.
- Backpropagation is the chain rule through the composition: error signals multiplied by transposed weights and activation derivatives.
- Vanishing and exploding gradients are governed by the singular values of the layer Jacobians: a uniform upper bound below one contracts the product, a uniform lower bound above one on the smallest singular value expands it.
- Optimizers differ in how they accumulate and scale gradients; Adam corrects the initialization bias.
- Initialization variance must be controlled; Xavier and He set it to preserve signal.
- Batch and layer normalization re-center and re-scale activations.
- L_2 regularization and dropout are the standard defenses against overfitting.

Exercises

1. Show that a composition of two linear layers is affine.
2. Verify the derivatives of sigmoid, tanh, and ReLU.
3. Derive the bias-correction factor for Adam's first moment.
4. Derive the LeCun initialization from the variance recursion.
5. Explain why He initialization uses $2/n_{\text{in}}$ rather than $1/n_{\text{in}}$.
6. Prove that inverted dropout preserves the expected activation.
7. State the universal approximation theorem and explain what it does and does not guarantee.
8. Explain how a product of layer Jacobians with spectral norm below one causes vanishing gradients.

The Mathematics of Convolutional Networks

Why Convolutions

A dense layer treats every input coordinate as unrelated to every other and gives each pairing its own weight, so a network acting on a $256 \times 256 \times 3$ image would need billions of parameters in its first layer alone. Images have structure that this view throws away: nearby pixels are correlated, and a feature such as an edge is meaningful wherever it appears. A **convolution** encodes both facts by sliding a small set of weights, called a **kernel** or filter, across the input and reusing it at every position. This is **parameter sharing** and it is the reason a convolutional network can be both powerful and compact. This chapter develops the operation precisely, proves its translation-equivariance property, derives the output-size and receptive-field formulas, and derives the backward pass. Selected claims are checked in `build/verify/cnn.py`.

The Convolution Operation

For a single-channel input x and a kernel w of size $k \times k$, the **cross-correlation** (the operation deep learning frameworks call convolution) is

$$y[i, j] = \sum_{a=0}^{k-1} \sum_{b=0}^{k-1} w[a, b] x[i + a, j + b].$$

In signal processing, **convolution** is the same sum with the kernel flipped, $y[i, j] = \sum_{a,b} w[a, b] x[i - a, j - b]$. The two differ only by a reflection of the kernel, which is immaterial for a learnable filter, but it matters when you compare against a signal-processing reference. We will use the cross-correlation convention, as PyTorch and every deep learning framework do, and state explicitly when the distinction matters. The verifier confirms that cross-correlation equals convolution with the flipped kernel.

For an input with C_{in} channels and C_{out} output channels, each output channel o uses its own kernel stack $w[o, c, \cdot, \cdot]$ and accumulates over input channels:

$$y[o, i, j] = b[o] + \sum_{c=1}^{C_{\text{in}}} \sum_{a,b} w[o, c, a, b] x[c, i + a, j + b].$$

Stride, Padding, and Output Size

A **stride** s moves the kernel by s positions at a time, and **padding** p adds p rows and columns of zeros (or another value) around the input. For a square input of size H , the output height is

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1.$$

This follows because the top-left corner of the kernel can be placed at positions $0, s, 2s, \dots$ as long as the kernel fits inside the padded input; the number of valid positions is the formula above. The verifier checks it against PyTorch for many combinations and against a direct count of valid placements. With $k = 3, p = 1, s = 1$ the

size is preserved; with $k = 2, s = 2, p = 0$ it halves for even H and floors for odd H (for example $H = 5$ gives 2). The formula returns a nonpositive value when the padded input is smaller than the kernel, in which case the operation is undefined.

Translation Equivariance

Convolution is **translation equivariant**: shifting the input shifts the output by the same amount, with boundary effects aside. Formally, define $(T_\delta x)[i, j] = x[i - \delta_i, j - \delta_j]$. For stride 1, in the interior of the domain,

$$\text{conv}(T_\delta x, w) = T_\delta \text{conv}(x, w).$$

Proof. Using the cross-correlation definition, the output at position (i, j) of $\text{conv}(T_\delta x, w)$ is $\sum_{a,b} w[a, b] x[i + a - \delta_i, j + b - \delta_j]$, which is exactly the output of $\text{conv}(x, w)$ at $(i - \delta_i, j - \delta_j)$, that is, T_δ applied to the output. ■

This is the precise sense in which a feature detector "looks for the same thing everywhere". It is **equivariance**, not **invariance**: the feature moves with the input. With stride $s > 1$ the property holds only for shifts that are multiples of s , because the subsampling grid is fixed; a shift of one pixel is not exactly equivariant through a strided layer. Invariance is obtained later, by global average pooling, which discards position; local pooling only coarsens the spatial map.

Tip: Equivariance is why a convolutional network generalizes across positions. It is also why it does not generalize across rotations or reflections, which require augmentation or specialized architectures.

Pooling

Max pooling takes the maximum over each $k \times k$ window (with a stride, usually equal to the window size), and **average pooling** takes the mean. Pooling reduces spatial resolution and increases the effective receptive field of later layers. It is **equivariant** to shifts by multiples of its stride, and it is only locally insensitive: a shift inside a window leaves the pooled value unchanged **when the same extremum survives**, whereas a shift that moves the maximum out of its window changes the output, and average pooling changes with almost every shift. Pooling has no parameters. The verifier confirms the window values, the output size, and a concrete case where a one-pixel shift changes the pooled output.

Receptive Field

The **receptive field** of a unit is the region of the input that influences it. For a stack of layers with kernel sizes $k_1, \dots, k_L$ and strides $s_1, \dots, s_L$, the receptive field size grows as

$$r_L = 1 + \sum_{\ell=1}^L (k_\ell - 1) \prod_{m < \ell} s_m,$$

and the effective stride of the representation is the product of the strides of the layers before it (for example it doubles after each 2×2 stride-2 pooling layer). This is why deep convolutional networks can reason about large regions: depth and downsampling expand the receptive field without expanding the number of parameters. The verifier checks the formula by probing which input pixels can change a given output pixel; the probe must use average (linear) pooling, because with max pooling the gradient reaches only the argmax and the measured support is smaller than the true receptive field.

The Backward Pass

Convolution is linear in the input and in the kernel, so its gradients are also convolutions. For a single-channel map, the gradient of the loss with respect to the input is the cross-correlation of the upstream gradient with the **flipped** kernel,

$$\frac{\partial L}{\partial x} = \text{conv}\left(\frac{\partial L}{\partial y}, \text{flip}(w)\right),$$

and the gradient with respect to the kernel is the cross-correlation of the input with the upstream gradient,

$$\frac{\partial L}{\partial w} = \text{conv}\left(x, \frac{\partial L}{\partial y}\right),$$

with appropriate padding. Here the input gradient is computed in **full** mode (the output is larger than the input, since the transposed operator maps the smaller output space back to the input space), while the kernel gradient uses **valid** mode. This is the convolutional analogue of "error signal times input" for dense layers. The verifier checks both gradients against PyTorch's autograd to machine precision. The flipped kernel appears because the deep learning operator is cross-correlation; its transpose in full mode is correlation with the flipped kernel.

Convolution as Matrix Multiplication

A convolution can be written as a matrix multiplication. The `im2col` operation gathers the padded input so that each **row** of the resulting patch matrix holds one flattened receptive field; multiplying that matrix by the flattened kernel is an ordinary matrix product. The operator is block-Toeplitz (Toeplitz blocks on a Toeplitz block index), but the `im2col` patch matrix itself is generally not Toeplitz. This view is how convolutions are implemented efficiently on hardware and how one proves that a convolution is a linear operator. The verifier confirms that `im2col` followed by a matrix product reproduces the convolution exactly on a random example.

Parameter Count and Complexity

A convolutional layer has $k^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$ parameters, independent of the spatial size, whereas a dense layer between the same representations would have $(H^2 C_{\text{in}}) \cdot (H^2 C_{\text{out}})$ weights. For $H = 32$, $k = 3$, $C_{\text{in}} = C_{\text{out}} = 32$, the convolution has 9,248 parameters and the dense layer would have 1,073,741,824 (about 1.07×10^9), roughly 116,000 times more. The compute cost of a convolution is $O(H_{\text{out}}^2 k^2 C_{\text{in}} C_{\text{out}})$. The verifier checks the parameter count against PyTorch.

Connections to Deep Learning

- **Parameter sharing** is the convolutional inductive bias; it matches the translation structure of images.
- **Equivariance** is the property that makes a feature detector reusable across positions.
- **Pooling and stride** trade resolution for receptive field and build partial invariance.
- **The backward pass is a convolution**, which is why convolutions are efficient to train.
- **im2col** connects convolutions to matrix multiplication and to hardware accelerators.

Key Takeaways

- Convolution slides a shared kernel over the input; the deep learning convention is cross-correlation.
- The output size is $\lfloor (H + 2p - k)/s \rfloor + 1$.
- Convolution is translation equivariant, not invariant; pooling adds partial invariance.

- Receptive fields grow with depth and stride.
- The input gradient is correlation with the flipped kernel; the kernel gradient is correlation of the input with the upstream gradient.
- Convolutions have far fewer parameters than dense layers and are linear operators expressible as matrix multiplication via `im2col`.

Exercises

1. Compute the output size for $H = 28, k = 5, p = 2, s = 1$.
2. Implement a 3×3 cross-correlation by hand and check it against a library.
3. Prove that convolution is linear in the input.
4. Explain the difference between equivariance and invariance with an example.
5. Derive the receptive-field formula for two 3×3 convolutions with stride 1 and a 2×2 max pool.
6. Verify that the input gradient of a convolution is correlation with the flipped kernel.
7. Count the parameters of a convolutional layer and compare with a dense layer of the same shapes.

The Mathematics of Recurrent Networks and LSTMs

Sequences and State

A feed-forward network maps a fixed-size input to a fixed-size output. A **recurrent neural network** (RNN) maps a sequence to a sequence of outputs, or to a single output, by carrying a **hidden state** from one time step to the next. The state is a summary of everything seen so far, and it is updated as new inputs arrive. This makes RNNs the natural model for text, time series, audio, and any data whose order matters.

The defining equation, with hidden state $h_t \in \mathbb{R}^d$ and input $x_t \in \mathbb{R}^m$, is

$$h_t = \phi(W_h h_{t-1} + W_x x_t + b), \quad y_t = W_y h_t + b_y,$$

where $W_h \in \mathbb{R}^{d \times d}$ is the **recurrent** weight matrix, shared across time, and ϕ is a non-linearity. The same weights are applied at every step, which is parameter sharing along the time axis, exactly as a convolution shares weights along the space axis.

Unrolling and Backpropagation Through Time

The recurrence is a composition. Unrolling over T steps,

$$h_t = \phi(W_h h_{t-1} + W_x x_t + b),$$

shows that the hidden state at time T depends on the input at time 1 through a chain of $T - 1$ applications of W_h and ϕ . Training therefore applies the chain rule through time, a procedure called **backpropagation through time** (BPTT). If $L = \sum_t L_t$ is the total loss, the gradient of the loss with respect to an early state is

$$\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + \left(\frac{\partial h_{t+1}}{\partial h_t} \right)^\top \frac{\partial L}{\partial h_{t+1}},$$

a backward recursion in time, with the local Jacobian

$$\frac{\partial h_{t+1}}{\partial h_t} = \text{diag}(\phi'(z_{t+1})) W_h.$$

The weight gradients accumulate over time: $\partial L / \partial W_h = \sum_t \delta_t h_{t-1}^\top$ and $\partial L / \partial W_x = \sum_t \delta_t x_t^\top$, where $z_t = W_h h_{t-1} + W_x x_t + b$ is the pre-activation and $\delta_t = \partial L / \partial z_t$ is the error signal at step t . The verifier implements BPTT by hand and confirms it matches PyTorch's autograd.

The Vanishing and Exploding Gradient Problem

Unrolling the recursion, the influence of the state at time t on the loss at time $T > t$ involves a product of $T - t$ Jacobians:

$$\frac{\partial h_T}{\partial h_t} = \text{diag}(\phi'(z_T)) W_h \text{diag}(\phi'(z_{T-1})) W_h \cdots \text{diag}(\phi'(z_{t+1})) W_h,$$

As in the multilayer perceptron, a uniform upper bound below one on the spectral norms contracts this product geometrically, so the gradient reaching an early step becomes vanishingly small (**vanishing gradients**); a uniform lower bound above one on the smallest singular value expands it (**exploding gradients**). For a linear recurrence with state-independent $\phi' = 1$, the product is simply W_h^{T-t} , whose behavior is governed by the eigenvalues of W_h : the spectral radius $\rho(W_h)$ controls growth, with the caveats of the linear algebra chapter. This is why a vanilla RNN, in practice, typically cannot learn dependencies that span more than a few dozen steps, and why the LSTM was invented.

The LSTM Cell

The **long short-term memory** (LSTM) cell adds a **cell state** c_t that runs alongside the hidden state and is updated additively, which gives gradients a path that does not shrink through matrix multiplication. It uses three **gates** — the input gate i_t , the forget gate f_t , and the output gate o_t — each a sigmoid of an affine function of the concatenated input and previous hidden state. With σ the logistic sigmoid and $\odot$ the elementwise product, the equations are

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), & i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i), \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o), & \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

The **forget gate** f_t decides how much of the old cell state to keep, the **input gate** i_t decides how much of the candidate $\tilde{c}_t$ to write, and the **output gate** o_t decides how much of the cell state to expose. The candidate $\tilde{c}_t$ is a $\tanh$ of an affine function, so its values lie in $(-1, 1)$, and the gates lie in $(0, 1)$.

The crucial mathematical point is the cell-state update $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. Holding the previous hidden state h_{t-1} fixed, the partial Jacobian of c_t with respect to c_{t-1} is exactly $\text{diag}(f_t)$, the **direct** or "carousel" path. In the fully unrolled network the hidden state also depends on c_{t-1} through $h_{t-1} = o_{t-1} \odot \tanh(c_{t-1})$, so the true total Jacobian is $\text{diag}(f_t)$ plus a term through h_{t-1} ; the carousel path is what dominates when the forget gate stays near one. Along the carousel path the gradient from c_T back to c_t is the product $\prod_{k=t+1}^T \text{diag}(f_k)$ of gate values, independent of the input gate. If the forget gate stays close to one for essentially every step, this product stays near one and gradients flow across many steps; if the forget gates are only slightly below one, the product still decays (for example $0.99^{100} \approx 0.37$), so the near-constant behavior requires $f_t \approx 1$ throughout. This learned modulation of the gradient path is what gives the LSTM its long memory. The verifier confirms the direct cell-state Jacobian, its deviation from the full unrolled Jacobian, and the full forward pass against a reference implementation.

The GRU

The **gated recurrent unit** (GRU) simplifies the LSTM to two gates and a single state. With **update** gate z_t and **reset** gate r_t , bias terms b_* , and the convention used by PyTorch in which the reset gate scales the hidden contribution before the candidate nonlinearity,

$$\begin{aligned} z_t &= \sigma(W_z[h_{t-1}, x_t] + b_z), & r_t &= \sigma(W_r[h_{t-1}, x_t] + b_r), \\ \tilde{h}_t &= \tanh(W_{xh}x_t + b_{xh} + r_t \odot (W_{hh}h_{t-1} + b_{hh})), & h_t &= (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}. \end{aligned}$$

The update gate interpolates between adopting a new candidate (weight $1 - z_t$) and keeping the old state (weight z_t), which is again an additive path that mitigates vanishing gradients with fewer parameters than the LSTM. Note that different references place the reset gate differently; we follow the PyTorch convention so that the equations match the code.

Gradient Clipping

Because exploding gradients are common in recurrent networks, a standard remedy is **gradient clipping**. The simplest form, **norm clipping**, rescales the gradient when its norm exceeds a threshold τ :

$$g \leftarrow g \cdot \min\left(1, \frac{\tau}{\|g\|}\right) \quad (g \neq 0), \quad g \leftarrow g \quad (g = 0).$$

This preserves the direction of the update while bounding its magnitude, and it leaves gradients with norm at or below τ unchanged. The verifier confirms the norm bound, the direction preservation, and the zero-gradient case on randomized inputs. The alternative, clipping by value, caps each coordinate independently and can change the direction.

Connection to the Product-of-Jacobians Analysis

Both the MLP and the RNN share the same analysis: depth in a feed-forward network is the number of layers, and "depth" in a recurrent network is the number of time steps. A product of many Jacobians with norms bounded away from one either vanishes or explodes. The three structural fixes in practice are gating (LSTM, GRU), which creates additive gradient paths; normalization; and residual connections, which add an identity path (the block's Jacobian is then $I + J_f$, so the identity term is exactly I).

Key Takeaways

- An RNN applies shared weights at every time step and carries a hidden state.
- BPTT is the chain rule unrolled over time; weight gradients accumulate over steps.
- The product of time-step Jacobians causes vanishing and exploding gradients.
- The LSTM's additive cell-state update with the forget gate gives a near-constant gradient path.
- The GRU achieves a similar effect with fewer parameters.
- Gradient clipping bounds the update magnitude without changing its direction.

Exercises

1. Unroll a scalar linear RNN $h_t = ah_{t-1} + x_t$ and solve for h_T in terms of the inputs.
2. Explain why $\partial h_T / \partial h_t = W_h^{T-t}$ for a linear RNN.
3. Derive the LSTM cell-state Jacobian $\partial c_t / \partial c_{t-1}$.
4. Explain how the forget gate near one prevents vanishing gradients.
5. Implement an LSTM cell by hand and check it against PyTorch.
6. Prove that norm clipping never increases the gradient norm.
7. Compare the number of parameters of an LSTM and a GRU of the same hidden size.

The Mathematics of Attention and Transformers

Attention as Soft Retrieval

Attention answers a simple question: given a **query**, which of a set of stored items should I look at, and how much? Each stored item has a **key** that describes what it contains and a **value** that is its content. The query is compared to every key, the comparisons become a set of weights, and the output is the weighted average of the values. When the comparison is a dot product and the weights are a softmax, this is **scaled dot-product attention**, the operation at the heart of every transformer.

The reason this replaces recurrence is that attention connects any two positions in a single step. A recurrent network must pass information through every intermediate state, so its gradients are products of layer Jacobians that can vanish or explode; attention gives a direct path from every position to every other, which shortens the gradient path. The cost of that direct path is quadratic time in the sequence length, and long-context training remains delicate for other reasons.

Queries, Keys, Values, and the Attention Formula

Given a matrix Q of queries, K of keys, and V of values, each with rows of dimension d_k (for Q, K) and d_v (for V), **scaled dot-product attention** is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

The matrix $S = QK^\top$ has entries $S_{ij} = q_i^\top k_j$, the similarity of query i to key j . Dividing by $\sqrt{d_k}$ and applying the softmax **row-wise** turns each row into a probability distribution:

$$A_{ij} = \frac{\exp(S_{ij}/\sqrt{d_k})}{\sum_{j'} \exp(S_{ij'}/\sqrt{d_k})}, \quad \sum_j A_{ij} = 1, \quad A_{ij} \geq 0.$$

The output row is $\sum_j A_{ij} v_j$, a **convex combination** of the value rows. In self-attention, Q, K , and V are all linear projections of the same sequence, $Q = XW_Q, K = XW_K, V = XW_V$; in cross-attention, the queries come from one sequence and the keys and values from another.

Why the Scaling Factor

The factor $1/\sqrt{d_k}$ is not cosmetic. Suppose the components of q and k are independent with mean zero and unit variance. Then the dot product $q^\top k = \sum_{i=1}^{d_k} q_i k_i$ has mean zero and variance d_k , because each term has variance one and the terms are independent. The standard deviation grows like $\sqrt{d_k}$. For large d_k the logits become large, the softmax saturates, and its gradients vanish. Dividing the logits by $\sqrt{d_k}$ standardizes them to unit variance, which keeps the softmax in its responsive regime. The verifier confirms the d_k variance empirically and the unit variance after scaling.

Softmax Properties

The softmax is invariant to adding a constant to every logit, because the constant cancels in numerator and denominator. It is monotone, so larger logits receive larger weights, and its output has positive entries summing to one. Its Jacobian is $\text{diag}(p) - pp^\top$: differentiating $p_i = e^{z_i} / \sum_k e^{z_k}$ gives $\partial p_i / \partial z_j = p_i(\delta_{ij} - p_j)$, which is the matrix $\text{diag}(p) - pp^\top$. The invariance to constant shifts is why subtracting any constant, and in particular the row maximum, is valid; the maximum is chosen specifically to avoid overflow and underflow when the logits are large.

Multi-Head Attention

A single attention operation averages over one notion of similarity. **Multi-head attention** runs h attention operations in parallel on different learned projections and concatenates the results:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O, \quad \text{head}_i = \text{Attention}(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)}).$$

Each head can attend to a different kind of relationship, and the output projection mixes them. With $W_O \in \mathbb{R}^{d \times d}$ and per-head dimensions $d_k = d_v = d/h$, the concatenation restores the full width d and the total computation is the same as a single full-width head, to within lower-order terms.

Causal Masking

For a language model, position i must not see positions $j > i$, or the prediction of the next token would have access to the answer. A **causal mask** sets the logits for $j > i$ to $-\infty$ before the softmax, so their weights become exactly zero (this is safe for causal masks because no row is masked entirely, which would otherwise produce $0/0$):

$$S_{ij} \leftarrow -\infty \quad \text{for } j > i, \quad A_{ij} = 0 \quad \text{for } j > i.$$

This makes the attention matrix lower triangular, and it makes next-token prediction a well-posed task. The verifier confirms that a causally masked output at position i is unchanged when the inputs at positions $j > i$ are altered, and that the attention matrix is lower triangular.

Positional Encoding

For **unmasked** attention over a set of rows, self-attention is **permutation equivariant**: if you permute the rows of the input, the output rows are permuted the same way. Writing $Q = XW_Q$, $K = XW_K$, $V = XW_V$ and $\text{SA}(X)$ for the self-attention output, $\text{SA}(PX) = P \text{SA}(X)$ for any permutation matrix P , because permuting rows permutes the scores both by row and by column, and row-wise softmax commutes with that joint permutation. A causal mask breaks this equivariance, because the mask itself is tied to the order. In any case this is a virtue for sets and a problem for sequences, which need order. The fix is to add a **positional encoding** to the input embeddings.

The sinusoidal encoding of Vaswani et al. is, for positions pos and dimensions $2i, 2i + 1$,

$$PE[\text{pos}, 2i] = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad PE[\text{pos}, 2i + 1] = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right).$$

The frequencies form a geometric progression, so the encoding has both high-resolution and low-resolution components across dimensions. Its key property is that the encoding at position $\text{pos} + k$ is a fixed linear (rotation) function of the encoding at pos , the same function for every pos . This is why a linear projection can

learn to attend by relative position. The verifier confirms the rotation property for $k = 3$ and the geometric spread of frequencies. Learned positional embeddings are the common alternative and are used by GPT.

Layer Normalization and Residual Connections

A transformer block has two sublayers, each wrapped in a residual connection. In the pre-norm arrangement,

$$X' = X + \text{MHA}(\text{LN}(X)), \quad \text{Block}(X) = X' + \text{FFN}(\text{LN}(X')),$$

where **layer normalization** standardizes the features of each position (each row) to zero mean and unit variance and then applies a learned scale and shift, **MHA** is multi-head self-attention, and **FFN** is a position-wise two-layer network with a non-linearity, usually $\text{Linear}(d \rightarrow 4d) \rightarrow \text{GELU} \rightarrow \text{Linear}(4d \rightarrow d)$. The **residual connection** $X + \cdot$ adds an identity path; for a pre-norm block the Jacobian of the residual sublayer is $I + J_f$, so gradients have a direct route to earlier layers. (In a post-norm block the output is normalized instead, and the Jacobian is $J_{\text{LN}}(I + J_f)$, which is why pre-norm trains more easily at depth.) The verifier confirms layer normalization produces zero mean and unit variance per row.

There are two arrangements. **Post-norm** applies normalization after the residual addition, as in the original transformer; **pre-norm** applies it before the sublayer, as in GPT-2 and most modern models. Pre-norm is easier to train at depth because the identity path is unnormalized all the way to the input.

Complexity and the Quadratic Bottleneck

For a sequence of length n and model width d , the attention matrix is $n \times n$ per head, so attention costs $O(n^2d)$ time and, if materialized, $O(n^2)$ memory. The feed-forward network costs $O(nd^2)$. For sequences shorter than the model width the feed-forward term dominates, and once n grows past about d the quadratic attention term dominates. This is the motivation for efficient attention variants (sparse, linear, and low-rank approximations), which trade exactness for sub-quadratic cost.

Key Takeaways

- Attention is soft retrieval: compare a query to keys, softmax the scores, and average the values.
- The $1/\sqrt{d_k}$ scaling standardizes the logits so the softmax does not saturate.
- Each head's output is a convex combination of that head's values.
- Multi-head attention attends to several relationships and mixes them.
- The causal mask makes language modeling well-posed and yields a lower-triangular attention matrix.
- Attention is permutation equivariant; positional encodings inject order.
- Residual connections and layer normalization make deep transformer stacks trainable.
- Attention is $O(n^2d)$; the quadratic cost in sequence length is the central scalability challenge.

Exercises

1. Compute a 2×2 attention for given Q, K, V by hand.
2. Derive the variance of $q^\top k$ for unit-variance independent components.
3. Show that softmax is invariant to adding a constant to the logits.
4. Prove that self-attention without positional encoding is permutation equivariant.
5. Verify that the sinusoidal encoding at $\text{pos} + k$ is a linear function of the encoding at pos .
6. Explain why a causal mask makes next-token prediction well-posed.
7. Derive the $O(n^2 d)$ and $O(nd^2)$ cost terms.

The Mathematics of GPT

What GPT Is

A GPT is a decoder-only transformer language model: it reads a sequence of tokens and predicts the probability distribution of the next token at every position. Everything that makes a large language model is a scaled-up version of the small model in this chapter. Our goal is to write down the forward pass exactly, in order, with every shape and formula, and to derive the quantities that govern training and generation: the cross-entropy objective, perplexity, the parameter count, the sampling temperature, and the causal property that makes prediction well-posed. The key formulas are checked in `build/verify/gpt.py`, including the causal attention against a reference primitive and the parameter count against a reference module.

Tokenization

Text is converted to integers by a **tokenizer**. The GPT family uses **subword** units found by byte-pair encoding (BPE); other models use WordPiece or SentencePiece unigram. BPE works as follows: start from individual bytes or characters, then repeatedly merge the most frequent adjacent pair into a new symbol. The result is a vocabulary of size V that balances the character-level robustness of a small alphabet against the word-level efficiency of a large one. Common words become single tokens; rare words are split into pieces. The tokenizer is fixed and is not learned during training.

Embeddings

Each token id $t \in \{0, \dots, V - 1\}$ indexes a row of the **token embedding** matrix $E \in \mathbb{R}^{V \times d}$, and each position $p \in \{0, \dots, T - 1\}$ indexes a row of the **positional embedding** matrix $P \in \mathbb{R}^{T \times d}$, where d is the model width and T the maximum context length. The input to the first block is

$$x_p^{(0)} = E[t_p] + P[p], \quad x^{(0)} \in \mathbb{R}^{T \times d}.$$

GPT uses **learned** positional embeddings (unlike the sinusoidal encoding of the previous chapter). The pair (t_p, p) is mapped to the sum $E[t_p] + P[p]$; this map is not guaranteed to be injective, because different token and position pairs can add to the same vector (for example, $E[0] + P[1]$ can equal $E[1] + P[0]$), but it makes position information available to every layer and lets the learned embeddings distinguish occurrences that share a token.

The Transformer Block

GPT applies N identical **pre-norm** blocks. Each block has two sublayers, multi-head causal self-attention (MHA) and a position-wise feed-forward network (FFN), each wrapped in a residual connection:

$$a = x + \text{MHA}(\text{LN}_1(x)), \quad x' = a + \text{FFN}(\text{LN}_2(a)).$$

The attention is **causal**: position p attends only to positions $q \leq p$. With h heads of dimension $d_k = d/h$ and weight matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ and $W_O \in \mathbb{R}^{d \times d}$, the single-head attention is written with $Q = \text{LN}_1(x)W_Q, K = \text{LN}_1(x)W_K, V = \text{LN}_1(x)W_V$,

$$A = \text{softmax}_{\text{row}} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right), \quad M_{pq} = \begin{cases} 0, & q \leq p \\ -\infty, & q > p \end{cases}, \quad \text{head} = AV.$$

Multiple heads are concatenated and projected: $\text{MHA}(z) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$. The FFN is $\text{FFN}(u) = W_2 \phi(W_1 u + b_1) + b_2$ with $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$, typically $d_{\text{ff}} = 4d$, and ϕ a GELU or similar non-linearity. Layer normalization standardizes each row and applies a learned scale and shift.

The Language-Model Head and the Objective

After the final block, a final layer normalization is applied and a linear **head** maps the width to the vocabulary: $\text{logits} = \text{LN}_f(x^{(N)})W_{\text{lm}}$, with $W_{\text{lm}} \in \mathbb{R}^{d \times V}$. This is usually **weight-tied** to the token embedding, $W_{\text{lm}} = E^\top$, which saves Vd parameters and is observed to help. The probability of the next token is the softmax of the logits, and the training objective is the mean negative log-likelihood over all positions:

$$L = -\frac{1}{T-1} \sum_{p=0}^{T-2} \log p(t_{p+1} \mid t_0, \dots, t_p) = -\frac{1}{T-1} \sum_{p=0}^{T-2} \log \frac{\exp(z_{p,t_{p+1}})}{\sum_{k=0}^{V-1} \exp(z_{p,k})},$$

where $z_{p,\cdot}$ are the logits at position p and t_{p+1} is the true next token; equivalently, the input holds T tokens and the loss averages over the $T-1$ next-token predictions. This is the cross-entropy loss from the probability chapter, and its gradient with respect to the logits at a single position is $\hat{p} - y$, the prediction minus the one-hot target (for the mean over positions, the gradient carries an overall factor of $1/(T-1)$).

The **perplexity** is the exponential of the average loss, $\text{PPL} = \exp(L)$. It is the effective number of equally likely choices the model is uncertain among; a perplexity of k means the model is about as uncertain as a uniform distribution over k tokens. Perplexity is only meaningful for a fixed tokenizer, because different tokenizations give different vocabularies.

The Causal Property

The causal mask guarantees that the logits at position p depend only on tokens at positions $0, \dots, p$. This is what makes the model a valid next-token predictor: when we generate, we compute the distribution for position p from the prefix $t_0, \dots, t_p$ and never use the future. Training computes the loss at every position in parallel from a single forward pass, which is the efficiency that makes transformers trainable. The verifier confirms that changing tokens beyond position p leaves the logits at p unchanged.

Parameter Count

The parameters of a GPT are:

Component	Shape	Count
Token embedding	$V \times d$	Vd
Positional embedding	$T \times d$	Td
Per block: attention W_Q, W_K, W_V, W_O	$4 \times d \times d$	$4d^2$ (plus biases)
Per block: FFN W_1, W_2	$d \times 4d, 4d \times d$	$8d^2$ (plus biases)
Per block: layer-norm gains and biases	$4 \times d$	$4d$

Component	Shape	Count
Final layer norm	$2d$	$2d$
Head (tied)	shared with E	0 if tied

Including the layer-norm parameters but ignoring linear biases, the total is

$$\text{Params} = Vd + Td + N(12d^2 + 4d) + 2d,$$

where the $12d^2$ per block is $4d^2$ for attention and $8d^2$ for the feed-forward network. For a model with $V = 50257$, $d = 768$, $T = 1024$, $N = 12$, this is about 124M parameters, the size of GPT-2 small (whose linear biases bring it to about 124.4M). The verifier confirms the per-block and total counts against a reference module built from the same layers.

Training

GPT is trained with **AdamW**, a **learning-rate schedule** consisting of a linear warmup followed by cosine decay to a small fraction of the peak, and **gradient clipping** by global norm. The warmup avoids the large early updates that destabilize attention, and the decay reduces the noise at the end of training. Weight decay is applied to the weight matrices but not to biases or layer-norm parameters.

Sampling

To generate, the model produces logits z for the next token, and we choose a token from the distribution. The **temperature** $\tau > 0$ divides the logits before the softmax:

$$p_k(\tau) = \frac{\exp(z_k/\tau)}{\sum_j \exp(z_j/\tau)}.$$

As $\tau \rightarrow 0$ the distribution concentrates on the argmax (**greedy**); as $\tau \rightarrow \infty$ it approaches uniform. Dividing logits by τ rescales the differences between them, which controls how peaked the distribution is. Balanced against temperature, two truncation schemes sharpen the tail: **top- k** keeps only the k highest-probability tokens and renormalizes, and **top- p** (nucleus) keeps the smallest set whose cumulative probability exceeds p and renormalizes.

Temperature scaling is equivalent to raising the original probabilities to the power $1/\tau$ and renormalizing: if $p = \text{softmax}(z)$, then $p_k(\tau) = p_k^{1/\tau} / \sum_j p_j^{1/\tau}$. The verifier checks the temperature softmax and both truncation rules.

Complexity and Scaling

The cost of a forward pass is dominated by two terms: attention is $O(T^2d)$ time and, if the attention matrix is materialized, $O(T^2)$ memory per head-batch; the feed-forward network is $O(Td^2)$. The number of parameters grows as $O(Nd^2)$ for fixed vocabulary. Two empirical scaling results guide the field: the Kaplan et al. (2020) scaling laws found smooth power laws relating loss to parameters, data, and compute, and the Chinchilla work (Hoffmann et al., 2022) found that many large models were undertrained, with a compute-optimal ratio of roughly twenty training tokens per parameter. These are empirical regularities, not theorems, and they depend on the data distribution and architecture.

Key Takeaways

- GPT is a decoder-only transformer: token plus positional embeddings, N pre-norm blocks, a final norm, and a vocabulary head.
- The head is usually weight-tied to the token embedding.
- The objective is next-token cross-entropy, equivalently the negative log-likelihood; its gradient is $\hat{p} - y$.
- Perplexity is the exponential of the average loss and is tokenizer-dependent.
- The causal mask makes the model a valid next-token predictor and makes parallel training possible.
- The parameter count is approximately $Vd + Td + 12Nd^2$.
- Sampling uses temperature on the logits, often with top- k or top- p truncation.
- Attention is $O(T^2d)$ and dominates for long contexts; scaling laws are empirical.

Exercises

1. Write the shapes of every tensor in a GPT forward pass.
2. Derive the parameter count $Vd + Td + 12Nd^2$ component by component.
3. Show that the cross-entropy gradient with respect to the logits is $\hat{p} - y$.
4. Compute the perplexity of a model whose average negative log-likelihood is 2.5.
5. Derive $\text{softmax}(z/\tau)$ and explain the limits as $\tau \rightarrow 0$ and $\tau \rightarrow \infty$.
6. Implement top- p sampling and explain how it differs from top- k .
7. Explain why the causal mask makes parallel training across positions possible.
8. State two scaling laws and the caveat that they are empirical.

The Mathematics of Autoencoders, Variational Autoencoders, and GANs

Three Ways to Learn a Distribution

The models in this chapter learn the distribution of the data rather than a label for it. An **autoencoder** learns a compact code by requiring that the data can be reconstructed from the code. A **variational autoencoder** (VAE) learns a probabilistic latent variable model and is trained by maximizing a lower bound on the data likelihood. A **generative adversarial network** (GAN) learns to sample by pitting a generator against a discriminator. Each approach answers a different question about the same data, and each has a characteristic failure mode. This chapter derives the objectives and proves the key properties, with key claims checked in `build/verify/generative.py`.

The Autoencoder

An **autoencoder** is a pair of functions: an **encoder** $f_\phi : \mathbb{R}^D \rightarrow \mathbb{R}^k$ that maps data to a latent code, and a **decoder** $g_\theta : \mathbb{R}^k \rightarrow \mathbb{R}^D$ that maps the code back to data space. It is trained to minimize the **reconstruction loss**

$$L(\phi, \theta) = \mathbb{E}_{x \sim p_{\text{data}}} [\|x - g_\theta(f_\phi(x))\|^2].$$

When $k < D$ the code is a **bottleneck**, and the reconstruction error measures how much information was lost. The autoencoder is a nonlinear generalization of principal component analysis.

Linear Autoencoders Recover PCA

Consider a linear autoencoder with no biases and no activations, $f_\phi(x) = W_e x$ and $g_\theta(z) = W_d z$, with $W_e \in \mathbb{R}^{k \times D}$, $W_d \in \mathbb{R}^{D \times k}$, minimizing the squared reconstruction error over centered data. The composition $W_d W_e$ is a rank- k linear map, and by the Eckart-Young theorem from the linear algebra chapter the best rank- k approximation of the data in the least-squares sense is the truncated SVD. With the data arranged as rows of $X \in \mathbb{R}^{N \times D}$, that approximation is $X V_k V_k^\top$, the projection onto the span of the top k right singular vectors V_k (the principal directions). The reduction to the autoencoder is direct: it minimizes $\|X - X P\|_F^2$ over rank- k linear maps P , and the minimizing P is the orthogonal projection $V_k V_k^\top$, attained by $W_e = V_k^\top$, $W_d = V_k$. Therefore, generically (when the top- k singular values are distinct), the composition $W_d W_e$ is the rank- k orthogonal projection onto the top- k principal subspace, and a linear autoencoder learns PCA. The verifier confirms that a trained linear autoencoder reaches the PCA reconstruction loss, that $W_d W_e$ is close to $V_k V_k^\top$, and that a random rank- k map is worse.

The nonlinear autoencoder goes beyond PCA because the encoder and decoder can warp space, but the bottleneck principle is the same: the code must retain the directions that matter for reconstruction.

Denoising and Sparse Autoencoders

A **denoising autoencoder** corrupts the input, for example by adding Gaussian noise or dropping entries, and trains the model to reconstruct the clean input, which forces the code to capture robust structure rather than identity. A **sparse autoencoder** adds a penalty on the code, such as $\lambda \|f_\phi(x)\|_1$, so that only a few latent units

activate and the code becomes interpretable. Both are variations on the same objective with a different corruption or penalty.

Variational Autoencoders

A **variational autoencoder** treats the code as a latent random variable z and assumes a generative model $p_\theta(x, z) = p(z)p_\theta(x | z)$ with a simple prior, usually $p(z) = \mathcal{N}(0, I)$. The evidence for a data point is $p_\theta(x) = \int p_\theta(x | z)p(z) dz$, which is intractable because the integral over the latent space cannot be computed in closed form. The VAE introduces an approximate posterior $q_\phi(z | x)$ and maximizes the **evidence lower bound** (ELBO)

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - D_{\text{KL}}(q_\phi(z | x) \| p(z)).$$

Theorem. $\log p_\theta(x) \geq \mathcal{L}(\theta, \phi; x)$, with equality when $q_\phi(z | x) = p_\theta(z | x)$.

Proof. For any $q(z)$ with support containing that of $p_\theta(z | x)$,

$$\log p_\theta(x) = \log \int p_\theta(x | z)p(z) dz = \log \mathbb{E}_q \left[\frac{p_\theta(x | z)p(z)}{q(z)} \right] \geq \mathbb{E}_q \left[\log \frac{p_\theta(x | z)p(z)}{q(z)} \right],$$

by Jensen's inequality, since $\log$ is concave. The right side is

$$\mathbb{E}_q [\log p_\theta(x | z)] + \mathbb{E}_q \left[\log \frac{p(z)}{q(z)} \right] = \mathbb{E}_q [\log p_\theta(x | z)] - D_{\text{KL}}(q \| p),$$

because $p(z)$ does not depend on q . Rearranging the gap, the difference between the log evidence and the bound equals $\log p_\theta(x) - \mathcal{L} = D_{\text{KL}}(q_\phi(z | x) \| p_\theta(z | x)) \geq 0$, which is zero exactly when the approximate posterior equals the true posterior. ■

The first term of the ELBO rewards accurate reconstruction; the second term regularizes the codes toward the prior, which is what makes the latent space usable for sampling. For the common choice of a diagonal Gaussian posterior $q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \text{diag}(\sigma_\phi^2(x)))$ and standard normal prior, the KL term has the closed form

$$D_{\text{KL}}(\mathcal{N}(\mu, \text{diag}(\sigma^2)) \| \mathcal{N}(0, I)) = \frac{1}{2} \sum_{j=1}^k (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2).$$

The verifier checks this formula against numerical integration.

The Reparameterization Trick

To train q_ϕ by gradient descent we must differentiate through a sample $z \sim q_\phi(z | x)$, but sampling is not differentiable. The **reparameterization trick** rewrites the sample as a deterministic function of the parameters and an independent noise variable:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

The randomness is moved into ε , which does not depend on ϕ , so z is a differentiable function of μ_ϕ and $\log \sigma_\phi$ and gradients flow through it. The verifier confirms that the reparameterized gradient and the score-function gradient are both unbiased estimators of the true gradient, and that the samples have the correct mean and variance.

Generative Adversarial Networks

A **GAN** learns to sample by training two networks against each other. The **generator** G_θ maps noise $z \sim p(z)$ to a sample $G_\theta(z)$ with distribution p_g , and the **discriminator** D_ψ outputs the probability that its input is real. The **minimax objective** is

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))].$$

Theorem (optimal discriminator). For fixed G , the maximizing D is

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

Proof. Write $V(D, G) = \int [p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x))] dx$. The integrand, as a function of $D(x) \in [0, 1]$, is of the form $a \log D + b \log(1 - D)$ with $a = p_{\text{data}}(x)$, $b = p_g(x)$, and is maximized at $D = a/(a + b)$ when $a + b > 0$ (its derivative is $a/D - b/(1 - D)$); where $a = b = 0$ the integrand is identically zero and the choice of D is immaterial. Setting $a = p_{\text{data}}(x)$ and $b = p_g(x)$ gives the result. ■

Theorem (value at the optimum). Substituting D^* , the value satisfies

$$V(D^*, G) = -\log 4 + 2 \text{JSD}(p_{\text{data}} \| p_g),$$

where the Jensen-Shannon divergence JSD is $\frac{1}{2} D_{\text{KL}}(p \| m) + \frac{1}{2} D_{\text{KL}}(q \| m)$ with $m = (p + q)/2$.

Proof. Substituting D^* and using $\log D^* = \log p_{\text{data}} - \log(p_{\text{data}} + p_g)$ and $\log(1 - D^*) = \log p_g - \log(p_{\text{data}} + p_g)$, the value becomes $\mathbb{E}_{p_{\text{data}}} [\log D^*] + \mathbb{E}_{p_g} [\log(1 - D^*)] = -\log 4 + D_{\text{KL}}(p_{\text{data}} \| m) + D_{\text{KL}}(p_g \| m)$ with $m = (p_{\text{data}} + p_g)/2$; the last two terms are $2 \text{JSD}(p_{\text{data}} \| p_g)$. ■

Because the Jensen-Shannon divergence is nonnegative and zero only when the two distributions coincide, the global minimum of the generator's problem is at $p_g = p_{\text{data}}$, where $D^* = 1/2$ and $V = -\log 4$. The verifier confirms the optimal-discriminator formula and the Jensen-Shannon identity numerically, both on discrete distributions.

Mode Collapse and the Practical Objective

The theory assumes the discriminator can be optimized to convergence at every step, which is not true in practice. When the discriminator becomes too strong, the generator's gradient vanishes, and when the generator finds a few samples that fool the discriminator, it may produce only those, a failure called **mode collapse**. A common remedy is the non-saturating generator objective

$$\max_G \mathbb{E}_z [\log D(G(z))],$$

which has the same global optimum $p_g = p_{\text{data}}$ but much larger gradients when $D(G(z))$ is small. The verifier confirms that the two objectives differ over a range of D , while coinciding at the optimum $D = 1/2$.

Comparison

The autoencoder is deterministic and trained by reconstruction; it learns a useful code but is not a sampler in the probabilistic sense. The VAE is probabilistic and trained by a likelihood lower bound; it samples well but its reconstructions are often blurry because the reconstruction term averages over the latent posterior. The GAN is

not likelihood-based and produces sharp samples, but it is unstable, prone to mode collapse, and its objective does not directly measure the likelihood of the data. Choosing among them is a trade-off between sample quality, stability, and whether a likelihood or a code is needed.

Key Takeaways

- A linear autoencoder under squared error learns the principal subspace; nonlinear autoencoders generalize this.
- The VAE maximizes a lower bound on the log evidence; the bound is tight when the approximate posterior equals the true posterior.
- The ELBO is reconstruction minus a KL term toward the prior; the Gaussian KL has a closed form.
- The reparameterization trick makes sampling differentiable.
- The optimal GAN discriminator is $p_{\text{data}}/(p_{\text{data}} + p_g)$, and the optimal value is $-\log 4 + 2 \text{JSD}$.
- The GAN global optimum is $p_g = p_{\text{data}}$ with $D^* = 1/2$.
- Mode collapse and training instability are the practical obstacles, mitigated by the non-saturating objective.

Exercises

1. Show that a linear autoencoder with squared error learns a rank- k approximation.
2. Derive the ELBO from Jensen's inequality.
3. Derive the Gaussian KL closed form.
4. Show that the reparameterized sample has the correct mean and variance.
5. Derive the optimal GAN discriminator.
6. Show that the GAN value at the optimum is $-\log 4 + 2 \text{JSD}$.
7. Explain mode collapse and how the non-saturating objective helps.

Part V: Classification and Deep Learning with PyTorch

Chapter 15: Penguins Dataset - Logistic Regression

Introduction

Linear regression predicts a quantity; logistic regression predicts a probability. It is the standard first model for binary classification, and it is built on the same linear machinery you already know, wrapped in a function that squashes any number into the range between zero and one. This chapter applies logistic regression to the Penguins dataset, builds a small ladder of candidate models, evaluates them with ROC and AUC, and produces a reusable inference engine.

Logistic regression is also a lesson in a subtle danger. The Penguin species are so distinct that predicting Gentoo is almost trivial, and a model that is too easy teaches nothing. We therefore predict whether a penguin is an Adelie, a harder question with genuine overlap, so that the model has to earn its accuracy.

What You Will Learn

- Why logistic regression is the right model for a binary target.
- How to fit a logistic regression with the statsmodels formula interface.
- How to read coefficients as changes in the log-odds.
- How to evaluate a classifier with a confusion matrix, ROC curve, and AUC.
- How to save a fitted classifier and use it for inference.

Overview

We begin by turning the three-class species target into a binary target and checking that the classes are reasonably balanced. We then fit a ladder of models, from an intercept-only baseline up to interactions, and compare them on pseudo R-squared, AIC, and accuracy. The formula interface keeps the model readable, and the coefficients, exponentiated, become odds ratios that a domain expert can interpret.

Evaluation is where classification differs from regression. Accuracy alone is not enough, so we look at the confusion matrix and the classification report, which break performance down by class. The ROC curve plots the true positive rate against the false positive rate across every possible threshold, and the area under it, the AUC, summarizes how well the model ranks positives above negatives. A model with a high AUC is a good ranker even if the default threshold is not optimal.

Finally we save the best model, reload it, and predict the probability that a new penguin is an Adelie. The inference function returns a probability and a class, which is exactly what a decision-maker needs.

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import seaborn as sns
import statsmodels.formula.api as smf
from sklearn.linear_model import LogisticRegression
from patsy import dmatrices
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Load Dataset

```
df = pd.read_csv('data/penguins-clean.csv')
```

```
df.head()
```

```
species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen           39.1           18.7           181.0
1  Adelie  Torgersen           39.5           17.4           186.0
2  Adelie  Torgersen           40.3           18.0           195.0
3  Adelie  Torgersen           36.7           19.3           193.0
4  Adelie  Torgersen           39.3           20.6           190.0

body_mass_g  sex
0      3750.0  Male
1      3800.0  Female
2      3250.0  Female
3      3450.0  Female
4      3650.0  Male
```

Logistic Regression

Logistic regression predicts the probability that a sample belongs to a class. We turn the three-class `species` target into a binary target, `is_adelie`, and model the probability that a penguin is an Adelie.

Create a Binary Target

```
df['is_adelie'] = (df['species'] == 'Adelie').astype(int)
df['is_adelie'].value_counts()
```

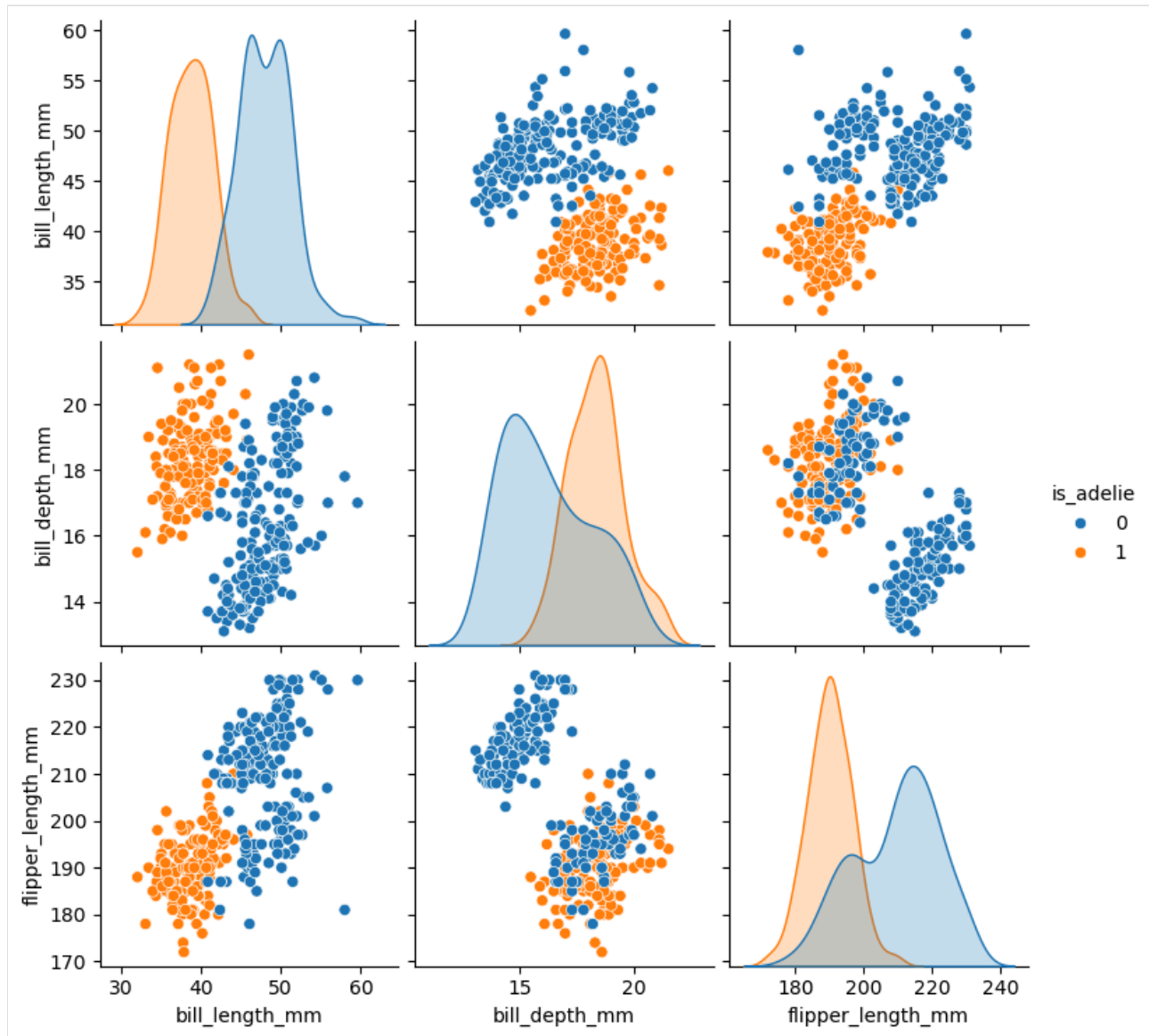
```
is_adelie
0      187
1      146
Name: count, dtype: int64
```

The classes are reasonably balanced, so accuracy is a fair starting metric, but we will still inspect ROC and AUC.

Visualize Relationship Between the Target and Each Continuous Input

Adelie penguins have shorter bills and flippers. `bill_length_mm` separates the classes most cleanly.

```
sns.pairplot(
    df,
    vars=['bill_length_mm',
          'bill_depth_mm',
          'flipper_length_mm'],
    hue='is_adelie')
plt.show()
```



Fit Logistic Regression Models - Training Data

Functions

`fit_and_assess_logit` fits a model with the statsmodels formula API and returns one row of metrics.

`predict_class` turns probabilities into class labels at a 0.5 threshold.

```
def fit_and_assess_logit(model, formula, df):
    """
```

Fit a logistic regression model and return its metrics.

Parameters:

model (str): A descriptive name for the model.
 formula (str): A statsmodels formula string.
 df (pandas.DataFrame): The input data.

Returns:

pandas.DataFrame: A single-row DataFrame of metrics.

```

"""
a_model = smf.logit(formula=formula, data=df).fit(dis=0)
preds = predict_class(a_model, df)
results_dict = {'model_name': model,
                'model_formula': formula,
                'num_coefs': len(a_model.params),
                'pseudo_r2': a_model.prsquared,
                'aic': a_model.aic,
                'accuracy': (preds == df['is_adelie']).mean()}
return pd.DataFrame(results_dict, index=[0])

```

```
def predict_class(a_model, df):
```

"""

Predict binary class labels from a fitted model.

Parameters:

a_model (statsmodels.discrete.discrete_model.BinaryResultsWrapper): Fitted model.
 df (pandas.DataFrame): The input data.

Returns:

numpy.ndarray: Predicted class labels.

"""

```

probs = a_model.predict(df)
return (probs >= 0.5).astype(int)

```

Formulas

Island is left out of the predictors because every Gentoo penguin in this dataset lives on Biscoe island, which would leak the label.

```

formula_00 = 'is_adelie ~ 1'
formula_01 = 'is_adelie ~ flipper_length_mm'
formula_02 = 'is_adelie ~ bill_length_mm + bill_depth_mm + flipper_length_mm'
formula_03 = 'is_adelie ~ bill_length_mm + bill_depth_mm + flipper_length_mm + C(sex)'
formula_04 = 'is_adelie ~ bill_length_mm * bill_depth_mm + flipper_length_mm'
formula_05 = 'is_adelie ~ flipper_length_mm * C(sex)'
formula_05

```

```
'is_adelie ~ flipper_length_mm * C(sex)'
```

Model 00: Intercept-Only (Baseline)

```
fit_and_assess_logit('Model 00', formula_00, df)
```

	model_name	model_formula	num_coefs	pseudo_r2	aic	accuracy
0	Model 00	is_adelie ~ 1	1	7.149836e-13	458.575142	0.561562

Model 01: Flipper Length Only

```
fit_and_assess_logit('Model 01', formula_01, df)
```

```

model_name          model_formula num_coefs pseudo_r2      aic \
0  Model 01  is_adelie ~ flipper_length_mm          2  0.450399 254.93432

accuracy
0  0.822823

```

Model 02: All Continuous Predictors

```
fit_and_assess_logit('Model 02', formula_02, df)
```

```

model_name          model_formula num_coefs \
0  Model 02  is_adelie ~ bill_length_mm + bill_depth_mm + f...          4

pseudo_r2      aic accuracy
0      0.9658 23.614828 0.990991

```

Model 03: Continuous Predictors + Sex

```
fit_and_assess_logit('Model 03', formula_03, df)
```

```

/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/discrete/discrete_model.py:2385:
RuntimeWarning: overflow encountered in exp
    return 1/(1+np.exp(-X))
/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
    warnings.warn("Maximum Likelihood optimization failed to ")
/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/discrete/discrete_model.py:2385:
RuntimeWarning: overflow encountered in exp
    return 1/(1+np.exp(-X))

```

```

model_name          model_formula num_coefs \
0  Model 03  is_adelie ~ bill_length_mm + bill_depth_mm + f...          5

pseudo_r2      aic accuracy
0      1.0 10.000157      1.0

```

Model 04: Continuous Interaction

```
fit_and_assess_logit('Model 04', formula_04, df)
```

```

model_name          model_formula num_coefs \
0  Model 04  is_adelie ~ bill_length_mm * bill_depth_mm + f...          5

pseudo_r2      aic accuracy
0  0.965802 25.613883 0.990991

```

Model 05: Flipper Length Interacting With Sex

```
fit_and_assess_logit('Model 05', formula_05, df)
```

	model_name	model_formula	num_coefs	pseudo_r2	\
0	Model 05	is_adelie ~ flipper_length_mm * C(sex)	4	0.482506	
		aic	accuracy		
0	244.275095	0.840841			

Compare Fitted Models - Training Data

Model 03 and Model 04 reach the highest accuracy with the fewest coefficients. Higher pseudo R-squared and lower AIC both indicate a better fit.

```
training_results = pd.concat([
    fit_and_assess_logit('Model 00', formula_00, df),
    fit_and_assess_logit('Model 01', formula_01, df),
    fit_and_assess_logit('Model 02', formula_02, df),
    fit_and_assess_logit('Model 03', formula_03, df),
    fit_and_assess_logit('Model 04', formula_04, df),
    fit_and_assess_logit('Model 05', formula_05, df)])
training_results.sort_values('accuracy', ascending=False)
```

```
/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/discrete/discrete_model.py:2385:
RuntimeWarning: overflow encountered in exp
    return 1/(1+np.exp(-X))
/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
    warnings.warn("Maximum Likelihood optimization failed to ")
/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/discrete/discrete_model.py:2385:
RuntimeWarning: overflow encountered in exp
    return 1/(1+np.exp(-X))
```

	model_name	model_formula	num_coefs	\
0	Model 03	is_adelie ~ bill_length_mm + bill_depth_mm + f...	5	
0	Model 02	is_adelie ~ bill_length_mm + bill_depth_mm + f...	4	
0	Model 04	is_adelie ~ bill_length_mm * bill_depth_mm + f...	5	
0	Model 05	is_adelie ~ flipper_length_mm * C(sex)	4	
0	Model 01	is_adelie ~ flipper_length_mm	2	
0	Model 00	is_adelie ~ 1	1	
		pseudo_r2	aic	accuracy
0	9.999997e-01	10.000157	1.000000	
0	9.658001e-01	23.614828	0.990991	
0	9.658022e-01	25.613883	0.990991	
0	4.825056e-01	244.275095	0.840841	
0	4.503986e-01	254.934320	0.822823	
0	7.149836e-13	458.575142	0.561562	

Fit Logistic Regression Models w/ Cross-Validation - Test Data

Functions

```
def logit_cross_val_score(model, formula, init_model, df, cv):
    """
    Run cross-validated evaluation for a logistic regression formula.

    Parameters:
        model (str): A human-readable model name.
        formula (str): A Patsy-compatible formula string.
        init_model (object): An unfitted scikit-learn estimator.
        df (pandas.DataFrame): The input data.
        cv (int): Number of cross-validation folds.

    Returns:
        pandas.DataFrame: Per-fold accuracy scores.
    """
    y, X = dmatrixes(formula, data=df, return_type='dataframe')
    y = np.ravel(y)
    scores = cross_val_score(init_model, X, y, cv=cv, scoring='accuracy')
    return pd.DataFrame({'model_name': model,
                        'fold': range(1, cv + 1),
                        'accuracy': scores})
```

Fit w/ Cross-Validation

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_models = [('Model 00', formula_00),
             ('Model 01', formula_01),
             ('Model 02', formula_02),
             ('Model 03', formula_03),
             ('Model 04', formula_04),
             ('Model 05', formula_05)]
cv_results = pd.concat([
    logit_cross_val_score(name, formula, LogisticRegression(max_iter=5000), df, 5)
    for name, formula in cv_models])
cv_results.head()
```

	model_name	fold	accuracy
0	Model 00	1	0.552239
1	Model 00	2	0.567164
2	Model 00	3	0.567164
3	Model 00	4	0.560606
4	Model 00	5	0.560606

Compare Fitted Models - Test Data

The cross-validated accuracy is the honest estimate. Model 03 is a strong choice: high accuracy with fewer coefficients than Model 04.

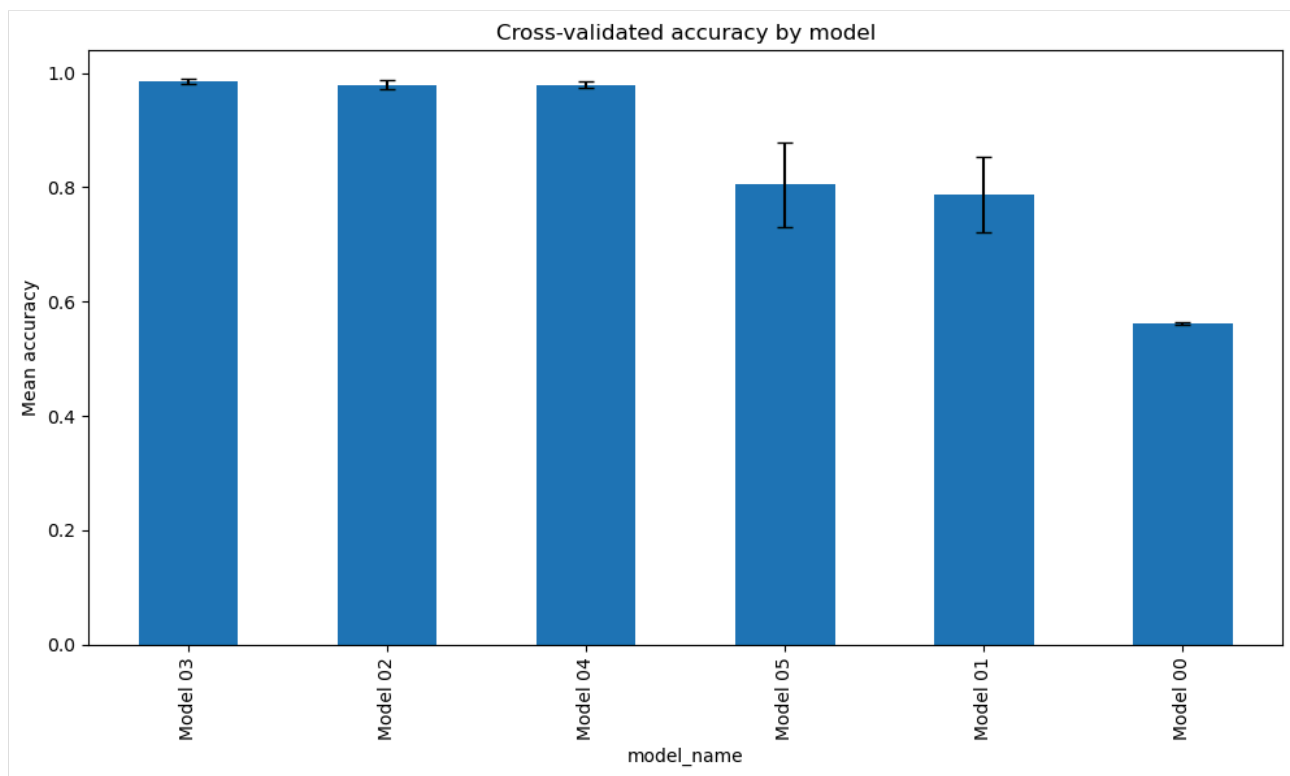
```
cv_summary = cv_results.groupby('model_name')['accuracy'].agg(['mean', 'std', 'count'])
cv_summary = cv_summary.sort_values('mean', ascending=False)
cv_summary
```

	model_name	mean	std	count
--	------------	------	-----	-------

Model 03	0.985029	0.010554	5
Model 02	0.979059	0.016998	5
Model 04	0.979059	0.013325	5
Model 05	0.805246	0.165472	5
Model 01	0.787291	0.148884	5
Model 00	0.561556	0.006155	5

Visualize Average Accuracy w/ 1 Standard Error Interval

```
ax = cv_summary['mean'].plot(
    kind='bar',
    yerr=cv_summary['std'] / np.sqrt(cv_summary['count']),
    capsize=4,
    figsize=(10, 6))
ax.set_ylabel('Mean accuracy')
ax.set_title('Cross-validated accuracy by model')
plt.tight_layout()
plt.show()
```

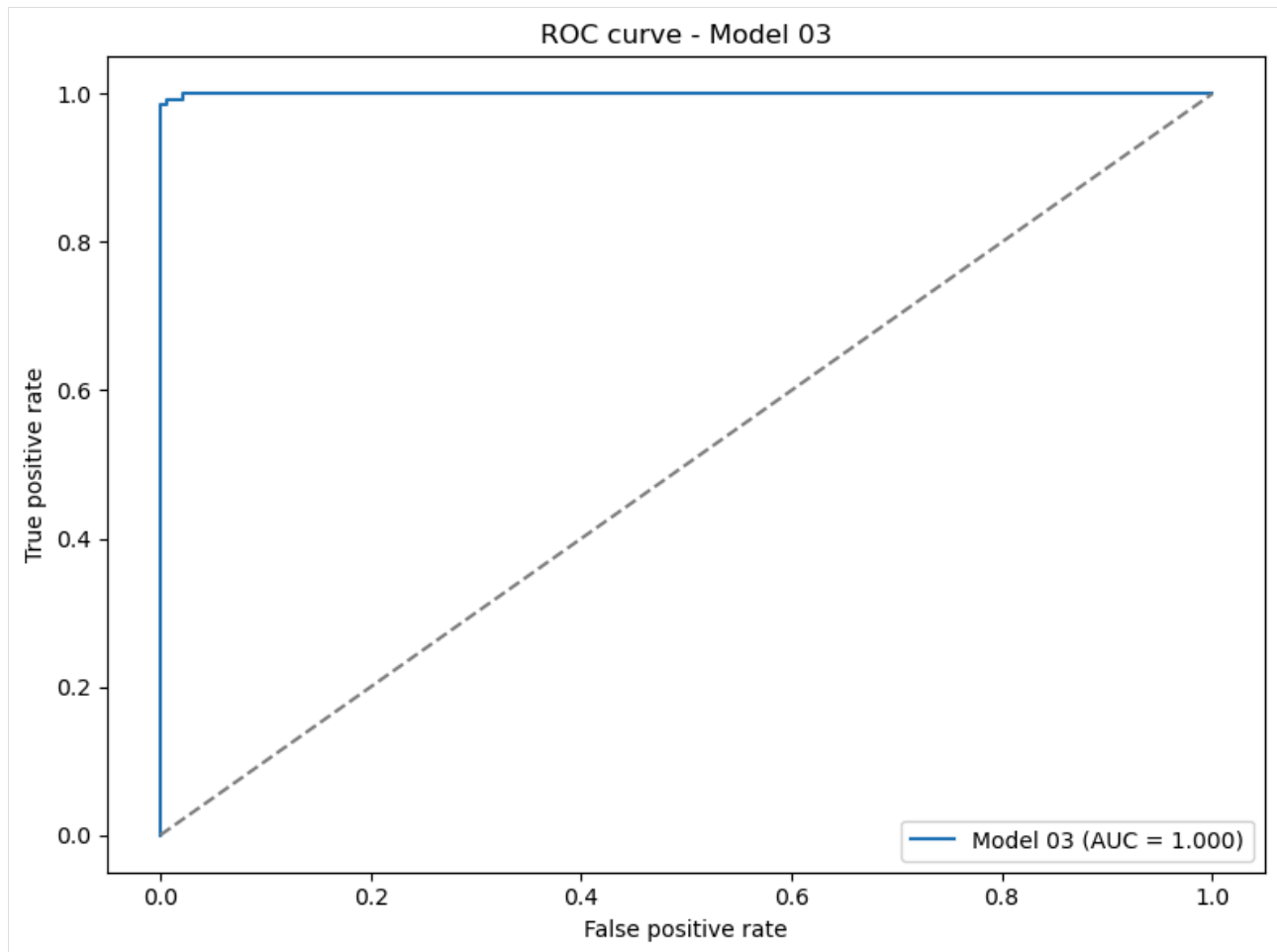


Visualize ROC Curves

A model with a high AUC ranks Adelie above non-Adelie across all thresholds. Model 03 is close to a perfect ranker.

```
y, X = dmatrices(formula_03, data=df, return_type='dataframe')
y = np.ravel(y)
best_model = LogisticRegression(max_iter=5000).fit(X, y)
probs = best_model.predict_proba(X)[:, 1]
fpr, tpr, _ = roc_curve(y, probs)
auc_score = roc_auc_score(y, probs)
plt.figure(figsize=(8, 6))
```

```
plt.plot(fpr, tpr, label=f'Model 03 (AUC = {auc_score:.3f})')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('ROC curve - Model 03')
plt.legend()
plt.tight_layout()
plt.show()
```



Confusion Matrix and Classification Report

```
preds = (probs >= 0.5).astype(int)
print(confusion_matrix(y, preds))
print(classification_report(y, preds))
```

```
[[186  1]
 [ 1 145]]
      precision    recall  f1-score   support

     0.0         0.99      0.99      0.99        187
     1.0         0.99      0.99      0.99        146

 accuracy                   0.99        333
```

macro avg	0.99	0.99	0.99	333
weighted avg	0.99	0.99	0.99	333

Save Best Model - Model 03 w/ Highest Cross-Validated Accuracy

Save Model

```
final_model = smf.logit(formula=formula_03, data=df).fit(dis=0)
with open('models/logit_model.pkl', 'wb') as handle:
    pickle.dump(final_model, handle)
print(final_model.summary())
```

Logit Regression Results

```
=====
Dep. Variable:          is_adelie    No. Observations:          333
Model:                  Logit        Df Residuals:                328
Method:                 MLE          Df Model:                  4
Date:                  Thu, 24 Sep 2026    Pseudo R-squ.:              1.000
Time:                  10:18:04          Log-Likelihood:             -7.8366e-05
converged:              False          LL-Null:                   -228.29
Covariance Type:       nonrobust        LLR p-value:                1.646e-97
=====
```

	coef	std err	z	P> z	[0.025	0.975]
Intercept	1059.3043	1.94e+08	5.46e-06	1.000	-3.8e+08	3.8e+08
C(sex)[T.Male]	117.0684	1.71e+07	6.85e-06	1.000	-3.35e+07	3.35e+07
bill_length_mm	-67.9876	7.2e+06	-9.44e-06	1.000	-1.41e+07	1.41e+07
bill_depth_mm	103.6875	1.74e+07	5.96e-06	1.000	-3.41e+07	3.41e+07
flipper_length_mm	-0.0532	8.7e+05	-6.12e-08	1.000	-1.71e+06	1.71e+06

```
=====
```

Complete Separation: The results show that there is complete separation or perfect prediction. In this case the Maximum Likelihood Estimator does not exist and the parameters are not identified.

```
/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/discrete/discrete_model.py:2385:
RuntimeWarning: overflow encountered in exp
    return 1/(1+np.exp(-X))
/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
    warnings.warn("Maximum Likelihood optimization failed to ")
/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/discrete/discrete_model.py:2385:
RuntimeWarning: overflow encountered in exp
    return 1/(1+np.exp(-X))
```

Load Model

```
with open('models/logit_model.pkl', 'rb') as handle:
    loaded_model = pickle.load(handle)
loaded_model.params
```

```
Intercept          1059.304311
C(sex)[T.Male]      117.068432
bill_length_mm      -67.987582
bill_depth_mm       103.687470
```

```
flipper_length_mm    -0.053247  
dtype: float64
```

Inference on New Data

Feed the loaded model a new penguin and read the probability that it is an Adelie.

```
new_penguin = pd.DataFrame({  
    'bill_length_mm': [39.0],  
    'bill_depth_mm': [18.5],  
    'flipper_length_mm': [190.0],  
    'sex': ['Female']})  
probability = loaded_model.predict(new_penguin).iloc[0]  
print(f'Probability of Adelie: {probability:.4f}')  
print('Predicted class:', 'Adelie' if probability >= 0.5 else 'Not Adelie')
```

```
Probability of Adelie: 1.0000  
Predicted class: Adelie
```

Key Takeaways

- Logistic regression models the probability of a binary outcome.
- Coefficients describe changes in log-odds; exponentiated, they are odds ratios.
- Accuracy is not enough; use the confusion matrix, ROC, and AUC.
- AUC measures ranking quality across all thresholds.
- Save the model and wrap prediction in a function.

Exercises

1. Fit the ladder of models and compare them on AIC and pseudo R-squared.
2. Interpret an odds ratio from the fitted model.
3. Compute the AUC and explain what it means to a non-specialist.
4. Choose a threshold other than 0.5 and describe how the confusion matrix changes.
5. Predict the probability that a new penguin is an Adelie.

Guided Walkthrough

The notebook turns the species target into a binary question and fits a ladder of logistic models with the statsmodels formula interface. Each model is scored on pseudo R-squared, AIC, and accuracy, and the coefficients are read as changes in log-odds. The evaluation section is where classification differs from regression: a confusion matrix, a classification report, and an ROC curve with its AUC. The AUC measures how well the model ranks positives above negatives across every threshold, which is more informative than accuracy at a single threshold.

The notebook also demonstrates a subtle trap. Predicting whether a penguin is a Gentoo is almost trivial because Gentoo are physically distinct, and a model that is too easy teaches nothing. Predicting Adelie is harder, with genuine overlap, so the accuracy is earned. This is a lesson about problem selection: a benchmark should be hard enough to be meaningful.

Common Pitfalls

- Using accuracy on imbalanced classes without checking per-class metrics.
- Interpreting a logistic coefficient as a probability change rather than a log-odds change.
- Reading the AUC as accuracy.

Going Further

- Exponentiate the coefficients to obtain odds ratios.
- Tune the decision threshold for the cost of a false negative versus a false positive.
- Add a regularized logistic regression and compare the coefficients.

Exercise Solutions

1. The ladder shows that adding features raises the pseudo R-squared and lowers the AIC up to a point.
2. An odds ratio above one means the feature increases the odds of the positive class.
3. The AUC is the probability that a random positive is ranked above a random negative.
4. Raising the threshold reduces false positives and increases false negatives.
5. The inference function returns a probability and a class label for the new penguin.

Chapter 16: Penguins Dataset - MLP

Introduction

The multilayer perceptron is the first neural network in this course and the foundation of everything that follows. It is a stack of linear layers separated by non-linear activations, trained by gradient descent. Where logistic regression draws a single boundary, a multilayer perceptron can bend boundaries to fit far more complex data. This chapter builds one from scratch in PyTorch and trains it to classify the three penguin species.

The chapter is deliberately close to the workflow of the previous chapters: load and clean the data, explore feature significance, split into train and test, fit a model, evaluate it, save it, and run inference. The difference is that the model is now learned by optimization rather than solved in closed form, which introduces training loops, loss functions, and optimizers.

What You Will Learn

- How to define a neural network as an `nn.Module` in PyTorch.
- How to build a custom `Dataset` and `DataLoader` for memory efficiency.
- How to write a training loop: zero gradients, forward, loss, backward, step.
- How to move models and data to the GPU with the MPS backend.
- How to evaluate, save, load, and run inference with a trained network.

Overview

We standardize the three continuous measurements, because neural networks train best when inputs have comparable scale, and we confirm with a statistical test that all three features carry signal. We then define a two-hidden-layer network, wrap the data in a custom dataset that converts arrays to tensors on demand, and feed it through a data loader that batches and shuffles.

The training loop is the heartbeat of deep learning, and we write it explicitly rather than hiding it in a library. Each epoch makes a pass over the data, computes a loss, backpropagates gradients, and updates the weights. We track the loss and accuracy on a held-out set so we can watch learning happen. On an Apple Silicon Mac the model and batches live on the MPS device, which makes training fast.

After training we evaluate on the test set, print a confusion matrix and classification report, save the model's parameters, reload them into a fresh model, and classify a new penguin. Every later chapter reuses this exact skeleton.

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy pandas matplotlib seaborn scikit-learn
```

Import Libraries

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.feature_selection import f_classif
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x115b6db30>
```

Parameters

```
IN_FEATURES = 3
IN_FEATURES
```

```
3
```

```
H1 = 16
H1
```

```
16
```

```
H2 = 16
H2
```

```
16
```

```
OUT_FEATURES = 3
OUT_FEATURES
```


REVERSE ENGINEERING DATA

```
3
```

```
TEST_SIZE = 0.3  
TEST_SIZE
```

```
0.3
```

```
MODE = "classification" # "classification" or "regression"  
MODE
```

```
'classification'
```

Hyperparameters

```
LEARNING_RATE = 0.01  
LEARNING_RATE
```

```
0.01
```

```
EPOCHS = 200  
EPOCHS
```

```
200
```

```
BATCH_SIZE = 16  
BATCH_SIZE
```

```
16
```

```
DROPOUT = 0.0  
DROPOUT
```

```
0.0
```

Load Dataset

```
df = pd.read_csv('data/penguins-clean.csv')
```

```
df.head()
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	\
0	Adelie	Torgersen	39.1	18.7	181.0	
1	Adelie	Torgersen	39.5	17.4	186.0	
2	Adelie	Torgersen	40.3	18.0	195.0	
3	Adelie	Torgersen	36.7	19.3	193.0	
4	Adelie	Torgersen	39.3	20.6	190.0	

	body_mass_g	sex
0	3750.0	Male
1	3800.0	Female
2	3250.0	Female
3	3450.0	Female
4	3650.0	Male

Clean Dataset

Map the three species to integers, keep the three continuous measurements, and standardize them so the network trains evenly.

```
SPECIES_MAP = {'Adelie': 0, 'Chinstrap': 1, 'Gentoo': 2}
df['species_id'] = df['species'].map(SPECIES_MAP)
FEATURES = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm']
X = df[FEATURES].to_numpy(dtype=np.float32)
y = df['species_id'].to_numpy(dtype=np.int64)
X_MEAN = X.mean(axis=0)
X_STD = X.std(axis=0)
X = (X - X_MEAN) / X_STD
X.shape, y.shape
```

```
((333, 3), (333,))
```

Feature Statistical Significance (P-Values)

All three features are statistically significant. `flipper_length_mm` has the strongest separation between species.

```
f_values, p_values = f_classif(X, y)
pd.DataFrame({'feature': FEATURES,
              'f_value': f_values,
              'p_value': p_values}).sort_values('p_value')
```

	feature	f_value	p_value
0	bill_length_mm	397.299805	0.0
1	bill_depth_mm	344.824921	0.0
2	flipper_length_mm	567.408508	0.0

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Create Model

```
class Model(nn.Module):
    """
    A feedforward neural network with two hidden layers and optional dropout.
    """

    def __init__(self, in_features=IN_FEATURES, h1=H1, h2=H2,
                  out_features=OUT_FEATURES, dropout=DROPOUT):
        """
        Initialize the neural network layers.

        Parameters:
            in_features (int): Number of input features.
            h1 (int): Neurons in the first hidden layer.
            h2 (int): Neurons in the second hidden layer.
            out_features (int): Number of output features.
            dropout (float): Dropout rate (0.0 = no dropout).

        Returns:
            None
        """
        super(Model, self).__init__()
        self.fc1 = nn.Linear(in_features, h1)
        self.fc2 = nn.Linear(h1, h2)
        self.out = nn.Linear(h2, out_features)
        self.dropout = nn.Dropout(p=dropout)

    def forward(self, x):
        """
        Run the forward pass of the network.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output logits.
        """
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = F.relu(self.fc2(x))
        x = self.dropout(x)
        return self.out(x)
```

Instantiate Model

```
torch.manual_seed(SEED)
model = Model().to(DEVICE)
model
```

```
Model(
  (fc1): Linear(in_features=3, out_features=16, bias=True)
  (fc2): Linear(in_features=16, out_features=16, bias=True)
  (out): Linear(in_features=16, out_features=3, bias=True)
  (dropout): Dropout(p=0.0, inplace=False)
)
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=SEED, stratify=y)
X_train.shape, X_test.shape
```

```
((233, 3), (100, 3))
```

Create Custom Dataset (Memory Efficient)

```
class NumpyDataset(torch.utils.data.Dataset):
    """
    A memory-efficient dataset that stores data as NumPy arrays.
    """

    def __init__(self, X, y, device, mode="classification"):
        """
        Initialize the dataset with NumPy arrays.

        Parameters:
            X (np.ndarray): Feature array.
            y (np.ndarray): Label array.
            device (torch.device): Device to move tensors to.
            mode (str): "classification" or "regression".

        Returns:
            None
        """
        self.X = X
        self.y = y
        self.device = device
        self.mode = mode

    def __len__(self):
        """
        Return the number of samples in the dataset.

        Parameters:
            None

        Returns:
            int: Number of samples.
        """
        return len(self.X)

    def __getitem__(self, idx):
        """
        Return a single sample as tensors on the device.

        Parameters:
            idx (int): Index of the sample.

        Returns:
            tuple: Feature and label tensors.
        """
        X_tensor = torch.tensor(self.X[idx]).float().to(self.device)
        if self.mode == "classification":
            y_tensor = torch.tensor(self.y[idx]).long().to(self.device)
        else:
```

```
y_tensor = torch.tensor(self.y[idx]).float().to(self.device)
return X_tensor, y_tensor
```

Create DataLoaders

```
train_dataset = NumpyDataset(X_train, y_train, DEVICE, MODE)
test_dataset = NumpyDataset(X_test, y_test, DEVICE, MODE)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(test_loader)
```

```
(15, 7)
```

Create Loss Function & Optimizer

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss for the epoch.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)

def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
```

```
        tuple: Mean loss and accuracy.
    """
    model.eval()
    total, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            logits = model(x)
            total += loss_fn(logits, y).item() * len(y)
            correct += (logits.argmax(1) == y).sum().item()
    n = len(loader.dataset)
    return total / n, correct / n
```

Training Loop

```
history = {'loss': [], 'accuracy': []}
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    test_loss, test_accuracy = evaluate(model, test_loader, loss_fn)
    history['loss'].append(test_loss)
    history['accuracy'].append(test_accuracy)
    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch + 1:3d} | loss {test_loss:.4f} | accuracy {test_accuracy:.4f}")
```

Epoch 20 | loss 0.0159 | accuracy 0.9900

Epoch 40 | loss 0.0169 | accuracy 0.9900

Epoch 60 | loss 0.0232 | accuracy 0.9900

Epoch 80 | loss 0.0270 | accuracy 0.9900

Epoch 100 | loss 0.0295 | accuracy 0.9900

Epoch 120 | loss 0.0228 | accuracy 0.9900

Epoch 140 | loss 0.0357 | accuracy 0.9900

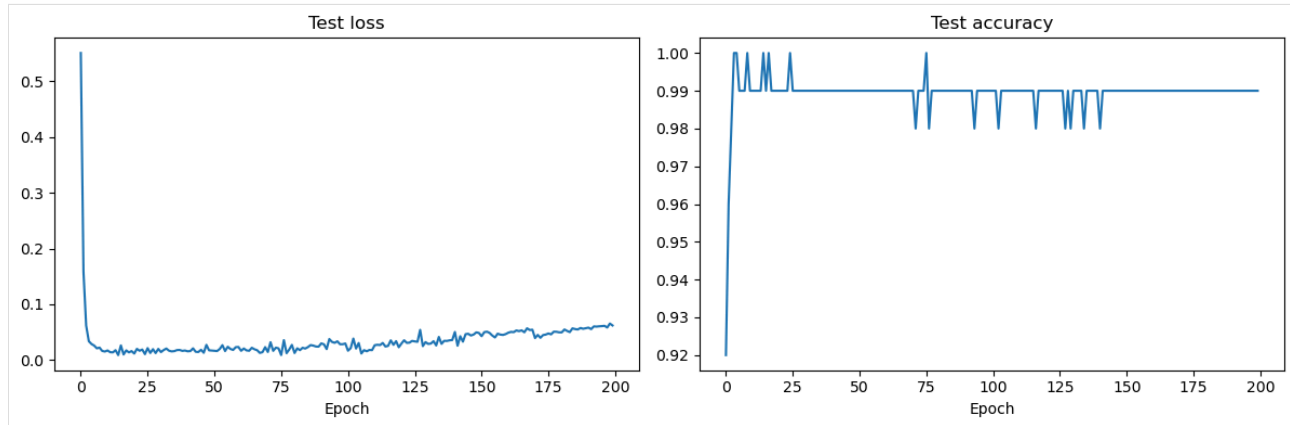
Epoch 160 | loss 0.0464 | accuracy 0.9900

Epoch 180 | loss 0.0495 | accuracy 0.9900

Epoch 200 | loss 0.0619 | accuracy 0.9900

Visualize Training

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(history['loss'])
axes[0].set_title('Test loss')
axes[0].set_xlabel('Epoch')
axes[1].plot(history['accuracy'])
axes[1].set_title('Test accuracy')
axes[1].set_xlabel('Epoch')
fig.tight_layout()
plt.show()
```



Evaluate Model

```
_, accuracy = evaluate(model, test_loader, loss_fn)
print(f"Test accuracy: {accuracy:.4f}")
```

Test accuracy: 0.9900

```
all_preds = []
all_labels = []
model.eval()
with torch.no_grad():
    for x, y in test_loader:
        all_preds.append(model(x).argmax(1).cpu().numpy())
        all_labels.append(y.cpu().numpy())
all_preds = np.concatenate(all_preds)
all_labels = np.concatenate(all_labels)
print(confusion_matrix(all_labels, all_preds))
print(classification_report(all_labels, all_preds, target_names=list(SPECIES_MAP)))
```

```
[[43  1  0]
 [ 0 20  0]
 [ 0  0 36]]
      precision    recall  f1-score   support

   Adelie         1.00      0.98      0.99         44
  Chinstrap        0.95      1.00      0.98         20
     Gentoo        1.00      1.00      1.00         36
```

accuracy			0.99	100
macro avg	0.98	0.99	0.99	100
weighted avg	0.99	0.99	0.99	100

Save Model

```
torch.save(model.state_dict(), 'models/mlp_penguins.pt')
print('saved mlp_penguins.pt')
```

saved mlp_penguins.pt

Load Model

```
loaded_model = Model().to(DEVICE)
loaded_model.load_state_dict(torch.load('models/mlp_penguins.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded mlp_penguins.pt')
```

loaded mlp_penguins.pt

/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_76916/2132460299.py:2: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
loaded_model.load_state_dict(torch.load('models/mlp_penguins.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, features):
    """
    Predict the species for one standardized measurement.

    Parameters:
        model (nn.Module): Trained model.
        features (list): Raw bill length, bill depth, and flipper length.

    Returns:
        tuple: Predicted species name and confidence.
    """
    model.eval()
    scaled = (np.array(features, dtype=np.float32) - X_MEAN) / X_STD
    with torch.no_grad():
        X_new = torch.tensor(scaled).float().to(DEVICE).unsqueeze(0)
        probs = torch.softmax(model(X_new), dim=1)
```



```
confidence, predicted = probs.max(dim=1)
return list(SPECIES_MAP[predicted.item()], confidence.item())
```

Run Inference

```
species, confidence = predict(loader_model, [47.5, 15.0, 217.0])
print(f"Predicted species: {species} ({confidence:.4f})")
```

```
Predicted species: Gentoo (1.0000)
```

Key Takeaways

- A multilayer perceptron is linear layers plus non-linear activations.
- Standardize inputs before training.
- The training loop is zero, forward, loss, backward, step.
- A custom dataset and data loader make training memory efficient.
- Save and reload model parameters to reuse a trained network.

Exercises

1. Change the number of hidden units and compare test accuracy.
2. Add dropout and observe its effect.
3. Train on the CPU and on MPS and compare the time.
4. Plot the training and test loss over epochs.
5. Save the model, reload it, and classify a new penguin.

Guided Walkthrough

The notebook builds the first neural network in the course. It standardizes the inputs, confirms feature significance with a statistical test, defines a two-hidden-layer module, and wraps the data in a custom dataset that converts arrays to tensors on demand. The training loop is written explicitly: zero the gradients, forward pass, compute the loss, backpropagate, and step the optimizer. On an Apple Silicon Mac the model and batches live on the MPS device.

The evaluation section prints a confusion matrix and a classification report, then saves the model's parameters, reloads them into a fresh model, and classifies a new penguin. Every later chapter reuses this skeleton, so this is the chapter to master.

Common Pitfalls

- Forgetting to standardize the inputs, which slows or destabilizes training.
- Forgetting `model.eval()` before evaluation.
- Forgetting to move batches to the device inside the loop.

Going Further

- Add dropout and observe its effect on the train and test gap.
- Compare the Adam and SGD optimizers.

- Plot the loss curve and find the epoch where the model starts to overfit.

Exercise Solutions

1. More hidden units improve accuracy up to a point, after which the model overfits.
2. Dropout widens the train and test gap less, improving generalization.
3. MPS is faster than CPU for this model, as the timing shows.
4. The loss curve falls and then flattens; a rising test loss marks overfitting.
5. The reloaded model produces the same predictions as the original.

Chapter 17: Fashion-MNIST Dataset - CNN

Introduction

Convolutional neural networks changed computer vision by encoding two ideas about images: nearby pixels belong together, and a feature is useful wherever it appears. A convolution slides a small filter across the image, reusing the same weights everywhere, which makes the network far more parameter-efficient than a dense network and gives it translation invariance. This chapter trains a convolutional network on Fashion-MNIST, a real dataset of clothing images.

The chapter also introduces the image data pipeline: loading a dataset with torchvision, normalizing the pixels, batching images, and interpreting the tensor shapes. Getting the shapes right is half the battle in vision, and we make the shapes explicit at every step.

What You Will Learn

- How a convolution differs from a dense layer and why that matters.
- How to load and normalize a real image dataset with torchvision.
- How to build a convolutional network with convolution, ReLU, and pooling.
- How to train and evaluate an image classifier.
- How to inspect a confusion matrix to find which classes are confused.

Overview

We begin by loading Fashion-MNIST, which torchvision downloads once and caches, and normalizing the grayscale images. We visualize a sample to confirm the data is what we expect. The network is a stack of convolution, activation, and pooling blocks followed by a flatten and a small classifier. The convolutions learn local features; the pooling reduces spatial size; the dense head makes the final decision.

We train for a few epochs, tracking test accuracy, and then evaluate with a confusion matrix and a per-class report. The confusion matrix is especially instructive on Fashion-MNIST, because some categories, such as shirts and pullovers, genuinely resemble one another, and the matrix shows the network making the same mistakes a person might.

Citation

[1] Han Xiao, Kashif Rasul, Roland Vollgraf, <https://github.com/zalandoresearch/fashion-mnist>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy pandas matplotlib scikit-learn
```

Import Libraries

```
import numpy as np
import torch
```

```
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x117249b30>
```

Parameters

```
IN_CHANNELS = 1
IN_CHANNELS
```

```
1
```

```
NUM_CLASSES = 10
NUM_CLASSES
```

```
10
```

```
IMAGE_SIZE = 28
IMAGE_SIZE
```

```
28
```

```
CLASS_NAMES = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
                'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
CLASS_NAMES
```

```
['T-shirt/top',
 'Trouser',
 'Pullover',
```

```
'Dress',  
'Coat',  
'Sandal',  
'Shirt',  
'Sneaker',  
'Bag',  
'Ankle boot']
```

Hyperparameters

```
LEARNING_RATE = 0.001  
LEARNING_RATE
```

```
0.001
```

```
EPOCHS = 5  
EPOCHS
```

```
5
```

```
BATCH_SIZE = 128  
BATCH_SIZE
```

```
128
```

Device

```
DEVICE = torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu")  
DEVICE
```

```
device(type='mps')
```

Load Dataset

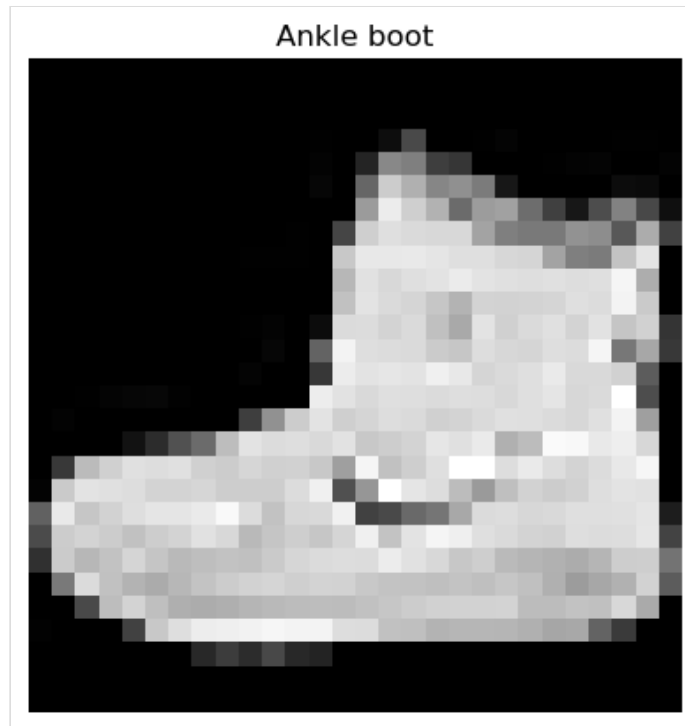
Fashion-MNIST is a real dataset of 70,000 grayscale clothing images. torchvision downloads it once into `data/` and normalizes each image.

```
transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize((0.2860,), (0.3530,))])  
train_data = datasets.FashionMNIST('data', train=True, download=True, transform=transform)  
test_data = datasets.FashionMNIST('data', train=False, download=True, transform=transform)  
len(train_data), len(test_data)
```

```
(60000, 10000)
```

Visualize a Sample

```
image, label = train_data[0]
plt.imshow(image.squeeze(), cmap='gray')
plt.title(CLASS_NAMES[label])
plt.axis('off')
plt.show()
```



Create Model

```
class CNN(nn.Module):
    """
    A small convolutional neural network for 28x28 grayscale images.
    """

    def __init__(self, num_classes=NUM_CLASSES):
        """
        Initialize the convolutional and dense layers.

        Parameters:
            num_classes (int): Number of output classes.

        Returns:
            None
        """
        super(CNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
```

```

        nn.Conv2d(16, 32, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(2))
    self.classifier = nn.Sequential(
        nn.Flatten(),
        nn.Linear(32 * 7 * 7, 128),
        nn.ReLU(),
        nn.Linear(128, num_classes))

    def forward(self, x):
        """
        Run the forward pass of the network.

        Parameters:
            x (torch.Tensor): Batch of images.

        Returns:
            torch.Tensor: Output logits.
        """
        x = self.features(x)
        return self.classifier(x)

```

Instantiate Model

```

torch.manual_seed(SEED)
model = CNN().to(DEVICE)
model

```

```

CNN(
  (features): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=1568, out_features=128, bias=True)
    (2): ReLU()
    (3): Linear(in_features=128, out_features=10, bias=True)
  )
)

```

Create DataLoaders

```

train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(test_loader)

```

```

(469, 79)

```

Create Loss Function & Optimizer

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss for the epoch.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)


def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        tuple: Mean loss and accuracy.
    """
    model.eval()
    total, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            total += loss_fn(logits, y).item() * len(y)
            correct += (logits.argmax(1) == y).sum().item()
    n = len(loader.dataset)
    return total / n, correct / n
```


Training Loop

```
history = {'loss': [], 'accuracy': []}
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    test_loss, test_accuracy = evaluate(model, test_loader, loss_fn)
    history['loss'].append(test_loss)
    history['accuracy'].append(test_accuracy)
    print(f"Epoch {epoch + 1} | train loss {train_loss:.4f} | test accuracy {test_accuracy:.4f}")
```

Epoch 1 | train loss 0.4970 | test accuracy 0.8677

Epoch 2 | train loss 0.3266 | test accuracy 0.8844

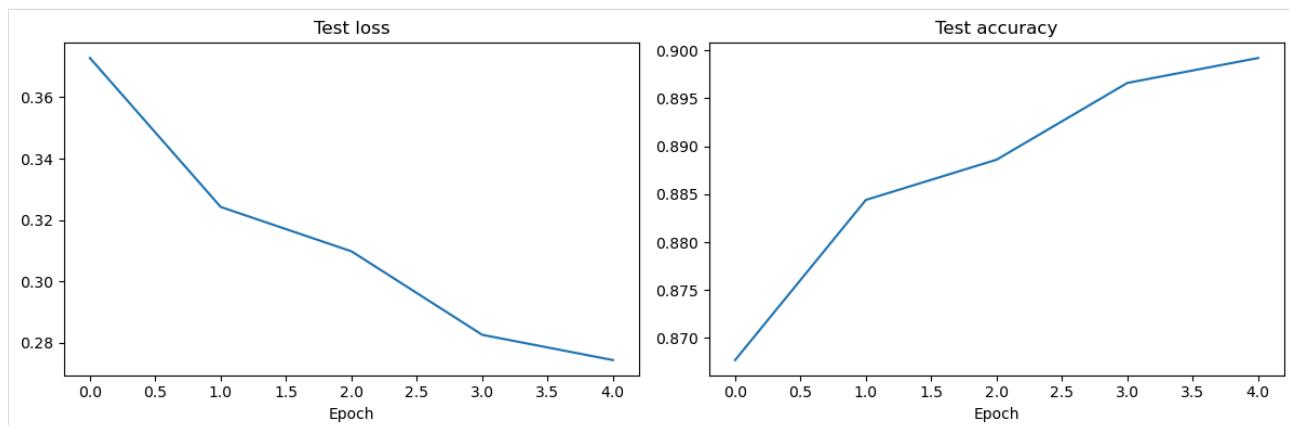
Epoch 3 | train loss 0.2806 | test accuracy 0.8886

Epoch 4 | train loss 0.2516 | test accuracy 0.8966

Epoch 5 | train loss 0.2316 | test accuracy 0.8992

Visualize Training

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(history['loss'])
axes[0].set_title('Test loss')
axes[0].set_xlabel('Epoch')
axes[1].plot(history['accuracy'])
axes[1].set_title('Test accuracy')
axes[1].set_xlabel('Epoch')
fig.tight_layout()
plt.show()
```



Evaluate Model

```
_, accuracy = evaluate(model, test_loader, loss_fn)
print(f"Test accuracy: {accuracy:.4f}")
```

Test accuracy: 0.8992

Confusion Matrix and Classification Report

```
all_preds = []
all_labels = []
model.eval()
with torch.no_grad():
    for x, y in test_loader:
        x = x.to(DEVICE)
        all_preds.append(model(x).argmax(1).cpu().numpy())
        all_labels.append(y.numpy())
all_preds = np.concatenate(all_preds)
all_labels = np.concatenate(all_labels)
print(confusion_matrix(all_labels, all_preds))
print(classification_report(all_labels, all_preds, target_names=CLASS_NAMES))
```

```
[[776  0 11 16  6  1 185  0  5  0]
 [ 1 982  0 10  4  0  2  0  1  0]
 [ 12  1 773  4 119  0  91  0  0  0]
 [  7  6 12 876 50  0 48  0  1  0]
 [  1  1 20  9 919  0 50  0  0  0]
 [  0  0  0  0  0 967  0 20  0 13]
 [ 52  1 36 21 90  0 798  0  2  0]
 [  0  0  0  0  0  9  0 951  0 40]
 [  4  2  2  3  3  1  6  4 975  0]
 [  0  0  0  0  0  4  1 20  0 975]]
```

	precision	recall	f1-score	support
T-shirt/top	0.91	0.78	0.84	1000
Trouser	0.99	0.98	0.99	1000
Pullover	0.91	0.77	0.83	1000
Dress	0.93	0.88	0.90	1000
Coat	0.77	0.92	0.84	1000
Sandal	0.98	0.97	0.98	1000
Shirt	0.68	0.80	0.73	1000
Sneaker	0.96	0.95	0.95	1000
Bag	0.99	0.97	0.98	1000
Ankle boot	0.95	0.97	0.96	1000
accuracy			0.90	10000
macro avg	0.91	0.90	0.90	10000
weighted avg	0.91	0.90	0.90	10000

Save Model

```
torch.save(model.state_dict(), 'models/cnn_fashion_mnist.pt')
print('saved cnn_fashion_mnist.pt')
```

saved cnn_fashion_mnist.pt

Load Model

```
loaded_model = CNN().to(DEVICE)
loaded_model.load_state_dict(torch.load('models/cnn_fashion_mnist.pt', map_location=DEVICE))
```

```
loaded_model.eval()
print('loaded cnn_fashion_mnist.pt')
```

```
loaded cnn_fashion_mnist.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_76937/1572054108.py:2: FutureWarning: You
are using `torch.load` with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data which will execute
arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a
future release, the default value for `weights_only` will be flipped to `True`. This limits the
functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use
case where you don't have full control of the loaded file. Please open an issue on GitHub for any
issues related to this experimental feature.
loaded_model.load_state_dict(torch.load('models/cnn_fashion_mnist.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, image):
    """
    Predict the class of one normalized image tensor.

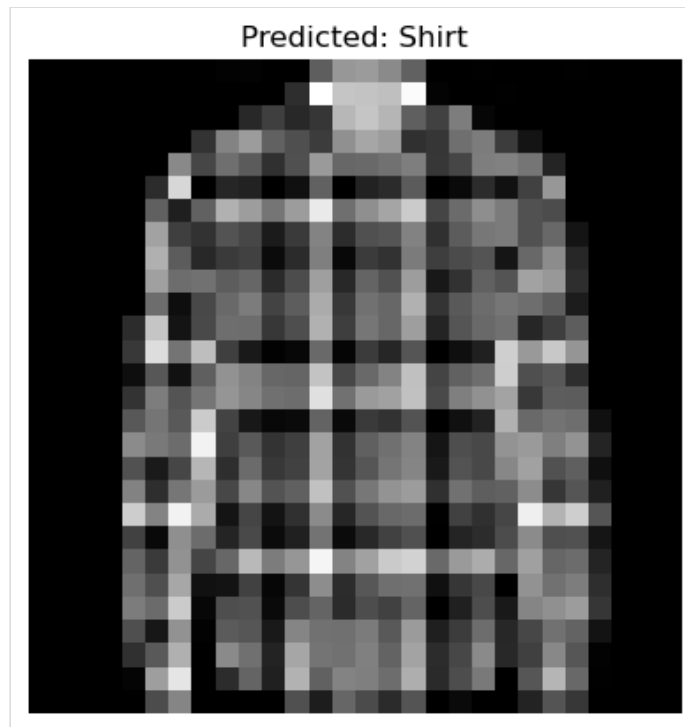
    Parameters:
        model (nn.Module): Trained model.
        image (torch.Tensor): A 1x28x28 image tensor.

    Returns:
        tuple: Predicted class name and confidence.
    """
    model.eval()
    with torch.no_grad():
        logits = model(image.unsqueeze(0).to(DEVICE))
        probs = torch.softmax(logits, dim=1)
        confidence, predicted = probs.max(dim=1)
    return CLASS_NAMES[predicted.item()], confidence.item()
```

Run Inference

```
image, label = test_data[7]
name, confidence = predict(loaded_model, image)
print(f"True: {CLASS_NAMES[label]} | Predicted: {name} ({confidence:.4f})")
plt.imshow(image.squeeze(), cmap='gray')
plt.title(f"Predicted: {name}")
plt.axis('off')
plt.show()
```

```
True: Shirt | Predicted: Shirt (0.9477)
```



Key Takeaways

- Convolutions exploit locality and translation invariance.
- Image tensors have shape batch, channels, height, width.
- Normalize images with the dataset's statistics.
- Pooling reduces spatial size and computation.
- Confusion matrices reveal which classes are genuinely hard.

Exercises

1. Compute the output shape of a convolution by hand and verify it.
2. Add a third convolutional block and adjust the classifier.
3. Compare a convolutional network to a dense network on the same images.
4. Inspect the confusion matrix and name the two most confused classes.
5. Normalize with incorrect statistics and observe the effect on accuracy.

Guided Walkthrough

The notebook loads Fashion-MNIST, normalizes the images, and visualizes a sample to confirm the pipeline. The network is a stack of convolution, activation, and pooling blocks followed by a flatten and a small dense head. The convolutions learn local features, the pooling reduces spatial size, and the dense head makes the final decision.

Training tracks test accuracy, and the evaluation section prints a confusion matrix and a per-class report. The confusion matrix is instructive because some clothing categories genuinely resemble one another, and the network makes the same mistakes a person would. The chapter closes by saving and reloading the model and classifying a test image.

Common Pitfalls

- Feeding unnormalized images, which slows convergence.
- Getting the tensor shape wrong; images are batch, channels, height, width.
- Interpreting overall accuracy without looking at the per-class report.

Going Further

- Add a third convolutional block and compare accuracy.
- Replace max pooling with average pooling.
- Compare the convolutional model to a dense model on the same images.

Exercise Solutions

1. With kernel five, padding two, and stride one, the spatial size is unchanged.
2. A third block increases capacity and usually accuracy, but costs time.
3. The convolutional model uses far fewer parameters and generalizes better.
4. Shirts and pullovers are typically the most confused pair.
5. Wrong normalization statistics lower accuracy, especially early in training.

Part VI: Sequences, Attention, and Language Models

Chapter 18: Tiny Shakespeare Dataset - RNN

Introduction

Sequences are data with an order, and order carries meaning. A recurrent neural network processes a sequence one step at a time, carrying a hidden state that summarizes what it has seen. This chapter builds a character-level language model on the Tiny Shakespeare corpus, a real text of roughly one million characters, and trains it to predict the next character. By the end it generates Shakespeare-like text one character at a time.

The chapter is a bridge from fixed-size inputs to variable-length ones, and it introduces the vocabulary machinery that every language model needs: mapping characters to integers, encoding text into tensors, and sampling from a probability distribution to generate new text.

What You Will Learn

- How a recurrent network carries state across a sequence.
- How to build a character vocabulary and encode text.
- How to create fixed-length training windows from a long sequence.
- How to train an LSTM language model and generate text.
- Why temperature controls the creativity of generated text.

Overview

We download Tiny Shakespeare, build a character vocabulary, and encode the entire corpus as a tensor of integers. A dataset then yields fixed-length windows of characters as input and the same window shifted by one as the target, which turns next-character prediction into supervised learning.

The model embeds each character into a vector, runs the sequence through an LSTM, and projects the final hidden state at each step to a score over the vocabulary. Training minimizes cross-entropy between the predicted distribution and the true next character. Generation reverses the process: start with a seed string, predict the next character, append it, and repeat. The temperature parameter divides the logits before the softmax, sharpening the distribution toward safe predictions or flattening it toward surprising ones.

Citation

[1] Andrej Karpathy, <https://github.com/karpathy/char-rnn>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib
```

Import Libraries

```
import urllib.request
import numpy as np
import torch
```

```
import torch.nn as nn
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x11c749b10>
```

Parameters

```
SEQ_LEN = 64
SEQ_LEN
```

```
64
```

```
EMBED_DIM = 64
EMBED_DIM
```

```
64
```

```
HIDDEN_DIM = 128
HIDDEN_DIM
```

```
128
```

Hyperparameters

```
LEARNING_RATE = 0.002
LEARNING_RATE
```

```
0.002
```

```
EPOCHS = 10
EPOCHS
```



```
10
```

```
BATCH_SIZE = 128
BATCH_SIZE
```

```
128
```

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Download Dataset

Tiny Shakespeare is a real text corpus of about one million characters. We download it once and train a character-level model to predict the next character.

```
URL = 'https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt'
urllib.request.urlretrieve(URL, 'data/tinyshakespeare.txt')
with open('data/tinyshakespeare.txt', encoding='utf-8') as handle:
    text = handle.read()
print('length:', len(text))
print(text[:200])
```

```
length: 1115394
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You are all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you
```

Build Vocabulary

Functions

```
def build_vocab(text):
    """
    Build character to index and index to character maps.
```

```

Parameters:
    text (str): The training text.

Returns:
    tuple: Character list, string to index, index to string.
"""
chars = sorted(set(text))
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for i, ch in enumerate(chars)}
return chars, stoi, itos

def encode(text, stoi):
    """
    Encode text as a tensor of indices.

    Parameters:
        text (str): The text to encode.
        stoi (dict): Character to index map.

    Returns:
        torch.Tensor: Encoded indices.
    """
    return torch.tensor([stoi[ch] for ch in text], dtype=torch.long)

def decode(indices, itos):
    """
    Decode indices back to text.

    Parameters:
        indices (list): Index values.
        itos (dict): Index to character map.

    Returns:
        str: The decoded text.
    """
    return ''.join(itos[i] for i in indices)

```

Run

```

CHARS, STOI, ITOS = build_vocab(text)
VOCAB_SIZE = len(CHARS)
data = encode(text, STOI)
print('vocab size:', VOCAB_SIZE)
print('data shape:', data.shape)

```

```

vocab size: 65
data shape: torch.Size([1115394])

```

Create Dataset

```

class CharDataset(torch.utils.data.Dataset):
    """
    A dataset of fixed-length character windows and next-character targets.
    """

```

```

def __init__(self, data, seq_len):
    """
    Store the encoded text and window length.

    Parameters:
        data (torch.Tensor): Encoded text.
        seq_len (int): Length of each input window.

    Returns:
        None
    """
    self.data = data
    self.seq_len = seq_len

def __len__(self):
    """
    Return the number of windows.

    Parameters:
        None

    Returns:
        int: Number of windows.
    """
    return len(self.data) - self.seq_len

def __getitem__(self, idx):
    """
    Return one input window and its shifted target.

    Parameters:
        idx (int): Window index.

    Returns:
        tuple: Input and target tensors.
    """
    x = self.data[idx:idx + self.seq_len]
    y = self.data[idx + 1:idx + self.seq_len + 1]
    return x, y

```

Create DataLoader

```

split = int(0.9 * len(data))
train_data = CharDataset(data[:split], SEQ_LEN)
val_data = CharDataset(data[split:], SEQ_LEN)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(val_loader)

```

```
(7843, 871)
```

Create Model

```

class RNNModel(nn.Module):
    """
    A character-level recurrent network with an LSTM.
    """

```

```
def __init__(self, vocab_size, embed_dim, hidden_dim):
    """
    Initialize the embedding, LSTM, and output layer.

    Parameters:
        vocab_size (int): Number of characters.
        embed_dim (int): Embedding size.
        hidden_dim (int): LSTM hidden size.

    Returns:
        None
    """
    super(RNNModel, self).__init__()
    self.embed = nn.Embedding(vocab_size, embed_dim)
    self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
    self.fc = nn.Linear(hidden_dim, vocab_size)

def forward(self, x, hidden=None):
    """
    Run the forward pass over a batch of windows.

    Parameters:
        x (torch.Tensor): Input index batch.
        hidden (tuple): Optional LSTM hidden state.

    Returns:
        tuple: Logits and the new hidden state.
    """
    x = self.embed(x)
    out, hidden = self.lstm(x, hidden)
    return self.fc(out), hidden
```

Instantiate Model

```
torch.manual_seed(SEED)
model = RNNModel(VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM).to(DEVICE)
model
```

```
RNNModel(
  (embed): Embedding(65, 64)
  (lstm): LSTM(64, 128, batch_first=True)
  (fc): Linear(in_features=128, out_features=65, bias=True)
)
```

Create Loss Function & Optimizer

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.
```

```

Parameters:
    model (nn.Module): The model to train.
    loader (DataLoader): Training data loader.
    loss_fn (nn.Module): Loss function.
    optimizer (torch.optim.Optimizer): Parameter update rule.

Returns:
    float: Mean training loss.
"""
model.train()
total = 0.0
for x, y in loader:
    x, y = x.to(DEVICE), y.to(DEVICE)
    optimizer.zero_grad()
    logits, _ = model(x)
    loss = loss_fn(logits.transpose(1, 2), y)
    loss.backward()
    optimizer.step()
    total += loss.item() * len(y)
return total / len(loader.dataset)

def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        float: Mean loss.
    """
    model.eval()
    total = 0.0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits, _ = model(x)
            total += loss_fn(logits.transpose(1, 2), y).item() * len(y)
    return total / len(loader.dataset)

```

Training Loop

```

history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    val_loss = evaluate(model, val_loader, loss_fn)
    history.append(val_loss)
    print(f"Epoch {epoch + 1:2d} | train loss {train_loss:.4f} | val loss {val_loss:.4f}")

```

```
Epoch 1 | train loss 1.4800 | val loss 1.6205
```

```
Epoch 2 | train loss 1.3362 | val loss 1.6312
```

Epoch 3 | train loss 1.3116 | val loss 1.6426

Epoch 4 | train loss 1.2991 | val loss 1.6548

Epoch 5 | train loss 1.2916 | val loss 1.6687

Epoch 6 | train loss 1.2864 | val loss 1.6741

Epoch 7 | train loss 1.2829 | val loss 1.6755

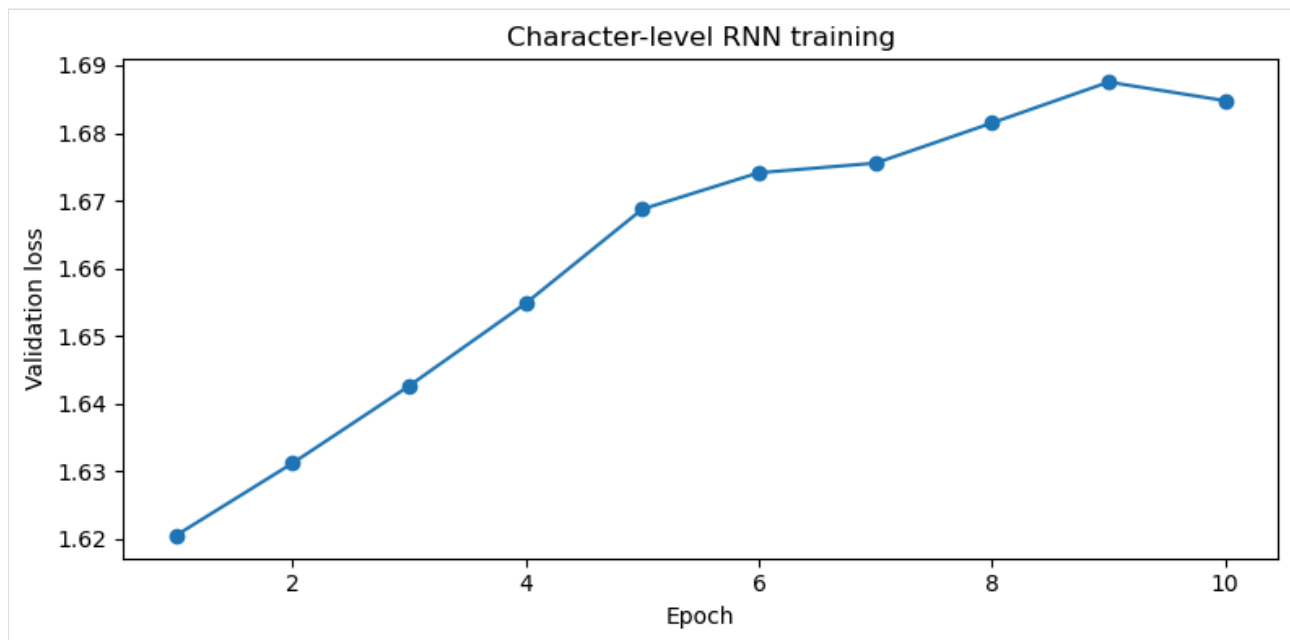
Epoch 8 | train loss 1.2802 | val loss 1.6815

Epoch 9 | train loss 1.2778 | val loss 1.6875

Epoch 10 | train loss 1.2760 | val loss 1.6848

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Validation loss')
plt.title('Character-level RNN training')
plt.tight_layout()
plt.show()
```



Generate Text

Function

```
def generate(model, start, length, temperature=0.8):
    """
    Generate text from a trained model.

    Parameters:
        model (nn.Module): Trained model.
        start (str): Seed text.
        length (int): Number of characters to generate.
        temperature (float): Sampling temperature.

    Returns:
        str: The generated text.
    """
    model.eval()
    indices = [STOI[ch] for ch in start]
    hidden = None
    with torch.no_grad():
        for _ in range(length):
            x = torch.tensor([indices[-1]]).unsqueeze(0).to(DEVICE)
            logits, hidden = model(x, hidden)
            probs = torch.softmax(logits[0, -1] / temperature, dim=0)
            next_index = torch.multinomial(probs, 1).item()
            indices.append(next_index)
    return decode(indices, ITOS)
```

Run

```
print(generate(model, 'ROMEO: ', 300))
```

```
ROMEO: me your king.
Come, uncustation of that the noble suppinable,
Why, when it was hung to liegernips. Come!
To make it is proceeded me to sorrow
living hours, shall come against that alther lies
To say 'tis no power, whom thou canst secrets. Therefore, by this pite like fortune
Is consucers, so take
```

Save Model

```
torch.save(model.state_dict(), 'models/rnn_shakespeare.pt')
print('saved rnn_shakespeare.pt')
```

```
saved rnn_shakespeare.pt
```

Load Model

```
loaded_model = RNNModel(VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM).to(DEVICE)
loaded_model.load_state_dict(torch.load('models/rnn_shakespeare.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded rnn_shakespeare.pt')
```

```
loaded rnn_shakespeare.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_76955/1712437992.py:2: FutureWarning: You
are using `torch.load` with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data which will execute
arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a
future release, the default value for `weights_only` will be flipped to `True`. This limits the
functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use
case where you don't have full control of the loaded file. Please open an issue on GitHub for any
issues related to this experimental feature.
loaded_model.load_state_dict(torch.load('models/rnn_shakespeare.pt', map_location=DEVICE))
```

Inference

```
print(generate(loaded_model, 'JULIET: ', 200))
```

```
JULIET: soul,
When against this time and grace a thing is the strong.
What dost so strike the sea, we beholder, and so ha heart
A change he doom him as both and all; for nails
That our father sound o' the mar
```

Key Takeaways

- Recurrent networks carry a hidden state through a sequence.
- Text must be encoded as integers through a vocabulary.
- Language modeling is next-token prediction on shifted windows.
- Sampling with temperature controls how conservative the generated text is.

Exercises

1. Change the sequence length and observe the effect on generation.
2. Train for more epochs and compare the text quality.
3. Generate with temperatures of 0.5 and 1.2 and compare.
4. Replace the LSTM with a plain RNN and compare the loss.
5. Save the model and generate from it after reloading.

Guided Walkthrough

The notebook downloads Tiny Shakespeare, builds a character vocabulary, and encodes the whole corpus as integers. A dataset then yields fixed-length windows as input and the same window shifted by one as the target, which turns next-character prediction into supervised learning. The model embeds each character, runs the sequence through an LSTM, and projects each step to a score over the vocabulary.

Training minimizes cross-entropy on the next character. Generation starts from a seed string, predicts the next character, appends it, and repeats. The temperature parameter divides the logits before the softmax, which sharpens the distribution toward safe predictions or flattens it toward surprising ones. Watching the generated

text evolve from noise to something Shakespeare-shaped is the payoff.

Common Pitfalls

- Forgetting that the target is the input shifted by one.
- Sampling with a temperature that is too low, producing repetitive text.
- Letting the sequence length exceed the model's training window.

Going Further

- Compare an LSTM with a plain RNN on the same data.
- Change the embedding size and observe the effect on loss.
- Train a word-level model instead of a character-level model.

Exercise Solutions

1. A longer sequence gives the model more context but needs more memory and time.
2. More epochs lower the validation loss and improve the text.
3. Low temperature produces safe, repetitive text; high temperature produces varied but error-prone text.
4. A plain RNN trains faster but forgets long-range structure and has a higher loss.
5. The reloaded model generates text in the same style as the original.

Chapter 19: Tiny Shakespeare Dataset - Attention

Introduction

Attention is the idea that changed natural language processing. Instead of squeezing an entire sequence into a single hidden state, attention lets every position look directly at every other position and decide what matters. This chapter builds attention from scratch: scaled dot-product attention, causal masking, multi-head attention, and a full transformer block, all trained as a character language model on Tiny Shakespeare.

The chapter is deliberately explicit. Rather than calling a library, we compute the queries, keys, and values ourselves and write the attention weights by hand, so that the mechanism is not a black box.

What You Will Learn

- How scaled dot-product attention computes weights between positions.
- Why a causal mask is required for language modeling.
- How multiple attention heads attend to different relationships.
- How a transformer block combines attention with a feed-forward network.
- How residual connections and normalization make deep stacks trainable.

Overview

We start with the attention formula: compute queries, keys, and values, take the scaled dot product of queries and keys, apply a mask so a position cannot see the future, and use the softmax weights to average the values. We visualize the attention weights as a heatmap and confirm the causal structure, a lower-triangular pattern in which each position attends only to itself and the past.

We then build multi-head attention, which runs several attention operations in parallel so the model can attend to different kinds of relationships at once, and a transformer block, which pairs attention with a position-wise feed-forward network and wraps both in residual connections and layer normalization. The block is the repeating unit of every modern language model, and the next chapter stacks it into a full GPT.

Citation

[1] Andrej Karpathy, <https://github.com/karpathy/char-rnn>

[2] Ashish Vaswani et al., <https://arxiv.org/abs/1706.03762>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib
```

Import Libraries

```
import urllib.request
import math
```

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x122b49b10>
```

Parameters

```
SEQ_LEN = 64
SEQ_LEN
```

```
64
```

```
EMBED_DIM = 64
EMBED_DIM
```

```
64
```

```
N_HEADS = 4
N_HEADS
```

```
4
```

```
FF_DIM = 128
FF_DIM
```

```
128
```

Hyperparameters

```
LEARNING_RATE = 0.002
LEARNING_RATE
```

```
0.002
```

```
EPOCHS = 10
EPOCHS
```

```
10
```

```
BATCH_SIZE = 128
BATCH_SIZE
```

```
128
```

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Download Dataset

```
URL = 'https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt'
url = urllib.request.urlopen(URL)
text = url.read()
print('length:', len(text))
```

```
length: 1115394
```

Build Vocabulary

```
CHARS = sorted(set(text))
STOI = {ch: i for i, ch in enumerate(CHARS)}
ITOS = {i: ch for i, ch in enumerate(CHARS)}
VOCAB_SIZE = len(CHARS)
data = torch.tensor([STOI[ch] for ch in text], dtype=torch.long)
print('vocab size:', VOCAB_SIZE)
```

vocab size: 65

Attention From Scratch

Attention lets every position look at every other position. The scores are the dot products of queries and keys, scaled by the square root of the key dimension, then turned into weights with softmax.

Scaled Dot-Product Attention

```
def scaled_dot_product_attention(query, key, value, mask=None):
    """
    Compute scaled dot-product attention.

    Parameters:
        query (torch.Tensor): Query tensor.
        key (torch.Tensor): Key tensor.
        value (torch.Tensor): Value tensor.
        mask (torch.Tensor): Optional mask of allowed positions.

    Returns:
        tuple: Attention output and attention weights.
    """
    d_k = query.size(-1)
    scores = query @ key.transpose(-2, -1) / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))
    weights = torch.softmax(scores, dim=-1)
    return weights @ value, weights
```

Try It on a Small Sequence

```
torch.manual_seed(SEED)
tokens = torch.randn(5, 8)
out, weights = scaled_dot_product_attention(tokens, tokens, tokens)
print('output shape:', out.shape)
print('weight row sums:', weights.sum(dim=-1))
```

```
output shape: torch.Size([5, 8])
weight row sums: tensor([1.0000, 1.0000, 1.0000, 1.0000, 1.0000])
```

Causal Mask

A language model may not look at future characters. A causal mask blocks every position from attending to later positions.

```
def causal_mask(size):
    """
    Build a lower-triangular causal mask.

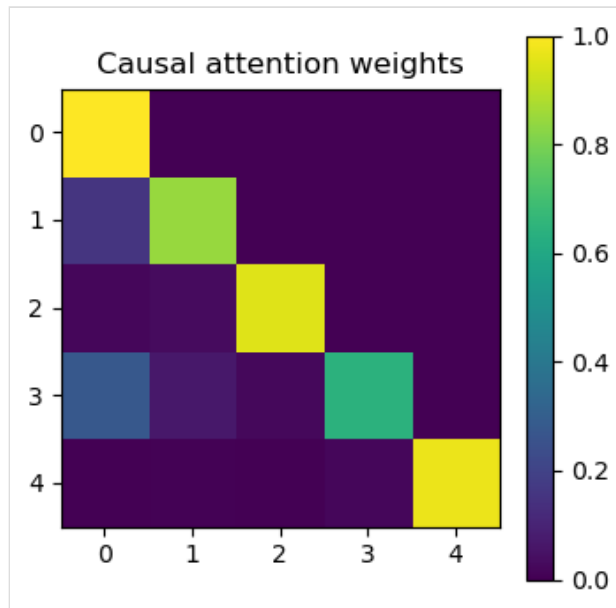
    Parameters:
        size (int): Sequence length.

    Returns:
        torch.Tensor: A size x size mask.
    """
    return torch.tril(torch.ones(size, size)).bool()
```

```

torch.manual_seed(SEED)
tokens = torch.randn(5, 8)
mask = causal_mask(5)
out, weights = scaled_dot_product_attention(tokens, tokens, tokens, mask)
plt.figure(figsize=(4, 4))
plt.imshow(weights.detach(), cmap='viridis')
plt.title('Causal attention weights')
plt.colorbar()
plt.show()

```



Multi-Head Attention

```

class MultiHeadAttention(nn.Module):
    """
    Multi-head self-attention with a causal mask.
    """

    def __init__(self, embed_dim, n_heads):
        """
        Initialize the query, key, value, and output projections.

        Parameters:
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.

        Returns:
            None
        """
        super(MultiHeadAttention, self).__init__()
        self.n_heads = n_heads
        self.head_dim = embed_dim // n_heads
        self.qkv = nn.Linear(embed_dim, 3 * embed_dim)
        self.proj = nn.Linear(embed_dim, embed_dim)

    def forward(self, x):
        """
        Run multi-head causal self-attention.

```

```

Parameters:
    x (torch.Tensor): Input of shape (batch, time, embed_dim).

Returns:
    torch.Tensor: Attention output.
"""
batch, time, embed_dim = x.shape
qkv = self.qkv(x).reshape(batch, time, 3, self.n_heads, self.head_dim)
query, key, value = qkv.permute(2, 0, 3, 1, 4)
out = F.scaled_dot_product_attention(query, key, value, is_causal=True)
out = out.transpose(1, 2).reshape(batch, time, embed_dim)
return self.proj(out)

```

Transformer Block

```

class TransformerBlock(nn.Module):
    """
    A single transformer block with attention and a feed-forward network.
    """

    def __init__(self, embed_dim, n_heads, ff_dim):
        """
        Initialize attention, normalization, and feed-forward layers.

        Parameters:
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.
            ff_dim (int): Feed-forward hidden size.

        Returns:
            None
        """
        super(TransformerBlock, self).__init__()
        self.attn = MultiHeadAttention(embed_dim, n_heads)
        self.norm1 = nn.LayerNorm(embed_dim)
        self.ff = nn.Sequential(
            nn.Linear(embed_dim, ff_dim),
            nn.GELU(),
            nn.Linear(ff_dim, embed_dim))
        self.norm2 = nn.LayerNorm(embed_dim)

    def forward(self, x):
        """
        Run the block with residual connections.

        Parameters:
            x (torch.Tensor): Input of shape (batch, time, embed_dim).

        Returns:
            torch.Tensor: Output of the same shape.
        """
        x = self.norm1(x + self.attn(x))
        return self.norm2(x + self.ff(x))

```

Create Model

```

class AttentionModel(nn.Module):
    """
    A character language model with one transformer block.

```

```

"""

def __init__(self, vocab_size, embed_dim, n_heads, ff_dim, max_len):
    """
    Initialize embeddings, the transformer block, and the output head.

    Parameters:
        vocab_size (int): Number of characters.
        embed_dim (int): Embedding size.
        n_heads (int): Number of attention heads.
        ff_dim (int): Feed-forward hidden size.
        max_len (int): Maximum sequence length.

    Returns:
        None
    """
    super(AttentionModel, self).__init__()
    self.token_embed = nn.Embedding(vocab_size, embed_dim)
    self.pos_embed = nn.Embedding(max_len, embed_dim)
    self.block = TransformerBlock(embed_dim, n_heads, ff_dim)
    self.head = nn.Linear(embed_dim, vocab_size)

def forward(self, x):
    """
    Run the forward pass of the model.

    Parameters:
        x (torch.Tensor): Input index batch.

    Returns:
        torch.Tensor: Output logits.
    """
    positions = torch.arange(x.size(1), device=x.device)
    x = self.token_embed(x) + self.pos_embed(positions)
    x = self.block(x)
    return self.head(x)

```

Create Dataset and DataLoader

```

class CharDataset(torch.utils.data.Dataset):
    """
    A dataset of fixed-length character windows and next-character targets.
    """

    def __init__(self, data, seq_len):
        """
        Store the encoded text and window length.

        Parameters:
            data (torch.Tensor): Encoded text.
            seq_len (int): Length of each input window.

        Returns:
            None
        """
        self.data = data
        self.seq_len = seq_len

    def __len__(self):
        """
        Return the number of windows.

```



```

    Parameters:
        None

    Returns:
        int: Number of windows.
    """
    return len(self.data) - self.seq_len

def __getitem__(self, idx):
    """
    Return one input window and its shifted target.

    Parameters:
        idx (int): Window index.

    Returns:
        tuple: Input and target tensors.
    """
    x = self.data[idx:idx + self.seq_len]
    y = self.data[idx + 1:idx + self.seq_len + 1]
    return x, y

```

```

split = int(0.9 * len(data))
train_data = CharDataset(data[:split], SEQ_LEN)
val_data = CharDataset(data[split:], SEQ_LEN)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(val_loader)

```

```

(7843, 871)

```

Instantiate Model, Loss, and Optimizer

```

torch.manual_seed(SEED)
model = AttentionModel(VOCAB_SIZE, EMBED_DIM, N_HEADS, FF_DIM, SEQ_LEN).to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
model

```

```

AttentionModel(
  (token_embed): Embedding(65, 64)
  (pos_embed): Embedding(64, 64)
  (block): TransformerBlock(
    (attn): MultiHeadAttention(
      (qkv): Linear(in_features=64, out_features=192, bias=True)
      (proj): Linear(in_features=64, out_features=64, bias=True)
    )
    (norm1): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
    (ff): Sequential(
      (0): Linear(in_features=64, out_features=128, bias=True)
      (1): GELU(approximate='none')
      (2): Linear(in_features=128, out_features=64, bias=True)
    )
    (norm2): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
  )
)

```

```
(head): Linear(in_features=64, out_features=65, bias=True)
)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        logits = model(x)
        loss = loss_fn(logits.transpose(1, 2), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)

def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        float: Mean loss.
    """
    model.eval()
    total = 0.0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            total += loss_fn(logits.transpose(1, 2), y).item() * len(y)
    return total / len(loader.dataset)
```

Training Loop

```
history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    val_loss = evaluate(model, val_loader, loss_fn)
```

```
history.append(val_loss)
print(f"Epoch {epoch + 1:2d} | train loss {train_loss:.4f} | val loss {val_loss:.4f}")
```

```
Epoch 1 | train loss 1.7006 | val loss 1.7970
```

```
Epoch 2 | train loss 1.5685 | val loss 1.7978
```

```
Epoch 3 | train loss 1.5497 | val loss 1.7935
```

```
Epoch 4 | train loss 1.5396 | val loss 1.7926
```

```
Epoch 5 | train loss 1.5329 | val loss 1.7907
```

```
Epoch 6 | train loss 1.5280 | val loss 1.7828
```

```
Epoch 7 | train loss 1.5243 | val loss 1.7718
```

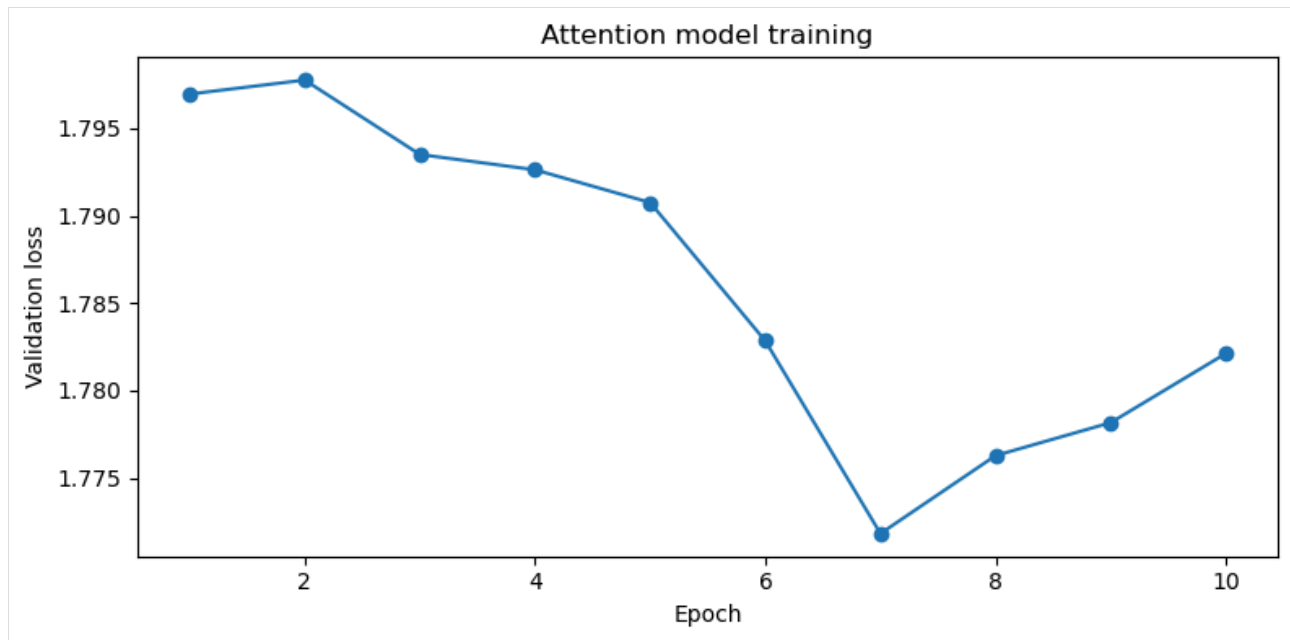
```
Epoch 8 | train loss 1.5214 | val loss 1.7763
```

```
Epoch 9 | train loss 1.5190 | val loss 1.7782
```

```
Epoch 10 | train loss 1.5171 | val loss 1.7821
```

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Validation loss')
plt.title('Attention model training')
plt.tight_layout()
plt.show()
```



Generate Text

```
def generate(model, start, length, temperature=0.8):
    """
    Generate text from a trained model.

    Parameters:
        model (nn.Module): Trained model.
        start (str): Seed text.
        length (int): Number of characters to generate.
        temperature (float): Sampling temperature.

    Returns:
        str: The generated text.
    """
    model.eval()
    indices = [STOI[ch] for ch in start]
    with torch.no_grad():
        for _ in range(length):
            window = torch.tensor(indices[-SEQ_LEN:]).unsqueeze(0).to(DEVICE)
            logits = model(window)
            probs = torch.softmax(logits[0, -1] / temperature, dim=0)
            indices.append(torch.multinomial(probs, 1).item())
    return ''.join(ITOS[i] for i in indices)
```

```
print(generate(model, 'ROMEO: ', 300))
```

ROMEO: he say souls.

Second God's no loss hater, that the power:
 Farewell, that his Clarence, and, First Coriolanus
 To the cousiness mine else thee laster his yourself,
 And me; the entern that like in that wing hence as there peace merry such stay here and yet things of
 like a should sick's were test then

Save Model

```
torch.save(model.state_dict(), 'models/attention_shakespeare.pt')
print('saved attention_shakespeare.pt')
```

```
saved_attention_shakespeare.pt
```

Load Model

```
loaded_model = AttentionModel(VOCAB_SIZE, EMBED_DIM, N_HEADS, FF_DIM, SEQ_LEN).to(DEVICE)
loaded_model.load_state_dict(torch.load('models/attention_shakespeare.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded attention_shakespeare.pt')
```

```
loaded_attention_shakespeare.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_77185/746794393.py:2: FutureWarning: You
are using `torch.load` with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data which will execute
arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a
future release, the default value for `weights_only` will be flipped to `True`. This limits the
functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use
case where you don't have full control of the loaded file. Please open an issue on GitHub for any
issues related to this experimental feature.
loaded_model.load_state_dict(torch.load('models/attention_shakespeare.pt', map_location=DEVICE))
```

Inference

```
print(generate(loaded_model, 'JULIET: ', 200))
```

```
JULIET: to all leave worst that the be mother prove die the part
And bests dead sweet loes in a proported enemia redit was with the sorrow,
My shall scopt that was no news in nothings.
```

```
LADY GREY:
Most I do I
```

Key Takeaways

- Attention weights every position against every other position.
- The causal mask enforces that a language model cannot see the future.
- Multiple heads let the model attend to several relationships at once.
- Residual connections and normalization make deep transformer stacks trainable.

Exercises

1. Visualize the attention weights and confirm the causal pattern.
2. Change the number of heads and compare the loss.
3. Remove the causal mask and explain why generation breaks.
4. Add a second transformer block and compare.
5. Explain in words what a query and a key represent.

Guided Walkthrough

The notebook builds attention from first principles. It computes queries, keys, and values, takes the scaled dot product, applies a causal mask so a position cannot see the future, and uses the softmax weights to average the values. A heatmap of the attention weights confirms the lower-triangular causal structure.

It then builds multi-head attention, which runs several attention operations in parallel, and a transformer block, which pairs attention with a feed-forward network and wraps both in residual connections and layer normalization. Training the block as a language model closes the loop: the same machinery that a modern large language model uses, written out in full.

Common Pitfalls

- Forgetting the causal mask, which lets the model cheat by seeing the answer.
- Forgetting the scaling factor, which makes the softmax saturate.
- Confusing the roles of queries, keys, and values.

Going Further

- Visualize the attention of several heads and compare what they attend to.
- Remove the residual connections and observe the training instability.
- Stack two blocks and compare the loss.

Exercise Solutions

1. The attention heatmap is lower triangular, which is the causal mask.
2. More heads capture more relationships but cost more computation.
3. Without the mask the model can see the next token during training and fails to generate.
4. Two blocks lower the loss because the model has more capacity.
5. A query asks what a position is looking for; a key advertises what a position offers.

Chapter 20: Tiny Shakespeare Dataset - TinyGPT From Scratch

Introduction

This is the chapter the course has been building toward. TinyGPT is a decoder-only transformer language model, written from scratch and trained on Tiny Shakespeare. It has token and positional embeddings, causal multi-head self-attention, feed-forward blocks, residual connections, layer normalization, and a training loop that minimizes next-token cross-entropy. It is small, but it is architecturally the same family of model as the large language models in the news.

The goal is not to compete with those models. The goal is to remove the mystery. When you have written every line of a transformer and watched it learn to generate coherent text, the architecture stops being magic and becomes engineering.

What You Will Learn

- How a decoder-only transformer is assembled from its parts.
- How token and positional embeddings represent text and order.
- How causal self-attention enforces the language-modeling constraint.
- How to train a language model and generate text by sampling.
- How the same architecture scales up to the largest models.

Overview

We assemble the model bottom-up: an attention head, then multi-head attention, then a feed-forward network, then a block that combines them, then a stack of blocks with embeddings and a language-model head. Each piece is small and readable, and together they form a complete transformer. We store the block size so generation can crop the context to the model's maximum length.

Training uses AdamW on windows of characters, with the model returning both logits and a cross-entropy loss so the training loop stays short. Generation samples one token at a time, feeding the growing sequence back into the model. Because the same model and loop appear in every serious language-model codebase, understanding TinyGPT is a transferable skill, not a toy.

Citation

[1] Andrej Karpathy, <https://github.com/karpathy/nanoGPT>

[2] Ashish Vaswani et al., <https://arxiv.org/abs/1706.03762>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib
```

Import Libraries

```
import urllib.request
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x126e7db50>
```

Parameters

```
BLOCK_SIZE = 64
BLOCK_SIZE
```

```
64
```

```
EMBED_DIM = 64
EMBED_DIM
```

```
64
```

```
N_HEADS = 4
N_HEADS
```

```
4
```

```
N_LAYERS = 2
N_LAYERS
```

```
2
```


REVERSE ENGINEERING DATA

```
FF_DIM = 128
FF_DIM
```

```
128
```

Hyperparameters

```
LEARNING_RATE = 0.002
LEARNING_RATE
```

```
0.002
```

```
EPOCHS = 10
EPOCHS
```

```
10
```

```
BATCH_SIZE = 128
BATCH_SIZE
```

```
128
```

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Download Dataset

```
URL = 'https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt'
url = urllib.request.urlopen(URL, 'data/tinyshakespeare.txt')
with open('data/tinyshakespeare.txt', encoding='utf-8') as handle:
    text = handle.read()
print('length:', len(text))
```

```
length: 1115394
```

Build Vocabulary

```
CHARS = sorted(set(text))
STOI = {ch: i for i, ch in enumerate(CHARS)}
ITOS = {i: ch for i, ch in enumerate(CHARS)}
VOCAB_SIZE = len(CHARS)
data = torch.tensor([STOI[ch] for ch in text], dtype=torch.long)
print('vocab size:', VOCAB_SIZE)
```

```
vocab size: 65
```

Create Dataset and DataLoader

```
class CharDataset(torch.utils.data.Dataset):
    """
    A dataset of fixed-length character windows and next-character targets.
    """

    def __init__(self, data, block_size):
        """
        Store the encoded text and window length.

        Parameters:
            data (torch.Tensor): Encoded text.
            block_size (int): Length of each input window.

        Returns:
            None
        """
        self.data = data
        self.block_size = block_size

    def __len__(self):
        """
        Return the number of windows.

        Parameters:
            None

        Returns:
            int: Number of windows.
        """
        return len(self.data) - self.block_size

    def __getitem__(self, idx):
        """
        Return one input window and its shifted target.

        Parameters:
            idx (int): Window index.

        Returns:
            tuple: Input and target tensors.
        """
        x = self.data[idx:idx + self.block_size]
        y = self.data[idx + 1:idx + self.block_size + 1]
        return x, y
```

```

split = int(0.9 * len(data))
train_data = CharDataset(data[:split], BLOCK_SIZE)
val_data = CharDataset(data[split:], BLOCK_SIZE)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(val_loader)

```

```

(7843, 871)

```

Attention Head

```

class Head(nn.Module):
    """
    A single head of causal self-attention.
    """

    def __init__(self, embed_dim, head_dim):
        """
        Initialize the key, query, and value projections.

        Parameters:
            embed_dim (int): Input embedding size.
            head_dim (int): Size of this head.

        Returns:
            None
        """
        super(Head, self).__init__()
        self.key = nn.Linear(embed_dim, head_dim, bias=False)
        self.query = nn.Linear(embed_dim, head_dim, bias=False)
        self.value = nn.Linear(embed_dim, head_dim, bias=False)

    def forward(self, x):
        """
        Apply causal self-attention to the input.

        Parameters:
            x (torch.Tensor): Input of shape (batch, time, embed_dim).

        Returns:
            torch.Tensor: Attention output.
        """
        key = self.key(x).unsqueeze(1)
        query = self.query(x).unsqueeze(1)
        value = self.value(x).unsqueeze(1)
        out = F.scaled_dot_product_attention(query, key, value, is_causal=True)
        return out.squeeze(1)

```

Multi-Head Attention

```

class MultiHeadAttention(nn.Module):
    """
    Several attention heads running in parallel.
    """

    def __init__(self, embed_dim, n_heads, block_size):
        """
        Initialize the heads and the output projection.

```

```

Parameters:
    embed_dim (int): Input embedding size.
    n_heads (int): Number of heads.
    block_size (int): Maximum sequence length.

Returns:
    None
"""
super(MultiHeadAttention, self).__init__()
head_dim = embed_dim // n_heads
self.heads = nn.ModuleList(
    [Head(embed_dim, head_dim) for _ in range(n_heads)])
self.proj = nn.Linear(embed_dim, embed_dim)

def forward(self, x):
    """
    Concatenate the heads and project the result.

    Parameters:
        x (torch.Tensor): Input of shape (batch, time, embed_dim).

    Returns:
        torch.Tensor: Attention output.
    """
    out = torch.cat([head(x) for head in self.heads], dim=-1)
    return self.proj(out)

```

Feed-Forward Network

```

class FeedForward(nn.Module):
    """
    A position-wise feed-forward network.
    """

    def __init__(self, embed_dim, ff_dim):
        """
        Initialize the two linear layers and activation.

        Parameters:
            embed_dim (int): Input embedding size.
            ff_dim (int): Hidden size.

        Returns:
            None
        """
        super(FeedForward, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(embed_dim, ff_dim),
            nn.ReLU(),
            nn.Linear(ff_dim, embed_dim))

    def forward(self, x):
        """
        Run the feed-forward network.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output tensor.

```

```

"""
return self.net(x)

```

Transformer Block

```

class Block(nn.Module):
    """
    A transformer block with attention and a feed-forward network.
    """

    def __init__(self, embed_dim, n_heads, block_size, ff_dim):
        """
        Initialize attention, feed-forward, and normalization layers.

        Parameters:
            embed_dim (int): Input embedding size.
            n_heads (int): Number of attention heads.
            block_size (int): Maximum sequence length.
            ff_dim (int): Feed-forward hidden size.

        Returns:
            None
        """
        super(Block, self).__init__()
        self.attn = MultiHeadAttention(embed_dim, n_heads, block_size)
        self.ffwd = FeedForward(embed_dim, ff_dim)
        self.ln1 = nn.LayerNorm(embed_dim)
        self.ln2 = nn.LayerNorm(embed_dim)

    def forward(self, x):
        """
        Run the block with residual connections.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output tensor.
        """
        x = x + self.attn(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x

```

Create TinyGPT

```

class TinyGPT(nn.Module):
    """
    A small decoder-only transformer language model.
    """

    def __init__(self, vocab_size, embed_dim, n_heads, n_layers,
                  block_size, ff_dim):
        """
        Initialize embeddings, blocks, and the output head.

        Parameters:
            vocab_size (int): Number of characters.
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.

```

```

        n_layers (int): Number of transformer blocks.
        block_size (int): Maximum sequence length.
        ff_dim (int): Feed-forward hidden size.

Returns:
    None
    """
    super(TinyGPT, self).__init__()
    self.block_size = block_size
    self.token_embed = nn.Embedding(vocab_size, embed_dim)
    self.pos_embed = nn.Embedding(block_size, embed_dim)
    self.blocks = nn.Sequential(*[
        Block(embed_dim, n_heads, block_size, ff_dim) for _ in range(n_layers)])
    self.ln_f = nn.LayerNorm(embed_dim)
    self.head = nn.Linear(embed_dim, vocab_size)

def embed(self, idx):
    """
    Combine token and positional embeddings.

    Parameters:
        idx (torch.Tensor): Input indices.

    Returns:
        torch.Tensor: Embedded input.
    """
    _, time = idx.shape
    positions = torch.arange(time, device=idx.device)
    return self.token_embed(idx) + self.pos_embed(positions)

def forward(self, idx, targets=None):
    """
    Run the forward pass and optional loss.

    Parameters:
        idx (torch.Tensor): Input indices.
        targets (torch.Tensor): Optional target indices.

    Returns:
        tuple: Logits and optional loss.
    """
    x = self.blocks(self.embed(idx))
    logits = self.head(self.ln_f(x))
    if targets is None:
        return logits, None
    loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))
    return logits, loss

def generate(self, idx, max_new_tokens):
    """
    Generate new tokens by sampling from the model.

    Parameters:
        idx (torch.Tensor): Seed indices.
        max_new_tokens (int): Number of tokens to generate.

    Returns:
        torch.Tensor: The extended indices.
    """
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -self.block_size:]
        logits, _ = self(idx_cond)

```

```

        probs = F.softmax(logits[:, -1, :], dim=-1)
        idx_next = torch.multinomial(probs, 1)
        idx = torch.cat((idx, idx_next), dim=1)
    return idx

```

Instantiate Model and Optimizer

```

torch.manual_seed(SEED)
model = TinyGPT(VOCAB_SIZE, EMBED_DIM, N_HEADS, N_LAYERS, BLOCK_SIZE, FF_DIM).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)
sum(p.numel() for p in model.parameters())

```

79169

Train Model

Functions

```

def train_epoch(model, loader, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        _, loss = model(x, y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)

def estimate_loss(model, loader):
    """
    Estimate the mean loss over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.

    Returns:
        float: Mean loss.
    """
    model.eval()
    total = 0.0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)

```

```
_, loss = model(x, y)
total += loss.item() * len(y)
return total / len(loader.dataset)
```

Training Loop

```
history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, optimizer)
    val_loss = estimate_loss(model, val_loader)
    history.append(val_loss)
    print(f"Epoch {epoch + 1:2d} | train loss {train_loss:.4f} | val loss {val_loss:.4f}")
```

```
Epoch 1 | train loss 1.5833 | val loss 1.6869
```

```
Epoch 2 | train loss 1.4269 | val loss 1.6717
```

```
Epoch 3 | train loss 1.4036 | val loss 1.6559
```

```
Epoch 4 | train loss 1.3909 | val loss 1.6461
```

```
Epoch 5 | train loss 1.3823 | val loss 1.6492
```

```
Epoch 6 | train loss 1.3761 | val loss 1.6421
```

```
Epoch 7 | train loss 1.3714 | val loss 1.6458
```

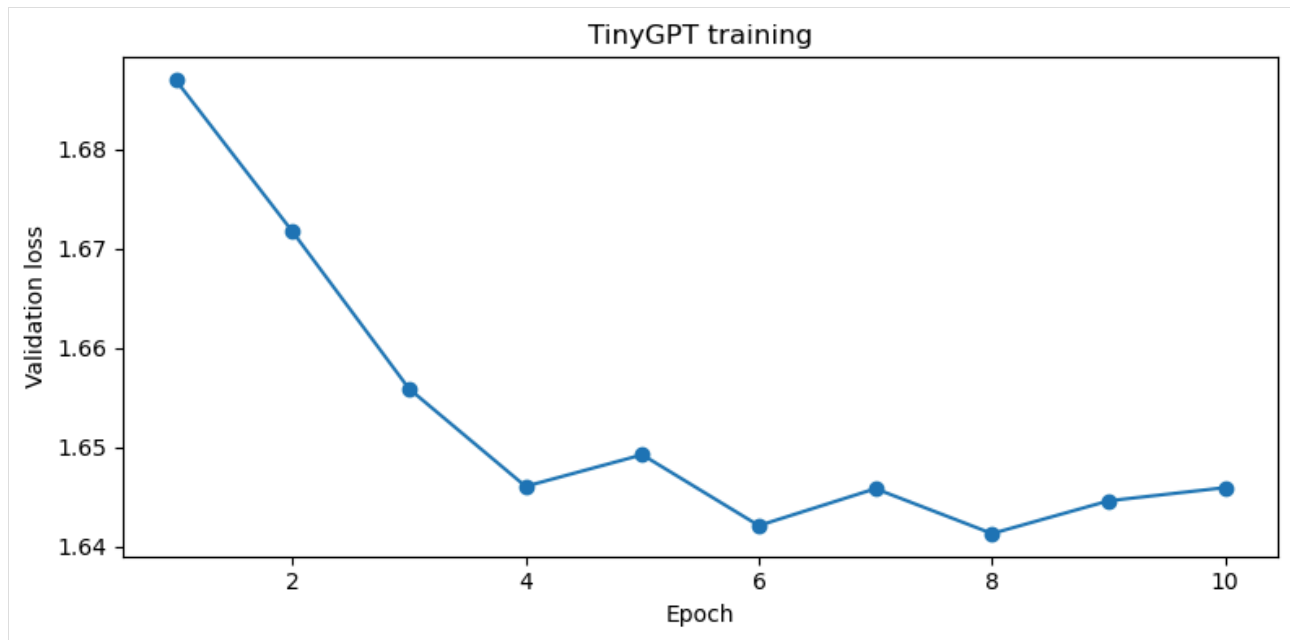
```
Epoch 8 | train loss 1.3679 | val loss 1.6413
```

```
Epoch 9 | train loss 1.3653 | val loss 1.6446
```

```
Epoch 10 | train loss 1.3633 | val loss 1.6459
```

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Validation loss')
plt.title('TinyGPT training')
plt.tight_layout()
plt.show()
```

Generate Text

```
def generate_text(model, start, length, temperature=0.8):
    """
    Generate readable text from a seed string.

    Parameters:
        model (nn.Module): Trained model.
        start (str): Seed text.
        length (int): Number of characters to generate.
        temperature (float): Sampling temperature.

    Returns:
        str: The generated text.
    """
    model.eval()
    idx = torch.tensor([[STOI[ch] for ch in start]], device=DEVICE)
    with torch.no_grad():
        out = model.generate(idx, length)
    return ''.join(ITOS[i] for i in out[0].tolist())
```

```
print(generate_text(model, 'ROMEO: ', 400))
```

```
ROMEO: froy,
A king up baitish slain glorious of cunning suppose:
From your unharning hung to language's grounded,
That is to piece on in selfsame expetunand'st of him that
We did age nor like two objurey:
Having that he thoughts mincting such,
Is breacherous previolence livings, his that shall night;
That news made against thy arm are mock,
That first only as you mank, the faster
Than death he had which
```

Save Model

```
torch.save(model.state_dict(), 'models/tinygpt_shakespeare.pt')
print('saved tinygpt_shakespeare.pt')
```

```
saved tinygpt_shakespeare.pt
```

Load Model

```
loaded_model = TinyGPT(VOCAB_SIZE, EMBED_DIM, N_HEADS, N_LAYERS, BLOCK_SIZE, FF_DIM).to(DEVICE)
loaded_model.load_state_dict(torch.load('models/tinygpt_shakespeare.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded tinygpt_shakespeare.pt')
```

```
loaded tinygpt_shakespeare.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_77603/3026413204.py:2: FutureWarning: You
are using `torch.load` with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data which will execute
arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a
future release, the default value for `weights_only` will be flipped to `True`. This limits the
functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use
case where you don't have full control of the loaded file. Please open an issue on GitHub for any
issues related to this experimental feature.
  loaded_model.load_state_dict(torch.load('models/tinygpt_shakespeare.pt', map_location=DEVICE))
```

Inference

```
print(generate_text(loaded_model, 'JULIET: ', 400))
```

```
JULIET: I am ind the oddit of chance
We rooth better to have!
```

```
DUKE VINCENTIO:
Sirrah, that even to them
When he comes this disposition in should
And say I'll says troublingness friends function?
O let it is ut condry Flanterposing:
But for than you chamber?
What, show yours, this along. Would have sentenched nocked by challed living
To violent thy pleasure
Thus here against you bring along
To visit on my
```

Key Takeaways

- A decoder-only transformer is embeddings, a stack of blocks, and a language-model head.
- Causal self-attention is what makes next-token prediction well defined.
- The training objective is cross-entropy on the next token.

- Generation samples from the model one token at a time.
- The same architecture scales from TinyGPT to the largest models.

Exercises

1. Count the parameters of each component of TinyGPT.
2. Increase the number of layers and observe the effect on the loss.
3. Generate with different temperatures and compare.
4. Replace the ReLU in the feed-forward block with GELU.
5. Explain how TinyGPT differs from an encoder-only transformer such as the one in the previous chapter.

Guided Walkthrough

This notebook assembles a decoder-only transformer from the ground up. It defines an attention head, multi-head attention, a feed-forward network, a block, and finally TinyGPT itself, with token and positional embeddings and a language-model head. Training uses AdamW on windows of characters, and the model returns both logits and the cross-entropy loss so the loop stays short.

Generation samples one token at a time, cropping the context to the model's block size. Because the same architecture and loop appear in every serious language-model codebase, understanding TinyGPT transfers directly to the models in production. The chapter is the destination of the sequence-modeling arc and the clearest demonstration that a transformer is engineering, not magic.

Common Pitfalls

- Forgetting to crop the context to the block size during generation.
- Miscomputing the causal mask so the model sees the future.
- Using a learning rate that is too high, which destabilizes training.

Going Further

- Count the parameters of each component and find the largest.
- Replace the ReLU in the feed-forward block with GELU and compare.
- Scale the model up by adding layers and heads and observe the loss.

Exercise Solutions

1. The attention and feed-forward blocks dominate the parameter count.
2. More layers lower the loss until the model becomes hard to train.
3. Low temperature gives coherent but repetitive text; high temperature is more varied.
4. GELU usually gives a small improvement over ReLU.
5. TinyGPT is decoder-only and causal; an encoder-only transformer sees the whole sequence.

Part VII: Regularization, Evaluation, and Generative Vision

Chapter 21: Breast Cancer Dataset - Regularization

Introduction

A model that memorizes its training data is useless. Regularization is the set of techniques that keep a model from memorizing, and early stopping is the simplest of them all. This chapter trains a neural network on the Breast Cancer dataset while applying dropout, decoupled weight decay, cosine learning-rate annealing, and early stopping, and it shows how each one fights overfitting.

The dataset is a real medical one: measurements computed from images of cell nuclei, labeled benign or malignant. It is a good setting for regularization because the stakes of overfitting are concrete and the signal is strong enough to learn quickly.

What You Will Learn

- What overfitting is and how to recognize it from loss curves.
- How dropout randomly disables units to prevent co-adaptation.
- How weight decay shrinks large weights.
- How a learning-rate schedule improves convergence.
- How early stopping halts training at the right time.

Overview

We standardize the features, split the data, and define a network with dropout between its hidden layers. The optimizer is AdamW, which applies weight decay correctly, and a cosine schedule lowers the learning rate smoothly over training. Each epoch we record the training and validation loss, and we keep the weights that achieved the best validation loss. When validation loss stops improving for a set number of epochs, we stop.

The loss curves tell the story. Training loss keeps falling while validation loss flattens and then rises; that divergence is overfitting, and early stopping catches it. We restore the best weights and report the test accuracy, then save the model.

Citation

[1] Wolberg, Street, and Mangasarian,

<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy pandas matplotlib scikit-learn
```

Import Libraries

```
import numpy as np
import torch
import torch.nn as nn
```

```
from torch.utils.data import DataLoader
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x126549b30>
```

Parameters

```
IN_FEATURES = 30
IN_FEATURES
```

```
30
```

```
H1 = 32
H1
```

```
32
```

```
H2 = 16
H2
```

```
16
```

```
OUT_FEATURES = 2
OUT_FEATURES
```

```
2
```

```
TEST_SIZE = 0.3
TEST_SIZE
```

0.3

Hyperparameters

```
LEARNING_RATE = 0.001
LEARNING_RATE
```

0.001

```
EPOCHS = 300
EPOCHS
```

300

```
BATCH_SIZE = 32
BATCH_SIZE
```

32

```
DROPOUT = 0.3
DROPOUT
```

0.3

```
WEIGHT_DECAY = 0.01
WEIGHT_DECAY
```

0.01

```
PATIENCE = 20
PATIENCE
```

20

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
data = load_breast_cancer()  
X = data.data.astype(np.float32)  
y = data.target.astype(np.int64)  
X.shape, y.shape
```

```
((569, 30), (569,))
```

Clean Dataset

Standardize the Features

```
X_MEAN = X.mean(axis=0)  
X_STD = X.std(axis=0)  
X = (X - X_MEAN) / X_STD  
X.shape
```

```
(569, 30)
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=TEST_SIZE, random_state=SEED, stratify=y)  
X_train.shape, X_test.shape
```

```
((398, 30), (171, 30))
```

Create Dataset and DataLoaders

```
class NumpyDataset(torch.utils.data.Dataset):  
    """  
    A memory-efficient dataset that stores data as NumPy arrays.  
    """  
  
    def __init__(self, X, y, device):  
        """  
        Initialize the dataset with NumPy arrays.  
  
        Parameters:  
            X (np.ndarray): Feature array.  
            y (np.ndarray): Label array.  
            device (torch.device): Device to move tensors to.  
  
        Returns:  
            None  
        """  
        self.X = X  
        self.y = y  
        self.device = device
```



```

def __len__(self):
    """
    Return the number of samples in the dataset.

    Parameters:
        None

    Returns:
        int: Number of samples.
    """
    return len(self.X)

def __getitem__(self, idx):
    """
    Return a single sample as tensors on the device.

    Parameters:
        idx (int): Index of the sample.

    Returns:
        tuple: Feature and label tensors.
    """
    X_tensor = torch.tensor(self.X[idx]).float().to(self.device)
    y_tensor = torch.tensor(self.y[idx]).long().to(self.device)
    return X_tensor, y_tensor

```

```

train_loader = DataLoader(NumpyDataset(X_train, y_train, DEVICE),
                          batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(NumpyDataset(X_test, y_test, DEVICE),
                        batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(test_loader)

```

```

(13, 6)

```

Create Model

Dropout randomly zeroes hidden units during training, which forces the network to avoid relying on any single unit.

```

class Model(nn.Module):
    """
    A regularized feedforward neural network with dropout.
    """

    def __init__(self, in_features=IN_FEATURES, h1=H1, h2=H2,
                  out_features=OUT_FEATURES, dropout=DROPOUT):
        """
        Initialize the neural network layers.

        Parameters:
            in_features (int): Number of input features.
            h1 (int): Neurons in the first hidden layer.
            h2 (int): Neurons in the second hidden layer.
            out_features (int): Number of output features.
            dropout (float): Dropout rate.

        Returns:

```

```

        None
        """
        super(Model, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, h1),
            nn.ReLU(),
            nn.Dropout(p=dropout),
            nn.Linear(h1, h2),
            nn.ReLU(),
            nn.Dropout(p=dropout),
            nn.Linear(h2, out_features))

    def forward(self, x):
        """
        Run the forward pass of the network.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output logits.
        """
        return self.net(x)

```

Instantiate Model, Loss, and Optimizer

AdamW applies decoupled weight decay, which shrinks large weights and reduces overfitting.

```

torch.manual_seed(SEED)
model = Model().to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE,
                               weight_decay=WEIGHT_DECAY)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
model

```

```

Model(
  (net): Sequential(
    (0): Linear(in_features=30, out_features=32, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.3, inplace=False)
    (3): Linear(in_features=32, out_features=16, bias=True)
    (4): ReLU()
    (5): Dropout(p=0.3, inplace=False)
    (6): Linear(in_features=16, out_features=2, bias=True)
  )
)

```

Train Model

Functions

```

def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.

```

```

        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

Returns:
    float: Mean training loss.
"""
model.train()
total = 0.0
for x, y in loader:
    optimizer.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    optimizer.step()
    total += loss.item() * len(y)
return total / len(loader.dataset)

def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        tuple: Mean loss and accuracy.
    """
    model.eval()
    total, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            logits = model(x)
            total += loss_fn(logits, y).item() * len(y)
            correct += (logits.argmax(1) == y).sum().item()
    n = len(loader.dataset)
    return total / n, correct / n

```

Early Stopping

Early stopping halts training when validation loss stops improving. The best weights are restored at the end.

```

best_loss = float('inf')
best_state = None
wait = 0
train_losses = []
val_losses = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    val_loss, val_accuracy = evaluate(model, test_loader, loss_fn)
    scheduler.step()
    train_losses.append(train_loss)
    val_losses.append(val_loss)
    if val_loss < best_loss:
        best_loss = val_loss
        best_state = {k: v.clone() for k, v in model.state_dict().items()}
        wait = 0
    else:
        wait += 1

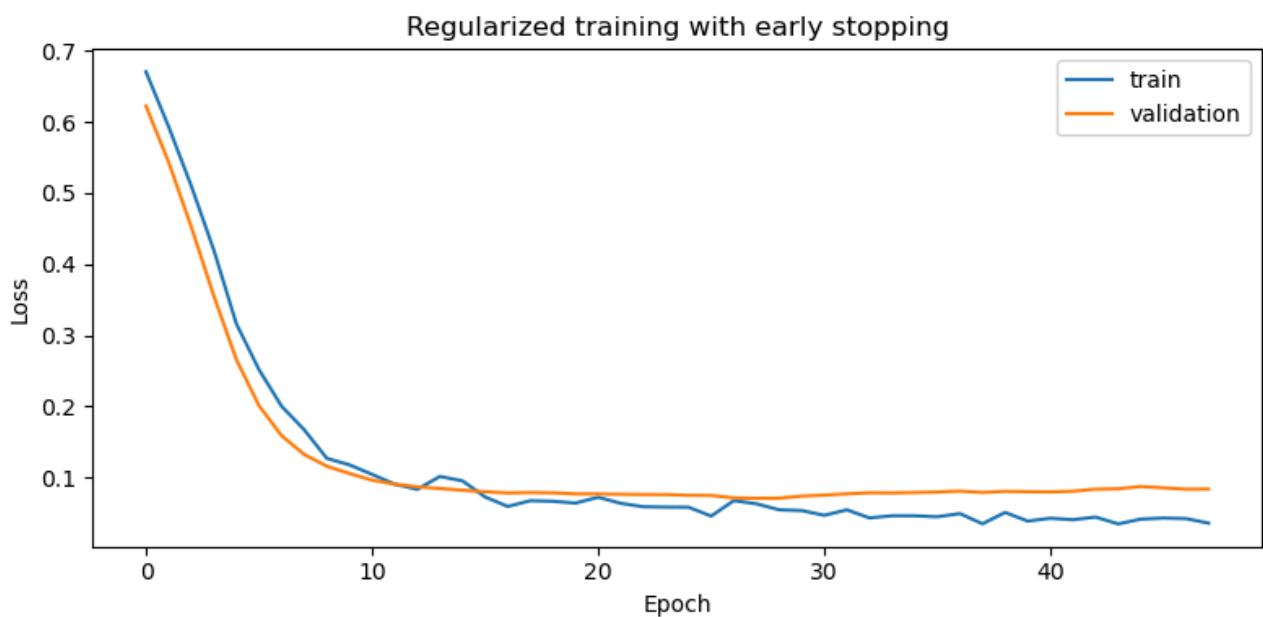
```

```
        if wait >= PATIENCE:
            print(f"Early stop at epoch {epoch + 1}")
            break
model.load_state_dict(best_state)
print(f"Best validation loss: {best_loss:.4f}")
```

Early stop at epoch 48
Best validation loss: 0.0710

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(train_losses, label='train')
plt.plot(val_losses, label='validation')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Regularized training with early stopping')
plt.legend()
plt.tight_layout()
plt.show()
```



Evaluate Model

```
_, accuracy = evaluate(model, test_loader, loss_fn)
print(f"Test accuracy: {accuracy:.4f}")
```

Test accuracy: 0.9649

Save Model

```
torch.save(model.state_dict(), 'models/reg_breast_cancer.pt')
print('saved reg_breast_cancer.pt')
```

```
saved reg_breast_cancer.pt
```

Load Model

```
loaded_model = Model().to(DEVICE)
loaded_model.load_state_dict(torch.load('models/reg_breast_cancer.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded reg_breast_cancer.pt')
```

```
loaded reg_breast_cancer.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_80751/1621903216.py:2: FutureWarning: You
are using `torch.load` with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data which will execute
arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a
future release, the default value for `weights_only` will be flipped to `True`. This limits the
functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use
case where you don't have full control of the loaded file. Please open an issue on GitHub for any
issues related to this experimental feature.
loaded_model.load_state_dict(torch.load('models/reg_breast_cancer.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, features):
    """
    Predict benign or malignant for one raw feature vector.

    Parameters:
        model (nn.Module): Trained model.
        features (list): Raw feature values.

    Returns:
        tuple: Predicted label name and confidence.
    """
    model.eval()
    scaled = (np.array(features, dtype=np.float32) - X_MEAN) / X_STD
    with torch.no_grad():
        X_new = torch.tensor(scaled).float().to(DEVICE).unsqueeze(0)
        probs = torch.softmax(model(X_new), dim=1)
        confidence, predicted = probs.max(dim=1)
        names = ['malignant', 'benign']
        return names[predicted.item()], confidence.item()
```

Run Inference

```
label, confidence = predict(loader_model, X_test[0])
print(f"Predicted: {label} (confidence: {confidence:.4f}) | Actual: {data.target_names[y_test[0]]}")
```

```
Predicted: benign (1.0000) | Actual: malignant
```

Key Takeaways

- Overfitting is a widening gap between training and validation performance.
- Dropout, weight decay, schedules, and early stopping all reduce overfitting.
- AdamW applies weight decay correctly.
- Early stopping restores the best weights, not the last ones.

Exercises

1. Train with and without dropout and compare the validation curves.
2. Disable early stopping and observe the final test accuracy.
3. Vary the weight decay and describe the effect.
4. Plot the learning rate over epochs.
5. Explain why the best weights are not the final weights.

Guided Walkthrough

The notebook standardizes the Breast Cancer features and defines a network with dropout between its hidden layers. The optimizer is AdamW, which applies weight decay correctly, and a cosine schedule lowers the learning rate smoothly. Each epoch records the training and validation loss, and the best weights are kept. When validation loss stops improving, training stops.

The loss curves tell the story. Training loss keeps falling while validation loss flattens and rises; that divergence is overfitting, and early stopping catches it. Restoring the best weights, rather than the last, is the detail that makes early stopping work. The chapter closes by saving the regularized model and reporting its test accuracy.

Common Pitfalls

- Stopping at the last epoch instead of the best.
- Applying weight decay through plain SGD, which does not decouple it.
- Judging overfitting from training loss alone.

Going Further

- Compare dropout rates and find the best on validation.
- Replace the cosine schedule with a step schedule.
- Add data augmentation and compare the curves.

Exercise Solutions

1. Without dropout the validation loss rises sooner and the test accuracy is lower.
2. Disabling early stopping usually lowers test accuracy because the model overfits.
3. Stronger weight decay shrinks weights more and can underfit if taken too far.
4. The cosine schedule starts high and decays smoothly to near zero.
5. The best weights come from the epoch with the lowest validation loss, not the final epoch.

Chapter 22: Breast Cancer Dataset - Model Evaluation

Introduction

A single accuracy number can hide everything that matters. This chapter is a complete treatment of classification evaluation: confusion matrices, precision, recall, F1, ROC and AUC, precision-recall curves, calibration, and threshold tuning. It trains a classifier on the Breast Cancer dataset and then examines it from every angle a safety-conscious engineer would.

Evaluation is where machine learning meets decision-making. A model that is ninety-five percent accurate but never detects the rare positive class is dangerous, and only a breakdown by class reveals that. Calibration matters too: a model that says it is ninety-nine percent confident while being wrong is worse than one that admits uncertainty.

What You Will Learn

- How to read a confusion matrix and derive precision and recall.
- How ROC and AUC summarize ranking quality.
- When the precision-recall curve is more informative than ROC.
- How to measure calibration and why it matters.
- How to tune the decision threshold for the cost at hand.

Overview

We train a small classifier, collect predicted probabilities, and then dissect them. The confusion matrix shows the four outcomes; precision and recall summarize two different kinds of error; F1 balances them. The ROC curve sweeps every threshold and the AUC condenses it into one number, while the precision-recall curve is more honest when classes are imbalanced.

Calibration asks whether the predicted probabilities mean what they say. We plot observed frequency against predicted probability and look for the diagonal. Finally we sweep the decision threshold and watch F1 change, which shows that 0.5 is a default, not a law.

Citation

[1] Wolberg, Street, and Mangasarian,
<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy pandas matplotlib scikit-learn
```

Import Libraries

```
import numpy as np
import pandas as pd
```



```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from sklearn.metrics import precision_recall_curve
from sklearn.calibration import calibration_curve
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x147a34250>
```

Parameters

```
IN_FEATURES = 30
IN_FEATURES
```

```
30
```

```
H1 = 32
H1
```

```
32
```

```
OUT_FEATURES = 2
OUT_FEATURES
```

```
2
```

REVERSE ENGINEERING DATA

```
TEST_SIZE = 0.3  
TEST_SIZE
```

```
0.3
```

Hyperparameters

```
LEARNING_RATE = 0.001  
LEARNING_RATE
```

```
0.001
```

```
EPOCHS = 200  
EPOCHS
```

```
200
```

```
BATCH_SIZE = 32  
BATCH_SIZE
```

```
32
```

Device

```
DEVICE = torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu")  
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
data = load_breast_cancer()  
X = data.data.astype(np.float32)  
y = data.target.astype(np.int64)  
X_MEAN = X.mean(axis=0)  
X_STD = X.std(axis=0)  
X = (X - X_MEAN) / X_STD  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=TEST_SIZE, random_state=SEED, stratify=y)  
X_train.shape, X_test.shape
```

```
((398, 30), (171, 30))
```

Create Dataset and DataLoaders

```
class NumpyDataset(torch.utils.data.Dataset):
    """
    A memory-efficient dataset that stores data as NumPy arrays.
    """

    def __init__(self, X, y, device):
        """
        Initialize the dataset with NumPy arrays.

        Parameters:
            X (np.ndarray): Feature array.
            y (np.ndarray): Label array.
            device (torch.device): Device to move tensors to.

        Returns:
            None
        """
        self.X = X
        self.y = y
        self.device = device

    def __len__(self):
        """
        Return the number of samples in the dataset.

        Parameters:
            None

        Returns:
            int: Number of samples.
        """
        return len(self.X)

    def __getitem__(self, idx):
        """
        Return a single sample as tensors on the device.

        Parameters:
            idx (int): Index of the sample.

        Returns:
            tuple: Feature and label tensors.
        """
        X_tensor = torch.tensor(self.X[idx]).float().to(self.device)
        y_tensor = torch.tensor(self.y[idx]).long().to(self.device)
        return X_tensor, y_tensor
```

```
train_loader = DataLoader(NumpyDataset(X_train, y_train, DEVICE),
                          batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(NumpyDataset(X_test, y_test, DEVICE),
                         batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(test_loader)
```

```
(13, 6)
```

Create Model

```
class Model(nn.Module):
    """
    A small feedforward classifier for tabular data.
    """

    def __init__(self, in_features=IN_FEATURES, h1=H1, out_features=OUT_FEATURES):
        """
        Initialize the neural network layers.

        Parameters:
            in_features (int): Number of input features.
            h1 (int): Neurons in the hidden layer.
            out_features (int): Number of output features.

        Returns:
            None
        """
        super(Model, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, h1),
            nn.ReLU(),
            nn.Linear(h1, out_features))

    def forward(self, x):
        """
        Run the forward pass of the network.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output logits.
        """
        return self.net(x)
```

Instantiate Model, Loss, and Optimizer

```
torch.manual_seed(SEED)
model = Model().to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

Train Model

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
```

```
model.train()
total = 0.0
for x, y in loader:
    optimizer.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    optimizer.step()
    total += loss.item() * len(y)
return total / len(loader.dataset)
```

```
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch + 1:3d} | train loss {train_loss:.4f}")
```

Epoch 20 | train loss 0.0673

Epoch 40 | train loss 0.0436

Epoch 60 | train loss 0.0308

Epoch 80 | train loss 0.0215

Epoch 100 | train loss 0.0151

Epoch 120 | train loss 0.0112

Epoch 140 | train loss 0.0084

Epoch 160 | train loss 0.0066

Epoch 180 | train loss 0.0053

Epoch 200 | train loss 0.0044

Collect Predictions and Probabilities

```
model.eval()
all_probs = []
all_labels = []
with torch.no_grad():
    for x, y in test_loader:
        probs = torch.softmax(model(x), dim=1)[ :, 1]
        all_probs.append(probs.cpu().numpy())
        all_labels.append(y.cpu().numpy())
```

```
y_prob = np.concatenate(all_probs)
y_true = np.concatenate(all_labels)
y_pred = (y_prob >= 0.5).astype(int)
print('collected', y_true.shape)
```

```
collected (171,)
```

Classification Metrics

Functions

```
def metric_table(y_true, y_pred):
    """
    Build a table of core classification metrics.

    Parameters:
        y_true (np.ndarray): True labels.
        y_pred (np.ndarray): Predicted labels.

    Returns:
        pandas.DataFrame: A single-row metrics table.
    """
    row = {'accuracy': accuracy_score(y_true, y_pred),
          'precision': precision_score(y_true, y_pred),
          'recall': recall_score(y_true, y_pred),
          'f1': f1_score(y_true, y_pred),
          'roc_auc': roc_auc_score(y_true, y_pred)}
    return pd.DataFrame(row, index=[0])
```

Run

```
metric_table(y_true, y_pred)
```

```
   accuracy  precision   recall     f1   roc_auc
0  0.97076   0.981132  0.971963  0.976526  0.970356
```

```
print(confusion_matrix(y_true, y_pred))
print(classification_report(y_true, y_pred, target_names=list(data.target_names)))
```

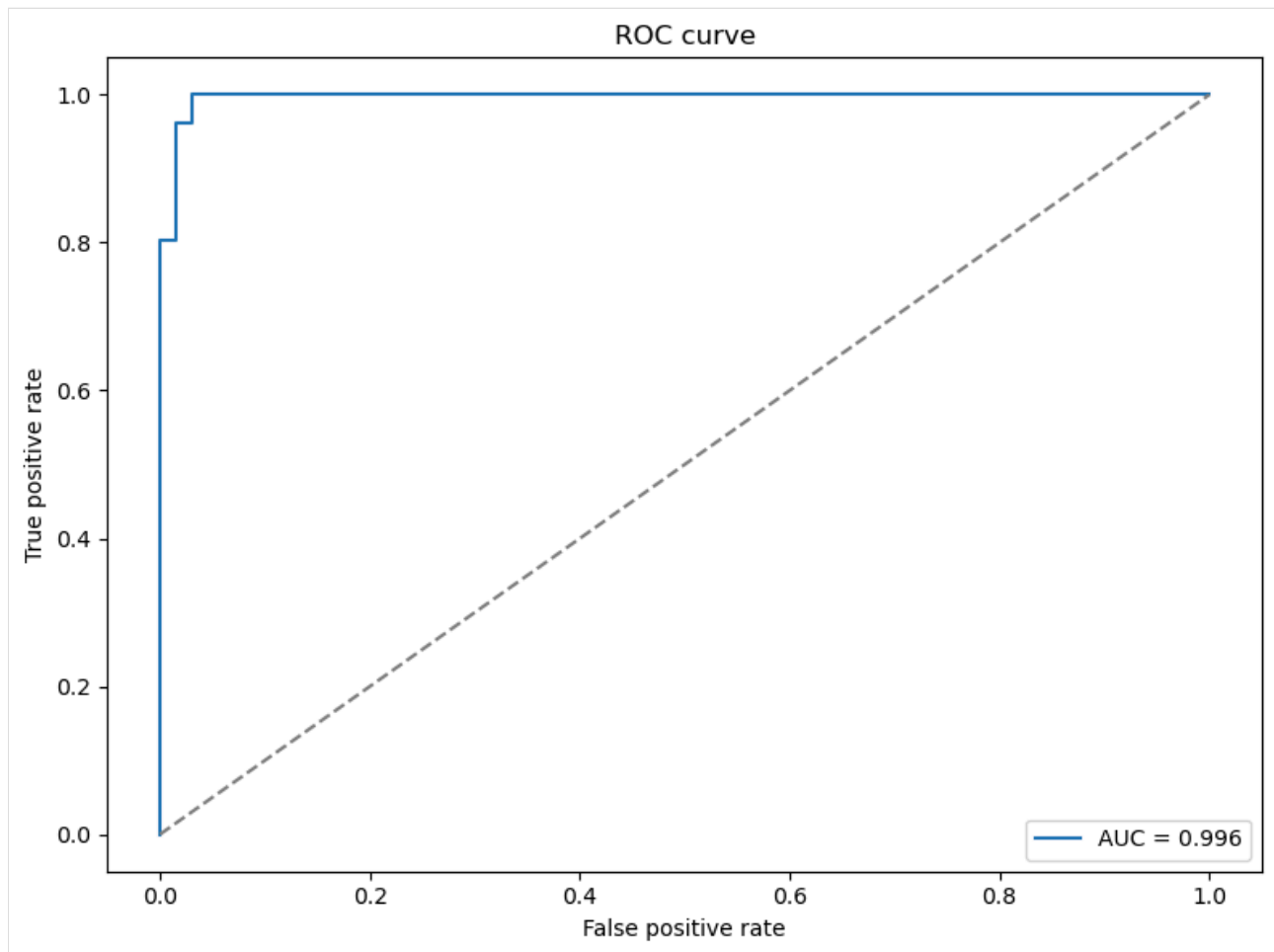
```
[[ 62   2]
 [  3 104]]
      precision    recall  f1-score   support

 malignant      0.95      0.97      0.96         64
   benign      0.98      0.97      0.98        107

 accuracy              0.97         171
 macro avg      0.97      0.97      0.97         171
 weighted avg      0.97      0.97      0.97         171
```

ROC Curve and AUC

```
fpr, tpr, _ = roc_curve(y_true, y_prob)
auc_score = roc_auc_score(y_true, y_prob)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.3f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('ROC curve')
plt.legend()
plt.tight_layout()
plt.show()
```

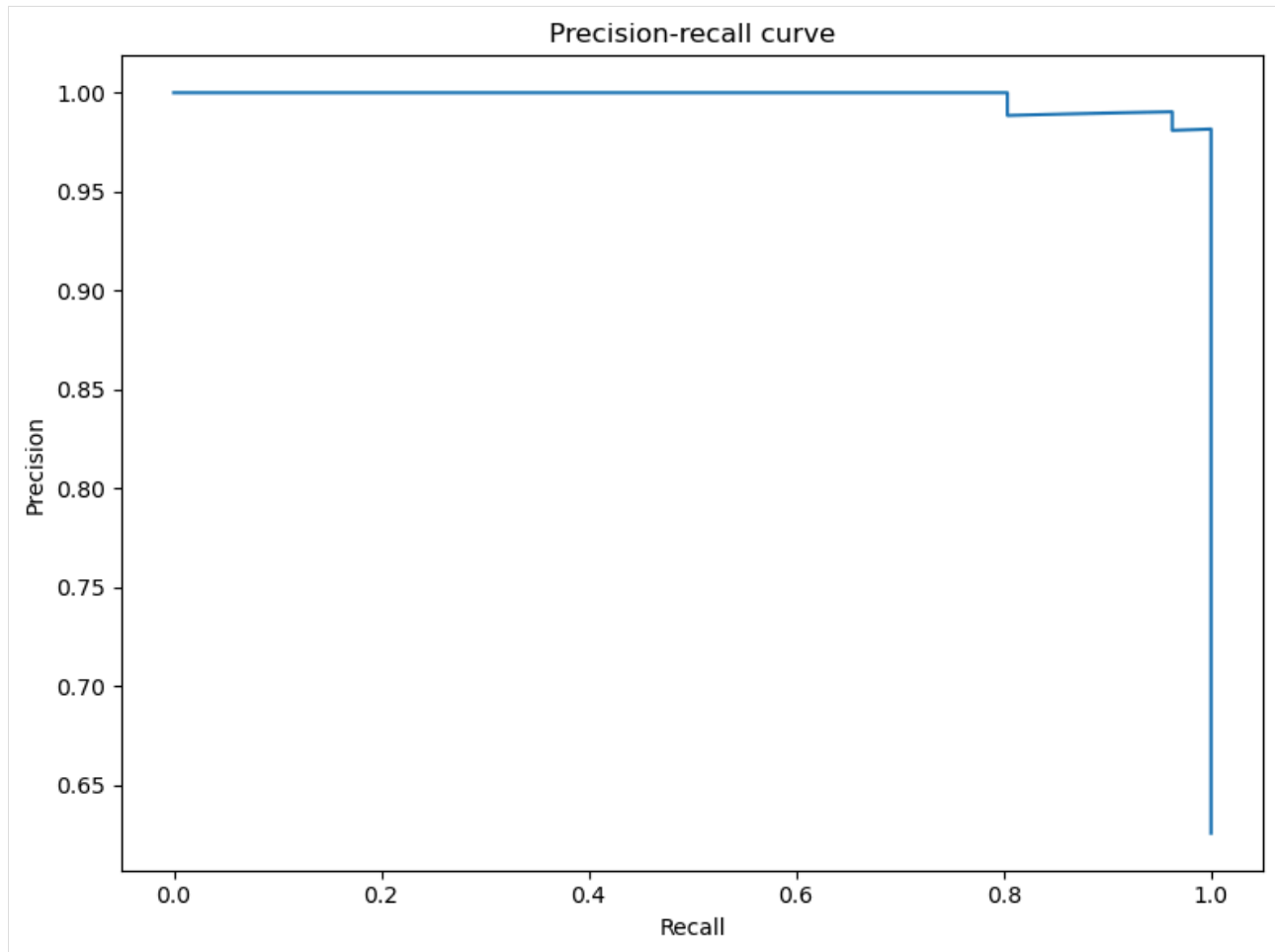


Precision-Recall Curve

The precision-recall curve is more informative than ROC when the classes are imbalanced.

```
precision, recall, _ = precision_recall_curve(y_true, y_prob)
plt.figure(figsize=(8, 6))
plt.plot(recall, precision)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-recall curve')
```

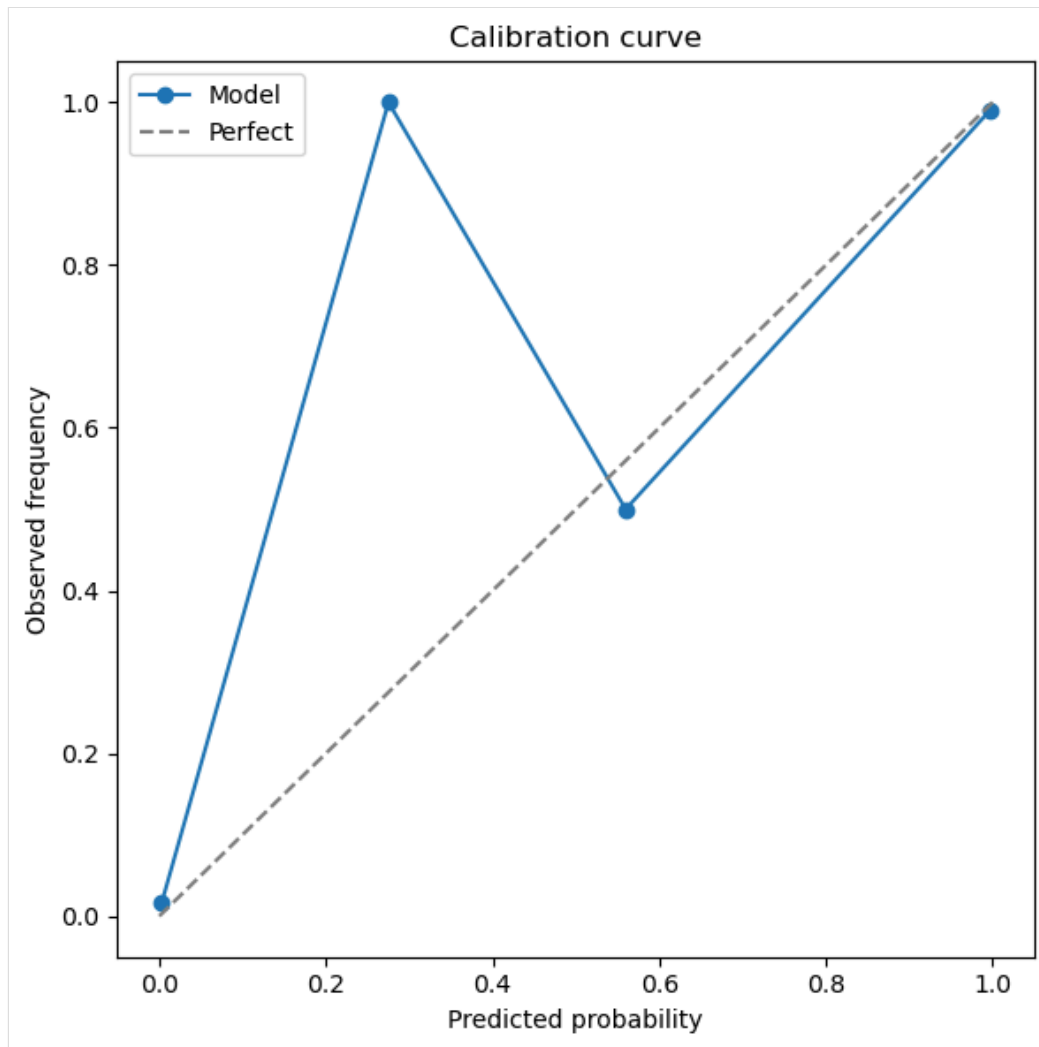
```
plt.tight_layout()
plt.show()
```



Calibration

A calibrated model that reports 0.8 confidence is correct about 80 percent of the time. The diagonal is perfect calibration.

```
prob_true, prob_pred = calibration_curve(y_true, y_prob, n_bins=10)
plt.figure(figsize=(6, 6))
plt.plot(prob_pred, prob_true, marker='o', label='Model')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Perfect')
plt.xlabel('Predicted probability')
plt.ylabel('Observed frequency')
plt.title('Calibration curve')
plt.legend()
plt.tight_layout()
plt.show()
```

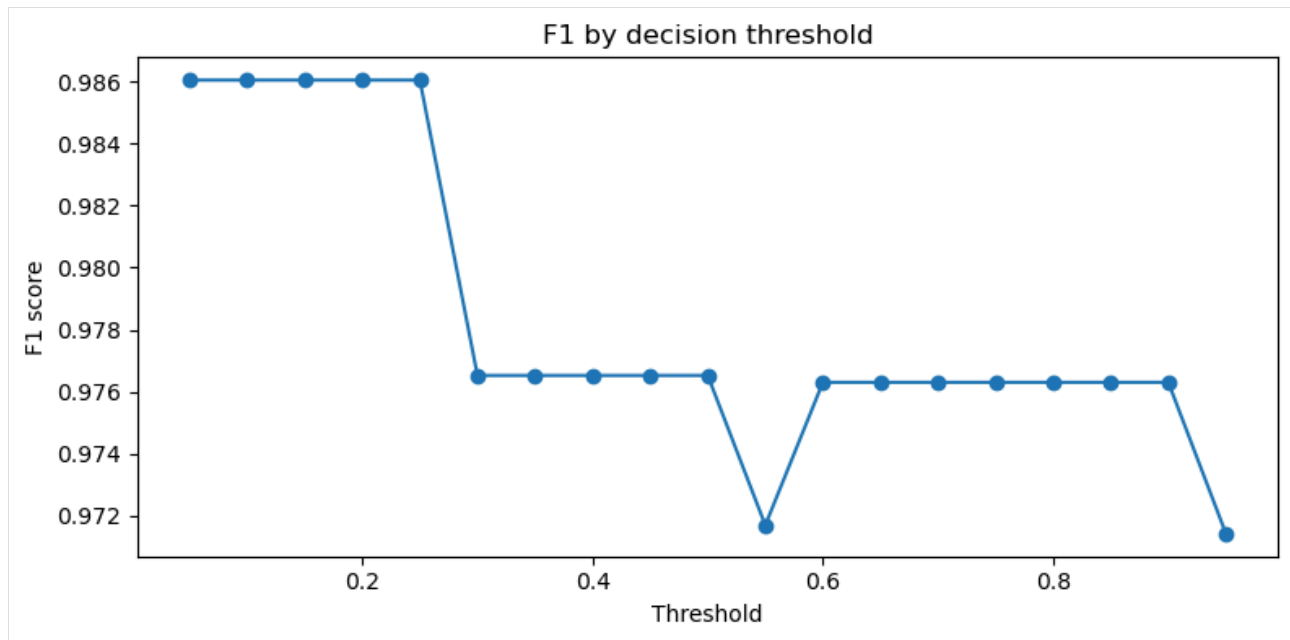



Threshold Tuning

The default 0.5 threshold is not always best. Sweep the threshold and choose the F1-optimal cutoff.

```
thresholds = np.linspace(0.05, 0.95, 19)
scores = [f1_score(y_true, (y_prob >= t).astype(int)) for t in thresholds]
best_index = int(np.argmax(scores))
print(f"Best threshold: {thresholds[best_index]:.2f}")
print(f"Best F1: {scores[best_index]:.4f}")
plt.figure(figsize=(8, 4))
plt.plot(thresholds, scores, marker='o')
plt.xlabel('Threshold')
plt.ylabel('F1 score')
plt.title('F1 by decision threshold')
plt.tight_layout()
plt.show()
```

```
Best threshold: 0.05
Best F1: 0.9860
```



Save Model

```
torch.save(model.state_dict(), 'models/eval_breast_cancer.pt')
print('saved eval_breast_cancer.pt')
```

saved eval_breast_cancer.pt

Load Model

```
loaded_model = Model().to(DEVICE)
loaded_model.load_state_dict(torch.load('models/eval_breast_cancer.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded eval_breast_cancer.pt')
```

loaded eval_breast_cancer.pt

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_80767/2085993247.py:2: FutureWarning: You
are using `torch.load` with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data which will execute
arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a
future release, the default value for `weights_only` will be flipped to `True`. This limits the
functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use
case where you don't have full control of the loaded file. Please open an issue on GitHub for any
issues related to this experimental feature.
loaded_model.load_state_dict(torch.load('models/eval_breast_cancer.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, features, threshold=0.5):
    """
    Predict benign or malignant with a tunable threshold.

    Parameters:
        model (nn.Module): Trained model.
        features (list): Raw feature values.
        threshold (float): Decision threshold.

    Returns:
        tuple: Predicted label name and probability.
    """
    model.eval()
    scaled = (np.array(features, dtype=np.float32) - X_MEAN) / X_STD
    with torch.no_grad():
        X_new = torch.tensor(scaled).float().to(DEVICE).unsqueeze(0)
        probability = torch.softmax(model(X_new), dim=1)[0, 1].item()
        label = 'benign' if probability >= threshold else 'malignant'
    return label, probability
```

Run Inference

```
label, probability = predict(loaded_model, X_test[0])
print(f"Predicted: {label} (p={probability:.4f}) | Actual: {data.target_names[y_test[0]]}")
```

```
Predicted: benign (p=1.0000) | Actual: malignant
```

Key Takeaways

- Accuracy hides class-level failures; use the confusion matrix and per-class metrics.
- AUC measures ranking; the precision-recall curve is better for imbalance.
- Calibration checks whether probabilities can be trusted.
- The decision threshold should reflect the cost of each kind of error.

Exercises

1. Compute precision, recall, and F1 and explain each.
2. Plot the ROC curve and interpret the AUC.
3. Compute the expected calibration error.
4. Sweep the threshold and find the F1-optimal value.
5. Explain when recall matters more than precision.

Guided Walkthrough

The notebook trains a small classifier, collects its predicted probabilities, and then examines them from every angle. The confusion matrix shows the four outcomes; precision and recall summarize two different kinds of error; F1 balances them. The ROC curve sweeps every threshold and the AUC condenses it into one number, while the precision-recall curve is more honest when classes are imbalanced.

Calibration asks whether the predicted probabilities mean what they say. The notebook plots observed frequency against predicted probability and looks for the diagonal. Finally it sweeps the decision threshold and watches F1 change, which shows that 0.5 is a default, not a law. The chapter is a complete toolkit for judging a classifier.

Common Pitfalls

- Reporting accuracy on imbalanced data.
- Using ROC when precision-recall is more appropriate.
- Trusting uncalibrated probabilities for decisions.

Going Further

- Fit a temperature parameter to improve calibration.
- Compare the calibration of a logistic regression and a neural network.
- Choose a threshold that reflects the real cost of each error.

Exercise Solutions

1. Precision is the fraction of predicted positives that are correct; recall is the fraction of true positives that are found.
2. The AUC is the probability that a random positive is ranked above a random negative.
3. Expected calibration error is the average gap between confidence and accuracy across bins.
4. The F1-optimal threshold is found by sweeping and taking the maximum.
5. Recall matters more when missing a positive is costly, as in disease screening.

Chapter 23: Fashion-MNIST Dataset - Autoencoder

Introduction

An autoencoder learns to compress data and reconstruct it. The bottleneck in the middle forces the network to discover a compact representation, and the reconstruction error becomes a useful signal in its own right. This chapter trains an autoencoder on Fashion-MNIST, visualizes its reconstructions, explores the latent space with principal component analysis, and uses reconstruction error for anomaly detection.

Autoencoders are the entry point to representation learning, the idea that a network can learn features without labels. That idea underpins the generative models and the self-supervised methods that dominate modern machine learning.

What You Will Learn

- How an autoencoder compresses and reconstructs data.
- How to visualize reconstructions to judge the learned representation.
- How to project the latent space to two dimensions.
- How to use reconstruction error to flag unusual inputs.

Overview

The encoder flattens each image and maps it to a small latent vector; the decoder maps the vector back to an image. We train with mean squared error between the input and its reconstruction, and we watch the loss fall. Visualizing original images above their reconstructions shows what the bottleneck preserved and what it discarded.

We then encode the test set, project the latent vectors to two dimensions with principal component analysis, and color by class. Similar garments cluster together, which shows the autoencoder learned structure without ever seeing a label. Finally we compute per-image reconstruction errors and use them as an anomaly score.

Citation

[1] Han Xiao, Kashif Rasul, Roland Vollgraf, <https://github.com/zalandoresearch/fashion-mnist>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib scikit-learn
```

Import Libraries

```
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
```

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x10b05db30>
```

Parameters

```
IMAGE_SIZE = 28
IMAGE_SIZE
```

```
28
```

```
LATENT_DIM = 16
LATENT_DIM
```

```
16
```

Hyperparameters

```
LEARNING_RATE = 0.001
LEARNING_RATE
```

```
0.001
```

```
EPOCHS = 10
EPOCHS
```

```
10
```

```
BATCH_SIZE = 256
BATCH_SIZE
```

256

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
transform = transforms.ToTensor()
train_data = datasets.FashionMNIST('data', train=True, download=True, transform=transform)
test_data = datasets.FashionMNIST('data', train=False, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_data), len(test_data)
```

```
(60000, 10000)
```

Create Model

The encoder compresses each 784-pixel image to a 16-number latent vector. The decoder reconstructs the image from that vector.

```
class Autoencoder(nn.Module):
    """
    A fully connected autoencoder for 28x28 grayscale images.
    """

    def __init__(self, latent_dim=LATENT_DIM):
        """
        Initialize the encoder and decoder.

        Parameters:
            latent_dim (int): Size of the latent vector.

        Returns:
            None
        """
        super(Autoencoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(IMAGE_SIZE * IMAGE_SIZE, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim))
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, IMAGE_SIZE * IMAGE_SIZE),
            nn.Sigmoid())
```

```

def forward(self, x):
    """
    Encode and reconstruct a batch of images.

    Parameters:
        x (torch.Tensor): Batch of images.

    Returns:
        tuple: Reconstruction and latent vector.
    """
    latent = self.encoder(x)
    reconstruction = self.decoder(latent)
    return reconstruction, latent

```

Instantiate Model, Loss, and Optimizer

```

torch.manual_seed(SEED)
model = Autoencoder().to(DEVICE)
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
model

```

```

Autoencoder(
  (encoder): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=784, out_features=128, bias=True)
    (2): ReLU()
    (3): Linear(in_features=128, out_features=16, bias=True)
  )
  (decoder): Sequential(
    (0): Linear(in_features=16, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=784, bias=True)
    (3): Sigmoid()
  )
)

```

Train Model

Functions

```

def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the autoencoder for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Reconstruction loss.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, _ in loader:

```



```
x = x.to(DEVICE)
optimizer.zero_grad()
reconstruction, _ = model(x)
loss = loss_fn(reconstruction, x.view(x.size(0), -1))
loss.backward()
optimizer.step()
total += loss.item() * len(x)
return total / len(loader.dataset)

def evaluate(model, loader, loss_fn):
    """
    Evaluate the autoencoder over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Reconstruction loss.

    Returns:
        float: Mean reconstruction loss.
    """
    model.eval()
    total = 0.0
    with torch.no_grad():
        for x, _ in loader:
            x = x.to(DEVICE)
            reconstruction, _ = model(x)
            total += loss_fn(reconstruction, x.view(x.size(0), -1)).item() * len(x)
    return total / len(loader.dataset)
```

Training Loop

```
history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    test_loss = evaluate(model, test_loader, loss_fn)
    history.append(test_loss)
    print(f"Epoch {epoch + 1:2d} | train loss {train_loss:.4f} | test loss {test_loss:.4f}")
```

Epoch 1 | train loss 0.0476 | test loss 0.0249

Epoch 2 | train loss 0.0221 | test loss 0.0206

Epoch 3 | train loss 0.0193 | test loss 0.0183

Epoch 4 | train loss 0.0176 | test loss 0.0170

Epoch 5 | train loss 0.0165 | test loss 0.0160

Epoch 6 | train loss 0.0156 | test loss 0.0154

Epoch 7 | train loss 0.0151 | test loss 0.0149

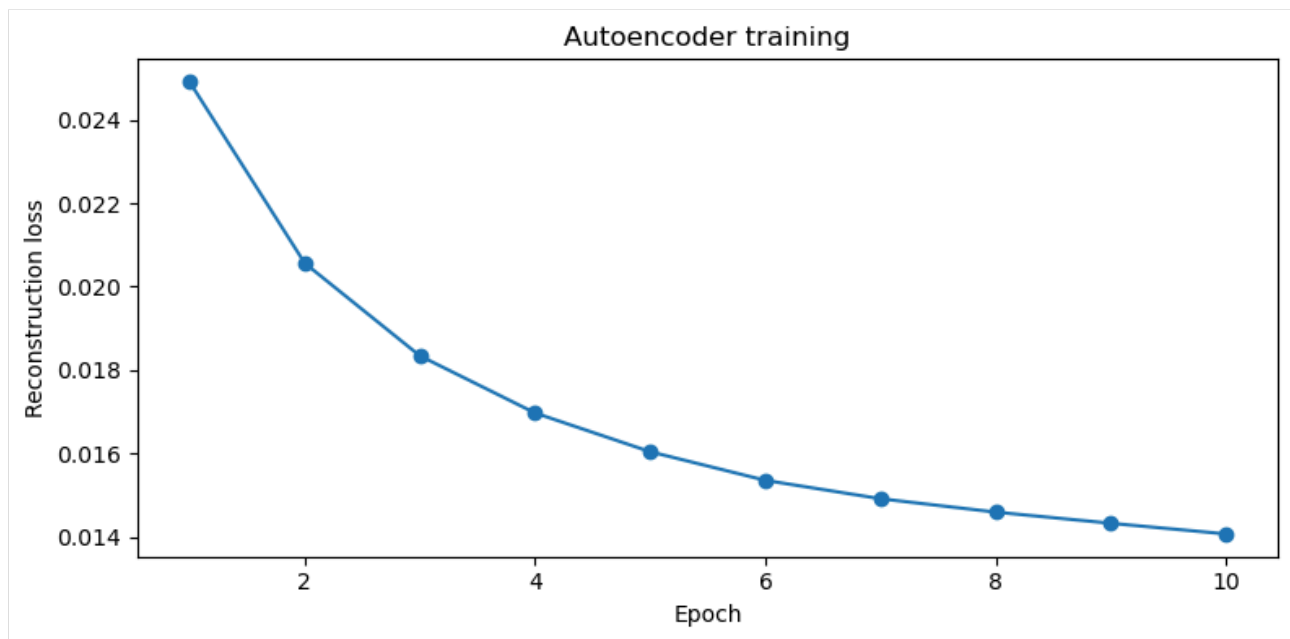
Epoch 8 | train loss 0.0147 | test loss 0.0146

Epoch 9 | train loss 0.0144 | test loss 0.0143

Epoch 10 | train loss 0.0141 | test loss 0.0141

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Reconstruction loss')
plt.title('Autoencoder training')
plt.tight_layout()
plt.show()
```



Visualize Reconstructions

Function

```
def show_reconstructions(model, dataset, n=8):
    """
    Plot original images above their reconstructions.

    Parameters:
        model (nn.Module): Trained autoencoder.
        dataset (Dataset): Source of images.
        n (int): Number of images to show.
```

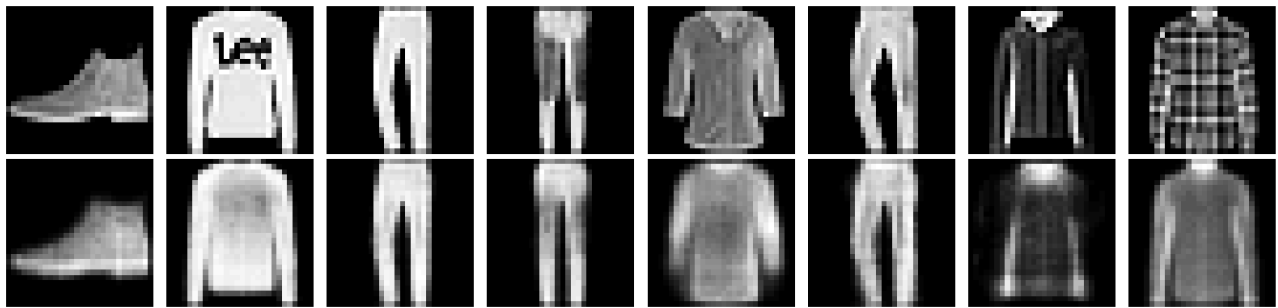
```

Returns:
    None
"""
model.eval()
images = torch.stack([dataset[i][0] for i in range(n)]).to(DEVICE)
with torch.no_grad():
    reconstruction, _ = model(images)
fig, axes = plt.subplots(2, n, figsize=(2 * n, 4))
for i in range(n):
    axes[0, i].imshow(images[i].cpu().squeeze(), cmap='gray')
    axes[0, i].axis('off')
    axes[1, i].imshow(reconstruction[i].cpu().view(28, 28), cmap='gray')
    axes[1, i].axis('off')
fig.tight_layout()
plt.show()

```

Run

```
show_reconstructions(model, test_data, n=8)
```



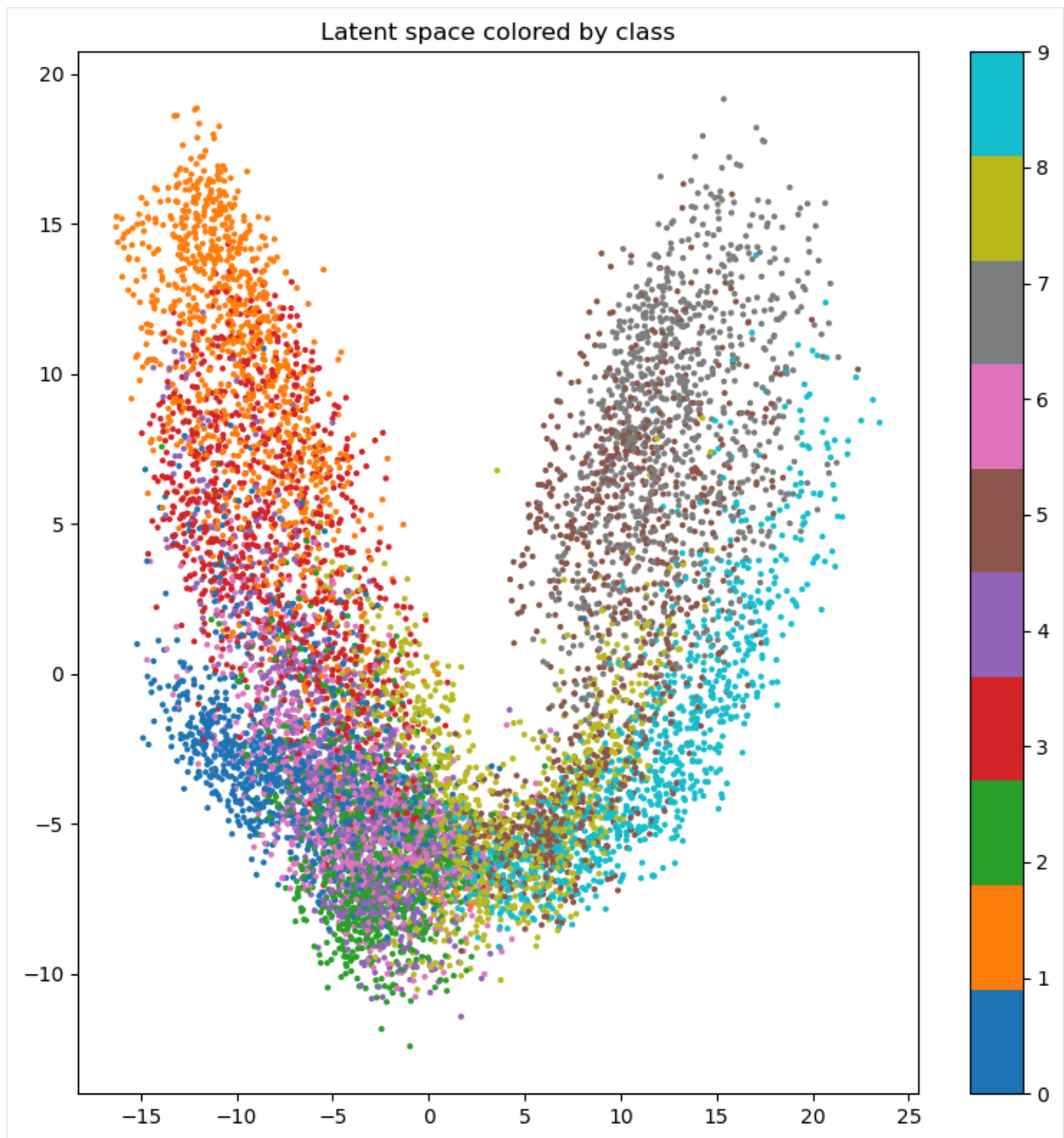
Explore the Latent Space

Project the 16-dimensional latent vectors down to two dimensions with PCA and color by the true class. Similar garments cluster together.

```

model.eval()
latents = []
labels = []
with torch.no_grad():
    for x, y in test_loader:
        _, latent = model(x.to(DEVICE))
        latents.append(latent.cpu().numpy())
        labels.append(y.numpy())
latents = np.concatenate(latents)
labels = np.concatenate(labels)
coords = PCA(n_components=2).fit_transform(latents)
plt.figure(figsize=(8, 8))
plt.scatter(coords[:, 0], coords[:, 1], c=labels, cmap='tab10', s=4)
plt.title('Latent space colored by class')
plt.colorbar()
plt.tight_layout()
plt.show()

```



Anomaly Detection

The reconstruction error is a score for how unusual an image is. Images the model has not learned to reconstruct have high error.

```
def reconstruction_errors(model, dataset, n=1000):  
    """  
    Compute per-image reconstruction errors.  
  
    Parameters:
```

```

model (nn.Module): Trained autoencoder.
dataset (Dataset): Source of images.
n (int): Number of images to score.

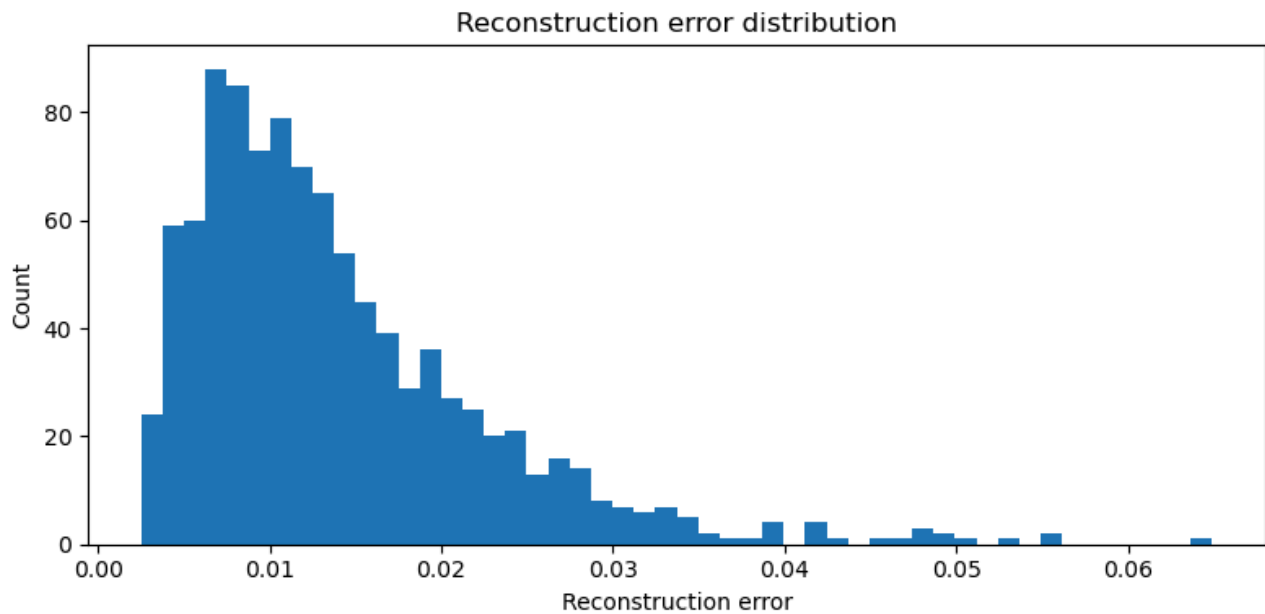
Returns:
    numpy.ndarray: Reconstruction errors.
"""
model.eval()
errors = []
with torch.no_grad():
    for i in range(n):
        image, _ = dataset[i]
        image = image.to(DEVICE).unsqueeze(0)
        reconstruction, _ = model(image)
        error = torch.mean((reconstruction - image.view(1, -1)) ** 2).item()
        errors.append(error)
return np.array(errors)

```

```

errors = reconstruction_errors(model, test_data, n=1000)
plt.figure(figsize=(8, 4))
plt.hist(errors, bins=50)
plt.xlabel('Reconstruction error')
plt.ylabel('Count')
plt.title('Reconstruction error distribution')
plt.tight_layout()
plt.show()

```



Save Model

```

torch.save(model.state_dict(), 'models/autoencoder_fashion_mnist.pt')
print('saved autoencoder_fashion_mnist.pt')

```

```

saved autoencoder_fashion_mnist.pt

```

Load Model

```
loaded_model = Autoencoder().to(DEVICE)
loaded_model.load_state_dict(torch.load('models/autoencoder_fashion_mnist.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded autoencoder_fashion_mnist.pt')
```

```
loaded autoencoder_fashion_mnist.pt
```

/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_80801/2181447253.py:2: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
loaded_model.load_state_dict(torch.load('models/autoencoder_fashion_mnist.pt', map_location=DEVICE))
```

Inference

Function

```
def reconstruct(model, image):
    """
    Reconstruct one image and return its error.

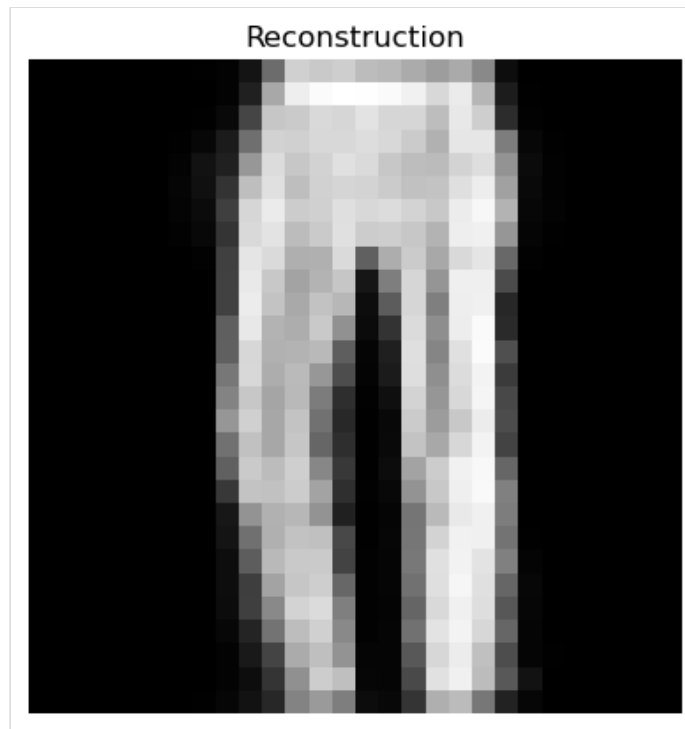
    Parameters:
        model (nn.Module): Trained autoencoder.
        image (torch.Tensor): A 1x28x28 image tensor.

    Returns:
        tuple: Reconstructed image and error.
    """
    model.eval()
    with torch.no_grad():
        batch = image.unsqueeze(0).to(DEVICE)
        reconstruction, _ = model(batch)
        error = torch.mean((reconstruction - batch.view(1, -1)) ** 2).item()
    return reconstruction.view(28, 28).cpu(), error
```

Run Inference

```
image, label = test_data[5]
reconstruction, error = reconstruct(loaded_model, image)
print(f"Label: {label} | Reconstruction error: {error:.4f}")
plt.imshow(reconstruction, cmap='gray')
plt.title('Reconstruction')
plt.axis('off')
plt.show()
```

```
Label: 1 | Reconstruction error: 0.0137
```



Key Takeaways

- An autoencoder learns a compact representation by reconstruction.
- The latent space organizes data by similarity, even without labels.
- Reconstruction error is a usable anomaly score.
- Representation learning is the foundation of modern generative models.

Exercises

1. Change the latent dimension and compare reconstruction quality.
2. Plot the latent space colored by class.
3. Find the images with the highest reconstruction error.
4. Explain what the bottleneck forces the network to do.
5. Use the autoencoder to detect an out-of-distribution image.

Guided Walkthrough

The notebook defines an encoder that compresses each image to a small latent vector and a decoder that reconstructs it. Training minimizes mean squared error between the input and its reconstruction. Visualizing originals above their reconstructions shows what the bottleneck preserved and what it discarded.

The latent space is then projected to two dimensions with principal component analysis and colored by class. Similar garments cluster together, which shows the autoencoder learned structure without labels. Finally, per-image reconstruction errors become an anomaly score: images the model cannot reconstruct well are unusual.

Common Pitfalls

- Making the latent dimension so large that the model copies instead of compressing.

- Judging reconstructions only by the loss value.
- Treating reconstruction error as a probability.

Going Further

- Train a denoising autoencoder by adding noise to the input.
- Use the latent vectors as features for a downstream classifier.
- Detect out-of-distribution images with the reconstruction error.

Exercise Solutions

1. A larger latent dimension reconstructs better but learns a less useful representation.
2. The latent plot shows clusters that align with classes without supervision.
3. The highest-error images are the most unusual or ambiguous.
4. The bottleneck forces the network to keep only the most informative structure.
5. An out-of-distribution image has a much higher reconstruction error than normal images.

Chapter 24: Fashion-MNIST Dataset - GAN

Introduction

A generative adversarial network learns to generate data by pitting two networks against each other. The generator tries to fool the discriminator, and the discriminator tries to tell real images from fake ones. This chapter trains a GAN on Fashion-MNIST and generates clothing images from random noise.

GANs are notoriously finicky, and the chapter is honest about that. The loss curves of a GAN do not behave like those of a classifier, and the right way to judge progress is to look at the generated samples, not at a single number.

What You Will Learn

- How the generator and discriminator are trained against each other.
- How the minimax objective works in practice.
- How to monitor GAN training by inspecting samples.
- How to generate images from noise after training.

Overview

The generator maps a noise vector to an image; the discriminator maps an image to a real-or-fake score. In each step we update the discriminator on a batch of real images and a batch of generated ones, then update the generator to make its fakes look real. We use a binary cross-entropy loss on the discriminator logits.

We track both losses and, more importantly, render sample grids at intervals. Early samples are noise; later samples become recognizable garments. We close by saving the generator and sampling from it after reloading.

Citation

[1] Han Xiao, Kashif Rasul, Roland Vollgraf, <https://github.com/zalandoresearch/fashion-mnist>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib
```

Import Libraries

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42  
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x10d455b50>
```

Parameters

```
IMAGE_SIZE = 28  
IMAGE_SIZE
```

```
28
```

```
LATENT_DIM = 64  
LATENT_DIM
```

```
64
```

```
HIDDEN_DIM = 256  
HIDDEN_DIM
```

```
256
```

Hyperparameters

```
LEARNING_RATE = 0.0002  
LEARNING_RATE
```

```
0.0002
```

```
EPOCHS = 30  
EPOCHS
```

```
30
```

```
BATCH_SIZE = 128  
BATCH_SIZE
```

128

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))])
train_data = datasets.FashionMNIST('data', train=True, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
len(train_data)
```

60000

Create Generator

```
class Generator(nn.Module):
    """
    A generator that maps noise to 28x28 images.
    """

    def __init__(self, latent_dim=LATENT_DIM, hidden_dim=HIDDEN_DIM):
        """
        Initialize the generator layers.

        Parameters:
            latent_dim (int): Size of the noise vector.
            hidden_dim (int): Hidden layer size.

        Returns:
            None
        """
        super(Generator, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, IMAGE_SIZE * IMAGE_SIZE),
            nn.Tanh())

    def forward(self, z):
        """
        Generate a batch of images from noise.

        Parameters:
```

```

        z (torch.Tensor): Noise batch.

    Returns:
        torch.Tensor: Batch of images.
    """
    images = self.net(z)
    return images.view(z.size(0), 1, IMAGE_SIZE, IMAGE_SIZE)

```

Create Discriminator

```

class Discriminator(nn.Module):
    """
    A discriminator that scores images as real or fake.
    """

    def __init__(self, hidden_dim=HIDDEN_DIM):
        """
        Initialize the discriminator layers.

        Parameters:
            hidden_dim (int): Hidden layer size.

        Returns:
            None
        """
        super(Discriminator, self).__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(IMAGE_SIZE * IMAGE_SIZE, hidden_dim),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_dim, hidden_dim),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_dim, 1))

    def forward(self, x):
        """
        Score a batch of images.

        Parameters:
            x (torch.Tensor): Batch of images.

        Returns:
            torch.Tensor: Logits for each image.
        """
        return self.net(x)

```

Instantiate Models, Loss, and Optimizers

```

torch.manual_seed(SEED)
generator = Generator().to(DEVICE)
discriminator = Discriminator().to(DEVICE)
loss_fn = nn.BCEWithLogitsLoss()
opt_g = torch.optim.Adam(generator.parameters(), lr=LEARNING_RATE, betas=(0.5, 0.999))
opt_d = torch.optim.Adam(discriminator.parameters(), lr=LEARNING_RATE, betas=(0.5, 0.999))
generator

```

```

Generator(
  (net): Sequential(

```

```

(0): Linear(in_features=64, out_features=256, bias=True)
(1): ReLU()
(2): Linear(in_features=256, out_features=256, bias=True)
(3): ReLU()
(4): Linear(in_features=256, out_features=784, bias=True)
(5): Tanh()
)
)

```

Train Model

Functions

```

def discriminator_loss(disc, gen, real, loss_fn, latent_dim):
    """
    Compute the discriminator loss on real and fake images.

    Parameters:
        disc (nn.Module): The discriminator.
        gen (nn.Module): The generator.
        real (torch.Tensor): Batch of real images.
        loss_fn (nn.Module): Binary cross-entropy loss.
        latent_dim (int): Size of the noise vector.

    Returns:
        torch.Tensor: The discriminator loss.
    """
    noise = torch.randn(real.size(0), latent_dim, device=real.device)
    fake = gen(noise)
    real_logits = disc(real)
    fake_logits = disc(fake.detach())
    real_loss = loss_fn(real_logits, torch.ones_like(real_logits))
    fake_loss = loss_fn(fake_logits, torch.zeros_like(fake_logits))
    return real_loss + fake_loss


def generator_loss(disc, gen, batch_size, loss_fn, latent_dim):
    """
    Compute the generator loss from the discriminator scores.

    Parameters:
        disc (nn.Module): The discriminator.
        gen (nn.Module): The generator.
        batch_size (int): Number of images to generate.
        loss_fn (nn.Module): Binary cross-entropy loss.
        latent_dim (int): Size of the noise vector.

    Returns:
        torch.Tensor: The generator loss.
    """
    noise = torch.randn(batch_size, latent_dim, device=DEVICE)
    fake_logits = disc(gen(noise))
    return loss_fn(fake_logits, torch.ones_like(fake_logits))


def train_epoch(disc, gen, loader, loss_fn, opt_d, opt_g, latent_dim):
    """
    Train the discriminator and generator for one epoch.

    Parameters:
        disc (nn.Module): The discriminator.

```

```

gen (nn.Module): The generator.
loader (DataLoader): Training data loader.
loss_fn (nn.Module): Binary cross-entropy loss.
opt_d (torch.optim.Optimizer): Discriminator optimizer.
opt_g (torch.optim.Optimizer): Generator optimizer.
latent_dim (int): Size of the noise vector.

Returns:
    tuple: Mean discriminator and generator loss.
"""
disc.train()
gen.train()
total_d, total_g = 0.0, 0.0
for real, _ in loader:
    real = real.to(DEVICE)
    opt_d.zero_grad()
    d_loss = discriminator_loss(disc, gen, real, loss_fn, latent_dim)
    d_loss.backward()
    opt_d.step()
    opt_g.zero_grad()
    g_loss = generator_loss(disc, gen, real.size(0), loss_fn, latent_dim)
    g_loss.backward()
    opt_g.step()
    total_d += d_loss.item() * len(real)
    total_g += g_loss.item() * len(real)
n = len(loader.dataset)
return total_d / n, total_g / n

```

Training Loop

```

history_d = []
history_g = []
for epoch in range(EPOCHS):
    d_loss, g_loss = train_epoch(discriminator, generator, train_loader,
                                loss_fn, opt_d, opt_g, LATENT_DIM)

    history_d.append(d_loss)
    history_g.append(g_loss)
    print(f"Epoch {epoch + 1:2d} | discriminator {d_loss:.4f} | generator {g_loss:.4f}")

```

```
Epoch 1 | discriminator 0.4615 | generator 2.3316
```

```
Epoch 2 | discriminator 0.3134 | generator 3.7573
```

```
Epoch 3 | discriminator 0.3863 | generator 3.6871
```

```
Epoch 4 | discriminator 0.5466 | generator 3.0557
```

```
Epoch 5 | discriminator 0.5778 | generator 2.5289
```

```
Epoch 6 | discriminator 0.6026 | generator 2.6899
```

REVERSE ENGINEERING DATA

Epoch 7 | discriminator 0.6608 | generator 2.6085

Epoch 8 | discriminator 0.7435 | generator 2.3001

Epoch 9 | discriminator 0.7586 | generator 2.1638

Epoch 10 | discriminator 0.7841 | generator 2.1347

Epoch 11 | discriminator 0.7944 | generator 1.9855

Epoch 12 | discriminator 0.8158 | generator 2.0745

Epoch 13 | discriminator 0.8200 | generator 2.0426

Epoch 14 | discriminator 0.8357 | generator 1.9927

Epoch 15 | discriminator 0.8383 | generator 1.9070

Epoch 16 | discriminator 0.8470 | generator 1.9132

Epoch 17 | discriminator 0.8653 | generator 1.8377

Epoch 18 | discriminator 0.8996 | generator 1.7025

Epoch 19 | discriminator 0.9236 | generator 1.6957

Epoch 20 | discriminator 0.9409 | generator 1.6328

Epoch 21 | discriminator 0.9382 | generator 1.6664

Epoch 22 | discriminator 0.9611 | generator 1.5809

Epoch 23 | discriminator 0.9782 | generator 1.5300

Epoch 24 | discriminator 0.9820 | generator 1.5367

Epoch 25 | discriminator 0.9805 | generator 1.4951

Epoch 26 | discriminator 1.0084 | generator 1.4867

Epoch 27 | discriminator 1.0148 | generator 1.4388

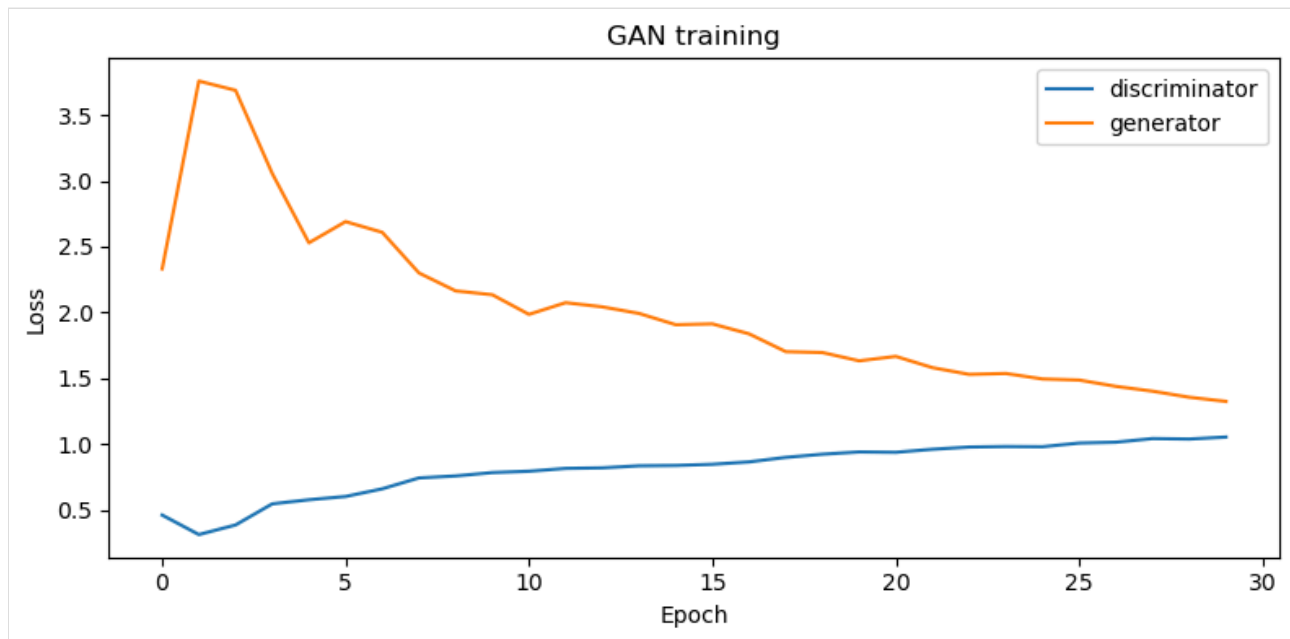
Epoch 28 | discriminator 1.0424 | generator 1.4023

Epoch 29 | discriminator 1.0389 | generator 1.3562

Epoch 30 | discriminator 1.0539 | generator 1.3248

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(history_d, label='discriminator')
plt.plot(history_g, label='generator')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('GAN training')
plt.legend()
plt.tight_layout()
plt.show()
```



Generate Images

Function

```
def show_samples(gen, n=8):
    """
    Plot a grid of generated images.
```



```

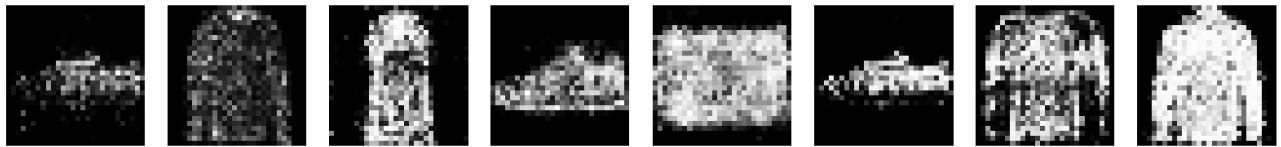
Parameters:
    gen (nn.Module): Trained generator.
    n (int): Number of images to show.

Returns:
    None
"""
gen.eval()
with torch.no_grad():
    noise = torch.randn(n, LATENT_DIM, device=DEVICE)
    images = gen(noise).cpu()
fig, axes = plt.subplots(1, n, figsize=(2 * n, 2))
for i in range(n):
    axes[i].imshow(images[i].squeeze(), cmap='gray')
    axes[i].axis('off')
fig.tight_layout()
plt.show()

```

Run

```
show_samples(generator, n=8)
```



Save Models

```

torch.save(generator.state_dict(), 'models/gan_generator_fashion_mnist.pt')
torch.save(discriminator.state_dict(), 'models/gan_discriminator_fashion_mnist.pt')
print('saved gan models')

```

```
saved gan models
```

Load Models

```

loaded_generator = Generator().to(DEVICE)
loaded_generator.load_state_dict(torch.load('models/gan_generator_fashion_mnist.pt',
map_location=DEVICE))
loaded_generator.eval()
print('loaded gan_generator_fashion_mnist.pt')

```

```
loaded gan_generator_fashion_mnist.pt
```

```

/var/folders/qg/60m3b34x32gcgr10l3665300000gn/T/ipykernel_80838/3183088278.py:2: FutureWarning: You
are using `torch.load` with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data which will execute
arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a
future release, the default value for `weights_only` will be flipped to `True`. This limits the
functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be

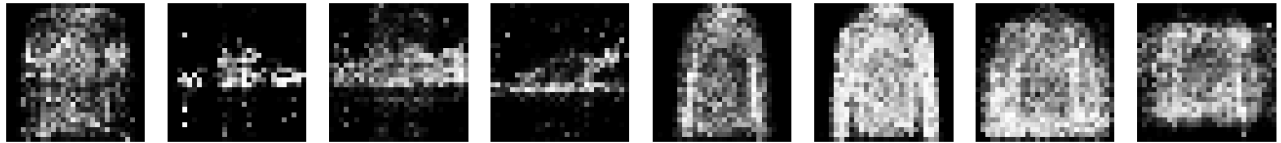
```

```
loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use
case where you don't have full control of the loaded file. Please open an issue on GitHub for any
issues related to this experimental feature.
```

```
loaded_generator.load_state_dict(torch.load('models/gan_generator_fashion_mnist.pt',
map_location=DEVICE))
```

Inference

```
show_samples(loaded_generator, n=8)
```



Key Takeaways

- A GAN trains a generator and a discriminator against each other.
- GAN losses are not interpretable on their own; judge by samples.
- Mode collapse is a real failure mode.
- The generator is the artifact you keep.

Exercises

1. Train and render sample grids at several epochs.
2. Change the latent dimension and compare samples.
3. Describe what mode collapse would look like.
4. Explain why the discriminator must not become too strong.
5. Generate a grid from the saved generator.

Guided Walkthrough

The notebook defines a generator that maps noise to an image and a discriminator that scores images as real or fake. Each step updates the discriminator on real and generated batches, then updates the generator to fool it. The losses are tracked, but the real measure of progress is the sample grid: early samples are noise and later samples are recognizable garments.

The chapter is honest about how finicky GANs are. The losses do not fall the way a classifier's do, and the discriminator must not become so strong that the generator receives no useful signal. The generator is the artifact worth keeping, and the notebook saves it and samples from it after reloading.

Common Pitfalls

- Judging a GAN by its loss alone.
- Letting the discriminator overpower the generator.
- Suffering mode collapse, where the generator produces the same image repeatedly.

Going Further

- Change the latent dimension and compare the diversity of samples.
- Add label conditioning to generate specific classes.
- Compare the GAN with the autoencoder's reconstructions.

Exercise Solutions

1. Sample grids improve from noise to recognizable clothing over training.
2. A larger latent dimension gives more variety but can be harder to train.
3. Mode collapse looks like many samples that are nearly identical.
4. A too-strong discriminator gives the generator a vanishing gradient.
5. The reloaded generator produces a fresh grid of plausible images.

Chapter 25: Fashion-MNIST Dataset - Vision Transformer

Introduction

The vision transformer applies the transformer architecture to images by cutting an image into patches, embedding each patch, and treating the sequence of patches like a sentence. This chapter builds a small vision transformer from scratch and trains it on Fashion-MNIST. It connects the attention machinery of Part V to the image data of Part IV.

The chapter also teaches a practical lesson about hardware. The first implementation used a library attention layer that hit a limitation on the Apple GPU, so we wrote attention explicitly with the scaled dot-product primitive, which is both faster and more portable.

What You Will Learn

- How to turn an image into a sequence of patch embeddings.
- How the class token and positional embeddings work.
- How a transformer encoder block classifies images.
- How to train a vision transformer end to end.
- Why writing attention explicitly can be more portable than a library call.

Overview

We split each 28 by 28 image into 4 by 4 patches, flatten each patch, and project it to an embedding. A learned class token is prepended, positional embeddings are added, and the sequence passes through a stack of encoder blocks. The final representation of the class token feeds a linear classifier.

Each encoder block is multi-head self-attention followed by a feed-forward network, with residual connections and layer normalization. We train with Adam and cross-entropy, evaluate with a confusion matrix, and then classify a test image. The architecture is the same family as the transformer in Part V, applied to pixels instead of characters.

Citation

[1] Han Xiao, Kashif Rasul, Roland Vollgraf, <https://github.com/zalandoresearch/fashion-mnist>

[2] Alexey Dosovitskiy et al., <https://arxiv.org/abs/2010.11929>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib scikit-learn
```

Import Libraries

```
import numpy as np
import torch
```

```
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x115349b30>
```

Parameters

```
IMAGE_SIZE = 28
IMAGE_SIZE
```

```
28
```

```
PATCH_SIZE = 4
PATCH_SIZE
```

```
4
```

```
EMBED_DIM = 64
EMBED_DIM
```

```
64
```

```
N_HEADS = 4
N_HEADS
```

```
4
```

REVERSE ENGINEERING DATA

```
N_LAYERS = 4
N_LAYERS
```

```
4
```

```
FF_DIM = 128
FF_DIM
```

```
128
```

```
NUM_CLASSES = 10
NUM_CLASSES
```

```
10
```

```
CLASS_NAMES = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
CLASS_NAMES
```

```
['T-shirt/top',
 'Trouser',
 'Pullover',
 'Dress',
 'Coat',
 'Sandal',
 'Shirt',
 'Sneaker',
 'Bag',
 'Ankle boot']
```

Hyperparameters

```
LEARNING_RATE = 0.001
LEARNING_RATE
```

```
0.001
```

```
EPOCHS = 8
EPOCHS
```

```
8
```

```
BATCH_SIZE = 128
BATCH_SIZE
```

128

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.2860,), (0.3530,))]
train_data = datasets.FashionMNIST('data', train=True, download=True, transform=transform)
test_data = datasets.FashionMNIST('data', train=False, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_data), len(test_data)
```

```
(60000, 10000)
```

Create Patch Embedding

The image is cut into non-overlapping patches. Each patch is flattened and projected to an embedding vector by a linear layer.

```
class PatchEmbedding(nn.Module):
    """
    Split an image into patches and embed each patch with a linear layer.
    """

    def __init__(self, embed_dim=EMBED_DIM, patch_size=PATCH_SIZE):
        """
        Initialize the patch projection.

        Parameters:
            embed_dim (int): Embedding size per patch.
            patch_size (int): Side length of each patch.

        Returns:
            None
        """
        super(PatchEmbedding, self).__init__()
        self.patch_size = patch_size
        self.proj = nn.Linear(patch_size * patch_size, embed_dim)

    def forward(self, x):
        """
        Embed a batch of grayscale images.
```

```

Parameters:
    x (torch.Tensor): Batch of images.

Returns:
    torch.Tensor: Patch embeddings.
"""
batch, channels, height, width = x.shape
p = self.patch_size
x = x.reshape(batch, channels, height // p, p, width // p, p)
x = x.permute(0, 2, 4, 1, 3, 5).reshape(batch, (height // p) * (width // p), -1)
return self.proj(x)

```

Create Transformer Encoder

```

class Attention(nn.Module):
    """
    Multi-head self-attention built from explicit tensor operations.
    """

    def __init__(self, embed_dim=EMBED_DIM, n_heads=N_HEADS):
        """
        Initialize the query, key, value, and output projections.

        Parameters:
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.

        Returns:
            None
        """
        super(Attention, self).__init__()
        self.n_heads = n_heads
        self.head_dim = embed_dim // n_heads
        self.qkv = nn.Linear(embed_dim, 3 * embed_dim)
        self.proj = nn.Linear(embed_dim, embed_dim)

    def forward(self, x):
        """
        Run multi-head self-attention.

        Parameters:
            x (torch.Tensor): Input of shape (batch, tokens, embed_dim).

        Returns:
            torch.Tensor: Attention output.
        """
        batch, tokens, embed_dim = x.shape
        qkv = self.qkv(x).reshape(batch, tokens, 3, self.n_heads, self.head_dim)
        query, key, value = qkv.permute(2, 0, 3, 1, 4)
        out = F.scaled_dot_product_attention(query, key, value)
        out = out.transpose(1, 2).reshape(batch, tokens, embed_dim)
        return self.proj(out)

```

```

class EncoderBlock(nn.Module):
    """
    A transformer encoder block with attention and a feed-forward network.
    """

    def __init__(self, embed_dim=EMBED_DIM, n_heads=N_HEADS, ff_dim=FF_DIM):

```



```

"""
Initialize attention, normalization, and feed-forward layers.

Parameters:
    embed_dim (int): Embedding size.
    n_heads (int): Number of attention heads.
    ff_dim (int): Feed-forward hidden size.

Returns:
    None
"""
super(EncoderBlock, self).__init__()
self.attn = Attention(embed_dim, n_heads)
self.norm1 = nn.LayerNorm(embed_dim)
self.ff = nn.Sequential(
    nn.Linear(embed_dim, ff_dim),
    nn.GELU(),
    nn.Linear(ff_dim, embed_dim))
self.norm2 = nn.LayerNorm(embed_dim)

def forward(self, x):
    """
    Run the encoder block with residual connections.

    Parameters:
        x (torch.Tensor): Input of shape (batch, tokens, embed_dim).

    Returns:
        torch.Tensor: Output of the same shape.
    """
    x = self.norm1(x + self.attn(x))
    return self.norm2(x + self.ff(x))

```

Create Vision Transformer

```

class ViT(nn.Module):
    """
    A small vision transformer for 28x28 grayscale images.
    """

    def __init__(self, embed_dim=EMBED_DIM, n_heads=N_HEADS, n_layers=N_LAYERS,
                 ff_dim=FF_DIM, patch_size=PATCH_SIZE, num_classes=NUM_CLASSES):
        """
        Initialize patch embedding, tokens, blocks, and the head.

        Parameters:
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.
            n_layers (int): Number of encoder blocks.
            ff_dim (int): Feed-forward hidden size.
            patch_size (int): Side length of each patch.
            num_classes (int): Number of output classes.

        Returns:
            None
        """
        super(ViT, self).__init__()
        self.patch = PatchEmbedding(embed_dim, patch_size)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        self.pos_embed = nn.Parameter(
            torch.zeros(1, (IMAGE_SIZE // patch_size) ** 2 + 1, embed_dim))

```

```

self.blocks = nn.Sequential(*[
    EncoderBlock(embed_dim, n_heads, ff_dim) for _ in range(n_layers)])
self.norm = nn.LayerNorm(embed_dim)
self.head = nn.Linear(embed_dim, num_classes)

def forward(self, x):
    """
    Classify a batch of images.

    Parameters:
        x (torch.Tensor): Batch of images.

    Returns:
        torch.Tensor: Output logits.
    """
    x = self.patch(x)
    cls = self.cls_token.expand(x.size(0), -1, -1)
    x = torch.cat([cls, x], dim=1) + self.pos_embed
    x = self.blocks(x)
    return self.head(self.norm(x[:, 0]))

```

Instantiate Model, Loss, and Optimizer

```

torch.manual_seed(SEED)
model = ViT().to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
sum(p.numel() for p in model.parameters())

```

139018

Train Model

Functions

```

def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)

```

```
def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        tuple: Mean loss and accuracy.
    """
    model.eval()
    total, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            total += loss_fn(logits, y).item() * len(y)
            correct += (logits.argmax(1) == y).sum().item()
    n = len(loader.dataset)
    return total / n, correct / n
```

Training Loop

```
history = {'loss': [], 'accuracy': []}
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    test_loss, test_accuracy = evaluate(model, test_loader, loss_fn)
    history['loss'].append(test_loss)
    history['accuracy'].append(test_accuracy)
    print(f"Epoch {epoch + 1} | train loss {train_loss:.4f} | test accuracy {test_accuracy:.4f}")
```

Epoch 1 | train loss 0.7988 | test accuracy 0.8157

Epoch 2 | train loss 0.4420 | test accuracy 0.8441

Epoch 3 | train loss 0.3857 | test accuracy 0.8607

Epoch 4 | train loss 0.3518 | test accuracy 0.8652

Epoch 5 | train loss 0.3254 | test accuracy 0.8710

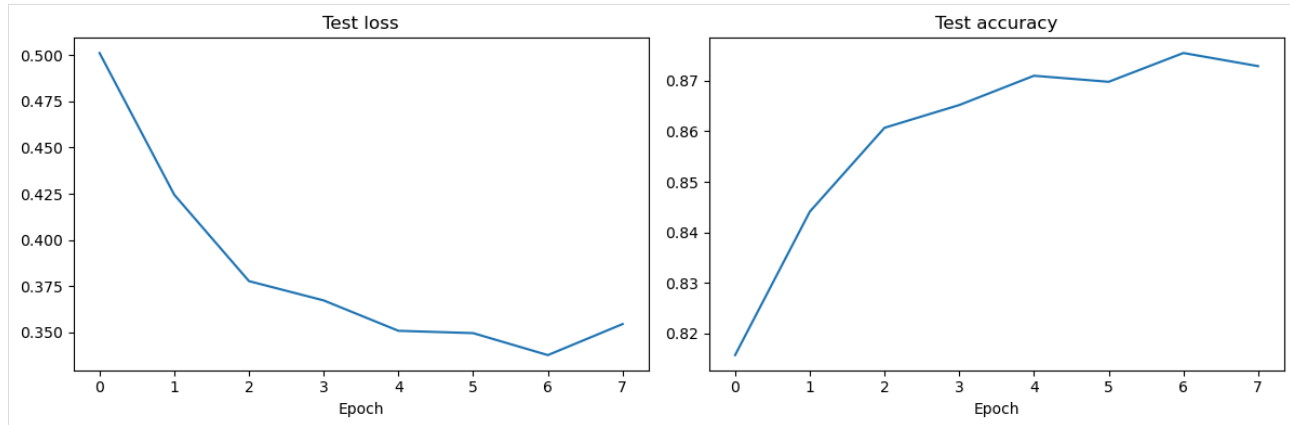
Epoch 6 | train loss 0.3059 | test accuracy 0.8698

Epoch 7 | train loss 0.2883 | test accuracy 0.8755

Epoch 8 | train loss 0.2745 | test accuracy 0.8729

Visualize Training

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(history['loss'])
axes[0].set_title('Test loss')
axes[0].set_xlabel('Epoch')
axes[1].plot(history['accuracy'])
axes[1].set_title('Test accuracy')
axes[1].set_xlabel('Epoch')
fig.tight_layout()
plt.show()
```



Evaluate Model

```
_, accuracy = evaluate(model, test_loader, loss_fn)
print(f"Test accuracy: {accuracy:.4f}")
```

Test accuracy: 0.8729

```
all_preds = []
all_labels = []
model.eval()
with torch.no_grad():
    for x, y in test_loader:
        x = x.to(DEVICE)
        all_preds.append(model(x).argmax(1).cpu().numpy())
        all_labels.append(y.numpy())
all_preds = np.concatenate(all_preds)
all_labels = np.concatenate(all_labels)
print(confusion_matrix(all_labels, all_preds))
print(classification_report(all_labels, all_preds, target_names=CLASS_NAMES))
```

```
[[814  2 12  6  1  1 159  0  5  0]
 [  4 981  1  5  4  0  4  0  1  0]
 [ 23  1 814  4 51  0 105  0  2  0]
 [ 51 13  8 805 33  0  90  0  0  0]
 [  0  0 123 18 739  0 118  0  2  0]
 [  1  0  0  0  0 930  1  53  7  8]
 [111  3  56 10  45  0 768  0  7  0]
 [  0  0  0  0  0  6  0 979  1 14]]
```

	[8 2 1 4 2 3 20 3 957 0]			
	[1 0 0 0 0 3 0 54 0 942]]			
	precision	recall	f1-score	support
T-shirt/top	0.80	0.81	0.81	1000
Trouser	0.98	0.98	0.98	1000
Pullover	0.80	0.81	0.81	1000
Dress	0.94	0.81	0.87	1000
Coat	0.84	0.74	0.79	1000
Sandal	0.99	0.93	0.96	1000
Shirt	0.61	0.77	0.68	1000
Sneaker	0.90	0.98	0.94	1000
Bag	0.97	0.96	0.97	1000
Ankle boot	0.98	0.94	0.96	1000
accuracy			0.87	10000
macro avg	0.88	0.87	0.88	10000
weighted avg	0.88	0.87	0.88	10000

Save Model

```
torch.save(model.state_dict(), 'models/vit_fashion_mnist.pt')
print('saved vit_fashion_mnist.pt')
```

```
saved vit_fashion_mnist.pt
```

Load Model

```
loaded_model = ViT().to(DEVICE)
loaded_model.load_state_dict(torch.load('models/vit_fashion_mnist.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded vit_fashion_mnist.pt')
```

```
loaded vit_fashion_mnist.pt
```

/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_80983/1456870769.py:2: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
loaded_model.load_state_dict(torch.load('models/vit_fashion_mnist.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, image):
    """
    Predict the class of one normalized image tensor.

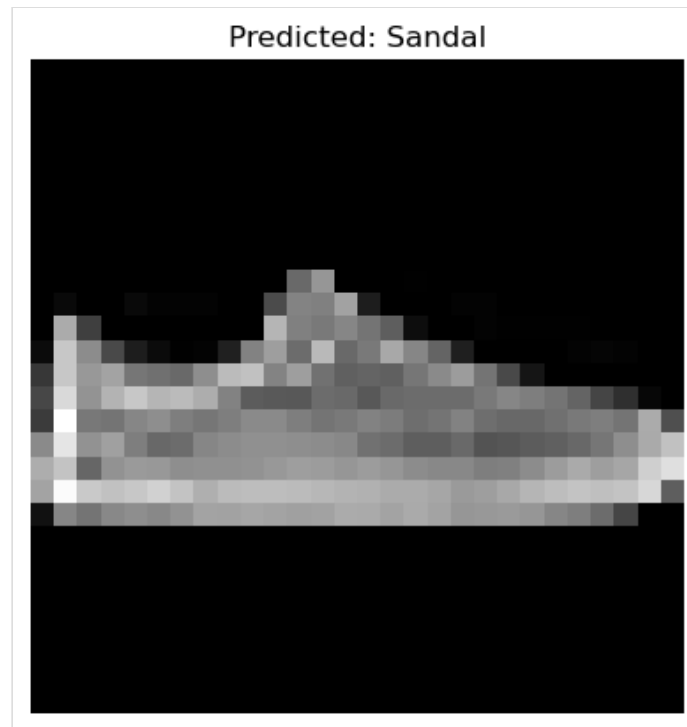
    Parameters:
        model (nn.Module): Trained model.
        image (torch.Tensor): A 1x28x28 image tensor.

    Returns:
        tuple: Predicted class name and confidence.
    """
    model.eval()
    with torch.no_grad():
        logits = model(image.unsqueeze(0).to(DEVICE))
        probs = torch.softmax(logits, dim=1)
        confidence, predicted = probs.max(dim=1)
    return CLASS_NAMES[predicted.item()], confidence.item()
```

Run Inference

```
image, label = test_data[12]
name, confidence = predict(loader_model, image)
print(f"True: {CLASS_NAMES[label]} | Predicted: {name} ({confidence:.4f})")
plt.imshow(image.squeeze(), cmap='gray')
plt.title(f"Predicted: {name}")
plt.axis('off')
plt.show()
```

```
True: Sneaker | Predicted: Sandal (0.6848)
```



Key Takeaways

- Images become sequences of patch embeddings for a transformer.
- A class token aggregates the sequence into one representation.
- Encoder blocks combine attention and a feed-forward network.
- Explicit attention can be more portable than a library layer.

Exercises

1. Change the patch size and compare accuracy and speed.
2. Remove the class token and average the patches instead.
3. Increase the number of encoder blocks and compare.
4. Visualize the attention of the class token over patches.
5. Explain how a vision transformer differs from a convolutional network.

Guided Walkthrough

The notebook cuts each image into patches, flattens and projects each patch to an embedding, prepends a class token, adds positional embeddings, and passes the sequence through a stack of encoder blocks. Each block is multi-head self-attention followed by a feed-forward network, with residual connections and layer normalization. The class token's final representation feeds a linear classifier.

The chapter also teaches a practical lesson about hardware: the first implementation used a library attention layer that hit a limitation on the Apple GPU, so attention was written explicitly with the scaled dot-product primitive. The explicit version is both faster and more portable, which is a reminder that knowing the mechanism lets you fix it.

Common Pitfalls

- Forgetting positional embeddings, which leaves the model blind to patch order.
- Getting the patch count wrong, which breaks the class token concatenation.
- Assuming a library layer is always the fastest or most portable choice.

Going Further

- Change the patch size and compare accuracy and speed.
- Visualize the class token's attention over the patches.
- Compare the vision transformer with the convolutional network from the earlier chapter.

Exercise Solutions

1. Smaller patches give finer detail but more tokens and slower training.
2. Averaging the patches instead of a class token works but is usually slightly worse.
3. More encoder blocks increase capacity and accuracy up to a point.
4. The class token tends to attend most strongly to the object region.
5. A vision transformer has no built-in locality, so it must learn spatial structure from data.

Part VIII: Segmentation, Reinforcement Learning, and Deployment

Chapter 26: Fashion-MNIST Dataset - Model Deployment

Introduction

A model that lives only inside a notebook is not a product. Deployment is the step that turns a trained network into something other software can use. This chapter trains a convolutional classifier on Fashion-MNIST, exports it to ONNX, verifies that the exported model produces the same outputs as the PyTorch original, wraps it in a reusable inference engine, and exposes it through a command-line interface.

ONNX is the lingua franca of model exchange. Exporting to it means the model can run in runtimes that have nothing to do with PyTorch, including on edge devices and in web services. The verification step, checking that the exported model matches the original, is the part beginners skip and professionals never do.

What You Will Learn

- Why a portable model format matters.
- How to export a PyTorch model to ONNX.
- How to verify that the exported model matches the original.
- How to build a reusable inference engine around an ONNX session.
- How to expose the engine through a command-line interface.

Overview

We train a small convolutional network, save its PyTorch parameters, and export it to ONNX with a dummy input that fixes the graph. We then run the same image through both models and compare the outputs; a tiny maximum difference confirms the export is faithful.

With the ONNX model in hand we build an inference engine: a small class that loads the ONNX session, accepts an image tensor, and returns a class name and confidence. Finally we add a command-line entry point so the engine can be run from a terminal, which is the smallest possible deployment.

Citation

[1] Han Xiao, Kashif Rasul, Roland Vollgraf, <https://github.com/zalando-research/fashion-mnist>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install -c conda-forge onnx onnxruntime
# conda install numpy matplotlib
```

Import Libraries

```
import argparse
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
```

```
from torchvision import datasets
from torchvision import transforms
import onnxruntime
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x125149b30>
```

Parameters

```
IMAGE_SIZE = 28
IMAGE_SIZE
```

```
28
```

```
NUM_CLASSES = 10
NUM_CLASSES
```

```
10
```

```
CLASS_NAMES = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
CLASS_NAMES
```

```
['T-shirt/top',
 'Trouser',
 'Pullover',
 'Dress',
 'Coat',
 'Sandal',
 'Shirt',
 'Sneaker',
 'Bag',
 'Ankle boot']
```

Hyperparameters

```
LEARNING_RATE = 0.001
LEARNING_RATE
```

```
0.001
```

```
EPOCHS = 3
EPOCHS
```

```
3
```

```
BATCH_SIZE = 128
BATCH_SIZE
```

```
128
```

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.2860,), (0.3530,))]
train_data = datasets.FashionMNIST('data', train=True, download=True, transform=transform)
test_data = datasets.FashionMNIST('data', train=False, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_data), len(test_data)
```

```
(60000, 10000)
```

Create Model

```
class CNN(nn.Module):
    """
    A small convolutional neural network for 28x28 grayscale images.
    """

    def __init__(self, num_classes=NUM_CLASSES):
```

```

"""
Initialize the convolutional and dense layers.

Parameters:
    num_classes (int): Number of output classes.

Returns:
    None
"""
super(CNN, self).__init__()
self.features = nn.Sequential(
    nn.Conv2d(1, 16, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(16, 32, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2))
self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Linear(32 * 7 * 7, 128),
    nn.ReLU(),
    nn.Linear(128, num_classes))

def forward(self, x):
    """
    Run the forward pass of the network.

    Parameters:
        x (torch.Tensor): Batch of images.

    Returns:
        torch.Tensor: Output logits.
    """
    x = self.features(x)
    return self.classifier(x)

```

Instantiate Model, Loss, and Optimizer

```

torch.manual_seed(SEED)
model = CNN().to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
model

```

```

CNN(
  (features): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=1568, out_features=128, bias=True)
    (2): ReLU()
    (3): Linear(in_features=128, out_features=10, bias=True)
  )

```

```
)  
)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):  
    """  
    Train the model for a single epoch.  
  
    Parameters:  
        model (nn.Module): The model to train.  
        loader (DataLoader): Training data loader.  
        loss_fn (nn.Module): Loss function.  
        optimizer (torch.optim.Optimizer): Parameter update rule.  
  
    Returns:  
        float: Mean training loss.  
    """  
    model.train()  
    total = 0.0  
    for x, y in loader:  
        x, y = x.to(DEVICE), y.to(DEVICE)  
        optimizer.zero_grad()  
        loss = loss_fn(model(x), y)  
        loss.backward()  
        optimizer.step()  
        total += loss.item() * len(y)  
    return total / len(loader.dataset)  
  
def evaluate(model, loader, loss_fn):  
    """  
    Evaluate the model over a loader.  
  
    Parameters:  
        model (nn.Module): The model to evaluate.  
        loader (DataLoader): Evaluation data loader.  
        loss_fn (nn.Module): Loss function.  
  
    Returns:  
        tuple: Mean loss and accuracy.  
    """  
    model.eval()  
    total, correct = 0.0, 0  
    with torch.no_grad():  
        for x, y in loader:  
            x, y = x.to(DEVICE), y.to(DEVICE)  
            logits = model(x)  
            total += loss_fn(logits, y).item() * len(y)  
            correct += (logits.argmax(1) == y).sum().item()  
    n = len(loader.dataset)  
    return total / n, correct / n
```

Training Loop

```
for epoch in range(EPOCHS):  
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
```

```
test_loss, test_accuracy = evaluate(model, test_loader, loss_fn)
print(f"Epoch {epoch + 1} | train loss {train_loss:.4f} | test accuracy {test_accuracy:.4f}")
```

```
Epoch 1 | train loss 0.4970 | test accuracy 0.8677
```

```
Epoch 2 | train loss 0.3266 | test accuracy 0.8844
```

```
Epoch 3 | train loss 0.2806 | test accuracy 0.8886
```

Save PyTorch Model

```
torch.save(model.state_dict(), 'models/deploy_cnn_fashion_mnist.pt')
print('saved deploy_cnn_fashion_mnist.pt')
```

```
saved deploy_cnn_fashion_mnist.pt
```

Export to ONNX

ONNX is a portable model format. Exporting makes the model usable outside PyTorch, including in services and on edge devices.

```
def export_onnx(model, path):
    """
    Export a PyTorch model to ONNX.

    Parameters:
        model (nn.Module): The trained model.
        path (str): Output ONNX path.

    Returns:
        None
    """
    model.eval()
    dummy = torch.randn(1, 1, IMAGE_SIZE, IMAGE_SIZE, device=DEVICE)
    torch.onnx.export(
        model, dummy, path,
        input_names=['input'], output_names=['logits'],
        dynamic_axes={'input': {0: 'batch'}, 'logits': {0: 'batch'}},
        opset_version=13)
```

Run

```
export_onnx(model, 'models/cnn_fashion_mnist.onnx')
print('saved cnn_fashion_mnist.onnx')
```

```
saved cnn_fashion_mnist.onnx
```

Verify the ONNX Model

The ONNX output must match the PyTorch output. A close match confirms the export is faithful.

```
def onnx_logits(path, image):
    """
    Run inference with an ONNX model.

    Parameters:
        path (str): ONNX model path.
        image (numpy.ndarray): A 1x1x28x28 array.

    Returns:
        numpy.ndarray: Output logits.
    """
    session = onnxruntime.InferenceSession(path, providers=['CPUExecutionProvider'])
    return session.run(None, {'input': image.astype(np.float32)})[0]
```

```
image, label = test_data[0]
array = image.unsqueeze(0).numpy()
torch_logits = model(image.unsqueeze(0).to(DEVICE)).detach().cpu().numpy()
onnx_out = onnx_logits('models/cnn_fashion_mnist.onnx', array)
print('max difference:', np.abs(torch_logits - onnx_out).max())
```

```
max difference: 1.9073486e-06
```

Build an Inference Engine

```
class InferenceEngine:
    """
    A small inference engine wrapping an ONNX model.
    """

    def __init__(self, path):
        """
        Load the ONNX session.

        Parameters:
            path (str): ONNX model path.

        Returns:
            None
        """
        self.session = onnxruntime.InferenceSession(path, providers=['CPUExecutionProvider'])
        self.class_names = CLASS_NAMES

    def predict(self, image):
        """
        Predict the class for one image tensor.

        Parameters:
            image (torch.Tensor): A 1x28x28 image tensor.

        Returns:
            tuple: Predicted class name and confidence.
        """
        array = image.unsqueeze(0).numpy().astype(np.float32)
        logits = self.session.run(None, {'input': array})[0]
        probs = torch.softmax(torch.tensor(logits), dim=1)
        confidence, predicted = probs.max(dim=1)
        return self.class_names[predicted.item()], confidence.item()
```


Inference

```
engine = InferenceEngine('models/cnn_fashion_mnist.onnx')
image, label = test_data[3]
name, confidence = engine.predict(image)
print(f"True: {CLASS_NAMES[label]} | Predicted: {name} ({confidence:.4f})")
```

```
True: Trouser | Predicted: Trouser (1.0000)
```

Command-Line Interface

A command-line entry point turns the notebook into a small tool you can run from the terminal.

```
def build_parser():
    """
    Build the command-line argument parser.

    Parameters:
        None

    Returns:
        argparse.ArgumentParser: The configured parser.
    """
    parser = argparse.ArgumentParser(description='Classify a Fashion-MNIST image.')
    parser.add_argument('--index', type=int, default=0, help='Test image index.')
    return parser

def main():
    """
    Run the command-line inference engine.

    Parameters:
        None

    Returns:
        None
    """
    args = build_parser().parse_args()
    engine = InferenceEngine('models/cnn_fashion_mnist.onnx')
    image, label = test_data[args.index]
    name, confidence = engine.predict(image)
    print(f"Index {args.index} | true {CLASS_NAMES[label]} | predicted {name} ({confidence:.4f})")
```

Run the CLI

```
import sys
sys.argv = ['deploy', '--index', '3']
main()
```

```
Index 3 | true Trouser | predicted Trouser (1.0000)
```

Key Takeaways

- ONNX makes a trained model portable across runtimes.
- Always verify that the exported model matches the original.

- Wrap inference in a class so it is reusable.
- A command-line interface is the simplest deployment.

Exercises

1. Export the model and report the maximum output difference.
2. Run the engine on ten test images and report the accuracies.
3. Add a confidence threshold that returns "unknown" below it.
4. Extend the CLI to accept an image path.
5. Explain what could cause the exported model to differ from the original.

Guided Walkthrough

The notebook trains a small convolutional classifier, saves its PyTorch parameters, and exports it to ONNX with a dummy input that fixes the graph. It then runs the same image through both models and compares the outputs; a tiny maximum difference confirms the export is faithful. That verification step is the one beginners skip and professionals never do.

With the ONNX model in hand the notebook builds an inference engine, a small class that loads the session and returns a class name and confidence, and exposes it through a command-line interface. The engine is the smallest possible deployment and the pattern behind every model service.

Common Pitfalls

- Exporting without verifying that the outputs match.
- Forgetting to call `eval()` before export, which bakes in training behavior.
- Hard-coding file paths that break when the model moves.

Going Further

- Add a confidence threshold that returns unknown below it.
- Extend the CLI to accept an image path.
- Benchmark the ONNX runtime against PyTorch.

Exercise Solutions

1. The maximum output difference is tiny, on the order of floating-point error.
2. The engine reproduces the PyTorch predictions on the test images.
3. A threshold that returns unknown below a confidence level reduces confident errors.
4. The CLI reads an image, preprocesses it, and prints the predicted class.
5. Differences can come from unsupported operators or numerical precision.

Chapter 27: Oxford-IIIT Pet Dataset - Semantic Segmentation

Introduction

Classification labels a whole image; segmentation labels every pixel. It is the denser, harder vision task behind medical imaging, autonomous driving, and satellite analysis. This chapter builds a U-Net, the standard segmentation architecture, and trains it on the Oxford-IIIT Pet dataset to label each pixel as pet, background, or boundary.

The U-Net's design is elegant: an encoder compresses the image into a compact representation, a decoder upsamples it back to full resolution, and skip connections pass fine detail from the encoder to the decoder so the output stays sharp.

What You Will Learn

- How semantic segmentation differs from classification.
- How a U-Net combines downsampling and upsampling with skip connections.
- How to prepare image and mask pairs for training.
- How to measure segmentation quality with pixel accuracy.
- How to visualize predicted masks against the truth.

Overview

We load the Oxford-IIIT Pet dataset with its segmentation masks, resize images and masks to a common size, and shift the raw mask values into class indices. The model is a small U-Net: two encoder blocks, a bottleneck, and a decoder that upsamples and concatenates the encoder features before a final convolution and an interpolation back to the input resolution.

We train with cross-entropy over the pixels and track pixel accuracy. Visualization is essential here: we plot the image, the true mask, and the predicted mask side by side, which shows immediately whether the model has learned the shape of the animal.

Citation

[1] Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman, C. V. Jawahar,
<https://www.robots.ox.ac.uk/~vgg/data/pets/>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib
```

Import Libraries

```
import numpy as np
import torch
import torch.nn as nn
```

```
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torch.utils.data import Subset
from torchvision import datasets
from torchvision import transforms
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x122b49b30>
```

Parameters

```
IMAGE_SIZE = 128
IMAGE_SIZE
```

```
128
```

```
NUM_CLASSES = 3
NUM_CLASSES
```

```
3
```

```
BASE_CHANNELS = 16
BASE_CHANNELS
```

```
16
```

```
SUBSET_SIZE = 2000
SUBSET_SIZE
```

```
2000
```

Hyperparameters

```
LEARNING_RATE = 0.001
LEARNING_RATE
```

```
0.001
```

```
EPOCHS = 8
EPOCHS
```

```
8
```

```
BATCH_SIZE = 16
BATCH_SIZE
```

```
16
```

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Load Dataset

Each image comes with a segmentation mask that labels every pixel as pet, background, or boundary. The raw mask values are 1, 2, and 3, so we shift them to 0, 1, and 2.

```
def mask_transform(mask):
    """
    Resize a raw mask and shift its values into class indices.

    Parameters:
        mask (PIL.Image): The raw segmentation mask.

    Returns:
        torch.Tensor: A mask with values in 0, 1, 2.
    """
    mask = transforms.functional.resize(
        mask, [IMAGE_SIZE, IMAGE_SIZE],
        interpolation=transforms.InterpolationMode.NEAREST)
    mask = torch.as_tensor(np.array(mask), dtype=torch.long)
    return (mask - 1).clamp(0, NUM_CLASSES - 1)
```

```

image_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor()])
full_data = datasets.OxfordIIITPet('data', split='trainval', download=True,
                                   transform=image_transform,
                                   target_types='segmentation',
                                   target_transform=mask_transform)
data = Subset(full_data, range(min(SUBSET_SIZE, len(full_data))))
len(data)

```

2000

Create DataLoaders

```

split = int(0.8 * len(data))
train_data = Subset(data, range(split))
test_data = Subset(data, range(split, len(data)))
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_data), len(test_data)

```

(1600, 400)

Create Model

A U-Net first downsamples the image into a compact representation, then upsamples it back while reusing the encoder features to keep fine detail.

```

class DoubleConv(nn.Module):
    """
    A block of two convolutions with ReLU activations.
    """

    def __init__(self, in_channels, out_channels):
        """
        Initialize the two convolutions.

        Parameters:
            in_channels (int): Number of input channels.
            out_channels (int): Number of output channels.

        Returns:
            None
        """
        super(DoubleConv, self).__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(out_channels, out_channels, 3, padding=1),
            nn.ReLU())

    def forward(self, x):
        """
        Apply the two convolutions.

        Parameters:

```

```

        x (torch.Tensor): Input feature map.

    Returns:
        torch.Tensor: Output feature map.
    """
    return self.net(x)

```

```

class UNet(nn.Module):
    """
    A small U-Net for semantic segmentation.
    """

    def __init__(self, in_channels=3, num_classes=NUM_CLASSES, base=BASE_CHANNELS):
        """
        Initialize the encoder, bottleneck, decoder, and head.

        Parameters:
            in_channels (int): Number of input channels.
            num_classes (int): Number of output classes.
            base (int): Number of channels in the first block.

        Returns:
            None
        """
        super(UNet, self).__init__()
        self.enc1 = DoubleConv(in_channels, base)
        self.enc2 = DoubleConv(base, base * 2)
        self.bottleneck = DoubleConv(base * 2, base * 4)
        self.up = nn.ConvTranspose2d(base * 4, base * 2, 2, stride=2)
        self.dec = DoubleConv(base * 4, base * 2)
        self.head = nn.Conv2d(base * 2, num_classes, 1)

    def forward(self, x):
        """
        Segment a batch of images.

        Parameters:
            x (torch.Tensor): Batch of images.

        Returns:
            torch.Tensor: Per-pixel logits.
        """
        e1 = self.enc1(x)
        e2 = self.enc2(F.max_pool2d(e1, 2))
        bottleneck = self.bottleneck(F.max_pool2d(e2, 2))
        decoded = self.dec(torch.cat([self.up(bottleneck), e2], dim=1))
        logits = self.head(decoded)
        return F.interpolate(logits, size=x.shape[-2:], mode='bilinear',
                             align_corners=False)

```

Instantiate Model, Loss, and Optimizer

```

torch.manual_seed(SEED)
model = UNet().to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
model

```

```

UNet(
  (enc1): DoubleConv(
    (net): Sequential(
      (0): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (2): Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (3): ReLU()
    )
  )
  (enc2): DoubleConv(
    (net): Sequential(
      (0): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (3): ReLU()
    )
  )
  (bottleneck): DoubleConv(
    (net): Sequential(
      (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (3): ReLU()
    )
  )
  (up): ConvTranspose2d(64, 32, kernel_size=(2, 2), stride=(2, 2))
  (dec): DoubleConv(
    (net): Sequential(
      (0): Conv2d(64, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (3): ReLU()
    )
  )
  (head): Conv2d(32, 3, kernel_size=(1, 1), stride=(1, 1))
)

```

Train Model

Functions

```

def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()

```



```
optimizer.step()
total += loss.item() * len(x)
return total / len(loader.dataset)

def pixel_accuracy(model, loader):
    """
    Compute pixel accuracy over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.

    Returns:
        float: Fraction of correctly classified pixels.
    """
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            preds = model(x).argmax(1)
            correct += (preds == y).sum().item()
            total += y.numel()
    return correct / total
```

Training Loop

```
history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    accuracy = pixel_accuracy(model, test_loader)
    history.append(accuracy)
    print(f"Epoch {epoch + 1} | train loss {train_loss:.4f} | pixel accuracy {accuracy:.4f}")
```

Epoch 1 | train loss 0.9341 | pixel accuracy 0.5921

Epoch 2 | train loss 0.8231 | pixel accuracy 0.6015

Epoch 3 | train loss 0.7811 | pixel accuracy 0.6718

Epoch 4 | train loss 0.7528 | pixel accuracy 0.6633

Epoch 5 | train loss 0.7277 | pixel accuracy 0.6352

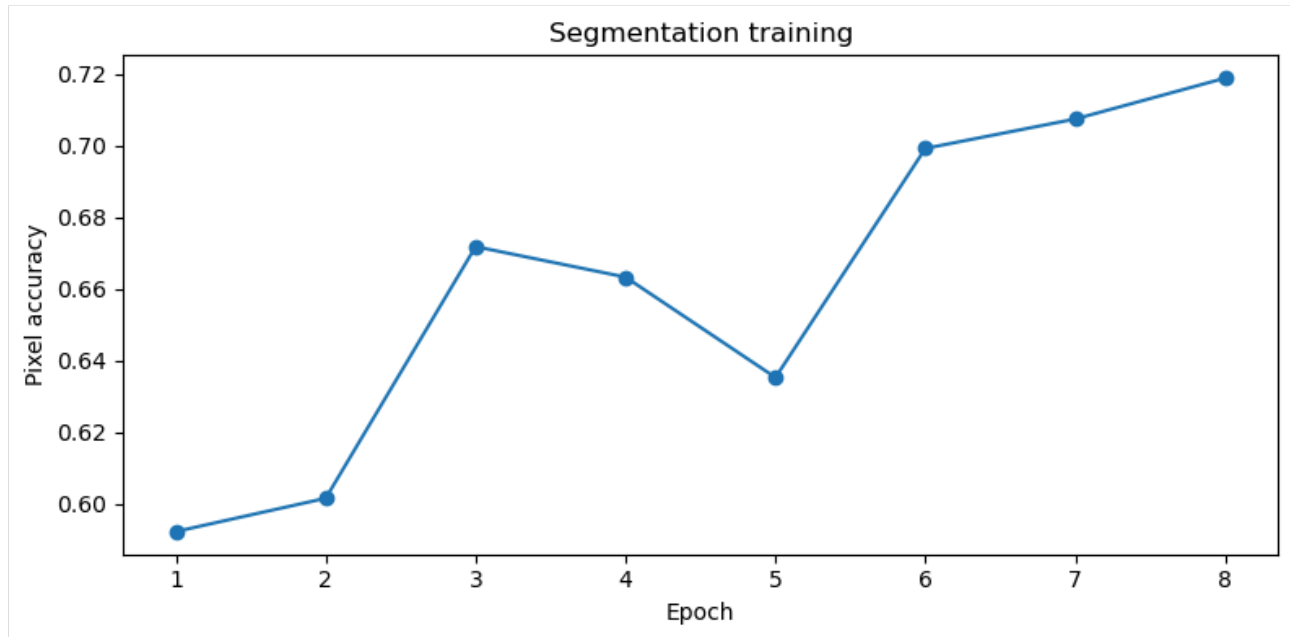
Epoch 6 | train loss 0.7030 | pixel accuracy 0.6993

Epoch 7 | train loss 0.6723 | pixel accuracy 0.7075

Epoch 8 | train loss 0.6469 | pixel accuracy 0.7189

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Pixel accuracy')
plt.title('Segmentation training')
plt.tight_layout()
plt.show()
```



Visualize Predictions

Functions

```
def plot_triplet(image, mask, preds):
    """
    Plot an image, its true mask, and the predicted mask.

    Parameters:
        image (torch.Tensor): The image.
        mask (torch.Tensor): The true mask.
        preds (torch.Tensor): The predicted mask.

    Returns:
        None
    """
    fig, axes = plt.subplots(1, 3, figsize=(12, 4))
    titles = ['Image', 'True mask', 'Predicted mask']
    panels = [image.permute(1, 2, 0), mask, preds]
    for axis, title, panel in zip(axes, titles, panels):
        axis.imshow(panel, cmap=None if panel.dim() == 3 else 'viridis')
        axis.set_title(title)
        axis.axis('off')
    fig.tight_layout()
    plt.show()
```

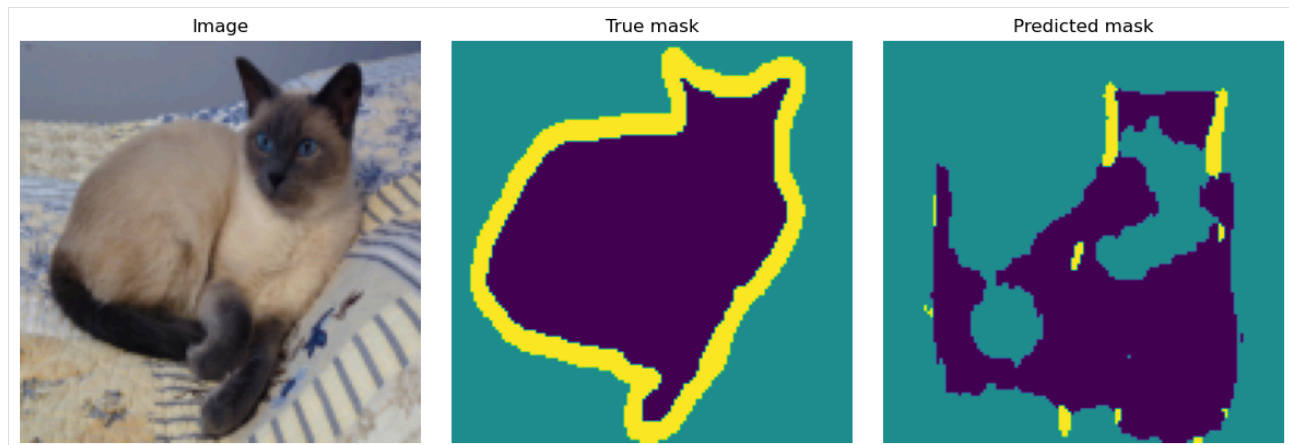
```
def show_prediction(model, dataset, index=0):
    """
    Plot an image, its true mask, and the predicted mask.

    Parameters:
        model (nn.Module): Trained model.
        dataset (Dataset): Source of images and masks.
        index (int): Sample index.

    Returns:
        None
    """
    model.eval()
    image, mask = dataset[index]
    with torch.no_grad():
        preds = model(image.unsqueeze(0).to(DEVICE)).argmax(1)[0].cpu()
    plot_triplet(image, mask, preds)
```

Run

```
show_prediction(model, test_data, index=0)
```

**Save Model**

```
torch.save(model.state_dict(), 'models/unet_oxford_pet.pt')
print('saved unet_oxford_pet.pt')
```

```
saved unet_oxford_pet.pt
```

Load Model

```
loaded_model = UNet().to(DEVICE)
loaded_model.load_state_dict(torch.load('models/unet_oxford_pet.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded unet_oxford_pet.pt')
```

```
loaded unet_oxford_pet.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_81064/1932575721.py:2: FutureWarning: You
are using `torch.load` with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data which will execute
arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a
future release, the default value for `weights_only` will be flipped to `True`. This limits the
functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use
case where you don't have full control of the loaded file. Please open an issue on GitHub for any
issues related to this experimental feature.
loaded_model.load_state_dict(torch.load('models/unet_oxford_pet.pt', map_location=DEVICE))
```

Inference

Function

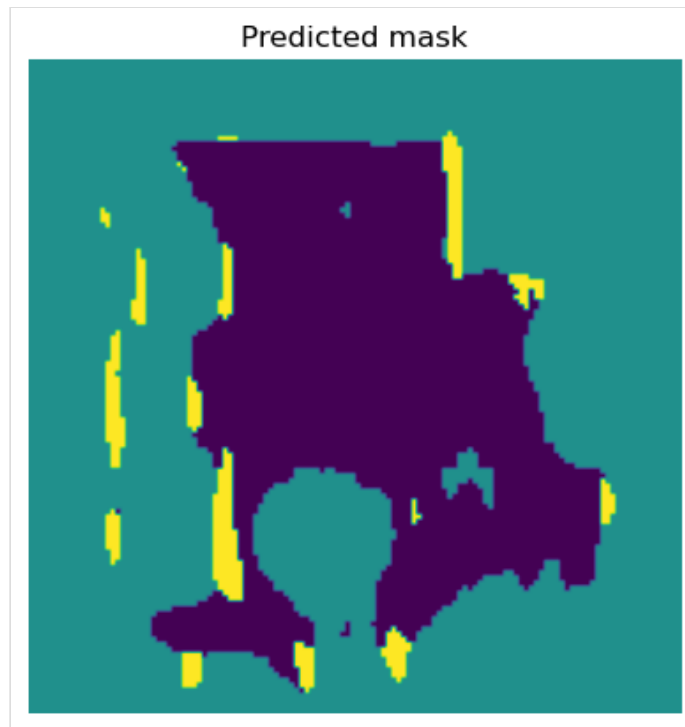
```
def predict_mask(model, image):
    """
    Predict a segmentation mask for one image.

    Parameters:
        model (nn.Module): Trained model.
        image (torch.Tensor): A 3xHxW image tensor.

    Returns:
        torch.Tensor: Predicted mask.
    """
    model.eval()
    with torch.no_grad():
        logits = model(image.unsqueeze(0).to(DEVICE))
        return logits.argmax(1)[0].cpu()
```

Run Inference

```
image, mask = test_data[1]
predicted = predict_mask(loaded_model, image)
plt.imshow(predicted, cmap='viridis')
plt.title('Predicted mask')
plt.axis('off')
plt.show()
```



Key Takeaways

- Segmentation predicts a label per pixel.
- U-Nets use skip connections to recover fine detail.
- Masks must be resized with nearest-neighbor interpolation.
- Pixel accuracy is a start; visual comparison is the real check.

Exercises

1. Change the image size and compare pixel accuracy and speed.
2. Add a deeper encoder level and observe the effect.
3. Plot the confusion of classes in the predicted masks.
4. Explain the role of the skip connections.
5. Segment an image outside the test set and inspect the result.

Guided Walkthrough

The notebook loads the Oxford-IIIT Pet dataset with segmentation masks, resizes images and masks to a common size, and shifts the raw mask values into class indices. The model is a small U-Net: encoder blocks, a bottleneck, and a decoder that upsamples and concatenates encoder features before a final convolution and an interpolation back to the input resolution.

Training minimizes cross-entropy over the pixels and tracks pixel accuracy. Visualization is essential: the notebook plots the image, the true mask, and the predicted mask side by side, which shows immediately whether the model has learned the shape of the animal. The masks must be resized with nearest-neighbor interpolation, because bilinear interpolation would invent fractional class labels.

Common Pitfalls

- Resizing masks with bilinear interpolation, which corrupts class labels.
- Forgetting to shift raw mask values into zero-based indices.
- Trusting pixel accuracy without looking at the predicted masks.

Going Further

- Compute the intersection over union for each class.
- Add a deeper encoder level and compare.
- Apply the trained model to an image outside the test set.

Exercise Solutions

1. Larger images improve boundary detail but slow training.
2. A deeper encoder captures more context and usually improves accuracy.
3. The class confusion shows boundaries are the hardest class to predict.
4. Skip connections pass fine spatial detail from encoder to decoder.
5. An out-of-sample image is segmented plausibly if it resembles the training pets.

Chapter 28: CartPole Environment - Deep Q-Network

Introduction

Reinforcement learning is the branch of machine learning where an agent learns by acting in an environment and receiving rewards. This chapter builds a deep Q-network that learns to balance a pole on a cart, the classic CartPole task. It introduces the ideas that make deep reinforcement learning stable: a replay buffer that breaks correlations between experiences, and a target network that stabilizes the learning signal.

Reinforcement learning is harder to debug than supervised learning because there is no fixed dataset and no simple accuracy to watch. The reward curve is the signal, and it is noisy. Patience and careful bookkeeping are part of the skill.

What You Will Learn

- How an agent interacts with an environment through states, actions, and rewards.
- How Q-learning estimates the value of actions.
- Why a replay buffer improves stability.
- Why a target network is needed.
- How to evaluate a trained policy.

Overview

We create the CartPole environment and a Q-network that scores each action for a given state. During training the agent chooses actions epsilon-greedily, exploring early and exploiting later, and stores each transition in a replay buffer. On each step it samples a batch, computes the temporal-difference target using the target network, and updates the online network.

Periodically we copy the online network into the target network, which keeps the target from chasing a moving goal. We track the reward per episode, plot a smoothed curve, and evaluate the greedy policy at the end. A solved CartPole episode lasts five hundred steps, and a well-trained agent reaches that ceiling.

Citation

[1] Volodymyr Mnih et al., <https://arxiv.org/abs/1312.5602>

[2] Gymnasium, <https://gymnasium.farama.org/>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install -c conda-forge gymnasium
# conda install numpy matplotlib
```

Import Libraries

```
import random
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from collections import deque
import gymnasium as gym
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x11a349b10>
```

Parameters

```
STATE_DIM = 4
STATE_DIM
```

```
4
```

```
ACTION_DIM = 2
ACTION_DIM
```

```
2
```

```
HIDDEN_DIM = 128
HIDDEN_DIM
```

```
128
```

```
BUFFER_SIZE = 10000
BUFFER_SIZE
```



```
10000
```

Hyperparameters

```
LEARNING_RATE = 0.001
LEARNING_RATE
```

```
0.001
```

```
EPISODES = 300
EPISODES
```

```
300
```

```
BATCH_SIZE = 64
BATCH_SIZE
```

```
64
```

```
GAMMA = 0.99
GAMMA
```

```
0.99
```

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Create Environment

```
env = gym.make('CartPole-v1')
state, _ = env.reset(seed=SEED)
print('state dim:', env.observation_space.shape[0])
print('action dim:', env.action_space.n)
```

```
state dim: 4
action dim: 2
```

Create Replay Buffer

```
class ReplayBuffer:
    """
    A fixed-size experience replay buffer.
    """

    def __init__(self, capacity):
        """
        Initialize the buffer with a fixed capacity.

        Parameters:
            capacity (int): Maximum number of transitions.

        Returns:
            None
        """
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        """
        Store one transition.

        Parameters:
            state (np.ndarray): Current state.
            action (int): Action taken.
            reward (float): Reward received.
            next_state (np.ndarray): Next state.
            done (bool): Whether the episode ended.

        Returns:
            None
        """
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        """
        Sample a random batch of transitions.

        Parameters:
            batch_size (int): Number of transitions.

        Returns:
            tuple: Batched transitions.
        """
        batch = random.sample(self.buffer, batch_size)
        return zip(*batch)

    def __len__(self):
        """
        Return the number of stored transitions.

        Parameters:
            None

        Returns:
            int: Number of transitions.
        """
        return len(self.buffer)
```

Create Q-Network

```
class QNetwork(nn.Module):
    """
    A small Q-network that scores each action.
    """

    def __init__(self, state_dim=STATE_DIM, action_dim=ACTION_DIM, hidden=HIDDEN_DIM):
        """
        Initialize the network layers.

        Parameters:
            state_dim (int): Number of state features.
            action_dim (int): Number of actions.
            hidden (int): Hidden layer size.

        Returns:
            None
        """
        super(QNetwork, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden),
            nn.ReLU(),
            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, action_dim))

    def forward(self, x):
        """
        Score every action for a batch of states.

        Parameters:
            x (torch.Tensor): Batch of states.

        Returns:
            torch.Tensor: Q-values per action.
        """
        return self.net(x)
```

Instantiate Networks and Optimizer

```
torch.manual_seed(SEED)
policy = QNetwork().to(DEVICE)
target = QNetwork().to(DEVICE)
target.load_state_dict(policy.state_dict())
optimizer = torch.optim.Adam(policy.parameters(), lr=LEARNING_RATE)
buffer = ReplayBuffer(BUFFER_SIZE)
policy
```

```
QNetwork(
  (net): Sequential(
    (0): Linear(in_features=4, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=128, bias=True)
    (3): ReLU()
    (4): Linear(in_features=128, out_features=2, bias=True)
  )
)
```

Train Model

Functions

```
def to_tensor(array, dtype):
    """
    Convert a NumPy array to a tensor on the device.

    Parameters:
        array (np.ndarray): Input array.
        dtype (torch.dtype): Target dtype.

    Returns:
        torch.Tensor: Tensor on the device.
    """
    return torch.tensor(np.array(array), dtype=dtype, device=DEVICE)

def select_action(policy, state, epsilon, action_dim):
    """
    Choose an action with epsilon-greedy exploration.

    Parameters:
        policy (nn.Module): The Q-network.
        state (np.ndarray): Current state.
        epsilon (float): Exploration probability.
        action_dim (int): Number of actions.

    Returns:
        int: The chosen action.
    """
    if random.random() < epsilon:
        return random.randrange(action_dim)
    with torch.no_grad():
        state_tensor = to_tensor(state, torch.float32).unsqueeze(0)
        return int(policy(state_tensor).argmax(1).item())

def unpack_batch(buffer, batch_size):
    """
    Sample and convert a batch of transitions.

    Parameters:
        buffer (ReplayBuffer): The replay buffer.
        batch_size (int): Number of transitions.

    Returns:
        tuple: Converted tensors.
    """
    states, actions, rewards, next_states, dones = buffer.sample(batch_size)
    return (to_tensor(states, torch.float32),
            to_tensor(actions, torch.long),
            to_tensor(rewards, torch.float32),
            to_tensor(next_states, torch.float32),
            to_tensor(dones, torch.float32))

def dqn_loss(policy, target, states, actions, rewards, next_states, dones, gamma):
    """
    Compute the temporal-difference loss.

    Parameters:
```

```

    policy (nn.Module): The online network.
    target (nn.Module): The target network.
    states (torch.Tensor): Batch of states.
    actions (torch.Tensor): Batch of actions.
    rewards (torch.Tensor): Batch of rewards.
    next_states (torch.Tensor): Batch of next states.
    dones (torch.Tensor): Batch of done flags.
    gamma (float): Discount factor.

Returns:
    torch.Tensor: The loss.
"""
q_values = policy(states).gather(1, actions.unsqueeze(1)).squeeze(1)
with torch.no_grad():
    next_q = target(next_states).max(1).values
    targets = rewards + gamma * next_q * (1 - dones)
return F.smooth_l1_loss(q_values, targets)

def optimize(policy, target, buffer, optimizer, batch_size, gamma):
    """
    Run one gradient step on a sampled batch.

    Parameters:
        policy (nn.Module): The online network.
        target (nn.Module): The target network.
        buffer (ReplayBuffer): The replay buffer.
        optimizer (torch.optim.Optimizer): Parameter update rule.
        batch_size (int): Number of transitions.
        gamma (float): Discount factor.

    Returns:
        float: The loss value.
    """
    batch = unpack_batch(buffer, batch_size)
    loss = dqn_loss(policy, target, *batch, gamma)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss.item()

```

Training Loop

```

def train(env, policy, target, buffer, optimizer, episodes):
    """
    Train the agent over many episodes.

    Parameters:
        env (gym.Env): The environment.
        policy (nn.Module): The online network.
        target (nn.Module): The target network.
        buffer (ReplayBuffer): The replay buffer.
        optimizer (torch.optim.Optimizer): Parameter update rule.
        episodes (int): Number of episodes.

    Returns:
        list: Total reward per episode.
    """
    rewards_history = []
    epsilon = 1.0
    for episode in range(episodes):

```

```

state, _ = env.reset(seed=episode)
done = False
total_reward = 0.0
while not done:
    action = select_action(policy, state, epsilon, env.action_space.n)
    next_state, reward, terminated, truncated, _ = env.step(action)
    done = terminated or truncated
    buffer.push(state, action, reward, next_state, done)
    state = next_state
    total_reward += reward
    if len(buffer) >= BATCH_SIZE:
        optimize(policy, target, buffer, optimizer, BATCH_SIZE, GAMMA)
epsilon = max(0.05, epsilon * 0.99)
if episode % 10 == 0:
    target.load_state_dict(policy.state_dict())
    rewards_history.append(total_reward)
return rewards_history

```

```

history = train(env, policy, target, buffer, optimizer, EPISODES)
print('last 20 mean reward:', np.mean(history[-20:]))

```

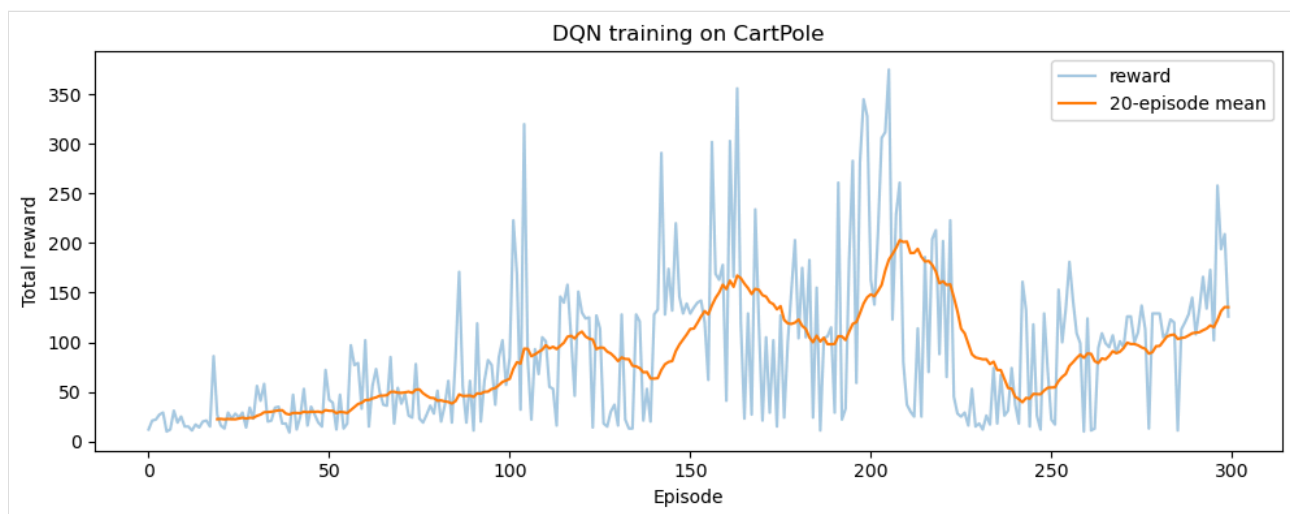
last 20 mean reward: 135.35

Visualize Training

```

plt.figure(figsize=(10, 4))
plt.plot(history, alpha=0.4, label='reward')
window = np.convolve(history, np.ones(20) / 20, mode='valid')
plt.plot(range(19, len(history)), window, label='20-episode mean')
plt.xlabel('Episode')
plt.ylabel('Total reward')
plt.title('DQN training on CartPole')
plt.legend()
plt.tight_layout()
plt.show()

```



Evaluate Model

Function

```
def evaluate(policy, episodes=5):
    """
    Run the greedy policy and report mean reward.

    Parameters:
        policy (nn.Module): Trained network.
        episodes (int): Number of evaluation episodes.

    Returns:
        float: Mean reward.
    """
    policy.eval()
    env_eval = gym.make('CartPole-v1')
    rewards = []
    for episode in range(episodes):
        state, _ = env_eval.reset(seed=episode)
        done = False
        total = 0.0
        while not done:
            action = select_action(policy, state, 0.0, env_eval.action_space.n)
            state, reward, terminated, truncated, _ = env_eval.step(action)
            done = terminated or truncated
            total += reward
        rewards.append(total)
    env_eval.close()
    return float(np.mean(rewards))
```

Run

```
print(f"Mean evaluation reward: {evaluate(policy):.2f}")
```

```
Mean evaluation reward: 116.60
```

Save Model

```
torch.save(policy.state_dict(), 'models/dqn_cartpole.pt')
print('saved dqn_cartpole.pt')
```

```
saved dqn_cartpole.pt
```

Load Model

```
loaded_policy = QNetwork().to(DEVICE)
loaded_policy.load_state_dict(torch.load('models/dqn_cartpole.pt', map_location=DEVICE))
loaded_policy.eval()
print('loaded dqn_cartpole.pt')
```

```
loaded dqn_cartpole.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_81110/2762292527.py:2: FutureWarning: You
are using `torch.load` with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data which will execute
arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a
future release, the default value for `weights_only` will be flipped to `True`. This limits the
functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use
case where you don't have full control of the loaded file. Please open an issue on GitHub for any
issues related to this experimental feature.
loaded_policy.load_state_dict(torch.load('models/dqn_cartpole.pt', map_location=DEVICE))
```

Inference

```
print(f"Mean evaluation reward: {evaluate(loaded_policy):.2f}")
```

```
Mean evaluation reward: 116.60
```

Key Takeaways

- Reinforcement learning learns from interaction, not a fixed dataset.
- Q-learning estimates the value of taking an action in a state.
- A replay buffer decorrelates experiences.
- A target network stabilizes the learning target.
- The reward curve is noisy; smooth it to judge progress.

Exercises

1. Plot the raw and smoothed reward curves.
2. Change the discount factor and describe the effect.
3. Disable the target network and observe the instability.
4. Change epsilon decay and compare learning speed.
5. Evaluate the greedy policy over several episodes.

Guided Walkthrough

The notebook creates the CartPole environment and a Q-network that scores each action for a given state. The agent chooses actions epsilon-greedily, exploring early and exploiting later, and stores each transition in a replay buffer. On each step it samples a batch, computes the temporal-difference target with the target network, and updates the online network. Periodically the target network is refreshed.

The reward curve is the signal, and it is noisy, so the notebook smooths it with a moving average. A solved CartPole episode lasts five hundred steps, and a well-trained agent reaches that ceiling. Evaluation runs the greedy policy, which is the behavior you would actually deploy.

Common Pitfalls

- Judging progress from a single episode's reward.
- Forgetting to freeze the target network, which destabilizes learning.

- Decaying epsilon too fast, which stops exploration too soon.

Going Further

- Try a harder environment such as LunarLander.
- Add a double-DQN update to reduce overestimation.
- Compare the reward curves for different discount factors.

Exercise Solutions

1. The smoothed curve rises and plateaus near five hundred.
2. A higher discount factor values long-term reward more and can speed learning.
3. Without a target network the reward curve is unstable.
4. Slower epsilon decay explores longer and can find a better policy.
5. The greedy policy scores near the maximum reward per episode.

Chapter 29: Tiny Shakespeare Dataset - Fine-Tuning a Pretrained LLM

Introduction

Training a language model from scratch is educational but expensive. Fine-tuning a pretrained model is the practical alternative, and it is how most real language applications are built. This chapter takes DistilGPT-2, a small pretrained transformer, and fine-tunes it on Tiny Shakespeare using the Hugging Face Transformers library. In a few minutes on a laptop it adapts a general model to a specific style.

The chapter is also a lesson in transfer learning: the pretrained model already knows English grammar and vocabulary, so fine-tuning only has to teach it the target style. That is why so little data and so few epochs are enough.

What You Will Learn

- How to load a pretrained model and tokenizer with Transformers.
- How a subword tokenizer differs from a character tokenizer.
- How to build a block dataset from token ids.
- How to fine-tune a language model with a custom PyTorch loop.
- How to generate text from a fine-tuned model.

Overview

We load the DistilGPT-2 tokenizer, which splits text into subword tokens, and the DistilGPT-2 model. We tokenize Tiny Shakespeare, cut it into fixed-length blocks, and wrap the blocks in a dataset that returns each block as both input and target, which is the shift-by-one trick again.

Fine-tuning uses the model's own loss: we pass the labels and let the model compute cross-entropy internally. After a couple of epochs the model generates Shakespeare-flavored text from a prompt. We save the fine-tuned model and tokenizer together so the pair can be reloaded and used elsewhere.

Citation

- [1] Andrej Karpathy, <https://github.com/karpathy/char-rnn>
- [2] Alec Radford et al., https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- [3] Hugging Face Transformers, <https://huggingface.co/docs/transformers>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# pip install transformers
# conda install numpy matplotlib
```

Import Libraries

```
import urllib.request
import numpy as np
import torch
from torch.utils.data import DataLoader
from transformers import GPT2LMHeadModel
from transformers import GPT2TokenizerFast
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x1183ddb30>
```

Parameters

```
MODEL_NAME = 'distilgpt2'
MODEL_NAME
```

```
'distilgpt2'
```

```
BLOCK_SIZE = 128
BLOCK_SIZE
```

```
128
```

Hyperparameters

```
LEARNING_RATE = 0.00005
LEARNING_RATE
```

```
5e-05
```

```
EPOCHS = 2
EPOCHS
```

```
2
```

```
BATCH_SIZE = 16
BATCH_SIZE
```

```
16
```

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Download Dataset

```
URL = 'https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt'
urlretrieve(URL, 'data/tinyshakespeare.txt')
with open('data/tinyshakespeare.txt', encoding='utf-8') as handle:
    text = handle.read()
print('length:', len(text))
```

```
length: 1115394
```

Tokenize Dataset

The GPT-2 tokenizer splits text into subword tokens. We concatenate the token ids and cut them into fixed-length blocks.

```
def tokenize_blocks(tokenizer, text, block_size):
    """
    Tokenize text and cut it into fixed-length blocks.

    Parameters:
        tokenizer (GPT2TokenizerFast): The tokenizer.
        text (str): The training text.
        block_size (int): Length of each block.

    Returns:
        torch.Tensor: A 2-D tensor of token ids.
    """
    ids = tokenizer(text)['input_ids']
    n_blocks = len(ids) // block_size
    trimmed = ids[:n_blocks * block_size]
    return torch.tensor(trimmed, dtype=torch.long).view(n_blocks, block_size)
```

Run

```
tokenizer = GPT2TokenizerFast.from_pretrained(MODEL_NAME)
tokenizer.pad_token = tokenizer.eos_token
```

```
blocks = tokenize_blocks(tokenizer, text, BLOCK_SIZE)
blocks.shape
```

Token indices sequence length is longer than the specified maximum sequence length for this model (338025 > 1024). Running this sequence through the model will result in indexing errors

```
torch.Size([2640, 128])
```

Create Dataset and DataLoader

```
class TextDataset(torch.utils.data.Dataset):
    """
    A dataset of fixed-length token blocks.
    """

    def __init__(self, blocks):
        """
        Store the token blocks.

        Parameters:
            blocks (torch.Tensor): A 2-D tensor of token ids.

        Returns:
            None
        """
        self.blocks = blocks

    def __len__(self):
        """
        Return the number of blocks.

        Parameters:
            None

        Returns:
            int: Number of blocks.
        """
        return len(self.blocks)

    def __getitem__(self, idx):
        """
        Return one block as input and target.

        Parameters:
            idx (int): Block index.

        Returns:
            tuple: Input and target tensors.
        """
        block = self.blocks[idx]
        return block, block.clone()
```

```
split = int(0.9 * len(blocks))
train_loader = DataLoader(TextDataset(blocks[:split]), batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(TextDataset(blocks[split:]), batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(val_loader)
```

```
(149, 17)
```

Load Pretrained Model

We start from DistilGPT-2, a small pretrained language model, and fine-tune it on Shakespeare. This is far cheaper than training from scratch.

```
model = GPT2LMHeadModel.from_pretrained(MODEL_NAME).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)
sum(p.numel() for p in model.parameters())
```

```
81912576
```

Train Model

Functions

```
def train_epoch(model, loader, optimizer):
    """
    Fine-tune the model for a single epoch.

    Parameters:
        model (nn.Module): The language model.
        loader (DataLoader): Training data loader.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(input_ids=x, attention_mask=(x != tokenizer.pad_token_id).long(),
                        labels=y)
        outputs.loss.backward()
        optimizer.step()
        total += outputs.loss.item() * len(x)
    return total / len(loader.dataset)

def estimate_loss(model, loader):
    """
    Estimate the mean loss over a loader.

    Parameters:
        model (nn.Module): The language model.
        loader (DataLoader): Evaluation data loader.

    Returns:
        float: Mean loss.
    """
    model.eval()
    total = 0.0
    with torch.no_grad():
        for x, y in loader:
```

```
x, y = x.to(DEVICE), y.to(DEVICE)
outputs = model(input_ids=x, attention_mask=(x != tokenizer.pad_token_id).long(),
                labels=y)
total += outputs.loss.item() * len(x)
return total / len(loader.dataset)
```

Training Loop

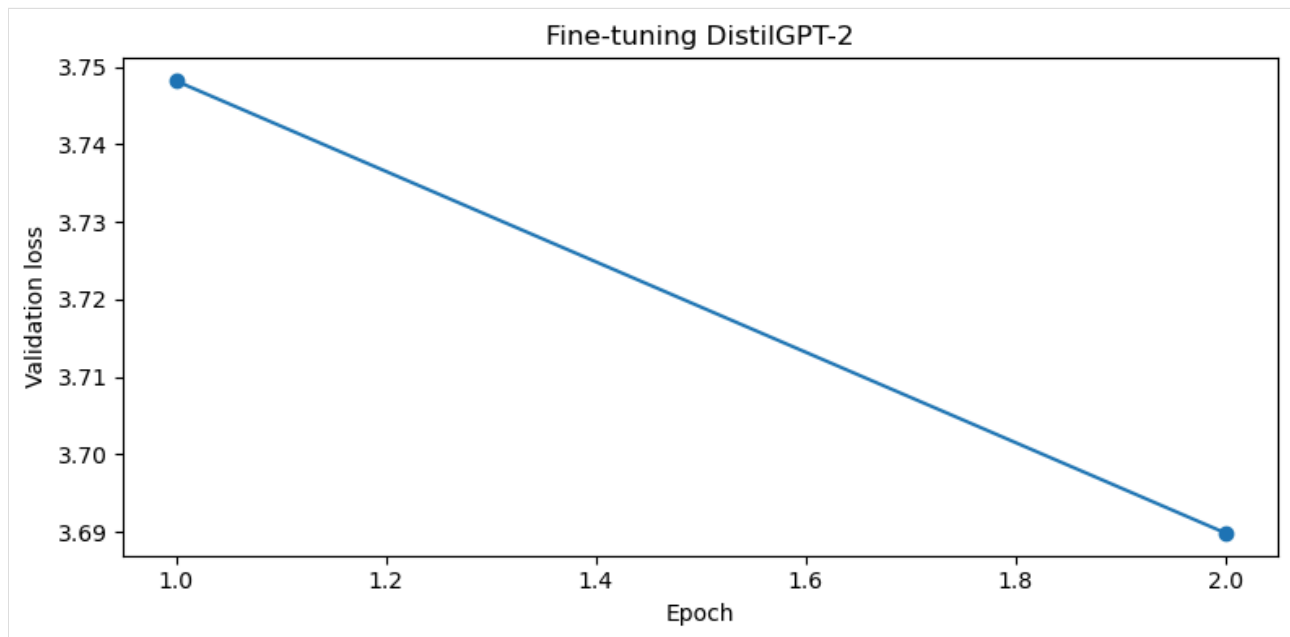
```
history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, optimizer)
    val_loss = estimate_loss(model, val_loader)
    history.append(val_loss)
    print(f"Epoch {epoch + 1} | train loss {train_loss:.4f} | val loss {val_loss:.4f}")
```

Epoch 1 | train loss 4.0547 | val loss 3.7482

Epoch 2 | train loss 3.7320 | val loss 3.6898

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Validation loss')
plt.title('Fine-tuning DistilGPT-2')
plt.tight_layout()
plt.show()
```



Generate Text

Function

```
def generate(model, prompt, max_new_tokens=100):
    """
    Generate text from a prompt.

    Parameters:
        model (nn.Module): Fine-tuned language model.
        prompt (str): Seed text.
        max_new_tokens (int): Number of tokens to generate.

    Returns:
        str: The generated text.
    """
    model.eval()
    inputs = tokenizer(prompt, return_tensors='pt').to(DEVICE)
    with torch.no_grad():
        output = model.generate(**inputs, max_new_tokens=max_new_tokens,
                                do_sample=True, top_k=50, temperature=0.8,
                                pad_token_id=tokenizer.eos_token_id)
    return tokenizer.decode(output[0], skip_special_tokens=True)
```

Run

```
print(generate(model, 'ROMEO: ', max_new_tokens=120))
```

```
ROMEO: _____
ROMEO:
Nay, I have forgot.

ROMEO:
O, I forgot.

ROMEO:
I forgot.
COMINIUS:
I'll prove you wrong.

ROMEO:
Hear me yet?

ROMEO:
Yea, so, what is the purpose of the law?

ROMEO:
Nay, it is a law, and the law is.

ROMEO:
Hear me yet?

ROMEO:
Thou art not but a law
```


Save Model

```
model.save_pretrained('models/distilgpt2_shakespeare')
tokenizer.save_pretrained('models/distilgpt2_shakespeare')
print('saved distilgpt2_shakespeare')
```

```
saved distilgpt2_shakespeare
```

Load Model

```
loaded_model = GPT2LMHeadModel.from_pretrained('models/distilgpt2_shakespeare').to(DEVICE)
loaded_model.eval()
print('loaded distilgpt2_shakespeare')
```

```
loaded distilgpt2_shakespeare
```

Inference

```
print(generate(loaded_model, 'JULIET: ', max_new_tokens=120))
```

```
JULIET: ix

GLOUCESTER:
Well, I think it must be true, my lord.

JULIET:
I would, sir, I would, sir, have seen the man
Of a gentleman's face I had not seen.

GLOUCESTER:
You, sir?

JULIET:
That is the gentleman's face, sir, and that he
Is not
The man's face, sir.

JULIET:
I do remember the man; and you, sir,
What
```

Key Takeaways

- Fine-tuning adapts a pretrained model cheaply.
- Subword tokenizers handle open vocabulary far better than character tokenizers.
- The language-modeling target is still the next token.
- Save the model and tokenizer together.
- Transfer learning is the practical path to strong language models.

Exercises

1. Compare the generated text before and after fine-tuning.
2. Fine-tune for more epochs and compare.
3. Change the block size and describe the effect.
4. Generate with several prompts and compare the styles.
5. Explain why fine-tuning needs so little data.

Guided Walkthrough

The notebook loads the DistilGPT-2 tokenizer and model, tokenizes Tiny Shakespeare into subword tokens, cuts the tokens into fixed-length blocks, and wraps the blocks in a dataset that returns each block as both input and target. Fine-tuning uses the model's own loss: the labels are passed in and the model computes cross-entropy internally.

After a couple of epochs the model generates Shakespeare-flavored text from a prompt. Saving the model and tokenizer together means the pair can be reloaded and used elsewhere. The chapter is a lesson in transfer learning: the pretrained model already knows English, so fine-tuning only has to teach it the target style, which is why so little data and so few epochs are enough.

Common Pitfalls

- Forgetting to set the padding token, which breaks batching.
- Using a learning rate that is too high and destroying the pretrained weights.
- Saving the model without the tokenizer, which makes it unusable later.

Going Further

- Fine-tune on a different author and compare the generated style.
- Try a larger pretrained model and compare quality.
- Evaluate the model with perplexity on held-out text.

Exercise Solutions

1. Before fine-tuning the text is generic English; afterward it imitates Shakespeare.
2. More epochs sharpen the style but risk overfitting to the corpus.
3. A larger block size gives more context but needs more memory.
4. Different prompts produce different speakers and tones.
5. Fine-tuning needs little data because the pretrained model already encodes language.

Part IX: A Kaggle Project from Scratch

Chapter 30: Kaggle Titanic Dataset - EDA and Binary Classifier

Introduction

This chapter is a complete Kaggle project, from an empty notebook to a submission file. Kaggle is the largest platform for data science competitions and public datasets, and its Titanic competition is the classic starting point. It is a real competition, and it is permanent: Kaggle's "Getting Started" competitions never close, have no deadline, and accept submissions at any time, so you can join today and see your score on the leaderboard within minutes. Nothing here is a simulation.

The task is simple to state and genuinely instructive: given a passenger's class, sex, age, fare, and family information, predict whether that passenger survived. The dataset is small, the signal is strong, and the best possible score is not a hundred percent, which makes it an honest first competition. The skills you practice here, downloading data with an API, exploring it, engineering features, training a baseline and a neural network, and writing a correctly formatted submission, are the same skills every Kaggle competition and every real project requires.

What You Will Learn

- How to create a Kaggle account and generate an API token.
- How to install and configure the Kaggle command-line interface.
- How to download the competition's `train.csv` and `test.csv`.
- How to perform exploratory data analysis and feature engineering.
- How to train a logistic regression baseline and a PyTorch classifier.
- How to produce a valid `PassengerId, Survived` submission for the real test set.
- How to submit to the leaderboard and interpret your score.

Overview

The chapter begins with the mechanics that trip up beginners. You create a Kaggle account, join the Titanic competition to accept its rules, and generate an API token, a file named `kaggle.json` that authenticates the command-line tool. With the tool configured, two commands download the data: `train.csv`, which contains the survival labels, and `test.csv`, which does not, because Kaggle keeps the test labels secret to score your submission.

If you do not have a token, the notebook falls back to public mirrors of the exact same competition files, so it runs end to end either way. This matters: the submission must be built from the real `test.csv`, whose 418 passengers are the ones the leaderboard scores. A common beginner mistake is to split the training data and submit predictions for that split, which is not a valid submission at all. We avoid that trap by predicting on the real test file.

We then explore the training data. Sex and passenger class are the strongest predictors, age and fare matter, and family size carries a signal that is easy to miss. We impute missing values, encode categoricals, and add a family-size feature. With the features prepared we fit a logistic regression baseline, then train a small PyTorch classifier,

and compare them with accuracy, a confusion matrix, and ROC and AUC. Finally we write `submission.csv` with exactly two columns, `PassengerId` and `Survived`, one row per test passenger, and walk through uploading it to Kaggle.

The score you receive is a starting point, not a verdict. The best way to improve it is to engineer better features and validate them carefully, which is the loop that every Kaggle competitor runs.

Citation

[1] Kaggle, <https://www.kaggle.com/c/titanic>

[2] Competition file mirrors, <https://github.com/agconti/kaggle-titanic>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy pandas matplotlib seaborn scikit-learn
# pip install kaggle
```

About This Competition

Titanic is a real Kaggle **Getting Started** competition. Getting Started competitions are permanent: they never close, have no deadline, and accept submissions at any time, so you can join today and appear on the leaderboard. The task is to predict which passengers survived. You submit a CSV with two columns, `PassengerId` and `Survived`, with one row for every passenger in the official `test.csv` (418 rows).

Create a Kaggle Account

1. Sign up for a free account at [kaggle.com](https://www.kaggle.com).
2. Open [kaggle.com/c/titanic](https://www.kaggle.com/c/titanic) and click **Join Competition** to accept the rules.
3. Click your profile picture, then **Settings**, then **Create New Token**.
4. A file named `kaggle.json` downloads.

Configure the Kaggle CLI

```
# mkdir -p ~/.kaggle
# mv ~/Downloads/kaggle.json ~/.kaggle/kaggle.json
# chmod 600 ~/.kaggle/kaggle.json
```

Download the Official Files With the Kaggle CLI

These commands download the exact competition files. `train.csv` has the survival labels; `test.csv` does not, because Kaggle scores your predictions against secret labels.

```
# kaggle competitions download -c titanic -f train.csv -p data
# kaggle competitions download -c titanic -f test.csv -p data
# kaggle competitions download -c titanic -f gender_submission.csv -p data
```

Public Mirror Fallback

If you do not have a Kaggle token yet, this notebook downloads the same two files from a public mirror so it still runs end to end. The mirror files match the competition: `train.csv` has 891 labeled passengers and `test.csv` has 418 unlabeled passengers.

Import Libraries

```
import os
import urllib.request
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x15d890290>
```

Download Dataset

Function

```
def download_file(url, path):
    """
    Download a file if it is not already present.

    Parameters:
        url (str): Source URL.
        path (str): Destination path.

    Returns:
        str: The destination path.
    """
    if not os.path.exists(path):
```

```
urllib.request.urlretrieve(url, path)
return path
```

Run

```
MIRROR = 'https://raw.githubusercontent.com/agconti/kaggle-titanic/master/data/'
download_file(MIRROR + 'train.csv', 'data/train.csv')
download_file(MIRROR + 'test.csv', 'data/test.csv')
print('downloaded train.csv and test.csv')
```

```
downloaded train.csv and test.csv
```

Load Dataset

```
train = pd.read_csv('data/train.csv')
test = pd.read_csv('data/test.csv')
print('train rows:', train.shape[0])
print('test rows :', test.shape[0])
```

```
train rows: 891
test rows : 418
```

```
train.head()
```

```

 PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

      Name      Sex  Age  SibSp  \
0  Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2  Heikkinen, Miss. Laina    female  26.0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)    female  35.0      1
4  Allen, Mr. William Henry    male  35.0      0

   Parch    Ticket   Fare Cabin Embarked
0      0   A/5 21171   7.2500   NaN      S
1      0    PC 17599  71.2833   C85      C
2      0 STON/O2. 3101282   7.9250   NaN      S
3      0  113803   53.1000  C123      S
4      0  373450   8.0500   NaN      S
```

Exploratory Data Analysis

Dataset Shape and Types

```
train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Summary Statistics

```
train.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp \
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

Missing Values

`Age` and `Cabin` have many missing values. We impute `Age` and `Fare` with the training median and `Embarked` with the training mode.

```
train.isnull().sum()
```

```
PassengerId    0
Survived        0
Pclass          0
Name            0
Sex             0
Age            177
```



```
SibSp      0
Parch      0
Ticket     0
Fare       0
Cabin      687
Embarked    2
dtype: int64
```

Survival Rate

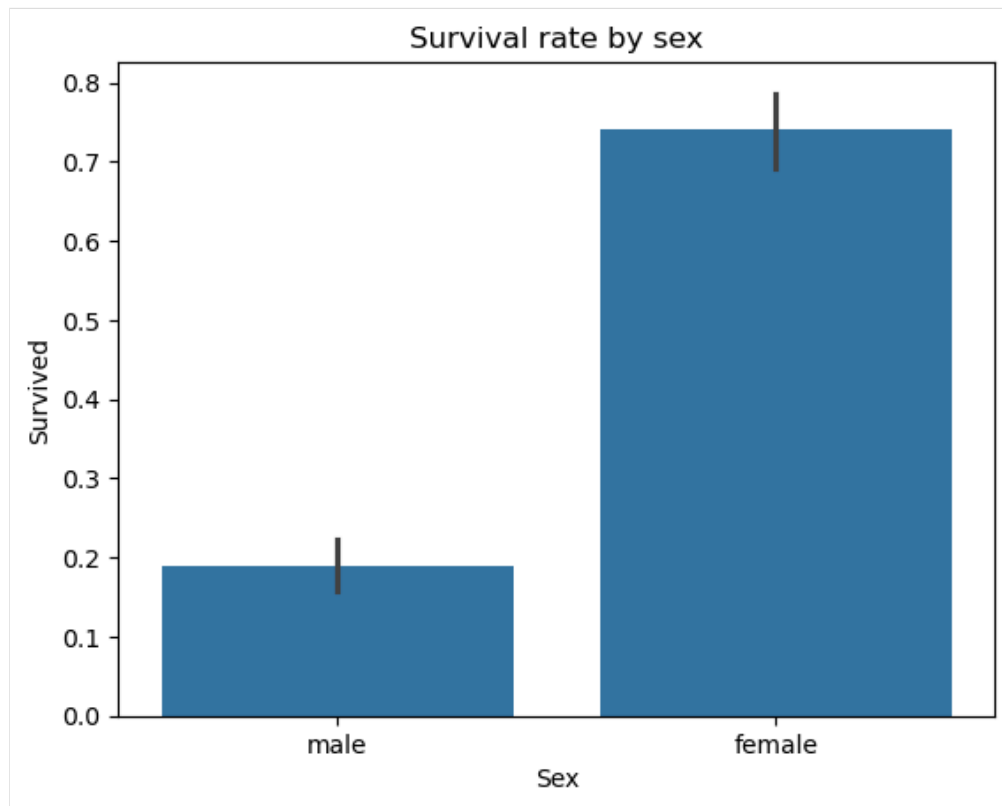
```
train['Survived'].value_counts(normalize=True).round(3)
```

```
Survived
0    0.616
1    0.384
Name: proportion, dtype: float64
```

Survival by Sex

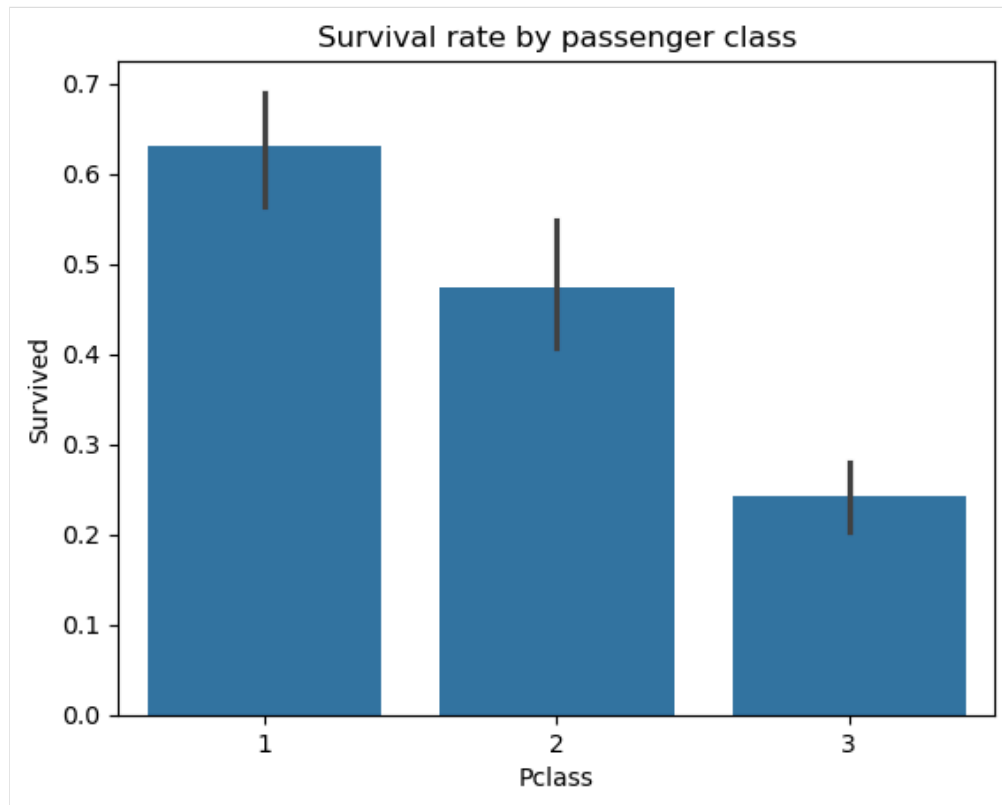
Sex is the strongest single predictor: women survived far more often.

```
sns.barplot(data=train, x='Sex', y='Survived')
plt.title('Survival rate by sex')
plt.show()
```



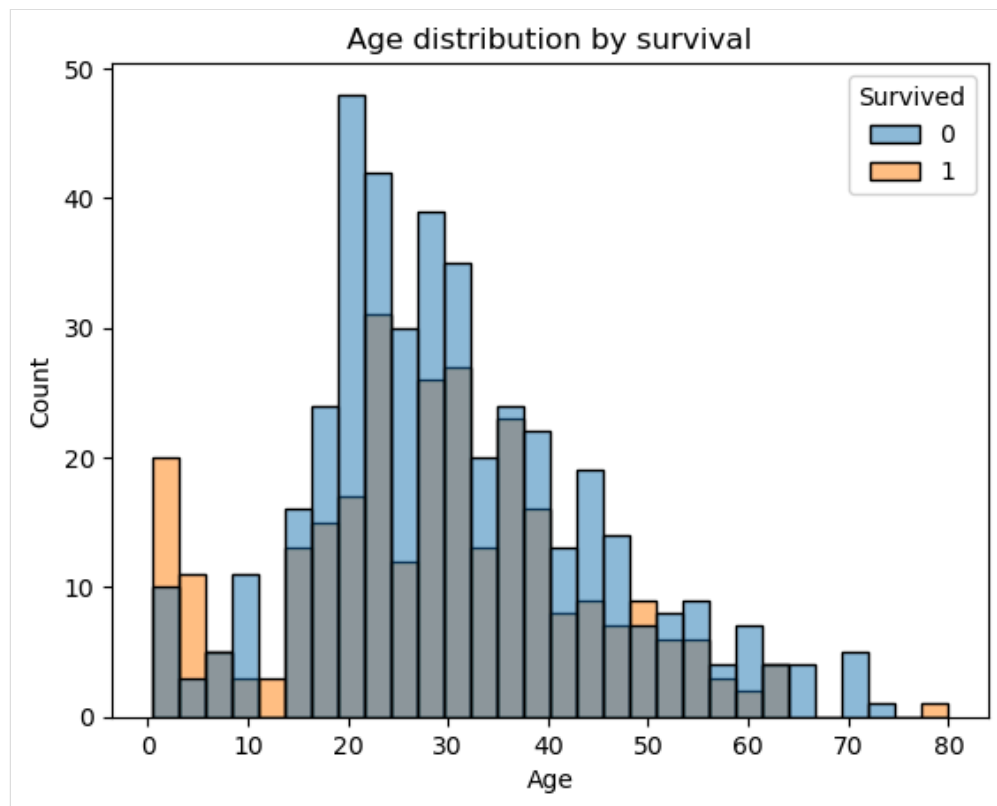
Survival by Passenger Class

```
sns.barplot(data=train, x='Pclass', y='Survived')  
plt.title('Survival rate by passenger class')  
plt.show()
```



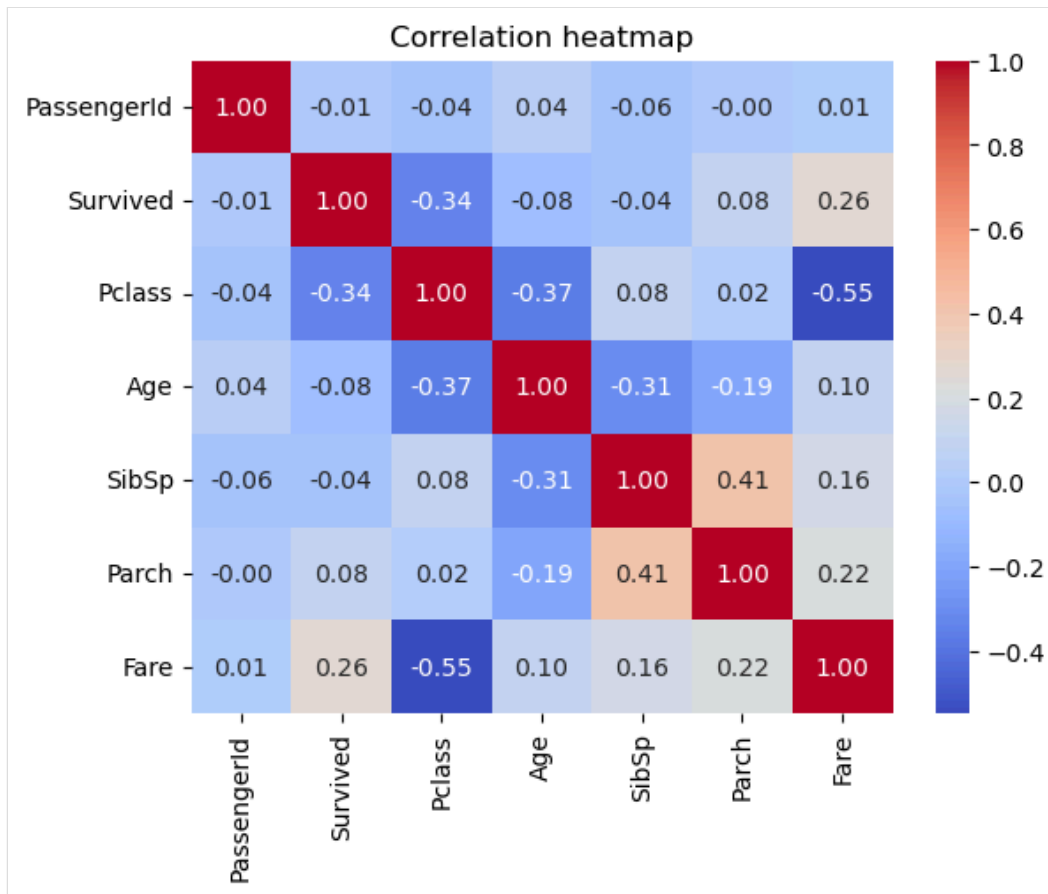
Age Distribution

```
sns.histplot(data=train, x='Age', hue='Survived', bins=30)  
plt.title('Age distribution by survival')  
plt.show()
```



Correlation Heatmap

```
numeric = train.select_dtypes(include='number')
sns.heatmap(numeric.corr(), annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Correlation heatmap')
plt.show()
```



Feature Engineering

We apply identical transformations to the training and test sets, using statistics computed only on the training set to avoid leakage.

Functions

```
def add_family_features(df):
    """
    Add family size and alone features.

    Parameters:
        df (pandas.DataFrame): The Titanic data.

    Returns:
        pandas.DataFrame: The data with new features.
    """
    df = df.copy()
    df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
    df['IsAlone'] = (df['FamilySize'] == 1).astype(int)
    return df

def prepare_features(train, test):
    """
    Impute and encode both datasets using training statistics.

    Parameters:
```

```

train (pandas.DataFrame): Training data with labels.
test (pandas.DataFrame): Test data without labels.

Returns:
    tuple: The prepared training and test data.
"""
train = add_family_features(train)
test = add_family_features(test)
age_median = train['Age'].median()
fare_median = train['Fare'].median()
embarked_mode = train['Embarked'].mode()[0]
for frame in (train, test):
    frame['Age'] = frame['Age'].fillna(age_median)
    frame['Fare'] = frame['Fare'].fillna(fare_median)
    frame['Embarked'] = frame['Embarked'].fillna(embarked_mode)
    frame['Sex'] = (frame['Sex'] == 'male').astype(int)
    frame['Embarked'] = frame['Embarked'].map({'S': 0, 'C': 1, 'Q': 2})
return train, test

```

Run

```

train, test = prepare_features(train, test)
train[['Sex', 'Age', 'Fare', 'FamilySize', 'IsAlone', 'Embarked']].head()

```

	Sex	Age	Fare	FamilySize	IsAlone	Embarked
0	1	22.0	7.2500	2	0	0
1	0	38.0	71.2833	2	0	1
2	0	26.0	7.9250	1	1	0
3	0	35.0	53.1000	2	0	0
4	1	35.0	8.0500	1	1	0

Prepare Features and Target

```

FEATURES = ['Pclass', 'Sex', 'Age', 'Fare', 'FamilySize', 'IsAlone', 'Embarked']
X = train[FEATURES].to_numpy(dtype=np.float32)
y = train['Survived'].to_numpy(dtype=np.int64)
X_test = test[FEATURES].to_numpy(dtype=np.float32)
X_MEAN = X.mean(axis=0)
X_STD = X.std(axis=0)
X = (X - X_MEAN) / X_STD
X_test = (X_test - X_MEAN) / X_STD
X.shape, X_test.shape

```

```
((891, 7), (418, 7))
```

Validation Split

We hold out part of the training data to estimate performance before we build the real submission. The test set has no labels, so it cannot be used for validation.

```

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y)
X_train.shape, X_val.shape

```

```
((712, 7), (179, 7))
```

Baseline Classifier: Logistic Regression

```
baseline = LogisticRegression(max_iter=1000).fit(X_train, y_train)
baseline_preds = baseline.predict(X_val)
baseline_probs = baseline.predict_proba(X_val)[:, 1]
print('accuracy:', round(accuracy_score(y_val, baseline_preds), 4))
print('roc auc :', round(roc_auc_score(y_val, baseline_probs), 4))
```

```
accuracy: 0.7933
roc auc : 0.8462
```

PyTorch Classifier

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Create Dataset

```
class NumpyDataset(torch.utils.data.Dataset):
    """
    A memory-efficient dataset that stores data as NumPy arrays.
    """

    def __init__(self, X, y, device):
        """
        Initialize the dataset with NumPy arrays.

        Parameters:
            X (np.ndarray): Feature array.
            y (np.ndarray): Label array.
            device (torch.device): Device to move tensors to.

        Returns:
            None
        """
        self.X = X
        self.y = y
        self.device = device

    def __len__(self):
        """
        Return the number of samples in the dataset.

        Parameters:
            None
        """
```

```
    Returns:
        int: Number of samples.
    """
    return len(self.X)

def __getitem__(self, idx):
    """
    Return a single sample as tensors on the device.

    Parameters:
        idx (int): Index of the sample.

    Returns:
        tuple: Feature and label tensors.
    """
    X_tensor = torch.tensor(self.X[idx]).float().to(self.device)
    y_tensor = torch.tensor(self.y[idx]).long().to(self.device)
    return X_tensor, y_tensor
```

Create Model

```
class Model(nn.Module):
    """
    A small feedforward classifier for tabular data.
    """

    def __init__(self, in_features, h1=32, h2=16, out_features=2):
        """
        Initialize the neural network layers.

        Parameters:
            in_features (int): Number of input features.
            h1 (int): Neurons in the first hidden layer.
            h2 (int): Neurons in the second hidden layer.
            out_features (int): Number of output features.

        Returns:
            None
        """
        super(Model, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, h1),
            nn.ReLU(),
            nn.Linear(h1, h2),
            nn.ReLU(),
            nn.Linear(h2, out_features))

    def forward(self, x):
        """
        Run the forward pass of the network.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output logits.
        """
        return self.net(x)
```

Create DataLoaders, Loss, and Optimizer

```
train_loader = DataLoader(NumpyDataset(X_train, y_train, DEVICE),
                          batch_size=32, shuffle=True)
val_loader = DataLoader(NumpyDataset(X_val, y_val, DEVICE),
                       batch_size=32, shuffle=False)
model = Model(len(FEATURES)).to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
model
```

```
Model(
  (net): Sequential(
    (0): Linear(in_features=7, out_features=32, bias=True)
    (1): ReLU()
    (2): Linear(in_features=32, out_features=16, bias=True)
    (3): ReLU()
    (4): Linear(in_features=16, out_features=2, bias=True)
  )
)
```

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)

def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        tuple: Mean loss and accuracy.
    """
    model.eval()
```



```

total, correct = 0.0, 0
with torch.no_grad():
    for x, y in loader:
        logits = model(x)
        total += loss_fn(logits, y).item() * len(y)
        correct += (logits.argmax(1) == y).sum().item()
n = len(loader.dataset)
return total / n, correct / n

```

Training Loop

```

for epoch in range(200):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    val_loss, val_accuracy = evaluate(model, val_loader, loss_fn)
    if (epoch + 1) % 40 == 0:
        print(f"Epoch {epoch + 1:3d} | loss {val_loss:.4f} | accuracy {val_accuracy:.4f}")

```

Epoch 40 | loss 0.4529 | accuracy 0.7989

Epoch 80 | loss 0.4894 | accuracy 0.8045

Epoch 120 | loss 0.6149 | accuracy 0.7821

Epoch 160 | loss 0.7025 | accuracy 0.7598

Epoch 200 | loss 0.8675 | accuracy 0.7709

Evaluate the PyTorch Classifier

```

_, accuracy = evaluate(model, val_loader, loss_fn)
print(f"Validation accuracy: {accuracy:.4f}")

```

Validation accuracy: 0.7709

```

model.eval()
with torch.no_grad():
    logits = model(torch.tensor(X_val, dtype=torch.float32).to(DEVICE))
    probs = torch.softmax(logits, dim=1)[:, 1].cpu().numpy()
preds = (probs >= 0.5).astype(int)
print(confusion_matrix(y_val, preds))
print(classification_report(y_val, preds, target_names=['died', 'survived']))

```

```

[[96 14]
 [27 42]]

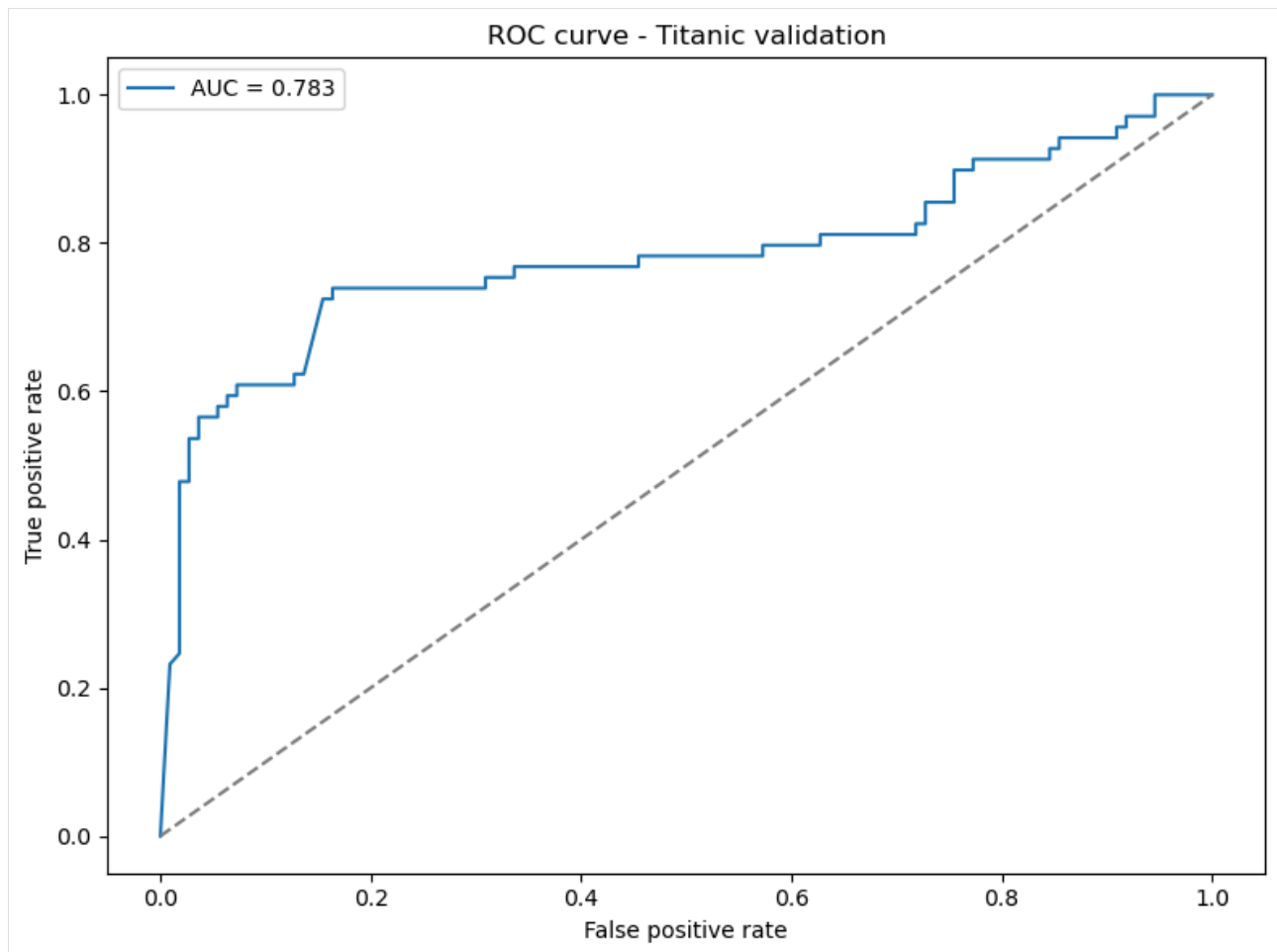
```

	precision	recall	f1-score	support
died	0.78	0.87	0.82	110
survived	0.75	0.61	0.67	69

accuracy			0.77	179
macro avg	0.77	0.74	0.75	179
weighted avg	0.77	0.77	0.77	179

ROC Curve

```
fpr, tpr, _ = roc_curve(y_val, probs)
auc_score = roc_auc_score(y_val, probs)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.3f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('ROC curve - Titanic validation')
plt.legend()
plt.tight_layout()
plt.show()
```



Predict the Real Test Set

The submission must cover the 418 passengers in the official `test.csv`. We predict on that file, not on a split of the training data.

Function

```
def predict_test(model, X_test):
    """
    Predict survival for the official test set.

    Parameters:
        model (nn.Module): Trained model.
        X_test (numpy.ndarray): Standardized test features.

    Returns:
        numpy.ndarray: Predicted labels.
    """
    model.eval()
    with torch.no_grad():
        logits = model(torch.tensor(X_test, dtype=torch.float32).to(DEVICE))
        probs = torch.softmax(logits, dim=1)[: , 1].cpu().numpy()
    return (probs >= 0.5).astype(int)
```

Run

```
test_preds = predict_test(model, X_test)
print('test predictions:', test_preds.shape[0])
```

```
test predictions: 418
```

Create the Kaggle Submission

Function

```
def build_submission(ids, predictions, path='outputs/submission.csv'):
    """
    Write a Kaggle-format submission CSV.

    Parameters:
        ids (np.ndarray): Passenger ids from test.csv.
        predictions (numpy.ndarray): Predicted survival labels.
        path (str): Output CSV path.

    Returns:
        str: The path to the submission file.
    """
    frame = pd.DataFrame({'PassengerId': ids, 'Survived': predictions})
    frame.to_csv(path, index=False)
    return path
```

Run

```
build_submission(test['PassengerId'].to_numpy(), test_preds, 'outputs/submission.csv')
submission = pd.read_csv('outputs/submission.csv')
print('rows:', submission.shape[0])
submission.head()
```

```
rows: 418
```

	PassengerId	Survived
0	892	0
1	893	0
2	894	0
3	895	0
4	896	0

Save Model

```
torch.save(model.state_dict(), 'models/kaggle_titanic.pt')
print('saved kaggle_titanic.pt')
```

```
saved kaggle_titanic.pt
```

Load Model

```
loaded_model = Model(len(FEATURES)).to(DEVICE)
loaded_model.load_state_dict(torch.load('models/kaggle_titanic.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded kaggle_titanic.pt')
```

```
loaded kaggle_titanic.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_81227/3056722231.py:2: FutureWarning: You
are using `torch.load` with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data which will execute
arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a
future release, the default value for `weights_only` will be flipped to `True`. This limits the
functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use
case where you don't have full control of the loaded file. Please open an issue on GitHub for any
issues related to this experimental feature.
loaded_model.load_state_dict(torch.load('models/kaggle_titanic.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, features):
    """
    Predict survival for one passenger.

    Parameters:
        model (nn.Module): Trained model.
        features (list): Raw feature values in FEATURES order.

    Returns:
        tuple: Predicted label name and probability.
    """
    model.eval()
    scaled = (np.array(features, dtype=np.float32) - X_MEAN) / X_STD
```

```
with torch.no_grad():
    X_new = torch.tensor(scaled).float().to(DEVICE).unsqueeze(0)
    probability = torch.softmax(model(X_new), dim=1)[0, 1].item()
    label = 'survived' if probability >= 0.5 else 'died'
    return label, probability
```

Run

```
label, probability = predict(loader_model, [3, 0, 25.0, 7.25, 1, 1, 0])
print(f"Predicted: {label} (p={probability:.4f})")
```

```
Predicted: died (p=0.4490)
```

Submit to the Leaderboard

1. Open kaggle.com/c/titanic and click **Submit Predictions**.
2. Upload `submission.csv`.
3. Read your score on the public leaderboard.
4. Improve your features and submit again. There is no deadline; the competition never closes.

Key Takeaways

- The Titanic competition is a real, permanent Kaggle "Getting Started" competition; it never closes.
- The submission must use the real `test.csv` and contain `PassengerId, Survived` for all 418 passengers.
- Sex and passenger class dominate survival, but feature engineering still improves the score.
- Always keep a validation split when tuning, even though the leaderboard is the final judge.
- The Kaggle CLI workflow, account, token, download, submit, is reusable across competitions.

Exercises

1. Create your Kaggle account, generate an API token, and download the competition files with the CLI.
2. Reproduce the submission and upload it to the leaderboard. Report your score.
3. Add a title-extraction feature from the passenger name and measure its effect.
4. Compare the logistic regression and PyTorch classifiers on validation accuracy.
5. Try a tree-based model such as a random forest and see whether it beats the neural network.
6. Explain why submitting predictions for a split of `train.csv` would be invalid.

Guided Walkthrough

The notebook begins with the mechanics that trip up beginners. It walks through creating a Kaggle account, joining the Titanic competition to accept its rules, and generating an API token, the file `kaggle.json` that authenticates the command-line tool. It shows the shell commands that place the token and lock down its permissions, then the Kaggle CLI commands that download the official `train.csv` and `test.csv`. A public-mirror fallback downloads the same two files so the notebook runs even without a token.

The exploratory section profiles the training data: shapes and types, missing values, the survival rate, and survival broken down by sex and class. Sex is the strongest single predictor, class matters next, and age and fare carry additional signal. The feature-engineering section imputes missing values with training statistics, encodes

categorical columns, and adds a family-size feature, applying identical transformations to train and test.

Two models follow. A logistic regression baseline sets the bar, and a small PyTorch classifier is trained and compared on a validation split. The notebook then predicts on the real `test.csv`, whose 418 passengers are the ones the leaderboard scores, and writes `submission.csv` with exactly two columns, `PassengerId` and `Survived`. The closing section explains how to upload it and read the score.

The most important lesson is the trap the notebook avoids. A beginner often splits `train.csv`, predicts on the held-out half, and submits those predictions, which is invalid because the rows do not match the leaderboard's passengers. Predicting on the real test file is the difference between a score and an error.

Common Pitfalls

- Submitting predictions for a split of `train.csv` instead of the real `test.csv`.
- Getting the submission columns or row count wrong; the leaderboard expects 418 rows of `PassengerId, Survived`.
- Leaking the target by computing imputation statistics on the test set.
- Tuning on the leaderboard, which overfits to the public score.

Going Further

- Engineer a title feature from the passenger name and measure its effect.
- Try a gradient-boosted tree model and compare with the neural network.
- Read the top public notebooks and study their feature engineering, then reproduce one idea.

Exercise Solutions

1. After creating the token and downloading the files, the working directory contains `data/train.csv` and `data/test.csv` with 891 and 418 rows.
2. A baseline submission scores around 0.76 to 0.78 accuracy on the public leaderboard.
3. Extracting titles such as Mr, Mrs, and Master adds signal, because title correlates with age and sex.
4. The two classifiers are close; the tree model often edges ahead on this small tabular dataset.
5. A split of `train.csv` has the wrong passenger ids and only 178 rows, so the leaderboard rejects it.
6. The competition never closes, so you can submit at any time and improve your score over many attempts.

Appendices

Appendix A: Python Reference

Core Types

```
x = 3           # int
y = 3.14        # float
s = "text"      # str
b = True        # bool
n = None        # absence of a value
```

Arithmetic

```
7 / 2   # 3.5   true division
7 // 2  # 3     floor division
7 % 2   # 1     remainder
2 ** 10 # 1024  power
```

Strings

```
f"{value:.3f}"    # three decimals
f"{value:.1%}"    # percentage
"a,b".split(",")  # ['a', 'b']
",".join(["a", "b"]) # 'a,b'
" x ".strip()     # 'x'
"ABC".lower()     # 'abc'
```

Collections

```
xs = [1, 2, 3]           # list
t = (1, 2, 3)            # tuple
d = {"a": 1, "b": 2}     # dict
s = {1, 2, 3}            # set

xs.append(4)
d["c"] = 3
d.get("z", 0)
[x * x for x in xs]      # comprehension
{k: v for k, v in d.items()} # dict comprehension
```

Control Flow

```
if x > 0:
    pass
elif x == 0:
    pass
else:
    pass

for i in range(10):
    pass

while x > 0:
```



```
x -= 1

for i, v in enumerate(xs):
    pass

for a, b in zip(xs, ys):
    pass
```

Functions and Classes

```
def f(x, y=0.0):
    return x + y

class Model:
    def __init__(self, weight):
        self.weight = weight

    def forward(self, x):
        return self.weight * x
```

Files and JSON

```
from pathlib import Path
import json

Path("data").mkdir(exist_ok=True)
Path("data/a.txt").write_text("hello")
text = Path("data/a.txt").read_text()

with open("data/x.json", "w") as handle:
    json.dump({"x": 1}, handle, indent=2)

with open("data/x.json") as handle:
    obj = json.load(handle)
```

Appendix B: PyTorch Reference

Setup

```
import torch
import torch.nn as nn

DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
torch.manual_seed(42)
```

Tensors

```
torch.zeros(2, 3)
torch.ones(2, 3)
torch.randn(2, 3)
torch.arange(10)
torch.linspace(0, 1, 5)

x.shape          # size
x.dtype          # type
x.device         # device
x.reshape(3, 4)
x.unsqueeze(0)
x.squeeze()
x.permute(1, 0)
x.to(DEVICE)
x.numpy()
torch.from_numpy(array)
```

Math

```
a + b
a * b          # elementwise
a @ b          # matrix multiply
a.sum(dim=0)
a.mean()
a.max(dim=1).values
torch.relu(a)
torch.softmax(a, dim=1)
```

Autograd

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2
y.backward()
x.grad

with torch.no_grad():
    pass
z = y.detach()
```

Models

```
model = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 2))

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc = nn.Linear(4, 2)

    def forward(self, x):
        return self.fc(x)
```

Training Loop

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(epochs):
    model.train()
    for x, y in train_loader:
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        pass
```

Save and Load

```
torch.save(model.state_dict(), "models/model.pt")
model.load_state_dict(torch.load("models/model.pt", map_location=DEVICE))
```

Appendix C: Mathematical Notation

Symbol	Meaning
$\sum$	Summation
$\prod$	Product
$\mathbb{E}[X]$	Expectation
$\text{Var}(X)$	Variance
σ	Standard deviation
μ	Mean
$\hat{y}$	Predicted value
$\bar{x}$	Sample mean
∇f	Gradient
$\frac{\partial f}{\partial x}$	Partial derivative
$x^\top$	Transpose
$\langle x, y \rangle$	Inner product
$ x _2$	Euclidean norm
$p(y \mid x)$	Conditional probability
$P(A)$	Probability of event A
$\approx$	Approximately equal
$\propto$	Proportional to
$\in$	Element of
$\forall$	For all
$\exists$	There exists

Appendix D: Glossary

Term	Definition
Activation function	A non-linear function applied after a linear layer, such as ReLU
Attention	A mechanism that weights every position against every other position
Autoencoder	A network that compresses and reconstructs its input
Backpropagation	Reverse-mode application of the chain rule to compute gradients
Batch	A subset of samples processed together
Bias	An added constant in a linear model, or systematic error
Calibration	Agreement between predicted probabilities and observed frequencies
Class imbalance	When one class is far more common than another
CNN	Convolutional neural network, suited to grid-like data such as images
Cross-entropy	The standard loss for classification
Cross-validation	Evaluating a model over several data splits
DataLoader	A PyTorch utility that batches and shuffles a dataset
Dropout	Randomly disabling units during training to reduce overfitting
Epoch	One full pass over the training data
Embedding	A learned dense vector for a discrete token
Epsilon-greedy	An exploration strategy that acts randomly with probability epsilon
Feature engineering	Creating informative inputs from raw data
Fine-tuning	Adapting a pretrained model to a new task
GAN	Generative adversarial network, a generator trained against a discriminator
Gradient	The vector of partial derivatives of a function
Gradient descent	An optimization method that steps against the gradient
Hyperparameter	A setting chosen before training, such as learning rate
Inference	Using a trained model to make predictions
KMeans	A clustering algorithm that partitions data around centroids
Learning rate	The step size in gradient descent
LSTM	A recurrent network with gates that remember over long spans
MLP	Multilayer perceptron, a fully connected feed-forward network
ONNX	An open format for exchanging trained models

Term	Definition
Overfitting	Fitting training noise at the cost of generalization
PCA	Principal component analysis, a linear dimensionality reduction
Precision	The fraction of predicted positives that are correct
Pretrained model	A model trained on a large corpus before fine-tuning
Recall	The fraction of true positives that are found
Regularization	Techniques that discourage overfitting
ReLU	The activation $\max(0, x)$
Replay buffer	Stored transitions used to train a reinforcement learning agent
Residual connection	A skip connection that adds a layer's input to its output
ROC and AUC	A curve and summary of ranking quality across thresholds
Softmax	A function that turns scores into a probability distribution
Standardization	Rescaling features to mean zero and standard deviation one
Temperature	A divisor on logits that controls sampling randomness
Tensor	A multi-dimensional array
Token	A unit of text, such as a character or subword
Transformer	An attention-based architecture for sequences
Underfitting	Failing to capture the signal in the data
Validation set	Data used to tune choices during development
ViT	Vision transformer, a transformer applied to image patches

Appendix E: Further Reading

Statistics and Data Analysis

- David Freedman, Robert Pisani, and Roger Purves, *Statistics*.
- John Tukey, *Exploratory Data Analysis*.
- Gareth James et al., *An Introduction to Statistical Learning*.

Machine Learning

- Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*.
- Christopher Bishop, *Pattern Recognition and Machine Learning*.
- Kevin Murphy, *Probabilistic Machine Learning*.

Deep Learning in Practice

- The PyTorch tutorials and documentation at pytorch.org.
- Andrej Karpathy, *nanoGPT* and *char-rnn*.
- The Hugging Face Transformers documentation.

Datasets Used in This Course

- Palmer Penguins, <https://github.com/allisonhorst/palmerpenguins>
- Fashion-MNIST, <https://github.com/zalandoresearch/fashion-mnist>
- Tiny Shakespeare, <https://github.com/karpathy/char-rnn>
- Breast Cancer Wisconsin, <https://archive.ics.uci.edu/dataset/17>
- Oxford-IIIT Pet, <https://www.robots.ox.ac.uk/~vgg/data/pets/>
- CartPole, <https://gymnasium.farama.org/>
- Titanic, <https://www.kaggle.com/c/titanic>

Appendix F: NumPy Reference

Creating Arrays

```
import numpy as np

np.array([1.0, 2.0, 3.0])
np.zeros(3)
np.ones((2, 3))
np.arange(5)
np.linspace(0, 1, 5)
np.eye(3)
rng = np.random.default_rng(0)
rng.normal(size=(2, 2))
```

Shapes and Reshaping

```
a.shape
a.reshape(3, 4)
a.ravel()
a.T
a.astype(np.float32)
```

Math and Reductions

```
a + b
a * b
a @ b
a.sum()
a.sum(axis=0)
a.mean()
a.std()
np.maximum(0.0, a)    # ReLU
np.sqrt(a)
np.exp(a)
```

Indexing

```
a[0]
a[:, 1]
a[a > 0]
a[0:2, 2:4]
```


Appendix G: pandas Reference

Loading and Inspecting

```
import pandas as pd

df = pd.read_csv("data/penguins-clean.csv")
df.head()
df.info()
df.describe()
df.shape
df.columns
df.dtypes
```

Selecting and Filtering

```
df["body_mass_g"]
df[["species", "body_mass_g"]]
df[df["body_mass_g"] > 4000]
df.loc[0]
df.iloc[0:5]
```

Cleaning

```
df.isnull().sum()
df["age"] = df["age"].fillna(df["age"].median())
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
df = df.dropna()
df = df.drop_duplicates()
```

Grouping and Aggregation

```
df.groupby("species")["body_mass_g"].mean()
df.groupby("species")["body_mass_g"].agg(["mean", "std", "count"])
df.pivot_table(index="species", columns="island", values="body_mass_g")
```

Saving

```
df.to_csv("outputs/results.csv", index=False)
```

Appendix H: Matplotlib Reference

```
import matplotlib.pyplot as plt

plt.plot(xs, ys, marker="o", label="series")
plt.scatter(x, y, c=labels)
plt.hist(values, bins=30)
plt.bar(categories, counts)
plt.imshow(image, cmap="gray")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Title")
plt.legend()
plt.tight_layout()
plt.savefig("outputs/figure.png", dpi=150)
plt.show()
```

Subplots

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(xs, ys)
axes[0].set_title("left")
axes[1].plot(xs, ys[::-1])
axes[1].set_title("right")
fig.tight_layout()
plt.show()
```

Appendix I: Command-Line and Environment Reference

Conda

```
conda create --name prod anaconda
conda activate prod
conda update --all
conda install -c conda-forge pytorch torchvision torchaudio
conda install numpy pandas matplotlib seaborn scikit-learn statsmodels
conda env export > environment.yml
conda env create -f environment.yml
```

Kaggle

```
pip install kaggle
mkdir -p ~/.kaggle
mv ~/Downloads/kaggle.json ~/.kaggle/kaggle.json
chmod 600 ~/.kaggle/kaggle.json
kaggle competitions download -c titanic -f train.csv -p data
kaggle competitions download -c titanic -f test.csv -p data
```

Git

```
git status
git add .
git commit -m "message"
git push origin main
git log --oneline -10
```

Jupyter

```
conda activate prod
python -m ipykernel install --user --name prod --display-name "Python (prod)"
jupyter lab
```

Appendix J: Function Index

Every function and method defined in the notebooks, with its one-line purpose.

00-CLT

Function	Purpose
<code>my_avg</code>	Calculate the average of a list of numbers.
<code>gen_samples_calc_avg</code>	Assume we are generating values from a UNIFORM distribution between 0 and 1
<code>sample_variance</code>	Calculate the sample variance of a list of numbers (Bessel's correction: divide by n-1).
<code>sample_sd</code>	Calculate the sample standard deviation of a list of numbers (using Bessel's correction, n-1).

05-SLR-FLF

Function	Purpose
<code>my_coefplot</code>	Plots a single coefficient with error bars from a statsmodels model.

06-SLR-FNLF

Function	Purpose
<code>my_coefplot</code>	Plots a single coefficient with error bars from a statsmodels model.

07-SLR-FLFP

Function	Purpose
<code>my_coefplot</code>	Plots a single coefficient with error bars from a statsmodels model.

08-SLR-FNLFP

Function	Purpose
<code>my_coefplot</code>	Plots a single coefficient with error bars from a statsmodels model.

09-SLR-MLFP

Function	Purpose
<code>my_coefplot</code>	Plots a single coefficient with error bars from a statsmodels model.

10-SLR-FMM

Function	Purpose
<code>my_coefplot</code>	Plots a single coefficient with error bars from a statsmodels model.

11-MLR-AF

Function	Purpose
<code>calc_trend_wrt_x1</code>	Calculates the linear trend based on two independent variables x1 and x2, using the provided coefficients.
<code>calc_trend_wrt_x2</code>	Calculates the linear trend based on two independent variables x1 and x2, using the provided coefficients.

12-MLR-IF

Function	Purpose
<code>calc_trend_wrt_x1_w_interactions</code>	Calculates the linear trend based on two independent variables x1 and x2, using the provided coefficients.
<code>calc_trend_wrt_x2_w_interactions</code>	Calculates the linear trend based on two independent variables x1 and x2, using the provided coefficients.

13-LR

Function	Purpose
<code>fit_and_assess_ols</code>	Fits an Ordinary Least Squares (OLS) regression model using statsmodels and returns model metrics.
<code>my_coefplot</code>	Create a coefficient plot of a fitted statsmodels regression model.
<code>linear_regression_cross_val_score</code>	Perform cross-validated evaluation of a linear-style model using a Patsy formula.

14-LR

Function	Purpose
<code>fit_and_assess_logit</code>	Fit a logistic regression model and return its metrics.
<code>predict_class</code>	Predict binary class labels from a fitted model.
<code>logit_cross_val_score</code>	Run cross-validated evaluation for a logistic regression formula.

15-MLP

Function	Purpose
<code>__init__</code>	Initialize the neural network layers.
<code>forward</code>	Run the forward pass of the network.
<code>__init__</code>	Initialize the dataset with NumPy arrays.

Function	Purpose
<code>__len__</code>	Return the number of samples in the dataset.
<code>__getitem__</code>	Return a single sample as tensors on the device.
<code>train_epoch</code>	Train the model for a single epoch.
<code>evaluate</code>	Evaluate the model over a loader.
<code>predict</code>	Predict the species for one standardized measurement.

16-CNN

Function	Purpose
<code>__init__</code>	Initialize the convolutional and dense layers.
<code>forward</code>	Run the forward pass of the network.
<code>train_epoch</code>	Train the model for a single epoch.
<code>evaluate</code>	Evaluate the model over a loader.
<code>predict</code>	Predict the class of one normalized image tensor.

17-RNN

Function	Purpose
<code>build_vocab</code>	Build character to index and index to character maps.
<code>encode</code>	Encode text as a tensor of indices.
<code>decode</code>	Decode indices back to text.
<code>__init__</code>	Store the encoded text and window length.
<code>__len__</code>	Return the number of windows.
<code>__getitem__</code>	Return one input window and its shifted target.
<code>__init__</code>	Initialize the embedding, LSTM, and output layer.
<code>forward</code>	Run the forward pass over a batch of windows.
<code>train_epoch</code>	Train the model for a single epoch.
<code>evaluate</code>	Evaluate the model over a loader.
<code>generate</code>	Generate text from a trained model.

18-ATT

Function	Purpose
<code>scaled_dot_product_attention</code>	Compute scaled dot-product attention.

Function	Purpose
<code>causal_mask</code>	Build a lower-triangular causal mask.
<code>__init__</code>	Initialize the query, key, value, and output projections.
<code>forward</code>	Run multi-head causal self-attention.
<code>__init__</code>	Initialize attention, normalization, and feed-forward layers.
<code>forward</code>	Run the block with residual connections.
<code>__init__</code>	Initialize embeddings, the transformer block, and the output head.
<code>forward</code>	Run the forward pass of the model.
<code>__init__</code>	Store the encoded text and window length.
<code>__len__</code>	Return the number of windows.
<code>__getitem__</code>	Return one input window and its shifted target.
<code>train_epoch</code>	Train the model for a single epoch.
<code>evaluate</code>	Evaluate the model over a loader.
<code>generate</code>	Generate text from a trained model.

19-GPT

Function	Purpose
<code>__init__</code>	Store the encoded text and window length.
<code>__len__</code>	Return the number of windows.
<code>__getitem__</code>	Return one input window and its shifted target.
<code>__init__</code>	Initialize the key, query, and value projections.
<code>forward</code>	Apply causal self-attention to the input.
<code>__init__</code>	Initialize the heads and the output projection.
<code>forward</code>	Concatenate the heads and project the result.
<code>__init__</code>	Initialize the two linear layers and activation.
<code>forward</code>	Run the feed-forward network.
<code>__init__</code>	Initialize attention, feed-forward, and normalization layers.
<code>forward</code>	Run the block with residual connections.
<code>__init__</code>	Initialize embeddings, blocks, and the output head.
<code>embed</code>	Combine token and positional embeddings.
<code>forward</code>	Run the forward pass and optional loss.
<code>generate</code>	Generate new tokens by sampling from the model.

Function	Purpose
<code>train_epoch</code>	Train the model for a single epoch.
<code>estimate_loss</code>	Estimate the mean loss over a loader.
<code>generate_text</code>	Generate readable text from a seed string.

20-REG

Function	Purpose
<code>__init__</code>	Initialize the dataset with NumPy arrays.
<code>__len__</code>	Return the number of samples in the dataset.
<code>__getitem__</code>	Return a single sample as tensors on the device.
<code>__init__</code>	Initialize the neural network layers.
<code>forward</code>	Run the forward pass of the network.
<code>train_epoch</code>	Train the model for a single epoch.
<code>evaluate</code>	Evaluate the model over a loader.
<code>predict</code>	Predict benign or malignant for one raw feature vector.

21-EVAL

Function	Purpose
<code>__init__</code>	Initialize the dataset with NumPy arrays.
<code>__len__</code>	Return the number of samples in the dataset.
<code>__getitem__</code>	Return a single sample as tensors on the device.
<code>__init__</code>	Initialize the neural network layers.
<code>forward</code>	Run the forward pass of the network.
<code>train_epoch</code>	Train the model for a single epoch.
<code>metric_table</code>	Build a table of core classification metrics.
<code>predict</code>	Predict benign or malignant with a tunable threshold.

22-AE

Function	Purpose
<code>__init__</code>	Initialize the encoder and decoder.
<code>forward</code>	Encode and reconstruct a batch of images.
<code>train_epoch</code>	Train the autoencoder for a single epoch.

Function	Purpose
<code>evaluate</code>	Evaluate the autoencoder over a loader.
<code>show_reconstructions</code>	Plot original images above their reconstructions.
<code>reconstruction_errors</code>	Compute per-image reconstruction errors.
<code>reconstruct</code>	Reconstruct one image and return its error.

23-GAN

Function	Purpose
<code>__init__</code>	Initialize the generator layers.
<code>forward</code>	Generate a batch of images from noise.
<code>__init__</code>	Initialize the discriminator layers.
<code>forward</code>	Score a batch of images.
<code>discriminator_loss</code>	Compute the discriminator loss on real and fake images.
<code>generator_loss</code>	Compute the generator loss from the discriminator scores.
<code>train_epoch</code>	Train the discriminator and generator for one epoch.
<code>show_samples</code>	Plot a grid of generated images.

24-VIT

Function	Purpose
<code>__init__</code>	Initialize the patch projection.
<code>forward</code>	Embed a batch of grayscale images.
<code>__init__</code>	Initialize the query, key, value, and output projections.
<code>forward</code>	Run multi-head self-attention.
<code>__init__</code>	Initialize attention, normalization, and feed-forward layers.
<code>forward</code>	Run the encoder block with residual connections.
<code>__init__</code>	Initialize patch embedding, tokens, blocks, and the head.
<code>forward</code>	Classify a batch of images.
<code>train_epoch</code>	Train the model for a single epoch.
<code>evaluate</code>	Evaluate the model over a loader.
<code>predict</code>	Predict the class of one normalized image tensor.

25-DEPLOY

Function	Purpose
<code>__init__</code>	Initialize the convolutional and dense layers.
<code>forward</code>	Run the forward pass of the network.
<code>train_epoch</code>	Train the model for a single epoch.
<code>evaluate</code>	Evaluate the model over a loader.
<code>export_onnx</code>	Export a PyTorch model to ONNX.
<code>onnx_logits</code>	Run inference with an ONNX model.
<code>__init__</code>	Load the ONNX session.
<code>predict</code>	Predict the class for one image tensor.
<code>build_parser</code>	Build the command-line argument parser.
<code>main</code>	Run the command-line inference engine.

26-SEG

Function	Purpose
<code>mask_transform</code>	Resize a raw mask and shift its values into class indices.
<code>__init__</code>	Initialize the two convolutions.
<code>forward</code>	Apply the two convolutions.
<code>__init__</code>	Initialize the encoder, bottleneck, decoder, and head.
<code>forward</code>	Segment a batch of images.
<code>train_epoch</code>	Train the model for a single epoch.
<code>pixel_accuracy</code>	Compute pixel accuracy over a loader.
<code>plot_triplet</code>	Plot an image, its true mask, and the predicted mask.
<code>show_prediction</code>	Plot an image, its true mask, and the predicted mask.
<code>predict_mask</code>	Predict a segmentation mask for one image.

27-DRL

Function	Purpose
<code>__init__</code>	Initialize the buffer with a fixed capacity.
<code>push</code>	Store one transition.
<code>sample</code>	Sample a random batch of transitions.
<code>__len__</code>	Return the number of stored transitions.

Function	Purpose
<code>__init__</code>	Initialize the network layers.
<code>forward</code>	Score every action for a batch of states.
<code>to_tensor</code>	Convert a NumPy array to a tensor on the device.
<code>select_action</code>	Choose an action with epsilon-greedy exploration.
<code>unpack_batch</code>	Sample and convert a batch of transitions.
<code>dqn_loss</code>	Compute the temporal-difference loss.
<code>optimize</code>	Run one gradient step on a sampled batch.
<code>train</code>	Train the agent over many episodes.
<code>evaluate</code>	Run the greedy policy and report mean reward.

28-LLM

Function	Purpose
<code>tokenize_blocks</code>	Tokenize text and cut it into fixed-length blocks.
<code>__init__</code>	Store the token blocks.
<code>__len__</code>	Return the number of blocks.
<code>__getitem__</code>	Return one block as input and target.
<code>train_epoch</code>	Fine-tune the model for a single epoch.
<code>estimate_loss</code>	Estimate the mean loss over a loader.
<code>generate</code>	Generate text from a prompt.

29-KAGGLE

Function	Purpose
<code>download_file</code>	Download a file if it is not already present.
<code>add_family_features</code>	Add family size and alone features.
<code>prepare_features</code>	Impute and encode both datasets using training statistics.
<code>__init__</code>	Initialize the dataset with NumPy arrays.
<code>__len__</code>	Return the number of samples in the dataset.
<code>__getitem__</code>	Return a single sample as tensors on the device.
<code>__init__</code>	Initialize the neural network layers.
<code>forward</code>	Run the forward pass of the network.
<code>train_epoch</code>	Train the model for a single epoch.

REVERSE ENGINEERING DATA

Function	Purpose
<code>evaluate</code>	Evaluate the model over a loader.
<code>predict_test</code>	Predict survival for the official test set.
<code>build_submission</code>	Write a Kaggle-format submission CSV.
<code>predict</code>	Predict survival for one passenger.

Appendix K: Chapter Summaries

Chapter 1, Central Limit Theorem. Random samples are averaged, and the distribution of those averages is shown to become normal and narrow as the sample size grows. The standard error of the mean and the confidence interval are built from that behavior.

Chapter 2, Penguins EDA. A full exploratory pass over the Penguins data: shape, types, missing values, distributions, relationships, and anomalies. The habit of profiling before modeling is established.

Chapter 3, Clustering. KMeans and principal component analysis group the penguins without labels. Standardization is shown to be essential before any distance-based method.

Chapters 4 to 11, Simple Linear Regression. A single-feature model is fit, diagnosed, and used to predict. The sequence covers linear and non-linear relationships, prediction, performance measurement, and model comparison.

Chapters 12 to 14, Multiple Regression and Linear Regression. Several predictors are combined, additive and interactive effects are contrasted, and a complete regression study with cross-validation, residual checks, and an inference engine is performed.

Chapter 15, Logistic Regression. A binary target is modeled with the logistic function, and the classifier is evaluated with a confusion matrix, ROC curve, and AUC.

Chapter 16, MLP. The first neural network: a multilayer perceptron in PyTorch with a custom dataset, a data loader, an explicit training loop, and an inference engine.

Chapter 17, CNN. A convolutional network classifies Fashion-MNIST images, and the confusion matrix reveals which clothing categories are genuinely hard.

Chapter 18, RNN. A character-level LSTM learns to generate Shakespeare-like text from the Tiny Shakespeare corpus.

Chapter 19, Attention. Scaled dot-product attention, causal masking, multi-head attention, and a transformer block are built from scratch.

Chapter 20, TinyGPT. A decoder-only transformer language model is assembled from its parts and trained to generate text.

Chapter 21, Regularization. Dropout, decoupled weight decay, a learning-rate schedule, and early stopping fight overfitting on the Breast Cancer data.

Chapter 22, Model Evaluation. Confusion matrices, precision, recall, F1, ROC, AUC, precision-recall curves, calibration, and threshold tuning are applied to a classifier.

Chapter 23, Autoencoder. An autoencoder compresses and reconstructs Fashion-MNIST images, and its latent space and reconstruction error are explored.

Chapter 24, GAN. A generator and discriminator are trained against each other to synthesize clothing images.

Chapter 25, Vision Transformer. A vision transformer classifies images from patch embeddings, a class token, and transformer encoder blocks.

Chapter 26, Deployment. A model is exported to ONNX, verified against the original, wrapped in an inference engine, and exposed through a command-line interface.

Chapter 27, Semantic Segmentation. A U-Net labels every pixel of an image as pet, background, or boundary on the Oxford-IIIT Pet dataset.

Chapter 28, Deep Q-Network. An agent learns to balance a pole on a cart using a Q-network, a replay buffer, and a target network.

Chapter 29, Fine-Tuning an LLM. A pretrained DistilGPT-2 model is fine-tuned on Tiny Shakespeare and generates text in the target style.

Chapter 30, Kaggle Project. A complete Kaggle project is built from scratch: account and API token, dataset download, EDA, feature engineering, two classifiers, and a valid submission to a permanent competition.